

Corso di Onboarding DevOps

1. Introduzione

Benvenuto/a a bordo! 🙌

Come è organizzato questo corso

Il contesto di lavoro: cosa fa il tuo team, in parole semplici

Il tuo percorso: da Junior Project Manager a Scrum Master

Come affrontare lo studio: qualche consiglio pratico

La mappa del corso

Pronto/a per iniziare?



Esercizi pratici



Collegamenti



Risorse

2. Fondamenti di informatica



Obiettivi della sezione

2.1 Cos'è un computer: hardware e software

2.2 La CPU: il cervello che calcola

2.3 La RAM: la memoria di lavoro temporanea

2.4 Il disco: la memoria permanente

2.5 Il Sistema Operativo: chi coordina tutto

2.6 Processi e Thread

2.7 Il File System: come sono organizzati i file

2.8 La rete: cos'è una rete di computer

2.9 TCP/IP: come i computer si "spediscono lettere"

2.10 HTTP e HTTPS: il protocollo del web

2.11 DNS: la rubrica telefonica di internet

2.12 API: come i software si parlano tra loro

2.13 REST: uno stile comune per costruire API

2.14 JSON e XML: i formati per scambiarsi dati

2.15 Database relazionali e SQL

2.16 Database NoSQL

2.17 Virtual Machine: un computer dentro un computer

2.18 Container: più leggeri di una VM

2.19 Docker: lo strumento più diffuso per i container

2.20 Kubernetes: l'orchestratore di container

2.21 Markdown e HTML: i due linguaggi per scrivere documenti e pagine

2.22 Riepilogo: come si incastrano tutti questi pezzi



Checklist di autoverifica



Esercizi pratici



Collegamenti



Risorse

3. Come nasce un software



Obiettivi della sezione



Il ciclo di vita del software

-  Requisiti: funzionali e non funzionali
-  Bug: quando qualcosa va storto
-  Feature: una nuova funzionalità
-  Versioning: perché numeriamo il software
-  Branching: lavorare in parallelo
-  Dal branch al merge: pull request e code review
-  ADR: perché una decisione tecnica va scritta
-  Come si collegano tutti questi concetti
-  Esercizi pratici
-  Collegamenti
-  Risorse

4. Git e GitHub

-  Obiettivi della sezione
- 4.1 Cos'è Git
- 4.2 Cos'è GitHub (e non è la stessa cosa di Git!)
- 4.3 Repository: il “progetto” versionato
- 4.4 Commit: uno scatto fotografico delle modifiche
- 4.5 Branch: un ramo di lavoro parallelo
- 4.6 Merge: unire due rami
- 4.7 Pull Request: il cuore della collaborazione su GitHub
- 4.8 Issue: tracciare bug e richieste
- 4.9 Release: una versione pubblicata ufficialmente
- 4.10 Tag: etichettare un punto preciso della storia
- 4.11 Git Flow: un workflow strutturato a più branch
- 4.12 Trunk Based Development: il workflow semplificato
- 4.13 Il modello “cactus” (OneFlow): una via di mezzo
- 4.14 Riepilogo dei comandi visti in questa sezione
-  Esercizi pratici
-  Collegamenti
-  Risorse

5. Agile

-  Obiettivi della sezione
- 5.1 Perché nasce l'Agile: il problema del modello a “cascata”
- 5.2 Il Manifesto Agile: i 4 valori fondamentali
- 5.3 I 12 principi Agile: raggruppati per temi
- 5.4 Il “mindset” Agile: più di un metodo, un modo di pensare
- 5.5 Waterfall vs Agile: il confronto visivo
- 5.6 Dall'Agile ai framework concreti: Scrum e Kanban
-  Esercizi pratici
-  Collegamenti
-  Risorse

6. Scrum

-  Obiettivi della sezione
- 6.1 Cos'è Scrum e perché è ovunque
- 6.2 I tre ruoli di Scrum
- 6.3 Gli eventi di Scrum

6.4 Gli artefatti di Scrum

6.5 User Story: come si descrive un requisito in Scrum

6.6 Story Point: stimare senza usare ore o giorni

6.7 Velocity: quanto il team riesce a fare in uno sprint

6.8 Definition of Ready: quando una storia può entrare in sprint

6.9 Definition of Done: quando una storia è davvero completata

6.10 Il flusso completo, in sintesi

6.11 Il ruolo dello Scrum Master nella pratica quotidiana



Esercizi pratici



Collegamenti



Risorse

7. Kanban



Obiettivi della sezione

7.1 Da dove viene Kanban

7.2 La board Kanban: colonne e card

7.3 WIP: il lavoro "in corso" e i suoi limiti

7.4 Lead Time e Cycle Time

7.5 Kanban vs Scrum: quando usare cosa

7.6 Riepilogo



Esercizi pratici



Collegamenti



Risorse

8. Project Management



Obiettivi della sezione

8.1 Stakeholder: chi ha interesse nel progetto

8.2 RAID Log: la mappa dei problemi (prima che diventino problemi)

8.3 RACI: chi fa cosa (e chi deve solo saperlo)

8.4 Milestone: un traguardo, non una scadenza qualsiasi

8.5 Rischio: identificare, valutare, mitigare

8.6 Pianificazione in un contesto Agile: livelli, non un unico piano

8.7 Monitoraggio: come si tiene traccia dell'andamento

8.8 KPI: gli indicatori chiave per un team Agile/DevOps

8.9 Reportistica: cosa riportare, a chi, e con quale livello di dettaglio

8.10 Il PMP e il PMBOK: che cosa sono e perché ti riguardano

8.11 I 5 gruppi di processi (process group)

8.12 Le aree di conoscenza (knowledge area)

8.13 Il Project Charter: l'atto di nascita del progetto

8.14 Scope e WBS (Work Breakdown Structure)

8.15 Tempi: schedule, dipendenze e critical path

8.16 Costi e Earned Value Management (EVM)

8.17 Il triplo vincolo e la gestione delle modifiche

8.18 Qualità, approvvigionamenti e comunicazione secondo il PMBOK

8.19 Chiusura del progetto e lessons learned

8.20 PMP e Agile: come convivono davvero

8.21 Jira: la gerarchia degli oggetti e le due viste

8.22 Excel: reporting e WBS

8.23 Riepilogo



Esercizi pratici



Collegamenti



Risorse

9. DevOps



Obiettivi della sezione

9.1 Cos'è DevOps: non è un tool, è una cultura

9.2 Perché nasce DevOps: il muro della confusione

9.3 Il ciclo DevOps: un percorso senza fine

9.4 La cultura DevOps: collaborazione, responsabilità, fiducia

9.5 Automazione: il motore pratico di DevOps

9.6 Continuous Integration (CI): integrare spesso, non alla fine

9.7 Continuous Delivery e Continuous Deployment: la differenza che confonde tutti

9.8 Infrastructure as Code (IaC): l'infrastruttura scritta come codice

9.9 Monitoring: sapere se le cose stanno andando bene

9.10 Observability: capire non solo il "cosa", ma il "perché"

9.11 Logging: la memoria degli eventi del sistema

9.12 Service Management: cosa succede quando il software è in produzione

9.13 Farm batch e processi schedulati: il lavoro che gira di notte

9.14 Riepilogo



Esercizi pratici



Collegamenti



Risorse

10. CI/CD



Obiettivi della sezione

10.1 Ripasso: cosa sono CI e CD

10.2 Anatomia di una pipeline: le fasi tipiche

10.3 Trigger: cosa fa scattare una pipeline

10.4 Ambienti: dove "vive" il codice, fase per fase

10.5 Artifact: il "pacchetto" che viaggia lungo la pipeline

10.6 Pipeline as Code: la pipeline è codice, non un click di troppo

10.7 Un esempio di pipeline in YAML

10.8 Quality Gate: i controlli che possono bloccare tutto

10.9 Rollback: cosa succede se un deploy va male comunque

10.10 Feature flag: disaccoppiare il rilascio dalla decisione di business

10.11 Strumenti concreti della pipeline

10.12 Riepilogo



Esercizi pratici



Collegamenti



Risorse

11. Architetture software



Obiettivi della sezione

11.1 Cos'è un'architettura software

11.2 Architettura Monolitica

11.3 Architettura a Microservizi

11.4 Come comunicano i componenti

11.5 Event-Driven Architecture: reagire invece di chiedere

11.6 Frontend vs Backend

11.7 Client-Server: il concetto base

11.8 Architettura a 3 livelli

11.9 Un accenno al Serverless

11.10 Riepilogo: cosa ti serve ricordare



Checklist di autoverifica



Esercizi pratici



Collegamenti



Risorse

12. Cloud



Obiettivi della sezione

12.1 Cos'è il cloud computing

12.2 I tre modelli principali: IaaS, PaaS, SaaS

12.3 Confronto: chi gestisce cosa

12.4 Azure: panoramica generale

12.5 AWS: la principale alternativa

12.6 Scalabilità: verticale vs orizzontale

12.7 Pay-as-you-go: paghi solo quello che usi

12.8 Regioni, data center e alta disponibilità

12.9 Platform Engineering: rendere semplice la strada giusta

12.10 DevOps Platform e Integration Platform: due mondi che si confondono

12.11 Riepilogo: cosa ti serve ricordare



Checklist di autoverifica



Esercizi pratici



Collegamenti



Risorse

13. Sicurezza



Obiettivi della sezione

13.1 Perché la sicurezza riguarda anche un PM, non solo i tecnici

13.2 Autenticazione vs Autorizzazione

13.3 Password, credenziali e MFA: le buone pratiche di base

13.4 HTTPS e crittografia: un richiamo veloce

13.5 Vulnerabilità comuni: perché il codice va "testato" anche per la sicurezza

13.6 OWASP Top 10: il riferimento standard del settore

13.7 DevSecOps: la sicurezza fin dall'inizio ("shift left")

13.8 Scan di sicurezza nella pipeline CI/CD

13.9 GDPR e privacy dei dati: un accenno essenziale

13.10 Backup e disaster recovery: il piano B quando qualcosa va storto

13.11 Business Continuity: cosa fa l'azienda mentre i sistemi sono giù

13.12 Riepilogo



Checklist di autoverifica



Esercizi pratici



Collegamenti



Risorse

14. Ambienti di sviluppo

Obiettivi della sezione

- 14.1 Perché servire diversi ambienti: la “prova costume”
- 14.2 Ambiente di Sviluppo (Dev)
- 14.3 Ambiente di Test / QA
- 14.4 Ambiente di Staging (Pre-produzione)
- 14.5 Ambiente di Produzione
- 14.6 I quattro ambienti, fase per fase
- 14.7 “Funziona sul mio computer”: perché gli ambienti devono somigliarsi
- 14.8 Dati di test vs dati reali
- 14.9 Configurazioni per ambiente
- 14.10 Chi ha accesso a quali ambienti

Confronto tra i quattro ambienti

14.11 Riepilogo

Checklist di autoverifica

Esercizi pratici

Collegamenti

Risorse

15. Intelligenza artificiale

Obiettivi della sezione

- 15.1 Cos'è l'intelligenza artificiale, in parole semplici
- 15.2 Machine learning: come “impara” un modello
- 15.3 AI generativa e LLM: cosa sono, e perché “sbagliano” in un modo particolare
- 15.4 RAG: far rispondere il modello sui documenti veri, non su quello che “ricorda”
- 15.5 MCP: dare al modello mani per agire, non solo occhi per leggere
- 15.6 AI negli strumenti di sviluppo: cosa cambia per Marco, Giulia e Ahmed
- 15.7 AI nel ciclo di vita DevOps: dall'idea al monitoraggio
- 15.8 AI nel lavoro di un Project Manager / Scrum Master
- 15.9 Rischi, limiti e uso responsabile
- 15.10 MLOps: come si porta un modello in produzione, in sintesi
- 15.11 Come usarla bene nel quotidiano
- 15.12 Riepilogo

Checklist di autoverifica

Esercizi pratici

Collegamenti

Risorse

16. Glossario

Mappa dei concetti: come si collegano

Collegamenti

Risorse

17. Piano di studio

Come usare questo piano

Vista d'insieme del percorso

Settimana 1 — Le fondamenta: come funziona un computer, come nasce un software

Settimana 2 — Come nasce un software e il controllo di versione con Git

Settimana 3 — Agile e Scrum: il mindset e il framework che il team usa ogni giorno

Settimana 4 — Kanban: un altro modo di visualizzare il lavoro

Settimana 5 — Project Management: come si tiene sotto controllo un progetto

Settimana 6 — DevOps e CI/CD: la cultura e la pipeline che uniscono sviluppo e operations

Settimana 7 — Il contesto tecnico e trasversale: architetture, cloud, sicurezza, ambienti

Settimana 8 — Intelligenza artificiale: come entra nel lavoro quotidiano del team

Settimana 9 — Consolidamento: ripasso generale, autovalutazione e prossimi passi

Tabella riepilogativa delle 9 settimane

 Collegamenti

 Risorse

18. Libri consigliati

La logica dei 5 livelli

Livello 0 – Fondamenti di informatica

Livello 1 – Sviluppo software

Livello 2 – Agile

Livello 3 – Project Management

Livello 4 – DevOps

Tabella riepilogativa

 Collegamenti

 Risorse

19. Risorse online

Come usare questa sezione

1. Fondamenti di informatica

2. Sviluppo software e Git/GitHub

3. Agile, Scrum, Kanban

4. Project Management

5. DevOps e CI/CD

6. Architetture, Cloud, Sicurezza

7. Intelligenza artificiale

Come restare aggiornato nel tempo

 Collegamenti

20. Backend Engineering Avanzato

 Obiettivi della sezione

20.1 Architettura di un sistema backend, in profondità

20.2 Dentro una richiesta HTTPS: il TLS handshake

20.3 Autenticazione stateless: sessioni, JWT e OAuth 2.0 in pratica

20.4 Progettare una API REST che invecchia bene

20.5 Transazioni e proprietà ACID

20.6 Isolamento e concorrenza: cosa può andare storto tra due transazioni

20.7 NoSQL e il teorema CAP

20.8 Scalare un database: replication e sharding

20.9 Caching: velocità in cambio di un problema nuovo

20.10 Concorrenza nel backend: thread, processi ed event loop

20.11 Query optimization: come una query lenta diventa rapida

20.12 Checklist finale: un'API pronta per la produzione

20.13 Riepilogo

 Esercizi pratici

 Collegamenti

 Risorse

CORSO DI ONBOARDING

Da neolaureato a Junior Project Manager / Scrum Master

1. Introduzione

Benvenuto/a a bordo! 🙌

Ciao, e benvenuto/a in azienda!

Se stai leggendo questa pagina, probabilmente ti sei appena seduto/a alla tua scrivania (fisica o virtuale) con un mucchio di acronimi nuovi in testa — Scrum, Sprint, DevOps, CI/CD, board, backlog — e la sensazione di dover imparare un'intera lingua straniera in poche settimane.

Respira: è normale, ed è esattamente per questo che esiste questo corso.

Sei stato/a assunto/a come **Junior Project Manager** e, nell'arco dei prossimi 2-3 mesi, affiancherai (e poi gradualmente sostituirai) una collega che oggi lavora come **Scrum Master / Project Manager** su un team che si occupa di una **piattaforma DevOps a supporto dello sviluppo software**. Non sai cosa significhi tutto questo? Perfetto, è il punto di partenza giusto: questo corso è pensato apposta per chi, come te, arriva da un percorso di studi gestionale (Ingegneria Gestionale) e parte da zero dal punto di vista tecnico e informatico.

Non ti verrà chiesto di scrivere codice o di diventare uno sviluppatore. Ti verrà chiesto di **capire come funziona il mondo in cui il tuo team lavora**, per poterlo organizzare, facilitare e rappresentare al meglio — verso il team stesso, verso i responsabili e verso il cliente.

Come è organizzato questo corso

Il corso è diviso in **19 sezioni numerate**, pensate per essere seguite **in ordine progressivo**: ogni sezione si appoggia sui concetti spiegati in quella precedente, un po' come i mattoncini di un Lego — non puoi costruire il tetto se non hai ancora messo le fondamenta.

Le sezioni si possono raggruppare in quattro grandi blocchi:

1. **Fondamenta tecniche** (sezioni 2-4): cosa è un computer, come “nasce” un software, cos'è Git/GitHub. Ti servono per capire il linguaggio di base che sviluppatori e tecnici usano ogni giorno.
2. **Metodologie di lavoro** (sezioni 5-8): Agile, Scrum, Kanban e Project Management. Sono i “metodi di organizzazione del lavoro” — qui il tuo background gestionale ti aiuterà moltissimo, perché sono concetti di organizzazione applicati al software.
3. **Mondo DevOps** (sezioni 9-15): DevOps, CI/CD, architetture software, cloud, sicurezza, ambienti di sviluppo e intelligenza artificiale. È il cuore tecnico del corso, quello più legato al ruolo che andrai a occupare: l'intelligenza artificiale chiude questo blocco perché ormai si intreccia con quasi tutti gli argomenti visti prima — dalla pipeline CI/CD alla code review, fino al monitoraggio.
4. **Strumenti di supporto e consultazione** (sezioni 16-19): glossario, piano di studio, libri e risorse online. Non sono da “studiare” una volta e basta: sono pensate per essere consultate spesso, come un dizionario o un'agenda.

🔗 **Esempio pratico:** se in una riunione senti dire “la pipeline è rossa, il rilascio è bloccato”, non serve che tu capisca subito cosa significhi nel dettaglio. Ti basta pensare “questo è un argomento del Blocco C, probabilmente della sezione 10 sul CI/CD” e prendere nota mentale per approfondirlo quando arrivi lì, invece di sentirti perso/a o di dover fingere di aver capito.

Oltre alle 19 sezioni, trovi due strumenti trasversali che ti accompagneranno per tutto il percorso:

- **Il Glossario** (sezione 16): ogni volta che incontri un termine che non ricordi — sia in questo corso, sia durante una riunione di lavoro — puoi cercarlo lì. Non c'è nulla di male nel non ricordare a memoria cosa significa “backlog” la terza volta che lo senti: è normale, ci vuole ripetizione.
- **Il Piano di studio** (sezione 17): un programma settimanale su **9 settimane** che ti dice, settimana per settimana, cosa leggere, quanto tempo dedicarci, e con quali attività pratiche verificare di aver capito (ad esempio: “questa settimana osserva una Daily Scrum del team e prendi nota di cosa succede”).

💡 **Consiglio pratico:** la prima volta che apri il corso, dai una scorsa veloce a tutti i titoli delle 19 sezioni (li trovi anche nel menu laterale). Non devi capire tutto subito: ti basta avere una mappa mentale di “cosa troverò più avanti”, così quando in una sezione si accenna a un concetto che verrà spiegato dopo, saprai che è normale e che arriverà il suo momento.

Il contesto di lavoro: cosa fa il tuo team, in parole semplici

Prima di addentrarci nei dettagli tecnici (che vedremo dalla sezione 2 in poi), proviamo a capire **a grandi linee** cosa significa lavorare su “una piattaforma DevOps a supporto dello sviluppo software”. Niente gergo tecnico per ora: solo un’analogia.

Immagina una grande cucina di un ristorante che deve servire centinaia di piatti ogni giorno, in modo veloce, corretto e senza errori. In quella cucina:

- ci sono **cuochi** che preparano i piatti (gli **sviluppatori**, che scrivono il codice del software);
- ci sono **linee di produzione** che passano il piatto da una stazione all'altra — dalla preparazione, alla cottura, all'impiattamento, al controllo qualità, fino al tavolo del cliente (questo, nel mondo software, si chiama **pipeline**, e lo vedremo nella sezione 10 sul CI/CD);
- ci sono **strumenti e attrezzature** condivise da tutta la cucina — i forni, i frigoriferi, i banchi di lavoro (nel mondo software, sono i **server, gli ambienti di test, gli strumenti di collaborazione**: lo vedremo nelle sezioni su DevOps e Cloud);
- e c'è un **responsabile di sala/cucina** che si assicura che gli ordini arrivino in tempo, che i cuochi non siano sovraccarichi, che eventuali problemi (un piatto tornato indietro, un ingrediente finito) vengano gestiti e comunicati al cliente. Questo è, in sostanza, il ruolo di **Project Manager / Scrum Master**: non cucini tu, ma fai in modo che la cucina funzioni, che il team lavori bene insieme, e che il cliente sia informato e soddisfatto.

Una **piattaforma DevOps** è l'insieme di strumenti e processi che permettono a questa “cucina” di funzionare in modo automatico, veloce e sicuro: dal momento in cui un cuoco scrive una nuova ricetta (scrive codice), al momento in cui quel piatto arriva davanti al cliente (il software è “in produzione”, cioè realmente usato dagli utenti finali), passando per tutti i controlli di qualità intermedi.

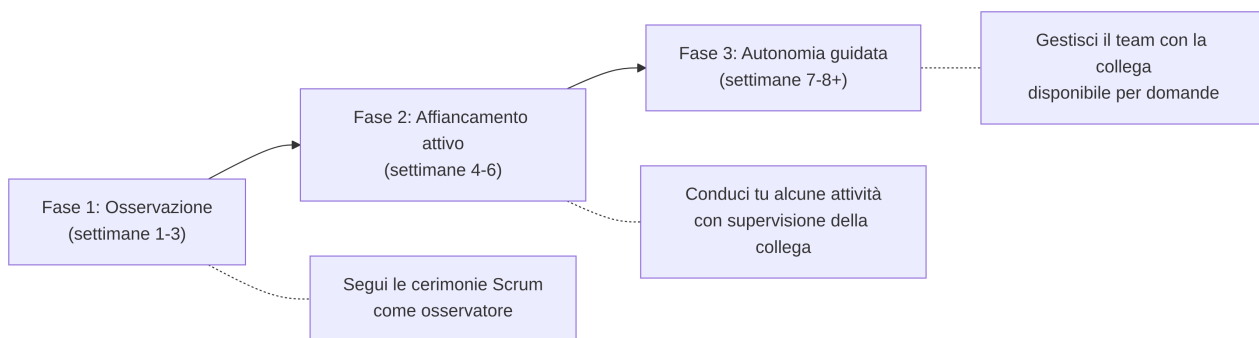
Non preoccuparti se questa descrizione ti sembra ancora vaga: è voluto. Da qui in avanti, sezione dopo sezione, ogni singolo “ingrediente” di questa cucina (Git, Scrum, Kanban, CI/CD, Cloud, Sicurezza...) verrà spiegato nel dettaglio, con esempi concreti tratti dal lavoro quotidiano del team.

Per rendere questi esempi davvero concreti (e non un'accozzaglia di casi slegati tra loro), in tutto il corso useremo sempre lo stesso scenario: **ShopFacile**, una piattaforma e-commerce (catalogo prodotti, carrello, ordini, pagamenti, sconti) sviluppata da un piccolo team interno. Il team è composto da **Marco** e **Giulia**, developer con esperienza medio-alta (Giulia è particolarmente attenta a test e qualità), **Ahmed**, developer più junior e in crescita, **Sara**, Product Owner che rappresenta le esigenze del business, e **Luca**, Scrum Master/Team Lead — il ruolo che tu stesso/a stai imparando a coprire. Ogni volta che nelle prossime sezioni troverai un esempio pratico, sarà quasi sempre un episodio della vita di ShopFacile e del suo team: così, invece di dover ricordare scenari diversi ogni volta, potrai concentrarti sul concetto nuovo riconoscendo persone e progetto già familiari.

Esempio pratico: Ahmed, il developer più junior del team, completa una nuova funzionalità di ShopFacile (un nuovo filtro di ricerca nel catalogo prodotti) e la carica su Git (il “quaderno delle ricette”, lo vedremo nella sezione 4). Da quel momento, in automatico, una serie di controlli (i “controlli qualità” della pipeline) verificano che il codice non rompa nulla di esistente. Se tutto va bene, la funzionalità arriva prima in un ambiente di test, poi — dopo l'ok di Giulia in fase di review — in produzione, cioè visibile ai clienti reali del negozio online. Il tuo ruolo, in questo flusso, non è scrivere codice: è assicurarti che tutti (incluse Sara e Luca) sappiano cosa sta succedendo, che eventuali blocchi (un test che fallisce, un ambiente non disponibile) vengano comunicati subito, e che il cliente sia aggiornato su quando la funzionalità sarà disponibile.

Il tuo percorso: da Junior Project Manager a Scrum Master

Il tuo percorso di crescita in questi 2-3 mesi seguirà tre fasi, che si sovrappongono gradualmente:



- **Fase 1 - Osservazione:** parteciperai alle riunioni del team (le vedremo nella sezione 6 su Scrum: si chiamano “cerimonie” o “eventi”, come la Daily Scrum o lo Sprint Planning) semplicemente osservando. In questa fase il tuo compito principale è studiare le sezioni di questo corso e collegare quello che leggi a quello che vedi succedere dal vivo.
- **Fase 2 - Affiancamento attivo:** comincerai a condurre tu stesso/a alcune attività — ad esempio facilitare una Daily Scrum, aggiornare la board del progetto, preparare un report per il cliente — mentre la tua collega osserva e ti dà feedback.
- **Fase 3 - Autonomia guidata:** gestirai in autonomia le attività di routine del ruolo, con la tua collega disponibile per domande su casi più complessi, finché il passaggio di consegne non sarà completo.

Questo non è un percorso “accademico” fine a se stesso: è un percorso pensato per portarti, passo dopo passo, a **sostituire operativamente** una persona con un ruolo di responsabilità. Per questo è fondamentale che, oltre a leggere le sezioni, tu **osservi il lavoro reale del team** e faccia domande — tante domande — alla tua collega e al resto del team.

Esempio pratico: nella Fase 1, durante una Daily Scrum, ti limiti ad ascoltare e prendere appunti su chi dice cosa. Nella Fase 2, è la tua collega a chiederti: “oggi la conduci tu, io resto in ascolto e ti do feedback alla fine” — magari dovrai gestire un membro del team che si dilunga troppo su un problema tecnico, riportando la discussione sui binari giusti. Nella Fase 3, sei tu a organizzare la Daily Scrum in autonomia, e la tua collega interviene solo se le chiedi aiuto su un caso particolare (ad esempio, come gestire un conflitto tra due membri del team sulle priorità).

Come affrontare lo studio: qualche consiglio pratico

Prima di iniziare, qualche indicazione su come vivere questo percorso senza scoraggiarti:

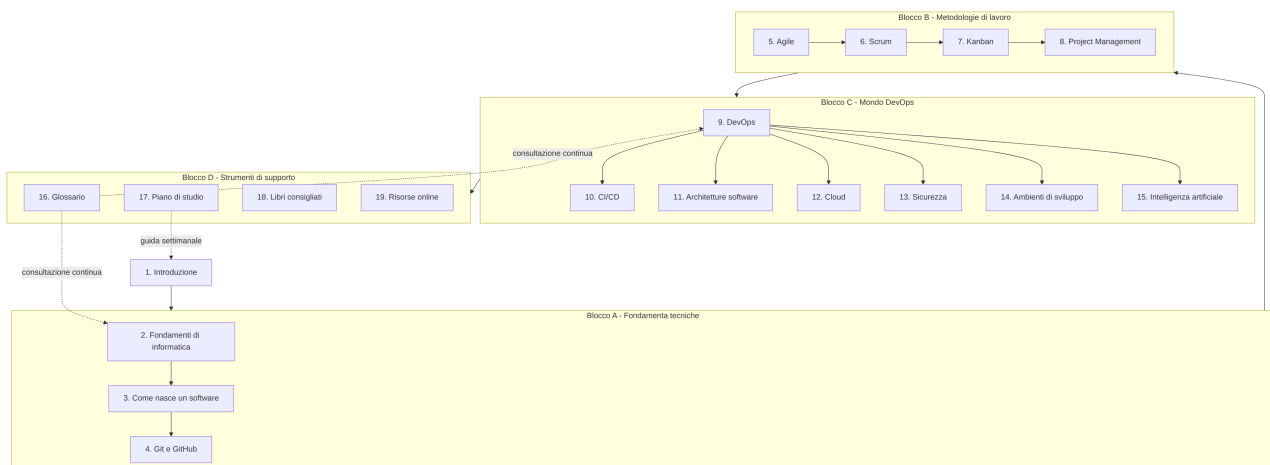
1. **Non devi capire tutto la prima volta.** Concetti come “pipeline CI/CD” o “architettura a microservizi” richiedono più di una lettura per sedimentare. È previsto che tu debba tornare indietro e rileggere: non è un tuo limite, è la normalità quando si impara qualcosa di completamente nuovo.

2. **Fai domande, sempre.** Non esiste una domanda “troppo banale” in questo percorso. Chi lavora ogni giorno con questi strumenti a volte dà per scontati concetti che per te sono nuovi: chiedere è il modo più veloce per colmare quel divario, molto più veloce che aspettare di capire tutto da solo/a leggendo.
3. **Collega la teoria a quello che vedi sul campo.** Ogni volta che in una riunione o in una chat di lavoro senti un termine nuovo, provalo a ricercare nel [Glossario](#) o pensa a quale sezione del corso lo tratta. Questo doppio binario (teoria + osservazione pratica) è il modo più efficace per imparare in questo contesto.
4. **Prova gli esempi, non limitarti a leggerli.** Dove il corso propone un esempio pratico (una board Kanban, un comando Git, un diagramma di pipeline), se possibile provalo tu stesso/a in un ambiente di prova. Capire leggendo è utile, ma capire “con le mani” resta molto più solido.
5. **Usa il piano di studio come bussola, non come gabbia.** Il [Piano di studio](#) a 9 settimane è una guida, non un obbligo rigido: se una settimana hai bisogno di più tempo su un argomento (capita spessissimo con i fondamenti di informatica, sezione 2), prenditelo. È molto meglio capire bene una sezione in più tempo che correre e arrivare in fondo con basi fragili.
6. **A fine settimana, fai un check-in con la tua collega o il tuo mentor.** Un breve confronto settimanale ti permette di validare cosa hai capito, chiarire dubbi rimasti in sospeso e ricevere indicazioni su cosa osservare la settimana successiva nel lavoro reale del team.

Esempio pratico: leggi nella sezione 7 cos'è un limite di WIP (Work In Progress) su una board Kanban. Il giorno dopo, durante una riunione, senti la tua collega dire “non possiamo prendere in carico un'altra card, siamo già al limite in ‘In Corso’”. Invece di lasciarlo passare, collegalo subito a quanto letto e magari chiedi: “è questo il limite di WIP di cui si parlava nella sezione 7?”. Questo piccolo gesto — collegare una lettura a una frase sentita dal vivo — è esattamente il “doppio binario” di cui si parla al punto 3.

La mappa del corso

Il diagramma seguente mostra come le 19 sezioni si collegano tra loro a grandi linee. Non è necessario memorizzarlo: serve solo per farti un'idea di dove ti troverai, sezione dopo sezione.



In sintesi: prima impari il **linguaggio di base** del software (Blocco A), poi **il modo in cui i team si organizzano** per costruirlo (Blocco B), poi **gli strumenti e i processi specifici** della piattaforma DevOps che gestirai, incluso l'impatto dell'intelligenza artificiale sul lavoro quotidiano (Blocco C), e infine hai a disposizione un **kit di strumenti di consultazione** (Blocco D) da usare per tutta la durata del percorso e anche dopo.

Pronto/a per iniziare?

Nella prossima sezione, [2. Fondamenti di informatica](#), partiremo dalle basi più elementari: cosa è un computer, come funziona una rete, cosa sono un server e un database. Sono i mattoncini su cui si costruirà tutto il resto del corso, quindi prenditi il tempo che serve.

Buon percorso!



Esercizi pratici

Prima di passare alla sezione 2, prova questi esercizi: non richiedono ancora conoscenze tecniche, servono solo a farti “digerire” la mappa del corso e il contesto in cui lavorerai.

1. **Disegna la mappa del corso a modo tuo.** Su un foglio o in un documento, riscrivi con parole tue (senza copiare) i quattro blocchi (Fondamenta tecniche, Metodologie di lavoro, Mondo DevOps, Strumenti di supporto) e per ciascuno annota una sola parola chiave che ti aiuti a ricordarlo. **Come verificare:** se riesci a spiegare a voce alta, in meno di 90 secondi e senza guardare il foglio, “di cosa parla ciascuno dei quattro blocchi”, l'esercizio è riuscito.
2. **Racconta l'analogia della cucina a qualcuno.** Prova a spiegare a un amico o familiare (anche non del settore) l'analogia della cucina del ristorante usata in questa sezione, sostituendo i termini tecnici con quelli che hai appena imparato (cuochi = sviluppatori, pipeline = linea di produzione, ecc.). **Come verificare:** se la persona a cui lo racconti riesce a ripeterti, con parole sue, cosa fa un Project Manager/Scrum Master in questa analogia, hai comunicato bene il concetto.
3. **Individua le tue tre fasi nel calendario.** Segna sul tuo calendario (anche approssimativamente) le date di inizio previste delle tre fasi del tuo percorso (Osservazione, Affiancamento attivo, Autonomia guidata) in base a quando sei stato/a assunto/a. **Come verificare:** mostra il calendario alla tua collega Scrum Master/PM e chiedile se le date che hai stimato sono realistiche rispetto al carico di lavoro reale del team in quel periodo.
4. **Fai una prima esplorazione del Glossario.** Apri il [Glossario](#) e scorilo per 5 minuti senza l'ansia di memorizzare nulla: individua 3 termini che hai già sentito nominare in azienda (anche di striscio, in corridoio o in chat) ma che non sapevi spiegare con precisione. **Come verificare:** scrivi i 3 termini scelti e una definizione con parole tue in una nota; a fine settimana 1 del piano di studio, riguardala e controlla se la tua definizione è ancora corretta o va corretta.

5. **Identifica chi è chi nel tuo contesto di lavoro.** Chiedi alla tua collega o a un membro del team di aiutarti a fare un elenco veloce (basta un elenco a punti, non serve un documento formale) di chi sono, nel progetto reale su cui lavorerai, gli “sviluppatori/cuochi”, chi si occupa di “operations” e chi è il referente lato cliente.  **Come verificare:** se dopo questo esercizio sai dire, senza esitare, a chi rivolgerti per un problema tecnico e a chi per una domanda del cliente, l'esercizio ha funzionato.

Collegamenti

- [2. Fondamenti di informatica](#) — le basi tecniche da cui parte tutto il resto del corso
- [16. Glossario](#) — da consultare ogni volta che incontri un termine nuovo
- [17. Piano di studio](#) — il programma dettagliato delle 9 settimane
- [19. Risorse online](#) — materiali extra se vuoi approfondire fin da subito

Risorse

- [Atlassian – Agile Coach: cos'è un team Agile](#) — introduzione generale al mindset Agile, utile come primo assaggio prima delle sezioni 5-7
- [GitHub Docs – Get started with Git](#) — guida ufficiale per chi non ha mai usato Git, utile in anticipo sulla sezione 4
- [Microsoft Learn – Cos'è DevOps](#) — panoramica ufficiale Microsoft sul concetto di DevOps, come anteprima della sezione 9
- [Project Management Institute – What is Project Management](#) — visione generale del ruolo di Project Manager, come richiamo al tuo percorso di studi

2. Fondamenti di informatica

Questa è una delle sezioni più importanti di tutto il corso. Non preoccuparti se non hai mai scritto una riga di codice o non sai cosa sia un server: qui costruiamo insieme, da zero, tutto il vocabolario tecnico che ti servirà per capire il resto del percorso (Git, Agile, DevOps, Cloud...).

Non serve leggere tutto in una sola sessione. Prendi questa sezione a piccoli pezzi, prova a rileggere gli esempi con i tuoi tempi e, se un concetto non è chiaro, chiedi al tuo mentor. È normale che alcuni termini richiedano più di una lettura per “entrare” davvero.

Obiettivi della sezione

Al termine di questa sezione saprai:

- distinguere hardware e software, e conoscere i componenti principali di un computer (CPU, RAM, disco);
- spiegare cosa fa un sistema operativo e cosa sono processi e thread;
- capire come sono organizzati i file su un computer;
- spiegare in modo semplice come funziona una rete, internet, e i protocolli TCP/IP, HTTP/HTTPS, DNS;
- capire cos'è un'API, cosa significa REST, riconoscere le quattro operazioni **CRUD** e leggere un semplice JSON, e capire a cosa servono **Swagger/OpenAPI** come contratto tra due team;
- capire la differenza tra database relazionali e NoSQL, cosa sono **chiave primaria**, **chiave esterna** e **JOIN**, e perché la **normalizzazione** evita di scrivere la stessa informazione più volte;
- capire cosa sono Virtual Machine, Container, Docker, Kubernetes e **OpenShift** — i mattoncini su cui si basa gran parte del mondo DevOps che incontrerai nel team;
- distinguere **Markdown** e **HTML**, i due linguaggi con cui è scritto anche questo stesso corso.

2.1 Cos'è un computer: hardware e software

Partiamo dalla domanda più semplice possibile: **cos'è un computer?**

Un computer è una macchina che esegue istruzioni. Punto. Tutto quello che fa — mostrarti una pagina web, farti scrivere un documento, farti guardare un video — è il risultato di miliardi di piccole istruzioni eseguite a altissima velocità.

Per capire come funziona, dobbiamo separare due concetti fondamentali:

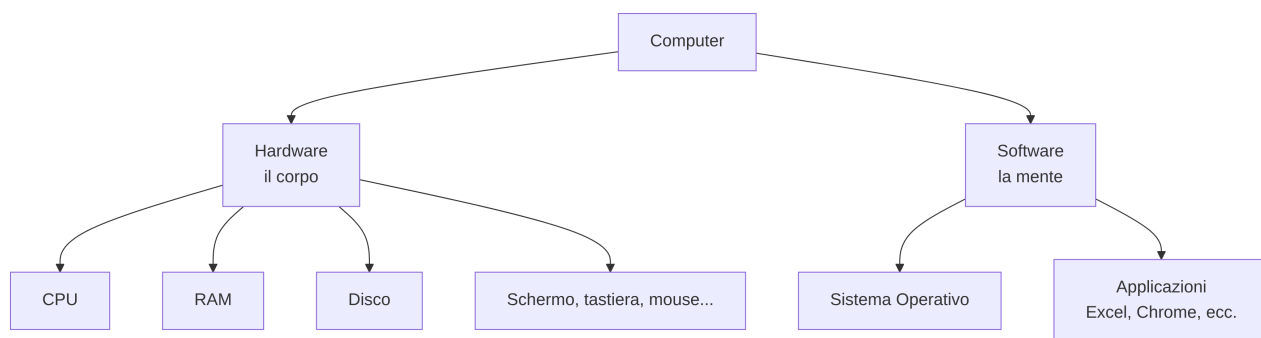
- **Hardware:** sono i componenti fisici, quelli che puoi toccare. Lo schermo, la tastiera, il “cervello” interno (CPU), la memoria, il disco.
- **Software:** sono le istruzioni, i programmi. Non li puoi toccare, ma “dicono” all'hardware cosa fare. Windows, Excel, Chrome, WhatsApp sono tutti software.

Analogia: il corpo e la mente

Pensa al computer come a una persona:

- l'**hardware** è il **corpo**: braccia, gambe, occhi, cervello (come organo fisico). Serve per agire nel mondo fisico.
- il **software** è la **mente**: i pensieri, le conoscenze, le decisioni. È ciò che dice al corpo cosa fare e come farlo.

Un corpo senza una mente che lo guidi non farebbe nulla di utile: sta lì, fermo. Una mente senza un corpo non potrebbe agire nel mondo. Hardware e software funzionano allo stesso modo: uno senza l'altro non serve a niente.



Nelle prossime sezioni vediamo nel dettaglio i pezzi principali dell'hardware (CPU, RAM, disco) e poi il software che li coordina (il sistema operativo).

Esempio pratico

Il laptop di lavoro di Marco è hardware (schermo, tastiera, CPU, RAM). Su di esso girano software come il sistema operativo, il browser e il client per collegarsi in VPN al progetto. Allo stesso modo, un server che ospita **ShopFacile**, la piattaforma e-commerce su cui lavora il team, è hardware (fisico o virtuale, in cloud) su cui girano software come il sistema operativo Linux, il motore container e l'applicazione stessa.

2.2 La CPU: il cervello che calcola

Abbiamo detto che l'hardware è il "corpo" del computer: cominciamo a sezionarlo, partendo dal componente che fa più "pensare" tra tutti, la CPU.

La **CPU** (Central Processing Unit, "unità centrale di elaborazione") è il componente che esegue le istruzioni. È lei che fa i calcoli, prende le decisioni logiche ("se questo è vero, fai questo, altrimenti fai quello") e coordina il lavoro di tutti gli altri componenti.

Analogia: la persona che fa i calcoli

Immagina un ufficio dove arrivano richieste continue: “calcola questo”, “confronta questi due numeri”, “copia questo dato qui”. La CPU è la persona seduta alla scrivania che esegue materialmente queste operazioni, una dopo l'altra, a una velocità incredibile: miliardi di operazioni al secondo.

Alcuni concetti utili da conoscere (non servono i dettagli tecnici, solo il significato generale, perché li sentirai nominare):

- **Core (nucleo):** una CPU moderna ha più “core”, cioè più unità di calcolo indipendenti nello stesso chip. È come avere più persone alla scrivania invece di una sola: possono lavorare su cose diverse in parallelo.
- **Frequenza (GHz):** quante operazioni al secondo può fare, in miliardi di cicli. Più alta è la frequenza, più velocemente lavora (semplificando molto).

Perché ti interessa? Perché quando in futuro sentirai parlare di “server con 8 core” o “la CPU è sotto stress”, saprai che si parla della capacità di calcolo della macchina.



Esempio pratico

Durante un picco di traffico sulla piattaforma, il team nota su una dashboard di monitoring che la CPU di un server è al 95% di utilizzo per diversi minuti. Questo è un segnale che la macchina fa fatica a gestire tutte le richieste in arrivo, e il team decide di aggiungere un altro container (ne parleremo più avanti) per distribuire il carico su più “persone alla scrivania”.

2.3 La RAM: la memoria di lavoro temporanea

La CPU, però, non calcola nel vuoto: ha bisogno di un posto dove tenere a portata di mano i dati su cui sta lavorando in questo preciso istante. Quel posto è la RAM.

La **RAM** (Random Access Memory) è la memoria che il computer usa mentre sta lavorando. È velocissima, ma ha due caratteristiche importanti:

1. è **temporanea**: quando spegni il computer, tutto quello che c'era in RAM viene perso;
2. è **limitata**: ha uno spazio finito (es. 8 GB, 16 GB, 32 GB).

Analogia: la scrivania

Pensa alla RAM come alla tua **scrivania** mentre lavori. Ci metti sopra i fogli, i libri, gli oggetti che ti servono in questo momento, perché prenderli dallo scaffale ogni volta sarebbe troppo lento. La scrivania è comoda e veloce da usare, ma ha spazio limitato: se hai troppi documenti, devi rimetterne alcuni nell'armadio per farne spazio ad altri. E quando vai via alla fine della giornata (spegni il PC), la scrivania viene “sgomberata”.

Quando apri un programma (es. il browser), il computer copia le istruzioni e i dati necessari dal disco (l'armadio, vedi sotto) alla RAM (la scrivania), perché lavorarci da lì è molto più rapido.

Se hai “troppe cose aperte” e il computer diventa lento, spesso è perché la RAM è piena: non c’è più spazio sulla scrivania.

Esempio pratico

Un’applicazione del progetto va in errore con un messaggio del tipo “out of memory” (memoria esaurita) perché il container in cui gira ha un limite di RAM troppo basso per il carico di lavoro reale. Il team operativo interviene aumentando il limite di memoria assegnato al container nella configurazione, e il problema si risolve.

2.4 Il disco: la memoria permanente

Abbiamo visto che la RAM si svuota allo spegnimento: ma allora dove restano salvati i dati di ShopFacile (ordini, prodotti, clienti) anche quando i server vengono riavviati? Serve una memoria che non “dimentichi” nulla.

Il **disco** (hard disk o, oggi più spesso, SSD - Solid State Drive) è la memoria dove i dati restano salvati **anche quando spegni il computer**. È più lento della RAM, ma ha molto più spazio ed è permanente.

Analogia: l’armadio

Il disco è come l’**armadio** di casa tua: ci conservi tutto quello che ti serve nel lungo periodo (documenti, vestiti, ricordi). Non è comodo come la scrivania per lavorare (aprire l’armadio, cercare la cosa giusta, richiuderlo richiede più tempo), ma è lì che le cose restano anche quando esci di casa o vai a dormire.

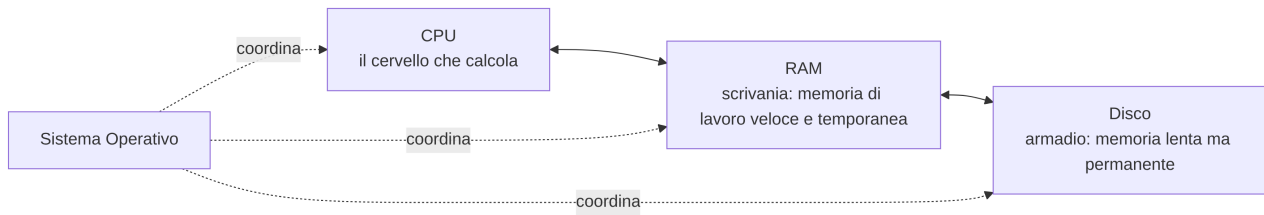
	RAM (scrivania)	Disco (armadio)
Velocità	Molto veloce	Più lenta (SSD comunque abbastanza rapido)
Persistenza	Si perde tutto allo spegnimento	Resta salvato
Spazio tipico	Pochi GB (es. 16 GB)	Centinaia di GB / TB
Uso	Lavoro “in corso”	Archiviazione permanente

Questa distinzione RAM/disco tornerà utile più avanti nel corso, ad esempio quando parleremo di database (dati salvati su disco) o di **cache**: una copia dei dati usati più di frequente, tenuta in RAM apposta per non dover andare a recuperarli ogni volta dal disco più lento, risparmiando tempo.

Esempio pratico

Il team riceve un alert automatico perché il disco del server che ospita il database è occupato all’85%. Se non si interviene (ad esempio archiviando o cancellando log vecchi, oppure aumentando lo spazio disponibile), il database rischia di non poter più scrivere nuovi dati quando il disco si riempie completamente.

CPU, RAM e Disco insieme



La CPU lavora sui dati che trova in RAM (velocissima ma volatile); la RAM, a sua volta, recupera dal disco (lento ma permanente) ciò che serve quando serve. Il Sistema Operativo, che vediamo ora, è il “direttore” che organizza tutto questo traffico.

2.5 Il Sistema Operativo: chi coordina tutto

Il **Sistema Operativo** (Operating System, OS) è il software che gestisce tutte le risorse hardware del computer (CPU, RAM, disco, schermo...) e permette agli altri programmi (Excel, Chrome, ecc.) di funzionare senza dover “parlare” direttamente con l'hardware. Perché esiste questo intermediario? Senza di lui, ogni sviluppatore dovrebbe riscrivere il proprio software per ogni modello di computer, e due programmi in esecuzione insieme potrebbero rovinarsi a vicenda memoria o file, senza nessuno che arbitri.

I sistemi operativi più diffusi sono:

- **Windows** (Microsoft) — il più usato sui PC aziendali e privati;
- **macOS** (Apple) — sui computer Apple;
- **Linux** — molto usato sui **server** (i computer che erogano servizi via internet) e nel mondo DevOps che vedrai più avanti nel corso.

Analogia: il direttore d'albergo

Immagina un albergo con tanti ospiti (i programmi) che hanno bisogno di risorse condivise: le camere (memoria), il personale (CPU), la cucina (disco). Il direttore dell'albergo (il sistema operativo) decide chi occupa cosa, per quanto tempo, evita che due ospiti litighino per la stessa risorsa, e fa in modo che tutto funzioni in armonia, senza che gli ospiti debbano parlarsi direttamente tra loro per organizzarsi.

Il sistema operativo si occupa, tra le altre cose, di:

- decidere quale programma può usare la CPU in un dato istante;
- gestire quale programma può usare quanta RAM;
- organizzare i file sul disco (vedi il “File System” più sotto);
- gestire la comunicazione con periferiche (stampanti, rete, mouse...);
- fornire un'interfaccia con cui l'utente interagisce (le finestre, le icone, o — su Linux — anche solo una riga di comando testuale, il “terminale”).

Nel mondo di cui ti occuperai come Project Manager su una piattaforma DevOps, sentirai parlare spesso di server **Linux**, perché è il sistema operativo più usato per far funzionare applicazioni web e infrastrutture.



Esempio pratico

Il team pianifica una finestra di manutenzione notturna per applicare un aggiornamento di sicurezza al sistema operativo Linux dei server di produzione, perché l'aggiornamento richiede il riavvio delle macchine: si sceglie un orario a basso traffico per minimizzare l'impatto sugli utenti.

2.6 Processi e Thread

Abbiamo detto che il sistema operativo decide chi usa la CPU e quanta RAM assegnare: ma a “chi”, concretamente? La risposta è: ai processi e ai thread, le unità di lavoro che il sistema operativo gestisce ogni istante.

Quando apri un programma (ad esempio il browser), il sistema operativo crea un **processo**: un'istanza del programma in esecuzione, con la sua porzione di memoria RAM dedicata, isolata dagli altri programmi.

Un processo può, a sua volta, dividersi in più **thread**: filoni di esecuzione più piccoli che lavorano sullo stesso processo, condividendone la memoria, ma potendo procedere in parti diverse del lavoro (anche in parallelo, se la CPU ha più core).

Analogia: un'azienda (processo) e i suoi dipendenti (thread)

Pensa a un **processo** come a un **reparto aziendale**: ha un proprio budget, un proprio spazio (l'ufficio), delle proprie risorse, separate da quelle degli altri reparti. Se un reparto va in crisi, in genere non manda in crisi automaticamente gli altri reparti (isolamento).

I **thread** sono i **dipendenti di quel reparto**: lavorano nello stesso spazio, condividono le stesse risorse (scrivanie, documenti, strumenti), e possono lavorare contemporaneamente su compiti diversi dello stesso progetto per essere più veloci. Se un dipendente combina un disastro con un documento condiviso, però, può creare problemi anche ai colleghi dello stesso reparto, perché condividono lo stesso spazio.

Esempio concreto: quando apri il browser (processo) e navighi su più schede, ogni scheda può essere gestita con thread (o addirittura processi) diversi, in modo che se una pagina web “va in crash” non blocchi necessariamente tutte le altre schede aperte.

Concetto	Cos'è	Analogia
Processo	Programma in esecuzione, con memoria propria isolata	Un reparto aziendale con il proprio ufficio
Thread	Filone di lavoro dentro un processo, condivide memoria con gli altri thread dello stesso processo	I dipendenti dello stesso reparto

Questo concetto tornerà utile quando, più avanti, parlerai di applicazioni che devono gestire **tante richieste contemporaneamente** (es. un sito web con migliaia di visitatori): il modo in cui un software gestisce processi e thread incide molto sulle sue performance.

Gestire tante richieste insieme apre però un problema delicato: cosa succede se due thread (o due processi) modificano lo **stesso dato** nello stesso istante? Se su ShopFacile è disponibile un solo pezzo di un prodotto e due clienti cliccano “Acquista” nello stesso millisecondo, senza una gestione corretta della contemporaneità entrambi gli ordini potrebbero essere accettati. Questo tipo di problema, legato al “chi arriva prima” tra operazioni simultanee, è anche la ragione per cui alcuni bug sono **intermittenti**: si manifestano solo in rare combinazioni di tempismo, e per questo sono tra i più difficili da riprodurre e correggere.

2.7 Il File System: come sono organizzati i file

Processi e thread lavorano sui dati in RAM, ma quei dati — codice, immagini, configurazioni — devono comunque “vivere” da qualche parte sul disco quando non sono in uso: è qui che entra in gioco il file system.

Il **file system** è il modo in cui il sistema operativo organizza e memorizza i file sul disco. Praticamente ogni informazione che il computer salva in modo permanente (documenti, foto, programmi, configurazioni) è un **file**, e i file sono organizzati in **cartelle** (chiamate anche “directory”), che possono contenere altre cartelle, in una struttura ad albero.

Analogia: l'archivio con cartelline

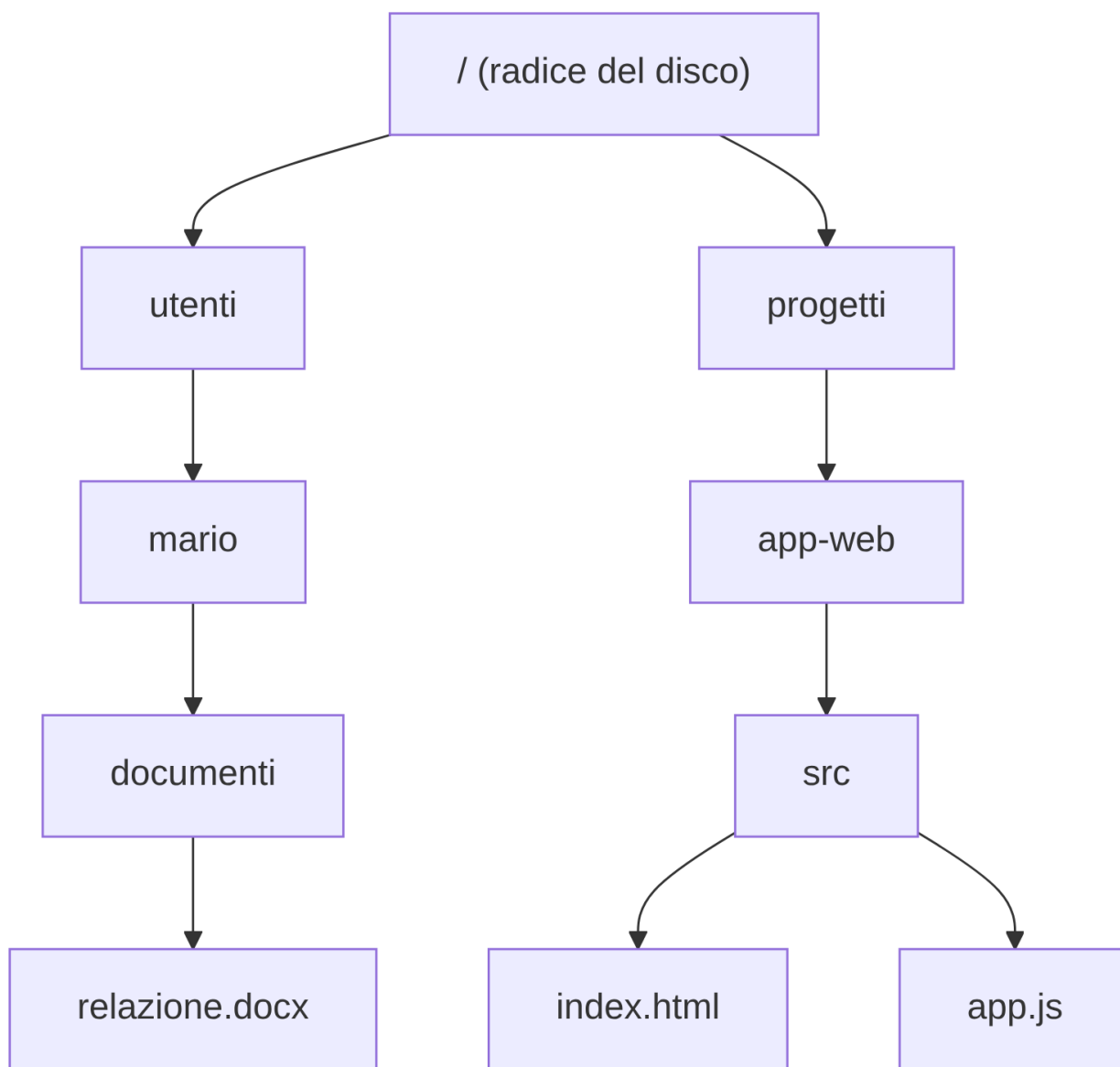
Pensa a un grande archivio fisico con tanti raccoglitori (cartelle), che possono contenere altri raccoglitori più piccoli, che a loro volta contengono i documenti (file). Per trovare un documento specifico, segui un percorso: “vai nell’armadio Contabilità, poi nel raccoglitore 2024, poi nella cartellina Gennaio, prendi il foglio Fattura_012”.

In informatica, questo percorso si chiama **path** (percorso), e si scrive così:

```
/contabilità/2024/gennaio/fattura_012.pdf
```

oppure, su Windows:

Ogni file ha tipicamente un **nome** e un'**estensione** che indica il suo tipo: `.pdf`, `.docx`, `.jpg`, `.txt`, `.py` (codice Python), `.json` (vedi più avanti), ecc.



Capire i path ti servirà moltissimo più avanti, ad esempio quando vedremo **Git** (sezione 4): i progetti software sono organizzati esattamente come cartelle e file su un file system.

💡 Esempio pratico

Marco ti scrive in chat: “il file di configurazione di ShopFacile è in `/app/config/produzione.json`”. Grazie a quello che hai appena imparato, sai leggere quel percorso: parti dalla radice (/), entri nella cartella `app`, poi in `config`, e lì trovi il file `produzione.json`. Non serve saperlo modificare, ma saperlo “leggere” ti permette di seguire la conversazione senza sentirti perso.

2.8 La rete: cos'è una rete di computer

Finora abbiamo parlato di un singolo computer — CPU, RAM, disco, sistema operativo, file. Ma un server con ShopFacile installato sopra, da solo e isolato, non servirebbe a nessuno: deve poter essere raggiunto dai clienti che navigano da casa loro. Il valore vero della tecnologia moderna nasce quando i computer **si parlano tra loro**: questo è possibile grazie alle **reti**.

Una **rete di computer** è semplicemente un insieme di dispositivi collegati tra loro, capaci di scambiarsi informazioni.

Può essere piccola (la rete Wi-Fi di casa tua, con il tuo PC, telefono, smart TV) o enorme: **internet** è la rete di reti più grande del mondo, che collega miliardi di dispositivi.

Analogia: il sistema postale

Pensa a una rete come al **sistema postale**: ogni casa ha un indirizzo univoco, le lettere vengono instradate attraverso uffici postali intermedi fino a raggiungere il destinatario giusto, indipendentemente da dove si trovi. Anche i computer hanno un “indirizzo” (si chiama **indirizzo IP**) e le informazioni viaggiano in “pacchetti” (come lettere) attraverso una serie di dispositivi intermedi (router) finché non arrivano a destinazione.



Esempio pratico

I clienti segnalano che ShopFacile “non si carica più”. Marco, che si occupa spesso di infrastruttura, scopre che il problema non è nell'applicazione stessa, ma in un guasto di rete tra due data center che impedisce ai server di raggiungersi: un classico problema “di rete”, da distinguere da un bug nel codice.

2.9 TCP/IP: come i computer si “spediscono lettere”

Abbiamo detto che una rete permette a dispositivi diversi di scambiarsi informazioni: ma con quali regole precise avviene questo scambio? Le definisce un protocollo di base che sta sotto (quasi) tutto il resto, TCP/IP.

TCP/IP è l'insieme di regole (un “protocollo”, cioè un linguaggio comune condiviso) che permette a due computer di scambiarsi dati su una rete, compreso internet. È, in un certo senso, l'alfabeto di base su cui si basa tutta la comunicazione online.

Semplificando molto:

- **IP** (Internet Protocol) si occupa dell'**indirizzamento**: assegna a ogni dispositivo un indirizzo univoco (l'**indirizzo IP**, es. **192.168.1.10**) e si occupa di far viaggiare i dati verso l'indirizzo giusto, passando per i router intermedi — proprio come il sistema postale smista le lettere in base all'indirizzo.

- **TCP** (Transmission Control Protocol) si occupa di **spezzare i dati in pacchetti**, spedirli, e assicurarsi che arrivino tutti, nell'ordine giusto, e — se qualcosa si perde per strada — di rispeditirlo. È come spedire un libro intero per posta spezzandolo in tante lettere numerate: il destinatario le riceve, le rimette in ordine con i numeri, e se manca la lettera 5 chiede di rispeditirla.

Analogia riassuntiva

Immagina di dover spedire un grosso pacco a un amico in un'altra città:

1. lo dividi in scatole più piccole numerate (TCP: divisione in pacchetti e controllo dell'ordine/integrità);
2. scrivi l'indirizzo del destinatario su ogni scatola (IP: indirizzamento);
3. le scatole passano per vari centri di smistamento (i router) prima di arrivare a destinazione;
4. il destinatario le riceve, controlla che siano tutte arrivate e le ricompone nell'ordine giusto.

Per il tuo lavoro da PM è utile soprattutto sapere che TCP/IP è il **livello base** su cui si costruiscono protocolli più “di alto livello” che userai spesso a parole, come HTTP.



Esempio pratico

Un ticket segnala: “il server non risponde sulla porta 443”. Se quella porta TCP (usata per HTTPS) è chiusa o bloccata sul firewall, nessuna richiesta arriva a destinazione, anche se il server è acceso e funzionante.

2.10 HTTP e HTTPS: il protocollo del web

TCP/IP garantisce che i dati arrivino a destinazione, ma non dice nulla su **come** un browser deve chiedere una pagina web a un server: per quello serve un protocollo di livello più alto, costruito sopra TCP/IP, che è HTTP.

HTTP (HyperText Transfer Protocol) è il protocollo usato per trasferire pagine web e dati tra un **client** (es. il tuo browser) e un **server** (il computer che “ospita” il sito). È costruito sopra TCP/IP: se TCP/IP è “come spedire un pacco”, HTTP è “il modulo che compili per chiedere qualcosa e come è strutturata la risposta”.

Il funzionamento di base è semplice: **richiesta e risposta**.

1. Il client manda una **richiesta** (“dammi la pagina della home”);
2. il server elabora la richiesta e manda una **risposta** (il contenuto della pagina, oppure un errore come il famoso “404 - pagina non trovata”).



La “S” di sicurezza: HTTPS

HTTPS è la versione **sicura** di HTTP: la “S” sta per “Secure”. La differenza è che i dati scambiati vengono **cifrati** (criptati), cioè trasformati in un codice illeggibile per chiunque li intercetti lungo il tragitto, e solo il client e il server legittimi possono “decifrarli” e leggerli.

Analogia: cartolina vs busta sigillata

- **HTTP** è come spedire una **cartolina postale**: chiunque la maneggi lungo il percorso (postini, magazzinieri) può leggerne il contenuto.
- **HTTPS** è come spedire una **lettera in una busta sigillata e cifrata**, che solo il destinatario ha la “chiave” per aprire e leggere.

Per questo, oggi, praticamente tutti i siti seri usano HTTPS: quando navighi e vedi il lucchetto  nella barra degli indirizzi del browser, significa che la connessione è protetta con HTTPS.

Esempio pratico

Durante un test di sicurezza su ShopFacile, viene segnalato che una vecchia pagina interna è ancora raggiungibile in HTTP (senza cifratura). Giulia, sempre attenta alla qualità, la corregge configurando un redirect automatico verso la versione HTTPS, così chi provasse ad accedere alla versione non sicura viene reindirizzato automaticamente a quella protetta.

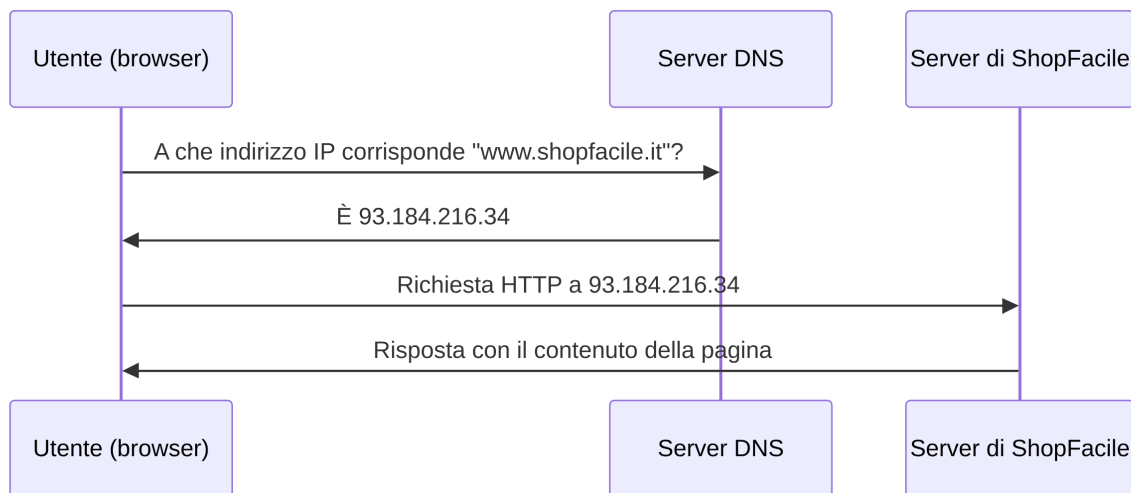
2.11 DNS: la rubrica telefonica di internet

Nella sezione precedente abbiamo visto che HTTP fa viaggiare richiesta e risposta tra client e server, ma un server, come ogni computer su internet, è identificato da un **indirizzo IP** (es. **142.250.180.4**). Ma un cliente, per visitare il sito, digita nel browser un nome facile da ricordare come **www.shopfacile.it**, non una sequenza di numeri. Chi fa la traduzione da nome a indirizzo IP?

Il **DNS** (Domain Name System)!

Analogia: la rubrica telefonica

Il DNS funziona come una **rubrica telefonica**: tu vuoi chiamare “Luca”, non ricordi il suo numero di telefono a memoria, quindi consulti la rubrica, trovi il nome “Luca” e la rubrica ti restituisce il numero corretto da chiamare. Il DNS fa esattamente questo per internet: un cliente chiede `www.shopfacile.it`, il DNS gli risponde con l'indirizzo IP del server giusto, e solo dopo il suo browser può effettivamente contattarlo.



Senza il DNS dovremmo ricordare a memoria stringhe di numeri per ogni sito che vogliamo visitare: praticamente impossibile.

Esempio pratico

Dopo aver spostato ShopFacile su un nuovo server, il team aggiorna il record DNS. Per alcune ore, però, alcuni clienti continuano a raggiungere il vecchio server: è un fenomeno normale, chiamato “propagazione DNS”, perché la modifica impiega un po’ di tempo a diffondersi su tutti i server DNS del mondo.

2.12 API: come i software si parlano tra loro

Finora abbiamo visto come un client raggiunge un server (DNS, HTTP). Ma una volta raggiunto, come chiede esattamente “dammi la lista prodotti” o “registra questo ordine” in un linguaggio che il server capisca? Con le API.

API sta per **Application Programming Interface** (“interfaccia di programmazione delle applicazioni”). È il modo in cui un software espone delle “funzionalità” che altri software possono usare, senza dover conoscere i dettagli interni di come funziona.

Analogia: il cameriere al ristorante

Pensa a un ristorante: tu (il **cliente/client**) non entri in cucina a prepararti da mangiare da solo. Ordini un piatto al **cameriere**, che porta la tua richiesta in **cucina** (il **server**), aspetta che il piatto sia pronto, e te lo riporta al tavolo.

Il cameriere è l'**API**: è l'intermediario con **regole precise** (il menù: puoi ordinare solo quello che è scritto lì, in un formato preciso) che permette a te (client) di ottenere qualcosa dalla cucina (server) senza sapere come funzionano i fornelli o le pentole.

In termini informatici:

- un'app di meteo sul tuo telefono non "misura" la temperatura da sola: fa una **richiesta API** a un servizio esterno che gestisce i dati meteorologici, e riceve una **risposta** con i dati richiesti;
- ShopFacile, al momento del pagamento, usa le API di un servizio di pagamento esterno per gestire il pagamento con carta, senza dover costruire da zero un intero sistema di pagamenti.



Le API sono uno dei concetti più importanti che incontrerai lavorando su una piattaforma software: praticamente tutte le applicazioni moderne sono fatte di tanti "pezzi" (servizi) che comunicano tra loro tramite API.

Esempio pratico

Sara chiede al team di aggiungere l'invio di notifiche via SMS ai clienti di ShopFacile quando un ordine viene spedito. Invece di costruire da zero un intero sistema per inviare SMS (server, connessioni con gli operatori telefonici, ecc.), Marco integra le API REST di un servizio esterno specializzato: basta inviare una richiesta con numero e testo del messaggio, e il servizio esterno si occupa di tutto il resto.

2.13 REST: uno stile comune per costruire API

Le API, come abbiamo appena visto, permettono a due software di parlarsi. Ma perché tutti riescano a capirsi senza reinventare ogni volta le regole del "dialogo", serve un modo condiviso di costruirle: lo stile REST.

REST (REpresentational State Transfer) non è una tecnologia specifica, ma uno **stile**, un insieme di convenzioni condivise su come costruire le API in modo semplice, standard e prevedibile, sfruttando proprio HTTP che abbiamo visto sopra.

Le API costruite seguendo lo stile REST si chiamano **API REST** (o “RESTful”), e sono lo standard più diffuso oggi per far comunicare applicazioni web.

L'idea di base: si usano gli indirizzi web (URL) per identificare le **risorse** (es. “un cliente”, “un ordine”), e si usano dei **verbi HTTP** standard per indicare l'azione da compiere su quella risorsa:

Verbo HTTP	Significato	Analogia
GET	Leggi/recupera dati	“Fammi vedere l'ordine numero 42”
POST	Crea un nuovo elemento	“Crea un nuovo ordine”
PUT / PATCH	Modifica un elemento esistente	“Aggiorna l'indirizzo di spedizione dell'ordine 42”
DELETE	Elimina un elemento	“Cancella l'ordine 42”

Esempio pratico: l'API REST che ShopFacile usa per gestire gli ordini potrebbe avere un indirizzo come:

```
GET https://api.shopfacile.it/ordini/42
```

per “recuperare i dettagli dell'ordine con id 42”.

Sentirai questo termine costantemente parlando con gli sviluppatori del team, quindi è importante che il concetto ti sia chiaro.

CRUD: le quattro operazioni fondamentali

Guarda di nuovo la tabella dei verbi HTTP appena vista: leggere, creare, modificare, eliminare. Non è un caso che si ripresentino quasi identiche in ogni sistema che gestisce dati — hanno un nome tecnico che sentirai usare in continuazione dagli sviluppatori: **CRUD**, l'acronimo di **Create, Read, Update, Delete** (Crea, Leggi, Aggiorna, Elimina).

Sono le quattro operazioni fondamentali che si possono fare su un qualsiasi dato salvato da qualche parte — un ordine, un cliente, una polizza — e si ritrovano a ogni livello che hai incontrato finora:

Operazione	Verbo HTTP	Comando SQL (sezione 2.15)
Create (crea)	POST	INSERT
Read (leggi)	GET	SELECT
Update (aggiorna)	PUT / PATCH	UPDATE
Delete (elimina)	DELETE	DELETE

Perché ti interessa da PM? Perché quando un tecnico dice “è solo un CRUD” sta dicendo qualcosa di preciso sulla stima: che la funzionalità richiesta si limita a creare, leggere, aggiornare ed eliminare un dato, senza logica di business particolare dietro — un lavoro relativamente standard e prevedibile. Quando invece dice “non è un CRUD” (o “c’è più logica sotto”), ti sta avvisando che dentro quell’operazione ci sono regole, calcoli, verifiche, integrazioni con altri sistemi: la stima va rivista al rialzo, ed è un segnale che vale la pena approfondire prima di fissare una scadenza.

Swagger e OpenAPI: il contratto tra due team

Immagina che Ahmed debba esporre un’API REST per gli ordini di ShopFacile, e che un partner esterno (un servizio di logistica) debba integrarsi con quell’API per ricevere gli ordini da spedire. Il modo “artigianale” di gestire questo accordo è un documento Word con l’elenco delle chiamate possibili, i campi richiesti, un paio di esempi. Funziona per una settimana: poi il documento invecchia, qualcuno cambia un campo nel codice senza aggiornarlo, e il team esterno scopre — in fase di test, non prima — che la specifica dice `codiceCliente` mentre l’API restituisce davvero `customerCode`. L’integrazione si rompe, e nessuno dei due team ha “sbagliato”: si stavano semplicemente fidando di un documento che nessuno aveva l’obbligo di mantenere sincronizzato con la realtà.

Da questo problema nasce **OpenAPI**: uno standard per descrivere un’API REST — le sue risorse, i verbi disponibili, i campi di ogni richiesta e risposta — in un formato leggibile sia da una persona che da una macchina (tipicamente JSON o YAML, il formato JSON lo hai visto in §2.14). **Swagger** è il nome della famiglia di strumenti storicamente associata a OpenAPI: a partire dalla stessa descrizione, genera automaticamente documentazione navigabile nel browser — che permette di “provare” le chiamate senza scrivere una riga di codice — oltre a codice di base (client e stub) per chi deve integrarsi.

Il punto per un PM non è lo strumento, ma il cambio di natura del documento: una specifica OpenAPI non è più “un documento di cui ci fidiamo”, è un **contratto** verificabile. Questo permette a due team di lavorare **in parallelo**: chi fornisce l’API e chi deve consumarla possono partire nello stesso momento, il secondo costruendo la propria integrazione contro la specifica anche prima che il servizio esista davvero.

 **Trade-off:** il contratto va comunque mantenuto a mano — se Ahmed cambia l’API e non aggiorna la specifica OpenAPI, il problema del documento Word invecchiato torna identico, solo con un formato più elegante. E una specifica formalmente corretta ma concettualmente sbagliata (es. descrive un campo come opzionale quando il servizio in realtà lo richiede sempre) dà a entrambi i team una falsa sicurezza: tutti si fidano di un contratto che non corrisponde al comportamento reale.

2.14 JSON e XML: i formati per scambiarsi dati

Abbiamo visto come un’API REST identifica una risorsa come “l’ordine 42” e come intervenireci (GET, POST...). Ma in che formato viaggiano concretamente i dati di quell’ordine dentro la richiesta e la risposta? Qui entrano in gioco JSON e XML.

Immagina cosa succederebbe senza un formato comune: ogni sistema inventerebbe il proprio modo di scrivere “l’ordine 42, cliente Mario Rossi, totale 129,90”, e ogni coppia di sistemi che deve parlarsi (il catalogo, i pagamenti, le spedizioni...) richiederebbe un “traduttore” dedicato, con il numero che esplode rapidamente al crescere dei sistemi coinvolti. Per questo due software che si scambiano dati (ad esempio tramite un’API) usano un **formato comune**, condiviso, che entrambi sappiano “leggere” senza bisogno di un intermediario su misura. I due formati più diffusi sono **JSON** e **XML**.

JSON

JSON (JavaScript Object Notation) è oggi il formato più usato per scambiare dati tra applicazioni web, perché è semplice da leggere sia per un umano che per un computer. Organizza i dati in coppie **chiave: valore**, proprio come una piccola scheda informativa.

Esempio: i dati di un ordine in formato JSON potrebbero essere così:

```
{
  "id_ordine": 42,
  "cliente": "Mario Rossi",
  "totale": 129.90,
  "spedito": false,
  "prodotti": [
    "Tastiera",
    "Mouse"
  ]
}
```

Si legge in modo abbastanza naturale: “l’ordine numero 42, del cliente Mario Rossi, con un totale di 129,90, non ancora spedito, contiene una tastiera e un mouse”. Le parentesi graffe `{ }` delimitano un “oggetto” (una scheda di dati), le parentesi quadre `[ ]` delimitano una lista di valori.

XML

XML (eXtensible Markup Language) è un formato più “vecchio” ma ancora molto usato in certi contesti aziendali (soprattutto in sistemi legacy o in alcuni settori come banking e pubblica amministrazione).

Usa dei **tag** che racchiudono i dati, un po’ come l’HTML delle pagine web:

```
<ordine>
  <id>42</id>
  <cliente>Mario Rossi</cliente>
  <totale>129.90</totale>
  <spedito>>false</spedito>
</ordine>
```

Quale scegliere?

Nella maggior parte dei progetti moderni con cui avrai a che fare, il formato predefinito sarà **JSON**: è più compatto, più leggibile, e più facile da usare nella programmazione moderna. XML lo incontrerai più raramente, magari in integrazioni con sistemi più datati.

Questo non significa che XML sia “superato”: resta preferibile quando serve una **validazione rigorosa** con uno schema formale (settori come banking o pubblica amministrazione) o quando servono i cosiddetti **namespace** — un modo di “etichettare” i tag con un prefisso, per evitare che due sistemi diversi usino per errore lo stesso nome di campo con significati diversi — capacità che JSON non offre allo stesso livello.

2.15 Database relazionali e SQL

Prima che ShopFacile avesse un database vero e proprio per gli ordini, il team operativo teneva l'elenco su un foglio Excel condiviso. Un giorno, due operatori aprono lo stesso ordine nello stesso minuto: uno aggiorna l'indirizzo, l'altro lo segna come “spedito”. Excel salva l'ultima modifica sopra l'altra senza avvisare nessuno, e non c'è cronologia da consultare per capire chi ha scritto cosa. È il problema della copia condivisa senza tracciamento: lo ritroverai identico, applicato al codice invece che a un foglio Excel, quando arriverai a Git nella sezione 4 — è lo stesso schema di fondo, con un file diverso.

Il JSON dell'ordine visto poco fa va quindi salvato in un modo che eviti il problema appena descritto, non basta scambiarlo in una risposta API. È qui che entrano in gioco i database.

Un **database** (base di dati) è un sistema organizzato per **salvare, organizzare e recuperare grandi quantità di dati** in modo efficiente e strutturato, gestendo correttamente gli accessi simultanei e mantenendo una cronologia delle modifiche — molto più potente e affidabile di un semplice file Excel per gestire dati che crescono in volume e complessità.

I **database relazionali** organizzano i dati in **tabelle**, esattamente come un foglio Excel: righe e colonne.

Analogia: il foglio Excel evoluto

Pensa a un database relazionale come a un insieme di fogli Excel collegati tra loro:

- ogni **tabella** è un foglio (es. “Clienti”, “Ordini”, “Prodotti”);
- ogni **riga** è un singolo elemento (es. un cliente specifico);
- ogni **colonna** è un attributo di quell'elemento (es. nome, cognome, email).

Le tabelle possono essere **collegate** tra loro: ad esempio, la tabella “Ordini” può avere una colonna che indica “a quale cliente appartiene questo ordine” — questa è la “relazione” da cui prende il nome “database relazionale”.

Esempio, tabella **Clienti** del database di ShopFacile:

id	nome	email
1	Mario Rossi	mario.rossi@email.com
2	Anna Bianchi	anna.bianchi@email.com

Chiave primaria: come distinguere due righe che si somigliano

Guarda di nuovo quella colonna `id`: perché esiste, se la tabella ha già `nome` ed `email`? Il problema che risolve è concreto: prova a immaginare due clienti che si chiamano entrambi “Mario Rossi” (capita più spesso di quanto si pensi). Se il database cercasse gli ordini di “Mario Rossi” per nome, potrebbe restituire — o modificare — gli ordini della persona sbagliata.

Per questo ogni tabella ha una **chiave primaria**: una colonna (o un piccolo gruppo di colonne) che identifica **in modo univoco e stabile** ogni riga, e che il database stesso si impegna a non far duplicare mai. Perché funzioni, una chiave primaria deve essere:

- **unica**: nessuna altra riga può averne lo stesso valore;
- **mai vuota**: ogni riga deve averne una;
- **stabile nel tempo**: non deve cambiare durante la vita della riga.

È tentante usare come chiave primaria qualcosa che sembra già unico, come il codice fiscale o l'indirizzo e-mail di un cliente — ma sono scelte rischiose: un'e-mail può cambiare (il cliente cambia provider), un codice fiscale può essere inserito con un errore di digitazione e va corretto, e in entrambi i casi “correggere la chiave primaria” è un'operazione delicata che rischia di rompere ogni collegamento con le altre tabelle. Per questo, nella grande maggioranza dei casi, si preferisce un numero generato apposta dal database (come l'`id` della tabella `Clienti`), che non ha alcun significato nel mondo reale e quindi non ha motivo di dover cambiare mai.

Esempio, tabella `Ordini`:

id	id_cliente	totale
101	1	129.90
102	2	59.00

Chiave esterna: il collegamento che il database fa rispettare

Nota la colonna `id_cliente` nella tabella `Ordini`: contiene lo stesso valore della chiave primaria della tabella `Clienti`. Questo collegamento si chiama **chiave esterna** (foreign key), ed è ciò che rende possibile la “relazione” da cui il database relazionale prende il nome.

Non è solo una convenzione di comodo: è una regola che il database **fa rispettare attivamente**. Se qualcuno provasse a creare un ordine con `id_cliente = 999` e nella tabella `Clienti` non esistesse nessun cliente con quell'id, il database **rifiuta l'operazione**. Questo si chiama vincolo di **integrità referenziale**, ed è un tassello dell'integrità dei dati di cui hai già sentito parlare in questa sezione: garantisce che non possano esistere in giro ordini “orfani”, intestati a un cliente inesistente.

Perché interessa a un PM: se un giorno senti che una funzionalità è stata bloccata perché “violerebbe un vincolo di integrità referenziale”, non è un capriccio tecnico — è il database che sta impedendo attivamente che i dati diventino incoerenti.

Relazioni: uno-a-molti e molti-a-molti

La relazione tra **Clienti** e **Ordini** è di tipo **uno-a-molti**: un cliente può avere molti ordini, ma ogni ordine appartiene a un solo cliente. È il caso più comune, e si riconosce così: la chiave esterna vive nella tabella “dalla parte dei molti” (**Ordini**).

Esiste anche il caso in cui **entrambi i lati possono avere molti elementi collegati** — le **relazioni molti-a-molti**. Pensa al dominio assicurativo che frequenterai: un cliente può avere più polizze, ma soprattutto ogni **polizza** può includere più **garanzie** (furto, incendio, responsabilità civile...), e la stessa garanzia (es. “furto”) può comparire in tante polizze diverse. Non puoi risolverlo con una singola chiave esterna da un lato o dall’altro, perché nessuna delle due tabelle “contiene” davvero l’altra. La soluzione è una terza tabella, chiamata **tabella ponte** (o tabella di associazione), che contiene solo coppie di chiavi esterne:

id_polizza	id_garanzia
1	Furto
1	Incendio
2	Furto
2	Responsabilità civile

Questa tabella dice, riga per riga, “questa polizza include questa garanzia”: la polizza 1 ha furto e incendio, la polizza 2 ha furto e responsabilità civile, e “furto” compare in entrambe senza essere duplicato come informazione.

SQL: il linguaggio per “interrogare” il database

SQL (Structured Query Language) è il linguaggio standard usato per “interrogare” (query) un database relazionale: chiedere dati, inserirne di nuovi, modificarli o eliminarli.

Esempio semplicissimo: per chiedere “dammi il nome e l’email di tutti i clienti”, in SQL scriveresti:

```
SELECT nome, email FROM Clienti;
```

Per chiedere “dammi solo gli ordini con un totale superiore a 100 euro”:

```
SELECT * FROM Ordini WHERE totale > 100;
```

Non ti verrà chiesto di scrivere SQL nel tuo ruolo di PM, ma capire questa logica di base ti aiuterà a seguire discussioni tecniche sul team riguardo “performance del database”, “query lente”, “migrazione dati”, ecc.

JOIN: perché i dati sono divisi in tabelle diverse

Torniamo alle due tabelle `Clients` e `Orders`. Un giorno Sara chiede: “mi serve l'elenco degli ordini con il nome del cliente accanto, per mandarlo al servizio spedizioni”. Le due informazioni, però, vivono in due tabelle diverse. La risposta è la **JOIN**: un'istruzione SQL che “unisce” temporaneamente le righe di due tabelle collegate tra loro tramite la chiave esterna vista sopra.

```
SELECT Orders.id, Clients.name, Orders.total
FROM Orders
JOIN Clients ON Orders.id_client = Clients.id;
```

Il risultato è una nuova tabella “virtuale” che mette insieme i dati delle due:

id	nome	totale
101	Mario Rossi	129.90
102	Anna Bianchi	59.00

Ma perché non scrivere semplicemente il nome del cliente **dentro** ogni riga della tabella `Orders`, evitando del tutto la JOIN? Perché il nome di Mario Rossi, se ha fatto 300 ordini, finirebbe scritto 300 volte, in 300 posti diversi. Se un giorno Mario Rossi corregge il proprio nome, bisognerebbe aggiornarlo in 300 righe — e se anche una sola venisse dimenticata, il database conterrebbe due versioni diverse della stessa verità. Il principio che evita questo problema si chiama **normalizzazione**: ogni informazione viene scritta **una volta sola**, nel posto giusto, e recuperata altrove tramite relazioni e JOIN quando serve.

Esistono due tipi principali di JOIN, e la differenza tra loro è una delle fonti più comuni di report “sbagliati” che un PM incontra:

- una **INNER JOIN** (quella vista sopra) restituisce solo le righe che hanno una corrispondenza in entrambe le tabelle: se un cliente non ha mai fatto un ordine, non compare affatto nel risultato;
- una **LEFT JOIN** restituisce **tutte** le righe della prima tabella, anche quelle senza corrispondenza nella seconda (con dei “vuoti” al posto dei dati mancanti).

Caso concreto: se Sara vuole “l'elenco di tutti i clienti, con il totale ordinato se ne hanno fatto uno”, e chi scrive la query usa una INNER JOIN invece di una LEFT JOIN, i clienti che non hanno mai ordinato **spariscono silenziosamente** dal report — non per un bug evidente, ma per la scelta di JOIN sbagliata rispetto alla domanda di business. È esattamente il tipo di errore che produce un numero “pulito” ma scorretto in una presentazione.

Vista (VIEW): dare un nome a una query complessa

Se la query con la JOIN qui sopra viene usata ogni settimana per il report delle spedizioni, ha senso salvarla con un nome, così chi si occupa di reporting non deve riscriverla — e non deve nemmeno sapere come sono strutturate le tabelle sottostanti. Questo si chiama **vista** (VIEW): una query salvata, che si comporta come se fosse essa stessa una tabella.


```
CREATE VIEW OrdiniConCliente AS
SELECT Ordini.id, Clienti.nome, Ordini.totale
FROM Ordini
JOIN Clienti ON Ordini.id_cliente = Clienti.id;
```

Da quel momento, chiunque può scrivere semplicemente `SELECT * FROM OrdiniConCliente` senza conoscere la JOIN che c'è dietro.

 **Trade-off:** la vista nasconde la complessità, ma non la elimina — la query dietro le quinte viene comunque eseguita ogni volta. Se qualcuno crea una vista che si appoggia su un'altra vista, che a sua volta si appoggia su un'altra ancora, il database finisce a eseguire in cascata query via via più pesanti a ogni chiamata: è un problema di performance che nessuno vede arrivare, perché ogni singola vista, presa da sola, sembra innocua.

Modellare i dati prima di scrivere codice

Tutto quello che hai visto in questa sottosezione — quali tabelle creare, quali colonne, quali relazioni, dove metterle — è il risultato di una fase che si chiama **modellazione dei dati**: decidere, prima ancora di scrivere una riga di codice, come rappresentare la realtà del progetto (clienti, ordini, polizze, garanzie) in tabelle e relazioni coerenti.

Non è un dettaglio tecnico da poco: un errore nel modello (una relazione uno-a-molti dove in realtà serviva molti-a-molti, una chiave primaria scelta male) scoperto quando il sistema è già in produzione, con migliaia di righe già scritte, costa ordini di grandezza in più da correggere rispetto a un errore scoperto sul tavolo da disegno, prima che una sola riga di dati sia mai stata salvata. È uno dei motivi per cui gli sviluppatori insistono a volte per “fermarsi a discutere il modello” prima di partire a costruire una nuova funzionalità, anche quando sembra di poter “iniziare subito a programmare”.

Esempi di database relazionali diffusi: **Microsoft SQL Server, PostgreSQL, MySQL, Oracle Database**.

2.16 Database NoSQL

Le tabelle rigide di SQL funzionano benissimo per dati regolari come clienti e ordini. Ma cosa succede quando i dati da salvare hanno una struttura molto variabile, come il catalogo prodotti di ShopFacile? Qui SQL comincia a scricchiolare, ed entrano in gioco i database NoSQL.

I database **NoSQL** (“Not Only SQL”) sono un'alternativa ai database relazionali, pensati per casi in cui la rigida struttura a tabelle non è la scelta migliore.

Perché esistono

Immagina di dover salvare dati molto **variabili** nella struttura (ad esempio: i prodotti nel catalogo di ShopFacile, dove ogni categoria ha attributi diversi — un libro ha “autore” e “numero pagine”, uno smartphone ha “memoria” e “colore”). Costringere questi dati in tabelle rigide e uniformi sarebbe scomodo. I database NoSQL permettono maggiore **flessibilità**.

Analogia: cartelle e documenti vs archivio rigido

Se il database relazionale è come un archivio con moduli standard identici per tutti (ogni modulo ha esattamente gli stessi campi da riempire), un database NoSQL è più come una serie di **cartelline con documenti liberi**: ogni documento può avere una struttura leggermente diversa, senza dover rispettare uno schema fisso e identico per tutti.

Molti database NoSQL salvano i dati proprio in formato **JSON** (quello che abbiamo visto sopra):

```
{
  "prodotto": "Smartphone X",
  "colore": "nero",
  "memoria_gb": 128
}
```

```
{
  "prodotto": "Libro Y",
  "autore": "M. Verdi",
  "pagine": 320
}
```

Due prodotti, campi diversi, nello stesso database: con un database relazionale classico sarebbe più complicato da gestire in modo pulito.

Quando si usano

- **Relazionali (SQL)**: dati molto strutturati, con relazioni chiare tra entità (es. sistemi gestionali, contabilità, ordini). Garantiscono forte coerenza dei dati.
- **NoSQL**: dati con struttura variabile o che cambia spesso, grandi volumi con necessità di scalare velocemente, casi come cataloghi prodotto, log, dati di sessione, messaggistica.

Esempi di database NoSQL diffusi: **MongoDB**, **Cosmos DB** (Azure), **Redis**, **Cassandra**.

Non esiste “il migliore in assoluto”: la scelta dipende dal tipo di dati e dal problema da risolvere. Sentirai il team discutere di questa scelta nelle fasi di progettazione di un nuovo servizio.

2.17 Virtual Machine: un computer dentro un computer

Fin qui abbiamo parlato di dati (SQL, NoSQL): ora torniamo all'infrastruttura che fa girare ShopFacile. Un modo classico per ricavare più “computer” indipendenti da un solo server fisico è la macchina virtuale.

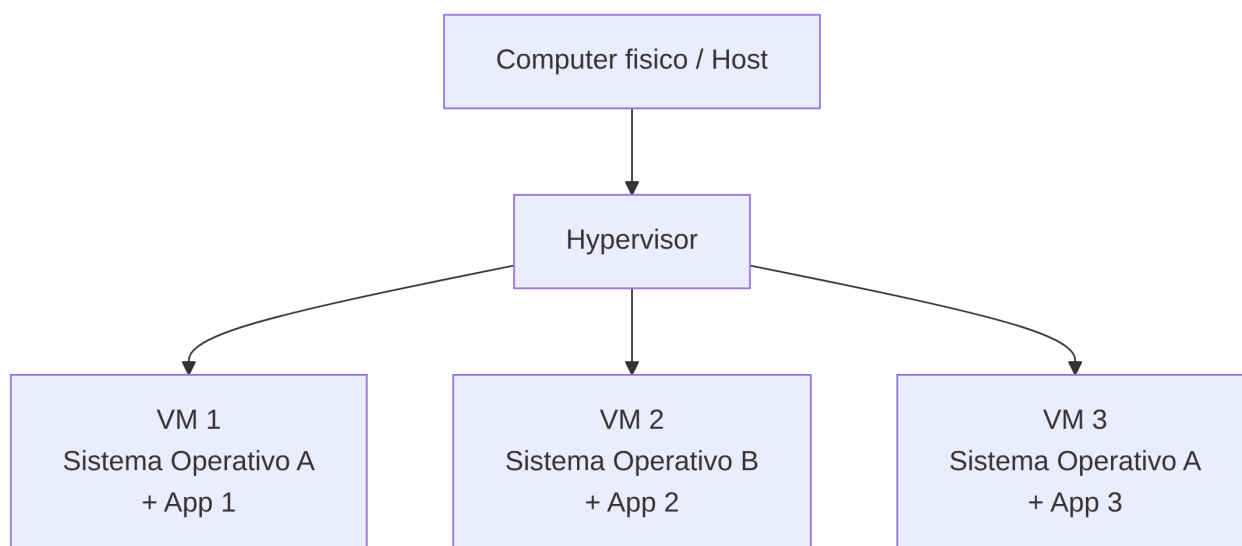
Una **Virtual Machine** (VM, macchina virtuale) è un software che **simula un computer completo** dentro un altro computer fisico. Dal punto di vista di chi la usa, si comporta esattamente come un computer vero e proprio: ha il suo sistema operativo, le sue applicazioni, il suo spazio disco — ma “vive” dentro un computer fisico più grande (l'**host**), condividendo con altre eventuali VM le risorse hardware reali.

Il software che crea e gestisce le macchine virtuali si chiama **hypervisor**.

Analogia: un condominio con appartamenti indipendenti

Pensa a un edificio (il computer fisico, “host”) diviso in **appartamenti** indipendenti (le VM). Ogni appartamento ha la sua porta blindata, i suoi mobili, la sua cucina — è completamente **isolato** dagli altri, anche se condividono le stesse fondamenta, gli stessi impianti elettrici e idraulici dell'edificio. Se un appartamento va a fuoco, in teoria gli altri restano al sicuro (isolamento).

Perché sono utili? Perché su un solo computer fisico potente puoi far “vivere” tante macchine virtuali diverse, ciascuna con il proprio sistema operativo e le proprie applicazioni, risparmiando sui costi hardware e semplificando la gestione (puoi creare, cancellare, spostare una VM molto più facilmente di un computer fisico).



Esempio pratico

Marco deve testare un aggiornamento importante prima di applicarlo ai server di produzione di ShopFacile. Crea una nuova VM di test partendo da un'immagine standard identica a quella di produzione, prova l'aggiornamento lì sopra senza alcun rischio per i clienti reali, e solo dopo aver verificato che tutto funziona procede con l'aggiornamento vero e proprio.

2.18 Container: più leggeri di una VM

Le VM funzionano, ma portare con sé un intero sistema operativo per ogni appartamento del “condominio” ha un costo in termini di peso e velocità. Esiste un'alternativa più leggera, molto usata oggi anche per ShopFacile: i container.

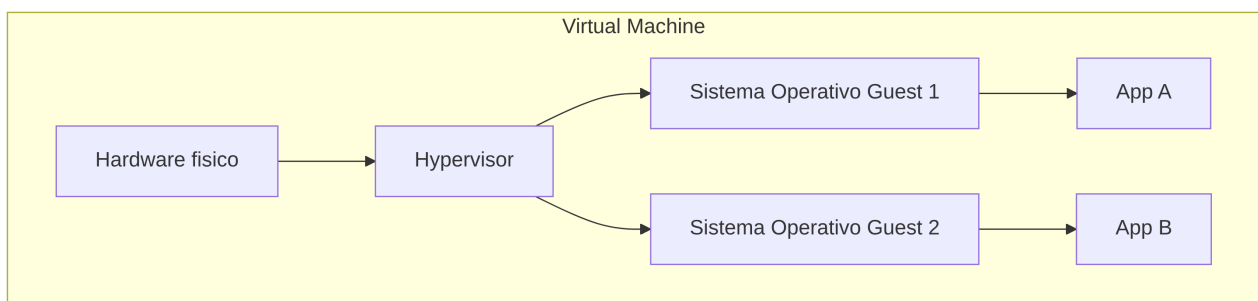
Un **container** è un altro modo per “isolare” un'applicazione e le sue dipendenze, ma in modo molto più **leggero** rispetto a una VM: invece di simulare un computer intero con il suo sistema operativo, un container condivide il sistema operativo della macchina che lo ospita, isolando solo l'applicazione e ciò che le serve per funzionare (librerie, configurazioni, file necessari).

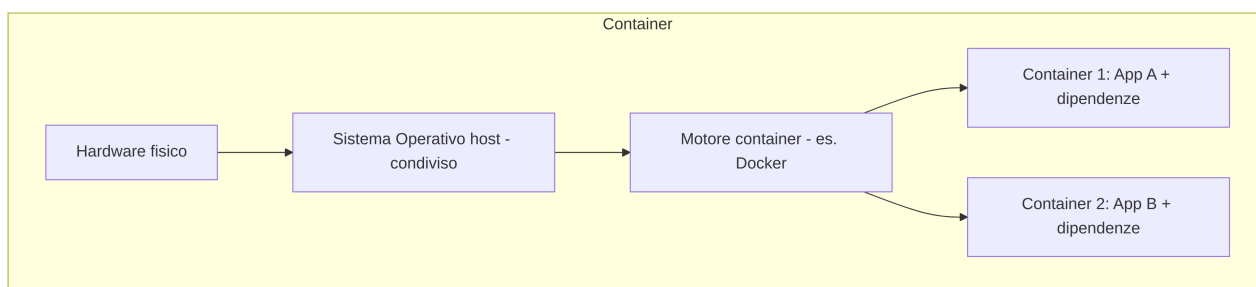
Analogia: la valigia pronta all'uso

Se la VM è un intero appartamento indipendente, il **container** è più come una **valigia già pronta con tutto il necessario**: vestiti, spazzolino, caricabatterie — tutto quello che serve per il viaggio, organizzato e pronto. Puoi portare questa valigia in qualsiasi “casa” (computer/server) e sarai subito operativo, perché hai già con te tutto ciò che ti serve, senza dover allestire un'intera casa da zero (come faresti con una VM).

Questo risolve un problema molto concreto e comune nello sviluppo software: *“sul mio computer funzionava, perché sul server non funziona?”*. Il container “porta con sé” esattamente le stesse versioni di librerie, configurazioni e dipendenze usate in fase di sviluppo, garantendo che l'applicazione si comporti allo stesso modo ovunque venga eseguita.

VM vs Container: il confronto visivo





La differenza chiave: ogni VM porta con sé un **intero sistema operativo** (pesante, lento da avviare), mentre i container **condividono** il sistema operativo della macchina host e sono quindi molto più leggeri, veloci da avviare (secondi, invece di minuti) e più efficienti in termini di risorse.

	Virtual Machine	Container
Cosa isola	Un intero computer virtuale, con proprio OS	Solo l'applicazione e le sue dipendenze
Peso	Pesante (GB), OS completo	Leggero (MB), condivide l'OS host
Tempo di avvio	Minuti	Secondi
Isolamento	Molto forte (OS separato)	Buono, ma leggermente meno forte della VM

Esempio pratico

Un cliente segnala un bug su ShopFacile in produzione. Ahmed, invece di provare a indovinare cosa non funzioni, lancia in locale sul proprio computer lo stesso identico container usato in produzione (stesse librerie, stessa versione del linguaggio, stessa configurazione) e riesce a riprodurre il problema in pochi minuti, con la certezza di lavorare in un ambiente identico a quello reale.

2.19 Docker: lo strumento più diffuso per i container

Abbiamo appena descritto cosa sono i container, ma “chi” li crea e li fa girare concretamente? Nella grande maggioranza dei casi, incluso il progetto ShopFacile, lo strumento usato è Docker.

Come abbiamo appena visto, il problema di fondo è quello del “funziona sul mio computer”: il laptop di Marco, il server di test e quello di produzione hanno spesso versioni leggermente diverse di librerie e configurazioni, e Docker è nato apposta per spedire il software **insieme al suo intero ambiente**, così che i tre coincidano sempre esattamente.

Docker è oggi lo strumento più popolare e diffuso per creare, distribuire ed eseguire container. È diventato talmente centrale nel mondo dello sviluppo software moderno che spesso “container” e “Docker” vengono usati quasi come sinonimi (anche se Docker è solo uno degli strumenti possibili).

Concetti chiave da conoscere (senza bisogno di saperli usare tecnicamente):

- **Immagine (image)**: è il “modello” o la “ricetta” del container: contiene l'applicazione e tutto ciò che le serve per funzionare (un po' come la lista precisa del contenuto della valigia, prima ancora di farla). È come una fotografia di uno stato pronto all'uso.
- **Container**: è un'immagine “in esecuzione”, cioè la valigia effettivamente in uso, attiva, con l'applicazione che sta girando.
- **Dockerfile**: è un file di testo con le istruzioni su come costruire un'immagine (es. “parti da questo sistema base, installa queste librerie, copia questi file, avvia questo comando”).
- **Registry** (es. Docker Hub): è un “magazzino” online dove si salvano e si condividono le immagini Docker, pronte per essere scaricate e usate.

Analogia riassuntiva

- il **Dockerfile** è la ricetta scritta;
- l'**immagine** è la valigia già fatta secondo quella ricetta, pronta ma non ancora “in viaggio”;
- il **container** è la valigia che stai effettivamente usando durante il viaggio (in esecuzione).

Nel team con cui lavorerai, sentirai spesso frasi come “buildiamo l'immagine”, “il container è andato in crash”, “pushiamo l'immagine sul registry”: ora sai cosa significano a grandi linee.



Esempio pratico

Marco scrive un Dockerfile per ShopFacile che parte da un'immagine base con il linguaggio di programmazione già installato, copia il codice dell'applicazione e installa le librerie necessarie. La pipeline di CI/CD (ne parleremo più avanti nel corso) usa questo Dockerfile per costruire automaticamente una nuova immagine ogni volta che il codice cambia, e la pubblica su un registry privato del progetto, pronta per essere distribuita.

2.20 Kubernetes: l'orchestratore di container

Docker ti permette di creare e avviare un singolo container, ma ShopFacile in produzione non ne ha uno solo: ne ha decine, distribuiti su più macchine. Gestirli a mano diventa presto impossibile, e qui entra in gioco Kubernetes.

Quando un'applicazione è composta da tanti container (magari decine o centinaia, distribuiti su tante macchine diverse per gestire tanti utenti contemporaneamente), diventa molto complicato gestirli a mano: avviarli, fermarli, sostituirli se si “rompono”, distribuirli sulle macchine giuste, farli comunicare tra loro...

Kubernetes (spesso abbreviato **K8s**) è lo strumento più diffuso per **orchestrare** (coordinare automaticamente) grandi quantità di container.

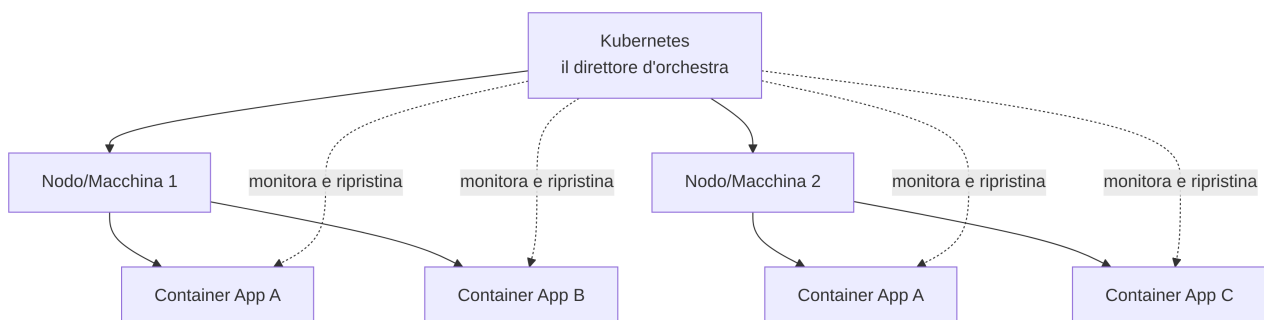
Analogia: il direttore d'orchestra

Pensa a un'orchestra con decine di musicisti (i container). Ogni musicista suona il suo strumento (esegue la sua applicazione), ma senza un **direttore** che coordini tempi, entrate e uscite di ciascuno, il risultato sarebbe caos. Il **direttore d'orchestra** (Kubernetes) decide:

- quando far “entrare” un nuovo musicista se ne serve uno in più (es. più utenti stanno usando l'app: servono più container per gestire il traffico);
- cosa fare se un musicista si sente male e deve uscire (un container si blocca: Kubernetes lo rileva e ne avvia automaticamente uno nuovo al suo posto, senza intervento umano);
- come distribuire i musicisti sul palco (su quali macchine far girare quali container, in base alle risorse disponibili).

Kubernetes gestisce automaticamente compiti come:

- **Scalabilità**: aumentare o diminuire il numero di container attivi in base al traffico/carico di lavoro;
- **Self-healing** (auto-guarigione): se un container si blocca o si rompe, Kubernetes lo rileva e lo riavvia o lo sostituisce automaticamente;
- **Distribuzione del carico**: smistare le richieste in arrivo tra i vari container disponibili, in modo che nessuno sia sovraccaricato mentre altri sono inattivi;
- **Deployment controllato**: aggiornare l'applicazione a una nuova versione gradualmente, riducendo il rischio di interruzioni per gli utenti.



Non dovrai mai configurare Kubernetes con le tue mani come Project Manager, ma sentirai questo nome citato molto spesso parlando dell'infrastruttura del progetto e dei tempi di rilascio: sapere cosa fa a grandi linee ti aiuterà a capire meglio le conversazioni tecniche del team e a stimare meglio la complessità di certe attività.

Kubernetes, però, non serve sempre. Un'applicazione singola con traffico modesto, gestita da un container o due su un solo server, spesso non ha bisogno di un direttore d'orchestra: adottarlo solo “perché è quello che usano tutti” ha un costo reale in complessità e competenze, che può superare il beneficio ottenuto.

Esempio pratico

Durante il Black Friday, con un picco di clienti su ShopFacile per una promozione con molto traffico, Kubernetes rileva l'aumento del carico e scala automaticamente da 3 a 10 container della stessa applicazione, per distribuire meglio le richieste. Passato il picco, riduce di nuovo il numero di container a 3, risparmiando risorse — tutto senza che nessuno del team debba intervenire manualmente nel cuore della notte.

OpenShift: Kubernetes “con il cruscotto”

C'è però un problema che Kubernetes da solo non risolve: installarlo e farlo funzionare bene non basta a renderlo utilizzabile in un'azienda. Serve anche un posto dove salvare le immagini Docker (il registry visto in §2.19), un sistema per autenticare chi può fare cosa, una gestione della rete e degli indirizzi, delle pipeline che colleghino il codice al deployment, uno strumento di monitoraggio che avvisi quando qualcosa non va. Kubernetes “nudo” è un motore potente, ma senza cruscotto: ogni azienda che lo adotta da zero finisce per scegliere, installare e mantenere decine di componenti aggiuntivi, spesso diversi da quelli scelti dall'azienda vicina — con il risultato che due aziende che dicono entrambe “usiamo Kubernetes” possono avere due piattaforme profondamente diverse da gestire e da cui assumere competenze.

Da questo problema nasce **OpenShift** (spesso abbreviato **OCP**, OpenShift Container Platform), il prodotto di Red Hat che è oggi una delle distribuzioni Kubernetes più diffuse nelle grandi aziende, incluso il settore assicurativo. OpenShift è Kubernetes — non lo sostituisce, lo racchiude — ma arriva già con registry, autenticazione, rete, pipeline e monitoraggio integrati, con supporto commerciale (un fornitore da chiamare quando qualcosa si rompe) e con vincoli di sicurezza più severi per impostazione predefinita.

Questo ultimo punto è quello che arriva più spesso sotto forma di ticket: per impostazione predefinita, OpenShift **non permette** ai container di girare con privilegi da amministratore (root), a differenza di molte installazioni “nude” di Kubernetes più permissive. Un'immagine Docker costruita e testata altrove, che assumeva implicitamente di poter girare come root, può quindi rifiutarsi di partire su OpenShift finché non viene corretta — non è un capriccio della piattaforma, è una scelta di sicurezza esplicita, ma è anche esattamente il tipo di attrito che arriva come segnalazione confusa (“il deployment non parte, dicono che è un problema di permessi”) sul tavolo di un PM.

	Kubernetes “nudo”	OpenShift (OCP)
Chi lo mantiene	Comunità open source	Red Hat (prodotto commerciale)
Componenti aggiuntivi (registry, rete, pipeline, monitoring...)	Da scegliere e integrare a parte	Già inclusi e integrati
Costo	Nessuna licenza (ma costo di integrazione interno)	Licenza a pagamento
Supporto	Community, nessun contratto	Supporto commerciale con SLA
Sicurezza predefinita	Configurabile, spesso più permissiva	Più restrittiva di default (es. niente root nei container)
Curva di adozione	Più lenta all'inizio (tutto da assemblare)	Più rapida all'inizio, con vincoli da imparare a rispettare

 **Trade-off:** scegliere OpenShift significa pagare una licenza e accettare un certo grado di vincolo verso un unico fornitore (lock-in), in cambio di tempo di setup risparmiato, sicurezza predefinita più solida e — concretamente — un numero di telefono da chiamare alle tre di notte quando qualcosa smette di funzionare in produzione. Nessuna delle due strade è “quella giusta” in assoluto: dipende da quante competenze interne l'azienda vuole (e può permettersi di) mantenere. In ogni caso, le competenze Kubernetes non si buttano: sotto OpenShift resta Kubernetes, e chi lo sa usare non deve ripartire da zero.

Questo è il primo passo di un'idea più grande, che vedremo nella sezione 12: costruire una piattaforma interna che renda semplice la strada giusta.

2.21 Markdown e HTML: i due linguaggi per scrivere documenti e pagine

Fin qui abbiamo parlato di come i dati viaggiano (API, JSON) e di come vengono salvati (SQL). C'è però un altro tipo di contenuto con cui avrai a che fare quotidianamente come PM, anche solo leggendolo: i **documenti** — questo stesso corso, un README di progetto, una pagina web. Anche questi hanno un “linguaggio” con cui vengono scritti, e vale la pena riconoscerlo.

Markdown: perché la documentazione di progetto raramente sta in un file Word

Immagina la documentazione di un progetto scritta in Word: vive come allegato in una lunga catena di e-mail, ognuno ha in locale la versione che gli è arrivata per ultima, nessuno sa con certezza qual è quella corretta e aggiornata, e per capire cosa è cambiato tra due revisioni bisogna aprire entrambi i file e confrontarli a occhio, riga per riga. Soprattutto, quel documento non sta **accanto al codice** di cui parla: vive altrove, in un altro sistema, con un altro ciclo di vita.

Da questo problema nasce **Markdown**: un modo di scrivere testo formattato usando pochi simboli semplici, direttamente in un file di testo puro. Un file Markdown si può salvare in Git esattamente come un file di codice (lo vedrai nella sezione 4): ha una cronologia di modifiche, si possono vedere le differenze esatte tra due versioni, e vive nella stessa cartella del codice a cui si riferisce.

💡 Il corso che stai leggendo in questo momento è scritto in Markdown. Ogni titolo, ogni tabella, ogni blocco di testo che hai letto finora è testo semplice con una sintassi minima, non un documento Word o PDF fatto a mano.

Sintassi	Risultato	Esempio
# Titolo	Titolo di primo livello	# Introduzione
## Sottotitolo	Titolo di livello inferiore	## 2.21 Markdown
testo	grassetto	**importante**
testo	<i>corsivo</i>	<i>*nota*</i>
- voce	Elenco puntato	- primo punto
[testo](url)	Link	[vai al sito](https://...)
`codice`	Testo in stile codice	<code>SELECT * FROM`</code>
\ a \ b \	Tabella	come quella che stai leggendo

Come PM incontrerai Markdown molto più spesso di quanto pensi, senza mai dover scrivere codice: i README dei progetti su GitHub, le pagine wiki interne, la descrizione di una issue in Jira o GitHub (che spesso accetta proprio questa sintassi), il testo di una pull request, un ADR (Architecture Decision Record). Saperlo *leggere* — capire che ****questo**** era pensato per apparire in grassetto, che una riga che inizia con **#** è un titolo — ti basta per non sentirti perso di fronte a un file `.md` grezzo, ad esempio aperto per errore fuori dal suo visualizzatore.

HTML: il linguaggio che dice al browser cosa mostrare

Quando il tuo browser mostra la pagina di ShopFacile, sta interpretando un altro linguaggio, pensato non per essere letto come testo semplice ma per essere **visualizzato**: l'**HTML** (HyperText Markup Language). L'HTML descrive la **struttura** di una pagina attraverso dei **tag** annidati uno dentro l'altro — non troppo diverso dai tag XML visti in §2.14, e non è un caso: HTML e XML condividono la stessa idea di fondo di racchiudere contenuto tra marcatori con un nome.

```
<html>
  <body>
    <h1>Benvenuto su ShopFacile</h1>
    <p>Il tuo carrello contiene <strong>2 articoli</strong>.</p>
  </body>
</html>
```

Si legge così: c'è un titolo di primo livello (`<h1>`) con il testo “Benvenuto su ShopFacile”, seguito da un paragrafo (`<p>`) che contiene a sua volta del testo in grassetto (`<strong>`) per evidenziare “2 articoli”. Ogni tag apre con `<nome>` e chiude con `</nome>`, e i tag possono contenerne altri, in una struttura ad albero — la stessa logica delle cartelle annidate vista in §2.7, applicata al contenuto di una pagina invece che ai file su disco.

L'HTML, da solo, descrive **solo la struttura**: cosa c'è e in che ordine. Non dice nulla su colori, font o posizionamento (se ne occupa il **CSS**, Cascading Style Sheets) né su cosa succede quando l'utente clicca un bottone (se ne occupa il **JavaScript**). Questa separazione — struttura (HTML), aspetto (CSS), comportamento (JavaScript) — è la ragione per cui, quando un tecnico ti dice “è solo un problema di CSS”, sta dicendo qualcosa di preciso: il contenuto e il comportamento della pagina sono corretti, cambia solo qualcosa nell'aspetto visivo (un colore sbagliato, un elemento allineato male). Non è però sempre un problema piccolo: un CSS che nasconde per errore un bottone di conferma pagamento è “solo CSS” nella causa tecnica, ma ha un impatto di business enorme — vale la pena non liquidare mai un ticket solo perché la causa dichiarata è “solo CSS”.

Il ponte tra i due: entrambi sono linguaggi di marcatura

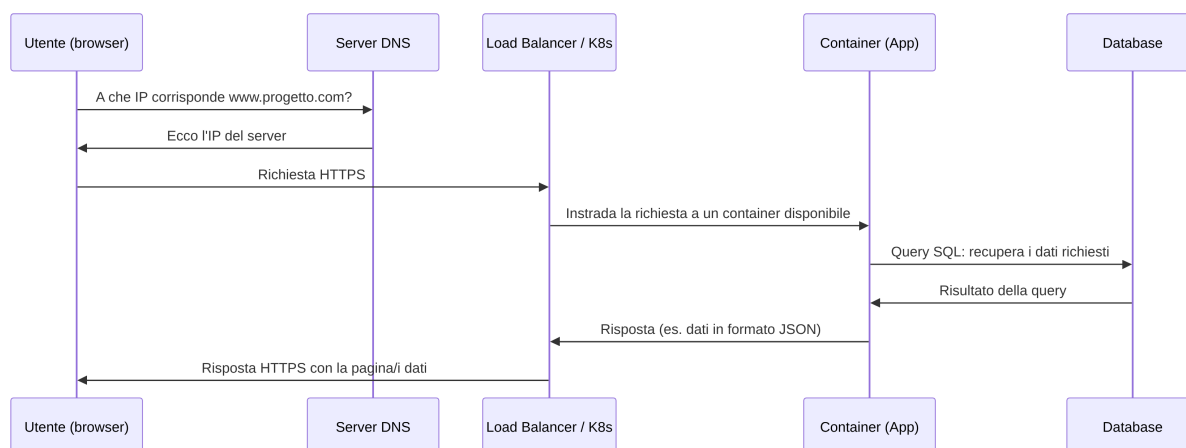
Markdown e HTML non sono in competizione: sono due linguaggi di **marcatura** (proprio come XML, §2.14) pensati per scopi diversi. Markdown è più semplice e leggibile anche “grezzo”, pensato per essere scritto rapidamente da una persona; HTML è più potente e dettagliato, pensato per essere interpretato da un browser. Non è un caso che siano collegati direttamente: quando un file Markdown viene mostrato in un browser (ad esempio proprio le pagine di questo corso), viene prima **tradotto** in HTML, perché è l'unico dei due linguaggi che il browser sa davvero interpretare e disegnare a schermo.

 **Trade-off:** Markdown è deliberatamente povero di proprietà. Non permette impaginazioni fini (margini precisi, numeri di pagina, intestazioni ripetute), non ha un vero sistema di commenti “in revisione” come quello di Word, e non è pensato per documenti destinati alla stampa formale (un contratto, una relazione ufficiale per un cliente esterno). Per quei casi, Word resta spesso la scelta giusta: la scelta tra i due non è “Markdown ha vinto”, è “quale documento sto scrivendo e chi lo deve leggere”. La documentazione tecnica che vive vicino al codice, sì, Markdown quasi sempre; un documento contrattuale per un cliente, quasi mai.

Non ti verrà mai chiesto di scrivere HTML o CSS in questo ruolo: l'obiettivo di questa sottosezione è che tu sappia **riconoscerli** quando li vedi citati in una conversazione tecnica, non produrli.

2.22 Riepilogo: come si incastrano tutti questi pezzi

Facciamo un ultimo passaggio per vedere come i concetti di questa sezione si incastrano in uno scenario reale — ad esempio, quando un utente usa un sito web che il tuo team ha costruito:



In questo singolo scenario compaiono quasi tutti i concetti visti in questa sezione: DNS, HTTPS, container orchestrati (Kubernetes), database e SQL, formato dati JSON scambiato via API. Non è un caso: questi sono davvero i “mattoncini” fondamentali di praticamente ogni sistema software moderno, e li ritroverai continuamente nel resto del corso.

✓ Checklist di autoverifica

Prima di passare alla sezione successiva, prova a rispondere (anche solo a voce, o scrivendo due righe) a queste domande. Se qualcuna ti mette in difficoltà, torna a rileggere il paragrafo corrispondente:

- Sapresti spiegare a un amico la differenza tra hardware e software?
- Sai spiegare la differenza tra RAM e disco con un’analogia tua?
- Sai dire a cosa serve un sistema operativo?
- Sai spiegare la differenza tra processo e thread?
- Sai spiegare, con parole tue, cosa fa il DNS?
- Sai spiegare perché HTTPS è più sicuro di HTTP?
- Sai spiegare cos’è un’API con l’analogia del ristorante?
- Sapresti leggere un piccolo blocco di dati in formato JSON?
- Sai spiegare la differenza tra database relazionale e NoSQL?
- Sai spiegare la differenza tra una VM e un container?
- Sai spiegare a cosa serve Kubernetes, a grandi linee?
- Sai spiegare cos’è il CRUD e cosa significa quando un tecnico dice “è solo un CRUD”?
- Sai spiegare, con parole tue, a cosa serve una specifica OpenAPI/Swagger tra due team che devono integrarsi?
- Sai spiegare la differenza tra chiave primaria e chiave esterna, e a cosa serve una JOIN?
- Sai spiegare la differenza tra OpenShift e Kubernetes “nudo”?
- Sai spiegare la differenza tra Markdown e HTML, e perché questo corso è scritto in Markdown?

Esercizi pratici

Gli esercizi che seguono ti aiutano a trasformare il vocabolario di questa sezione in comprensione reale. Non serve saper scrivere codice per farli: servono soprattutto occhi curiosi e la disponibilità a fare qualche domanda ai colleghi.

1. **Guarda “sotto il cofano” del tuo computer.** Apri il Task Manager (Windows) o il Monitoraggio Attività (macOS) e osserva per un paio di minuti quanta CPU e quanta RAM stanno usando le applicazioni aperte. Prova ad aprire molte schede del browser insieme e osserva come cambiano i numeri.  **Come verificare:** sai indicare quale processo, in quel momento, sta consumando più CPU e quale più RAM, e sai spiegare la differenza tra le due cose usando le analogie di questa sezione (scrivania vs persona che calcola)?
2. **Traccia il percorso di una richiesta web.** Apri gli strumenti di sviluppo del browser (F12 o tasto destro → “Ispeziona”), vai sulla scheda “Network”/“Rete”, visita un sito qualsiasi e osserva la prima richiesta HTTP che parte: nota il metodo (`GET`), il codice di risposta (es. `200`) e se il protocollo usato è HTTP o HTTPS.  **Come verificare:** sai indicare, guardando la schermata, se la connessione era protetta (HTTPS) e sai spiegare cosa significa il codice di risposta che hai visto?
3. **Interroga un DNS a mano.** Da riga di comando (terminale su Mac/Linux, Prompt dei comandi o PowerShell su Windows), esegui il comando `nslookup www.google.com` (o `ping www.google.com`) e osserva l'indirizzo IP che viene restituito.  **Come verificare:** sai spiegare a parole tue cosa ha fatto il comando, collegandolo all'analogia della “rubrica telefonica” vista in questa sezione?
4. **Leggi e “traduci” un JSON reale.** Chiedi a un developer del team di mostrarti un piccolo esempio di risposta JSON restituita da un'API del progetto (o, in alternativa, apri in un browser un endpoint pubblico come `https://api.github.com/users/github`). Prova a identificare le coppie chiave-valore principali.  **Come verificare:** sapresti riscrivere a voce, in una frase in italiano, cosa dice quel JSON, come abbiamo fatto con l'esempio dell'ordine in questa sezione?
5. **Confronta VM e container con parole tue.** Senza guardare il testo, scrivi in 3-4 righe la differenza tra una macchina virtuale e un container, e almeno un motivo per cui un team potrebbe preferire il secondo per distribuire un'applicazione.  **Come verificare:** fai leggere quello che hai scritto a un collega tecnico (o confrontalo con la tabella di questa sezione): la tua spiegazione coglie sia la differenza di “peso” che quella di isolamento?
6. **Disegna lo schema di una richiesta completa.** Su un foglio (anche a mano), disegna il percorso di una richiesta utente che visita una pagina della piattaforma: browser → DNS → server/container → database → e ritorno, etichettando ogni passaggio con il termine giusto (IP, HTTP/S, API, JSON, SQL...), ispirandoti al diagramma della sezione 2.22.  **Come verificare:** riesci a spiegare il tuo disegno a un collega non tecnico in meno di 3 minuti, senza dover consultare gli appunti?

7. **Riconosci un CRUD.** Prendi una funzionalità qualsiasi già rilasciata sul tuo progetto (o chiedine una a un collega) e chiedi al team: “questa era solo un CRUD, o c’era logica di business dentro?”. Ascolta la risposta e prova a capire, dal loro tono, se la stima era stata semplice o complicata da fare.  **Come verificare:** sapresti spiegare a un collega non tecnico, con parole tue, la differenza tra “è solo un CRUD” e “non è un CRUD” in termini di stima e rischio?
8. **Trova una specifica OpenAPI/Swagger reale.** Chiedi a uno sviluppatore del team se esiste una documentazione Swagger per una delle API del progetto (spesso è raggiungibile da un browser, con un indirizzo che contiene `/swagger` o `/api-docs`) e fatti mostrare la pagina.  **Come verificare:** sapresti indicare, guardando quella pagina, quali operazioni sono disponibili su una risorsa e distinguere GET, POST, PUT e DELETE tra loro?
9. **Leggi due tabelle collegate.** Chiedi a un collega tecnico di mostrarti (anche solo a voce, con carta e penna) due tabelle reali del progetto collegate da una chiave esterna, e chiedi cosa succede se provi a inserire una riga che punta a un id inesistente.  **Come verificare:** sapresti spiegare cosa significa “integrità referenziale” con parole tue, partendo da quell’esempio?
10. **Apri il codice sorgente di una pagina.** Su un sito qualsiasi, apri gli strumenti di sviluppo del browser (F12 o tasto destro → “Ispeziona elemento”) e guarda il codice HTML della pagina per un minuto, senza modificare nulla.  **Come verificare:** sapresti indicare almeno un tag di apertura e la sua chiusura corrispondente, e distinguere a colpo d’occhio cosa è struttura (HTML) da cosa potrebbe essere solo aspetto (CSS)?

Collegamenti

- Prossima sezione: [3. Come nasce un software](#)
- [4. Git e GitHub](#) — vedrai come i concetti di file e cartelle si applicano al codice
- [9. DevOps](#) — dove Docker e Kubernetes tornano centrali
- [11. Architetture software](#) — dove API e REST vengono approfonditi
- [12. Cloud](#) — dove VM, container e OpenShift vengono usati concretamente, e dove ritroverai il tema della piattaforma interna
- [16. Glossario](#) — per ripassare rapidamente ogni termine visto qui

Risorse

- [MDN Web Docs – Come funziona internet](#) — spiegazione approfondita di rete, DNS, HTTP
- [MDN Web Docs – Panoramica su HTTP](#) — approfondimento sul protocollo HTTP/HTTPS
- [Microsoft Learn – Cos’è un container?](#) — confronto ufficiale container vs VM
- [Documentazione ufficiale Docker – Introduzione](#) — guida introduttiva ufficiale a Docker
- [Red Hat – Cos’è OpenShift?](#) — panoramica ufficiale della piattaforma
- [Swagger – Cos’è OpenAPI?](#) — introduzione ufficiale allo standard OpenAPI
- [Markdown Guide \(in inglese\)](#) — riferimento rapido alla sintassi Markdown
- [Documentazione ufficiale Kubernetes – Concetti base](#) — introduzione ufficiale a Kubernetes (disponibile anche in italiano)

3. Come nasce un software

Nella sezione precedente hai imparato cosa sono hardware, software, reti e altri concetti di base dell'informatica. Ora facciamo un passo avanti: come nasce, concretamente, un software? Chi lo progetta? Chi lo scrive? Chi decide quando è pronto per essere usato dalle persone?

Capire questo processo è fondamentale per te, futuro Junior Project Manager, perché il tuo lavoro consisterà proprio nel coordinare le persone e le attività che compongono questo processo. Non dovrai scrivere codice, ma dovrai capire di cosa parlano gli sviluppatori, cosa significa “abbiamo trovato un bug in produzione” o “la pull request è ancora in review”, e come si inserisce ogni attività nel percorso complessivo di un progetto software.

Pensa a questa sezione come alla mappa di un territorio che esplorerai per tutta la tua carriera: oggi impari i nomi delle vie principali, poi con l'esperienza scoprirai i dettagli di ogni singolo quartiere.

Per non restare sul piano astratto, seguiremo come filo conduttore **ShopFacile**, la piattaforma e-commerce che hai già incontrato nella sezione precedente, con il suo piccolo team interno: Marco e Giulia (developer, lei molto attenta a test e qualità), Ahmed (developer junior, in crescita), Sara (Product Owner) e Luca (Scrum Master).

Obiettivi della sezione

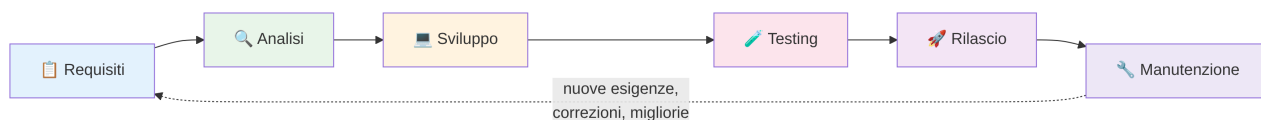
Alla fine di questa sezione saprai:

- Descrivere le fasi del ciclo di vita di un software e cosa succede in ciascuna
- Distinguere un requisito funzionale da un requisito non funzionale
- Spiegare la differenza tra bug e feature
- Capire perché il software viene “numerato” con le versioni
- Comprendere, a livello concettuale, cosa sono branch, pull request, code review e merge
- Capire perché una decisione tecnica importante va scritta in un ADR (Architecture Decision Record), e cosa significa che un ADR viene “superato”

Il ciclo di vita del software

Immagina di dover costruire una casa. Non inizi comprando mattoni a caso: prima parli con la famiglia che ci vivrà per capire quante stanze servono (requisiti), poi un architetto disegna la piantina (analisi), poi gli operai costruiscono davvero la casa (sviluppo), poi un ingegnere controlla che tutto sia sicuro e a norma (testing), poi la famiglia ci si trasferisce (rilascio), e infine, anno dopo anno, la casa avrà bisogno di manutenzione: una caldaia da sostituire, un tetto da riparare (manutenzione).

Il software funziona esattamente allo stesso modo. Questo percorso si chiama **ciclo di vita del software** (in inglese *Software Development Life Cycle*, o **SDLC**), ed è composto da fasi che si ripetono in ogni progetto, grande o piccolo.



Nota la freccia tratteggiata che torna da “Manutenzione” a “Requisiti”: il ciclo di vita non è quasi mai un percorso lineare che finisce una volta per tutte. È più simile a una spirale: ogni volta che il software viene usato nel mondo reale, emergono nuove esigenze, si scoprono difetti, arrivano richieste di nuove funzionalità, e il ciclo riparte. Nei prossimi capitoli (in particolare nella sezione sull’Agile) vedrai che i team moderni ripetono questo ciclo molte volte, anche ogni due settimane, invece di farlo una sola volta in anni.

Vediamo ora ogni fase nel dettaglio.

1. Requisiti: cosa deve fare il software

È la fase in cui si stabilisce **cosa** deve fare il software, senza ancora preoccuparsi di **come** lo farà tecnicamente. È come quando parli con un architetto e gli dici “vogliamo tre camere, una cucina abitabile e un balcone”: stai descrivendo un bisogno, non stai disegnando la pianta.

In questa fase il project manager, il cliente e gli analisti si incontrano per capire cosa serve davvero. Il risultato è un documento (o una lista di elementi in uno strumento come Jira o GitHub Issues, che vedremo più avanti) che elenca i **requisiti**.

Esempio pratico: durante una riunione su ShopFacile, il cliente dice “vogliamo che i nostri utenti possano cercare i prodotti nel catalogo per nome o categoria”. Sara, la Product Owner, trascrive questa richiesta in una card del backlog, magari intitolata “Ricerca prodotti nel catalogo”, senza ancora sapere se si useranno filtri, una barra di ricerca testuale o entrambe le cose: quella scelta arriverà nella fase successiva.

Approfondiamo questo concetto nella prossima sezione perché è cruciale.

2. Analisi: dai requisiti alla soluzione

Una volta chiaro **cosa** serve, bisogna capire **come** realizzarlo. È il lavoro dell’architetto che trasforma “vogliamo tre camere” in una piantina con misure precise, posizione dei muri portanti, degli impianti elettrici e idraulici.

Nel software, questa fase si chiama **analisi** (o analisi tecnica/progettazione). Chi si occupa di analisi — spesso un architetto software o un tech lead — decide:

- Come sarà strutturato il software (quali “pezzi” lo compongono e come comunicano tra loro — ne parleremo nella sezione sulle architetture software)
- Quali tecnologie usare (linguaggi di programmazione, database, strumenti)
- Come i dati saranno organizzati e salvati
- Quali rischi tecnici ci sono e come affrontarli

Esempio pratico: il requisito è “l’utente deve poter recuperare la password se la dimentica”. Marco, che spesso si occupa di infrastruttura e di questo tipo di scelte tecniche, stabilisce come: si invierà un’email con un link temporaneo, quel link scadrà dopo 30 minuti, i dati della password saranno salvati in un certo modo per motivi di sicurezza. È il passaggio dall’idea al progetto tecnico.

Una volta che si sa “come” farlo, resta solo un passaggio: farlo davvero. Passiamo quindi alla fase di sviluppo.

3. Sviluppo: si scrive il codice

Questa è la fase che tutti immaginano quando pensano all’informatica: gli sviluppatori “scrivono codice”. Ma cosa significa davvero?

Scrivere codice significa dare istruzioni precise e dettagliate al computer, usando un linguaggio di programmazione (Python, Java, C#, JavaScript, e tanti altri). Il computer non capisce l’italiano o l’inglese naturale: capisce solo istruzioni scritte in un modo molto rigido e specifico, un po’ come una ricetta di cucina scritta per una persona che esegue esattamente e solo quello che è scritto, senza improvvisare nulla.

Un’analogia utile: se dici a un amico “fammi un caffè”, lui capisce dal contesto cosa fare. Se dovessi scrivere le istruzioni per un robot che non ha mai visto un caffè, dovresti specificare ogni singolo passaggio: “apri lo sportello della macchina, prendi la capsula dal contenitore in alto a destra, inseriscila nell’apposito slot, chiudi lo sportello, premi il tasto con la scritta ESPRESSO, attendi che il led diventi verde...”. Scrivere codice è simile: bisogna essere estremamente precisi, perché il computer esegue letteralmente quello che gli viene scritto, senza intuire le intenzioni.

Lo sviluppatore prende i documenti di analisi e li traduce in queste istruzioni dettagliate, organizzate in file di testo (i “file di codice”) che insieme compongono il software. Il codice viene poi salvato e condiviso con il resto del team usando strumenti come Git (che vedremo nella prossima sezione).

Esempio pratico: l’analisi ha stabilito che serve una funzione che calcoli lo sconto finale su un carrello di ShopFacile. Marco scrive, in un linguaggio come Python o Java, una serie di istruzioni tipo “prendi il totale del carrello, verifica se esiste un codice sconto valido, se sì calcola la percentuale da sottrarre, altrimenti restituisci il totale invariato”. Ogni singolo caso (carrello vuoto, codice sconto scaduto, sconto superiore al totale) deve essere previsto esplicitamente: il computer non “capisce” cosa intendevi, esegue solo quello che hai scritto.

Il codice è scritto, ma “sembra funzionare” non basta: prima di fidarsene bisogna verificarlo sistematicamente. È qui che entra in gioco il testing.

4. Testing: si verifica che funzioni

Immaginiamo che gli operai abbiano finito di costruire la casa. Prima di far entrare la famiglia, un ingegnere collaudatore verifica che gli impianti funzionino, che le porte si aprano bene, che non ci siano crepe nei muri. Non basta che la casa “sembri” pronta: bisogna testarla.

Nel software questa fase si chiama **testing** (o test, o collaudo). Consiste nel verificare, in modo sistematico, che il software faccia esattamente quello che deve fare, senza comportamenti indesiderati.

Esistono diversi tipi di test, con diversi livelli di dettaglio:

- **Test unitari:** verificano un singolo “pezzettino” di codice isolato, ad esempio una funzione che calcola uno sconto. È come testare una singola presa elettrica prima di collegare l'intero impianto.
- **Test di integrazione:** verificano che più pezzi di codice funzionino bene insieme, ad esempio che il sistema di pagamento comunichi correttamente con il catalogo prodotti. È come verificare che l'impianto elettrico e quello idraulico non entrino in conflitto.
- **Test manuali:** una persona (spesso chiamata tester o QA, *Quality Assurance*) usa il software come farebbe un utente reale, cliccando in giro, provando casi limite, cercando di “romperlo”. È come far vivere qualcuno nella casa per un weekend di prova prima del trasferimento definitivo.

Perché si testa? Perché è molto più economico e veloce trovare un errore prima che il software arrivi agli utenti, piuttosto che scoprirlo quando migliaia di persone lo stanno già usando (e magari perdendo dati o soldi a causa di quell'errore).

Ogni tipo di test, però, ha anche un costo in tempo: un test unitario si scrive ed esegue in pochi secondi, un test di integrazione richiede più preparazione, un test manuale approfondito può occupare ore di lavoro di una persona come Giulia. Per questo, nella pratica, un PM non applica meccanicamente tutta la scala a ogni modifica: negozia con il team **quanto** testare in base al rischio reale di quella modifica. Un piccolo cambio al colore di un bottone non richiede lo stesso investimento di test di una modifica al calcolo del prezzo nel carrello.

Esempio pratico: per la nuova funzionalità di ricerca prodotti di ShopFacile, Ahmed scrive un test unitario che verifica che la funzione di ricerca restituisca risultati corretti con un solo prodotto nel catalogo; Giulia, sempre molto attenta alla qualità, scrive un test di integrazione che verifica che la ricerca funzioni correttamente insieme al filtro dei prezzi, e poi prova lei stessa a digitare caratteri strani (accenti, emoji, campo vuoto) nella barra di ricerca per vedere se qualcosa si rompe. Solo dopo che tutti e tre i livelli di test danno esito positivo, la funzionalità viene considerata pronta per il passo successivo.

I test sono tutti verdi: la funzionalità può finalmente lasciare l'ambiente del team ed essere messa nelle mani degli utenti veri.

5. Rilascio (Deploy): si va “in produzione”

Quando il software ha superato i test, è pronto per essere usato davvero dagli utenti finali. Questo passaggio si chiama **rilascio** o, con il termine tecnico che sentirai usare moltissimo, **deploy**.

“Mettere il software in produzione” significa renderlo disponibile e funzionante nell'ambiente reale, quello usato dai clienti veri — a differenza degli ambienti di test o sviluppo, che sono come delle “copie di prova” usate solo internamente dal team (ne parleremo nella sezione sugli ambienti di sviluppo).

Un'analogia: è il giorno del trasferimento nella casa nuova. Tutto quello che è stato progettato, costruito e verificato diventa finalmente reale e utilizzabile dalla famiglia.

Il deploy può essere un'operazione delicata: si tratta di installare la nuova versione del software sui sistemi che gli utenti usano davvero, a volte senza nemmeno interrompere il servizio (pensa a un aggiornamento di un'app che non ti fa nemmeno notare che è cambiato qualcosa). Nella sezione dedicata al DevOps e alla CI/CD scoprirai come oggi questo processo venga spesso automatizzato per essere più rapido e sicuro.

Esempio pratico: il team decide di rilasciare la nuova funzionalità di ricerca prodotti di ShopFacile di notte, in una fascia oraria con pochissimi utenti collegati, per limitare l'impatto nel caso qualcosa vada storto. Il deploy viene fatto prima solo su una piccola parte dei server (un cosiddetto "rilascio graduale"): se dopo un'ora tutto funziona bene, la nuova versione viene estesa a tutti gli utenti; se emergono problemi, si può tornare rapidamente alla versione precedente (un'operazione chiamata *rollback*).

Il software è finalmente in produzione, ma la storia non finisce qui: da questo momento inizia una fase che dura molto più a lungo di tutte le altre messe insieme, la manutenzione.

6. Manutenzione: la cura continua

Una casa, una volta costruita, non resta perfetta per sempre: la caldaia si guasta, il tetto va rifatto, magari dopo qualche anno si decide di ristrutturare la cucina. Lo stesso vale per il software.

La **manutenzione** comprende tutte le attività che avvengono dopo il rilascio:

- Correggere errori che emergono durante l'uso reale (i famosi **bug**, che vedremo tra poco)
- Aggiornare il software per farlo funzionare con nuove versioni di sistemi operativi o browser
- Migliorare le prestazioni
- Aggiungere piccole modifiche richieste dagli utenti

Un progetto software di successo può restare "in manutenzione" per anni, anche decenni. È una fase spesso sottovalutata da chi è alle prime armi, ma in realtà occupa una parte enorme del tempo e del budget di un progetto informatico nel lungo periodo.

Esempio pratico: sei mesi dopo il rilascio, la funzionalità di ricerca prodotti di ShopFacile inizia a rispondere sempre più lentamente perché il catalogo è cresciuto da 500 a 50.000 prodotti. Nessun requisito iniziale parlava di questo scenario: è un'attività di manutenzione, in cui Marco introduce un meccanismo di **caching** (una sorta di "memoria veloce" che conserva i risultati delle ricerche più frequenti) per riportare i tempi di risposta a livelli accettabili.

Abbiamo visto le sei fasi in azione. Ma per parlarne con precisione nel lavoro quotidiano, serve anche il vocabolario giusto: cominciamo dai requisiti, distinguendo due categorie che confonderai spesso all'inizio.



Requisiti: funzionali e non funzionali

Torniamo sui requisiti, perché è un concetto che userai costantemente nel tuo ruolo di Project Manager.

Un **requisito** è una descrizione di qualcosa che il software deve fare o di come deve comportarsi. Si dividono in due grandi categorie.

Requisiti funzionali

Descrivono **cosa** il software deve fare, cioè le sue funzionalità concrete. Rispondono alla domanda: “quali azioni deve poter compiere l'utente (o il sistema)?”

Esempi:

- “L'app deve permettere all'utente di effettuare il login con email e password”
- “Il sistema deve permettere di aggiungere un prodotto al carrello”
- “L'utente deve poter scaricare la fattura in formato PDF”

Sono requisiti facili da immaginare, perché descrivono azioni concrete e visibili, un po' come l'elenco delle stanze di una casa: “vogliamo una cucina, due camere, un bagno”.

Requisiti non funzionali

Descrivono **come** il software deve comportarsi, cioè la qualità con cui svolge le sue funzioni. Non riguardano una singola azione specifica, ma una caratteristica generale del sistema. Rispondono alla domanda: “quanto bene, quanto velocemente, quanto in sicurezza deve funzionare?”

Esempi:

- “L'app deve rispondere in meno di 2 secondi a ogni richiesta”
- “Il sistema deve poter gestire fino a 10.000 utenti collegati contemporaneamente”
- “I dati personali degli utenti devono essere cifrati”
- “L'applicazione deve essere disponibile (funzionante) almeno il 99,9% del tempo in un anno”

Tornando all'analogia della casa: se il requisito funzionale è “vogliamo una cucina”, il requisito non funzionale è “la cucina deve essere isolata acusticamente e avere abbastanza corrente per usare più elettrodomestici insieme senza far scattare il salvavita”. Non è una stanza in più, è una qualità che tutte le stanze devono avere.

Aspetto	Requisito funzionale	Requisito non funzionale
Domanda a cui risponde	Cosa fa?	Quanto bene lo fa?
Esempio	“Login con email e password”	“Login in meno di 2 secondi”
Facile da vedere?	Sì, si nota subito se manca	No, spesso si nota solo quando manca (es. sistema lento)
Chi lo richiede spesso	Cliente, utenti	Team tecnico, esigenze di business (sicurezza, scalabilità)

Un errore comune di chi inizia in questo settore è concentrarsi solo sui requisiti funzionali, perché sono i più “visibili” e i clienti li richiedono esplicitamente. Ma un buon Project Manager sa che i requisiti non funzionali (velocità, sicurezza, affidabilità) sono altrettanto critici: un'app che fa tutto quello che deve, ma è lentissima o insicura, è comunque un fallimento.

Perché stimare software è così difficile

Quando Sara chiede “quanto tempo ci vuole per il recupero password?”, Marco non può rispondere guardando un catalogo prezzi: ogni funzionalità di ShopFacile è, in un certo senso, unica, e a differenza di una casa — dove chi costruisce sa quasi esattamente quante ore servono per averne già fatte centinaia simili — un software raramente è “la stessa cosa” di qualcosa già fatto prima: cambiano i requisiti, il codice esistente su cui si innesta, i casi limite che emergono solo scrivendo il codice. Buona parte della difficoltà reale **emerge mentre ci si lavora**, non prima: Marco può stimare “due giorni” e poi scoprire a metà lavoro che il sistema di invio email esistente non supporta link temporanei e va modificato anche quello. Cosa significa questo per te, futuro PM? Che una stima è una **previsione con un margine di errore**, non una promessa: comunicarla al cliente come una data certa crea aspettative che il team, prima o poi, non riuscirà a rispettare. Riprenderemo questo concetto quando parleremo di Story Point, nella sezione 6 su Scrum.

Anche il requisito meglio scritto, però, non garantisce che il codice finale si comporti come previsto: a volte qualcosa va storto, ed è lì che entra in scena il bug.



Bug: quando qualcosa va storto

Un **bug** è un difetto nel software: un comportamento non corretto, non previsto o non voluto. Il termine in inglese significa letteralmente “insetto”, e c'è una storia (in parte leggendaria, ma bellissima) dietro questo nome.

Curiosità storica: si racconta che nel 1947, mentre lavorava su uno dei primi grandi calcolatori (il Harvard Mark II), la programmatrice e informatica Grace Hopper e il suo team trovarono una falena vera e propria incastrata tra i contatti di un relè elettromeccanico, che causava un malfunzionamento. Il team incollò l'insetto su un foglio del registro di laboratorio con la scritta “First actual case of bug being found” (primo caso reale di un bug trovato). Da allora, “bug” è diventato il termine universale per indicare un errore nel software (anche se il termine era già usato in ambito ingegneristico prima di quell'episodio).

Esempio concreto di bug: su ShopFacile, Giulia scopre durante un test che se metti nel carrello più di 99 pezzi dello stesso prodotto, il prezzo totale diventa negativo per un errore nel calcolo. L'utente potrebbe finire per “guadagnare” acquistando, invece di pagare! Questo è esattamente il tipo di comportamento inatteso che un bug produce: qualcosa che nessuno ha previsto o voluto, che si scopre spesso solo quando il software viene usato in modi che gli sviluppatori non avevano immaginato.

Perché succedono cose come questa, se il team ha già scritto dei test? La ragione di fondo è che un software come ShopFacile ha un numero enorme di stati e combinazioni possibili: quanti pezzi nel carrello, quali codici sconto, quale browser, quale sequenza di click. Anche pochi elementi che si combinano tra loro generano rapidamente migliaia di casi diversi (una vera e propria “esplosione combinatoria”), e nessun team, per quanto bravo, ha il tempo di percorrerli tutti prima di ogni rilascio. Per questo “testare tutto” non è un obiettivo realistico: testare significa sempre **scegliere dove**

guardare, concentrando l'attenzione sulle combinazioni più probabili o più rischiose. È anche per questo che, come futuro PM, non potrai mai pretendere “zero bug”: potrai solo negoziare quanto rischio il team accetta di correre su ogni singola area del prodotto.

I bug possono essere piccoli e quasi invisibili (un testo scritto con il colore sbagliato) o gravissimi (un sistema

bancario che addebita importi errati). Parte del lavoro di un Project Manager è proprio aiutare a **classificare** i bug per gravità e priorità, per decidere quali vanno risolti immediatamente e quali possono aspettare: il bug del prezzo negativo di ShopFacile, per esempio, è abbastanza grave da bloccare tutto il resto.

✨ Feature: una nuova funzionalità

Un bug corregge qualcosa che non va, ma non tutto il lavoro del team nasce da un errore da correggere: molto lavoro nasce dal desiderio di aggiungere qualcosa di nuovo e utile. Questo si chiama feature.

Una **feature** è, al contrario del bug, qualcosa di voluto: una nuova funzionalità che si aggiunge al software per offrire più valore agli utenti.

Esempio: aggiungere la possibilità di pagare con Apple Pay in un'app che finora accettava solo carta di credito è una feature. È una scelta, una decisione di business, non la correzione di un errore.

La differenza tra bug fix (correzione di un bug) e feature è quindi:

	Bug fix	Feature
Perché si fa	Qualcosa non funziona come dovrebbe	Si vuole aggiungere qualcosa di nuovo
È previsto dai requisiti originali?	Sì, il comportamento corretto era già previsto ma non funzionava	No, è un'aggiunta rispetto a quanto già esisteva
Urgenza tipica	Spesso alta, soprattutto se blocca l'uso del software	Variabile, spesso pianificata per una versione futura
Esempio	“Il bottone ‘Acquista’ non risponde al click su alcuni telefoni”	“Aggiungere la possibilità di salvare i prodotti preferiti”

Questa distinzione è fondamentale perché, come Project Manager, dovrai spesso gestire un elenco (detto **backlog**, termine che approfondiremo nella sezione sull'Agile) contenente sia bug da correggere che feature da sviluppare, e aiutare il team a decidere le priorità.

Ogni volta che un bug viene corretto o una feature aggiunta, il software cambia rispetto a prima: serve un modo per identificare con precisione “quale versione” stiamo usando o discutendo in un dato momento.

Versioning: perché numeriamo il software

Hai mai notato che le app sul telefono si aggiornano e mostrano numeri come “2.4.1” o “10.0.3”? Questo si chiama **versioning** (o numerazione delle versioni).

Perché è importante? Immagina di dover parlare con un collega di un problema e dire semplicemente “l'app fa una cosa strana”. Quale app? In che momento? Con quali funzionalità presenti? Senza un numero di versione preciso, sarebbe come descrivere un'auto senza dire il modello o l'anno: informazioni troppo vaghe per essere utili.

Il numero di versione permette di:

- Sapere esattamente quale “fotografia” del software si sta usando o discutendo
- Capire se un problema è già stato risolto in una versione più recente
- Comunicare chiaramente ai clienti cosa cambia da una versione all'altra

Un sistema molto diffuso si chiama **Semantic Versioning** (versionamento semantico), che usa tre numeri nel formato **MAJOR.MINOR.PATCH** (ad esempio **1.2.3**):

- **MAJOR** (il primo numero, “1”): cambia quando si introducono modifiche importanti, che potrebbero non essere compatibili con le versioni precedenti. Ad esempio, se cambia completamente il modo in cui l'app comunica con altri sistemi.
- **MINOR** (il secondo numero, “2”): cambia quando si aggiungono nuove feature, ma tutto quello che c'era prima continua a funzionare come prima.
- **PATCH** (il terzo numero, “3”): cambia quando si correggono bug, senza aggiungere nuove funzionalità.

Esempio pratico: se il software è alla versione 1.2.3 e il team:

- corregge un bug → diventa 1.2.4
- aggiunge una nuova feature (es. il pagamento con Apple Pay) → diventa 1.3.0
- cambia qualcosa in modo così profondo da rompere la compatibilità con l'esterno → diventa 2.0.0

Questo sistema è uno standard molto diffuso nel mondo dello sviluppo software, adottato da moltissime librerie e applicazioni.

Numerare le versioni presuppone però che il codice arrivi in modo ordinato a uno stato “pubblicabile” — e questo, con più persone che scrivono codice insieme, non è affatto scontato. Vediamo come si organizza concretamente il lavoro di squadra sul codice.

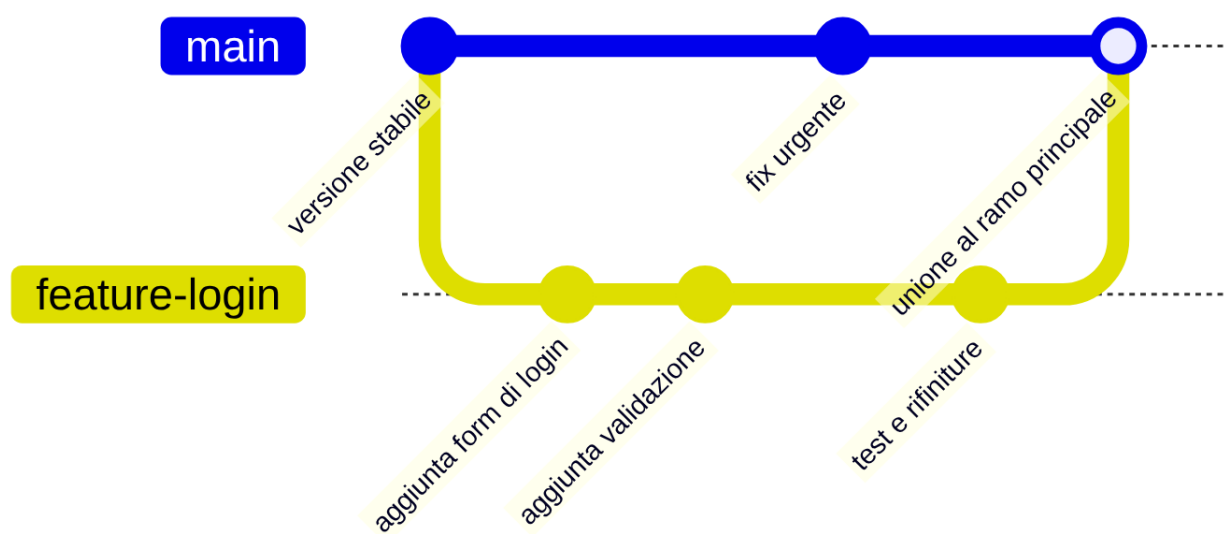
Branching: lavorare in parallelo

A Marco e Ahmed è capitato di recente un piccolo disastro proprio per questo motivo: si sono scambiati via chat lo stesso file per un intervento urgente, ognuno partendo dalla propria copia locale, e uno dei due ha finito per salvare sopra la correzione dell'altro senza saperlo. Lo vedrai raccontato passo passo,

con tutti i dettagli, nella prossima sezione dedicata a Git e GitHub — qui basta l'idea: lavorare sulla stessa copia di un file, in parallelo e senza un sistema che tenga traccia delle modifiche, è un rischio concreto, non un'ipotesi da manuale.

Per questo esiste il concetto di **branch** (in italiano “ramo”). Un branch è come una copia parallela e temporanea del progetto, su cui una persona (o un piccolo gruppo) può lavorare in isolamento, senza disturbare il lavoro degli altri, per poi ricongiungere le modifiche al progetto principale quando sono pronte.

L'analogia migliore è quella dell'albero: il tronco principale rappresenta la versione “ufficiale” e stabile del software (spesso chiamata branch `main` o `master`). Ogni volta che uno sviluppatore deve lavorare su qualcosa — una nuova feature, la correzione di un bug — crea un ramo che si distacca dal tronco, ci lavora in tranquillità, e alla fine quel ramo viene “ricongiunto” al tronco principale.



Per ora ti basta capire l'idea: il branching permette a più persone di lavorare in parallelo sullo stesso progetto senza pestarsi i piedi. I dettagli pratici (comandi, strumenti, esempi passo-passo) arrivano nella prossima sezione.

Esempio pratico: mentre Ahmed lavora sul branch `feature-ricerca-prodotti` per aggiungere la barra di ricerca, Giulia lavora in parallelo sul branch `fix-carrello` per correggere il bug del prezzo negativo visto prima. Nessuno dei due deve aspettare che l'altro finisca: lavorano entrambi sul proprio ramo, isolati, e uniranno le modifiche al tronco principale quando saranno pronti, ciascuno con i suoi tempi.



Dal branch al merge: pull request e code review

Un branch isolato, però, non può restare isolato per sempre: prima o poi il lavoro va ricongiunto al tronco principale. Ma una volta che uno sviluppatore ha finito di lavorare sul proprio ramo (ad esempio ha completato la feature “login con email”), non può semplicemente “ricongiungere” le sue modifiche al tronco principale senza controlli. Sarebbe rischioso: e se il suo codice avesse errori? E se rompesse qualcosa che già funzionava?

Per questo, prima di unire le modifiche, si passa attraverso due passaggi fondamentali: la **pull request** e la **code review**.

Pull Request: “propongo questa modifica”

Una **pull request** (spesso abbreviata **PR** — su altre piattaforme lo stesso concetto può avere un nome diverso, ma la logica è identica) è, in parole semplici, una richiesta formale: “ho finito di lavorare su questo ramo, per favore controllate il mio lavoro e, se va bene, unitelo al progetto principale”.

È come quando, dopo aver scritto una relazione di lavoro, non la invii direttamente al cliente, ma la mandi prima al tuo responsabile per un controllo. La pull request è esattamente quella richiesta di controllo, applicata al codice.

Esempio pratico: Ahmed, che ha finito la barra di ricerca prodotti, apre una pull request con un titolo come “Aggiunta ricerca prodotti per nome e categoria” e una descrizione che spiega cosa cambia, come è stato testato e magari uno screenshot del risultato. A quel punto la pull request compare in una lista visibile a tutto il team su GitHub, pronta per essere revisionata.

Aprire la pull request, però, non basta: qualcuno deve davvero leggere quel codice prima che diventi parte ufficiale del progetto. È il momento della code review.

Code Review: il controllo dei colleghi

La **code review** è il momento in cui uno o più colleghi (spesso sviluppatori più esperti, o semplicemente altri membri del team) leggono il codice scritto e verificano che:

- Faccia davvero quello che deve fare
- Sia scritto in modo chiaro e mantenibile (cioè che altre persone, in futuro, possano capirlo e modificarlo facilmente)
- Non introduca bug o problemi di sicurezza
- Segua le convenzioni e gli standard adottati dal team

Perché è importante che sia qualcun altro a controllare, e non solo chi ha scritto il codice? Per lo stesso motivo per cui è utile far leggere una mail importante a un collega prima di inviarla: chi scrive tende a non vedere i propri errori, mentre un occhio esterno nota cose che altrimenti sfuggirebbero. La code review, inoltre, aiuta a diffondere la conoscenza del progetto tra i membri del team, così che non ci sia una sola persona che “sa tutto” e nessun altro capisce quel pezzo di codice.

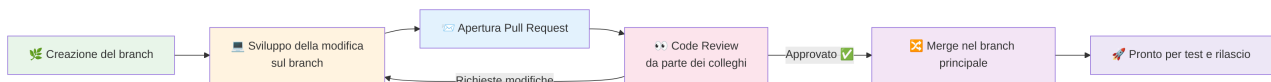
Se durante la code review emergono problemi, chi ha scritto il codice apporta le correzioni richieste, e il processo si ripete finché tutti sono soddisfatti.

Esempio pratico: Giulia, revisionando la pull request della ricerca prodotti, lascia un commento del tipo “qui la ricerca fa distinzione tra maiuscole e minuscole, un utente che scrive ‘scarpe’ con la S minuscola non troverebbe ‘Scarpe’ scritto con la maiuscola: puoi correggerlo?”. Ahmed corregge il codice, aggiorna la pull request, e Giulia la approva.

Merge: l'unione finale

La code review è stata superata: non resta che rendere ufficiale il lavoro. Quando la pull request è stata approvata dai colleghi durante la code review, si procede al **merge**: l'unione definitiva delle modifiche del ramo (branch) al progetto principale. A questo punto, quella nuova funzionalità o correzione diventa parte ufficiale del software, pronta per le fasi successive (ulteriori test, e infine il rilascio).

Esempio pratico: dopo l'approvazione, Ahmed (o un collega con i permessi adeguati) clicca sul bottone “Merge” nello strumento usato dal team. Da quel momento, il codice della barra di ricerca fa parte del branch principale `main` di ShopFacile, insieme al lavoro di tutti gli altri membri del team, e sarà incluso nel prossimo rilascio pianificato.



Questo flusso — branch, pull request, code review, merge — è oggi lo standard adottato dalla stragrande maggioranza dei team di sviluppo professionali nel mondo, e lo ritroverai costantemente nel tuo lavoro quotidiano come Project Manager, anche solo per monitorare lo stato di avanzamento delle attività del team.



ADR: perché una decisione tecnica va scritta

Otto mesi dopo il rilascio del modulo di pagamento, Giulia deve modificare il modo in cui ShopFacile salva i documenti dei clienti (fatture, ricevute) e si imbatte in una scelta che la lascia perplessa: quei file non stanno nel database insieme al resto dei dati, ma su uno storage separato, dedicato solo a loro. Chi ha preso questa decisione non lavora più in azienda. Il codice le dice **cosa** succede — i file vengono scritti su quello storage — ma non **perché** qualcuno abbia scelto di separarli dal database. A questo punto Giulia ha solo due strade, entrambe rischiose: rifare la scelta da zero (rischiando di ripetere lo stesso errore, se era un errore, o di buttare via una buona ragione che semplicemente non conosce), oppure non toccare nulla per paura di rompere qualcosa che non capisce. Entrambe le strade costano tempo e certezza. Il vero costo non è la decisione originale: è la decisione **perduta**.

Luca vede la stessa dinamica da un'angolazione diversa, più politica che tecnica: in riunione, la discussione su “dove mettiamo i documenti dei clienti” torna a galla ogni sei mesi, e ogni volta si riparte da zero, come se non se ne fosse mai parlato.

Per questo esistono gli **ADR (Architecture Decision Record, record di decisione architetturale)**: un documento breve, uno per ogni decisione tecnica rilevante, scritto nel momento in cui la decisione viene presa — non ricostruito a posteriori — e versionato nel repository, accanto al codice che quella decisione riguarda. Il formato è stato proposto nel 2011 da Michael Nygard, ed è oggi ampiamente adottato nei team che si sono scontrati almeno una volta con il problema visto sopra.

Un ADR tipico ha una struttura fissa e volutamente breve:

ADR-NNN: Titolo della decisione

Data: AAAA-MM-GG

Stato: proposta | accettata | superata (da ADR-XXX)

Contesto

Qual è il problema? Quali vincoli tecnici o di business rendono necessaria questa decisione, in questo momento?

Decisione

Cosa si è deciso di fare, in modo diretto e verificabile.

Conseguenze

Cosa cambia, in positivo e in negativo, per il sistema e per chi ci lavorerà in futuro.

Alternative considerate

Quali altre opzioni erano sul tavolo, e perché sono state scartate.

Un esempio compilato, realistico per il caso di Giulia:

ADR-014: Archiviare i documenti dei clienti su object storage, non nel database

Stato: accettata

Contesto: i documenti dei clienti (fatture, ricevute) crescono di volume rapidamente e vengono letti raramente dopo i primi giorni. Tenerli come campi binari nel database appesantisce i backup e le query su una tabella che, per il resto, è consultata di continuo.

Decisione: i documenti vengono salvati su uno storage a oggetti dedicato; il database conserva solo un riferimento (un identificativo) al file, non il file stesso.

Conseguenze: i backup del database restano leggeri e veloci; in cambio, il sistema ora dipende da un servizio esterno in più, che va monitorato separatamente.

Alternative considerate: tenere i file nel database (scartata per il peso sui backup); un file system condiviso tra i server applicativi (scartata perché non scala bene se si aggiungono altri server).

Il dettaglio che vale più di ogni altro per un PM è lo stato “**superata**”: un ADR non si cancella e non si riscrive mai. Se una decisione viene ripensata, si scrive un **nuovo** ADR che dichiara esplicitamente di superare il precedente (e il vecchio ADR resta lì, marcato come superato, non sparisce). Così la storia del ragionamento — cosa si sapeva, cosa si è deciso, perché — resta intatta e consultabile. È, per le

decisioni, lo stesso principio che hai appena visto per il codice con Git: la cronologia non si riscrive per “far finta che non sia successo”, si aggiunge un nuovo capitolo (lo ritroverai in pratica nella prossima sezione).

Come ogni strumento di documentazione, però, gli ADR hanno un costo reale, e un Project Manager deve conoscerlo per non farne un rito vuoto: scriverli richiede tempo che si sottrae ad altro, e se pubblicarne uno richiede un’approvazione lenta e burocratica, il team semplicemente smette di scriverli. All'estremo opposto, se si scrive un ADR per ogni microscopica scelta (il nome di una variabile, l'ordine di due parametri), il formato perde senso: nessuno legge più trenta ADR per trovare quello che conta. Il criterio pratico che vale la pena portarsi via: **scrivi un ADR quando la decisione è costosa da cambiare dopo** — un cambio di database, la scelta di un fornitore esterno, il modo in cui due sistemi si parlano. Per il resto, un buon commento o una riga nella pull request bastano.

Come si collegano tutti questi concetti

Abbiamo visto tanti termini uno alla volta: requisiti, bug, feature, versioning, branch, pull request, code review, merge. Facciamo un piccolo riepilogo con un esempio end-to-end sul progetto ShopFacile, per vedere come tutti i concetti di questa sezione si incastrano insieme in un caso reale (semplificato):

1. Il cliente chiede una nuova funzionalità: “vogliamo che gli utenti possano recuperare la password” → è un **requisito funzionale**, che Sara fa finire nel backlog di ShopFacile come **feature**.
2. Marco fa l'**analisi**: decide di implementarla con un'email contenente un link temporaneo.
3. Ahmed crea un **branch** dedicato e inizia lo **sviluppo**, scrivendo il codice necessario.
4. Durante lo sviluppo scopre e corregge anche un piccolo **bug** preesistente in una funzione che invia le email.
5. Finito il lavoro, apre una **pull request**.
6. Giulia esegue la **code review**, chiede una piccola modifica per rispettare un **requisito non funzionale** (il link deve scadere dopo 30 minuti per motivi di sicurezza).
7. Ahmed corregge, Giulia approva, si fa il **merge**.
8. Il team esegue il **testing** (unitario, di integrazione e manuale) sulla nuova funzionalità.
9. Tutto ok: si prepara il **rilascio** e si assegna un nuovo numero di **versione** a ShopFacile, ad esempio da 2.3.1 a 2.4.0 (è una nuova feature, quindi cambia il numero MINOR).
10. Il software va in **produzione**. Da qui in avanti, entra nella fase di **manutenzione**: se emergono nuovi bug legati a questa funzionalità, il ciclo ripartirà da capo su un nuovo branch.

Come vedi, tutti questi termini che oggi possono sembrare astratti sono in realtà i mattoni con cui è costruito il lavoro quotidiano di qualsiasi team software, ShopFacile incluso. Nelle prossime sezioni approfondirai molti di questi concetti con strumenti e pratiche concrete.

Esercizi pratici

Questi esercizi ti aiutano a consolidare i concetti di questa sezione prima di passare a Git e GitHub. Non servono strumenti particolari: bastano carta, penna e, dove indicato, una breve chiacchierata con un collega.

1. **Mappa il ciclo di vita su un caso reale.** Pensa a una funzionalità che sai essere stata rilasciata di recente nel progetto (chiedi a un collega se non ne conosci una) e scrivi, riga per riga, come pensi si siano svolte le sei fasi del ciclo di vita (requisiti, analisi, sviluppo, testing, rilascio, manutenzione) per quel caso specifico. Non preoccuparti di essere preciso al 100%: l'obiettivo è esercitarti a “vedere” le fasi in un esempio concreto.  **Come verificare:** mostra la tua mappa a un collega del team e chiedigli di correggere i punti in cui hai sbagliato o semplificato troppo.
2. **Classifica 5 requisiti.** Scrivi 5 frasi che potrebbero essere requisiti di un software (puoi inventarle o prenderle da una card reale del backlog del progetto) e per ciascuna indica se è un requisito funzionale o non funzionale, motivando la scelta in una riga.  **Come verificare:** rileggi ogni frase chiedendoti “risponde a cosa fa?” (funzionale) o a “quanto bene lo fa?” (non funzionale); se la risposta non è immediata, probabilmente la frase è scritta in modo ambiguo — prova a riformulare.
3. **Distingui bug da feature su casi reali.** Chiedi a un developer o al tuo referente di mostrarti 3-4 elementi del backlog del progetto (o della board del team) e, prima che te lo dicano loro, provaci a indovinare quali sono bug e quali sono feature, spiegando il perché della tua scelta.  **Come verificare:** confronta le tue risposte con la classificazione reale usata dal team; se hai sbagliato qualcosa, capisci perché (spesso la differenza sta nel fatto che un bug viola un comportamento già previsto, mentre la feature è un'aggiunta nuova).
4. **Simula un versioning.** Immagina che il software del progetto sia alla versione 3.4.2. Elenca 4 eventi ipotetici (2 bug fix, 1 nuova feature, 1 cambiamento che rompe la compatibilità) e scrivi, in ordine, come cambierebbe il numero di versione dopo ciascun evento.  **Come verificare:** rileggi le regole MAJOR.MINOR.PATCH di questa sezione e verifica che ogni tuo passaggio le rispetti (ad esempio, un nuovo MAJOR deve riportare MINOR e PATCH a zero).
5. **Disegna il flusso branch → merge con un esempio del progetto.** Su un foglio (anche a mano), disegna il flowchart “branch, sviluppo, pull request, code review, merge” visto in questa sezione, ma sostituendo le etichette generiche con un esempio verosimile del progetto (es. “branch fix-notifiche-email” invece di “creazione del branch”). Poi racconta il disegno a voce alta, come se lo spiegassi a un collega nuovo arrivato.  **Come verificare:** se riesci a raccontare il disegno senza guardare gli appunti e senza incepparti su nessun passaggio, hai interiorizzato bene il flusso.
6. **Riconosci quando serve un ADR.** Pensa a 4 decisioni tecniche plausibili nel progetto ShopFacile (es. “cambiare il database di produzione”, “rinominare una variabile interna”, “scegliere un fornitore esterno di pagamenti”, “aggiungere un commento nel codice”): per ciascuna, decidi se merita un ADR o no, motivando con il criterio “quanto è costosa da cambiare dopo”.  **Come verificare:** se hai risposto “sì” a tutte e 4 o “no” a tutte e 4, riguarda il criterio: dovrebbero emergere sia casi che meritano un ADR sia casi che non lo meritano.

Collegamenti

- [4. Git e GitHub](#) — per capire in pratica, con comandi ed esempi, come funzionano davvero branch, commit, pull request e merge, e per vedere il modello “cactus” a metà strada tra Git Flow e Trunk Based
- [5. Agile](#) — per scoprire come i team moderni organizzano il ciclo di vita del software in iterazioni brevi e continue
- [9. DevOps](#) — per approfondire come le fasi di sviluppo, testing e rilascio vengono oggi collegate e automatizzate
- [10. CI/CD](#) — per capire come build, test e deploy possono essere automatizzati end-to-end

Risorse

- [Atlassian — What is SDLC? Software Development Life Cycle Explained](#) — panoramica chiara sul ciclo di vita del software
- [Michael Nygard — Documenting Architecture Decisions \(2011\)](#) — il post che ha proposto per primo il formato ADR
- [Semantic Versioning 2.0.0](#) — la specifica ufficiale del versionamento semantico (in inglese, ma con esempi semplici)
- [GitHub Docs — About pull requests](#) — documentazione ufficiale su cosa sono e come funzionano le pull request
- [Atlassian — Code Review Best Practices](#) — guida pratica sul perché e come si fa code review
- [Wikipedia — Software bug \(storia del termine, inclusa la storia della falena di Grace Hopper\)](#) — approfondimento storico e tecnico sull'origine del termine “bug”

4. Git e GitHub

Una settimana dopo aver corretto un bug critico nel checkout di ShopFacile, Marco lo ritrova identico, di nuovo in produzione. Ricostruendo i fatti con Ahmed capisce cosa è successo: si erano scambiati il file `checkout.js` via chat per un intervento urgente, ognuno partendo dalla propria copia locale. Ahmed ha salvato sopra la sua versione più vecchia senza saperlo, cancellando la correzione di Marco — e nessuna delle due copie porta scritto chi l'ha modificata, quando, o perché.

Git e GitHub esistono per risolvere esattamente questo problema, ma applicato al codice sorgente di un software: migliaia di file che decine di persone modificano ogni giorno, in parallelo, senza perdere traccia di chi ha cambiato cosa.

Questa è probabilmente la sezione più “pratica” del corso: gli strumenti che imparerai qui li vedrai citati in ogni standup, in ogni pull request, in ogni pipeline di CI/CD. Non serve che tu diventi uno sviluppatore, ma è fondamentale che tu capisca il vocabolario e la logica di fondo, perché è il linguaggio con cui il tuo team lavora ogni giorno.

Obiettivi della sezione

Alla fine di questa sezione saprai:

- spiegare la differenza tra Git e GitHub;
- capire cosa sono repository, commit, branch e merge;
- capire come funziona una Pull Request e perché è il cuore della collaborazione su GitHub;
- distinguere Issue, Release e Tag;
- conoscere tre modelli di lavoro (workflow) diffusi: **Git Flow**, il modello **cactus (OneFlow)** e il **Trunk Based Development**, e capire quando si usa l'uno o l'altro;
- riconoscere a cosa serve **Maven** e il file `pom.xml` nei progetti Java, e capire perché la gestione delle dipendenze esterne è un problema comune a ogni linguaggio di programmazione.

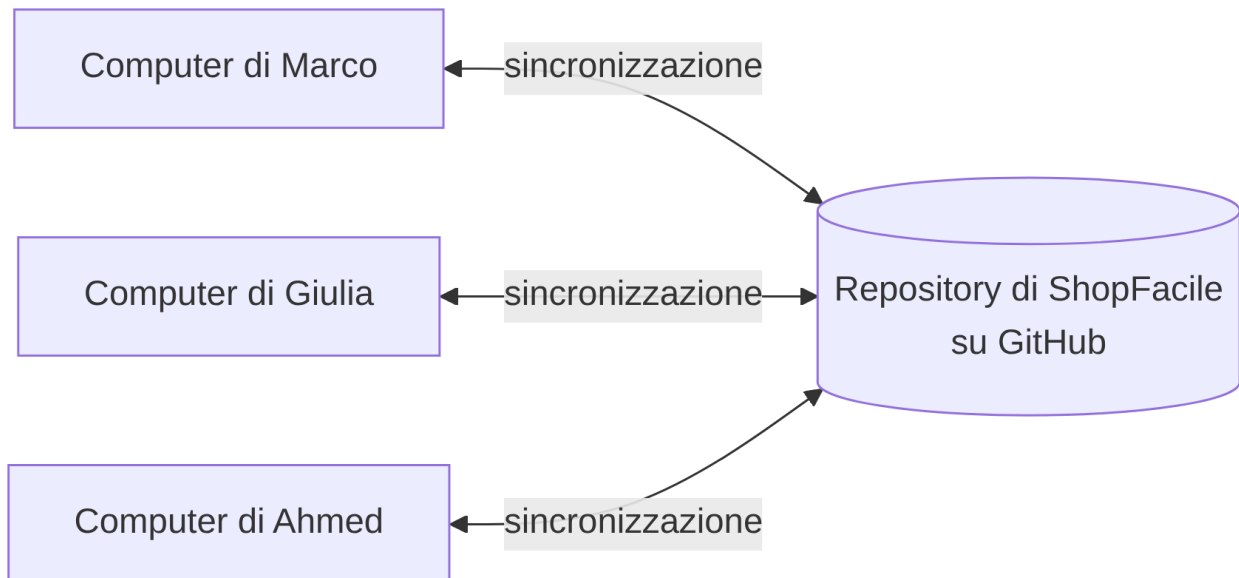
4.1 Cos'è Git

Git è un **sistema di controllo di versione** (in inglese *Version Control System*, o VCS). In parole semplici: è un programma che tiene traccia di **ogni modifica** fatta ai file di un progetto, salvando nel tempo tutta la “cronologia” di quelle modifiche.

 **Analogia:** pensa alla cronologia delle modifiche di Google Docs o alla funzione “Traccia modifiche” di Word, ma molto più potente. Con Git non solo puoi vedere chi ha cambiato cosa e quando, ma puoi anche: - tornare indietro a una versione precedente in qualsiasi momento; - far lavorare **più persone in parallelo** sullo stesso progetto senza che si sovrascrivano il lavoro a vicenda; - “unire” automaticamente le modifiche fatte da persone diverse.

Git è stato creato nel 2005 da Linus Torvalds (lo stesso creatore di Linux) per gestire lo sviluppo del kernel Linux, un progetto con migliaia di collaboratori. Oggi è lo standard de facto per versionare codice, usato praticamente da ogni azienda software al mondo.

Un punto fondamentale: **Git è distribuito**. Questo significa che ogni persona che lavora su un progetto ha, sul proprio computer, una **copia completa** di tutta la cronologia del progetto — non solo l'ultima versione, ma *tutta la storia* fin dall'inizio. Questo è molto diverso dai vecchi sistemi centralizzati, dove esisteva un solo “archivio” centrale e se quello andava giù, si perdeva tutto.



Git funziona da **riga di comando** (il terminale), ma esistono anche interfacce grafiche (come GitHub Desktop, GitKraken, o l'integrazione diretta in Visual Studio Code) che rendono tutto più visuale. In questo corso vedrai comandi da terminale a scopo illustrativo: non devi memorizzarli a memoria, l'obiettivo è che tu capisca **la logica**.

4.2 Cos'è GitHub (e non è la stessa cosa di Git!)

Nel diagramma di prima abbiamo scritto “GitHub” sopra il server remoto un po' di sfuggita: è il momento di chiarire di cosa si tratta davvero. Uno degli equivoci più comuni per chi inizia, infatti, è pensare che Git e GitHub siano la stessa cosa. Non lo sono.

	Git	GitHub
Cos'è	Uno strumento (software)	Una piattaforma online (un servizio)
Dove vive	Sul tuo computer	Su internet (cloud), o su un server aziendale (GitHub Enterprise)
Cosa fa	Gestisce la cronologia delle modifiche	Ospita i repository online e aggiunge funzionalità di collaborazione
Necessario?	Sì, è il motore	No, è un servizio che <i>usa</i> Git

 **Analogia:** Git è come il motore di un'auto: fa il lavoro tecnico di spostare la macchina. GitHub è come una concessionaria con parcheggio, assistenza e servizi extra: ospita l'auto, la rende visibile ad altri, offre strumenti di manutenzione condivisa. Potresti guidare l'auto (usare Git) senza mai passare dalla concessionaria (senza mai usare GitHub) — ma se lavori in team, avere un posto centrale dove parcheggiare e collaborare è estremamente comodo.

GitHub, in concreto, offre:

- uno spazio online dove “ospitare” i repository (i progetti);
- strumenti di collaborazione come **Pull Request**, **Issue**, revisioni del codice, commenti;
- automazioni (**GitHub Actions**, che vedremo nella sezione CI/CD);
- gestione di permessi, team, sicurezza.

GitHub **non è l'unica piattaforma** di questo tipo: esistono alternative diffuse con funzionalità molto simili, a volte con nomi leggermente diversi per gli stessi concetti — ma la logica di fondo resta sempre identica: Git sotto il cofano, una piattaforma di collaborazione sopra.

Il concetto di fondo è sempre lo stesso (Git sotto il cofano), cambia la piattaforma “concessionaria” sopra. In questo corso useremo GitHub come riferimento perché è la piattaforma usata realmente dal team, sia in versione cloud (github.com) sia, in alcuni contesti aziendali più grandi, in versione **self-hosted** con **GitHub Enterprise Server** (installata sui server dell'azienda, ad esempio su un indirizzo come github.tuaazienda.it), quando l'azienda vuole mantenere il controllo completo della propria infrastruttura.

4.3 Repository: il “progetto” versionato

GitHub ospita i progetti, dicevamo: ma cosa contiene esattamente uno di quei progetti da un punto di vista tecnico? La risposta è il repository. Un **repository** (spesso abbreviato “repo”) è semplicemente **la cartella di un progetto**, ma tracciata da Git. Contiene tutti i file del progetto (codice, documentazione, configurazioni) più una cartella speciale invisibile chiamata **.git**, dove Git conserva tutta la cronologia delle modifiche.

 **Analogia:** se il progetto è un libro, il repository è sia il libro stesso sia l'archivio completo di tutte le bozze precedenti, capitolo per capitolo, con la data e l'autore di ogni modifica.

Un repository può essere: - **locale**: la copia che vive sul tuo computer; - **remoto**: la copia ospitata online (su GitHub, cloud o self-hosted), che funge da punto di incontro condiviso tra tutti i membri del team.

```
# Creare un nuovo repository da zero, dentro una cartella
git init

# Oppure "clonare" (scaricare) un repository che esiste già online
git clone https://github.com/nome-organizzazione/nome-progetto.git
```

Dopo un `git clone`, avrai sul tuo computer una copia completa del progetto, compresa tutta la sua storia — pronta per essere modificata.

 **Esempio pratico:** immagina che Marco ti mandi il link al repository GitHub del progetto, ad esempio `https://github.com/shopfacile/backend-api` (o, se l'azienda usa un'istanza self-hosted, qualcosa come `https://github.tuaazienda.it/shopfacile/backend-api`). Aprendolo nel browser vedrai: una lista di cartelle e file (il codice), un pulsante verde “Code” per clonarlo, il numero di branch attivi, e una voce “Commits” con la cronologia di tutte le modifiche fatte finora. Se lo clonassi sul tuo computer con `git clone https://github.com/shopfacile/backend-api.git`, ti ritroveresti in una cartella `backend-api` con dentro tutti quei file, pronti per essere letti — anche se non li modifichi mai.

Maven e il file POM: gestire le librerie che il team non ha scritto

Un repository come `backend-api`, però, non contiene *tutto* il codice che il software usa davvero. Il software di oggi non è scritto tutto in casa: un'applicazione tipica si appoggia a decine di librerie scritte da altri (per parlare con un database, per gestire le date, per validare un'email), e ognuna di quelle librerie usa a sua volta altre librerie. Scaricarle a mano una per una e infilarle nel repository è ingestibile — e il classico “sul mio computer funziona” nasce spesso proprio da due sviluppatori con due versioni leggermente diverse della stessa libreria installate a mano.

Nel mondo Java, lo strumento che risolve questo problema si chiama **Maven**: gestisce sia la build del progetto (compilazione, test, creazione del pacchetto finale) sia le sue dipendenze esterne. Il cuore di Maven è un file chiamato **POM (Project Object Model)**, quasi sempre salvato come `pom.xml` nella radice del repository — è lì che, aprendo un progetto Java, riconoscerai a colpo d'occhio “questo usa Maven”.

```

<project>
  <groupId>it.shopfacile</groupId>
  <artifactId>backend-api</artifactId>
  <version>2.4.0</version>          <!-- versione del progetto (sezione 3) -->

  <dependencies>
    <dependency>
      <groupId>org.postgresql</groupId>
      <artifactId>postgresql</artifactId>
      <version>42.7.3</version>      <!-- una dipendenza dichiarata, con versione
        precisa -->
    </dependency>
  </dependencies>
</project>

```

Poche cose contano davvero, per riconoscere e dialogare, non per scrivere XML: la dipendenza dichiarata nel `pom.xml` porta con sé anche le **dipendenze transitive**, quelle di cui nessuno nel team ha scritto una riga né chiesto esplicitamente — arrivano perché la libreria che hai dichiarato ne usa altre a sua volta — e sono spesso il punto in cui compaiono le vulnerabilità note di sicurezza (approfondito nella sezione 13). In azienda, queste librerie non si scaricano da internet a ogni build: vengono scaricate una volta e conservate in un **repository di artifact** interno (strumenti come Nexus o Artifactory), sia per velocità sia per non dipendere da internet in produzione. Il ciclo tipico di build Maven è `compile → test → package → install → deploy`, lo stesso spirito delle fasi di pipeline che vedrai nella sezione 10. Ecosistemi diversi risolvono lo stesso problema con nomi diversi: **npm** per JavaScript (già citato nella sezione 10), **pip** per Python, **NuGet** per .NET — stesso ruolo di Maven, in un altro mondo.

4.4 Commit: uno scatto fotografico delle modifiche

Il repository conserva “tutta la cronologia delle modifiche”, abbiamo detto: ma di cosa è fatta, concretamente, questa cronologia? È fatta di commit. Un **commit** è un salvataggio “ufficiale” di un insieme di modifiche, con un messaggio che spiega **cosa** e **perché** è stato cambiato.



Analogia: immagina di scattare una fotografia allo stato attuale del progetto. Ogni commit è uno scatto: cattura esattamente come erano i file in quel momento, con un’etichetta (il messaggio di commit) che descrive cosa è successo da uno scatto all’altro. Con tante fotografie in sequenza, hai un album completo della storia del progetto.

Il flusso tipico per creare un commit è:

```
# 1. Vedo quali file ho modificato
git status

# 2. "Metto in valigia" le modifiche che voglio salvare (staging)
git add nome-file.txt
# oppure, per aggiungere tutti i file modificati:
git add .

# 3. Creo il commit con un messaggio descrittivo
git commit -m "Aggiunge la validazione del campo email nel form di registrazione"
```

Alcune buone pratiche sui messaggi di commit (le vedrai spesso menzionate nelle Pull Request del team):

- il messaggio deve spiegare **il perché**, non solo il “cosa” (il “cosa” si vede già dal codice cambiato);
- meglio tanti commit piccoli e mirati che un unico commit enorme con scritto “fix vari”;
- molti team adottano convenzioni standard, ad esempio i [Conventional Commits](#) (**feat:** , **fix:** , **docs:** , **chore:** ...).

Ogni commit ha un **identificativo univoco** (un codice esadecimale, es. **a1b2c3d**), che permette di riferirsi esattamente a quello scatto in qualsiasi momento futuro — utile per capire, ad esempio, “in quale commit è stato introdotto questo bug?”.

 **Esempio pratico:** dopo tre commit di Marco su ShopFacile, se lanci **git log --oneline** vedresti qualcosa così:

```
a1b2c3d Aggiunge la validazione del campo email nel form di registrazione
9f8e7d6 Aggiunge il form di registrazione utente
3c4d5e6 Setup iniziale del progetto
```

Ogni riga è un commit: il codice a sinistra (es. **a1b2c3d**) è l'identificativo univoco, il testo a destra è il messaggio. Se un giorno Giulia nota un bug nella validazione email, potrà usare **git log** per capire esattamente in quale commit — e quindi in che momento — è stata introdotta quella modifica.

 **La cronologia dei commit è per sempre:** qualsiasi cosa finisca in un commit resta nella storia del repository per sempre, anche se in un commit successivo cancelli quel file. Chiunque abbia accesso al repository può tornare indietro fino al primo commit e recuperarla.

La conseguenza operativa, importante anche per un PM: se una password, una chiave API o un altro segreto finisce per errore in un commit, **non basta rimuovere il file**. Quella credenziale va considerata compromessa e va **sostituita subito**, ovunque venga usata — riscrivere la cronologia per “far finta che non sia successo” è un’operazione tecnica complessa e rischiosa, non una soluzione al problema.

Lo strumento che previene tutto questo a monte è il file `.gitignore`: elenca quali file (chiavi, credenziali, file temporanei, cartelle di build) Git deve ignorare, evitando che entrino per errore in un commit.

4.5 Branch: un ramo di lavoro parallelo

Ogni commit di Marco si accumula in sequenza sulla stessa linea temporale: ma cosa succede quando Giulia e Ahmed devono lavorare *contemporaneamente* su cose diverse, senza pestarsi i piedi? Serve poter deviare dalla linea principale, ed è qui che entra in gioco il branch.

C’è una posta in gioco più seria del semplice “non ostacolarsi”: ShopFacile è già online, con clienti che stanno pagando in questo momento. Se Marco, per aggiungere un nuovo metodo di pagamento, lavorasse direttamente su `main`, un suo errore fermerebbe il checkout subito, per tutti. Il branch è prima di tutto una **rete di sicurezza verso la produzione**, e solo in secondo luogo un modo per non ostacolare i colleghi.

Un **branch** (letteralmente “ramo”) è una linea di sviluppo indipendente che parte da un punto della storia del progetto. Permette di lavorare su qualcosa di nuovo (una funzionalità, una correzione) **senza toccare** la versione “stabile” del progetto, che di solito vive nel branch principale chiamato `main` (in passato spesso chiamato `master`). La transizione da un nome all’altro si è diffusa intorno al 2020: la ragione più citata è la volontà di abbandonare una coppia di termini (`master` / `slave`, usata in informatica per indicare un componente che comanda e uno che obbedisce) letta da molti come inopportuna. Il passaggio non è stato universale, così entrambi i nomi si incontrano ancora nei progetti esistenti: se apri un repository più vecchio e trovi `master`, è semplicemente il branch principale con il vecchio nome.



Analogia: pensa alla linea principale della storia (`main`) come alla strada maestra di un progetto: deve restare sempre percorribile e affidabile. Un branch è come un cantiere laterale in cui provi cose nuove: se qualcosa va storto, il cantiere non blocca la strada principale. Quando il lavoro nel cantiere è pronto e testato, lo “collegi” alla strada maestra.

```
# Creo un nuovo branch chiamato "feature/login-utente"
git branch feature/login-utente

# Mi sposto su quel branch per iniziare a lavorarci
git checkout feature/login-utente

# In alternativa, questi due comandi si possono unire in uno solo:
git checkout -b feature/login-utente
```

Convenzioni di naming comuni per i branch (utili da riconoscere quando le vedi nelle Pull Request):

- `feature/...` → nuova funzionalità (es. `feature/login-utente`)
- `fix/...` o `bugfix/...` → correzione di un bug
- `hotfix/...` → correzione urgente su produzione
- `release/...` → preparazione di una nuova versione

Lavorare su branch separati è ciò che permette a più persone del team di lavorare **in parallelo sullo stesso progetto** senza pestarsi i piedi.



Esempio pratico: il team di ShopFacile deve sviluppare due cose in parallelo: una nuova pagina di login e la correzione di un bug nel checkout. Marco e Ahmed lanceranno:

```
git checkout -b feature/login-utente      # Marco
git checkout -b fix/checkout-prezzo-errato # Ahmed
```

Da questo momento lavorano su due “cantieri” separati: i commit di Marco non toccano i file su cui lavora Ahmed (a meno che non modifichino esattamente gli stessi file), e nessuno dei due tocca `main`, che resta stabile per tutti gli altri.

4.6 Merge: unire due rami

Marco e Ahmed, però, non possono lavorare per sempre su rami separati: prima o poi il loro lavoro deve tornare a far parte di un'unica versione condivisa. L'operazione che lo rende possibile è il merge. Il **merge** è l'operazione che **unisce le modifiche** fatte su un branch dentro un altro branch (tipicamente, si uniscono le modifiche di un branch di feature dentro `main`, una volta che il lavoro è completo e testato).



Analogia: riprendendo l'immagine di prima, il merge è il momento in cui il cantiere laterale viene collegato alla strada principale: da quel momento, chi percorre la strada maestra passa anche dal nuovo tratto.

```
# Mi sposto sul branch che deve "ricevere" le modifiche
git checkout main

# Unisco le modifiche del branch di feature dentro main
git merge feature/login-utente
```

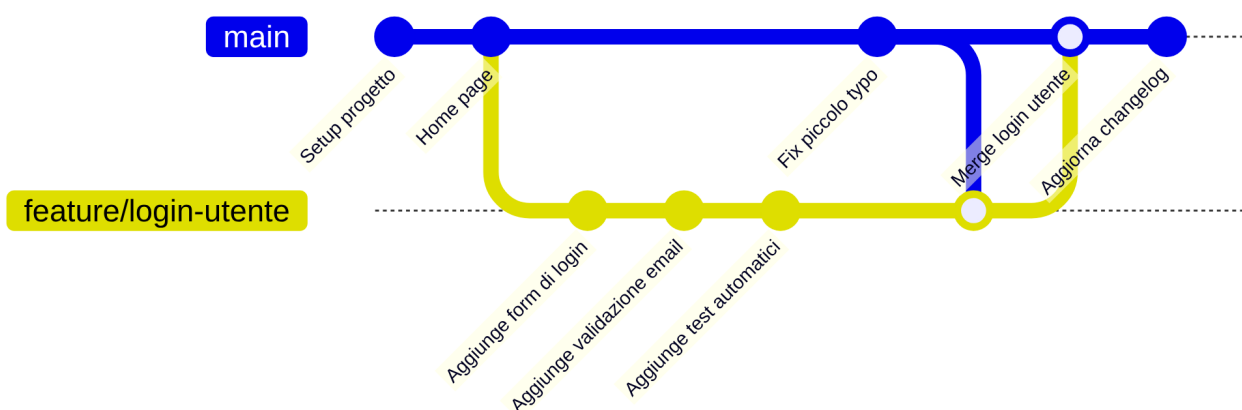
Git è molto bravo a unire automaticamente modifiche fatte su parti diverse dei file. Ma se due persone hanno modificato **esattamente la stessa riga** in modo diverso, Git non sa quale versione tenere: si genera un **conflitto di merge**, che va risolto manualmente decidendo quale modifica (o quale combinazione) mantenere. Non preoccuparti: è un evento normalissimo nel lavoro quotidiano, non un errore grave.

🔧 **Esempio pratico:** supponiamo che il branch `feature/login-utente` sia pronto. Da `main` lanci:

```
git checkout main
git merge feature/login-utente
```

Se nessuno ha toccato le stesse righe su `main` nel frattempo, Git risponde con qualcosa come `Fast-forward` o `Merge made by the 'ort' strategy` e il lavoro è unito automaticamente. Se invece Giulia aveva modificato lo stesso file nello stesso punto, Git si ferma e restituisce un messaggio come `CONFLICT (content): Merge conflict in login.js` — a quel punto tocca aprire il file, decidere quale versione tenere (o combinarle), e completare il merge con `git add + git commit`.

Ecco un diagramma che riassume l'intero ciclo di vita di un branch di feature, dalla creazione al merge:



Cosa racconta questo diagramma: mentre il branch `feature/login-utente` procedeva con i suoi commit, anche `main` è avanzato con un piccolo fix. Il branch di feature ha "assorbito" quell'aggiornamento (merge di `main` dentro il branch), per essere sicuro di lavorare sulla versione più recente, e infine è stato unito (merge) dentro `main`, portando tutto il lavoro fatto nella linea principale.

4.7 Pull Request: il cuore della collaborazione su GitHub

Abbiamo visto come funziona un merge da riga di comando, ma nella pratica quotidiana su GitHub il merge non viene quasi mai lanciato “a freddo”: passa prima per uno strumento che formalizza la richiesta e apre lo spazio per la revisione, la Pull Request. Una **Pull Request** (spesso abbreviata **PR**) è una **richiesta formale** di unire le modifiche di un branch dentro un altro branch (di solito dentro `main`), passando prima per una **revisione** da parte di altri membri del team.

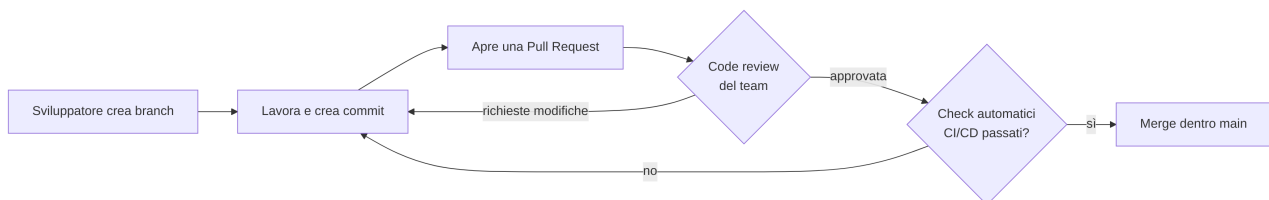
Il nome, tradotto alla lettera, è “richiesta di *tirare dentro*” (pull) il proprio lavoro nel ramo principale: eredità dell’epoca in cui si chiedeva a chi manteneva un progetto open source di prelevare le modifiche da un’altra copia del repository, su un altro computer.

Questo è probabilmente il concetto più importante di tutta la sezione, perché è il meccanismo attraverso cui lavora quasi ogni team che usa GitHub. (Su altre piattaforme lo stesso identico concetto può avere un nome diverso — non farti confondere dal nome: la logica è la stessa.)

 **Analogia:** è come sottoporre una bozza di documento a un collega prima di renderla definitiva. Non modifichi direttamente il documento ufficiale: proponi le tue modifiche, il collega le legge, lascia commenti, magari chiede correzioni, e solo quando tutti sono d’accordo la bozza diventa la versione ufficiale.

Il flusso tipico di una Pull Request:

1. Uno sviluppatore (ad esempio Marco) crea un branch e ci lavora (con i suoi commit).
2. Quando il lavoro è pronto, apre una **Pull Request** su GitHub, proponendo di unire quel branch dentro `main` (o dentro un altro branch di destinazione).
3. Altri membri del team fanno la **code review**: leggono le modifiche, lasciano commenti, chiedono chiarimenti o correzioni.
4. Spesso, in parallelo, delle **pipeline automatiche** (CI/CD, che vedremo nella sezione 10) eseguono automaticamente test e controlli di qualità sul codice della PR.
5. Quando tutto è approvato (“approved”) e i controlli automatici sono passati (i cosiddetti “check verdi”), la Pull Request viene unita (merge) — spesso con un semplice click sul pulsante “Merge” su GitHub.



Perché le Pull Request sono così importanti per un Project Manager? Perché sono il punto in cui puoi **misurare visivamente lo stato di avanzamento** del lavoro: quante PR sono aperte, quante sono in attesa di revisione, quanto tempo restano aperte prima di essere unite (un indicatore utile di quanto è fluido il processo del team).

 **Esempio pratico:** Marco ha finito la feature login su ShopFacile. I passaggi concreti su GitHub sarebbero: 1. Push del branch: `git push origin feature/login-utente`. 2. Su GitHub compare un banner “Compare & pull request”: clicca, scrivi un titolo (es. “Aggiunge login utente con validazione email”) e una descrizione di cosa cambia e perché. 3. Assegna Giulia come reviewer. 4. Giulia lascia commenti tipo “Perché qui usiamo `==` invece di `===` ?” — Marco risponde o corregge con un nuovo commit sullo stesso branch, che aggiorna automaticamente la PR. 5. Quando i commenti sono risolti e i check automatici (CI/CD) sono verdi, Giulia clicca “Approve”, poi qualcuno clicca “Merge pull request”.

Da Project Manager, vedresti questa PR nella sezione “Pull requests” del repository con un’etichetta verde “Open” che diventa viola “Merged” a fine processo.

4.8 Issue: tracciare bug e richieste

Finora abbiamo parlato di codice già scritto: branch, commit, pull request. Ma da dove nasce il lavoro stesso, cioè “cosa” bisogna scrivere o correggere?

Prima che ShopFacile adottasse le Issue, i bug segnalati dai clienti finivano in email sparse o in un foglio Excel condiviso aggiornato a intermittenza: Sara si ritrovava a chiedere in chat “qualcuno si sta già occupando del bug del checkout?” senza risposta certa, perché nessuno sapeva se un problema fosse già noto, già assegnato o già risolto.

Spesso il lavoro nasce da una Issue. Una **Issue** è una “voce” che descrive un problema da risolvere o una richiesta da realizzare: un bug da correggere, una nuova funzionalità da sviluppare, un miglioramento da valutare.

 **Analogia:** è come un “biglietto” (ticket) di un help desk: qualcuno segnala un problema o una richiesta, il biglietto viene assegnato a qualcuno, discusso, e chiuso quando risolto.

Una Issue tipicamente contiene: - un titolo chiaro; - una descrizione del problema o della richiesta (spesso con passi per riprodurre un bug, screenshot, comportamento atteso vs. osservato); - **etichette** (label), es. `bug`, `enhancement`, `documentazione`; - un **assegnatario** (chi ci lavora); - commenti di discussione.

Le Issue si possono collegare direttamente alle Pull Request: è comune vedere in una PR la frase “Closes #42”, che indica che, una volta unita quella PR, la Issue numero 42 verrà chiusa automaticamente perché risolta.

Molti team di progetto (incluso probabilmente il tuo) usano le **Issue di GitHub** come base per la gestione del backlog che vedremo nelle sezioni su Agile, Scrum e Kanban.

🔧 **Esempio pratico:** un cliente di ShopFacile segnala che il pulsante “Conferma ordine” non funziona su un certo browser. Sara apre una Issue così: - **Titolo:** “Il pulsante Conferma ordine non risponde su Safari” - **Descrizione:** passi per riprodurlo, screenshot, browser e versione usati - **Label:** `bug`, `priorità-alta` - **Assegnatario:** Ahmed, che se ne occuperà

Ahmed apre un branch `fix/conferma-ordine-safari`, risolve il problema, e nella descrizione della Pull Request scrive “Closes #57” (dove 57 è il numero della Issue): una volta unita la PR, GitHub chiude automaticamente la Issue numero 57.

4.9 Release: una versione pubblicata ufficialmente

La Issue #57 è stata chiusa, il fix è unito in `main`: ma quando quel fix arriva davvero nelle mani degli utenti, con quale nome viene comunicato? Con quello di una Release. Una **Release** è un punto preciso della storia del progetto che viene “pubblicato” ufficialmente come versione consegnabile agli utenti finali (o al cliente).

💡 **Analogia:** se i commit sono gli scatti fotografici quotidiani di un album, la release è la **foto scelta per la copertina** di un capitolo: un momento speciale, marcato e comunicato a tutti come “questa è la versione 2.3.0, disponibile da oggi”.

Le release seguono spesso uno schema di numerazione chiamato **Semantic Versioning** (`MAJOR.MINOR.PATCH`, es. `2.3.1`): - **MAJOR:** cambiamenti grandi, potenzialmente non compatibili con le versioni precedenti (es. `1.x` → `2.0.0`); - **MINOR:** nuove funzionalità compatibili con le versioni precedenti (es. `2.2.x` → `2.3.0`); - **PATCH:** correzioni di bug, senza nuove funzionalità (es. `2.3.0` → `2.3.1`).

Su GitHub, una Release viene solitamente accompagnata da **note di rilascio** (release notes): un elenco leggibile di cosa è cambiato, utilissimo anche per un Project Manager che deve comunicare al cliente o agli stakeholder cosa contiene la nuova versione.

🔧 **Esempio pratico:** il team di ShopFacile rilascia la versione `2.3.0` del prodotto. Sulla pagina “Releases” di GitHub troveresti una voce con: - tag `v2.3.0`; - titolo “Versione 2.3.0 — Gestione utenti”; - note di rilascio tipo: “✨ Nuove funzionalità: gestione ruoli utente. 🐛 Fix: corretto un bug nel checkout. ⚠️ Nota: richiede un aggiornamento del database.”; - eventuali file scaricabili (es. un installer).

Un Project Manager potrebbe riprendere direttamente queste note di rilascio — magari semplificandole — in una comunicazione al cliente o in un aggiornamento agli stakeholder.

4.10 Tag: etichettare un punto preciso della storia

Ogni Release che abbiamo appena visto, in realtà, si appoggia su un meccanismo più elementare di Git per individuare quel punto preciso della storia: il tag. Un **Tag** è un'etichetta che punta a un commit specifico, per marcarlo come "punto di interesse" — quasi sempre usato insieme alle release.

 **Analogia:** se la storia del progetto è un lungo rotolo di pellicola fotografica, un tag è un segnalibro adesivo attaccato a uno scatto preciso, con scritto sopra "v2.3.0" — così puoi ritrovarlo istantaneamente anche tra migliaia di altri scatti.

```
# Creo un tag "annotato" (con messaggio) sul commit corrente
git tag -a v2.3.0 -m "Versione 2.3.0: aggiunta gestione utenti"

# Invio il tag anche al repository remoto (altrimenti resta solo locale)
git push origin v2.3.0
```

Differenza pratica tra Tag e Release: il **tag** è un concetto di Git (un puntatore a un commit), mentre la **Release** è un concetto di GitHub costruito sopra un tag, con in più note descrittive, file scaricabili (binari, installer) e visibilità nella pagina del progetto.

 **Esempio pratico:** dopo aver taggato e pubblicato la versione **v2.3.0**, se sei sul repository e lanci **git tag**, vedresti l'elenco di tutti i tag creati nel tempo:

```
v1.0.0
v1.1.0
v2.0.0
v2.1.0
v2.2.0
v2.3.0
```

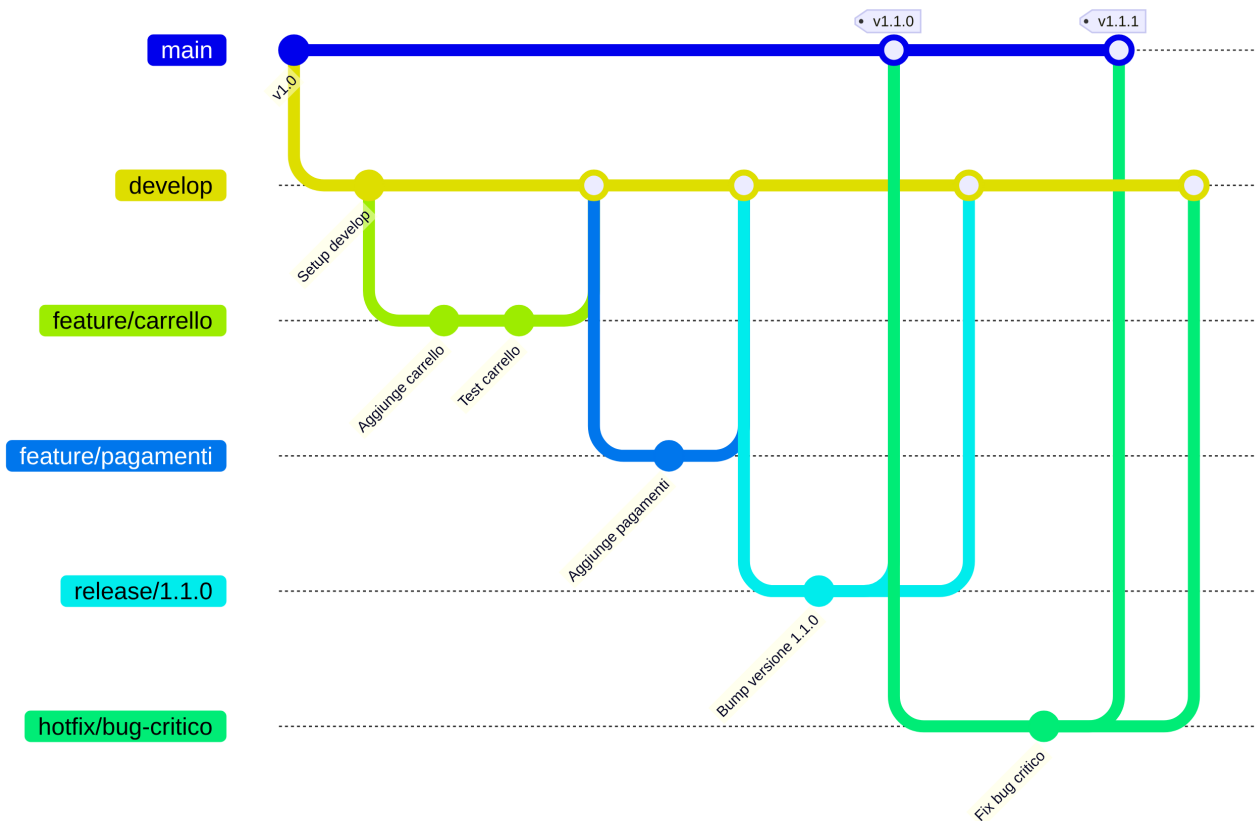
Se un giorno serve tornare esattamente alla versione rilasciata qualche mese prima, basta fare **git checkout v2.1.0** (se quello era il tag di quella release) per vedere il progetto esattamente come era in quel momento, senza dover cercare "a occhio" nella cronologia dei commit.

4.11 Git Flow: un workflow strutturato a più branch

Finora abbiamo visto i singoli “mattoncini” (branch, merge, PR, tag, release): ma come si combinano insieme in una strategia coerente, giorno per giorno, per tutto il team? Un primo modello molto diffuso è Git Flow. **Git Flow** è un modello di organizzazione dei branch molto diffuso, pensato per progetti con **cicli di rilascio pianificati** (es. una nuova versione ogni mese) e la necessità di gestire più cose in parallelo: nuove funzionalità, preparazione di una release, correzioni urgenti su produzione.

Git Flow prevede branch con ruoli specifici:

- **main** : contiene sempre il codice in produzione, stabile al 100%.
- **develop** : il branch di “integrazione”, dove confluiscono le funzionalità completate, in attesa della prossima release.
- **feature/...** : branch temporanei per sviluppare singole funzionalità, che partono da **develop** e ci ritornano a lavoro finito.
- **release/...** : branch temporanei usati per finalizzare una versione (test, piccoli fix, aggiornamento numero di versione) prima di pubblicarla.
- **hotfix/...** : branch temporanei per correzioni urgenti direttamente su **main**, per problemi critici in produzione che non possono aspettare il prossimo ciclo di release.



Quando usare Git Flow: è utile quando il progetto ha rilasci pianificati e non continui (es. release mensili), quando serve mantenere in parallelo più versioni in produzione, o quando il processo di approvazione/rilascio è complesso e richiede una fase di stabilizzazione prima di ogni release. Lo svantaggio è che è un processo più pesante, con più branch da gestire e più passaggi di merge.

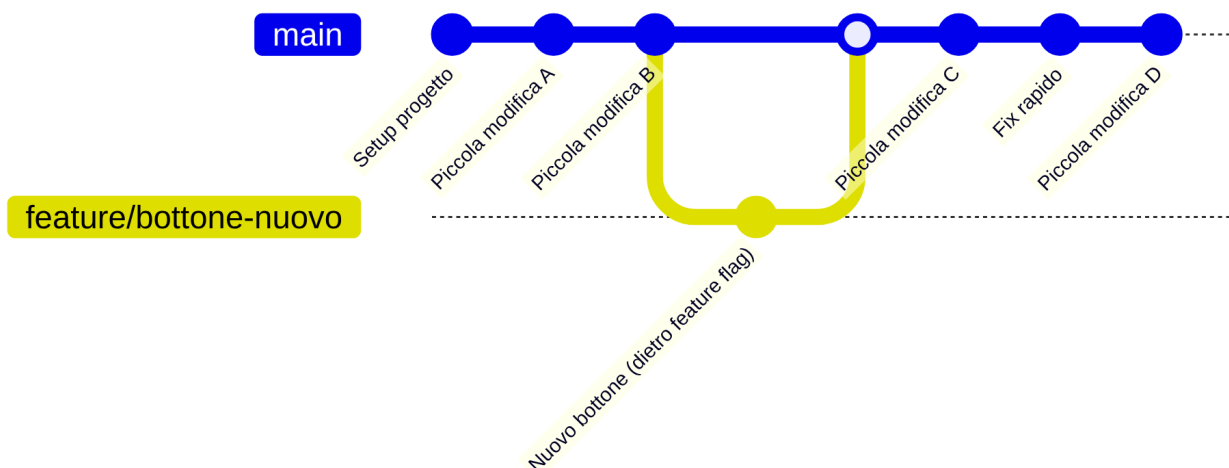
🔧 **Esempio pratico:** immaginiamo che ShopFacile rilasci una nuova versione ogni mese. Lo scenario tipico con Git Flow: - Marco, Giulia e Ahmed lavorano tutto il mese su vari branch `feature/...` che partono da `develop` e ci tornano quando pronti; - una settimana prima del rilascio si crea `release/1.4.0` da `develop`: qui si fanno solo piccoli fix e test finali, niente nuove funzionalità; - quando è tutto stabile, `release/1.4.0` viene unito sia in `main` (con tag `v1.4.0`, che diventa la produzione) sia di nuovo in `develop`; - se due giorni dopo il rilascio emerge un bug critico in produzione, si crea `hotfix/bug-pagamenti` direttamente da `main`, si corregge, si unisce in `main` (nuovo tag `v1.4.1`) e anche in `develop`, così il fix non si perde nel prossimo rilascio.

4.12 Trunk Based Development: il workflow semplificato

Git Flow, con i suoi cinque tipi di branch, funziona bene per rilasci pianificati, ma è un processo pesante per chi rilascia molto più spesso: esiste un'alternativa decisamente più snella. Il **Trunk Based Development** (TBD) è un approccio molto più semplice: tutti i membri del team lavorano il più possibile direttamente su **un unico branch principale** (il "trunk", cioè `main`), con **commit frequenti e piccoli**, integrando il proprio lavoro più volte al giorno.

💡 **Analogia:** se Git Flow è come costruire un edificio con tanti cantieri separati che vengono collegati alla struttura principale solo a lavori ultimati, il Trunk Based Development è come aggiungere un mattone alla volta direttamente sulla struttura principale, controllando ogni singolo mattone prima di posarlo, così l'edificio resta sempre in piedi e visitabile.

I branch di feature, quando si usano, sono **molto brevi** (durano ore o al massimo un paio di giorni, non settimane), e le funzionalità non ancora pronte per gli utenti finali vengono spesso "nascoste" tramite **feature flag** (interruttori che attivano/disattivano una funzionalità) invece che tenute isolate su un branch a lungo termine.



Quando usare Trunk Based Development: funziona molto bene quando il team ha una **suite di test automatici solida** e una pipeline di CI/CD matura, capace di verificare rapidamente che ogni piccolo cambiamento non rompa nulla. È il modello preferito dai team che fanno rilasci **continui** (più volte al giorno o alla settimana), tipico della cultura DevOps che vedremo più avanti nel corso. Lo svantaggio è che richiede molta disciplina e automazione: senza test solidi, integrare spesso su un unico branch diventa rischioso.

 **Esempio pratico:** immaginiamo che il team di ShopFacile lavori con Trunk Based Development e una pipeline CI/CD solida. Giulia deve aggiungere un nuovo filtro di ricerca: - crea un branch `feature/filtro-ricerca` la mattina; - fa 2-3 commit piccoli nel corso della giornata, ciascuno verificato automaticamente dalla pipeline; - il filtro non è ancora pronto per tutti gli utenti, quindi lo nasconde dietro un feature flag chiamato `nuovo_filtro_ricerca_attivo` (inizialmente spento); - entro sera apre una PR piccola e veloce da revisionare, e la unisce in `main`; - il codice è ora in produzione ma invisibile agli utenti (flag spento); quando il team è pronto, attiva il flag per tutti (o per un gruppo di test) senza dover rilasciare nulla di nuovo.

Confronto rapido

	Git Flow	Trunk Based Development
Struttura branch	Molti branch a lungo termine (<code>main</code> , <code>develop</code> , <code>feature</code> , <code>release</code> , <code>hotfix</code>)	Un solo branch principale, branch di feature brevissimi
Frequenza integrazione	Bassa (le feature restano isolate a lungo)	Alta (più commit al giorno su <code>main</code>)
Adatto a	Rilasci pianificati, versioni multiple in produzione	Rilasci continui, cultura DevOps, forte automazione
Serve	Processo chiaro e disciplina sui merge	Test automatici solidi, feature flag
Rischio principale	Conflitti di merge grandi e complessi	Rischio di “rompere” <code>main</code> se manca automazione

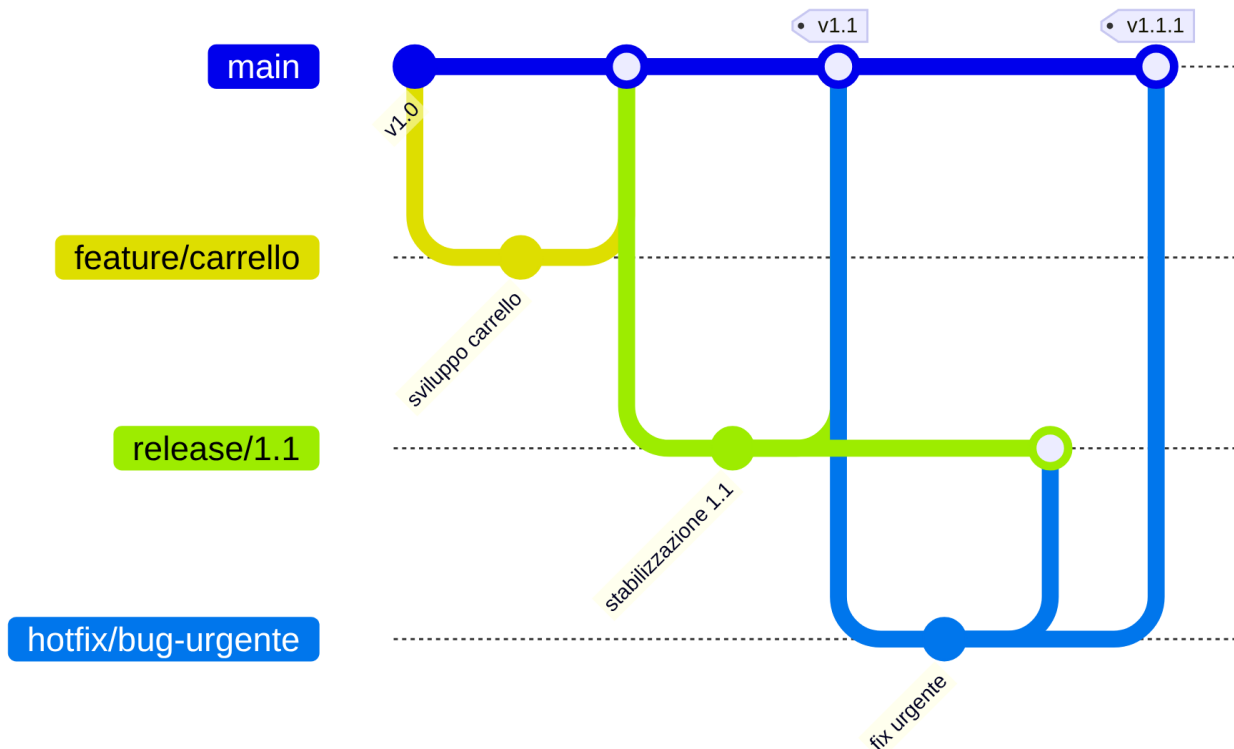
Non esiste un modello “migliore in assoluto”: la scelta dipende dalla maturità del team, dal tipo di prodotto e dalla frequenza di rilascio desiderata. Molti team reali usano versioni semplificate o ibride di questi due modelli, adattandole al proprio contesto — e la via di mezzo più diffusa ha un nome proprio, che vale la pena conoscere prima di incontrarlo in una riunione.

4.13 Il modello “cactus” (OneFlow): una via di mezzo

Git Flow, con `main` e `develop` sempre vivi in parallelo, raddoppia i punti in cui il codice può divergere: ogni feature branch deve tenersi allineato con `develop`, e `develop` a sua volta con `main` a ogni release. Trunk Based Development risolve il problema riducendo tutto a un solo branch di lunga

vita, ma **richiede** una suite di test automatici molto solida e feature flag maturi — condizioni che non tutti i team hanno il primo giorno. Nella pratica, molte aziende non stanno né perfettamente nell'uno né nell'altro: stanno in mezzo.

Il modello che descrive questa via di mezzo è conosciuto informalmente come **modello “cactus”**, e anche come **OneFlow**. L'idea: un solo tronco (**main**) da cui partono ed a cui tornano rami corti di feature — come in Trunk Based — ma da cui, quando serve preparare o mantenere una release, si staccano anche rami di release più longevi, che a loro volta si biforcano solo per le correzioni urgenti (hotfix). Visto nel suo insieme, con il tronco centrale e questi rami corti che si irradiano ai lati, la forma ricorda vagamente un cactus — da cui il nome.



A differenza di Git Flow, però, non esiste un branch **develop** permanente: le feature vanno dirette su **main** (o quasi), e i rami di release sono temporanei, legati a una specifica versione da mantenere, non un secondo tronco parallelo da tenere sincronizzato per sempre.

Un avvertimento pratico più importante del nome stesso: a differenza di “Git Flow”, la terminologia del modello cactus **non è standardizzata**. Team diversi, quando dicono “usiamo un modello cactus” (o “OneFlow”), possono intendere varianti anche sensibilmente diverse tra loro. Se un collega te lo dice in una riunione, la cosa giusta da fare non è annuire come se il nome bastasse a spiegare tutto: è chiedere di disegnarlo sulla lavagna, per capire davvero quali branch vivono a lungo e quali sono temporanei nel progetto specifico su cui lavori.

	Git Flow	Cactus (OneFlow)	Trunk Based
Branch di lunga vita	Due (<code>main</code> , <code>develop</code>)	Uno (<code>main</code>), più i rami di release finché sono mantenuti	Uno solo (<code>main</code>)
Frequenza di rilascio	Pianificata, es. mensile	Flessibile: rilasci frequenti dal tronco, release più stabili quando serve	Continua, più volte al giorno
Correzioni urgenti	Branch <code>hotfix/...</code> da <code>main</code> , poi riportato anche in <code>develop</code>	Branch <code>hotfix/...</code> dal ramo di release interessato, poi riportato su <code>main</code>	Fix diretto su <code>main</code> , spesso dietro feature flag
Requisiti di test automatici	Moderati: la fase di stabilizzazione del branch <code>release/...</code> compensa test meno maturi	Medio-alti: servono buoni test, ma il ramo di release dà un margine di sicurezza in più	Molto alti: senza test solidi, ogni commit su <code>main</code> è un rischio
Contesto in cui conviene	Rilasci pianificati, più versioni in produzione contemporaneamente	Prodotto con rilasci abbastanza frequenti ma che deve anche mantenere versioni precedenti (es. per clienti enterprise)	Prodotto SaaS con un'unica versione in produzione, cultura DevOps matura

4.14 Riepilogo dei comandi visti in questa sezione

Abbiamo visto molti comandi sparsi nei vari esempi di questa sezione: raccogliamoli qui in un unico prontuario, utile come riferimento veloce quando ti troverai a seguire una conversazione tecnica del team.

Comando	Cosa fa
<code>git init</code>	crea un nuovo repository
<code>git clone <url></code>	scarica un repository esistente
<code>git status</code>	mostra i file modificati
<code>git add <file></code>	prepara le modifiche per il commit
<code>git commit -m "messaggio"</code>	salva uno "scatto" delle modifiche
<code>git branch <nome></code>	crea un nuovo branch
<code>git checkout <nome></code>	mi sposto su un branch
<code>git checkout -b <nome></code>	crea e mi sposto su un nuovo branch
<code>git merge <branch></code>	unisce un branch in quello corrente
<code>git tag -a <nome> -m "messaggio"</code>	crea un tag annotato
<code>git push origin <branch\ tag></code>	invia branch/tag al repository remoto

Ciò che conta, nel tuo ruolo, è capire **cosa rappresentano concettualmente** (uno scatto, un ramo, un'unione) per poter seguire con consapevolezza le conversazioni tecniche del team, leggere lo stato di una Pull Request, e capire perché il team ha scelto un certo workflow.



Esercizi pratici

Gli esercizi che seguono ti servono a consolidare i concetti visti in questa sezione con le mani sulla tastiera. Non serve nessun progetto reale: puoi farli tutti con un account GitHub gratuito e un repository di prova che poi puoi anche eliminare.

1. **Crea un repository e fai il tuo primo commit.** Crea un repository di prova su GitHub (pubblico o privato, non importa), clonalo sul tuo computer con `git clone`, crea un file `note.txt` con dentro una riga di testo a piacere, poi lancia `git add note.txt` e `git commit -m "Primo commit di prova"`. **Come verificare:** lanciando `git log`, deve apparire il tuo commit con il messaggio che hai scritto e un identificativo esadecimale univoco.
2. **Crea un branch, modificalo e uniscilo.** Nel repository di prova, crea un branch con `git checkout -b feature/prova`, modifica `note.txt` aggiungendo una seconda riga, fai un commit, poi torna su `main` con `git checkout main` e lancia `git merge feature/prova`. **Come verificare:** dopo il merge, `note.txt` su `main` deve contenere anche la riga che hai aggiunto sul branch, e `git log` deve mostrare il commit del branch unito nella storia di `main`.
3. **Genera un conflitto di merge e risolvi.** Sempre su `main`, modifica la prima riga di `note.txt` e fai un commit. Poi crea un nuovo branch da un punto *precedente* a quella modifica (o modifica la stessa riga su un secondo branch prima del merge) e prova a fare il merge: dovresti ottenere un `CONFLICT (content)`. Apri il file, scegli quale testo mantenere, rimuovi i

marcatori `<<<<<<`, `=====`, `>>>>>>` che Git inserisce, e completa il merge con `git add note.txt` e `git commit`.  **Come verificare:** `git status` non deve più segnalare conflitti, e `note.txt` deve contenere solo il testo finale che hai scelto, senza marcatori residui.

4. **Apri una Pull Request fittizia su GitHub.** Fai il push del branch `feature/prova` (o uno nuovo) con `git push origin feature/prova`, poi su GitHub apri una Pull Request verso `main`, scrivendo un titolo e una breve descrizione di cosa cambia. Se hai un amico o un collega disponibile, chiedigli di lasciarti un commento; altrimenti lasciatelo da solo per vedere come funziona l'interfaccia.  **Come verificare:** la Pull Request compare nella sezione "Pull requests" del repository con stato "Open", e mostra correttamente i file modificati nel tab "Files changed".
5. **Crea un tag e una Release.** Sul repository di prova, crea un tag annotato con `git tag -a v0.1.0 -m "Prima versione di prova"`, invialo con `git push origin v0.1.0`, poi su GitHub vai nella sezione "Releases" e pubblica una Release basata su quel tag, con qualche nota di rilascio scritta da te.  **Come verificare:** il tag `v0.1.0` compare lanciando `git tag`, e la Release compare nella pagina "Releases" del repository con le note che hai scritto.
6. **Disegna il workflow del tuo progetto reale.** Chiedi a un collega developer quale workflow (Git Flow, Trunk Based Development, modello cactus/OneFlow, o una via di mezzo) usa il progetto su cui sei inserito/a, e provate insieme a disegnare — anche su un foglio — la sequenza di branch che si crea per una feature "tipica" del progetto, dalla creazione del branch fino al rilascio in produzione.  **Come verificare:** sai indicare, senza guardare gli appunti, a quale dei tre modelli visti in questa sezione il progetto si avvicina di più (o se è un ibrido), e sai spiegare perché quella scelta si adatta al ritmo di rilascio del progetto.
7. **Riconosci un `pom.xml`.** Chiedi a un developer del progetto (se il backend è Java) di farti vedere il file `pom.xml` reale, oppure cercane uno di esempio online. Individua senza aiuto: il nome e la versione del progetto, almeno una dipendenza dichiarata con la sua versione.  **Come verificare:** sai indicare a un collega, guardando il file, quale libreria esterna il progetto usa e quale versione — senza dover capire una riga di XML in più del necessario.

Collegamenti

- [3. Come nasce un software](#) — dove si inserisce Git nel ciclo di vita di un progetto software, e dove trovi gli ADR per documentare le decisioni tecniche
- [5. Agile](#) — il mindset dietro le pratiche di integrazione frequente che abbiamo visto con il Trunk Based Development
- [9. DevOps](#) — la cultura di automazione e rilascio continuo che rende possibile il Trunk Based Development
- [10. CI/CD](#) — le pipeline automatiche (GitHub Actions) che si attivano su commit e Pull Request, e dove il ciclo di build di Maven si collega alla generazione dell'artifact
- [13. Sicurezza](#) — dove si approfondisce come le dipendenze (comprese quelle transitive viste in questa sezione) possono introdurre vulnerabilità note
- [16. Glossario](#) — per ripassare velocemente i termini di questa sezione

- [Documentazione ufficiale di Git](#)
- [Git — Book \(Pro Git, gratuito e in italiano\)](#)
- [GitHub Docs](#)
- [Learn Git Branching \(esercizi interattivi nel browser\)](#)
- [GitHub Docs — Get started](#)
- [About pull requests — GitHub Docs](#)
- [Conventional Commits](#)
- [Semantic Versioning](#)
- [Trunk Based Development](#)
- [OneFlow — a Git branching model and workflow](#) — la descrizione originale del modello “cactus”
- [Apache Maven — Guida introduttiva](#)
- [A successful Git branching model \(Git Flow, Vincent Driessen\)](#)

5. Agile

Nelle sezioni precedenti hai visto come nasce un software (requisiti, analisi, sviluppo, testing, rilascio, manutenzione) e come Git e GitHub permettano a più persone di lavorare insieme sullo stesso codice. Ora facciamo un passo diverso, ma altrettanto importante: non uno strumento, non una tecnologia, ma un **modo di pensare** e di organizzare il lavoro.

Si chiama **Agile**, e se lavori (o lavorerai) in un team di sviluppo software, è quasi certo che lo sentirai nominare fin dal primo giorno: “il nostro team lavora in modo agile”, “facciamo Scrum”, “usiamo una board Kanban”. Questa sezione ti dà le basi per capire da dove viene questo termine, cosa significa davvero, e perché ha cambiato il modo in cui si costruisce il software negli ultimi 20 anni.

Per rendere i concetti meno astratti, in questa sezione e nelle prossime due (Scrum e Kanban) useremo sempre lo stesso progetto di riferimento: **ShopFacile**, una piattaforma e-commerce (catalogo prodotti, carrello, ordini, pagamenti, sconti e promozioni) sviluppata da un piccolo team interno. Il team è composto da **Marco** e **Giulia** (developer, lei molto attenta a test e qualità), **Ahmed** (developer junior, in crescita), **Sara** (Product Owner, si occupa di business e priorità) e **Luca** (Scrum Master, facilita il lavoro del team). Li ritroverai in quasi tutti gli esempi pratici che seguono.

Obiettivi della sezione

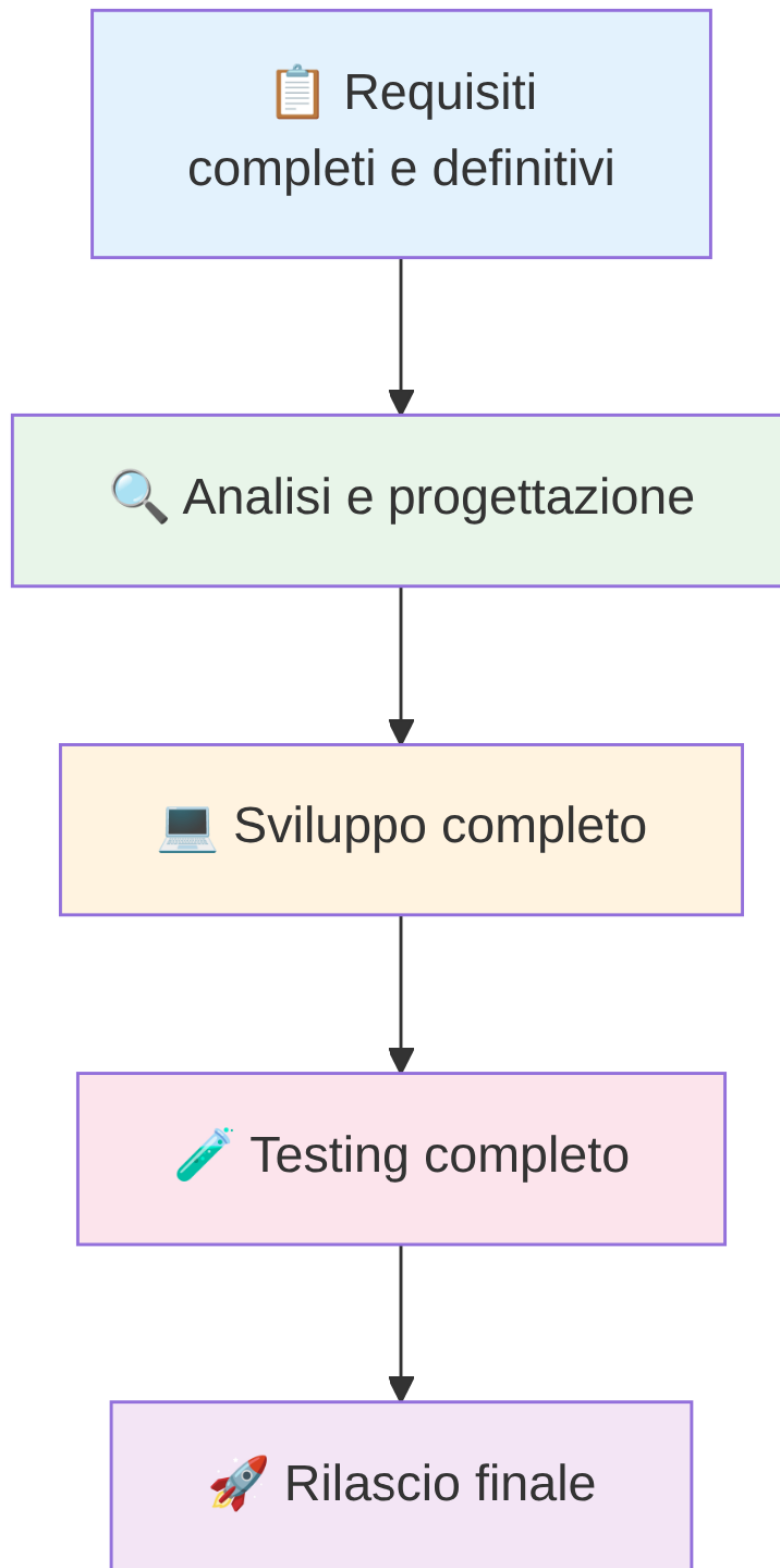
Alla fine di questa sezione saprai:

- spiegare perché è nato l'Agile e da cosa si differenzia rispetto ai modelli precedenti;
- elencare e spiegare con parole tue i 4 valori del Manifesto Agile;
- riconoscere i temi principali dei 12 principi Agile, senza doverli imparare a memoria;
- descrivere cosa significa avere un “mindset agile” nel lavoro quotidiano;
- distinguere concettualmente un approccio Waterfall da un approccio Agile;
- sapere che Scrum e Kanban sono due modi concreti (framework) per applicare l'Agile.

5.1 Perché nasce l'Agile: il problema del modello a “cascata”

Per capire perché è nato l'Agile, dobbiamo prima capire cosa c'era **prima**.

Per decenni, il modo più comune di gestire un progetto software è stato il modello **Waterfall** (in italiano “a cascata”). L'idea è semplice: il progetto si divide in fasi rigide, una dopo l'altra, e si passa alla fase successiva solo quando quella precedente è **completamente** finita. Prima si raccolgono *tutti* i requisiti, poi si fa *tutta* l'analisi, poi si scrive *tutto* il codice, poi si testa *tutto*, e solo alla fine si rilascia il prodotto finito.



💡 **Analogia:** il Waterfall è come costruire una casa seguendo un progetto architettonico immutabile, firmato e bloccato fin dal primo giorno. Solo alla fine dei lavori, dopo mesi o anni, la famiglia entra per la prima volta nella casa finita — e solo in quel momento scopre se le misure della cucina erano davvero quelle giuste per le sue esigenze.

Immagina se il team di ShopFacile avesse dovuto raccogliere e bloccare *tutti* i requisiti su catalogo, carrello e pagamenti prima di scrivere una sola riga di codice: qualsiasi promozione lanciata da un concorrente nel frattempo avrebbe reso il progetto già in ritardo sul mercato ancora prima del rilascio.

Il problema di questo approccio è diventato via via più evidente, soprattutto a partire dagli anni '90, quando il software ha iniziato a diventare sempre più complesso e i mercati sempre più rapidi a cambiare:

- **Il cliente vede il risultato troppo tardi.** Se il progetto dura un anno, il cliente scopre il prodotto finito dopo un anno — e nel frattempo le sue esigenze possono essere cambiate, o si accorge che aveva descritto male ciò di cui aveva davvero bisogno.
- **Cambiare idea a metà strada è costosissimo.** In un modello a cascata, tornare indietro (es. modificare un requisito durante lo sviluppo) significa spesso rifare da capo intere fasi già “chiuse”.
- **I rischi si scoprono tardi.** Un problema tecnico grave, o un errore nei requisiti iniziali, viene spesso a galla solo nella fase di testing o, peggio, dopo il rilascio — quando correggerlo costa molto di più che se fosse stato scoperto all'inizio.

Negli anni '90 e nei primi anni 2000, diversi gruppi di sviluppatori (indipendenti tra loro) iniziarono a sperimentare approcci alternativi, più flessibili e incentrati sul consegnare valore al cliente in modo rapido e frequente, piuttosto che pianificare tutto in anticipo nei minimi dettagli. Nacquero così metodi con nomi come Scrum, Extreme Programming (XP), Crystal, e altri — tutti con filosofie simili ma pratiche diverse.

Nel **febbraio 2001**, 17 di questi sviluppatori si incontrarono in una località di montagna (Snowbird, Utah, USA) per discutere cosa avessero in comune i loro approcci. Da quell'incontro nacque un documento breve e diretto: il **Manifesto per lo Sviluppo Agile di Software** (*Manifesto Agile*). Non era un metodo, non era una checklist di regole: era un insieme di **valori e principi condivisi**, un vero e proprio cambio di mentalità. Vediamo subito questi 4 valori nel concreto, applicandoli al team di ShopFacile che ci accompagnerà per il resto della sezione.

5.2 Il Manifesto Agile: i 4 valori fondamentali

Il Manifesto Agile è composto, prima di tutto, da **4 valori**. Sono espressi con una struttura particolare: “preferiamo A rispetto a B”, il che **non significa che B non conti**, ma che, quando bisogna scegliere, A ha la priorità.

💡 Testo originale (in inglese) di riferimento: agilemanifesto.org — esiste anche la versione ufficiale in italiano.

1. Persone e interazioni, più che processi e strumenti

Non significa “i processi e gli strumenti non servono” — Git, Jira, le pipeline di CI/CD sono tutti strumenti utilissimi che vedrai in questo corso. Significa che uno strumento perfetto usato da un team che non comunica bene produce risultati peggiori di un team che comunica bene con strumenti semplici.

Esempio pratico: quando il carrello di ShopFacile mostra un totale sbagliato dopo l'applicazione di un codice sconto, Marco e Giulia preferiscono chiarirsi con una videochiamata di 10 minuti piuttosto che scambiarsi 15 email formali su un sistema di ticket, sperando che il messaggio arrivi comunque chiaro.

2. Software funzionante, più che documentazione esaustiva

Non significa “non scriviamo documentazione” — la documentazione utile (come questo stesso corso!) ha valore. Significa che l'obiettivo primario di un progetto software è **produrre qualcosa che funziona e che le persone possono usare**, non produrre migliaia di pagine di specifiche che nessuno leggerà mai o che diventeranno obsolete in due settimane.

Esempio pratico: a metà di uno sprint, il team di ShopFacile può scegliere di dedicare tempo a completare il flusso di checkout che Sara può già provare sul carrello, invece di scrivere un documento di 40 pagine che descrive nel dettaglio come funzionerà quel flusso.

3. Collaborazione con il cliente, più che negoziazione dei contratti

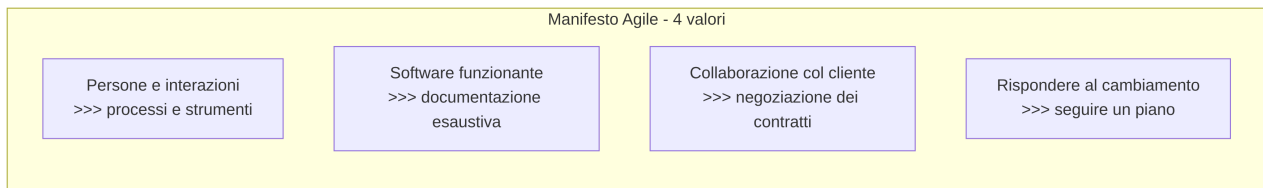
Non significa “i contratti non servono” — restano necessari, soprattutto in ambito aziendale. Significa che il rapporto con il cliente non dovrebbe limitarsi a “firmiamo un contratto all'inizio e ci rivediamo alla consegna finale”, ma dovrebbe essere una **collaborazione continua**, con feedback frequenti durante tutto il progetto.

Esempio pratico: invece di consegnare il modulo pagamenti di ShopFacile finito dopo 6 mesi e scoprire solo allora che “non era esattamente quello che intendevamo”, il team mostra a Sara (e al cliente, tramite lei) una versione parziale ma funzionante del carrello ogni 2 settimane, raccogliendo feedback che permettono di correggere la direzione strada per strada.

4. Rispondere al cambiamento, più che seguire un piano

Non significa “non pianifichiamo nulla” — pianificare resta fondamentale, e lo vedrai bene nella sezione sul Project Management. Significa che un piano fatto a inizio progetto non può prevedere tutto, e quando emergono nuove informazioni (il mercato cambia, il cliente capisce meglio cosa vuole, si scopre un vincolo tecnico imprevisto), è più intelligente **adattare il piano** che seguirlo alla lettera solo perché “era scritto così”.

Esempio pratico: durante lo sviluppo, un concorrente lancia una funzionalità di sconti aggressivi che cambia le priorità del mercato. Sara può rivedere il backlog di ShopFacile (l'elenco delle attività da fare) e riorganizzare le priorità già alla prossima iterazione, invece di aspettare la fine di un piano annuale immutabile.



5.3 I 12 principi Agile: raggruppati per temi

I 4 valori che abbiamo appena visto restano un po' astratti se non si traducono in comportamenti concreti: è esattamente questo che fanno i **12 principi** più operativi elencati nel Manifesto. Non serve impararli a memoria uno per uno: è molto più utile raggrupparli per temi, perché in fondo raccontano poche idee ripetute con angolazioni diverse.

Tema 1 — Soddisfazione del cliente al centro

I principi Agile insistono sulla **consegna frequente** di valore reale al cliente, preferendo cicli di poche settimane piuttosto che consegne rare e lontane nel tempo. La priorità più alta è soddisfare il cliente attraverso consegne di valore continue e anticipate.

Esempio pratico: il team di ShopFacile non aspetta di avere “tutto il sito perfetto” per farlo vedere a Sara e al cliente. Consegna prima la funzionalità di ricerca prodotti, poi quella del carrello, poi quella dei pagamenti — ogni pezzo utilizzabile e verificabile appena pronto.

Tema 2 — Collaborazione quotidiana tra le persone coinvolte

I principi sottolineano che le persone di business (chi rappresenta gli interessi del cliente/progetto) e gli sviluppatori devono lavorare **insieme ogni giorno**, non in compartimenti stagni che si parlano solo a inizio e fine progetto. Si valorizzano anche gli incontri di persona (o in videochiamata) come modo più efficace di comunicare rispetto a lunghe catene di documenti scritti.

Esempio pratico: è per questo motivo che la maggior parte dei team Agile fa un breve incontro giornaliero (il “daily standup”, che vedrai nella sezione su Scrum): 15 minuti in cui ognuno racconta cosa ha fatto, cosa farà, e se ha ostacoli — invece di scoprire un problema solo a fine settimana leggendo un report.

Tema 3 — Adattamento al cambiamento

Un principio fondamentale afferma che il cambiamento dei requisiti va **accolto con favore**, anche a stadi avanzati dello sviluppo, perché i processi agili permettono di sfruttarlo a vantaggio del cliente, offrendo un vantaggio competitivo. Questo è un ribaltamento radicale rispetto alla mentalità Waterfall, dove il cambiamento era visto come un problema da evitare.

Esempio pratico: se durante il progetto il cliente si rende conto che una funzionalità pianificata non serve più, e ne serve invece un'altra più urgente (come i famosi sconti aggressivi del concorrente visti in 5.1), il team di ShopFacile riorganizza le priorità del proprio lavoro futuro senza dover “riaprire” un intero progetto formalmente chiuso.

Tema 4 — Semplicità ed efficienza

Un principio recita che “la semplicità — l'arte di massimizzare la quantità di lavoro non svolto — è essenziale”. In altre parole: fare solo il lavoro necessario per raggiungere l'obiettivo, evitando di costruire funzionalità complesse “che potrebbero servire un giorno” ma che nessuno ha davvero richiesto.

Esempio pratico: se un cliente chiede a ShopFacile un modulo per esportare in PDF lo storico dei propri ordini, il team costruisce prima la funzione più semplice che risolve il problema, invece di progettare da subito un sistema di esportazione universale con 20 formati diversi che nessuno userà.

Tema 5 — Team auto-organizzati e motivati

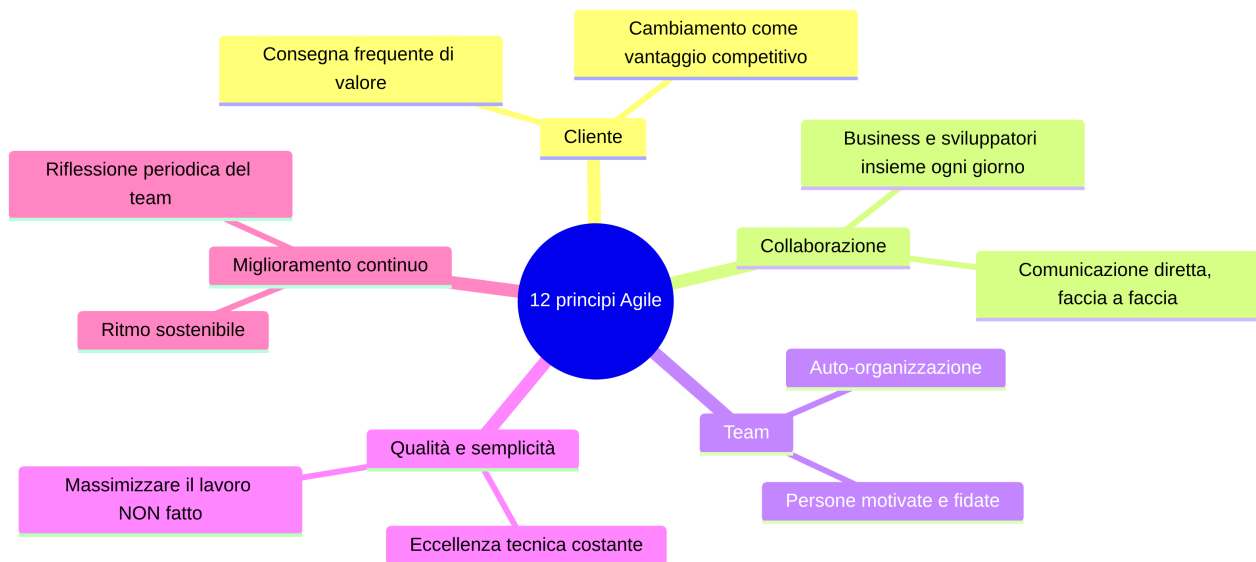
I principi insistono sul costruire progetti attorno a **persone motivate**, dando loro l'ambiente e il supporto di cui hanno bisogno, e fidandosi che sapranno portare a termine il lavoro. Le architetture, i requisiti e i progetti migliori emergono da **team che si auto-organizzano**, cioè che decidono internamente come dividersi il lavoro, piuttosto che ricevere ogni compito imposto dall'alto nei minimi dettagli.

Esempio pratico: invece che Luca assegni manualmente ogni singolo task a Marco, Giulia e Ahmed, il team stesso, durante la pianificazione dello sprint, decide chi si occuperà di cosa in base a competenze e disponibilità — Luca facilita il processo, non lo comanda dall'alto.

Tema 6 — Miglioramento continuo

Un ultimo tema, altrettanto importante: a intervalli regolari, il team deve **riflettere su come diventare più efficace**, per poi mettere in pratica quanto imparato. Questo principio è il seme di quella che nella sezione su Scrum chiamerai “retrospettiva”: una pratica dedicata proprio a imparare dagli errori e dai successi, iterazione dopo iterazione.

Esempio pratico: alla fine di ogni sprint di ShopFacile (ad esempio ogni 2 settimane), Luca facilita una retrospettiva di 30-60 minuti in cui il team si chiede: cosa è andato bene? Cosa possiamo migliorare? Il team prova concretamente a cambiare qualcosa nel ciclo successivo, senza aspettare la fine del progetto per correggere le proprie abitudini.



5.4 Il “mindset” Agile: più di un metodo, un modo di pensare

Valori e principi sono scritti su carta, ma conoscerli a memoria non basta: quello che conta davvero è come si comportano le persone quando nessuno controlla la checklist. Una delle cose più difficili da capire per chi arriva da fuori (te compreso, oggi) è che **Agile non è una checklist di regole da seguire**. È soprattutto un **mindset**, cioè un atteggiamento mentale che guida le decisioni quotidiane del team.

Cosa significa davvero avere un mindset agile, in pratica?

- **Accettare che il piano iniziale non sarà perfetto, e va bene così.** Un team agile non cerca di prevedere ogni dettaglio a inizio progetto: sa che imparerà cose nuove strada facendo, e costruisce processi che permettono di **adattarsi** invece che resistere al cambiamento.
- **Lavorare per piccoli passi verificabili (iterare).** Invece di lavorare per mesi su qualcosa senza mai mostrarlo a nessuno, il team consegna piccoli pezzi funzionanti spesso, verifica se sono nella direzione giusta, e corregge la rotta rapidamente se serve.
- **Collaborare in modo trasparente, anche quando qualcosa va storto.** Un mindset agile preferisce dire presto “questa cosa non funziona come pensavamo” piuttosto che nascondere fino alla scadenza finale.
- **Puntare all'adattamento continuo, non alla perfezione del piano.** Non è un fallimento se il piano cambia: è un segnale che il team sta imparando e reagendo bene alla realtà, invece di ignorarla per “restare fedele” a un documento scritto mesi prima.
- **Concentrarsi sul valore per l'utente/cliente, non sulla semplice esecuzione di task.** La domanda guida non è solo “abbiamo completato l'attività X?”, ma “questa attività ha davvero portato valore al cliente?”.



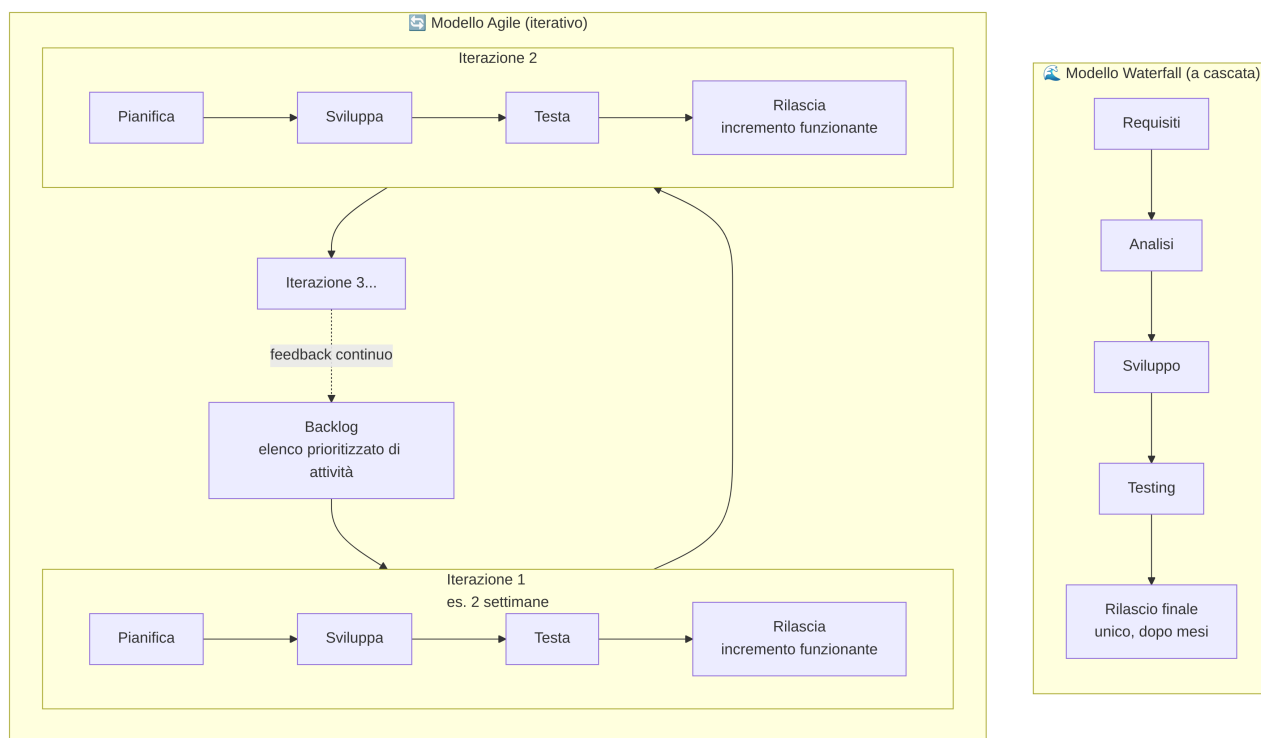
Analogia: pensa alla differenza tra seguire una ricetta di cucina alla lettera, ignorando che gli ospiti sono arrivati con un'ora di ritardo e i tempi di cottura non funzionano più, oppure adattarsi in corsa — magari cambiando l'ordine dei piatti — mantenendo l'obiettivo (una cena buona e godibile) più importante del rispetto rigido della procedura scritta. Il mindset agile è proprio questo: l'obiettivo (valore per il cliente) conta più della fedeltà al piano originale.

Esempio pratico: due team lavorano entrambi con “sprint di 2 settimane” e “daily standup” — le stesse pratiche, sulla carta identiche. Nel primo, quando a metà sprint emerge un problema imprevisto, il Product Owner e il team si fermano, ne discutono e ridefiniscono insieme le priorità. Nel secondo, il piano dello sprint resta “intoccabile” fino alla fine, anche quando è ormai chiaro che non porterà valore reale: si segue il rituale alla lettera, ma si è perso il mindset. Solo il primo dei due sta davvero applicando l'Agile — il secondo fa solo “teatro Agile”.

Un errore comune, anche in aziende che si definiscono “agili”, è applicare le **pratiche** (i daily standup, le board Kanban, gli sprint) senza davvero adottare il **mindset** sottostante. Il risultato è quello che nel settore si chiama scherzosamente “fare Agile senza essere agili”: si seguono i rituali nella forma, ma le decisioni continuano a essere prese come in un progetto rigido e tradizionale. Il tuo compito, da futuro Project Manager, sarà anche capire questa differenza e favorire davvero il mindset, non solo il rituale.

5.5 Waterfall vs Agile: il confronto visivo

Dopo aver visto valori, principi e mindset uno per uno, è utile fare un passo indietro e guardare il quadro d'insieme, confrontando direttamente il modello Agile con quello Waterfall visto nella sezione 5.1. Riassumiamo visivamente la differenza principale tra i due approcci: il Waterfall procede in linea retta, fase dopo fase, con un unico rilascio alla fine; l'Agile procede per **cicli brevi e ripetuti** (spesso chiamati iterazioni o sprint), ognuno dei quali produce qualcosa di utilizzabile.



Alcune differenze chiave riassunte in tabella:

Aspetto	Waterfall	Agile
Struttura	Fasi sequenziali, una alla volta	Cicli brevi e ripetuti (iterazioni)
Quando si vede il risultato	Solo alla fine del progetto	Ad ogni iterazione (es. ogni 2 settimane)
Gestione del cambiamento	Difficile e costosa a metà progetto	Prevista e “benvenuta” come opportunità
Coinvolgimento del cliente	Soprattutto a inizio e fine progetto	Continuo, durante tutto il progetto
Documentazione	Spesso estesa e dettagliata a priori	Leggera, quanto basta, aggiornata nel tempo
Rischio scoperto	Tardi (in fase di testing o dopo il rilascio)	Presto, a ogni iterazione
Adatto a	Progetti con requisiti molto stabili e ben noti (es. costruzioni fisiche, normative rigide)	Progetti con requisiti che possono evolvere, mercati dinamici, software

Non significa che il Waterfall sia “sbagliato” in assoluto: in contesti dove i requisiti sono davvero stabili e immutabili (pensa a certi progetti di ingegneria civile con vincoli normativi fissi), un approccio più sequenziale può avere senso. Ma nella maggior parte dei progetti software moderni — dove i requisiti evolvono, il mercato cambia rapidamente e il feedback degli utenti è preziosissimo — l’approccio Agile si è dimostrato, negli ultimi 20 anni, molto più efficace.

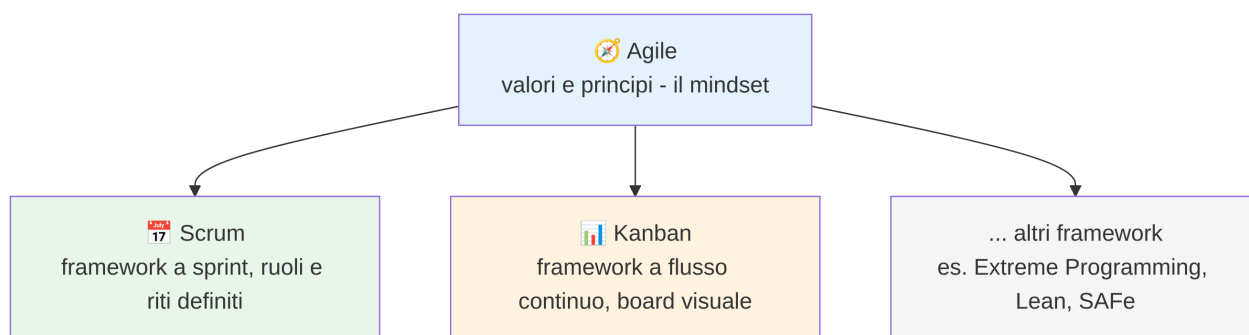
Vale la pena essere altrettanto onesti sui limiti dell'Agile, perché come futuro Project Manager li incontrerai davvero, non solo sui libri. Ci sono contesti in cui un impianto rigoroso di documentazione e pianificazione anticipata non è una preferenza di metodo, ma un obbligo: un contratto **a prezzo e perimetro fissi** (fixed price, fixed scope), firmato prima di iniziare, lascia poco spazio ad “adattarsi durante il progetto” se il cliente vuole sapere fin da subito esattamente cosa riceverà e quanto pagherà. Allo stesso modo, settori **regolamentati** — farmaceutico, bancario, aerospaziale — impongono per legge una documentazione formale e tracciabile di ogni fase, non perché sia “più prudente”, ma perché un'autorità di controllo esterna la richiede esplicitamente, a prescindere da quanto il team sia bravo a lavorare in modo agile. Infine, l'Agile nasce pensato per un **singolo team**: quando un progetto richiede di coordinare venti o trenta persone su più team in parallelo, le pratiche che funzionano bene per 7-8 persone (un daily di 15 minuti, una board condivisa, una retrospettiva) non si scalano da sole, e serve un livello di coordinamento aggiuntivo che l'Agile “puro” non descrive — è proprio il problema che vedrai tra poco a proposito di SAFe.

5.6 Dall'Agile ai framework concreti: Scrum e Kanban

Sappiamo ora perché l'Agile è nato e cosa lo distingue dal Waterfall: resta da capire come il team di ShopFacile applica concretamente questi valori nel lavoro di ogni giorno. Un punto importante da chiarire prima di procedere: **Agile non è un metodo con regole precise da seguire passo passo**. È un insieme di valori e principi — quello che abbiamo visto finora. Per applicarlo concretamente nel lavoro di tutti i giorni, negli anni sono nati diversi **framework** (cornici operative, con ruoli, riti e artefatti specifici) che *implementano* la filosofia Agile in modo pratico.

I due framework più diffusi nel mondo dello sviluppo software — e quelli che approfondirai nelle prossime due sezioni di questo corso — sono:

- **Scrum**: un framework strutturato, basato su cicli di lavoro a tempo fisso chiamati **sprint** (tipicamente 1-4 settimane), con ruoli precisi (Scrum Master, Product Owner, Development Team) e riti definiti (sprint planning, daily standup, sprint review, retrospettiva).
- **Kanban**: un framework più fluido, basato su un flusso continuo di lavoro visualizzato su una **board** (bacheca) con colonne come “Da fare”, “In corso”, “Fatto”, con un forte focus sulla limitazione del lavoro in corso (*Work In Progress*) per evitare colli di bottiglia.



Non sono in competizione tra loro in modo netto: molti team, in realtà, usano versioni ibride (a volte chiamate “Scrumban”) che mescolano elementi di entrambi. Ciò che conta è che **entrambi nascono per applicare gli stessi valori Agile** che hai visto in questa sezione: consegnare valore frequentemente, adattarsi al cambiamento, collaborare in modo trasparente, migliorare continuamente.

Nello schema qui sopra hai notato una terza voce, **SAFe** (Scaled Agile Framework), accennata solo come “altro framework”. Merita due righe di spiegazione perché nasce da un problema molto concreto: Scrum e Kanban funzionano bene per un singolo team di poche persone, ma non dicono nulla su come far collaborare venti team che lavorano sullo stesso prodotto. SAFe è uno dei tentativi più diffusi di rispondere a questa domanda, aggiungendo livelli di pianificazione e coordinamento sopra i singoli team Scrum/Kanban. È anche, va detto, uno dei framework più discussi: molti professionisti Agile lo criticano proprio perché la sua struttura, con ruoli e riti aggiuntivi a livello di programma e di portafoglio, rischia di reintrodurre la burocrazia e la rigidità che l'Agile era nato per superare.

Esempio pratico: un team che sviluppa un nuovo prodotto da zero, con requisiti che cambiano spesso e la necessità di pianificare con un certo anticipo il lavoro delle prossime due settimane, troverà probabilmente più utile Scrum, con la sua cadenza regolare a sprint. Un team che si occupa soprattutto di supporto e correzione di problemi — richieste che arrivano in modo imprevedibile, una alla volta, senza un ritmo fisso — troverà invece più naturale Kanban, che non impone di pianificare in blocco cosa fare nelle prossime due settimane.

Prima di passare alla pratica, vale la pena spendere due righe anche sulle critiche più recenti che vengono fatte all'Agile, perché come Project Manager dovrai saperle riconoscere per non subirle senza accorgertene. La prima è il rischio di **sprint consecutivi senza respiro**: un team che consegna ogni due settimane, per anni, senza mai una pausa strutturale tra un ciclo di lavoro e il successivo, può arrivare a un ritmo insostenibile — l'Agile parla esplicitamente di “ritmo sostenibile” (l'hai visto nel Tema 6 di 5.3), ma nella pratica quotidiana questo principio è spesso il primo a essere sacrificato quando le scadenze si accumulano. La seconda critica riguarda un uso distorto e comodo della parola stessa: “siamo agili” viene talvolta usato come alibi per non pianificare mai nulla oltre le prossime due settimane, anche quando un minimo di visione a medio termine (che vedrai nella sezione sul Project Management) sarebbe utile sia al team che al cliente. L'Agile ben applicato non è un rifiuto della pianificazione: è pianificare in modo diverso, non smettere di pianificare.

Nelle prossime due sezioni vedrai nel dettaglio come funzionano davvero, con ruoli, artefatti, riti e strumenti concreti che userai (o osserverai) nel tuo lavoro quotidiano da Junior Project Manager.



Esercizi pratici

Gli esercizi che seguono ti aiutano a consolidare i concetti *teorici* di questa sezione (valori, principi, mindset, Waterfall vs Agile, scelta del framework) prima di passare alla pratica operativa di Scrum e Kanban, che approfondirai nelle prossime due sezioni.

1. **Riscrivi i 4 valori del Manifesto con un esempio tuo.** Per ciascuno dei 4 valori (5.2), scrivi un esempio concreto diverso da quelli di questa pagina — puoi prenderlo dal progetto che osservi, dalla vita universitaria o anche da un'esperienza personale (es. organizzare un evento con altre

persone).  **Come verificare:** fatti leggere i tuoi 4 esempi da un collega o dalla tua collega Scrum Master/PM e chiedi se colgono davvero il “preferiamo A rispetto a B, ma B resta importante” — non un aut-aut assoluto.

2. **Scegli 3 principi su 12 e trova un esempio “sul campo”.** Tra i 6 temi visti in 5.3, scegline 3 e osserva (o chiedi a un collega di raccontarti) un episodio reale, anche piccolo, del progetto che li illustra — diverso dagli esempi di questa pagina.  **Come verificare:** riesci a raccontare a voce, in meno di un minuto per principio e senza guardare gli appunti, sia il principio che l'esempio reale collegato?
3. **Costruisci la tua tabella Waterfall vs Agile su un caso non software.** Prendi un progetto qualsiasi che conosci bene, anche non informatico (organizzare un trasferimento, pianificare un evento, scrivere una tesi) e compila una mini-tabella con almeno 3 righe della tabella di 5.5, valutando se in quel caso specifico converrebbe un approccio più Waterfall o più Agile.  **Come verificare:** per ogni riga hai scritto un motivo concreto (non solo “è più veloce” o “è più moderno”) che giustifica la scelta in quel contesto specifico?
4. **Riconosci il “teatro Agile”.** Scrivi 3-4 righe che descrivono una situazione (immaginaria o osservata) in cui un team segue le pratiche agili (sprint, daily, board) ma non il mindset — ispirandoti all'esempio pratico di 5.4 ma con un caso tuo.  **Come verificare:** la tua descrizione indica chiaramente sia *quale pratica* viene seguita sia *quale decisione concreta* rivela che il mindset agile in realtà manca.
5. **Scrum o Kanban? Decidi per due scenari diversi.** Immagina due contesti di lavoro molto diversi tra loro (ad esempio: un team che costruisce da zero una nuova funzionalità complessa vs un team che gestisce solo richieste di supporto che arrivano senza un ritmo prevedibile) e per ciascuno indica se propenderesti più per Scrum o per Kanban, motivando con almeno 2 argomenti presi da 5.6.  **Come verificare:** la tua motivazione fa riferimento esplicito alle caratteristiche di Scrum e Kanban descritte in questa sezione (cadenza fissa vs flusso continuo, pianificazione a blocchi vs limite di WIP), non solo a una preferenza personale.
6. **Prepara un “elevator pitch” dell'Agile.** Scrivi (o registra a voce) una spiegazione di massimo 5 frasi dell'Agile per una persona che non ha mai sentito questo termine, senza usare la parola “framework” e senza elencare i 12 principi uno per uno.  **Come verificare:** fatti ascoltare da qualcuno non tecnico (un amico, un familiare, un collega di un altro reparto) — se in meno di 2 minuti capisce la differenza principale rispetto al “fare tutto e consegnare solo alla fine”, il pitch funziona.

Collegamenti

- [6. Scrum](#) — il framework Agile più diffuso, con sprint, ruoli e riti concreti
- [7. Kanban](#) — il framework Agile basato su flusso continuo e board visuale
- [8. Project Management](#) — come il tuo ruolo di Project Manager si inserisce concretamente in un contesto Agile

- [Manifesto Agile \(versione ufficiale in italiano\)](#)
- [I 12 principi del Manifesto Agile \(italiano\)](#)
- [Atlassian — What is Agile?](#)
- [Scrum.org — What is Scrum?](#)
- [Atlassian — Agile vs Waterfall](#)
- [Project Management Institute — What is Agile?](#)

6. Scrum

Nella sezione precedente hai scoperto l'Agile come **filosofia**: un insieme di valori e principi (persone prima dei processi, collaborazione con il cliente, adattamento al cambiamento, consegne frequenti). Ma una filosofia, da sola, non ti dice cosa fare il lunedì mattina quando entri in ufficio.

Serve un **framework**, cioè un insieme di regole concrete, ruoli definiti e riunioni con un nome e uno scopo preciso, che traduca la filosofia Agile in pratica quotidiana. Il framework più usato al mondo per farlo si chiama **Scrum**, ed è quello che il progetto a cui sarai affiancato utilizza (o utilizzerà) ogni giorno.

Questa è la sezione più importante del corso per te, perché **Scrum Master** è il ruolo che andrai a occupare. Ogni concetto che leggerai qui non è teoria astratta: è il tuo futuro lavoro quotidiano.

 **Analogia iniziale:** se l'Agile è la filosofia “cucina leggera, con ingredienti freschi, adattandosi a quello che il mercato offre oggi”, Scrum è la **ricetta strutturata** che dice esattamente quando fare la spesa, quanti piatti preparare, con quale squadra in cucina e quando assaggiare per correggere il sapore prima di servire in tavola.

Per non restare sul piano astratto, in questa sezione seguiremo lo stesso progetto di riferimento delle altre sezioni del corso: **ShopFacile**, la piattaforma di e-commerce (catalogo prodotti, carrello, ordini, pagamenti, sconti) su cui lavora un piccolo team interno. È proprio in questa sezione che i membri di quel team assumono, per la prima volta in modo formale, i tre ruoli di Scrum: **Sara** è la Product Owner, **Luca** è lo Scrum Master — il ruolo che anche tu andrai a occupare — e **Marco, Giulia e Ahmed** compongono il team di sviluppo. Li ritroverai in ogni paragrafo che segue, mentre applicano concretamente Scrum al lavoro quotidiano su ShopFacile.

Obiettivi della sezione

Alla fine di questa sezione saprai:

- spiegare cos'è Scrum e perché è diventato il framework Agile più diffuso;
- distinguere i tre ruoli di Scrum (Product Owner, Scrum Master, Team di sviluppo) e in particolare il ruolo che occuperai tu;
- descrivere i cinque eventi di Scrum (Sprint, Sprint Planning, Daily Scrum, Sprint Review, Sprint Retrospective): durata, partecipanti, obiettivo;
- distinguere i tre artefatti (Product Backlog, Sprint Backlog, Increment);
- scrivere e riconoscere una User Story;
- capire cosa sono Story Point e Velocity e perché si usano al posto delle ore/giorni;
- applicare i concetti di Definition of Ready e Definition of Done;
- avere un quadro concreto delle attività quotidiane di uno Scrum Master.

6.1 Cos'è Scrum e perché è ovunque

Scrum è un framework leggero per gestire il lavoro di sviluppo di un prodotto in modo iterativo e incrementale. È stato formalizzato negli anni '90 da Ken Schwaber e Jeff Sutherland (due dei firmatari del Manifesto Agile che hai visto nella sezione precedente) e oggi è, di gran lunga, il framework Agile più utilizzato nel mondo del software — e sempre più anche fuori dal software (marketing, HR, persino organizzazione di eventi).

Il nome “Scrum” viene dal rugby: indica la **mischia**, la fase di gioco in cui l'intera squadra si compatta e avanza insieme, come un blocco unico, spingendo nella stessa direzione. Non è un caso: Scrum nasce proprio dall'idea che un progetto complesso si affronta meglio con una squadra compatta e auto-organizzata, piuttosto che con singoli individui che lavorano ognuno per proprio conto secondo un piano rigido stabilito a tavolino mesi prima.

Perché Scrum ha avuto così tanto successo? Alcuni motivi concreti:

- è **semplice da descrivere** (la Guida ufficiale a Scrum, la “Scrum Guide”, è lunga poche decine di pagine) ma **profondo da padroneggiare**;
- fornisce una **cadenza regolare** (lo Sprint) che dà al team e agli stakeholder un ritmo prevedibile di consegne e verifiche;
- rende **visibile** lo stato del lavoro, riducendo le sorprese a fine progetto;
- si adatta bene a contesti dove i requisiti **non sono chiari al 100% fin dall'inizio** — che, nella pratica, è quasi sempre il caso di un progetto software reale.

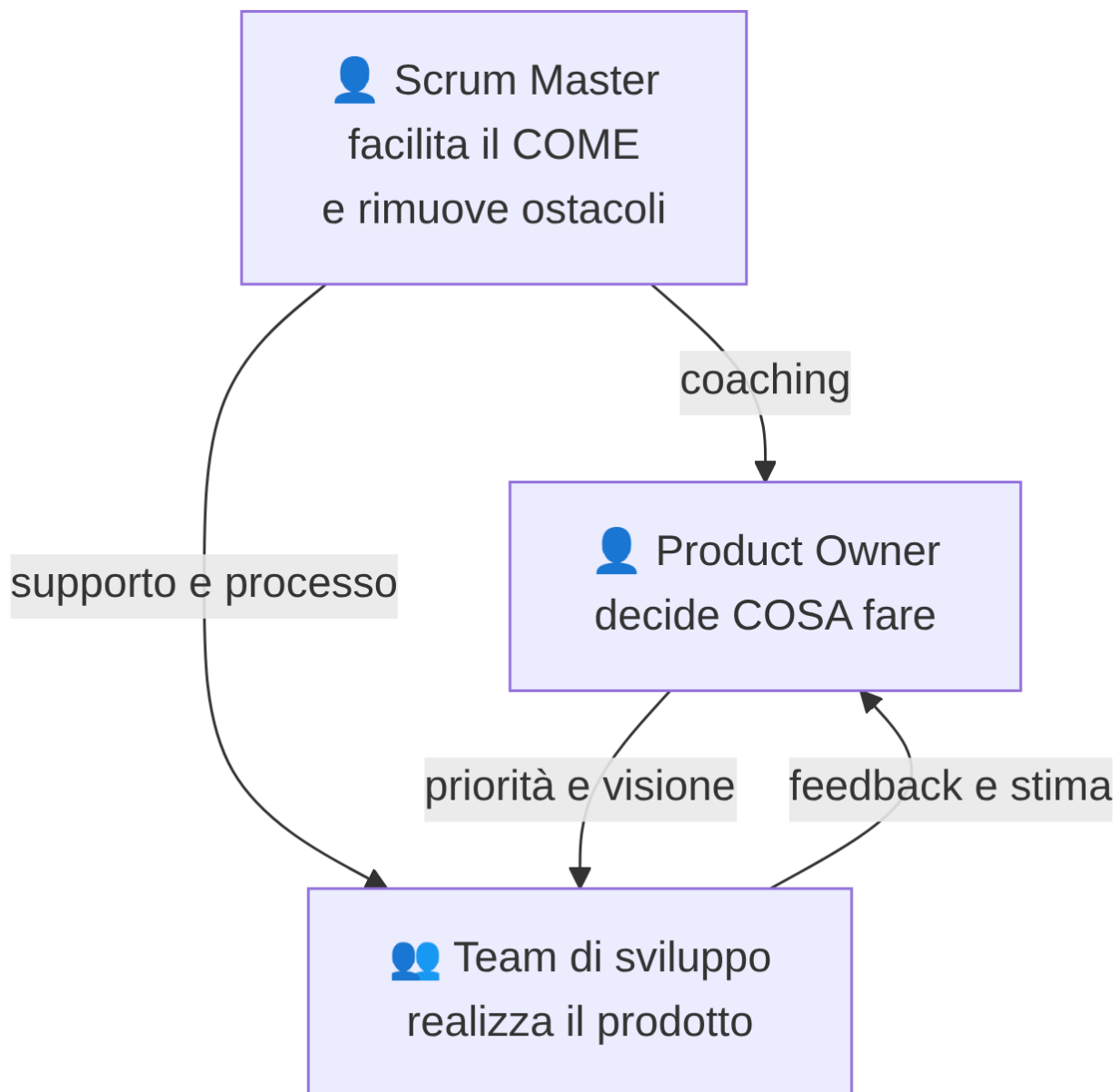


Analogia: pensa a una squadra di calcio che prepara la stagione. Non pianifica ogni singola azione di ogni singola partita dell'anno prima che inizi il campionato: pianifica una partita alla volta, si allena, gioca, guarda i risultati, corregge la tattica per la partita successiva. Scrum applica la stessa logica al lavoro di un team: pianifica “una partita alla volta” (lo Sprint), gioca, osserva il risultato, si adatta.

Un punto importante da ricordare: **Scrum non è un metodo per fare tutto e subito più velocemente**. È un modo per rendere visibile e gestibile la complessità, imparando dall'esperienza reale sprint dopo sprint, invece di affidarsi a un piano perfetto scritto a inizio progetto che quasi certamente si rivelerà sbagliato su qualche punto.

6.2 I tre ruoli di Scrum

Scrum definisce esattamente **tre ruoli** (nella terminologia ufficiale più recente si chiamano “accountability”, cioè “responsabilità”, ma nella pratica quotidiana si continua a parlare di ruoli). Non uno di più, non uno di meno. Vediamoli uno per uno.



6.2.1 Product Owner: la voce del business

Prima che ShopFacile avesse un Product Owner formalmente riconosciuto, capitava una scena come questa: in una stessa settimana, il commerciale chiede ad **Ahmed** di aggiungere “con urgenza” un banner promozionale per un cliente importante, quello stesso cliente importante scrive direttamente al team chiedendo invece una correzione al proprio account, e l'amministratore delegato, passando in ufficio, dice che la vera priorità è “finalmente” il nuovo modulo di reportistica. Ahmed, che vuole fare bene con tutti, comincia il banner, lo abbandona a metà per la correzione dell'account, poi passa al modulo di reportistica quando arriva la richiesta dell'AD. A fine settimana, nessuna delle tre cose è finita, e tre persone diverse sono convinte che “la loro” fosse la priorità assoluta che nessuno ha rispettato.

Questo è esattamente il problema che il ruolo di Product Owner esiste per prevenire. Il **Product Owner** (spesso abbreviato **PO**) non è semplicemente “la persona che decide cosa fare”: è la persona che fa esistere **una sola lista di priorità**, condivisa e visibile, così che il team non diventi il campo di battaglia

di richieste in conflitto che arrivano da direzioni diverse. Oggi, su ShopFacile, quel ruolo lo occupa **Sara**: è lei il punto a cui commerciale, cliente e management devono rivolgersi, ed è lei che decide come le tre richieste dell'esempio sopra si ordinano davvero, non i toni di voce di chi le fa.

Più formalmente, il Product Owner è la persona responsabile di **massimizzare il valore** del prodotto che il team sta costruendo. In pratica, decide **cosa** il team deve realizzare e **in che ordine di priorità**, rappresentando gli interessi del cliente, degli utenti finali e del business.

 **Analogia:** il Product Owner è come il **capo cuoco che decide il menù** di un ristorante. Non è lui che cucina personalmente ogni piatto (questo lo fa la brigata di cucina, cioè il team di sviluppo), ma decide quali piatti proporre, in quale ordine introdurli nel menù, e quali ingredienti sono davvero prioritari da avere in cucina questa settimana.

Le responsabilità principali del Product Owner:

- gestisce e ordina per priorità il **Product Backlog** (lo vedremo tra poco): decide cosa viene fatto prima e cosa dopo;
- si assicura che il team capisca **perché** una funzionalità è importante, non solo “cosa” va costruito;
- è il punto di contatto principale con il cliente/gli stakeholder per le domande su requisiti e priorità;
- accetta o rifiuta il lavoro completato a fine sprint, verificando che soddisfi quanto richiesto.

 **Esempio pratico:** Sara, Product Owner di ShopFacile, ha in cima al Product Backlog tre voci: “Aggiungere il pagamento con carta di credito”, “Correggere un bug che blocca il login su mobile” e “Aggiungere un filtro di ricerca avanzata”. Nella prossima Sprint Planning decide di dare priorità massima al bug di login (blocca utenti reali ogni giorno), poi al pagamento con carta (richiesto da molti clienti), lasciando il filtro di ricerca più in basso: non è meno utile, ma nessun cliente lo ha ancora richiesto con urgenza.

Un errore comune da evitare: il Product Owner **non è un manager che dà ordini al team** su come lavorare, e non decide **come** tecnicamente realizzare qualcosa — quella è una responsabilità del team di sviluppo. Il PO decide le priorità di business, non l'implementazione tecnica.

6.2.2 Scrum Master: il facilitatore (il TUO ruolo)

Prima che Luca presidiasse questo ruolo su ShopFacile, era successo questo: **Giulia** aveva bisogno dell'accesso a un ambiente di test per completare il proprio lavoro. Lo aveva segnalato, ma nessuno si sentiva responsabile di seguirlo fino in fondo: non toccava a Marco, che aveva i suoi task; non toccava a Sara, che si occupava di priorità di business, non di permessi IT. Il ticket aperto con il reparto IT è rimasto in coda, insieme ad altri cento ticket simili, per **una settimana intera**. Giulia, nel frattempo, non è rimasta ferma: ha iniziato altro lavoro per non “perdere tempo”, ma questo ha significato accumulare più cose iniziate e non finite, proprio il tipo di situazione che un limite di WIP (lo vedrai nella sezione 7) cerca di evitare.

Il problema, in quel caso, non era tecnico: era che **nessuno aveva la responsabilità esplicita di farsi carico degli ostacoli del team**. Il **Scrum Master** è la persona responsabile di **far funzionare bene il processo Scrum**: si assicura che il team capisca e applichi correttamente la teoria, le pratiche e le regole di Scrum, e lavora attivamente per rimuovere tutto ciò che ostacola il team — compreso un ticket IT che altrimenti resterebbe bloccato una settimana senza che nessuno lo sblocchi.

Questo è il ruolo che tu andrai a occupare, quindi vale la pena spenderci più tempo degli altri due.

 **Analogia:** se il Product Owner è il capo cuoco che decide il menù e il team di sviluppo è la brigata che cucina, lo **Scrum Master è il direttore di sala** — o meglio, un allenatore/coach della squadra di cucina. Non cucina lui i piatti, non decide il menù, ma si assicura che: la cucina abbia tutto il necessario per lavorare senza intoppi, che i tempi tra un piatto e l'altro siano rispettati, che eventuali problemi (un fornitore in ritardo, un forno che non funziona) vengano risolti rapidamente, e che la squadra migliori il proprio modo di lavorare service dopo service.

Punti fondamentali da capire su questo ruolo, perché generano spesso confusione in chi inizia:

- **Lo Scrum Master non è un capo del team.** Non assegna task, non valuta le prestazioni individuali, non decide chi fa cosa. È un ruolo di **servizio**, non di autorità gerarchica.
- **Lo Scrum Master non è un Project Manager tradizionale.** Non tiene un piano di progetto dettagliato con scadenze imposte dall'alto, non fa micro-management delle attività. Il team di sviluppo si auto-organizza: decide da solo come distribuire il lavoro tra i suoi membri.
- **Lo Scrum Master è un servant leader** ("leader al servizio"): il suo potere non viene dal comandare, ma dal facilitare, proteggere e migliorare continuamente le condizioni di lavoro del team.

Le responsabilità concrete di uno Scrum Master, secondo la Scrum Guide, si dividono in tre direzioni:

1. **Verso il team di sviluppo:** aiuta il team ad auto-organizzarsi, a restare focalizzato, a rimuovere gli impedimenti (ostacoli) che incontra, e facilita tutti gli eventi Scrum.
2. **Verso il Product Owner:** aiuta a gestire e comunicare in modo efficace il Product Backlog, aiuta a trovare tecniche per ordinare le priorità in modo chiaro.
3. **Verso l'organizzazione:** guida e aiuta l'azienda ad adottare Scrum correttamente, rimuovendo barriere organizzative che ostacolano il team (es. troppi meeting esterni, dipendenze non chiare da altri team, processi burocratici lenti).

Un impedimento (in inglese *impediment* o *blocker*) è **qualsiasi cosa che rallenta o blocca il team**: un ambiente di test che non funziona, l'attesa di una risposta dal cliente, un accesso non ancora concesso, un conflitto tra due colleghi, un requisito ambiguo. Parte enorme del lavoro quotidiano di uno Scrum Master è **individuare questi ostacoli e lavorare per rimuoverli**, anche quando la soluzione non dipende direttamente da lui/lei (in quel caso, il compito diventa "scalare" il problema alla persona giusta e seguirne la risoluzione).

📌 **Esempio pratico:** oggi, con Luca nel ruolo, un impedimento come quello di Giulia non resterebbe più bloccato una settimana. Emerge nel Daily Scrum, Luca lo annota senza discuterlo lì (per non allungare il daily) e subito dopo scrive al responsabile IT per sollecitarlo, aggiornando il team il giorno seguente. Non risolve lui stesso il problema tecnico: si assicura che chi può risolverlo lo faccia in tempi rapidi.

6.2.3 Team di sviluppo: chi realizza il prodotto

Immagina, per un momento, una versione di ShopFacile in cui Marco, Giulia e Ahmed non sono un team con una responsabilità collettiva, ma tre individui con compiti separati e nessuna visione d'insieme. A fine sprint, la funzionalità di Ahmed è “finita” — il codice è scritto — ma nessuno l'ha testata. Quella di Marco è testata, ma non è mai stata integrata con il resto del sistema. Alla Sprint Review non c'è niente di realmente funzionante da mostrare, anche se, tecnicamente, ognuno dei tre ha completato il proprio pezzo di lavoro individuale. “Io ho fatto la mia parte” è vero per ciascuno, e allo stesso tempo il prodotto non avanza.

È per evitare esattamente questo che Scrum non definisce il terzo ruolo come una somma di individui, ma come un **team con una responsabilità condivisa sul risultato**. Il **Team di sviluppo** (in inglese *Developers*, termine che nella Scrum Guide più recente indica chiunque contribuisca a realizzare l'Increment: programmatori, ma anche designer, tester, analisti, a seconda del contesto) è il gruppo di persone che **trasforma le voci del Product Backlog in un prodotto funzionante**, sprint dopo sprint — non ciascuno il proprio pezzo, ma tutti insieme l'incremento completo.

💡 **Analogia:** continuando il paragone gastronomico, il team di sviluppo è la **brigata di cucina**: chi taglia le verdure, chi cucina la carne, chi impiatta. Decidono loro, internamente, come organizzarsi per preparare i piatti richiesti nei tempi previsti — nessuno dall'esterno gli dice esattamente “tu tagli la cipolla per primo, tu impiatti per secondo”.

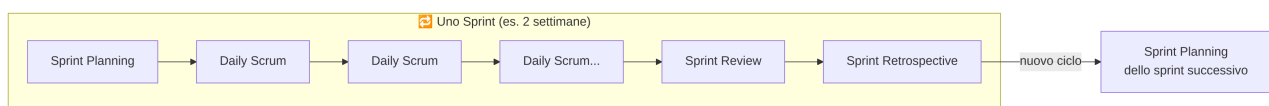
Caratteristiche chiave del team di sviluppo in Scrum:

- è **auto-organizzato**: decide internamente come distribuire il lavoro, nessuno (nemmeno lo Scrum Master) glielo impone dall'esterno;
- è **cross-funzionale**: ha, al suo interno, tutte le competenze necessarie per portare a termine il lavoro senza dover dipendere costantemente da persone esterne al team;
- è tipicamente composto da **3 a 9 persone** (indicazione tipica, non una regola rigida): squadre più piccole comunicano meglio, squadre troppo grandi diventano difficili da coordinare;
- è collettivamente responsabile della qualità del lavoro consegnato — non esiste “il problema è di quel singolo sviluppatore”, esiste “il problema è del team”.

📌 **Esempio pratico:** lo Sprint Backlog di ShopFacile contiene una User Story che richiede sia lavoro di back-end che di front-end. Nessuno assegna i task dall'esterno: durante lo Sprint Planning **Marco** e **Ahmed** si dividono spontaneamente il lavoro ("io mi occupo dell'API, tu del componente grafico") e si aggiornano a vicenda nei Daily Scrum successivi, senza bisogno che il Product Owner o lo Scrum Master decidano chi fa cosa.

6.3 Gli eventi di Scrum

Scrum organizza il lavoro attorno a **cinque eventi** con una cadenza fissa e ripetuta. Il contenitore che li racchiude tutti è lo **Sprint**.



6.3.1 Sprint: il contenitore di tutto

Lo **Sprint** è un periodo di tempo fisso (in inglese si dice *time-boxed*, cioè con una durata massima non superabile) durante il quale il team lavora per produrre un incremento di prodotto potenzialmente utilizzabile.

- **Durata tipica:** da 1 a 4 settimane, più comunemente **2 settimane**. La durata resta **costante** sprint dopo sprint (non si allunga "solo questa volta" perché il lavoro non è finito).
- **Partecipanti:** tutto il team Scrum (Product Owner, Scrum Master, Team di sviluppo).
- **Obiettivo:** consegnare un incremento di valore, utilizzabile e di qualità sufficiente, entro la fine del periodo stabilito.

Quella durata, "1 a 4 settimane", non è un dettaglio arbitrario: nasconde un compromesso reale che dovrai gestire come Scrum Master. Sprint più **corti** (una settimana) danno un feedback più rapido — se il team ha sbagliato direzione, lo scopri quasi subito — ma il costo dei riti (Planning, Review, Retrospective) resta più o meno lo stesso in valore assoluto e quindi pesa proporzionalmente di più: passare 3 ore su 5 giorni lavorativi in riunioni di sprint è un carico ben diverso da 3 ore su 10 giorni. Sprint più **lunghi** (un mese) diluiscono quel costo, ma il rovescio della medaglia è che il team scopre più tardi di aver preso una direzione sbagliata, e nel frattempo ha continuato a lavorarci per settimane. Non esiste una durata "giusta" in assoluto: dipende da quanto rapidamente cambia il contesto in cui il team lavora. Un team che deve reagire in fretta a un mercato che cambia spesso (come ShopFacile durante un periodo di promozioni aggressive dei concorrenti) trova più valore in sprint corti; un team con requisiti più stabili può permettersi sprint più lunghi senza perderci molto.



Analogia: lo Sprint è come una **puntata di una serie TV**: ha una durata fissa, racconta un pezzo di storia autoconclusivo (l'incremento di prodotto), e si inserisce in una stagione più ampia (il progetto). Ogni puntata ha un suo mini-arco narrativo (lo Sprint Goal, l'obiettivo dello sprint) anche se la storia complessiva continua nella puntata successiva.

Dentro ogni Sprint si svolgono, in ordine, gli altri quattro eventi: la Sprint Planning all'inizio, i Daily Scrum ogni giorno, e la Sprint Review e la Sprint Retrospective alla fine.

Esempio pratico: il team di ShopFacile lavora a sprint di 2 settimane, che iniziano sempre di lunedì. Lunedì mattina si tiene la Sprint Planning, ogni mattina alle 9:30 il Daily Scrum, e il venerdì della seconda settimana si tengono, in sequenza, Sprint Review e Sprint Retrospective. Il lunedì successivo inizia già il nuovo sprint.

Visto il contenitore nel suo complesso, entriamo ora nel primo evento che lo apre concretamente: come il team decide, sprint per sprint, cosa finirà davvero dentro quelle due settimane.

6.3.2 Sprint Planning: cosa faremo in questo sprint

La **Sprint Planning** è la riunione che apre ogni Sprint, in cui il team decide **cosa** verrà realizzato nello sprint e, a grandi linee, **come**.

- **Durata tipica:** per uno sprint di 2 settimane, circa **2-4 ore** (in generale, si stima circa un'ora per ogni settimana di durata dello sprint).
- **Partecipanti:** tutto il team Scrum.
- **Obiettivo:** selezionare le voci più prioritarie del Product Backlog che il team ritiene di poter completare nello sprint, e definire lo **Sprint Goal** (l'obiettivo/tema principale dello sprint, in una frase).

Come si svolge in pratica: il Product Owner presenta le voci più in alto nel Product Backlog (quelle a maggiore priorità) e spiega il “perché” di ciascuna. Il team di sviluppo discute, fa domande, e stima quante di quelle voci può realisticamente completare nello sprint, sulla base della propria capacità (spesso guidata dalla Velocity, che vedremo più avanti). Il risultato finale è lo **Sprint Backlog**: l'elenco delle voci scelte per questo sprint, spesso scomposte in task più piccoli e operativi.



Analogia: è come la riunione della squadra prima della partita in cui l'allenatore, insieme ai giocatori, decide la formazione e la tattica di gioco per la partita di questa settimana — non per tutto il campionato, solo per la prossima partita.



Esempio pratico: il team di ShopFacile ha una Velocity media di 25 punti. In Sprint Planning, **Sara** presenta le prime 6 voci del Product Backlog (che valgono, in totale, 34 punti). Dopo discussione, il team si accorge che l'ultima voce (8 punti) è troppo rischiosa da completare

insieme alle altre e la rimanda al prossimo sprint, portando l'impegno totale a 26 punti — vicino alla propria Velocity storica. Lo Sprint Goal che ne esce è: "Permettere ai clienti di completare un ordine dall'inizio alla fine, incluso il pagamento".

Una volta chiuso lo Sprint Backlog, il team di ShopFacile non lo mette in un cassetto: lo esegue giorno per giorno, e il primo strumento con cui lo fa è proprio il Daily Scrum.

6.3.3 Daily Scrum: il check-in quotidiano

Il **Daily Scrum** (spesso chiamato semplicemente "il daily" o "lo stand-up") è una breve riunione quotidiana in cui il team di sviluppo sincronizza il proprio lavoro e pianifica le prossime 24 ore.

Vale la pena immaginare cosa succede a un team che decide di saltarlo "perché siamo tutti impegnati" o "abbiamo già detto tutto in chat". Marco e Ahmed, senza volerlo, finiscono per lavorare tre giorni sulla stessa parte del carrello, scoprendolo solo quando aprono due Pull Request che si sovrappongono. Nel frattempo, il blocco di Giulia sull'ambiente di test (lo hai visto nell'esempio del paragrafo 6.2.2) resta invisibile a tutti tranne a lei fino al giorno prima della fine dello sprint, quando è troppo tardi per risolverlo in tempo. Il Daily Scrum non è un rituale burocratico: è il meccanismo che fa emergere questi problemi entro 24 ore, non entro una settimana.

- **Durata tipica: massimo 15 minuti**, sempre alla stessa ora e nello stesso luogo (fisico o virtuale) per creare abitudine e ridurre attrito.
- **Partecipanti**: principalmente il team di sviluppo. Lo Scrum Master di solito partecipa per facilitare (soprattutto nelle prime fasi di vita di un team) ma non è tenuto a "guidare" la riunione: idealmente è il team a auto-gestirla. Il Product Owner può partecipare, ma non è obbligatorio.
- **Obiettivo**: ispezionare l'avanzamento verso lo Sprint Goal e adattare il piano di lavoro dei giorni successivi.

Come si svolge in pratica: un formato molto diffuso (anche se non è l'unico previsto) è che ogni persona risponda brevemente a tre domande: "cosa ho fatto ieri per lo Sprint Goal?", "cosa farò oggi?", "ci sono ostacoli che mi bloccano?". Se emerge un impedimento (es. "sono bloccato, aspetto una risposta dal reparto sicurezza"), **non si discute la soluzione durante il daily** (per non farlo diventare una riunione lunga): si segna, e se ne parla subito dopo, in un incontro più ristretto — spesso è esattamente qui che entra in gioco lo Scrum Master, per farsi carico di seguire e risolvere quell'ostacolo.

Esempio pratico — un mini Daily Scrum:

- **Marco**: "Ieri ho finito l'endpoint di login. Oggi lavoro sui test automatici. Nessun blocco."
- **Giulia**: "Ieri ho lavorato sulla pagina del profilo utente. Oggi continuo, ma sono bloccata: aspetto l'accesso all'ambiente di test da ieri."
- **Ahmed**: "Ieri ho aiutato Giulia a capire un errore di configurazione. Oggi riprendo la User Story sul carrello."

Tutto il daily dura meno di 10 minuti. Il blocco di Giulia viene segnato dallo Scrum Master, che se ne occupa subito dopo, fuori dal daily.

 **Analogia:** è come il **check-in veloce dello spogliatoio a metà allenamento**: non si riprogetta la tattica da zero ogni giorno, ci si aggiorna velocemente su chi ha un problema (un infortunio, una difficoltà) e si aggiusta la rotta per il resto della sessione.

Daily dopo daily, il team di ShopFacile arriva così alla fine delle due settimane: è il momento di mostrare a chi non ha seguito il lavoro giorno per giorno cosa è stato davvero costruito.

6.3.4 Sprint Review: cosa abbiamo costruito

La **Sprint Review** è la riunione di chiusura in cui il team mostra il lavoro realizzato durante lo sprint e raccoglie feedback.

- **Durata tipica:** per uno sprint di 2 settimane, circa **1-2 ore** (proporzionale alla durata dello sprint, come la Planning).
- **Partecipanti:** tutto il team Scrum, più — ed è importante — gli **stakeholder** invitati: il cliente, altri reparti, chiunque abbia interesse a vedere l'avanzamento del prodotto.
- **Obiettivo:** ispezionare l'incremento realizzato, discutere cosa è cambiato nel contesto (mercato, priorità, vincoli) e adattare il Product Backlog di conseguenza.

Come si svolge in pratica: il team fa una **demo dal vivo** di ciò che ha realizzato (non slide che descrivono il lavoro: il prodotto funzionante, mostrato in azione). Gli stakeholder fanno domande, danno feedback, a volte propongono nuove idee o cambi di priorità. Il Product Owner tiene traccia di tutto questo per aggiornare il Product Backlog.

 **Analogia:** è come l'**assaggio finale prima di servire il piatto ai clienti del ristorante**: la cucina porta il piatto (l'incremento) al tavolo dei giudici (gli stakeholder), che lo assaggiano, commentano, e magari chiedono “la prossima volta, un po' meno piccante” — feedback che entra direttamente nel menù (Product Backlog) della prossima settimana.

 **Esempio pratico:** il team di ShopFacile mostra dal vivo, sull'ambiente di test, il nuovo flusso di checkout appena completato: uno sviluppatore inserisce un ordine di prova davanti a tutti, mostrando ogni passaggio fino alla conferma. Uno stakeholder nota che manca un messaggio di errore chiaro se la carta di credito viene rifiutata: **Sara** annota la richiesta e la aggiunge, come nuova voce, al Product Backlog per una prossima Sprint Planning.

Un errore comune da evitare come Scrum Master: la Sprint Review **non è una presentazione formale con slide** preparata a parte. È una demo pratica e informale del prodotto, pensata per generare conversazione e feedback reale, non per “fare bella figura”.

Con il prodotto mostrato e il backlog aggiornato, resta un'ultima domanda prima di chiudere lo sprint: non cosa è stato costruito, ma come il team ci ha lavorato — ed è esattamente il tema della Sprint Retrospective.

6.3.5 Sprint Retrospective: come possiamo lavorare meglio

La **Sprint Retrospective** (spesso chiamata solo “retro”) è l'ultimo evento dello sprint: il momento in cui il team riflette su **come** ha lavorato (non su cosa ha costruito, quello è già stato discusso nella Review) per trovare modi di migliorare nel prossimo sprint.

- **Durata tipica:** per uno sprint di 2 settimane, circa **1-1,5 ore**.
- **Partecipanti:** solo il team Scrum (Product Owner, Scrum Master, team di sviluppo) — niente stakeholder esterni, per permettere un dialogo aperto e sincero.
- **Obiettivo:** identificare cosa ha funzionato bene, cosa no, e definire **almeno un'azione concreta di miglioramento** da provare nel prossimo sprint.

Come si svolge in pratica: esistono decine di formati per condurre una retrospettiva, ma uno dei più semplici e diffusi si chiama “Start / Stop / Continue”: ogni persona scrive su dei post-it (fisici o su una lavagna digitale) cosa il team dovrebbe **iniziare** a fare, cosa dovrebbe **smettere** di fare, e cosa dovrebbe **continuare** a fare così com'è. Si raggruppano le idee simili, si discutono le più votate, e si scelgono 1-2 azioni concrete da provare nel prossimo sprint (non 15 azioni: meglio poche e realizzabili).



Analogia: è la **riunione tecnica dello staff dopo la partita**, a bocce ferme: non si riguarda solo il punteggio finale, si analizza come si è giocato, cosa ha funzionato nello schema tattico e cosa va corretto per la partita successiva. È il momento in cui una squadra migliora davvero nel tempo, partita dopo partita.



Esempio pratico — risultato di una retrospettiva con il formato Start/Stop/Continue:

- **Start** (iniziare a fare): scrivere una breve nota nella descrizione di ogni Pull Request per spiegare “perché”, non solo “cosa” cambia.
- **Stop** (smettere di fare): accettare nuove richieste urgenti a metà sprint senza discuterne prima con il Product Owner.
- **Continue** (continuare a fare): il pairing tra un developer senior e uno junior sulle User Story più complesse, che ha ridotto gli errori nell'ultimo sprint.

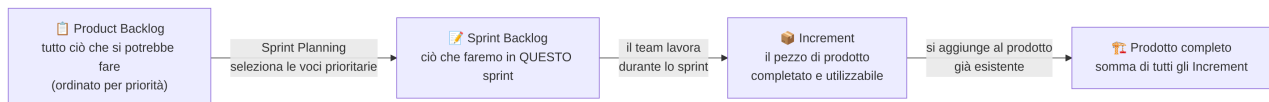
Il team vota le idee più sentite e sceglie una sola azione concreta da provare nel prossimo sprint: introdurre la nota “perché” nelle Pull Request.

Facilitare bene la Retrospective è una delle competenze più importanti di uno Scrum Master: è il principale motore del **miglioramento continuo** del team, uno dei pilastri di Scrum e dell'Agile in generale.

Con la Retrospective si chiude il ciclo degli eventi: il team di ShopFacile riparte da capo con una nuova Sprint Planning, ma quel ciclo, ad ogni giro, produce e trasforma sempre gli stessi tre elementi tangibili. Vediamoli nel dettaglio.

6.4 Gli artefatti di Scrum

Scrum definisce anche **tre artefatti**: elementi tangibili (documenti, liste, prodotti) che rappresentano lavoro o valore, e che rendono trasparente lo stato del progetto a chiunque li guardi.



6.4.1 Product Backlog: la lista completa dei desideri

Il **Product Backlog** è l'elenco completo, ordinato per priorità, di **tutto ciò che potrebbe servire al prodotto**: nuove funzionalità, modifiche, correzioni di bug, miglioramenti tecnici. È di proprietà e responsabilità del **Product Owner**.

 **Analogia:** è la **lista della spesa generale di casa**, quella che tieni sul frigorifero e continui ad aggiornare: non compri tutto oggi, ma sai cosa ti serve, in ordine di urgenza (il latte è finito, serve subito; le tovaglette nuove possono aspettare).

 **Esempio pratico:** il Product Backlog di ShopFacile potrebbe contenere, tra le tante voci: “Come cliente, voglio salvare più indirizzi di spedizione” (priorità alta, già stimata), “Come cliente, voglio ricevere una notifica quando il mio ordine è spedito” (priorità media), “Come amministratore, voglio esportare un report vendite mensile” (priorità bassa, ancora da approfondire). Le prime sono in cima e ben dettagliate, l'ultima è più in basso e resta volutamente vaga per ora.

Caratteristiche importanti del Product Backlog:

- è **vivo e mai completamente finito**: si aggiornano ed evolve continuamente, man mano che si scopre di più sul prodotto, sul mercato, sui bisogni degli utenti;
- è **ordinato per priorità**: le voci più importanti (quelle che il team affronterà prima) stanno in alto;
- le voci in alto sono di solito più **dettagliate e “pronte”** (ne parliamo con la Definition of Ready), mentre quelle in basso possono restare vaghe finché non si avvicina il momento di affrontarle — non ha senso dettagliare oggi qualcosa che verrà realizzato forse tra sei mesi e potrebbe cambiare completamente nel frattempo.

Le voci del Product Backlog sono tipicamente scritte in formato **User Story** (le vediamo nel prossimo paragrafo).

Il Product Backlog di ShopFacile, per quanto ben ordinato, resta però una lista di *possibilità*: solo una piccola parte di quelle voci diventa lavoro concreto in un dato sprint. È esattamente quel sottoinsieme a formare il secondo artefatto.

6.4.2 Sprint Backlog: il piano di questo sprint

Lo **Sprint Backlog** è il sottoinsieme di voci del Product Backlog che il team ha scelto di realizzare **in questo sprint**, insieme al piano per realizzarle (spesso scomposte in task operativi più piccoli) e allo Sprint Goal.

 **Analogia:** se il Product Backlog è la lista della spesa generale di casa, lo Sprint Backlog è la **lista della spesa di oggi**: hai scelto cosa comprare in questo giro, in base a quanto tempo e quanti soldi hai disponibili adesso.

 **Esempio pratico** — uno Sprint Backlog reale potrebbe contenere:

1. Come cliente, voglio salvare più indirizzi di spedizione nel mio profilo (5 punti) — task: creare tabella database, endpoint API, interfaccia utente.
2. Come cliente, voglio ricevere una email di conferma dopo l'acquisto (3 punti) — task: template email, integrazione con servizio di invio.
3. Correggere il bug che duplica il prodotto nel carrello in alcuni casi (2 punti) — task: riprodurre il bug, correggere, scrivere test di regressione.
4. Come amministratore, voglio poter disattivare temporaneamente un prodotto dal catalogo (3 punti) — task: nuovo campo nel database, pulsante in interfaccia admin.

Totale: 13 punti, coerente con la Velocity del team in quello sprint.

Di proprietà del **team di sviluppo** (non del Product Owner): è il team che decide come organizzare il lavoro dentro lo sprint, ed è il team che può aggiornarlo giorno per giorno (ad esempio aggiungendo un task che si scopre necessario durante lo sprint) — sempre restando fedele allo Sprint Goal stabilito in Planning.

Man mano che il team di ShopFacile porta a termine le voci dello Sprint Backlog, quelle righe scritte su una lista si trasformano in qualcosa di concreto e verificabile: è qui che entra in gioco il terzo artefatto, l'Increment.

6.4.3 Increment: il pezzo di prodotto realmente costruito

L'**Increment** è il pezzo di prodotto realmente completato durante lo sprint: funzionante, testato, e — questo è il punto chiave — **conforme alla Definition of Done** del team (ne parliamo tra poco).

 **Analogia:** se il prodotto finale è una casa, ogni Increment è una stanza completamente finita e abitabile — non un muro a metà con i cavi elettrici scoperti. Ogni stanza che si aggiunge deve poter essere già vissuta, anche se la casa nel complesso non è ancora finita.

 **Esempio pratico:** a fine sprint, la funzionalità “salvataggio di più indirizzi di spedizione” di ShopFacile è conforme alla Definition of Done: codice scritto e revisionato, test automatici che passano, verificata manualmente, distribuita in ambiente di test. Questo pezzo di prodotto è l'Increment di quello sprint — anche se **Sara** decide di non rilasciarlo subito agli utenti finali, aspettando di raggrupparlo con altre due funzionalità nella prossima release.

Ogni Increment si somma a tutti quelli realizzati negli sprint precedenti, costruendo progressivamente il prodotto completo. Un punto fondamentale: un Increment deve essere **potenzialmente utilizzabile**, anche se il Product Owner alla fine decide di non rilasciarlo subito agli utenti finali (magari perché si preferisce raggrupparlo con altre funzionalità in un'unica release). “Potenzialmente utilizzabile” significa che, dal punto di vista tecnico e qualitativo, **sarebbe già pronto per andare in produzione**.

Ruoli, eventi e artefatti danno la struttura; ma per capire davvero come una voce del Product Backlog di ShopFacile passa dall'essere un'idea a un compito che il team sa stimare e realizzare, serve guardare al formato con cui quelle voci vengono scritte concretamente.

6.5 User Story: come si descrive un requisito in Scrum

Una **User Story** (letteralmente “storia dell'utente”) è il formato più comune per descrivere una voce del Product Backlog, scritta dal punto di vista di chi utilizzerà quella funzionalità.

Il formato standard è:

Come [tipo di utente/ruolo], **voglio** [azione/funzionalità], **per** [beneficio/motivo].

Esempio concreto:

Come **cliente che sta effettuando un acquisto online**, voglio **poter salvare più indirizzi di spedizione nel mio profilo**, per **non dover reinserire l'indirizzo ogni volta che effettuo un nuovo ordine**.

Perché questo formato è così efficace ed è diventato uno standard:

- costringe a chiarire **chi** beneficia della funzionalità (non è la stessa cosa scrivere una funzionalità per un amministratore o per un cliente finale);

- costringe a esplicitare il **beneficio**, cioè il “perché” — che aiuta tutto il team a capire il valore reale, non solo il compito tecnico da eseguire, e permette anche soluzioni alternative se emergono durante lo sviluppo;
- è **breve e leggibile da chiunque**, anche da chi non ha background tecnico (perfetto per un contesto come il tuo).

Una User Story, di per sé, non è ancora sufficientemente dettagliata per essere sviluppata: quasi sempre si accompagna a **criteri di accettazione** (condizioni precise che permettono di verificare se la storia è stata implementata correttamente). Per la storia sopra, un criterio di accettazione potrebbe essere: *“Il cliente può salvare fino a 5 indirizzi diversi; al momento del checkout può selezionare uno degli indirizzi salvati da un menu a tendina.”*

Le User Story più grandi vengono spesso raggruppate in **Epic** (storie troppo grandi per essere completate in un solo sprint, che vanno scomposte in User Story più piccole prima di poter entrare in uno sprint).

Scrivere bene una User Story, però, risponde solo alla domanda “cosa serve fare e perché”. Resta da rispondere a un'altra domanda, altrettanto concreta per chi pianifica uno sprint: quanto lavoro richiede davvero.

6.6 Story Point: stimare senza usare ore o giorni

Un **Story Point** è un'unità di misura **relativa** che il team usa per stimare quanto “sforzo complessivo” richiede realizzare una User Story, combinando insieme complessità, quantità di lavoro e incertezza/rischio.

Il punto fondamentale da capire — e su cui i neofiti si confondono spesso — è: **i Story Point non sono ore o giorni**. Non esiste una formula fissa del tipo “1 Story Point = 4 ore”. Sono un numero **relativo**: dicono “questa storia è più grande/complessa di quest'altra”, non “questa storia richiederà esattamente questo tempo”.

 **Analogia:** pensa a come stimeresti la difficoltà di alcune scalate in montagna, senza sapere esattamente quanto tempo impiegherai (dipende dal meteo, dalla forma del giorno, da imprevisti): diresti “questa è una scalata facile”, “questa è medio-difficile”, “questa è molto impegnativa e rischiosa”. Non stai dando un tempo in ore, stai dando una valutazione relativa di sforzo e difficoltà — ed è più facile e più affidabile essere d'accordo su “questa è più difficile di quella” che sul tempo esatto in minuti.

Perché il team preferisce i Story Point alle ore/giorni?

- **Le persone sono pessime a stimare il tempo assoluto** (quante volte hai stimato “questo esercizio mi prende 2 ore” e ne ha richieste 5?), ma sono molto più brave a **confrontare due cose** e dire quale è più grande;

- i Story Point si “liberano” dalla velocità specifica di ciascuna persona: una stima in ore cambia da persona a persona (un sviluppatore esperto ci mette 2 ore, un junior magari 6), mentre una stima in punti descrive la complessità del problema, indipendentemente da chi lo risolverà;
- riducono la falsa precisione: dire “questa storia vale 5 punti” comunica onestamente il livello di incertezza, mentre dire “questa storia richiede esattamente 13 ore” dà una falsa sensazione di precisione che quasi mai si rivela vera.

Per stimare, molti team usano la **sequenza di Fibonacci** (o una sua versione semplificata): **1, 2, 3, 5, 8, 13, 21...** I numeri crescono sempre più distanziati tra loro apposta: più una storia è grande, più diventa difficile essere precisi, quindi ha senso avere “scalini” più larghi (la differenza tra una storia da 1 e una da 2 punti deve essere chiara e netta, così come la differenza tra una da 13 e una da 21).

Esempio pratico di sessione di stima (tecnica molto diffusa chiamata *Planning Poker*): il team si riunisce, il Product Owner presenta una User Story, ogni membro del team di sviluppo sceglie in privato una carta con un numero della sequenza di Fibonacci che rappresenta la propria stima, poi tutti scoprono le carte insieme. Se le stime sono simili (es. 5 e 8), si prende una media o si discute brevemente e si converge. Se sono molto diverse (es. 2 e 21), è un segnale che le persone stanno immaginando scenari diversi: si discute finché non emerge un’intesa comune, e spesso proprio questa discussione fa emergere dettagli o rischi nascosti nella storia — che è, in realtà, uno dei benefici più grandi di questa tecnica, al di là del numero finale.

Una User Story troppo grande per essere stimata con sicurezza (es. “vale tra 40 e 100 punti, non sappiamo”) è quasi sempre un segnale che va **scomposta** in storie più piccole prima di poter entrare in uno sprint.

Stimare ogni singola storia è utile sprint dopo sprint, ma il vero valore di quei numeri emerge solo quando li si osserva nel tempo, aggregati: è proprio questo che fa la Velocity.

6.7 Velocity: quanto il team riesce a fare in uno sprint

La **Velocity** (velocità) è il numero medio di Story Point che un team completa in uno sprint, calcolato osservando gli sprint passati.

Esempio concreto: se negli ultimi 4 sprint il team ha completato rispettivamente 23, 27, 25 e 29 Story Point, la sua Velocity media è attorno a **26 punti a sprint** ($(23+27+25+29)/4 = 26$).



Analogia: è come il **ritmo medio di una maratona**: se sai che in allenamento riesci a coprire in media 10 km all'ora, puoi stimare con ragionevole affidabilità quanto tempo ti servirà per completare i 42 km della maratona — non con precisione assoluta (dipende dal giorno, dal meteo, dalla forma), ma con una stima utile per pianificare.

Perché la Velocity è utile, soprattutto per un ruolo come il tuo che dovrà poi parlare anche di pianificazione con il team e gli stakeholder:

- permette di fare **previsioni realistiche**: se il Product Backlog rimanente vale circa 200 punti e la Velocity media è 26 punti a sprint, ci si può aspettare che il lavoro richieda circa 8 sprint ($200/26 \approx 7,7$), un'informazione preziosa per rispondere alla classica domanda del cliente “quando sarà pronto?”;
- aiuta il team, durante la Sprint Planning, a non caricarsi di troppo lavoro in uno sprint (un errore molto comune nei team giovani): se la Velocity media è 26, pianificare uno sprint da 45 punti è un campanello d'allarme;
- diventa più stabile e affidabile nel tempo, man mano che il team accumula storia di sprint passati — nei primi sprint di un team nuovo è normale che la Velocity sia molto variabile.

Un avvertimento importante da Scrum Master: **la Velocity non va mai usata per confrontare team diversi** (“il team A fa 40 punti, il team B solo 20, quindi il team A lavora meglio”) né come metrica di performance individuale. I Story Point sono relativi e specifici di ogni singolo team: un punto per il team A non equivale a un punto per il team B, perché ogni team ha una propria scala interna di riferimento. È uno degli errori più dannosi che si possano fare con questa metrica, e spinge facilmente i team a “gonfiare” artificialmente le stime — proteggere il team da questo uso scorretto è anch'esso un compito dello Scrum Master (lo ritroverai anche tra gli antipattern in 6.11).

Sapere quanto il team riesce a fare non basta, però, se le storie che gli arrivano in Sprint Planning sono ancora ambigue o mal definite: serve un filtro condiviso prima ancora di iniziare a stimarle e pianificarle.

6.8 Definition of Ready: quando una storia può entrare in sprint

La **Definition of Ready** (DoR) è un elenco di criteri condivisi dal team che una User Story deve soddisfare **prima** di poter essere selezionata in una Sprint Planning ed entrare in uno sprint.

 **Analogia:** è come il controllo dei documenti all'imbarco di un aereo: prima di salire a bordo (entrare nello sprint), il biglietto deve avere tutte le informazioni corrette, il documento d'identità deve essere valido, il bagaglio deve rispettare le regole. Se manca qualcosa, non sali a bordo — non perché il viaggio non sia importante, ma perché partire senza tutto il necessario crea problemi a metà volo.

Esempi tipici di criteri di Definition of Ready (variano da team a team, ma un insieme comune potrebbe essere):

- la User Story è scritta secondo il formato standard (Come/Voglio/Per) ed è comprensibile a tutto il team;
- ha criteri di accettazione chiari e verificabili;
- è stata stimata dal team (ha un valore in Story Point);
- le eventuali dipendenze da altri team o sistemi esterni sono note e gestibili;
- è sufficientemente piccola da poter essere completata in un singolo sprint.

📌 **Esempio pratico:** nel progetto ShopFacile, la User Story “Come cliente, voglio pagare con carta di credito” arriva in Sprint Planning ma il team si accorge che non ha ancora criteri di accettazione chiari (cosa succede se la carta viene rifiutata? quali carte sono supportate?) né una stima. Non soddisfa la Definition of Ready: resta nel Product Backlog, **Sara** la raffina con il team in una sessione di refinement, e potrà entrare in uno sprint successivo, quando sarà davvero “pronta”.

Quella “sessione di refinement” citata nell’esempio non è un evento ufficiale di Scrum (non è nell’elenco dei cinque visti in 6.3), ma è una pratica così diffusa che vale la pena definirla: il **refinement** (o raffinamento del backlog) è il momento, spesso ricorrente ma non fisso, in cui il team e il Product Owner **chiariscono e dimensionano** le voci del Product Backlog **prima** che arrivino a una Sprint Planning — è esattamente il lavoro che permette a una storia di passare da “vaga” a conforme alla Definition of Ready.

Perché la Definition of Ready è utile: evita che il team scopra, a metà sprint, che una storia era troppo ambigua per essere realizzata bene, causando ritardi, rilavorazioni o discussioni infinite proprio quando ormai il tempo dello sprint è già stato impegnato. È molto meglio scoprire l’ambiguità **prima** che la storia entri nello sprint.

La Definition of Ready presidia l’ingresso di una storia nello sprint; la sua “gemella” presidia invece l’uscita, cioè il momento in cui il team può davvero dire di aver finito.

6.9 Definition of Done: quando una storia è davvero completata

La **Definition of Done** (DoD) è un elenco di criteri condivisi dal team che stabilisce quando un pezzo di lavoro (una User Story, e più in generale l’Increment) può essere considerato **realmente completato** — e non solo “quasi finito” o “finito sulla carta”.

💡 **Analogia:** è come la checklist di un pilota prima del decollo: non basta che l’aereo “sembri” pronto a occhio. Ci sono controlli precisi e non negoziabili (carburante, strumentazione, comunicazioni) che devono essere tutti spuntati prima di poter dire davvero “pronti al decollo”. Saltare un controllo perché “abbiamo fretta” è esattamente il tipo di scorciatoia che, prima o poi, si paga cara.

Esempi tipici di criteri di Definition of Done (di nuovo, variano da team a team e spesso si arricchiscono nel tempo):

- il codice è stato scritto e sottoposto a code review (vedi la sezione su Git e GitHub — spesso corrisponde a una Pull Request approvata);
- i test automatici sono stati scritti e passano;
- la funzionalità è stata verificata manualmente secondo i criteri di accettazione della User Story;
- la documentazione (se necessaria) è stata aggiornata;

- il codice è stato distribuito almeno in un ambiente di test/staging;
- non introduce bug noti bloccanti.

 **Esempio pratico** — la Definition of Done applicata alla User Story “Come cliente, voglio salvare più indirizzi di spedizione”:

- ☒ Codice scritto e Pull Request approvata da almeno un altro developer
- ☒ Test automatici scritti e superati con successo
- ☒ Funzionalità verificata manualmente rispetto ai criteri di accettazione (salvare fino a 5 indirizzi, selezionarne uno al checkout)
- ☒ Distribuita con successo in ambiente di test
- ☐ Nessun bug bloccante noto

Se anche un solo punto non è spuntato, la storia **non** può essere considerata completata: resta “in corso” fino a che tutti i controlli non sono superati.

Perché la Definition of Done è cruciale, e perché sarà una delle tue responsabilità quotidiane più importanti come Scrum Master: senza una DoD condivisa e rispettata, il team rischia di accumulare **debito tecnico nascosto** — storie segnate come “fatte” che in realtà mancano di test, di qualità, di rifiniture (ne vedi subito il meccanismo nel dettaglio). Uno dei compiti meno visibili ma più importanti dello Scrum Master è proprio **vigilare che il team non “bari” sulla Definition of Done** solo per dichiarare più storie completate in fretta.

Definition of Ready vs. Definition of Done: non confonderle

	Definition of Ready	Definition of Done
Quando si applica	Prima che una storia entri in sprint	Prima che una storia si consideri completata
Risponde alla domanda	“Siamo pronti a iniziare a lavorarci?”	“Abbiamo davvero finito di lavorarci?”
Se non soddisfatta	La storia resta nel Product Backlog, non entra in sprint	La storia resta “in corso”, non si conta come completata

Debito tecnico: il conto che arriva dopo

Il “debito tecnico nascosto” accennato sopra merita una spiegazione propria, perché lo incontrerai spesso e nessuno te lo definirà per te sul lavoro. La metafora è di **Ward Cunningham**, uno dei firmatari del Manifesto Agile: prendere una scorciatoia tecnica per andare più veloci oggi è come prendere un **prestito**. Guadagni tempo subito, ma paghi **interessi** dopo, sotto forma di lavoro che diventa via via più lento e più rischioso, ogni volta che tocchi quella parte di codice.

📌 **Esempio pratico:** per rispettare una scadenza commerciale su ShopFacile, il team decide di saltare la copertura dei test automatici su una parte del flusso di checkout — “la aggiungiamo dopo”, si dice in quel momento. Tre mesi dopo, una modifica in quella stessa area del checkout, che normalmente richiederebbe un giorno di lavoro, ne richiede il triplo: senza test, nessuno sa con certezza cosa si romperà toccando quel codice, e ogni modifica va verificata a mano, riga per riga.

Il punto operativo per il tuo ruolo è questo: il debito tecnico **non è un fallimento tecnico** del team, è una **decisione di business** presa consapevolmente — a volte è del tutto ragionevole accettarla, se la scadenza vale davvero quel rischio. Il problema non è contrarre debito tecnico, è farlo **in silenzio**: se resta nascosto, nessuno lo ripaga mai, perché non compare da nessuna parte finché non esplode. Il tuo compito è renderlo **visibile nel Product Backlog** come qualsiasi altra voce (es. “Aggiungere test automatici al flusso di checkout”), così che Sara possa decidere consapevolmente quando dargli priorità, invece di scoprirlo solo quando causa un incidente in produzione.

Ruoli, eventi, artefatti, stime e criteri di qualità: abbiamo visto tutti i pezzi che il team di ShopFacile usa ogni sprint. Vale la pena, prima di chiudere la sezione, vederli riuniti in un unico schema.

6.10 Il flusso completo, in sintesi

Per fissare le idee, ecco come tutti i pezzi visti finora si incastrano insieme in un ciclo che si ripete sprint dopo sprint:

121

6.11 Il ruolo dello Scrum Master nella pratica quotidiana

Chiudiamo la sezione con la parte più concreta per te: **cosa farai davvero**, ogni settimana, una volta preso in mano questo ruolo. Ecco un elenco realistico di attività quotidiane e ricorrenti:

- **Facilitare gli eventi Scrum:** preparare e condurre Sprint Planning, Daily Scrum, Sprint Review e Sprint Retrospective, assicurandoti che abbiano uno scopo chiaro, rispettino i tempi previsti (*time-box*) e producano risultati concreti (non “riunioni per riunioni”).
- **Tracciare e seguire gli impedimenti:** mantenere una lista visibile degli ostacoli segnalati dal team (spesso emergono proprio nel Daily Scrum), capire chi può risolverli, seguirli fino alla chiusura, e segnalare a manager o altri reparti quelli che richiedono un intervento esterno al team.
- **Aggiornare e mantenere leggibile la board del team** (fisica o digitale, es. su GitHub Projects o Jira): verificare che lo stato delle User Story rifletta la realtà, che nulla resti “bloccato” senza che nessuno se ne accorga, che la board comunichi a colpo d’occhio lo stato reale del lavoro.
- **Vigilare sulla Definition of Done:** aiutare il team a non “accontentarsi” e a non dichiarare completato ciò che, in realtà, manca di qualche passaggio essenziale (test, review, documentazione).
- **Aiutare il Product Owner a preparare un Product Backlog “pronto”:** facilitare sessioni di raffinamento (*refinement*) del backlog, in cui le storie in alto vengono discusse, chiarite e stimate, così da essere pronte per la prossima Planning (Definition of Ready).
- **Proteggere il focus del team:** schermare il team da richieste esterne non pianificate, riunioni superflue, interruzioni che minano la concentrazione durante lo sprint — senza isolare il team dal mondo, ma facendo da filtro intelligente.
- **Monitorare metriche utili** (Velocity, numero di storie completate, eventuali “burndown chart”, che vediamo subito nel dettaglio), non per giudicare le persone, ma per aiutare il team a capire il proprio ritmo e migliorare le previsioni.
- **Coaching del team sull’Agile mindset:** aiutare, nel tempo, il team a interiorizzare i principi Agile visti nella sezione precedente, non solo a “eseguire” meccanicamente i rituali di Scrum.
- **Curare la comunicazione con gli stakeholder:** preparare, insieme al Product Owner, aggiornamenti chiari sull’avanzamento del progetto per il cliente e per il management, spesso appoggiandosi a strumenti come quelli che vedrai nella sezione sul Project Management.
- **Migliorare continuamente il processo del team:** dare seguito concreto alle azioni decise nelle Retrospective, verificando negli sprint successivi se hanno davvero funzionato, e aggiustando se non è così.

Il burndown chart: leggere l’andamento di uno sprint

Il **burndown chart** è un grafico semplice che mostra, giorno per giorno durante lo sprint, quanto lavoro **resta ancora da fare** (di solito in Story Point) rispetto a quanto ne era stato pianificato. L’asse orizzontale rappresenta i giorni dello sprint, l’asse verticale il lavoro residuo: idealmente la linea scende in modo abbastanza regolare da “tutto il lavoro” a “zero” esattamente all’ultimo giorno.

Ecco un burndown realistico per uno sprint di 10 giorni lavorativi partito con 26 punti da completare:

Giorno	1	2	3	4	5	6	7	8	9	10
Punti residui	26	24	24	24	21	15	15	15	6	0

Il numero non conta da solo: conta **come si legge** l'andamento. Dal giorno 2 al giorno 4 la linea resta **piatta a 24**: nessun punto viene chiuso per tre giorni di fila. È un segnale preciso, quasi sempre di uno di questi problemi: una storia è “quasi finita” da giorni ma nessuno la chiude davvero, un impedimento non segnalato blocca qualcuno in silenzio, oppure il team ha sottovalutato la complessità del lavoro in Sprint Planning. Una linea piatta per giorni consecutivi dice, a chi conduce il team, *quando* è nato il problema — non solo che esiste.

Antipattern comuni del neo Scrum Master

Chi arriva a questo ruolo per la prima volta cade quasi sempre in alcuni errori tipici. Conoscerli in anticipo aiuta a evitarli:

- **Trasformare il Daily in un rapporto di stato rivolto a sé.** Se il team riferisce a te invece che agli altri membri, il Daily ha perso il suo scopo. *Correzione*: fai un passo indietro e lascia che il team parli tra sé, tu ascolti e intervieni solo sugli impedimenti.
- **Assegnare i task al posto del team.** Decidere tu chi fa cosa contraddice l'auto-organizzazione vista in 6.2.3. *Correzione*: proponi, non imponi — lascia che sia il team a dividersi il lavoro.
- **Usare la Velocity per confrontare team o persone.** L'hai già visto in 6.7: è uno degli usi più dannosi della metrica. *Correzione*: usala solo per le previsioni interne dello stesso team, mai come classifica.
- **Accettare che la Retrospective produca solo lamentele.** Una retro che finisce senza un'azione concreta è tempo perso. *Correzione*: chiudi sempre con almeno un'azione scelta dal team, piccola e verificabile.
- **Proteggere il team fino a isolarlo dagli stakeholder.** Fare da filtro non significa costruire un muro: un team completamente isolato perde il contatto con il “perché” del proprio lavoro. *Correzione*: filtra le interruzioni inutili, ma mantieni visibili i canali di comunicazione che contano davvero (come la Sprint Review).

Un consiglio pratico per iniziare: nei primi sprint che seguirai, il tuo obiettivo principale non è “cambiare tutto e ottimizzare subito”. È **osservare, capire le dinamiche reali del team** (chi si blocca spesso su cosa, quali riunioni funzionano e quali sono percepite come inutili, dove nascono davvero gli impedimenti) e solo dopo iniziare a proporre piccoli miglioramenti, uno alla volta — esattamente come farebbe il team stesso in una Retrospective ben condotta.

È esattamente questo l'equilibrio che **Luca** ha imparato a trovare lavorando con **Sara, Marco, Giulia e Ahmed** su ShopFacile: lo stesso equilibrio che, da qui in avanti nel corso, dovrai imparare a trovare tu.

Esercizi pratici

Gli esercizi seguenti servono a consolidare quanto visto in questa sezione con qualcosa che puoi davvero fare, non solo leggere. Non serve un progetto reale per farli: puoi usare scenari immaginari, e diversi da quelli già proposti nel piano di studio (sezione 17) — qui l'obiettivo è approfondire con più dettaglio, proprio perché questa è la sezione più importante del corso per il tuo ruolo.

1. **Scrivi 3 User Story** per un'ipotetica app di noleggio di biciclette in condivisione (bike sharing), seguendo il formato Come/Voglio/Per, e aggiungi almeno un criterio di accettazione per ciascuna.  **Come verificare:** ogni storia indica chiaramente un tipo di utente (non “come utente” generico, ma es. “come cliente che deve sbloccare una bici”), un beneficio concreto (non solo un'azione tecnica), e almeno un criterio di accettazione scritto in modo specifico e verificabile (non “deve funzionare bene”).
2. **Simula una sessione di Planning Poker** con 2-3 colleghi, amici o familiari (non serve che siano tecnici): scegli 5 attività quotidiane non lavorative (es. “cucinare la cena per 6 persone”, “organizzare una festa a sorpresa”, “traslocare in una nuova casa”) e stimale con la sequenza di Fibonacci (1, 2, 3, 5, 8, 13, 21), scoprendo le carte insieme e discutendo le differenze.  **Come verificare:** hai almeno un caso in cui le stime iniziali erano molto diverse tra loro (es. 2 contro 13) e la discussione successiva ha fatto emergere un motivo concreto della differenza (non solo “abbiamo scelto un numero a caso”).
3. **Costruisci uno Sprint Backlog di esempio:** immagina un team con Velocity media di 20 punti e uno sprint di 2 settimane. Scegli 4-5 User Story (puoi riusare quelle scritte nell'esercizio 1 o inventarne altre), assegna a ciascuna una stima in punti, e scrivi anche uno Sprint Goal in una sola frase.  **Come verificare:** il totale dei punti scelti è vicino alla Velocity indicata (né troppo sopra né troppo sotto, es. tra 17 e 22 punti), e lo Sprint Goal riassume in una frase il “tema” comune delle storie scelte, non un semplice elenco.
4. **Calcola una Velocity media:** un team fittizio ha completato, negli ultimi 5 sprint, rispettivamente 18, 22, 19, 24 e 21 Story Point. Usa questo numero per stimare quanti sprint servirebbero per completare un Product Backlog residuo di 150 punti.  **Come verificare:** la Velocity media calcolata è 20,8 punti (arrotondabile a 21), e la stima è di circa $150/21 \approx 7$ sprint.
5. **Conduci una mini-retrospettiva** di circa 20 minuti con colleghi, amici o familiari su un'attività di gruppo recente qualsiasi (non lavorativa: un viaggio organizzato insieme, un progetto universitario di gruppo, l'organizzazione di un evento), usando il formato Start / Stop / Continue, e scegli una sola azione di miglioramento concreta da provare la prossima volta.  **Come verificare:** alla fine hai una sola azione scritta, concreta e verificabile (non “comunicare meglio”, ma qualcosa come “condividere l'itinerario su un documento condiviso almeno 3 giorni prima”).
6. **Scrivi una Definition of Done di 5 punti** per un'attività non informatica (es. “organizzare una cena per 10 persone” o “preparare la presentazione per il primo giorno di lavoro”), poi verifica a fine attività quali punti hai davvero rispettato.  **Come verificare:** la tua DoD ha esattamente 5 criteri verificabili con un sì/no (non vaghi come “tutto ok”), e a lavoro concluso sei in grado di dire con certezza quali hai soddisfatto e quali no.

7. **Ricostruisci un impedimento reale:** chiedi alla tua collega Scrum Master/PM di raccontarti un impedimento gestito di recente nel progetto (chi lo ha segnalato, in quale evento è emerso, chi lo ha risolto e in quanto tempo). Poi scrivi in 5 righe come ti saresti comportato tu, da Scrum Master, in quella stessa situazione.  **Come verificare:** la tua ricostruzione indica chiaramente in quale evento Scrum è emerso l'impedimento (quasi sempre il Daily Scrum) e chi, concretamente, ha agito per rimuoverlo — non solo “il problema si è risolto da sé”.
-

Collegamenti

- [7. Kanban](#) — un altro framework Agile, spesso complementare o alternativo a Scrum
- [8. Project Management](#) — come gli strumenti di gestione progetto si integrano con la pratica quotidiana di uno Scrum Master
- [16. Glossario](#) — per ripassare velocemente i termini di questa sezione

Risorse

- [Scrum Guide ufficiale \(scrumguides.org\)](https://www.scrumguides.org/)
- [Scrum.org](https://www.scrum.org/) — What is Scrum?
- [Atlassian](#) — Agile Coach: cos'è Scrum

7. Kanban

Nella sezione precedente hai visto Scrum: sprint a tempo fisso, ruoli definiti, eventi ricorrenti (Sprint Planning, Daily Scrum, Sprint Review, Sprint Retrospective). Immagina il team di **ShopFacile** appena uscito da una Sprint Retrospective: tra i punti emersi c'è che i bug urgenti segnalati dagli utenti, arrivati a metà sprint senza preavviso, hanno continuato a “saltare la fila” e a scombinare la pianificazione fatta a inizio sprint. Sara e Luca si chiedono se, almeno per questo tipo di lavoro imprevedibile, un flusso continuo non sia più adatto di uno sprint a tempo fisso. È esattamente il problema che Kanban è pensato per risolvere.

Scrum è un framework molto “strutturato”. Kanban parte da un’idea diversa e più semplice: **rendere visibile il lavoro e limitare quanto ne facciamo contemporaneamente**, senza necessariamente imporre sprint o ruoli fissi.

Se Scrum è come organizzare il lavoro a “capitoli” di durata fissa, Kanban è più simile a un nastro trasportatore continuo: il lavoro scorre, una voce alla volta, dall’inizio alla fine, senza tappe temporali predefinite.

Obiettivi della sezione

Alla fine di questa sezione saprai:

- da dove viene il metodo Kanban e perché si chiama così;
- come è strutturata una board Kanban e cosa rappresentano colonne e card;
- cos'è il WIP (Work In Progress) e perché limitarlo migliora il flusso di lavoro del team;
- la differenza tra Lead Time e Cycle Time, con un esempio numerico;
- quando conviene usare Kanban invece di Scrum (o entrambi insieme).

7.1 Da dove viene Kanban

La parola **Kanban** (看板) viene dal giapponese e significa più o meno “cartellino visivo” o “insegna”. Il metodo nasce **non nel software**, ma in fabbrica: negli anni '40-'50 **Toyota** lo introdusse nelle sue linee di produzione automobilistica come parte del sistema di produzione “lean” (cioè “snello”, senza sprechi).

L'idea originale era molto concreta: in una linea di montaggio, ogni reparto usava dei cartellini fisici per segnalare al reparto precedente “ho bisogno di altri pezzi” — invece di produrre pezzi in grandi quantità “a caso” e ammassarli in magazzino (con il rischio di produrne troppi o troppo pochi), si produceva **solo quello che serviva, quando serviva**. Questo riduceva sprechi, magazzini pieni di materiale inutilizzato, e colli di bottiglia nascosti.

Decenni dopo, a partire dagli anni 2000, questa filosofia è stata adattata al lavoro di team software (il nome più associato a questa trasposizione è David J. Anderson). L'idea di fondo resta identica: **rendere visibile il flusso di lavoro e non accumulare più lavoro “in corso” di quanto il team riesca effettivamente a gestire.**

7.2 La board Kanban: colonne e card

Il cuore pratico di Kanban è la **board** (bacheca), divisa in **colonne** che rappresentano le fasi che un elemento di lavoro attraversa, da quando viene richiesto a quando è completato.

La versione più semplice ha tre colonne:

- **To Do** (da fare): lavoro non ancora iniziato;
- **In Progress** (in corso): lavoro su cui qualcuno sta lavorando ora;
- **Done** (fatto): lavoro completato.

Nella pratica reale, quasi nessun team si ferma a tre colonne: le colonne vengono **personalizzate** per rispecchiare le fasi reali del lavoro del team. Un esempio molto comune in un contesto software:

To Do → In Analisi → In Sviluppo → In Test → Done

Ogni **card** (cartellino, che riprende proprio l'idea del cartellino giapponese originale) rappresenta un singolo elemento di lavoro: una user story, un bug da correggere, un task tecnico. La card si sposta da sinistra a destra sulla board, colonna dopo colonna, man mano che avanza. Ogni card contiene tipicamente: un titolo, una breve descrizione, chi ci sta lavorando (assegnatario), e a volte etichette o una stima di dimensione.



 **Analogia:** pensa alla board come al monitor delle partenze in un aeroporto. Ogni volo (card) parte da “In attesa”, passa per “Imbarco in corso” e arriva a “Partito”. A colpo d'occhio, chiunque guardi il monitor capisce subito lo stato di ogni volo — senza dover chiedere a nessuno “a che punto siamo?”.

Le board Kanban possono essere fisiche (un muro con post-it) o digitali (strumenti come GitHub Projects, Trello, Jira). Su GitHub, ad esempio, una board di questo tipo si costruisce con **GitHub Projects**: le colonne diventano stati personalizzabili e le card possono collegarsi direttamente a Issue e Pull Request del repository, così lo stato del lavoro sul codice si riflette automaticamente sulla board.

💡 **Esempio pratico:** immagina la board Kanban del team di **ShopFacile**, con 4 colonne — **To Do**, **In Sviluppo**, **In Test**, **Done** — e in un dato lunedì mattina la situazione è questa:

- **To Do** (6 card): richieste appena arrivate, non ancora prese in carico — es. “Aggiungere validazione campo email”, “Bug: pagina lenta su mobile”.
- **In Sviluppo** (3 card): “Fix errore export PDF” (assegnata a Marco), “Aggiornare libreria di logging” (assegnata a Giulia), “Aggiungere filtro categoria al catalogo” (assegnata ad Ahmed).
- **In Test** (1 card): “Nuovo filtro ricerca prodotti”, in verifica dal tester.
- **Done** (3 card, completate questa settimana): “Fix bottone disabilitato”, “Aggiornamento pagina contatti”, “Correzione traduzione IT”.

Guardando solo questa riga di numeri (6 / 3 / 1 / 3), un Project Manager può già farsi una prima idea del flusso: c'è più lavoro in attesa (6) di quanto il team ne stia lavorando attivamente (3+1=4) — non è necessariamente un problema, ma è un segnale da tenere d'occhio se il numero in “To Do” continua solo a crescere.

Avere una board così, visibile a tutti, è già un grande passo avanti — ma non basta da solo: bisogna anche controllare **quanto lavoro il team si mette a fare contemporaneamente**, non solo renderlo visibile. È qui che entra in gioco il concetto di **WIP**.

7.3 WIP: il lavoro “in corso” e i suoi limiti

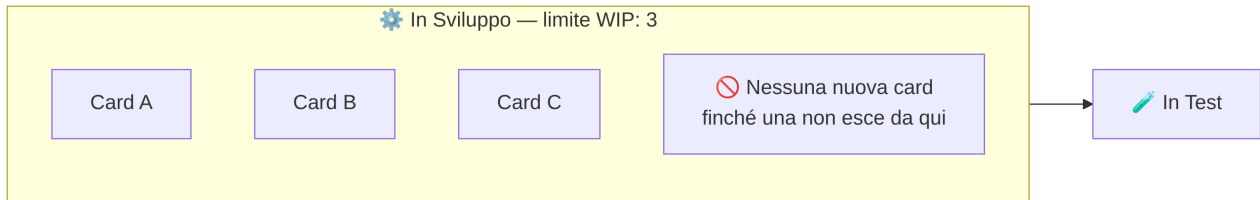
WIP è l'acronimo di **Work In Progress**, cioè “lavoro in corso”: il numero di elementi che, in un dato momento, sono già iniziati ma non ancora completati.

Uno dei principi cardine di Kanban è il **limite di WIP**: si stabilisce un numero massimo di card che possono trovarsi contemporaneamente in una determinata colonna (tipicamente le colonne di lavoro attivo, come “In Sviluppo” o “In Test”). Se una colonna ha già raggiunto il suo limite, **nessuno può iniziare una nuova card in quella colonna** finché non se ne libera una completandola e facendola avanzare.

Detto così può sembrare controintuitivo: perché limitare volontariamente quanto lavoro il team può iniziare? La risposta è che **fare tante cose contemporaneamente, in realtà, le fa fare tutte più lentamente**.

💡 **Analogia:** pensa al casello di un'autostrada durante un weekend di esodo. Se tutte le macchine potessero entrare in autostrada contemporaneamente senza controllo, si formerebbe un ingorgo che blocca *tutti*, e alla fine le macchine arriverebbero più tardi di quanto arriverebbero se il casello facesse passare le auto in modo controllato, poche alla volta. È lo stesso principio di un **imbuto**: se versi il liquido troppo in fretta, l'imbuto si intasa e in realtà il liquido scende **più lentamente** rispetto a versarlo con un flusso costante e misurato.

Nel lavoro di un team succede la stessa cosa: se ognuno ha 6 task “aperti” contemporaneamente, continua a passare da uno all'altro (context switching), nessun task avanza davvero in fretta, e la sensazione di “essere sempre indaffarati” nasconde il fatto che **poco viene realmente completato**. Limitare il WIP forza il team a **finire prima di iniziare altro**, il che in pratica fa terminare più lavoro, più velocemente, con meno errori (perché ci si concentra su meno cose alla volta).



Un segnale tipico da Project Manager/Scrum Master: se vedi una colonna della board costantemente “esplosa” oltre il suo limite di WIP, è un sintomo concreto di un **collo di bottiglia** nel flusso di lavoro del team, su cui vale la pena indagare (manca una competenza? una fase richiede un’approvazione lenta? c’è troppa dipendenza da una sola persona?).

 **Esempio pratico:** nel team di ShopFacile, la colonna “In Sviluppo” ha un limite di WIP fissato a 3. Martedì la colonna ha già 3 card (A, B, C) e **Ahmed**, che ha appena finito un task, vorrebbe iniziarne uno nuovo, la card D. Con il limite di WIP attivo, **non può farlo**: deve prima aiutare a completare A, B o C (magari facendo code review a un collega, o testando manualmente una delle tre) e farla avanzare in “In Test”. Solo quando una delle tre card lascia la colonna, si “libera uno slot” e la card D può entrare. Il risultato pratico è che Ahmed, invece di aprire un quarto fronte, spinge a chiudere qualcosa che è già a metà — ed è esattamente l'effetto voluto.

Limitare il WIP non serve solo a evitare che il team apra troppi fronti contemporaneamente: come vedremo subito, avere poche card “in corso” alla volta rende anche possibile **misurare con precisione** quanto tempo impiega davvero una card ad attraversare la board, dall'ingresso in To Do al completamento.

C'è però un effetto collaterale del limite di WIP che nessun manuale racconta volentieri: quando la colonna è piena e qualcuno resta senza un task “proprio” da avviare, si sente in colpa, come se stesse “non facendo niente”. È un riflesso comprensibile — siamo abituati a misurare il proprio valore dal numero di cose che abbiamo aperto — ma è esattamente il comportamento opposto a quello che il limite di WIP vuole ottenere. Il compito di chi facilita il team (che tu incontrerai nella pratica come Scrum Master, vedi la sezione 6.2.2) è ricordare, con costanza, che **aiutare a chiudere il lavoro di un collega è lavoro**, non meno prezioso di aprirne uno nuovo: è proprio quello che fa Ahmed nell'esempio sopra, e non è “tempo perso” solo perché non produce una nuova card con il suo nome sopra.

Classi di servizio e corsia di emergenza: e i bug urgenti?

Il capitolo si è aperto con la domanda di Sara e Luca: cosa si fa quando arriva un bug critico che non può aspettare il proprio turno in coda? La risposta di Kanban non è “si ignora la board e si lavora fuori sistema”: è definire, **prima** che il problema si presenti e insieme al team, una **classe di servizio** dedicata all'urgenza reale, spesso visualizzata come una corsia separata in alto sulla board (in inglese si parla di *swimlane*).

Tre decisioni vanno prese in anticipo, non nel momento della crisi:

- **Quante corsie di emergenza esistono.** Tipicamente **una sola**: se ogni tipo di richiesta ha la sua corsia “prioritaria”, nessuna lo è davvero più delle altre, e il limite di WIP perde significato.
- **Quali criteri rendono un lavoro davvero urgente.** Non basta “il cliente dice che è urgente”: un criterio concreto per ShopFacile potrebbe essere “il sito è irraggiungibile o un cliente non riesce a pagare, in produzione, in questo momento” — non “sarebbe comodo averlo prima”.
- **Cosa succede al lavoro normale quando la corsia di emergenza viene usata.** La card urgente entra e viene lavorata subito, anche superando il limite di WIP della colonna — ma il team ne discute apertamente nel Daily successivo, perché quel superamento ha un costo: qualcos'altro, inevitabilmente, avanza più lentamente.

 **Esempio pratico:** tornando al team di ShopFacile, dopo la Retrospective con cui si è aperto questo capitolo, Sara e Luca definiscono insieme al team una singola corsia di emergenza, riservata solo a bug che bloccano un pagamento o rendono il sito irraggiungibile. La settimana dopo arriva esattamente un caso così: un bug impedisce a una parte dei clienti di completare il checkout. La card entra nella corsia di emergenza, **Giulia** la prende in carico subito, superando per un giorno il limite di WIP della colonna “In Sviluppo”. Nel Daily seguente il team ne discute apertamente: la card “Aggiornare libreria di logging” che stava a metà è rimasta ferma un giorno in più, ma è una conseguenza accettata, non un imprevisto subito in silenzio. I bug urgenti non “saltano la fila” in modo caotico e invisibile come prima: hanno una fila propria, dichiarata e con regole chiare.

7.4 Lead Time e Cycle Time

Kanban, essendo un metodo orientato al “flusso” continuo di lavoro, si misura soprattutto con due metriche di tempo, spesso confuse tra loro ma concettualmente diverse. Vediamole applicate alla board di ShopFacile che abbiamo appena descritto.

Lead Time

Il **Lead Time** è il tempo totale che passa **dal momento in cui una richiesta viene fatta** (la card entra nella colonna “To Do”, cioè nel backlog) **al momento in cui viene completata** (la card arriva in “Done”). È il tempo che interessa di più a **chi ha fatto la richiesta** (un cliente, uno stakeholder): “quanto tempo devo aspettare per avere questa cosa?”.

Cycle Time

Il **Cycle Time** è il tempo che passa **dal momento in cui il team inizia effettivamente a lavorarci** (la card entra in “In Progress” o “In Analisi”) **al momento in cui viene completata**. È il tempo che interessa di più al **team**, perché misura quanto efficiente è il lavoro una volta avviato, senza contare l'attesa in coda.

La differenza pratica è quindi il tempo di **attesa in coda**, prima che qualcuno inizi effettivamente a lavorare sulla richiesta.

Esempio numerico

Immagina questa card: “Aggiungere filtro di ricerca alla pagina prodotti”.

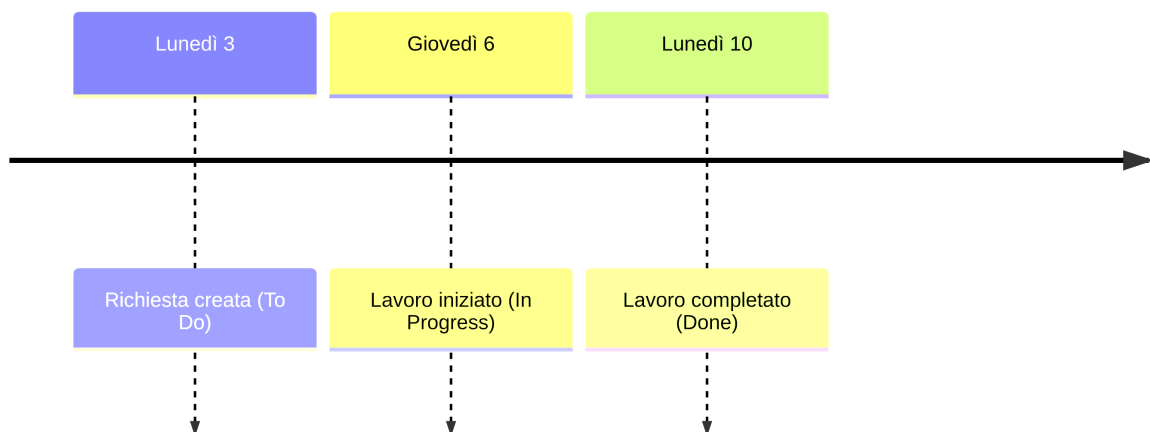
- **Lunedì 3**: la richiesta viene registrata e messa in “To Do”.
- **Giovedì 6**: uno sviluppatore inizia effettivamente a lavorarci (“In Progress”).
- **Lunedì 10**: il lavoro è completato e la card arriva in “Done”.

Calcoliamo:

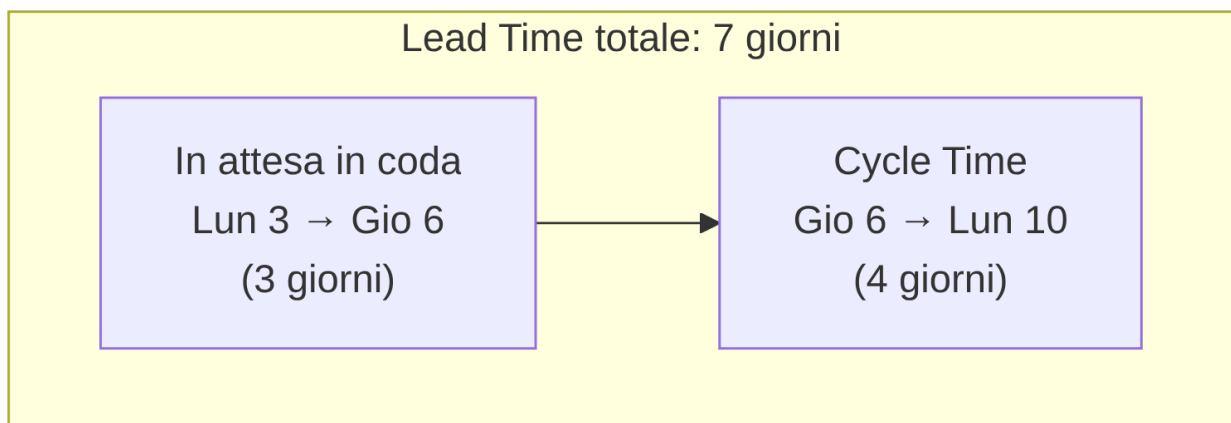
- **Lead Time** = da lunedì 3 a lunedì 10 = **7 giorni**
- **Cycle Time** = da giovedì 6 a lunedì 10 = **4 giorni**

La differenza (3 giorni) è il tempo che la richiesta ha passato “in coda”, in attesa che qualcuno se ne occupasse.

Lead Time vs Cycle Time



Rappresentazione a barre, per confrontare visivamente i due intervalli:



💡 **Perché la distinzione conta per un Project Manager:** se il cliente si lamenta che “le richieste ci metton troppo ad arrivare”, devi capire **dove** si perde tempo. Se il Cycle Time è basso ma il Lead Time è alto, il problema non è che il team lavora lentamente: è che le richieste restano troppo tempo in coda prima che qualcuno le prenda in carico. Sono due problemi diversi, con soluzioni diverse (il primo si risolve migliorando l’efficienza del team, il secondo migliorando come si priorizza e si smista il lavoro in arrivo).

💡 **Esempio pratico:** confrontiamo due card diverse della stessa settimana.

- Card “Bug critico: pagamento non va a buon fine” — entra in To Do **martedì 4** alle 9:00, un developer la prende in carico **subito** (martedì 4, 9:30) perché è urgente, ed è completata **mercoledì 5**. Lead Time $\approx$ Cycle Time $\approx$ **1 giorno**: nessuna attesa in coda.
- Card “Migliorare testo pagina ‘Chi siamo’” — entra in To Do **martedì 4**, ma essendo bassa priorità resta in coda per **12 giorni**; qualcuno la prende in carico **giovedì 16** e la completa lo stesso giorno in 2 ore. Lead Time = **12 giorni**, Cycle Time = **meno di 1 giorno**.

Stesso team, stessa efficienza di esecuzione (Cycle Time basso in entrambi i casi) — ma un Lead Time completamente diverso, perché dipende da **quanto la card ha aspettato in coda**, non da quanto ci ha messo il team a farla una volta iniziata.

Lead Time e Cycle Time sono le metriche con cui Kanban misura il flusso continuo di lavoro; Scrum, che lavora a sprint, misura invece l’avanzamento con una metrica diversa, la Velocity. Vale la pena confrontare i due approcci fianco a fianco, per capire quando ha senso usare l’uno o l’altro.

7.5 Kanban vs Scrum: quando usare cosa

Kanban e Scrum condividono lo stesso spirito Agile (visto nella sezione 5), ma organizzano il lavoro in modo diverso.

	Scrum	Kanban
Struttura del tempo	A sprint di durata fissa (es. 2 settimane)	Flusso continuo , senza intervalli di tempo fissi
Consegna del lavoro	Un “pacchetto” di elementi consegnato a fine sprint	Ogni elemento viene consegnato appena è pronto
Ruoli	Ruoli definiti (Scrum Master, Product Owner, Team)	Nessun ruolo obbligatorio specifico
Eventi ricorrenti	Planning, Daily, Review, Retrospettiva	Nessun evento obbligatorio (anche se molti team fanno comunque una daily)
Cosa si limita	La quantità di lavoro pianificata per lo sprint (lo Sprint Backlog)	Il WIP, cioè quante card possono essere “in corso” in ogni momento
Metriche tipiche	Velocity (punti completati per sprint)	Lead Time, Cycle Time, Throughput
Cambiamenti a metà lavoro	Scoraggiati a metà sprint (si aspetta il prossimo)	Benvenuti in qualsiasi momento, il flusso è continuo

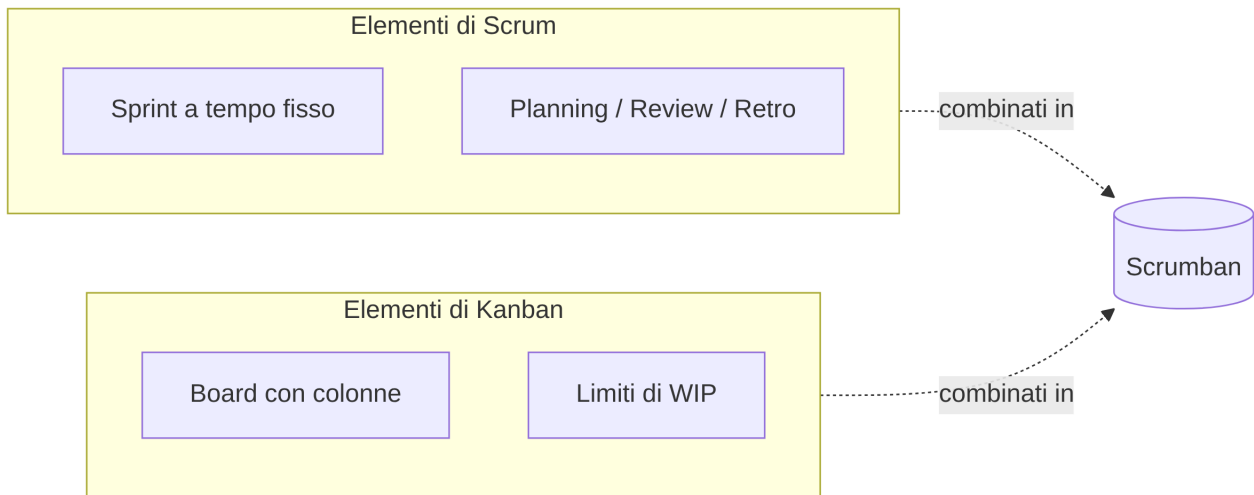
Nella tabella compare un termine che non hai ancora incontrato: il **Throughput** (letteralmente “capacità di elaborazione”) è semplicemente **quante card il team completa in un dato periodo** (es. “12 card a settimana”), a differenza di Lead Time e Cycle Time che misurano una durata. Se Lead Time e Cycle Time rispondono a “quanto tempo ci vuole”, il Throughput risponde a “quanto ne finiamo” — le due informazioni insieme danno un quadro completo del flusso: un team può avere un Cycle Time basso ma un Throughput scarso, se lavora bene sulle poche card che porta avanti ma ne avvia troppo poche in parallelo.

Quando usare Scrum: quando il lavoro si presta a essere pianificato in blocchi (sprint), il team vuole momenti regolari di revisione e retrospettiva, e serve una cadenza predicibile per pianificare e comunicare con gli stakeholder (es. “a fine sprint mostriamo una demo”).

Quando usare Kanban: quando il lavoro arriva in modo **imprevedibile e continuo**, con priorità che cambiano spesso — è il caso tipico di team di **supporto, manutenzione, gestione incident** o di piattaforme DevOps, dove non ha molto senso “pianificare due settimane fisse” perché una richiesta urgente (es. un bug critico in produzione) può arrivare in qualsiasi momento e deve poter “saltare la fila”.

Scrumban: il meglio dei due mondi

Molti team reali non scelgono in modo rigido l'uno o l'altro, ma **combinano i due approcci**, in un ibrido informalmente chiamato **Scrumban**: si mantiene la cadenza degli sprint e degli eventi Scrum (planning, review, retrospettiva) per la pianificazione e la comunicazione, ma si gestisce il lavoro **dentro** lo sprint con una board Kanban e limiti di WIP, per governare meglio il flusso quotidiano. È un approccio molto comune nei team che gestiscono sia nuovo sviluppo pianificato sia richieste impreviste (bug urgenti, richieste di supporto).



Prima di chiudere, però, un'ultima onestà intellettuale: Kanban non è privo di costi. La sua stessa forza — l'assenza di una cadenza fissa — è anche il suo limite principale. Senza sprint, è più difficile dare al cliente una data di consegna prevedibile per un gruppo di funzionalità (“quando avremo tutte queste 5 cose insieme?” ha una risposta più naturale in Scrum che in Kanban, dove ogni card procede per conto proprio). E senza gli eventi ricorrenti di Scrum, mancano i momenti naturali in cui il team si ferma a fare una demo o a riflettere su come sta lavorando: una Review e una Retrospective non nascono da sole nel flusso continuo, vanno **istituite deliberatamente** — molti team Kanban le mantengono comunque, a cadenza fissa, anche senza uno sprint che le “obblighi”.

Chiudiamo qui il confronto con Scrum: riprendiamo in breve i punti chiave di questa sezione su Kanban.

7.6 Riepilogo

- Kanban nasce nella produzione industriale (Toyota, filosofia “lean”) e si è poi adattato al lavoro dei team software.
- La **board**, con le sue colonne personalizzabili, rende visibile lo stato di ogni card.
- Limitare il **WIP** (lavoro in corso) migliora il flusso complessivo: meno cose iniziate contemporaneamente, più cose finite rapidamente.
- **Lead Time** misura l'attesa dal punto di vista di chi richiede; **Cycle Time** misura l'efficienza del team una volta che il lavoro è iniziato.
- Scrum lavora a **sprint**, Kanban lavora a **flusso continuo**: la scelta dipende dal tipo di lavoro del team, e i due approcci si possono anche combinare (**Scrumban**).



Esercizi pratici

1. **Disegna una board a 4 colonne.** Su carta o con un tool gratuito online, crea una board con le colonne **To Do** → **In Sviluppo** → **In Test** → **Done** e inventa 6 card realistiche per un progetto a tua scelta (anche immaginario, es. un sito di prenotazioni). Distribuisci le card nelle colonne

come faresti se fossi tu a gestire il flusso in questo momento.  **Come verificare:** se riesci a spiegare a voce, per ciascuna card, “perché è in quella colonna e non in un'altra”, l'esercizio è fatto bene.

2. **Applica un limite di WIP e osserva l'effetto.** Riprendi la board dell'esercizio 1 e imposta un limite di WIP di 2 sulla colonna “In Sviluppo”. Se hai messo più di 2 card in quella colonna, decidi quali spostare (indietro in To Do, o avanti se sono davvero pronte) per rispettare il limite.  **Come verificare:** alla fine, la colonna “In Sviluppo” deve contenere esattamente 2 card (o meno), e devi saper indicare quale card verrebbe “sbloccata” per prima quando una delle due card attuali viene completata.
3. **Calcola Lead Time e Cycle Time con date a tua scelta.** Inventi una card con tre date: quando entra in “To Do”, quando qualcuno la prende in carico (“In Sviluppo”), quando viene completata (“Done”). Calcola a mano Lead Time e Cycle Time, ed evidenzia quanti giorni sono di “attesa in coda”.  **Come verificare:** Lead Time deve sempre essere **maggiore o uguale** al Cycle Time (mai il contrario) — se il tuo calcolo dà un Cycle Time più alto del Lead Time, hai invertito una data.
4. **Osserva la board reale del team.** Chiedi a un collega di farti vedere la board Kanban (o Scrum) usata realmente nel progetto: annota quante colonne ha, se ci sono limiti di WIP visibili e su quali colonne, e quante card ci sono in ciascuna colonna in questo momento.  **Come verificare:** sei in grado di riportare, senza guardare di nuovo la board, il nome esatto di tutte le colonne e almeno un esempio di card per ciascuna.
5. **Individua un possibile collo di bottiglia.** Guardando la stessa board reale (o quella dell'esercizio 1), individua se c'è una colonna con più card delle altre, e formula un'ipotesi sul perché (manca una competenza? un'approvazione lenta? una sola persona sovraccarica?).  **Come verificare:** prova la tua ipotesi con un collega o con la tua collega Scrum Master/PM — se conferma (anche parzialmente) la tua lettura, hai capito il concetto.
6. **Confronta Scrum e Kanban su un caso reale.** Pensa a un tipo di richiesta che arriva spesso nel progetto (es. un bug urgente in produzione, oppure una nuova feature pianificata) e scrivi 2-3 righe su quale dei due approcci (Scrum a sprint, o Kanban a flusso continuo) si adatterebbe meglio a gestirla, e perché.  **Come verificare:** la tua risposta deve citare almeno uno dei criteri visti nella tabella della sezione 7.5 (es. prevedibilità del lavoro, necessità di “saltare la fila”, cadenza di consegna).

Collegamenti

- [4. Git e GitHub](#) — dove Issue e Pull Request diventano le card di una board Kanban costruita con GitHub Projects
- [8. Project Management](#) — come usare le metriche di flusso (Lead Time, Cycle Time, Throughput) per monitorare un progetto

- [Kanbanize — What is Kanban?](#)
- [Atlassian — Kanban](#)
- [Atlassian — Lead Time vs Cycle Time](#)
- [Atlassian — What is WIP limit?](#)
- [Kanban University — What is Kanban?](#)
- [Scrumban — Definizione ed esempi](#)

8. Project Management

Hai già visto Agile, Scrum e Kanban: sai quindi **come** un team organizza il lavoro giorno per giorno (sprint, board, colonne, cerimonie). Questa sezione affronta un livello diverso, complementare: quello del **Project Management**, cioè l'insieme di strumenti e pratiche che permettono di tenere sotto controllo un progetto nel suo complesso — chi è coinvolto, cosa può andare storto, a che punto siamo, e come lo comunichiamo a chi non è nella daily standup ogni giorno.

Attenzione a un equivoco comune: il Project Management **non** è la pianificazione rigida “a cascata” (Waterfall) che magari hai studiato all'università, con Gantt chart bloccati e fasi sequenziali immutabili.

Per capire perché questo corso non torna a quel modello, vale la pena vedere cosa succedeva davvero con Waterfall, non solo definirlo. In un progetto a cascata tipico, i requisiti vengono **congelati mesi prima** in un documento lungo, approvato una volta. Il team sviluppa per mesi seguendo quel documento, senza mostrare nulla al cliente fino alla fine — la **prima verifica reale** avviene solo alla consegna. È a quel punto che emerge il problema: il software corrisponde esattamente al documento, ma nel frattempo il mercato o la comprensione del problema sono cambiati, e il cliente scopre di aver ricevuto — con grande precisione tecnica — **la cosa sbagliata**. Non è un errore di programmazione: è un difetto strutturale dell'aver bloccato i requisiti troppo presto e verificato troppo tardi — l'argomento che spesso convince uno sponsor scettico a non chiedere “un Gantt dettagliato di tutto il progetto” il primo giorno.

In un contesto Agile/DevOps, il project management convive con l'iterazione continua e l'incertezza: si pianifica, sì, ma a più livelli e con la consapevolezza che il piano cambierà. Il ruolo del Project Manager (o di chi ne fa le funzioni, come uno Scrum Master “aumentato” o un Delivery Manager) è più simile a un **navigatore che aggiorna la rotta ogni giorno** che a un ingegnere che disegna un progetto immutabile su carta.

Anche in questa sezione useremo come filo conduttore il team di **ShopFacile**, la piattaforma e-commerce che hai già incontrato: **Sara** (Product Owner) sarà la voce del business tra gli stakeholder, **Luca** (Scrum Master) seguirà da vicino processo e rischi — RAID Log, RACI, monitoraggio — mentre **Marco**, **Giulia** e **Ahmed**, il team di sviluppo, daranno concretezza agli esempi di KPI e reportistica.

Obiettivi della sezione

Alla fine di questa sezione saprai:

- identificare gli **stakeholder** di un progetto software e capire perché vanno mappati;
- usare un **RAID Log** per tenere traccia di rischi, assunzioni, problemi e dipendenze;
- leggere e costruire una **matrice RACI** per chiarire le responsabilità;
- distinguere una **milestone** da una semplice scadenza;
- valutare un **rischio** in termini di probabilità e impatto, e capire come si mitiga;
- capire come funziona la **pianificazione a più livelli** in un contesto Agile (roadmap, release plan, sprint plan);

- riconoscere gli strumenti di **monitoraggio** dell'andamento di un team (burndown/burnup, stand-up, dashboard);
- conoscere i principali **KPI** usati in un team Agile/DevOps;
- distinguere i diversi livelli di **reportistica**, a seconda di chi la legge;
- capire cosa sono il **PMI**, la certificazione **PMP** e la **PMBOK Guide**, e come si collocano rispetto ad Agile/Scrum;
- riconoscere i **5 process group** e le **10 knowledge area** del PMBOK, collegandoli agli strumenti già visti nel corso;
- scrivere un **project charter** essenziale e distinguere una **WBS** (scomposizione del lavoro) da un Product Backlog;
- leggere un **percorso critico (critical path)** semplice e interpretare le metriche base dell'**Earned Value Management** (PV, EV, AC, SPI, CPI);
- muoverti con sicurezza in un contesto **ibrido**, che unisce pratiche PMP e pratiche Agile, come accade in molte aziende reali;
- riconoscere in **Jira** la gerarchia Epic/Story/Task/Sub-task e leggere backlog, board e Gantt come viste diverse sugli stessi dati;
- usare **Excel** con criterio per WBS e reportistica, sapendo quando è la scelta giusta e quando invece serve lo strumento condiviso del team;
- distinguere il ruolo del **Solution Architect** da quello di Tech Lead, Product Owner ed Enterprise Architect, e capire quando coinvolgerlo.

8.1 Stakeholder: chi ha interesse nel progetto

Uno **stakeholder** (letteralmente “portatore di interesse”) è **qualsiasi persona o gruppo che può influenzare il progetto, oppure che ne è influenzato**. Non sono solo “quelli che pagano”: sono tutti quelli che, in un modo o nell'altro, hanno voce in capitolo o subiscono le conseguenze delle decisioni prese.

 **Analogia:** organizzare un progetto software senza mappare gli stakeholder è come organizzare un matrimonio pensando solo agli sposi. In realtà ci sono genitori che pagano parte del conto, invitati con esigenze alimentari particolari, il fotografo che deve essere informato sugli orari, il comune che deve validare i documenti. Se ignori qualcuno di questi soggetti, il giorno dell'evento emergono problemi che potevi prevedere con un minimo di mappatura.

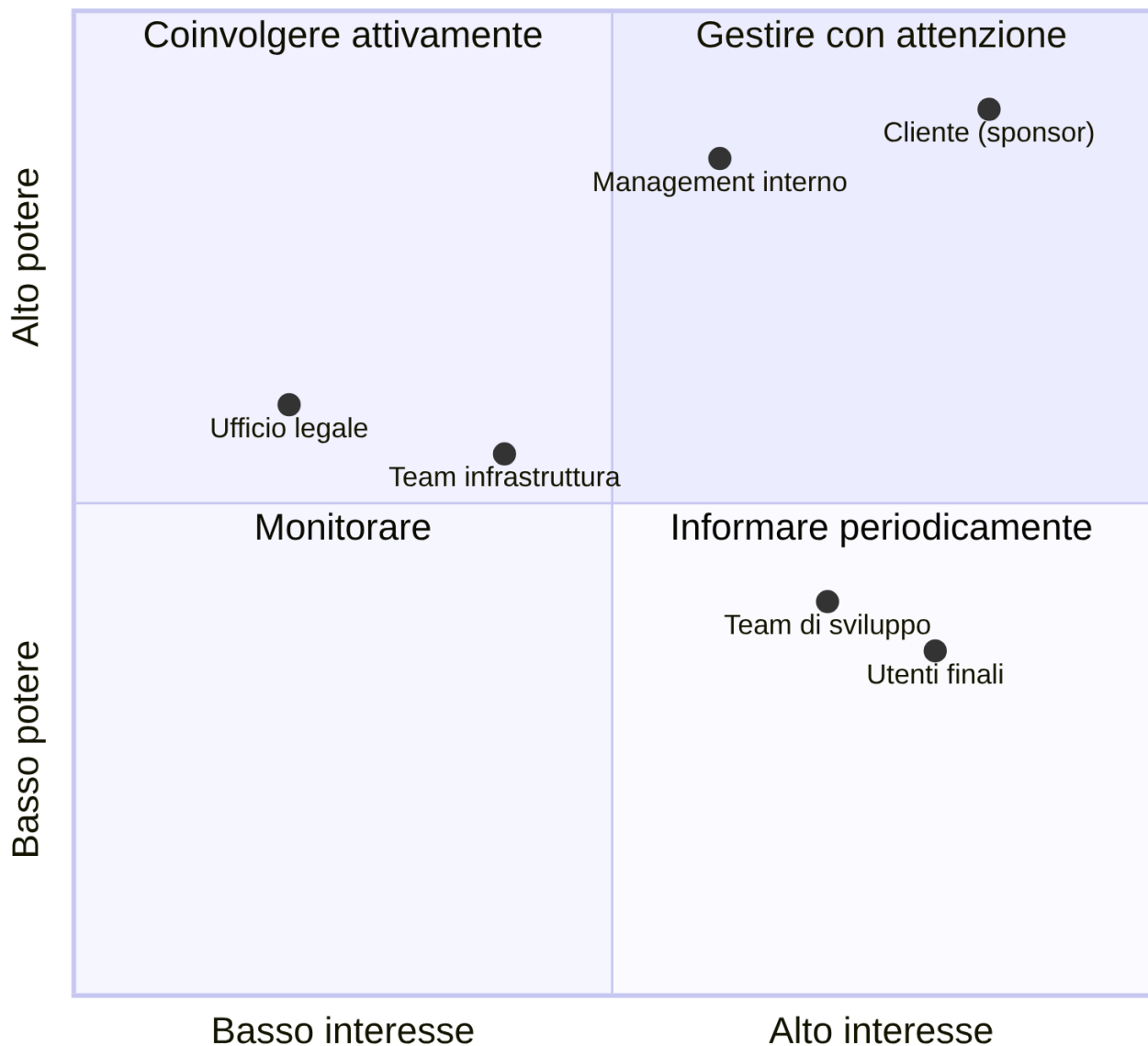
In un progetto software tipico, gli stakeholder principali sono:

- **Il cliente** (o il committente): chi ha commissionato il progetto e ne finanzia lo sviluppo. Vuole vedere valore consegnato, rispetto dei tempi e dei costi concordati.
- **Il management** (interno all'azienda che sviluppa): responsabili di linea, direttori, chi deve rendere conto a sua volta di come vanno i progetti del team.
- **Gli utenti finali:** le persone che useranno concretamente il software ogni giorno. Spesso non partecipano alle decisioni, ma sono chi subisce più direttamente le conseguenze di scelte sbagliate (un'interfaccia scomoda, una funzionalità mancante).

- **Il team:** sviluppatori, tester, designer, chiunque contribuisca alla realizzazione. Sono stakeholder anche loro: un carico di lavoro insostenibile o requisiti ambigui li riguardano direttamente.
- **Altri team o reparti:** un team di sicurezza che deve validare il rilascio, un team di infrastruttura che gestisce i server, un ufficio legale che deve approvare termini contrattuali.

Una mappatura semplice, spesso usata nella pratica, classifica gli stakeholder in base a due dimensioni: **quanto potere/influenza hanno** sul progetto e **quanto interesse** hanno nel suo esito.

Mappa degli stakeholder (potere vs interesse)



Esempio pratico: nel progetto ShopFacile per una nuova funzionalità di pagamento online, il cliente/committente ha alto potere e alto interesse (va coinvolto attivamente in ogni decisione importante) — nella pratica di ShopFacile è **Sara**, la Product Owner, a rappresentare questa voce del business nelle decisioni quotidiane; il team di sicurezza interno ha alto potere ma interesse più episodico (va coinvolto nei momenti critici, come la validazione prima del rilascio in produzione); gli utenti finali hanno alto interesse ma poco potere diretto sulle decisioni (vanno rappresentati tramite ricerche utente, feedback, test di usabilità).

Perché questo conta per un Junior Project Manager? Perché la causa più comune di problemi in un progetto non è “il codice non funziona”, ma **qualcuno importante non è stato informato o coinvolto al momento giusto**. Mappare gli stakeholder all’inizio ti evita sorprese a metà progetto — ma sapere *chi* coinvolgere non basta se non hai anche un posto dove tracciare cosa può andare storto: è il compito del RAID Log, che vediamo subito.

8.2 RAID Log: la mappa dei problemi (prima che diventino problemi)

Immagina questa situazione, capitata più volte in team come quello di ShopFacile: solo una persona sa che il fornitore di hosting ha in programma una manutenzione delicata — ma va in ferie senza scriverlo da nessuna parte. Nello stesso periodo, il rilascio di una funzionalità dipende da un fornitore esterno di pagamenti, ma nessuno ha mai tracciato formalmente questa dipendenza né chiesto una data di conferma. Quando i due problemi si sommano, il progetto si **ferma per due settimane**, e il team scopre solo allora che entrambe le informazioni erano note a qualcuno, ma non condivise né scritte da nessuna parte. È esattamente il problema che il RAID Log esiste per prevenire.

RAID è un acronimo che raggruppa quattro categorie di informazioni che un team di progetto deve tracciare costantemente:

- **R — Risks** (Rischi): eventi che *potrebbero* accadere e che, se accadono, avrebbero un impatto negativo sul progetto.
- **A — Assumptions** (Assunzioni): cose che stiamo dando per scontate, senza averle verificate al 100%, per poter procedere con la pianificazione.
- **I — Issues** (Problemi): cose che *sono già accadute* e stanno attivamente impattando il progetto ora.
- **D — Dependencies** (Dipendenze): elementi esterni al controllo diretto del team, di cui il progetto ha bisogno per procedere.

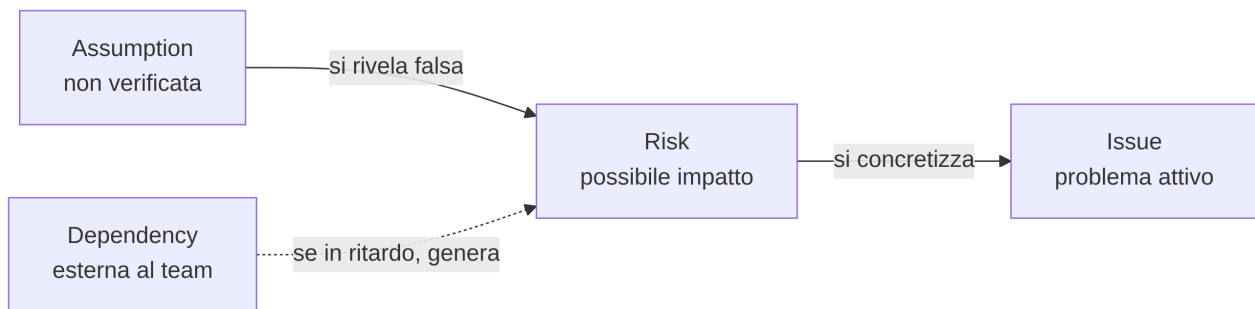
 **Analogia:** il RAID Log è come il cruscotto di un’automobile durante un lungo viaggio. Il rischio è la spia della benzina che si accende prima che tu resti a secco (un avviso preventivo). L’assunzione è il presupposto “ci sarà un distributore aperto lungo la strada” — non l’hai verificato, ma stai viaggiando come se fosse vero. Il problema è il pneumatico che si è già bucato: è già successo, va gestito subito. La dipendenza è il fatto che, per attraversare un ponte, devi aspettare che apra alle 8, che tu lo voglia o no: non lo controlli, ma ne dipendi.

Un esempio concreto di ciascuna categoria, per un progetto software reale:

Categoria	Esempio concreto
Risk	"C'è il rischio che il carico di traffico previsto per il Black Friday superi la capacità attuale dei server, causando lentezza o down del sito." (non è ancora successo, ma è possibile)
Assumption	"Stiamo pianificando la release assumendo che l'ambiente di test resti disponibile per tutto il mese; non abbiamo una conferma scritta dal team infrastruttura."
Issue	"Il servizio di invio email di terze parti è down da ieri: gli utenti non ricevono le email di conferma registrazione." (è già un problema attivo, in corso)
Dependency	"Il rilascio della nuova funzionalità di pagamento dipende dall'approvazione di sicurezza da parte di un team esterno al progetto, previsto per la settimana prossima."

Nella pratica, un RAID Log è semplicemente una tabella condivisa (spesso in un foglio, in una board dedicata, o in una Issue "fissata" in cima al repository GitHub del progetto), aggiornata regolarmente, con colonne tipo: descrizione, categoria, responsabile, impatto, stato, data di apertura, azione prevista.

Un aspetto importante da capire: le **assunzioni**, se non verificate per tempo, diventano facilmente **rischi**; e i **rischi**, se non mitigati per tempo, diventano **issue** attive. Il RAID Log serve proprio a intercettare questa "scala di gravità" prima che il problema esploda a valle.



Esempio pratico: un estratto reale (semplificato) di RAID Log a metà dello Sprint 6 di ShopFacile, così come potrebbe apparire in una board condivisa con il team. In ShopFacile non esiste un Project Manager dedicato: le responsabilità di "Tech Lead" e "Project Manager" viste in tabella sono spesso portate avanti da **Luca**, insieme al team, accanto al suo ruolo di Scrum Master:

ID	Categoria	Descrizione	Responsabile	Impatto (1-5)	Stato	Aperto il
R-12	Risk	Il fornitore di hosting ha comunicato una finestra di manutenzione non pianificata nella settimana del rilascio	Luca (Tech Lead)	4	Aperto	03/06
R-13	Risk	Se non testiamo il carico prima del Black Friday, rischio di rallentamenti sotto picco di traffico	Giulia (Developer/ QA)	5	In mitigazione	05/06
A-07	Assumption	Stiamo assumendo che l'ambiente di staging resti disponibile fino a fine mese	Luca (Project Manager)	3	Da verificare	01/06
A-08	Assumption	Assumiamo che Sara fornisca il feedback sul design entro 3 giorni lavorativi	Luca (Project Manager)	2	Da verificare	04/06
I-04	Issue	Il servizio di invio SMS di terze parti restituisce errori 500 dal 06/06	Marco (Sviluppatore backend)	4	In corso	06/06
D-05	Dependency	Il rilascio del modulo pagamenti dipende dall'approvazione del team sicurezza, prevista per il 12/06	Luca (Tech Lead)	5	In attesa	02/06

Con soli 6 righe, un Project Manager junior può già farsi una domanda utile: **quali elementi con impatto alto (4-5) non hanno ancora un'azione di mitigazione chiara?** In questo estratto sono tre (R-13, I-04, D-05): sono i primi tre di cui parlare nel prossimo check-in con il cliente, prima che diventino la causa di uno slittamento non comunicato per tempo.

8.3 RACI: chi fa cosa (e chi deve solo saperlo)

Una delle domande più comuni — e più fonte di attriti — in un progetto è: **“chi decide su questo?”** o **“a chi tocca fare questa cosa?”**. La matrice **RACI** è uno strumento semplice per chiarirlo prima che diventi un problema, assegnando a ogni attività uno dei quattro ruoli:

- **R — Responsible** (Esecutore): chi materialmente **fa il lavoro**.
- **A — Accountable** (Responsabile finale): chi **risponde del risultato** davanti a tutti — è la persona che, se qualcosa va storto, deve rispondere. Per ogni attività dovrebbe essercene **uno solo**, per evitare ambiguità.
- **C — Consulted** (Consultato): chi viene **coinvolto per un parere** prima che la decisione/attività sia completata, con una comunicazione a doppio senso (gli chiedi qualcosa, lui risponde).
- **I — Informed** (Informato): chi viene **aggiornato dopo il fatto**, con comunicazione a senso unico (non partecipa alla decisione, ma deve saperlo).

 **Analogia:** pensa a un ristorante durante il servizio. Il cameriere che porta il piatto al tavolo è il *Responsible*: esegue l'azione. Lo chef di cucina è l'*Accountable*: risponde della qualità del piatto che esce dalla sua cucina. Il sommelier viene *Consulted* se il cliente chiede un abbinamento particolare di vino. Il titolare del ristorante è *Informed*: alla fine della serata gli viene detto quanti coperti sono stati serviti, ma non ha partecipato a nessun piatto.

Esempio pratico: matrice RACI per l'attività “rilascio di una nuova funzionalità in produzione” — lo stesso rilascio del modulo pagamenti visto nel RAID Log di ShopFacile qui sopra. I ruoli della tabella restano generici (così puoi riusarla per qualsiasi progetto), ma nel team ShopFacile sono coperti così: Sviluppatore → **Ahmed**, Tech Lead/Project Manager → **Luca**, QA/Tester → **Giulia**, Cliente → **Sara**, in quanto voce del business.

Attività	Sviluppatore	Tech Lead	Project Manager	QA / Tester	Cliente
Sviluppo del codice	R	C	I	I	—
Code review	C	A / R	I	I	—
Test della funzionalità	I	C	I	R	—
Decisione go/no-go al rilascio	I	C	A	C	I
Esecuzione del rilascio (deploy)	C	R	I	I	—
Comunicazione al cliente dell'avvenuto rilascio	I	I	R / A	I	I

Qualche osservazione utile da tenere a mente quando costruisci una RACI:

- Per ogni riga ci dovrebbe essere **esattamente una A** (Accountable): se nessuno è accountable, nessuno risponde davvero del risultato; se ce ne sono troppe, si diluisce la responsabilità e nelle emergenze “si scaricano” a vicenda.
- Una stessa persona può essere sia R che A sulla stessa attività (come il Tech Lead nella code review, sopra), specialmente in team piccoli.
- Troppi “C” su un’attività la rendono lenta: ogni consultazione è un potenziale collo di bottiglia. Usa “C” solo dove il parere serve davvero.

Il Solution Architect: chi tiene insieme la soluzione tra i team

La RACI qui sopra elenca ruoli di progetto — Sviluppatore, Tech Lead, PM, QA, Cliente — ma su un progetto più grande di ShopFacile, con più team che lavorano su sistemi diversi che devono parlarsi tra loro, manca spesso un ruolo che nessuno dei precedenti coprirebbe davvero.

Immagina questo scenario, plausibile appena l'azienda ha più di un team: il team di **Marco** decide di gestire l'autenticazione degli utenti con un proprio sistema di login; in parallelo, un altro team che lavora sull'app mobile di ShopFacile decide, indipendentemente e per motivi altrettanto ragionevoli, di implementarne uno diverso; un terzo team, quello che si occupa delle notifiche, scopre solo a integrazione quasi ultimata che nessuno dei due sistemi di login gli fornisce l'informazione di cui ha bisogno per inviare un avviso push. Ogni scelta, presa da sola, era sensata. Il risultato **complessivo** non sta in piedi: due sistemi che risolvono lo stesso problema in modo incompatibile, e un'integrazione che nessuno ha progettato perché apparteneva esattamente al confine tra due team, quindi a nessuno.

Il **Solution Architect** è il ruolo che esiste per questo: tiene insieme la soluzione **tra** i sistemi e **tra** i team, non dentro a uno solo. In concreto:

- traduce i requisiti di business in scelte tecniche, e — altrettanto importante — traduce i vincoli tecnici in conseguenze di business (costi, tempi, rischi) che uno sponsor può capire senza background tecnico;
- decide anche dove **non** costruire (usare un servizio già esistente invece di farne un altro identico in un altro team);
- presidia i **requisiti non funzionali** del sistema nel suo complesso — li hai già incontrati nella sezione 3 — perché sono quasi sempre loro, non i requisiti funzionali, a rompersi ai confini tra sistemi diversi (un sistema pensato per 100 richieste al secondo che deve integrarsi con uno che ne sopporta 10, per esempio).

Un ruolo con un nome simile, ma con un ambito diverso, genera confusione quasi automatica: eccola chiarita in una tabella.

Ruolo	Ambito	Orizzonte temporale	Tipo di decisione	Risponde a
Solution Architect	Una soluzione specifica, che attraversa più sistemi/team	Il progetto o programma in corso	Come i sistemi coinvolti si integrano tra loro	Sponsor del progetto, Enterprise Architect (se presente)
Enterprise Architect	Tutta l'organizzazione, tutti i sistemi	Anni, a prescindere dal singolo progetto	Standard tecnologici, quali sistemi l'azienda deve avere nel suo insieme	Direzione IT/CTO
Tech Lead	Un singolo team	Lo sprint o la release corrente	Come si scrive e si organizza il codice di quel team	Il team, e spesso il Solution Architect
Product Owner	Il valore per l'utente di un prodotto	Il backlog, sprint dopo sprint	Cosa costruire, in che ordine di priorità	Il business/gli stakeholder

Cosa significa questo, in pratica, per un Junior Project Manager? Il Solution Architect è l'interlocutore giusto per capire se una richiesta è “grande” **prima** di stimarla con il team — e va coinvolto prima, non dopo: se lo coinvolgi solo a valle, quando le scelte tecniche sono già state fatte team per team, l'unico ruolo che gli resta è quello di dire no a integrazioni che a quel punto sono già difficili da cambiare.

Come ogni ruolo di questo tipo, anche il Solution Architect ha un trade-off da conoscere. Un architetto troppo lontano dal codice — che disegna soluzioni sulla carta senza mai verificarne la fattibilità con chi scrive davvero il software — produce disegni eleganti che poi non si realizzano: è il rischio noto come “**ivory tower architect**” (architetto nella torre d'avorio). Il rischio opposto è altrettanto reale: un architetto che vive dentro un solo team perde esattamente la visione d'insieme che dovrebbe garantire.

La contromisura più efficace contro entrambi gli estremi è la stessa: un Solution Architect che scrive **ADR** per le decisioni importanti (li hai già incontrati nella sezione 3) e le discute apertamente con i team coinvolti, invece di deciderle in isolamento e comunicarle già chiuse.

Una RACI ben fatta chiarisce chi fa cosa lungo il percorso di un rilascio, ma non dice ancora **quando** quel rilascio conta davvero per chi guarda il progetto da fuori. È qui che entra in gioco il concetto di milestone.

8.4 Milestone: un traguardo, non una scadenza qualsiasi

Una **milestone** (pietra miliare) è un **punto significativo** nel percorso del progetto: segna il raggiungimento di qualcosa di rilevante, non semplicemente il passaggio di una data sul calendario.

 **Analogia:** pensa a una gara ciclistica a tappe. Ogni tappa ha ovviamente una data di arrivo, ma alcuni punti del percorso sono **traguardi volanti** o **cime di montagna**: superarli non è solo “un altro chilometro fatto”, è un momento che tutti notano, che dà punteggio a parte, che segna davvero un salto di fase della corsa. Una scadenza qualsiasi è come un cartello chilometrico: utile per orientarsi, ma non cambia la narrazione della corsa. Una milestone è il traguardo di tappa.

La differenza pratica:

	Scadenza semplice	Milestone
Cosa segna	Il completamento di un compito operativo	Il raggiungimento di un obiettivo significativo per il progetto
Visibilità	Interna al team, spesso operativa	Comunicata anche a stakeholder esterni
Esempio	“Completare la migrazione del database di test entro venerdì”	“Ambiente di test pronto e validato dal cliente”
Conseguenza se manca	Si riorganizza il lavoro della settimana	Può avere impatto su comunicazioni contrattuali, pianificazione di altri team, aspettative del cliente

Esempi concreti di milestone in un progetto software: “fine della fase di analisi dei requisiti”, “prima release disponibile per un gruppo pilota di utenti (beta)”, “completamento dei test di sicurezza”, “go-live in produzione”, “primo mese di produzione senza incidenti critici”. Non sono attività in sé, ma **traguardi** che raccontano un salto di stato del progetto — per ShopFacile, il go-live del nuovo modulo pagamenti che abbiamo visto nel RAID Log e nella RACI è esattamente questo tipo di milestone: non un task tra tanti, ma un momento che Sara comunicherà anche al di fuori del team.

In un contesto Agile, le milestone convivono bene con gli sprint: uno sprint può contribuire a una milestone senza esaurirla, e una milestone spesso coincide con la fine di una serie di sprint (ad esempio, la fine di una release che raggruppa più sprint). Ma un traguardo così visibile porta con sé anche più da perdere se qualcosa va storto lungo il percorso — ed è proprio quello che il prossimo paragrafo insegna a gestire.

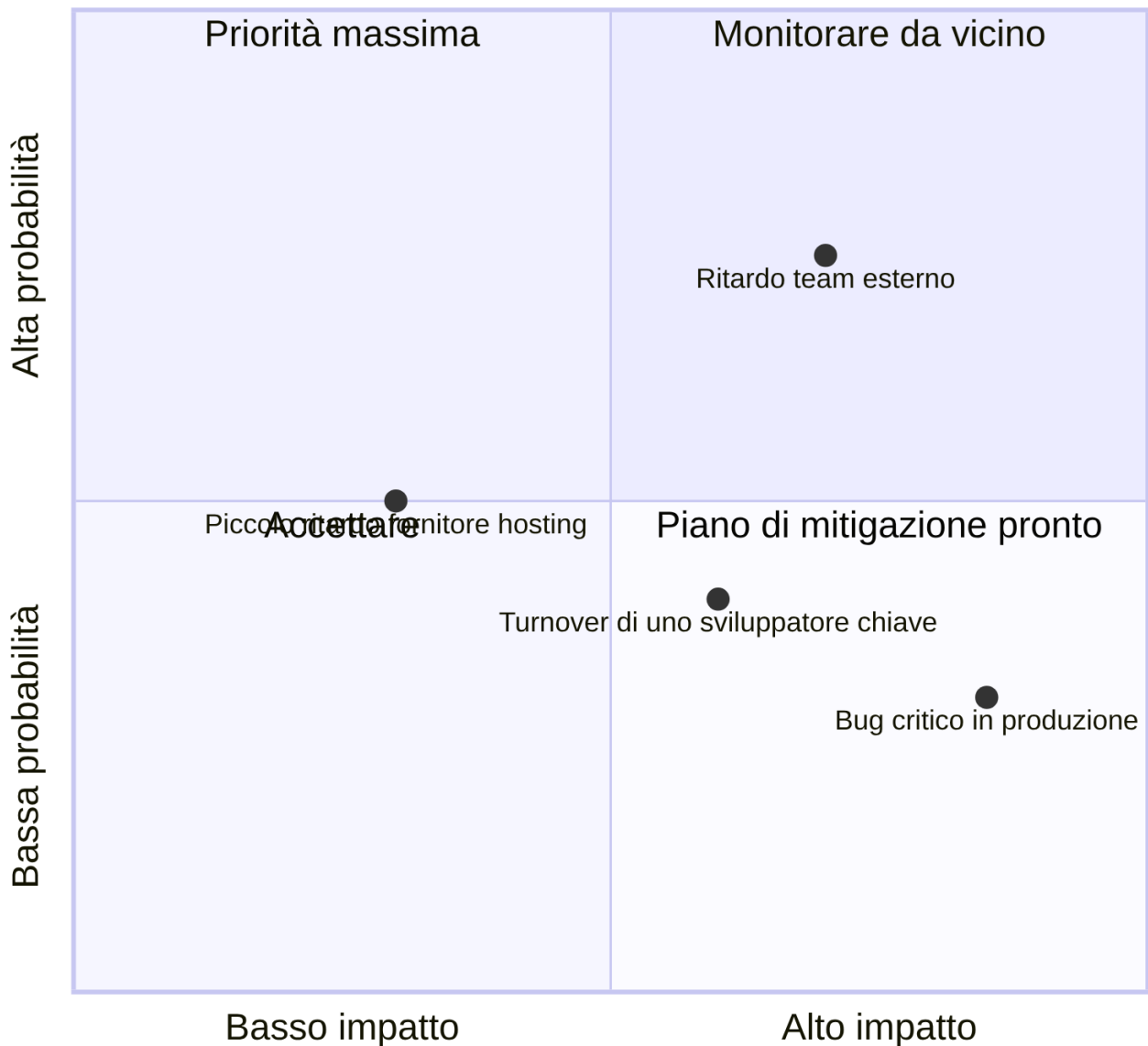
8.5 Rischio: identificare, valutare, mitigare

Un **rischio** è un evento incerto che, se si verifica, ha un impatto (di solito negativo, anche se esistono rischi “positivi”, cioè opportunità) sul progetto. Gestire i rischi non significa eliminarli tutti — è impossibile — ma **conoscerli in anticipo** e decidere consapevolmente cosa fare.

Il processo tipico ha tre fasi:

1. **Identificazione**: elencare cosa potrebbe andare storto. Si fa in team, spesso in un momento dedicato (es. durante la pianificazione di una release), con tecniche semplici come il brainstorming o rivedendo i rischi materializzati in progetti simili passati.
2. **Valutazione**: per ogni rischio, stimare **probabilità** (quanto è plausibile che accada) e **impatto** (quanto danno farebbe se accadesse). Spesso si usa una scala semplice (basso/medio/alto) e si visualizza in una matrice.
3. **Mitigazione**: decidere una strategia. Le opzioni classiche sono:
 - **evitare** il rischio (cambiare approccio per non incontrarlo);
 - **ridurre** probabilità o impatto (azioni preventive);
 - **trasferire** il rischio (es. a un fornitore, con un'assicurazione contrattuale);
 - **accettare** il rischio consapevolmente, se il costo di mitigarlo supera il danno potenziale, tenendo comunque un piano B pronto.

Matrice probabilità/impatto



Esempio concreto: “rischio di ritardo per dipendenza da un altro team” — lo stesso rischio D-05 già anticipato nel RAID Log di ShopFacile.

- **Descrizione del rischio:** la nuova funzionalità di pagamento di ShopFacile richiede che il team di infrastruttura fornisca un nuovo ambiente sicuro entro due settimane; quel team ha già altri due progetti in corso.
- **Probabilità:** media-alta (il team esterno è oggettivamente sovraccarico, lo si vede dal loro backlog).
- **Impatto:** alto (senza quell'ambiente, il rilascio non può partire, e la data comunicata al cliente è a rischio).

- **Mitigazione:** Luca decide di chiedere una conferma scritta della data al team infrastruttura appena possibile (per trasformare l'assunzione in un impegno verificato); valuta con il team un piano B con un ambiente temporaneo meno performante ma disponibile prima; e si accorda con Sara per comunicare per tempo al cliente che esiste un rischio, invece di farlo scoprire a due giorni dalla scadenza.

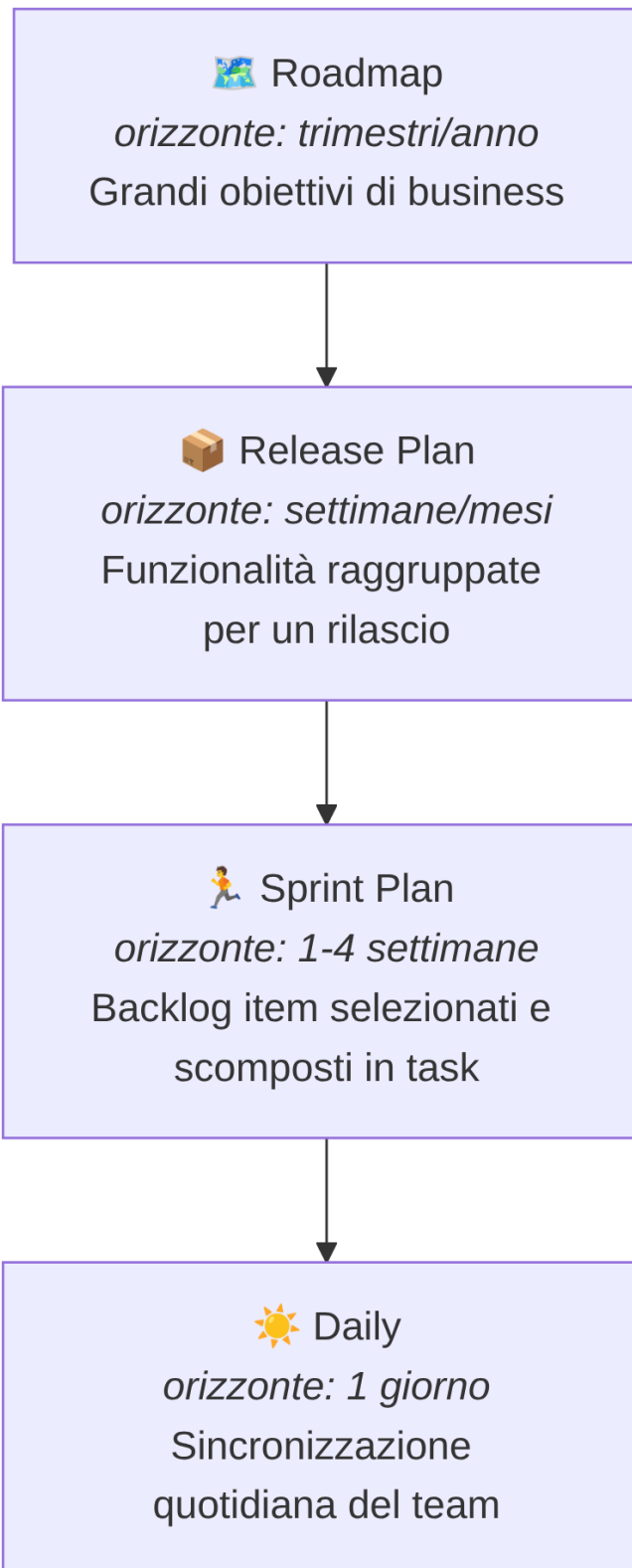
Da notare come questo esempio collega direttamente rischio, assunzione e dipendenza — gli stessi concetti visti nel RAID Log: non sono strumenti separati, ma **lenti diverse sullo stesso terreno**.

8.6 Pianificazione in un contesto Agile: livelli, non un unico piano

Nel project management “a cascata” tradizionale, l'idea è pianificare tutto in anticipo, in un unico grande piano dettagliato. In un contesto Agile/DevOps si pianifica invece **a più livelli**, con dettaglio crescente man mano che ci si avvicina all'esecuzione — perché sappiamo che i dettagli lontani nel tempo cambieranno comunque, mentre quelli vicini devono essere affidabili.

I livelli tipici, dal più generale al più operativo:

1. **Roadmap:** la visione a lungo termine (mesi/trimestri/anno), con i grandi temi e obiettivi di business. Non contiene dettagli tecnici, ma direzioni: nella roadmap di ShopFacile che Sara porta agli stakeholder potrebbe leggersi “nel primo trimestre miglioriamo l'esperienza di checkout”, “nel secondo trimestre lanciamo l'app mobile”.
2. **Release Plan:** un livello più concreto, che raggruppa più sprint per arrivare a un rilascio significativo per gli utenti o il cliente. Qui iniziano a comparire funzionalità specifiche e stime di massima.
3. **Sprint Plan:** il piano del singolo sprint (di solito 1-4 settimane, come hai visto nella sezione su Scrum), con gli elementi di backlog selezionati e scomposti in attività concrete.
4. **Daily:** l'allineamento quotidiano del team (la daily standup), dove si sincronizza il lavoro del singolo giorno rispetto all'obiettivo dello sprint.



Il punto chiave da capire, come futuro Project Manager: **più ti allontani nel tempo, meno il piano deve essere dettagliato — e va bene così**. Una roadmap troppo dettagliata è quasi sempre sbagliata (perché il futuro lontano è incerto), mentre uno sprint plan troppo vago è inutile (perché il team deve poter lavorare su qualcosa di concreto già domani mattina). Il tuo compito non è “prevedere tutto”, ma tenere questi livelli **coerenti tra loro** e aggiornarli quando la realtà li smentisce.

Avere un piano a più livelli, però, non ti dice ancora se ShopFacile lo sta effettivamente rispettando giorno per giorno: per quello serve osservare l'andamento reale del team, non solo il piano scritto.

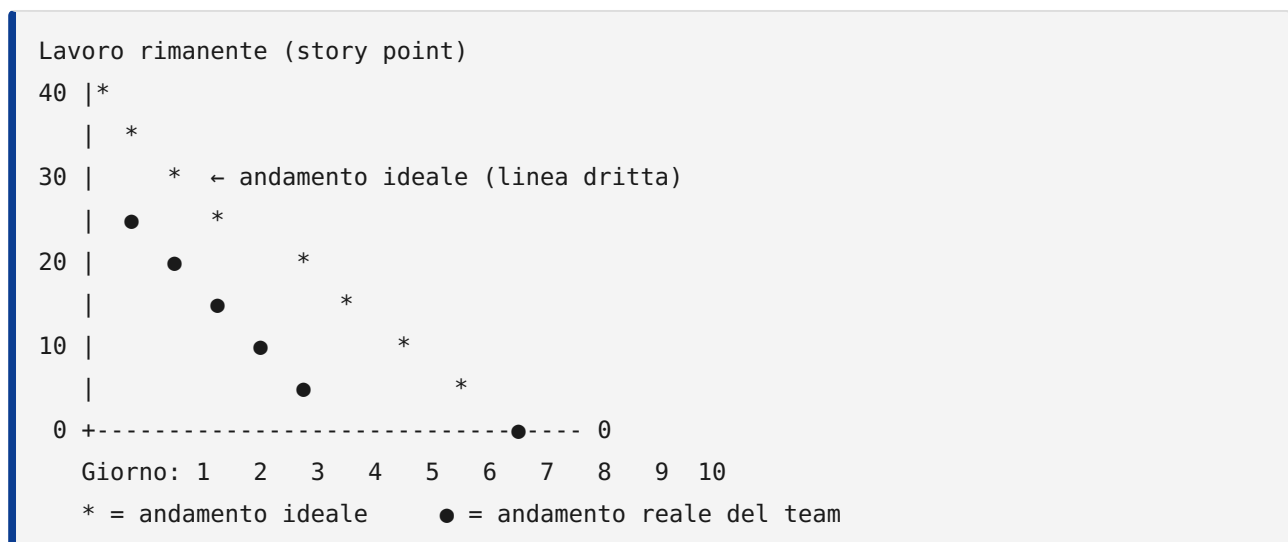
8.7 Monitoraggio: come si tiene traccia dell'andamento

Pianificare non basta: bisogna anche **verificare costantemente** se si sta procedendo come previsto, per poter reagire in tempo (non a fine progetto, quando è troppo tardi).

Gli strumenti principali di monitoraggio in un contesto Agile:

- **Daily stand-up**: il momento quotidiano (di solito 15 minuti, in piedi) in cui il team si sincronizza su cosa è stato fatto, cosa si farà e quali ostacoli ci sono. È monitoraggio “in tempo reale”, fatto dal team per il team.
- **Burndown chart**: mostra quanto lavoro **rimane** da fare in uno sprint (o in una release), giorno per giorno. La linea scende verso zero man mano che il lavoro si completa; una linea ideale tratteggiata mostra dove “dovremmo essere” per finire in tempo.
- **Burnup chart**: mostra quanto lavoro **è stato completato**, rispetto al totale previsto — utile soprattutto se lo scope cambia durante il progetto (con il burnup vedi anche se il “totale” sale, mentre il burndown da solo può confondere un aumento di scope con un rallentamento del team).
- **Dashboard**: pannelli visuali (spesso in GitHub Projects, Jira, o simili) che aggregano lo stato del backlog, delle Pull Request aperte, dei risultati delle pipeline di CI/CD, in un colpo d'occhio.

Un burndown chart “sano” tende ad avvicinarsi a una linea diagonale che scende gradualmente verso zero. Una rappresentazione testuale semplificata, per capire visivamente la forma:



In questo esempio, il team di ShopFacile parte più lento del previsto (la linea reale sta sopra quella ideale nei primi giorni — un possibile segnale di rischio da discutere in stand-up), ma recupera nella seconda metà dello sprint, chiudendo comunque a zero entro la fine. È tipicamente **Luca**, in quanto Scrum Master, a tenere d'occhio questo burndown e la dashboard del team ogni giorno: non guarda solo “se” la linea arriva a zero, ma **come** ci arriva — una discesa brusca solo nell'ultimo giorno è spesso il sintomo di lavoro “spuntato” tutto insieme a fine sprint, non di un reale progresso quotidiano.

Questi stessi grafici che Luca osserva ogni giorno sono anche la materia prima da cui si ricavano numeri di sintesi più stabili, utili per confrontare uno sprint con l'altro: i KPI.

8.8 KPI: gli indicatori chiave per un team Agile/DevOps

Un **KPI** (Key Performance Indicator, indicatore chiave di prestazione) è una misura quantitativa che aiuta a capire se le cose stanno andando bene, senza doversi fidare solo di impressioni soggettive (“mi pare che stiamo andando bene” non è un dato).

Alcuni KPI molto usati nei team Agile e DevOps:

KPI	Cosa misura	Perché è utile
Velocity	Quanti story point (o elementi di backlog) il team completa in media per sprint	Aiuta a pianificare quanto lavoro inserire negli sprint futuri, in modo realistico
Lead time	Tempo totale dalla richiesta di una funzionalità (apertura dell'elemento di backlog) al suo rilascio effettivo agli utenti	Misura quanto velocemente il team trasforma un'idea in valore consegnato
Cycle time	Tempo dall'inizio effettivo del lavoro su un elemento (non dalla richiesta) al suo completamento	Più mirato del lead time: misura l'efficienza del “fare”, non dell'attesa in coda
Defect rate	Numero di bug rilevati (spesso in produzione) rispetto al volume di lavoro consegnato	Segnala la qualità di quanto viene rilasciato
Deployment frequency	Quante volte si rilascia in produzione in un dato periodo	KPI tipicamente DevOps: più alta è, di solito, più il processo di rilascio è maturo e automatizzato
Mean Time To Recovery (MTTR)	Tempo medio per ripristinare il servizio dopo un incidente in produzione	Misura la resilienza del team/ sistema, non solo la sua velocità
Sprint goal success rate	Percentuale di sprint in cui l'obiettivo dello sprint viene effettivamente raggiunto	Indica se la pianificazione degli sprint è realistica

Un avvertimento importante, da futuro Project Manager: i KPI vanno usati per **capire e migliorare il processo**, non per giudicare o “punire” le singole persone del team (es. confrontare la velocity di due sviluppatori diversi è quasi sempre un errore metodologico, perché la velocity dipende dal team nel suo insieme, dalla complessità del lavoro assegnato, e non è comparabile tra persone o addirittura tra team diversi). Usato male, un KPI distrugge la fiducia del team; usato bene, aiuta tutti a vedere dove migliorare.

Esempio pratico: confronto tra due sprint consecutivi del team di ShopFacile, con alcuni KPI di sintesi:

KPI	Sprint 8	Sprint 9	Cosa può significare
Velocity	32 story point	24 story point	Da solo non basta a dire se è un problema: va guardato insieme agli altri KPI
Lead time medio	6 giorni	9 giorni	Le richieste restano più tempo "in coda" prima di essere completate
Defect rate	2 bug su 15 elementi consegnati	5 bug su 11 elementi consegnati	Segnale di qualità in calo, non solo di velocità
Deployment frequency	4 rilasci in produzione	1 rilascio in produzione	Il processo di rilascio ha subito un rallentamento
Sprint goal success rate	Raggiunto	Non raggiunto	Conferma che qualcosa, nello sprint, non ha funzionato come previsto

Guardando la sola velocity, si potrebbe pensare "il team ha lavorato di meno". Ma leggendo la riga del defect rate e quella della deployment frequency insieme, l'ipotesi più plausibile è diversa: il team ha probabilmente **rallentato per gestire un numero maggiore di bug**, riducendo sia la velocity che la frequenza dei rilasci. Un Project Manager che si fermasse al primo numero (velocity in calo = "il team lavora peggio") trarrebbe la conclusione sbagliata; guardare i KPI **come insieme coerente**, non uno per uno isolato, è quello che permette di fare la domanda giusta in retrospettiva: "cosa ha causato l'aumento dei bug nello Sprint 9?", invece di "perché avete lavorato di meno?".

Questi KPI, però, restano numeri grezzi finché qualcuno non li traduce in un messaggio comprensibile per chi non li guarda ogni giorno: è il compito della reportistica.

8.9 Reportistica: cosa riportare, a chi, e con quale livello di dettaglio

Riportare lo stato di avanzamento è una delle attività più quotidiane di un Project Manager, ma non tutti i report sono uguali: **il livello di dettaglio deve adattarsi a chi lo legge**.



Analogia: pensa a come racconti la tua giornata a persone diverse. A un collega stretto racconti ogni dettaglio tecnico di un problema che hai risolto. A tuo padre, la sera, dici semplicemente "giornata impegnativa ma è andata bene". Nessuna delle due versioni è "più vera" dell'altra: sono adatte a interlocutori diversi, con bisogni informativi diversi.

Due macro-categorie di report, con caratteristiche molto diverse:

	Report per il management/ cliente	Report operativi per il team
Frequenza tipica	Settimanale o mensile	Quotidiana o per sprint
Livello di dettaglio	Alto livello: stato generale, rischi principali, milestone raggiunte o a rischio	Dettagliato: singoli task, blocchi tecnici, chi sta facendo cosa
Linguaggio	Orientato al business, senza gergo tecnico	Tecnico, con riferimenti diretti a task, PR, ambienti
Contenuto tipico	Avanzamento % rispetto alla roadmap, KPI di sintesi, rischi da RAID Log con impatto su tempi/costi	Burndown/burnup dello sprint, stato delle Pull Request, esito delle pipeline CI/CD
Obiettivo	Dare fiducia e visibilità decisionale a chi non segue i dettagli ogni giorno	Coordinare il lavoro quotidiano e risolvere blocchi in tempo reale

Esempio pratico: la stessa informazione — “il rilascio della nuova funzionalità di pagamento di ShopFacile slitta di una settimana per un problema di sicurezza emerso nei test” — viene comunicata in due modi molto diversi:

- **Al cliente/management, a cura di Sara:** “Il rilascio della funzionalità di pagamento è posticipato di una settimana per garantire il rispetto degli standard di sicurezza richiesti. Non ci sono impatti sul budget. Nuova data prevista: [data].”
- **Al team, in stand-up o nella dashboard operativa, a cura di Luca:** “Il test di sicurezza automatizzato ha rilevato una vulnerabilità nella gestione dei token di sessione nel modulo di pagamento (vedi issue #128); serve un fix nel branch `fix/token-sicurezza` prima di riaprire la Pull Request e far ripartire la pipeline.”

Stessa realtà, due livelli di risoluzione informativa. Un errore comune dei Project Manager junior è portare al cliente il livello di dettaglio tecnico del team (che genera confusione e ansia inutile), oppure, al contrario, dare al team solo informazioni di alto livello (che non bastano per lavorare). Imparare a “tradurre” tra questi due livelli è una delle competenze più preziose del ruolo.

8.10 Il PMP e il PMBOK: che cosa sono e perché ti riguardano



Le prossime sottosezioni sono più lunghe delle precedenti: studiale per **riconoscere il vocabolario** che troverai in azienda, certificazioni e contratti — non per applicarlo integralmente ogni giorno, dato che il tuo lavoro quotidiano in un team come ShopFacile resterà per lo più Agile.

Un altro pezzo di vocabolario PMI, utile appena l'azienda avrà più di un progetto in corso, è la distinzione gerarchica tra **progetto**, **programma** e **portfolio**. Un **progetto** è uno sforzo temporaneo con obiettivo e fine definiti — il modulo pagamenti di ShopFacile. Un **programma** raggruppa progetti

correlati (es. pagamenti, UI del carrello, sconti in un programma “Rifacimento checkout”) perché coordinarli genera benefici in più. Un **portfolio** è il livello più alto: l'insieme di progetti e programmi di un'organizzazione, gestiti per allinearli alla strategia e decidere dove investire risorse limitate. Da Junior PM lavorerai quasi sempre sul singolo progetto, ma sapere che qualcuno decide priorità **tra** progetti spiega perché, a volte, un progetto in linea col piano viene comunque rallentato da una decisione presa più in alto.

Finora hai visto strumenti singoli — RAID Log, RACI, milestone, KPI — presi in prestito dalla pratica di progetto senza dire da dove arrivano. In realtà molti di questi strumenti fanno parte di un corpo di conoscenza molto più ampio e riconosciuto a livello internazionale: quello del **PMI**, il **Project Management Institute**, l'associazione professionale di riferimento per chi fa project management in senso tradizionale.

Perché è nato un corpo di conoscenza così formale? Il PMI nasce dal bisogno concreto di grandi progetti di ingegneria e costruzione, che sforavano **sistematicamente** costi e tempi previsti. Organizzazioni diverse dovevano collaborare sullo stesso progetto e comprare servizi da fornitori con un linguaggio diverso dal proprio: senza un vocabolario condiviso su cosa significa “rischio” o “scope”, ogni collaborazione ripartiva da zero solo per capirsi. Codificare tutto in un'unica guida è stata la risposta a quel problema concreto, non un esercizio accademico.

Il PMI pubblica una guida di riferimento chiamata **PMBOK Guide (Project Management Body of Knowledge)**, cioè “corpo di conoscenza del project management”): non un metodo rigido da applicare uguale ovunque, ma una raccolta organizzata di terminologia, processi e buone pratiche riconosciute a livello globale. È un po' come un grande manuale di riferimento — non un romanzo da leggere in ordine, ma un'enciclopedia da consultare per trovare il vocabolario giusto per un problema di progetto.

Il PMI offre anche una certificazione professionale molto conosciuta nel settore, il **PMP (Project Management Professional)**. In sintesi, senza entrare in numeri precisi che cambiano nel tempo, per ottenerla occorre tipicamente dimostrare:

- un certo numero di **ore di esperienza pratica** nella gestione di progetti (l'esatta soglia varia in base al titolo di studio posseduto);
- un certo numero di **ore di formazione specifica** in project management;
- il superamento di un **esame** che verifica la conoscenza dei processi, delle aree di conoscenza e la capacità di applicarli a scenari pratici.

 **Nota prudente:** i requisiti esatti (ore minime di esperienza e di formazione, formato e durata dell'esame) cambiano periodicamente e variano in base al percorso di studi di chi si candida. Se in futuro ti interesserà la certificazione, verifica sempre i requisiti aggiornati direttamente sul sito ufficiale del PMI, non su fonti secondarie.

Un punto che genera spesso confusione, e che è importante chiarire subito: **il PMP/PMBOK non è “l’opposto” di Agile e Scrum**. Non stai per imparare un metodo waterfall in conflitto con tutto quello visto finora. Alcuni elementi utili a capire perché:

- Il PMI stesso ha una certificazione dedicata all’Agile, la **PMI-ACP (Agile Certified Practitioner)**, segno che riconosce l’approccio Agile come uno standard professionale a sé, non come una moda da ignorare.
- La versione più recente della guida, la **PMBOK Guide, 7th Edition** (pubblicata nel 2021), ha cambiato profondamente impostazione rispetto alle edizioni precedenti: non è più organizzata per “processi da eseguire in un certo ordine” (i 5 process group e le 10 knowledge area che vedrai nelle prossime due sottosezioni, ancora oggi il modo più diffuso con cui questi temi vengono insegnati e discussi), ma per **12 principi** generali (es. “essere un guardiano diligente e rispettoso”, “adattarsi in base al contesto”) e **8 performance domain** (aree di prestazione, come stakeholder, pianificazione, incertezza). Entrambi i set sono pensati per essere applicabili sia a progetti pianificati in anticipo sia a progetti gestiti in modo iterativo/Agile — un segnale chiaro che anche il PMI riconosce che “un solo metodo per tutti i progetti” non è più un’idea sostenibile.
- Molte aziende reali — probabilmente anche quella per cui lavorerai — non scelgono “Agile puro” o “PMP puro”, ma un **approccio ibrido**: usano il linguaggio e alcuni strumenti del PMBOK (stakeholder, rischio, budget, reportistica) dentro un modo di lavorare quotidiano che resta Agile (sprint, backlog, retrospettive).

Nel contesto di questo corso, quindi, il modo più utile di vedere il PMP e il PMBOK non è “il metodo alternativo a Scrum”, ma un **vocabolario comune e una cassetta degli attrezzi**: tanti degli strumenti che hai già usato con ShopFacile (RAID Log, RACI, stakeholder, KPI) sono, di fatto, pratiche che il PMBOK descrive in modo formale — tu le hai già viste “in azione” nel contesto Agile, ora le colleghiamo al loro nome e alla loro origine “ufficiale”.

8.11 I 5 gruppi di processi (process group)

Il primo di questi collegamenti riguarda **come si organizza il lavoro di un progetto nel tempo**: il PMBOK (nell’impostazione più tradizionale, quella della 6th Edition, di cui parliamo ancora perché è la più diffusa nel vocabolario comune del settore) organizza le attività di un progetto in **5 gruppi di processi** (process group). Il punto da capire bene, soprattutto se il termine “processo” ti suona come “fase rigida e sequenziale”: **non sono fasi che si susseguono una volta e basta**, ma gruppi di attività che **si ripetono**, spesso più volte, durante la vita del progetto.

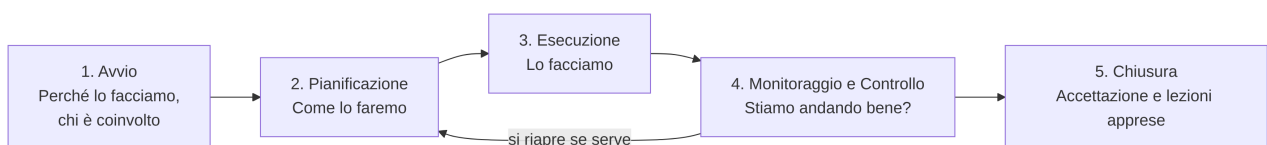
I 5 process group sono:

1. **Avvio (Initiating)**: si definisce il progetto (o una sua fase/ iterazione) ad alto livello — obiettivi, stakeholder chiave, perché vale la pena farlo. È il momento del **project charter**, che vedremo nella prossima sottosezione.
2. **Pianificazione (Planning)**: si definisce come si raggiungeranno gli obiettivi — scope, tempi, costi, qualità, rischi, comunicazione. È il gruppo di processi più corposo nel PMBOK tradizionale.
3. **Esecuzione (Executing)**: si fa il lavoro previsto dal piano — sviluppo, test, coordinamento del team.

4. **Monitoraggio e Controllo (Monitoring & Controlling)**: si verifica l'andamento rispetto al piano e si interviene se serve — è la stessa logica del monitoraggio che hai visto in 8.7, formalizzata a livello di intero progetto.
5. **Chiusura (Closing)**: si conclude formalmente il progetto (o una sua fase), si verifica che il lavoro sia stato accettato, si raccolgono le lezioni apprese (lo vedremo in 8.19).

 **Analogia:** pensa ai process group non come alle fasi di un viaggio in treno (parti, viaggi, arrivi, fine), ma come ai **ruoli di un cruscotto d'auto durante tutto il viaggio**: c'è sempre un momento in cui decidi la destinazione (avvio), uno in cui scegli la strada (pianifica), uno in cui guidi (esecuzione), uno in cui guardi lo specchietto e il tachimetro (monitoraggio e controllo) — e questi quattro si alternano e si ripetono in continuazione, non uno alla volta e mai più. Solo quando parcheggi definitivamente a destinazione c'è la chiusura.

Il collegamento con ciò che già conosci è diretto: **ogni sprint di ShopFacile contiene, in piccolo, tutti e cinque i process group**. Lo Sprint Planning è un mini-avvio più pianificazione; i giorni di sviluppo sono esecuzione; la Daily Scrum e il burndown sono monitoraggio e controllo; la Sprint Review è, in un certo senso, una piccola chiusura (si verifica cosa è stato completato) — e la Sprint Retrospective aggiunge la raccolta delle lezioni apprese, proprio come la chiusura formale di un progetto più grande.



Esempio pratico: nel progetto ShopFacile, il rilascio del modulo pagamenti che hai già incontrato più volte in questa sezione può essere letto con la lente dei process group: **Avvio** è il momento in cui Sara propone l'iniziativa e se ne definisce l'obiettivo di business; **Pianificazione** è quando Luca e il team stimano il lavoro e lo distribuiscono su più sprint; **Esecuzione** sono gli sprint in cui Marco, Giulia e Ahmed sviluppano e testano; **Monitoraggio e Controllo** è il RAID Log, la RACI e il burndown che hai già visto tenere sotto controllo l'andamento; **Chiusura** è il go-live in produzione seguito dalla retrospettiva sul rilascio.

Sapere **quando** si lavora (i process group) non dice ancora **su cosa** si lavora. Il PMBOK organizza anche i contenuti del project management per temi: sono le knowledge area, ed è la mappa più utile per collegare tutto quello che hai già imparato in questa sezione.

8.12 Le aree di conoscenza (knowledge area)

Oltre ai process group (“quando”), il PMBOK tradizionale (6th Edition) organizza il contenuto del project management in **10 aree di conoscenza** (knowledge area): temi trasversali che si applicano, in misura diversa, in tutti i process group. Questa è probabilmente la tabella più importante di questa sezione, perché **funziona da mappa**: quasi tutto quello che hai già imparato in 8.1-8.9 trova qui il suo “cassetto” ufficiale.

Area di conoscenza	Di cosa si occupa	Dove l'hai già vista nel corso
Integrazione (Integration Management)	Coordinare tutte le altre aree in un piano coerente; gestire le modifiche in modo controllato	8.13, 8.17
Ambito (Scope Management)	Definire cosa è (e cosa NON è) incluso nel progetto, ed evitare che cresca senza controllo	8.14 (WBS e scope creep)
Tempi (Schedule Management)	Pianificare attività, dipendenze e durate nel tempo	8.6, 8.15
Costi (Cost Management)	Stimare, allocare e controllare il budget	8.16 (EVM)
Qualità (Quality Management)	Garantire che il risultato soddisfi gli standard richiesti	8.18 (lo ritroverai anche nella sezione 10 su CI/CD)
Risorse (Resource Management)	Gestire le persone e gli strumenti necessari al progetto	8.3
Comunicazione (Communications Management)	Decidere cosa comunicare, a chi, con quale frequenza e formato	8.9 (reportistica)
Rischi (Risk Management)	Identificare, valutare e mitigare eventi incerti che possono impattare il progetto	8.2, 8.5
Approvvigionamenti (Procurement Management)	Gestire fornitori esterni e contratti	8.18
Stakeholder (Stakeholder Management)	Identificare e gestire le persone/ gruppi con un interesse nel progetto	8.1

 **Da notare:** leggendo la terza colonna, salta all'occhio che **non stai imparando dieci concetti nuovi**. Stai scoprendo che gli strumenti già usati con ShopFacile — RAID Log, RACI, reportistica, pianificazione a livelli — sono, con un altro nome, esattamente le knowledge area del PMBOK. È lo stesso terreno visto da due vocabolari diversi, non due materie diverse.

Un chiarimento importante sulla versione della guida: questa organizzazione in 5 process group e 10 knowledge area è quella della **PMBOK Guide, 6th Edition** — vedi il richiamo alla 7th Edition già fatto in 8.10, che ha cambiato struttura ma non ha reso obsoleto questo vocabolario, ancora il più diffuso nella pratica quotidiana.

Con questa mappa in mano, puoi ora ripartire dall'inizio del “ciclo di vita” di un progetto secondo il PMBOK: il documento che lo fa nascere ufficialmente, il project charter.

8.13 Il Project Charter: l'atto di nascita del progetto

Se il process group di Avvio (8.11) è il momento in cui un progetto nasce ufficialmente, il **project charter** è il documento che lo certifica per scritto. È, in un certo senso, l'**atto di nascita del progetto**: prima che esista un charter approvato, formalmente il progetto non dovrebbe nemmeno iniziare (niente budget allocato, niente team assegnato in modo ufficiale).

Un project charter, anche in versione semplice, contiene tipicamente:

- **Obiettivi del progetto**: perché lo facciamo, cosa vogliamo ottenere.
- **Scope di alto livello**: cosa è dentro e cosa è fuori dal progetto (in poche righe, non nel dettaglio — quello arriva con la WBS, sezione 8.14).
- **Stakeholder chiave**: chi ha un ruolo decisivo (si collega diretto a 8.1).
- **Budget e tempi indicativi**: un ordine di grandezza, non ancora una pianificazione dettagliata.
- **Sponsor**: la persona (o il gruppo) che autorizza formalmente il progetto e ne è responsabile finale davanti all'organizzazione.
- **Criteri di successo**: come sapremo, alla fine, se il progetto ha raggiunto l'obiettivo.

Il charter viene tipicamente **approvato dallo sponsor** (o da chi ha il potere di allocare budget e risorse), non dal Project Manager stesso: chi gestisce il progetto lo scrive (o lo redige insieme allo sponsor), ma serve un'autorizzazione che venga “da sopra” per renderlo legittimo.

 **Esempio pratico**: un project charter essenziale per ShopFacile, per il progetto “nuovo modulo di pagamento” che hai già incontrato più volte in questa sezione:

Campo	Contenuto
Obiettivo	Aggiungere un nuovo metodo di pagamento (carta di credito con autenticazione forte) per ridurre gli abbandoni al checkout
Scope di alto livello	Incluso: integrazione con il nuovo fornitore di pagamenti, test di sicurezza, rilascio in produzione. Escluso: rifacimento dell'intero flusso di checkout
Stakeholder chiave	Sara (Product Owner/business), team sicurezza interno, fornitore esterno di pagamenti
Budget e tempi indicativi	Circa 3 sprint, nessun costo aggiuntivo oltre l'abbonamento al servizio del fornitore
Sponsor	Il responsabile e-commerce dell'azienda che possiede ShopFacile
Criteri di successo	Il nuovo metodo di pagamento è disponibile in produzione, supera i test di sicurezza, e non introduce un aumento del tasso di errore al checkout

Da notare quanto questo charter sia in realtà “leggero”: non è un documento di 40 pagine, ma un riassunto di una pagina che allinea tutti su obiettivo, confini e responsabilità **prima** di iniziare a lavorare. Una volta chiarito il “cosa e perché” a livello generale, il passo successivo è scomporlo in pezzi di lavoro gestibili: è il compito della WBS.

8.14 Scope e WBS (Work Breakdown Structure)

Il charter definisce lo scope solo “a grandi linee”. Il **Scope Management** (gestione dell’ambito) si occupa di renderlo preciso e di difenderlo nel tempo — perché il rischio più comune, in qualsiasi progetto, è che l’ambito cresca senza controllo: si chiama **scope creep**.

 **Esempio concreto di scope creep:** lo sprint del team ShopFacile è già iniziato con un obiettivo chiaro (“integrare il nuovo metodo di pagamento”). A metà sprint, **Sara** chiede di aggiungere “già che ci siamo, mostriamo anche lo storico delle transazioni nel profilo utente”. Presa isolatamente, la richiesta ha senso per il business — ma se viene accettata senza discussione, senza rivalutare tempi o rimuovere altro lavoro dallo sprint, è **scope creep**: lo scope cresce “di nascosto”, mettendo a rischio l’obiettivo originale dello sprint. La risposta corretta non è “mai accettare nulla”, ma **gestire la richiesta come una change request** (la vedremo in 8.17): valutarla, e decidere insieme cosa togliere o rimandare per farle spazio.

Lo strumento principale del PMBOK per definire lo scope in modo preciso è la **WBS, Work Breakdown Structure** (struttura di scomposizione del lavoro): una scomposizione **gerarchica** del lavoro totale del progetto in parti sempre più piccole e gestibili, fino ad arrivare a “pacchetti di lavoro” (work package) stimabili e assegnabili.

Un piccolo esempio di WBS per il progetto “modulo pagamenti” di ShopFacile:

1. Modulo pagamenti ShopFacile
 - 1.1 Integrazione con il fornitore esterno
 - 1.1.1 Configurazione ambiente di test del fornitore
 - 1.1.2 Sviluppo del connettore di pagamento
 - 1.1.3 Test di integrazione
 - 1.2 Sicurezza
 - 1.2.1 Test di sicurezza sui token di sessione
 - 1.2.2 Revisione del team sicurezza interno
 - 1.3 Interfaccia utente
 - 1.3.1 Schermata di scelta del metodo di pagamento
 - 1.3.2 Schermata di conferma pagamento
 - 1.4 Rilascio
 - 1.4.1 Deploy in ambiente di staging
 - 1.4.2 Go-live in produzione
 - 1.4.3 Monitoraggio post-rilascio

Una domanda che ti farai subito, arrivato a questo punto: **la WBS non è semplicemente il Product Backlog che ho già visto in Scrum?** Sono strumenti simili nello spirito (scomporre il lavoro), ma diversi per natura:

	WBS	Product Backlog
Struttura	Gerarchica, ad albero (scomposizione del <i>lavoro</i>)	Lista ordinata e piatta (elenco di <i>elementi</i> di valore)
Stabilità	Tende a essere definita presto e a cambiare poco	Cambia continuamente, per definizione (è “vivo”)
Unità di scomposizione	Attività/pacchetti di lavoro (cosa va fatto)	User Story/elementi di valore (cosa vuole l'utente)
Ordinamento	Non ha un ordine di priorità intrinseco	È ordinato per priorità (in cima le cose più importanti)
Chi lo definisce tipicamente	Project Manager/team, in fase di pianificazione	Product Owner (Sara, per ShopFacile), in modo continuativo

Nella pratica di un progetto ibrido (che vedremo meglio in 8.20), è comune usare **entrambi**: una WBS di alto livello per capire i grandi blocchi di lavoro e stimare budget/tempi complessivi, e un Product Backlog per la gestione operativa e continua del lavoro sprint per sprint. Definito **cosa** c'è da fare, il passo successivo naturale è capire **quando**: è il territorio dello schedule e del percorso critico.

8.15 Tempi: schedule, dipendenze e critical path

Una volta scomposto il lavoro (WBS), il **Schedule Management** (gestione dei tempi) si occupa di ordinarlo nel tempo: stimare la durata di ogni attività, individuare le **dipendenze** tra attività (quali devono finire prima che un'altra possa iniziare) e costruire un piano temporale.

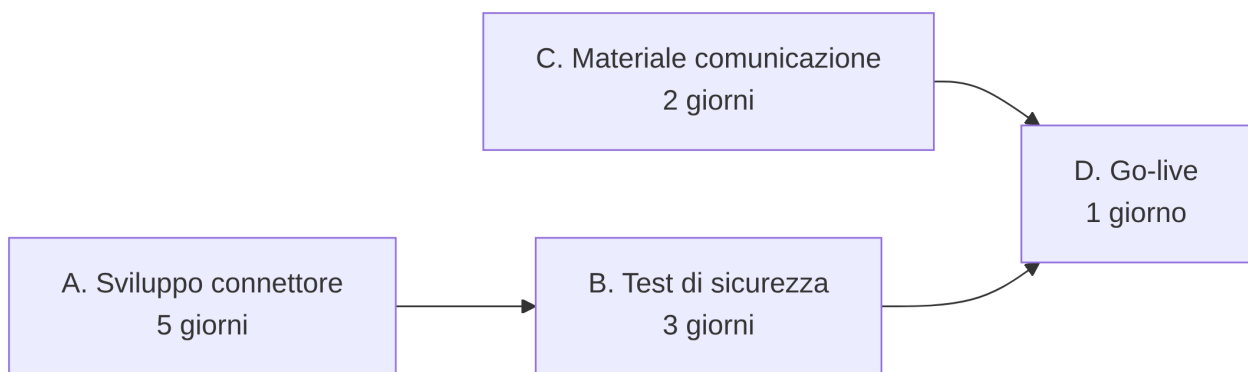
Lo strumento visivo più classico per rappresentare questo piano è il **diagramma di Gantt**: un grafico a barre orizzontali, una per attività, posizionate nel tempo, che mostra a colpo d'occhio quando inizia e finisce ciascuna attività e come si sovrappongono. È utile per avere una vista d'insieme su progetti con attività numerose e dipendenze complesse, ma ha un limite importante in contesto Agile: un Gantt dettagliato **presuppone di conoscere in anticipo** durate e dipendenze precise, cosa che in un progetto iterativo (dove backlog e priorità cambiano sprint dopo sprint) è spesso un'illusione di precisione più che un piano realistico.

Il concetto più utile da portare a casa da questa sottosezione è quello di **percorso critico** (critical path): la **sequenza di attività dipendenti tra loro che determina la durata minima totale del progetto**. Se un'attività sul percorso critico slitta, **slitta tutto il progetto**; se un'attività fuori dal percorso critico slitta (fino a un certo limite), il progetto nel suo complesso non ne risente.

Quel “limite” ha un nome: **float** (o **slack**), cioè quanto una singola attività può slittare senza spostare la data finale del progetto. Le attività sul percorso critico hanno float pari a zero, per definizione: non hanno margine.

Esempio semplice con ShopFacile: immagina solo 4 attività necessarie prima del go-live del modulo pagamenti, con le loro durate stimate e dipendenze:

Attività	Durata stimata	Dipende da
A — Sviluppo del connettore di pagamento	5 giorni	—
B — Test di sicurezza	3 giorni	A
C — Preparazione materiale di comunicazione per gli utenti	2 giorni	—
D — Go-live in produzione	1 giorno	B e C



Il percorso $A \rightarrow B \rightarrow D$ dura $5 + 3 + 1 = \mathbf{9 \text{ giorni}}$. Il percorso $C \rightarrow D$ dura $2 + 1 = \mathbf{3 \text{ giorni}}$. Il percorso più lungo (quello che determina la durata minima del progetto) è **$A \rightarrow B \rightarrow D$, con 9 giorni**: questo è il percorso critico. L'attività C ha 6 giorni di float: può iniziare fino a 6 giorni più tardi (o richiedere 6 giorni in più del previsto) senza far slittare il go-live — mentre un solo giorno di ritardo su A o su B fa slittare l'intero progetto di un giorno.

Quando ha senso questo tipo di analisi? Quando le attività sono poche, ben definite, con dipendenze chiare — tipicamente in progetti a scope fisso, con una data di consegna esterna vincolante (es. una normativa, un evento). In un contesto Agile continuo come lo sviluppo ordinario di ShopFacile, dove il lavoro è organizzato in backlog e sprint ricorrenti più che in un albero di attività con dipendenze rigide, il team **non calcola un percorso critico ogni sprint**: usa piuttosto la **velocity** (vista in 8.8) per stimare quanto lavoro completerà nei prossimi sprint. Il percorso critico resta utile soprattutto per **eventi puntuali con una scadenza esterna rigida** (un lancio in una data fissa, una scadenza legale), non per il flusso continuo di lavoro sprint dopo sprint.

Sapere **quando** finirà un'attività, però, non dice ancora nulla su un'altra domanda altrettanto cruciale per uno sponsor: quanto sta costando, e se lo si sta facendo **al costo previsto**.

8.16 Costi e Earned Value Management (EVM)

Dopo aver pianificato i tempi (8.15), la domanda naturale successiva è **quanto costa** e **se si sta rispettando il budget**: è il compito del **Cost Management** (gestione dei costi), che si occupa di stimare, allocare e controllare il budget del progetto. Il punto di partenza è la **baseline**: il piano di riferimento (quanto lavoro dovrebbe essere completato, a quale costo, a ogni punto nel tempo) rispetto al quale misurare poi lo scostamento reale.

Il PMBOK propone una tecnica per misurare questo scostamento in modo integrato (non solo “quanto abbiamo speso”, ma “quanto lavoro utile stiamo davvero producendo per quella spesa”): l'**Earned Value Management (EVM)**, gestione del valore realizzato. Si basa su tre misure di base, spiegate qui **senza formule complicate**, solo con il loro significato:

- **PV — Planned Value** (valore pianificato): quanto lavoro **avremmo dovuto** aver completato, a questo punto, secondo il piano originale — espresso in valore economico (es. “a questo punto dovremmo aver completato lavoro per 10.000 €”).
- **EV — Earned Value** (valore realizzato/guadagnato): quanto lavoro **abbiamo davvero completato**, a questo punto, sempre espresso in valore economico (non quanto abbiamo speso, ma quanto lavoro *utile* è stato fatto).
- **AC — Actual Cost** (costo reale): quanto **abbiamo davvero speso**, a questo punto, per arrivare dove siamo.

Da queste tre misure si ottengono due indicatori sintetici, con una formula semplice e un'interpretazione altrettanto semplice:

- **SPI — Schedule Performance Index** = EV / PV → misura se siamo in linea con i **tempi** pianificati.
- **CPI — Cost Performance Index** = EV / AC → misura se siamo in linea con i **costi** pianificati.

In entrambi i casi: **un valore maggiore di 1 è positivo** (stiamo producendo più valore di quanto pianificato/speso), **un valore minore di 1 è negativo** (siamo in ritardo o stiamo spendendo più del valore prodotto), **un valore vicino a 1 significa che siamo in linea col piano**.



Esempio pratico numerico, con numeri tondi: a metà del progetto “modulo pagamenti” di ShopFacile, il piano prevedeva di aver completato lavoro per un valore di **10.000 €** ($PV = 10.000$ €). Guardando cosa è stato davvero consegnato (funzionalità completate e testate), il team stima di aver prodotto un valore di **8.000 €** ($EV = 8.000$ €). I costi effettivamente sostenuti finora (ore del team, costi del fornitore) sono stati **9.000 €** ($AC = 9.000$ €).

- **SPI** = $EV / PV = 8.000 / 10.000 = 0,8$ → è **minore di 1**: il team è **in ritardo** rispetto al piano (ha completato meno lavoro di quanto previsto a questo punto).
- **CPI** = $EV / AC = 8.000 / 9.000 \approx 0,89$ → è **minore di 1**: il progetto sta **spendendo di più** del valore che sta producendo (per ogni euro speso, sta “tornando” meno di un euro di lavoro utile).

Interpretazione in linguaggio semplice, quella che porteresti a Luca e Sara: **il progetto è sia in ritardo sui tempi sia sopra budget rispetto al valore prodotto** — un doppio segnale che richiede attenzione, non necessariamente panico: può bastare capire *perché* (es. un imprevisto tecnico) e decidere un'azione correttiva, come faresti con un rischio nel RAID Log.

Una nota di realismo, utile da avere chiara come futuro Junior PM: in **molti contesti Agile**, l'EVM completo (con tutte le formule e varianti del PMBOK) **non viene usato così com'è** — è pensato per progetti con una baseline dettagliata e stabile, che in Agile per definizione cambia sprint dopo sprint. Restano però **utilissimi i concetti sottostanti**: avere una baseline di riferimento (anche solo “quanti story point pensavamo di completare in questa release”) e misurare regolarmente lo **scostamento** tra pianificato e reale è esattamente lo stesso principio che guida un burndown/burnup chart (8.7) — solo espresso in story point invece che in euro.

8.17 Il triplo vincolo e la gestione delle modifiche

Fin qui abbiamo trattato scope (8.14), tempi (8.15) e costi (8.16) come tre variabili separate, ma in realtà sono profondamente collegate tra loro: è il concetto del **triplo vincolo** (triple constraint), uno dei concetti più citati del project management. **Ambito (scope), tempi e costi** sono legati tra loro come i lati di un triangolo — **non puoi cambiare uno senza che almeno un altro ne sia influenzato**, a parità di qualità del risultato (spesso rappresentata come un quarto elemento, al centro del triangolo, che tutti gli altri tre influenzano).

 **Analogia:** pensa a una coperta di dimensioni fisse. Se la tiri per coprire meglio le spalle (più scope), scoprirai i piedi (meno tempo disponibile o più costo, oppure qualità più bassa se la tiri senza aggiungere stoffa). Non esiste un modo di “tirare la coperta” che allunghi tutti i lati contemporaneamente senza un costo da qualche parte.

Il comportamento del triangolo cambia in base a **cosa è fisso** nel tuo progetto:

- **Progetto a scope fisso** (tipico di molti progetti PMP tradizionali, es. una normativa da rispettare entro una data): lo scope non si discute, quindi se emergono imprevisti si agisce su **tempi** o **costi** (più persone, più tempo, o entrambi).
- **Progetto a tempo fisso** (tipico di uno sprint Scrum): la data non si discute (lo sprint dura quanto deciso), quindi se il lavoro non ci sta, si agisce sullo **scope** — si riduce cosa entra nello sprint, non si allunga lo sprint per farci stare tutto.

Questo è esattamente il collegamento con Agile che vale la pena sottolineare: **in Scrum, lo scope è la variabile “elastica”, mentre tempo (la durata dello sprint) e team sono fissi**. È l'esatto opposto dell'approccio “a scope fisso, tempi che si adattano” tipico di molta pianificazione tradizionale — ed è anche il motivo per cui lo scope creep (8.14) è un rischio particolarmente insidioso in Scrum: se qualcuno aggiunge scope senza togliere nulla, e il tempo non può allungarsi, l'unica variabile che cede silenziosamente è la qualità (corner cutting) o il carico di lavoro sostenibile del team.

Quando una modifica a scope, tempi o costi viene proposta formalmente, il PMBOK la chiama **change request** (richiesta di modifica): non un cambiamento fatto “a braccio”, ma una proposta che va **valutata prima di essere applicata**, guardando l’impatto su tutti i lati del triangolo. In progetti di dimensioni maggiori, questa valutazione passa da un **Change Control Board (CCB)**: un piccolo gruppo (spesso sponsor, PM, rappresentanti degli stakeholder principali) che approva o rifiuta le richieste di modifica più significative, in modo che non sia una sola persona a decidere unilateralmente di alterare l’ambito di un progetto già avviato.

Esempio pratico: torna alla richiesta di Sara vista in 8.14 (aggiungere lo storico transazioni a metà sprint). In un progetto Agile ben gestito, questa richiesta non viene né accettata automaticamente né respinta seccamente: viene trattata come una piccola change request, discussa con il team (magari in una versione informale, senza un vero e proprio Change Control Board formale in un contesto così piccolo) — si valuta cosa togliere dallo sprint corrente per farle spazio, oppure si conferma che andrà nel prossimo Sprint Planning, aggiungendola al Product Backlog. Il principio del PMBOK (valutare prima di applicare) resta identico, anche se il “board” che decide, in ShopFacile, è semplicemente Luca e Sara insieme, non un comitato formale.

Fin qui ambito, tempi e costi. Le altre knowledge area della tabella in 8.12 — qualità, approvvigionamenti, comunicazione — meritano solo un completamento più breve, perché in gran parte le hai già incontrate altrove nel corso.

8.18 Qualità, approvvigionamenti e comunicazione secondo il PMBOK

Tre aree di conoscenza del PMBOK che, a differenza di scope/tempi/costi, non richiedono un approfondimento lungo in questa sezione: le hai già viste (o le vedrai) altrove nel corso, e qui basta collegarle esplicitamente al vocabolario PMP.

Quality Management (gestione della qualità) distingue due attività spesso confuse:

- **Quality Assurance (QA):** attività **preventive**, orientate al processo — assicurarsi che il modo in cui si lavora produca qualità “per costruzione” (es. standard di code review, test automatizzati previsti dal processo).
- **Quality Control (QC):** attività **di verifica**, orientate al prodotto — controllare che il singolo risultato concreto soddisfi i requisiti (es. eseguire i test su una specifica funzionalità appena sviluppata).

In ShopFacile, questa distinzione ha un volto familiare: **Giulia**, nel suo ruolo di Developer/QA, incarna entrambe le attività — quando definisce insieme al team gli standard di test e code review sta facendo Quality Assurance; quando verifica concretamente che una funzionalità superi i test prima del rilascio sta facendo Quality Control. E i **quality gate** della pipeline CI/CD, che hai visto (o vedrai) nella sezione 10, sono proprio l’automazione del Quality Control: un controllo oggettivo e ripetibile, al posto di un giudizio manuale ogni volta.

Procurement Management (gestione degli approvvigionamenti) riguarda i rapporti con **fornitori esterni**: contratti, criteri di scelta, gestione della relazione. Il caso più concreto in ShopFacile è il **fornitore esterno del servizio di pagamento**, già incontrato più volte in questa sezione (nel charter di 8.13, nella WBS di 8.14): non è parte del team ShopFacile, ma il progetto dipende dal suo servizio, dai suoi tempi di integrazione e dal contratto che regola cosa succede se qualcosa va storto (es. livelli di servizio garantiti, SLA — Service Level Agreement). Gestire bene questa relazione è project management, anche se non scrivi una riga di codice.

Communications Management (gestione della comunicazione) è l'area che hai già visto in dettaglio in **8.9**: un piano di comunicazione — anche informale — dovrebbe chiarire, per ogni tipo di stakeholder, **cosa** comunicare, **con quale frequenza**, **con quale livello di dettaglio** e **attraverso quale canale**. Il PMBOK formalizza esattamente questo, sotto il nome di communication management plan.

Con qualità, approvvigionamenti e comunicazione richiamati al loro posto nel vocabolario PMBOK, resta un'ultima fase del ciclo di vita del progetto da collegare a qualcosa che già conosci molto bene: la chiusura.

8.19 Chiusura del progetto e lessons learned

Il process group di Chiusura (8.11) non è solo “il progetto è finito, si passa al prossimo”: il PMBOK lo tratta come un momento formale, con attività precise:

- **Accettazione del deliverable**: qualcuno con l'autorità per farlo (lo sponsor, il cliente) verifica esplicitamente che il risultato consegnato soddisfi i criteri di successo definiti nel charter (8.13) — senza questa accettazione formale, un progetto rischia di restare in un limbo dove “sembra finito” ma nessuno lo ha davvero confermato.
- **Archiviazione**: documentazione, decisioni prese, contratti chiusi vengono organizzati e conservati, così che chi lavorerà su un progetto simile in futuro possa ritrovarli.
- **Lessons learned** (lezioni apprese): un momento dedicato a chiedersi, collettivamente, cosa ha funzionato e cosa no durante il progetto, per fare meglio la prossima volta.

Ed è qui che arriva il collegamento più diretto con tutto ciò che hai già imparato su Scrum (sezione 6): **le lessons learned del PMBOK e la Sprint Retrospective sono, nella sostanza, lo stesso principio**, applicato con una cadenza diversa. La Retrospective la fai alla fine di ogni sprint (ogni 1-4 settimane); le lessons learned del PMBOK si fanno tipicamente alla fine di un intero progetto (mesi). Non sono due tecniche diverse: è la stessa domanda — “cosa impariamo da questa esperienza per la prossima?” — fatta a due zoom diversi. Un team maturo, in un contesto ibrido, fa entrambe le cose: retrospettive frequenti per correggere la rotta sprint dopo sprint, e una revisione più ampia a fine progetto (o release) per cogliere pattern che si notano solo su un orizzonte più lungo.

Esempio pratico: alla chiusura del progetto “modulo pagamenti” di ShopFacile, oltre alla retrospettiva dell'ultimo sprint (focalizzata sugli ultimi due settimane di lavoro), il team con Luca dedica un'ora a guardare l'intero progetto, dal charter iniziale al go-live: cosa ha funzionato bene (es. “coinvolgere il team sicurezza fin dall'inizio ha evitato ritardi”), cosa ha funzionato male (es. “abbiamo sottostimato il

tempo di integrazione con il fornitore esterno di due settimane”), e cosa portare nel prossimo progetto simile (es. “la prossima volta, chiediamo una stima di integrazione al fornitore prima di fissare il budget nel charter”).

Con l'intero ciclo di vita del progetto — dal charter alla chiusura — ora collegato al vocabolario Agile che già conoscevi, resta una domanda di sintesi da affrontare esplicitamente: come convivono, davvero, nella pratica di un'azienda reale, PMP e Agile?

8.20 PMP e Agile: come convivono davvero

Hai visto, sottosezione dopo sottosezione, che quasi ogni concetto del PMBOK ha un corrispettivo (o un parente stretto) nel vocabolario Agile già noto. Vale la pena riassumerlo in una tabella di “traduzione”, utile da tenere a mente quando ti troverai in una riunione dove qualcuno usa termini PMP e qualcun altro termini Scrum per parlare, in fondo, delle stesse cose:

Termine PMP/PMBOK	Corrispettivo Agile/Scrum	Nota
Project Charter	Product Vision	Entrambi rispondono a “perché lo facciamo e cosa vogliamo ottenere”
WBS (Work Breakdown Structure)	Product Backlog	Entrambi scompongono il lavoro, ma con struttura e stabilità diverse (8.14)
Diagramma di Gantt	Roadmap / Release Plan	Entrambi mostrano una vista nel tempo, ma il Gantt è più rigido e dettagliato
Project Manager	Scrum Master + Product Owner	Nel PMBOK tradizionale un solo ruolo copre “processo” e “priorità del business”; in Scrum sono due ruoli distinti
Earned Value Management (SPI/ CPI)	Velocity / Burndown-Burnup	Lo SPI ha un analogo ragionevole (scostamento sui tempi/quantità di lavoro); il CPI no — vedi nota sotto
Change Control Board	Sprint Planning / Product Owner che ri-prioritizza il backlog	Entrambi sono il “filtro” che decide se una modifica entra nel lavoro pianificato
Lessons Learned	Sprint Retrospective	Stesso principio, cadenza diversa (8.19)

⚠ Attenzione a non forzare l'equivalenza: l'analogia EVM ↔ velocity/burndown funziona bene per lo **SPI**, perché sia lo SPI sia il burndown/burnup misurano uno scostamento su tempi e quantità di lavoro. Non funziona per il **CPI** (EV/AC): velocity e burndown non tracciano alcun costo, perché in un team stabile il costo per sprint è praticamente fisso indipendentemente dai punti completati. Il CPI, in Agile, semplicemente non ha un vero corrispettivo.

Va detto con chiarezza, perché è la sintesi più utile di tutta questa sezione: **nella pratica quotidiana di molte aziende reali, non esiste “o PMP o Agile”**. Esiste un **approccio ibrido**: si usa il linguaggio e alcuni strumenti PMBOK per parlare con il management e per governare budget, contratti con fornitori esterni e reportistica verso l'alto, e si lavora concretamente in sprint, con backlog e retrospettive, per la gestione quotidiana del team. Non è un compromesso “a metà” imperfetto: è semplicemente il riconoscimento che strumenti diversi servono bene a scopi diversi.

Questo, non a caso, è anche la direzione presa dal PMI stesso con la **PMBOK Guide, 7th Edition** (2021), già introdotta in 8.10: un segnale chiaro che anche il PMI riconosce che “un solo metodo per tutti i progetti” non è più un'idea sostenibile.

Cosa conviene sapere, in concreto, a un Junior Project Manager (o Scrum Master) che lavora in un team DevOps come quello di ShopFacile?

- Conoscere il **vocabolario PMP** ti permette di comunicare con manager, clienti o fornitori che lo usano abitualmente, senza dover “tradurre” ogni volta da zero.
- Gli **strumenti pratici** (RAID Log, RACI, stakeholder mapping, piano di comunicazione) sono utili **indipendentemente** dal metodo scelto per il lavoro quotidiano del team.
- Non è necessario (né sempre utile) applicare l'EVM completo o un Gantt dettagliato in un team Agile maturo — ma è utile **sapere cosa sono**, per riconoscerli quando un cliente o un altro reparto li richiede.
- Il tuo valore, come Junior PM/Scrum Master, è nella capacità di **muoverti tra i due mondi**: parlare “PMP” quando serve assicurare lo sponsor con numeri e milestone, parlare “Agile” quando serve coordinare il lavoro quotidiano del team — sapendo che, sotto il vocabolario diverso, il problema che stai risolvendo è quasi sempre lo stesso.

Come si decide, in pratica: predittivo, Agile o ibrido

I criteri per scegliere un approccio più predittivo o più Agile sono già apparsi in questa sezione — nel triplo vincolo (8.17), nel percorso critico (8.15), nell'EVM (8.16) — ma sparsi. Raccoglierli in poche domande operative aiuta a decidere davvero, all'inizio di un progetto:

1. **Il perimetro è negoziabile, o fissato contrattualmente?** Se un contratto elenca esattamente cosa va consegnato, hai poco margine per lo scope “elastico” tipico di Scrum (8.17).
2. **Esiste una scadenza esterna non spostabile** (una normativa, un evento già annunciato)? In questi casi il percorso critico (8.15) resta utile: la data non è negoziabile.
3. **Quanto è probabile che i requisiti cambino mentre il team lavora?** Più il dominio è incerto (il caso tipico di ShopFacile), più ha senso un approccio iterativo che corregga la rotta sprint dopo sprint.

4. **Lo sponsor pretende reportistica in formato tradizionale?** Un cliente abituato a Gantt e SPI/CPI potrebbe richiedere quel linguaggio nei report verso l'alto (8.9), anche se il team lavora in Scrum.

Più le risposte spingono verso “fisso, rigido, poco cambiamento”, più senso ha un approccio predittivo; più spingono verso “flessibile, incerto”, più senso ha l'Agile. Nella pratica, gli **approcci ibridi sono la norma, non l'eccezione**: raro trovare un progetto puramente predittivo o puramente Agile.

8.21 Jira: la gerarchia degli oggetti e le due viste

Prima di parlare di Jira in sé, vale la pena vedere il problema che risolve con una scena fin troppo comune. Luca segue lo stato di avanzamento di ShopFacile su un foglio di calcolo che aggiorna la sera, quando ha tempo; gli sviluppatori, nel frattempo, scrivono lo stato reale del loro lavoro in chat, a chi lo chiede. Il risultato, puntuale ogni settimana: alla riunione di stato non si discute più di **cosa fare**, ma di **quale numero sia vero** — il foglio di Luca o l'ultimo messaggio in chat. Il motivo per cui esistono strumenti come **Jira** è avere **un unico posto dove il lavoro esiste**, aggiornato da chi lo fa, non ricostruito la sera da chi lo insegue.

Nella sezione 10 incontrerai Jira come alternativa alle Issue di GitHub per il tracking del lavoro. Qui vediamo già la parte che lì resterà sullo sfondo: come Jira organizza quel lavoro internamente, e perché la stessa informazione appare in modo così diverso in una board e in un Gantt.

Issue: il termine ombrello che confonde tutti all'inizio

La prima cosa che confonde chiunque arrivi da GitHub Issues: in Jira, **tutto** è una **issue**. Un bug, una nuova funzionalità, un compito tecnico, un piccolo pezzo di un lavoro più grande — sono tutte, sotto il cofano, lo stesso tipo di oggetto. Quello che li distingue è il **tipo di issue** (Epic, Story, Task, Bug, Sub-task): un'etichetta sul contenitore, non un contenitore diverso. Quando qualcuno dice “ho aperto una issue”, sta dicendo pochissimo su cosa sia davvero: bisogna sempre chiedere (o guardare) il tipo.

La gerarchia: da Epic a Sub-task

Le issue si organizzano in una gerarchia, dal più grande al più piccolo:

Tipo	Cosa rappresenta	Chi la scrive tipicamente	Orizzonte temporale	Esempio in ShopFacile
Initiative / Epic	Un obiettivo ampio, che raggruppa più Story/Task	Product Owner, a volte con il Solution Architect	Mesi, più release	“Preventivatore online”
Story	Una User Story: valore per l'utente, scritta dal suo punto di vista	Product Owner (con il team)	Uno o pochi sprint	“Come cliente voglio salvare un preventivo per riprenderlo dopo”
Task	Lavoro tecnico necessario, che nessun utente chiederebbe mai esplicitamente	Il team di sviluppo	Uno sprint o meno	“Configurare il job di pulizia automatica dei preventivi scaduti”
Bug	Un difetto da correggere (già visto nella sezione 3)	Chiunque lo scopre	Variabile, spesso urgente	“Il preventivo salvato perde lo sconto applicato”
Sub-task	Un pezzo di lavoro dentro una Story/Task più grande, non stimabile da solo come issue a sé	Chi lavora sulla Story/Task	Ore o giorni	“Predisporre la tabella preventivi nel database”

Il punto che vale la pena chiarire bene, perché genera confusione quanto il termine “issue”: **Story e Task convivono, non sono alternative**. La Story “salvare un preventivo” ha valore diretto per il cliente di ShopFacile; per realizzarla, il team la scompone in Sub-task (uno dei quali, “predisporre la tabella preventivi”, è un Task tecnico puro — nessun cliente lo chiederebbe mai, ma senza di esso la Story non esiste). Una Epic come “Preventivatore online” raggruppa più Story di questo tipo, ciascuna scomponibile a sua volta.

Backlog, sprint, board: lo stesso lavoro, vista diversa

Le stesse issue, con lo stesso stato, si vedono in modi diversi secondo cosa stai cercando di capire: nel **backlog** le vedi come lista ordinata, in attesa di essere pianificate; nello **sprint** attivo le vedi come il sottoinsieme selezionato per le prossime settimane; nella **board** (già vista nelle sezioni 6 e 7) le vedi disposte per colonna, secondo lo stato di avanzamento (Da fare, In corso, Fatto, e le colonne intermedie che il team ha definito).

Il passaggio da una colonna all'altra segue un **workflow**: le transizioni di stato permesse (una issue non può “saltare” da Da fare a Fatto senza passare da In corso, se il workflow non lo consente). Qui sta il punto che vale oro per un Project Manager: **se la board non rispecchia il modo in cui il lavoro scorre davvero**, le persone smettono di aggiornarla in tempo reale e la aggiornano tutta insieme a fine

giornata — a quel punto i dati che stai guardando sono una **ricostruzione a posteriori**, non lo stato reale del lavoro. Una board che mente non è un problema di Jira: è un problema di processo che Jira si limita a rendere visibile (o a nascondere, se nessuno la aggiorna).

Le due viste sullo stesso dato: board e Gantt

Le due sezioni precedenti — la WBS in 8.14 e le dipendenze/critical path in 8.15 — hanno già introdotto il ragionamento su **come le attività si incastrano nel tempo**. In Jira, quel ragionamento prende la forma di una vista **Gantt/timeline**, affiancata alla board Agile. Non sono due metodi in competizione: sono **due domande diverse sugli stessi dati**.

- La **board Agile** risponde a: chi sta facendo cosa **adesso**?
- Il **Gantt/timeline** risponde a: come le cose si incastrano **nel tempo**, con quali dipendenze tra loro?

Il limite da non dimenticare, in entrambi i casi: il Gantt di Jira è affidabile esattamente quanto le stime e le dipendenze che **qualcuno ha inserito a mano**. Se nessuno collega due Story con una dipendenza reale, il Gantt disegnerà due barre indipendenti che, nella realtà, non lo sono — la stessa illusione di precisione già vista nella sezione 8.15 a proposito del Gantt tradizionale.

JQL e i report che un PM userà davvero

Le issue di Jira si possono interrogare con un linguaggio di filtro chiamato **JQL** (Jira Query Language): permette di costruire ricerche precise (“tutte le Story dell'Epic Preventivatore, non completate, con priorità alta”) senza dover scorrere manualmente il backlog. Non serve scriverlo per capirsi con il team: basta sapere che esiste, per riconoscere un filtro salvato quando qualcuno lo condivide.

I report che userai più spesso: il **burndown** (già visto nella sezione 6, qui è semplicemente la stessa vista applicata dentro lo strumento), la **velocity** (sezione 6.7, aggregata sprint dopo sprint) e il **cumulative flow diagram**, che mostra quante issue si trovano in ciascuna colonna del workflow nel tempo — utile soprattutto per individuare un **collo di bottiglia**: se la colonna “In review” si allarga sempre di più mentre le altre restano stabili, il problema non è “il team scrive poco codice”, è che nessuno ha tempo per revisionarlo.

Trade-off: lo strumento amplifica il processo, non lo crea

Jira, come ogni strumento di questo tipo, non risolve un processo debole: lo **amplifica**, nel bene e nel male. Un team con un processo chiaro lo rende visibile e misurabile; un team senza un processo condiviso lo usa per litigare più velocemente su numeri diversi. Due rischi concreti da conoscere: un Jira personalizzato all'eccesso (campi su misura, workflow complicati, automazioni su automazioni) diventa un **secondo lavoro** da mantenere, che qualcuno deve gestire oltre al progetto vero; e i **campi obbligatori che nessuno usa davvero** (perché troppi, perché nessuno ne capisce lo scopo) producono dati completi solo nella forma — che nessuno, poi, guarda mai per decidere qualcosa.

8.22 Excel: reporting e WBS

Nel corso, finora, Excel è comparso solo come il foglio condiviso che Luca aggiorna la sera — raccontato, giustamente, come parte del problema che Jira risolve. Ma sarebbe un errore concludere che Excel è solo “lo strumento di ripiego di chi non ha ancora adottato uno strumento vero”: è, di fatto, lo strumento di project management più usato al mondo, e ci sono casi in cui **resta la scelta migliore**: prototipare velocemente una struttura prima di formalizzarla in Jira, un calcolo ad hoc che nessuno strumento di ticketing prevede, un’analisi una tantum che non merita un processo permanente, un report per un pubblico (un fornitore, un revisore esterno) che non ha e non avrà accesso al tuo strumento di ticketing.

La WBS in Excel

La WBS vista in 8.14 si presta particolarmente bene a un foglio Excel, proprio perché è gerarchica e stabile (a differenza del backlog, che cambia di continuo). Una struttura tipica usa **codici gerarchici** (1, 1.1, 1.1.1...) con colonne per attività, responsabile, stima e dipendenze:

Codice	Attività	Responsabile	Stima (giorni)	Dipende da
1	Modulo pagamenti ShopFacile	—	—	—
1.1	Integrazione fornitore esterno	—	—	—
1.1.1	Configurazione ambiente di test	Marco	2	—
1.1.2	Sviluppo connettore di pagamento	Marco	5	1.1.1
1.2	Sicurezza	—	—	—
1.2.1	Test di sicurezza sui token	Giulia	3	1.1.2

La numerazione gerarchica, da sola, è già un’informazione: leggendo solo i codici (senza nemmeno i nomi delle attività) capisci istantaneamente cosa è contenuto in cosa, e a quale livello di dettaglio ti trovi. Scendendo nei livelli si arriva al **work package** (già introdotto in 8.14): il livello più basso della scomposizione, quello abbastanza piccolo da poter essere stimato con sicurezza e assegnato a una singola persona — esattamente le righe 1.1.1, 1.1.2 e 1.2.1 della tabella sopra.

Reporting: le poche funzioni che contano davvero

Per il reporting in Excel, poche funzioni risolvono la maggior parte dei casi reali:

- le **tabelle pivot**, per riassumere rapidamente grandi elenchi (quante issue per stato, quante ore per persona, senza scrivere formule);
- **CERCA.VERT** o **CERCA.X**, per unire due elenchi che condividono una colonna in comune (es. un elenco di attività e un elenco di responsabili) — lo stesso principio della JOIN vista nella sezione 2 a proposito delle tabelle di un database, solo applicato a due fogli invece che a due tabelle SQL;
- la **formattazione condizionale**, per far saltare all’occhio subito i problemi (una cella che si colora di rosso se una data è passata e l’attività non è ancora completata);

- i **grafici**, per rendere leggibile a chi non guarda il dettaglio ogni giorno un andamento che altrimenti resterebbe sepolto in righe di numeri.

Un errore classico da conoscere, perché lo vedrai capitare: un report costruito su un'estrazione di dati fatta **a mano** una volta, che nessuno rifà nello stesso modo il mese dopo. Il numero nel report sembra identico nella forma, ma la procedura che lo ha prodotto è già cambiata (magari un filtro diverso, una colonna in più): il confronto tra un mese e l'altro, a quel punto, confronta cose diverse senza che nessuno se ne accorga.

Trade-off e limiti, senza sconti

I limiti di Excel, per essere onesti, sono altrettanto reali dei suoi vantaggi: **nessuna storia delle modifiche** utile quanto quella di Git (sezione 4) — capire chi ha cambiato cosa e quando è, nella pratica, quasi impossibile; **nessuna fonte unica di verità**, perché un file si copia, si scarica, si modifica in locale; il classico scenario delle **tre versioni dello stesso file in tre e-mail diverse**, con nessuno sicuro di quale sia quella corretta. E un rischio meno ovvio ma reale: un foglio Excel che, con il tempo, **diventa un processo aziendale** — usato da tutti, ma senza controllo di versione, senza backup dedicato, senza nessuno che ne sia davvero responsabile — è un rischio operativo silenzioso, non uno strumento “leggero e senza conseguenze”.

Il criterio pratico da portarti via: **Excel per pensare, lo strumento condiviso (Jira o equivalente) per far scorrere il lavoro**. Il primo è perfetto per esplorare un'idea prima che diventi un processo; il secondo è indispensabile una volta che quel lavoro deve essere visibile, tracciabile e condiviso da più persone nel tempo.

8.23 Riepilogo

In questa sezione hai visto come i concetti di project management si inseriscano in un contesto Agile/DevOps, senza contraddire ciò che hai imparato su Scrum e Kanban, ma completandolo con una vista più ampia sul progetto nel suo insieme:

- gli **stakeholder** ti dicono chi coinvolgere e come;
- il **RAID Log** ti dà un metodo per non farti sorprendere da rischi, assunzioni non verificate, problemi e dipendenze;
- la **RACI** chiarisce chi fa cosa, evitando ambiguità e conflitti, e il **Solution Architect** è il ruolo che tiene insieme la soluzione tra sistemi e team diversi, quando la RACI di un singolo team non basta più;
- le **milestone** segnano i traguardi che contano davvero, distinti dalle scadenze operative quotidiane;
- la gestione del **rischio** ti allena a pensare in anticipo, non solo a reagire;
- la **pianificazione a più livelli** (roadmap, release plan, sprint plan, daily) convive naturalmente con l'incertezza tipica di un progetto Agile;
- il **monitoraggio** (stand-up, burndown/burnup, dashboard) ti permette di intercettare deviazioni mentre sono ancora piccole;
- i **KPI** ti danno numeri oggettivi su cui basare le decisioni, se usati con criterio;

- la **reportistica** ti insegna che comunicare bene significa adattare il dettaglio a chi ascolta, non ripetere sempre lo stesso messaggio.

Hai poi collegato tutto questo al vocabolario più formale e riconosciuto a livello internazionale del **PMI**, la **PMBOK Guide** e la certificazione **PMP**, scoprendo che non è un metodo alternativo e in conflitto con Agile, ma una cassetta degli attrezzi con cui parlare la stessa lingua di manager, clienti e fornitori:

- i **5 process group** (Avvio, Pianificazione, Esecuzione, Monitoraggio e Controllo, Chiusura) descrivono attività che si ripetono, non fasi rigide — e si ritrovano, in piccolo, dentro ogni sprint;
- le **10 knowledge area** del PMBOK 6 (Integrazione, Ambito, Tempi, Costi, Qualità, Risorse, Comunicazione, Rischi, Approvvigionamenti, Stakeholder) sono, di fatto, gli stessi temi già visti in questa sezione, solo con un nome più formale;
- il **project charter** è l'atto di nascita del progetto, la **WBS** ne scompone il lavoro (in modo diverso da un Product Backlog), e insieme raccontano il “cosa” e il “quanto” del progetto;
- **schedule, percorso critico ed Earned Value Management** offrono un modo per misurare tempi e costi in modo rigoroso, utile soprattutto per scadenze esterne fisse, anche se in Agile il loro ruolo è spesso preso dalla velocity e dal burndown;
- il **triplo vincolo** (ambito, tempi, costi) e le **change request** spiegano perché una modifica non è mai “gratis”, e perché in Scrum è lo scope — non il tempo — la variabile che si adatta;
- **qualità, approvvigionamenti e comunicazione** completano la mappa, collegandosi rispettivamente ai quality gate della pipeline, ai fornitori esterni e alla reportistica già vista;
- la **chiusura del progetto** e le **lessons learned** sono, in fondo, la stessa idea della Sprint Retrospective, con una cadenza più ampia;
- l'**approccio ibrido** — usare il vocabolario PMP per governare budget e stakeholder, e il metodo Agile per il lavoro quotidiano del team — è la realtà di gran parte delle aziende in cui lavorerai.

Infine hai visto due strumenti molto concreti, che userai quasi ogni giorno: **Jira**, con la sua gerarchia di Epic/Story/Task/Sub-task e le due viste (board e Gantt) sullo stesso lavoro; ed **Excel**, che resta indispensabile per prototipare, calcolare e riportare, ma va tenuto separato dal ruolo di “fonte unica di verità” che spetta allo strumento condiviso del team.

Nelle prossime sezioni vedrai come molti di questi concetti si traducano in strumenti concreti: la cultura DevOps (sezione 9) e piattaforme come GitHub (già vista nella sezione 4) e la pipeline di CI/CD (sezione 10) offrono board, dashboard, Issue e pipeline che mettono in pratica esattamente ciò che hai visto qui — RAID Log, RACI, burndown chart e KPI diventano funzionalità cliccabili in uno strumento reale.



Esercizi pratici

Gli esercizi che seguono ti aiutano a passare dal “ho capito il concetto leggendolo” al “saprei applicarlo su un progetto reale”. Puoi farli con carta e penna, un foglio di calcolo, o uno strumento online gratuito: lo strumento conta meno del ragionamento che ci metti dietro.

1. **Mappa gli stakeholder di un progetto immaginario.** Scegli un progetto software semplice (es. “un’app per prenotare turni in palestra”) ed elenca almeno 6 stakeholder diversi. Per ciascuno, assegna una posizione sulla matrice potere/interesse (come quella della sezione 8.1) e scrivi in

una riga **come** li terrestri aggiornati (es. “email mensile”, “invito al Sprint Review”, “coinvolgimento diretto nelle decisioni”).  **Come verificare:** se per almeno uno stakeholder non sai dire “cosa vuole” e “come lo tieni informato” in una frase, la mappatura non è ancora abbastanza concreta — torna a rileggere la sezione 8.1.

2. **Costruisci un RAID Log di 8 righe per un progetto reale o immaginario.** Deve contenere almeno 2 rischi, 2 assunzioni, 2 issue e 2 dipendenze, ciascuna con una stima di impatto (scala 1-5) e un responsabile assegnato.  **Come verificare:** prova a ordinare le righe per impatto decrescente. Se le prime tre righe della lista non hanno ancora un’azione di mitigazione scritta, il RAID Log è incompleto — un RAID Log senza azioni è solo un elenco di preoccupazioni, non uno strumento di gestione.
3. **Disegna una matrice RACI per un’attività che conosci.** Scegli un’attività diversa da quella già usata come esempio nella sezione 8.3 (ad esempio: “pianificazione di uno sprint”, “gestione di un incidente in produzione”, “onboarding di un nuovo membro del team”) e costruisci una tabella RACI con almeno 5 righe/attività e almeno 4 ruoli.  **Come verificare:** conta le “A” in ogni riga. Se una riga ha zero A o più di una A, correggila: è l’errore più comune di chi costruisce una RACI per la prima volta.
4. **Calcola e interpreta due KPI su dati inventati.** Immagina un team che, in 4 sprint consecutivi, completa rispettivamente 28, 25, 30 e 18 story point, con 1, 2, 1 e 6 bug rilevati in produzione nello stesso periodo. Calcola la velocity media dei primi tre sprint e confrontala con il quarto; scrivi in 3-4 righe quale ipotesi formularesti su cosa è successo nel quarto sprint, motivando la risposta con i dati (non solo con un’impressione).  **Come verificare:** la tua ipotesi deve citare **entrambi** i KPI (velocity e defect rate) insieme, non uno isolato — se la tua conclusione si basa solo sul calo della velocity, rileggi l’avvertimento sui KPI della sezione 8.8.
5. **Scrivi due versioni dello stesso aggiornamento di stato.** Pensa a un contrattempo plausibile in un progetto software (un bug bloccante trovato tardi, un ritardo di un fornitore, un test di sicurezza fallito) e scrivi due comunicazioni brevi (massimo 3-4 righe ciascuna): una per il cliente/management, una per il team in stand-up.  **Come verificare:** fai leggere la versione “per il cliente” a qualcuno senza background tecnico (anche un amico non del settore): se non capisce cosa sta succedendo e cosa aspettarsi, è ancora troppo tecnica.
6. **Simula una retrospettiva sui rischi di un progetto concluso (anche immaginario).** Elenca 3 rischi che, con il senno di poi, si sono effettivamente concretizzati in issue, e per ciascuno scrivi: come lo avresti potuto identificare prima, e quale delle quattro strategie di mitigazione (evitare, ridurre, trasferire, accettare) avrebbe avuto più senso applicare in anticipo.  **Come verificare:** per ogni rischio, la strategia di mitigazione che scegli deve essere diversa da un generico “fare più attenzione” — deve essere un’azione concreta e verificabile (es. “richiedere conferma scritta entro una data”, non “stare più attenti alle dipendenze”).
7. **Scrivi un mini project charter.** Scegli un progetto software semplice (può essere lo stesso “app per prenotare turni in palestra” usato nell’esercizio 1, o un progetto tuo) e compila le sei voci viste nella sezione 8.13 (obiettivo, scope di alto livello, stakeholder chiave, budget/tempi indicativi, sponsor, criteri di successo) in non più di una pagina.  **Come verificare:** fai leggere

solo la riga “criteri di successo” a qualcuno che non conosce il progetto: deve capire, da quella riga sola, come si misurerà se il progetto ha funzionato. Se non ci riesce, il criterio è troppo vago (es. “andare bene” non è un criterio verificabile).

8. **Disegna una WBS a 2 livelli.** Scegli una funzionalità software di media complessità (es. “sistema di recensioni prodotto” per ShopFacile) e scomponila in una struttura gerarchica di almeno 3 rami principali, ciascuno con 2-3 sotto-attività, sul modello dell'esempio della sezione 8.14.  **Come verificare:** ogni “foglia” della tua WBS (l'ultimo livello di scomposizione) deve essere un'attività che una singola persona potrebbe stimare in giorni, non settimane. Se non riesci a stimarla, scomponila ancora.
9. **Calcola SPI e CPI su dati forniti.** Un progetto ha una PV (Planned Value) di 20.000 €, un EV (Earned Value) di 15.000 € e un AC (Actual Cost) di 18.000 €. Calcola SPI e CPI e scrivi, in 2-3 righe, cosa diresti a uno sponsor che ti chiede “come va il progetto?”, usando entrambi gli indicatori.  **Come verificare:** $SPI = 15.000/20.000 = 0,75$ (in ritardo sui tempi); $CPI = 15.000/18.000 \approx 0,83$ (sopra budget rispetto al valore prodotto). Se la tua risposta allo sponsor cita solo uno dei due numeri, rileggi la sezione 8.16: i due indicatori raccontano cose diverse (tempi vs costi) e vanno letti insieme, come i KPI della sezione 8.8.
10. **Traduci 5 termini tra vocabolario PMP e vocabolario Agile.** Usando la tabella della sezione 8.20 come riferimento (ma senza guardarla mentre rispondi), scrivi a memoria il corrispettivo Agile di: Project Charter, WBS, Change Control Board, Earned Value Management, Lessons Learned. Per ciascuna coppia, scrivi anche una riga su **cosa cambia** tra i due (non solo il nome diverso).  **Come verificare:** controlla le tue risposte con la tabella della sezione 8.20. Se per una coppia non riesci a scrivere “cosa cambia” (e ti limiti a dire che sono “la stessa cosa con un altro nome”), rileggi la sottosezione corrispondente: quasi sempre c'è una differenza concreta (es. struttura, stabilità, chi decide).
11. **Classifica issue nella gerarchia di Jira.** Scegli una funzionalità software di media complessità (puoi riusare quella dell'esercizio 8) e scrivi: una possibile Epic che la contiene, 2 Story con valore diretto per l'utente, 1 Task tecnico che nessun utente chiederebbe mai, e 2 Sub-task per una delle Story.  **Come verificare:** se il tuo Task “tecnico” in realtà descrive un beneficio visibile all'utente, probabilmente è una Story mascherata da Task — rileggi la distinzione della sezione 8.21.
12. **Riconosci quando un lavoro sta meglio in Excel o in Jira.** Elenca 4 situazioni plausibili in un progetto (es. “calcolare una volta il costo di un'opzione tecnica alternativa”, “tracciare lo stato di 40 Story per lo sprint in corso”) e per ciascuna decidi quale dei due strumenti useresti, motivando con il criterio della sezione 8.22.  **Come verificare:** se hai scelto Excel per un caso che richiede aggiornamenti quotidiani da più persone, o Jira per un calcolo isolato che nessuno riuserà, rileggi il criterio “Excel per pensare, lo strumento condiviso per far scorrere il lavoro”.
13. **Distingui Solution Architect da Tech Lead in uno scenario.** Immagina che due team di ShopFacile debbano integrare i rispettivi sistemi (es. catalogo prodotti e gestione ordini) e stai decidendo chi coinvolgere e quando. Scrivi in 3-4 righe cosa chiederesti al Solution Architect e cosa chiederesti invece al Tech Lead di ciascun team.  **Come verificare:** se le domande che faresti ai due ruoli sono intercambiabili, rileggi la tabella di confronto della sezione 8.3 — dovrebbe emergere una differenza di ambito (tra i team vs dentro un team), non solo di titolo.

Collegamenti

- [3. Come nasce un software](#) — i requisiti non funzionali che il Solution Architect presidia, e gli ADR con cui documenta le decisioni tra sistemi
- [4. Git e GitHub](#) — dove Issue, Pull Request e GitHub Projects diventano gli strumenti concreti per tracciare RAID Log e dashboard
- [5. Agile](#) — i principi che spiegano perché, in un contesto Agile, lo scope è la variabile che si adatta e non il tempo
- [6. Scrum](#) — i ruoli (Scrum Master, Product Owner), le User Story e le Epic che in Jira diventano oggetti concreti, e la Sprint Retrospective che dialoga con Project Manager, sponsor e lessons learned del PMBOK
- [9. DevOps](#) — la cultura e le pratiche che rendono possibile un monitoraggio continuo e KPI come deployment frequency e MTTR
- [10. CI/CD](#) — i quality gate della pipeline come automazione concreta del Quality Control visto in 8.18, e Jira Software come strumento di tracking collegato alla pipeline
- [16. Glossario](#) — le definizioni sintetiche di RAID Log, RACI, milestone, change request e degli altri termini incontrati in questa sezione

Risorse

- [PMI — Project Management Institute](#)
- [PMBOK Guide — panoramica](#)
- PMBOK Guide, 7th Edition (2021) — l'edizione più recente, basata su 12 principi e 8 performance domain invece dei 5 process group/10 knowledge area della 6th Edition
- PMI Standard for Project Management — lo standard incluso insieme alla PMBOK Guide 7th Edition, che ne definisce i principi generali
- Certificazione PMP (Project Management Professional) — verifica sempre i requisiti aggiornati (ore di esperienza, formazione, esame) direttamente sul sito PMI, perché cambiano periodicamente
- Certificazione PMI-ACP (Agile Certified Practitioner) — la certificazione del PMI dedicata specificamente all'Agile
- [Atlassian — Guida ai KPI Agile](#)
- [Atlassian — Cos'è un RACI Chart](#)
- [Atlassian — Stakeholder analysis](#)
- [Atlassian — Burndown chart](#)
- [Scrum.org — Risk Management in Scrum](#)
- [Axelos — PRINCE2 e RAID Log](#)
- [Atlassian — Jira Software, panoramica](#)
- [Atlassian — Guida a Epic, Story e Task in Jira](#)
- [Atlassian — Cumulative flow diagram](#)
- [Microsoft Support — Panoramica delle tabelle pivot in Excel](#)
- [Wikipedia — Enterprise Architecture](#) — per la differenza di ambito tra Enterprise Architect e Solution Architect

9. DevOps

Nelle sezioni precedenti hai imparato **come un team organizza il lavoro** (Agile, Scrum, Kanban) e **come si tiene sotto controllo un progetto** nel suo complesso (Project Management, con stakeholder, RAID Log, RACI, KPI). Questa sezione affronta un tema diverso, e altrettanto centrale per il lavoro che farai ogni giorno: **come il software, una volta scritto, arriva davvero nelle mani degli utenti — e come ci resta, funzionando bene, nel tempo.**

Questo è il mondo di **DevOps**. È una delle parole che sentirai pronunciare più spesso nel tuo team, quasi sempre associata a strumenti molto concreti (pipeline, ambienti, dashboard) che vedrai nel dettaglio nella prossima sezione, dedicata al CI/CD. Ma prima di usare gli strumenti, devi capire **l'idea** che c'è dietro — perché DevOps, prima di essere una piattaforma o un insieme di pratiche tecniche, è **un modo di pensare al lavoro del team**. Se capisci bene questa sezione, tutto quello che vedrai dopo (pipeline, ambienti, monitoraggio, sicurezza) ti sembrerà la conseguenza naturale di un'idea semplice, non un elenco di strumenti scollegati tra loro.

Per rendere tutto più concreto, in questa sezione useremo un unico progetto di riferimento, che ritroverai anche nelle sezioni successive: **ShopFacile**, una piattaforma e-commerce (catalogo prodotti, carrello, ordini, pagamenti, sconti) sviluppata da un piccolo team interno composto, tra gli altri, da **Marco** e **Giulia** (developer), **Ahmed** (developer junior, in crescita), **Sara** (Product Owner) e **Luca** (Scrum Master). Vedrai questi nomi tornare più volte: non sono persone diverse in ogni esempio, ma lo stesso team che affronta, di volta in volta, un problema diverso.

Obiettivi della sezione

Alla fine di questa sezione saprai:

- spiegare **cos'è DevOps** e perché non è “uno strumento” ma una cultura;
 - raccontare **perché è nato** DevOps, partendo dal problema storico del “muro della confusione” tra sviluppo e infrastruttura;
 - riconoscere gli elementi chiave della **cultura DevOps** (collaborazione, responsabilità condivisa, blameless culture) e il modello **CALMS**;
 - capire perché **l'automazione** è al centro di DevOps, con esempi concreti;
 - distinguere con precisione **Continuous Integration**, **Continuous Delivery** e **Continuous Deployment** — tre termini che si confondono facilmente, ma che indicano cose diverse;
 - capire cos'è **l'Infrastructure as Code** e perché ha cambiato il modo di gestire server e reti;
 - distinguere **monitoring**, **observability** e **logging**, tre attività complementari per capire lo stato di un sistema in produzione;
 - riconoscere i processi principali dell'**ITSM** (incident, problem e change management, service request) e il vocabolario **ITIL** che li descrive;
 - capire cos'è **l'elaborazione batch** e perché la finestra notturna e le dipendenze tra job sono un vincolo di progetto, non solo un dettaglio tecnico.
-

9.1 Cos'è DevOps: non è un tool, è una cultura

La parola **DevOps** nasce dalla fusione di due parole inglesi:

- **Development** — lo **sviluppo** del software: scrivere codice, creare nuove funzionalità, correggere bug.
- **Operations** — le **operazioni**: tutto ciò che serve per far funzionare quel software in un ambiente reale, con utenti reali — server, reti, database, sicurezza, backup, monitoraggio.

DevOps è quindi, letteralmente, **l'unione di questi due mondi**. Ma qui sta il punto più importante da capire, e su cui vale la pena insistere subito: **DevOps non è un software che si installa, non è un ruolo con la targhetta “DevOps Engineer” sulla porta, e non è nemmeno un insieme fisso di pipeline**. DevOps è prima di tutto **una cultura**: un modo di organizzare le persone, i processi e (solo successivamente) gli strumenti, in modo che chi scrive il codice e chi lo fa funzionare in produzione lavorino **insieme**, verso lo stesso obiettivo, invece che come due squadre separate con interessi contrapposti.

 **Analogia:** pensa a un ristorante. Se la cucina (chi cucina i piatti, lo “sviluppo”) e la sala (chi li serve al cliente e gestisce i reclami, le “operazioni”) lavorano come due mondi separati — la cucina si disinteressa di cosa succede quando il piatto arriva al tavolo, la sala non sa mai perché un piatto è in ritardo o come è stato preparato — il risultato è un servizio lento, con errori che nessuno riesce a spiegare e tutti scaricano sull'altro reparto. Un ristorante che funziona bene ha invece cucina e sala che comunicano costantemente: sanno cosa succede dall'altra parte, si scambiano informazioni in tempo reale, e se un piatto torna indietro non si accusano a vicenda, ma capiscono insieme cosa è andato storto per non farlo succedere di nuovo. DevOps è esattamente questo, applicato allo sviluppo software: fare in modo che “chi cucina il codice” e “chi lo serve agli utenti” lavorino come **una sola squadra**, non come due squadre che si passano un pacco.

Vale la pena essere ancora più precisi su un equivoco molto comune, perché lo sentirai spesso sul lavoro: quando qualcuno dice “usiamo GitHub Actions” o “abbiamo automatizzato con strumenti DevOps”, si riferisce agli **strumenti** che rendono più facile *applicare* la cultura DevOps (ad esempio le pipeline di cui parleremo più avanti). Ma installare quegli strumenti **non rende automaticamente un team “DevOps”**, così come comprare attrezzi da falegname non fa di te un falegname. Gli strumenti aiutano, e sono importanti — li vedrai nel dettaglio nella prossima sezione — ma senza il cambiamento di cultura e di collaborazione, restano scatole vuote.

 **Nota per un futuro Project Manager:** quel titolo “DevOps Engineer” esiste davvero. Sopra abbiamo detto che DevOps non dovrebbe essere un ruolo con una targhetta sulla porta — ed è un punto di vista difendibile. Ma se guardi un organigramma reale, o un annuncio di lavoro, quel titolo c'è, e continuerai a incontrarlo per tutta la carriera. In pratica, chi porta quel titolo si occupa soprattutto di pipeline, di automazione dell'infrastruttura (l'IaC che vedrai al paragrafo 9.8), di piattaforme interne (le vedrai nella sezione 12) e di affidabilità dei sistemi in produzione: fa, insomma, il lavoro tecnico che rende possibile la cultura DevOps. Il purista storce il naso perché

teme che dare a una sola persona la targhetta “DevOps” faccia sentire il resto del team autorizzato a disinteressarsene — ricostruendo, con un’etichetta diversa, lo stesso muro della confusione. Per te, in pratica: leggere quel titolo su un CV o su un organigramma ti dice “questa persona sa di pipeline e infrastruttura”, non “questa persona è responsabile lei sola della qualità in produzione”.

Lo stesso vale per **Operations**: non è finita insieme al muro della confusione. Esiste ancora oggi come disciplina, con persone che se ne occupano a tempo pieno — cambia *come* lavora (in modo più collaborativo, spesso nella forma di **SRE**, che vedrai al paragrafo 9.3), non il fatto che continui a esistere.

Ma se DevOps è “solo” una cultura, perché è diventata così centrale nel lavoro quotidiano di un team come quello di ShopFacile? Per rispondere bisogna guardare al problema concreto che ha spinto a inventarla: torniamo indietro a come lavoravano i team **prima** che DevOps esistesse.

9.2 Perché nasce DevOps: il muro della confusione

Per capire perché DevOps è diventato così importante, vale la pena vederlo succedere davvero, non solo descriverlo in astratto — tornando a un periodo di ShopFacile precedente a quello che conosci finora, quando sviluppo e infrastruttura erano ancora due reparti separati e la cultura DevOps non era stata ancora adottata.

Marco ha pronto un fix urgente per un bug che blocca il checkout di una parte dei clienti di ShopFacile: vuole rilasciarlo subito. Il responsabile dell’infrastruttura di allora blocca il rilascio: un cambiamento non pianificato è, sui suoi dati, la causa più frequente dei guasti notturni di cui risponde lui. **Entrambi hanno ragione**: Marco è valutato su quanto in fretta risolve i problemi degli utenti, il responsabile su quanto resta stabile il sistema durante la notte — due obiettivi legittimi, in quel momento, in conflitto.

Non è un problema specifico di ShopFacile: è così comune che ha preso un nome — il **muro della confusione**. Vediamo da dove nasce, in generale.

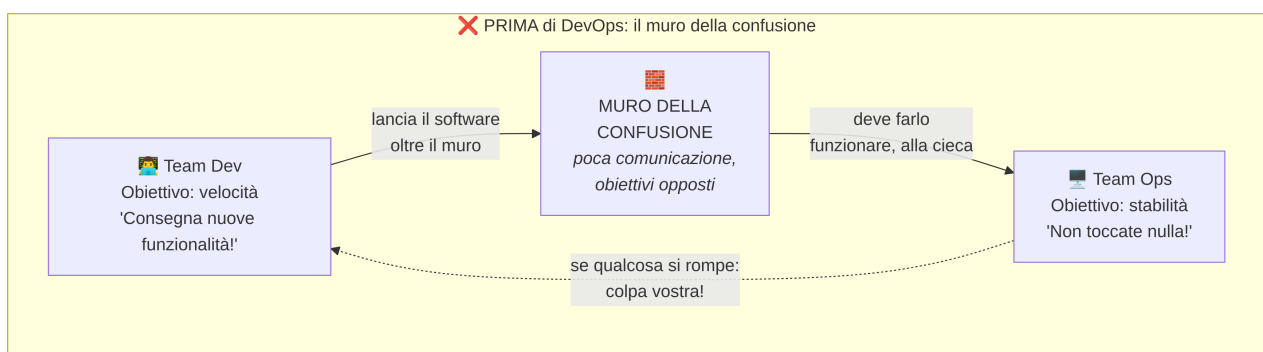
Il mondo prima di DevOps: due team, due obiettivi opposti

Tradizionalmente, in molte organizzazioni, esistevano due reparti separati, spesso con manager diversi, obiettivi diversi e persino uffici diversi:

- Il team di **sviluppo (Dev)**, il cui obiettivo — spesso l’unico su cui veniva valutato — era **consegnare nuove funzionalità velocemente**. Più release, più funzionalità, più velocità: questo veniva premiato.
- Il team di **operazioni/infrastruttura (Ops)**, il cui obiettivo era **mantenere i sistemi stabili, sicuri e sempre disponibili**. Ogni cambiamento nei sistemi in produzione è un potenziale rischio di down: il loro obiettivo, spesso l’unico su cui venivano valutati, era **la stabilità**, non la velocità.

Il problema è evidente non appena lo si scrive esplicitamente: **questi due obiettivi sono in tensione tra loro**. Il team di sviluppo vuole rilasciare spesso e velocemente; il team di operazioni vuole cambiare le cose il meno possibile, perché ogni cambiamento è un rischio per la stabilità che deve garantire. Non è che uno dei due team avesse torto: **ciascuno stava ottimizzando esattamente per quello su cui veniva giudicato**, ma il risultato complessivo era un conflitto strutturale.

Questa frizione, con il tempo, ha preso un nome molto azzeccato nella comunità tecnica: “**il muro della confusione**” (**wall of confusion**). Il team di sviluppo “lanciava” il software oltre un muro metaforico verso il team di operazioni, che doveva farlo funzionare in produzione senza averlo scritto, spesso senza documentazione adeguata, e senza aver preso parte alle decisioni di progettazione. Quando qualcosa si rompeva in produzione, ciascun team tendeva ad accusare l'altro: “il codice ha un bug” contro “l'infrastruttura non era configurata bene” — e nessuno aveva una visione completa per capire davvero cosa fosse successo.



Perché questo era un problema reale (non solo “scomodo”)

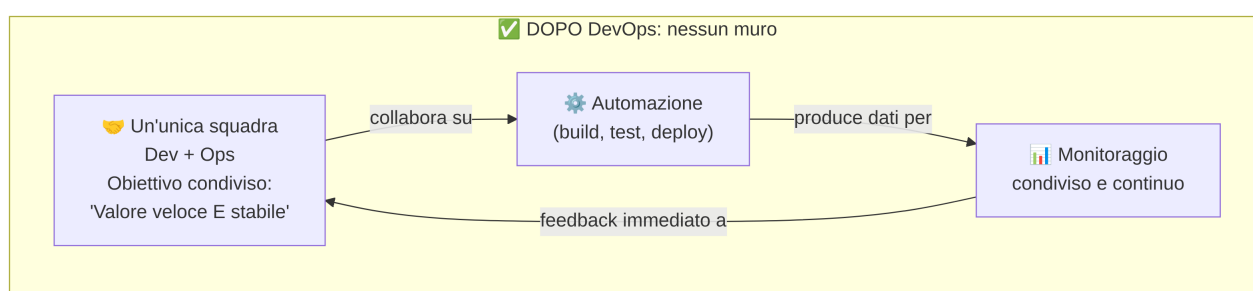
Questo modello creava conseguenze molto concrete, che vale la pena elencare perché le riconoscerai facilmente in aziende che non hanno ancora adottato una cultura DevOps:

- **Rilasci rari e rischiosi:** se rilasciare è complicato, doloroso e richiede il coinvolgimento manuale di tante persone, i team tendono a rilasciare raramente (una volta al mese, o anche meno). Ma rilasci rari significano rilasci **grandi**, che accumulano tanti cambiamenti insieme — e più cambiamenti insieme, più è probabile che qualcosa si rompa, e più è difficile capire *quale* cambiamento specifico ha causato il problema.
- **Colpa reciproca invece di soluzioni:** quando qualcosa andava storto in produzione, il tempo veniva spesso speso a capire “di chi è la colpa” invece che a risolvere il problema e capire come evitarlo in futuro.
- **Comunicazione tardiva:** gli sviluppatori progettavano soluzioni senza sapere come sarebbero state effettivamente eseguite in produzione (quante risorse servono, quali vincoli di sicurezza esistono); il team operazioni riceveva richieste dell'ultimo minuto, senza tempo per prepararsi.
- **Lentezza percepita dal cliente:** il risultato finale, quello che conta davvero per chi ha commissionato il progetto, era che le nuove funzionalità richiedevano troppo tempo per arrivare davvero in mano agli utenti, anche quando il codice era pronto da settimane.

💡 Nota per un futuro Project Manager: molti dei rischi e delle issue che hai imparato a tracciare nel RAID Log (sezione 8) — ritardi nei rilasci, problemi di comunicazione tra team, dipendenze che si scoprono troppo tardi — sono spesso **sintomi diretti** dell'assenza di una cultura DevOps. Non è un caso che questi due argomenti siano collegati.

La soluzione: eliminare il muro, non spostarlo

DevOps nasce esattamente per rispondere a questo problema, e lo fa non “velocizzando” il lancio oltre il muro, ma **eliminando il muro stesso**. L'idea è che sviluppo e operazioni non siano più due team separati con obiettivi opposti, ma **un'unica squadra, con un obiettivo condiviso**: consegnare valore agli utenti **velocemente e in modo stabile**, non l'uno a scapito dell'altro.



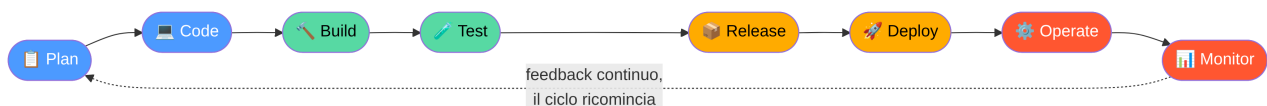
Il punto chiave visualizzato nel diagramma sopra è che il flusso non è più una linea a senso unico che “lancia” il lavoro da un team all'altro, ma un **ciclo continuo di collaborazione e feedback**: lo stesso team che scrive il codice si occupa (insieme, non da solo) anche di come funziona in produzione, usa l'automazione per ridurre il rischio dei rilasci frequenti, e riceve informazioni immediate su come il software si comporta nel mondo reale — informazioni che tornano utili per il prossimo ciclo di sviluppo. Vedremo nella prossima sezione (9.3) come questo ciclo si rappresenta in modo ancora più esplicito con il celebre “simbolo a infinito” di DevOps.

9.3 Il ciclo DevOps: un percorso senza fine

Uno dei modi più famosi per rappresentare DevOps visivamente è un simbolo a forma di **infinito** (∞), diviso in otto fasi che si susseguono e non finiscono mai: appena termina “Operate/Monitor”, si riparte immediatamente con “Plan” per il ciclo successivo. Questo simbolo comunica un messaggio molto preciso: **DevOps non è un progetto con un inizio e una fine, ma un flusso continuo**.

Le otto fasi tipiche del ciclo sono:

Fase	Cosa succede	Chi è coinvolto principalmente
Plan	Si decide cosa costruire: requisiti, priorità, backlog (collegamento diretto con quanto visto in Agile/Scrum)	Product Owner, team, stakeholder
Code	Si scrive il codice della nuova funzionalità o della correzione	Sviluppatori
Build	Il codice viene compilato/assemblato in un pacchetto eseguibile, in modo automatico	Automazione (pipeline)
Test	Il pacchetto viene testato automaticamente (test unitari, di integrazione, talvolta di sicurezza)	Automazione + QA
Release	Il pacchetto testato viene preparato e approvato per il rilascio	Team + approvazioni
Deploy	Il pacchetto viene effettivamente installato/attivato nell'ambiente di produzione (o di test)	Automazione (pipeline)
Operate	Il sistema funziona in produzione, gestito e mantenuto	Operations / SRE
Monitor	Si osserva come si comporta il sistema in produzione, raccogliendo dati utili per il prossimo ciclo di pianificazione	Tutti, grazie a strumenti di monitoraggio



Nota una cosa importante nella tabella e nel diagramma: le prime due fasi (Plan, Code) corrispondono grosso modo al mondo **Dev** “classico”; le ultime due (Operate, Monitor) al mondo **Ops** “classico”. Le fasi centrali (Build, Test, Release, Deploy) sono la “zona di confine” dove, storicamente, si trovava il muro della confusione — ed è proprio lì che l'**automazione** gioca il ruolo più importante, come vedremo nella prossima sezione, perché è lì che il lavoro passa fisicamente da un mondo all'altro.

Il dettaglio da ricordare: **Monitor non è la fine del ciclo, è l'inizio del ciclo successivo**. I dati raccolti osservando il sistema in produzione (quanti utenti usano una funzionalità, dove si verificano errori, quali parti sono lente) diventano input diretto per la prossima fase di Plan. Questo è il motivo per cui il simbolo è un infinito e non una linea con un punto di arrivo.

Nella riga “Operate” hai notato la sigla **SRE**: vale la pena chiarirla, perché in azienda sentirai usare “DevOps” e “SRE” come sinonimi, spesso a torto. **Site Reliability Engineering (SRE)** è una disciplina nata in Google che applica un approccio ingegneristico, con obiettivi numerici precisi (gli *error budget*,

quanta

indisponibilità ci si può permettere), al mantenere un sistema affidabile in produzione. In sintesi:

DevOps è la cultura che abbatte il muro tra Dev e Ops; **SRE è un modo specifico, orientato ai dati, di metterla in pratica**, soprattutto in Operate e Monitor — non tutte le aziende “DevOps” fanno anche SRE in senso stretto.

Conoscere le otto fasi del ciclo, però, non basta a farle funzionare bene insieme: perché quel ciclo scorra senza attriti, serve che le persone che lo attraversano — chi scrive il codice, chi lo gestisce in produzione — si comportino secondo alcuni principi condivisi. È proprio a questi principi, cioè alla cultura DevOps vera e propria, che dedichiamo il prossimo paragrafo.

9.4 La cultura DevOps: collaborazione, responsabilità, fiducia

Abbiamo detto che DevOps è prima di tutto una cultura. Ma cosa significa concretamente? Significa un insieme di comportamenti e valori condivisi dal team, che si traducono in scelte quotidiane precise.

Collaborazione, non “passaggio di consegne”

Nel modello vecchio, lo sviluppo “consegnava” al team operazioni, e la comunicazione era spesso a senso unico e tardiva (un po’ come il modello “Informed” della RACI che hai visto nella sezione 8, applicato però a qualcosa che avrebbe avuto bisogno di molto più coinvolgimento). Nella cultura DevOps, sviluppo e operazioni **collaborano dall’inizio**: chi scrive il codice pensa già a come sarà eseguito in produzione (quante risorse servono, come si comporta sotto carico, come si può monitorare); chi gestisce l’infrastruttura è coinvolto già nelle fasi di progettazione, non solo quando il software è “pronto” e deve solo essere installato.

Responsabilità condivisa: “you build it, you run it”

Una delle frasi più citate nella cultura DevOps è “**you build it, you run it**” (“chi lo costruisce, lo gestisce anche”). Significa che il team che scrive una funzionalità non se ne “disinteressa” una volta rilasciata: resta coinvolto (spesso tramite turni di reperibilità, i cosiddetti *on-call*) anche quando quella funzionalità è in produzione e qualcosa va storto.



Analogia: è la differenza tra un ristorante dove il cuoco cucina e se ne va, lasciando alla sala il compito di gestire ogni reclamo su un piatto che non conosce nei dettagli, e un ristorante dove — se un cliente segnala un problema con un piatto — quel piatto torna direttamente al cuoco che lo ha preparato, perché è lui che ha davvero le informazioni e gli strumenti per capire cosa è andato storto e correggerlo.

Questo cambia gli incentivi in modo molto concreto: se sai che sarai tu (o il tuo team) a dover gestire un problema alle tre di notte, scriverai codice più robusto, penserai di più a come monitorarlo, e sarai più motivato a collaborare con chi gestisce l’infrastruttura, perché non è più “un problema di qualcun altro”.

Questo beneficio, però, ha anche un costo umano reale, che un futuro Scrum Master non può ignorare: essere svegliati di notte per un guasto in produzione non è gratis per nessuno. Un turno di reperibilità mal gestito — sempre la stessa persona, nessuna compensazione, nessuna rotazione tra Marco, Giulia e Ahmed — logora chi lo sostiene e finisce per scontrarsi con uno dei valori che hai visto in Agile (sezione 5): un ritmo di lavoro **sostenibile** nel tempo. Un team DevOps maturo organizza l'on-call a rotazione, lo compensa esplicitamente e ne tiene conto nella pianificazione degli sprint successivi — non lo tratta come un dettaglio tecnico invisibile al project management.

Blameless culture: imparare dagli errori senza cercare un colpevole

Uno dei pilastri più importanti — e spesso più difficili da costruire davvero — della cultura DevOps è la cosiddetta **blameless culture** (cultura “senza colpa”). L'idea è semplice da enunciare ma richiede maturità organizzativa per essere applicata davvero: **quando qualcosa va storto (un bug critico, un'interruzione del servizio), l'obiettivo dell'analisi che segue non è trovare “di chi è la colpa” per punirlo, ma capire le cause reali per evitare che accada di nuovo.**

 **Analogia:** pensa alla differenza tra un'aviazione civile e un ambiente dove ogni errore viene punito severamente. Quando un aereo ha un incidente o un “quasi incidente”, gli investigatori non cercano primariamente “chi punire”: cercano di ricostruire la catena di eventi (un errore umano, ma anche una procedura poco chiara, uno strumento mal progettato, una comunicazione ambigua) per correggere il **sistema**, non solo la persona. Questo è uno dei motivi per cui il trasporto aereo è diventato via via più sicuro nel tempo: chi commette un errore in buona fede lo segnala subito, perché sa che verrà usato per migliorare, non per essere licenziato.

Nel contesto DevOps, questo si traduce in pratiche concrete come il **post-mortem** (o *retrospettiva sull'incidente*): dopo un problema importante in produzione, il team si riunisce per ricostruire cosa è successo, con una linea temporale precisa, e per proporre azioni concrete di miglioramento — senza che il documento serva a puntare il dito su una persona specifica. Se le persone temono di essere colpevolizzate, tendono a **nascondere i problemi o segnalarli in ritardo** — esattamente il contrario di quello che un team DevOps efficace ha bisogno di fare.

Il modello CALMS: cinque pilastri per riconoscere la cultura DevOps

Un modo diffuso per ricordare gli elementi fondamentali della cultura DevOps è l'acronimo **CALMS**:

Lettera	Pilastro	Significato
C	Culture (Cultura)	Collaborazione, fiducia reciproca, responsabilità condivisa tra Dev e Ops, blameless culture
A	Automation (Automazione)	Automatizzare compiti ripetitivi e a rischio di errore umano: build, test, deploy, provisioning dell'infrastruttura
L	Lean (Snellezza)	Eliminare gli sprechi nel processo: passaggi inutili, attese, lavoro in eccesso non necessario (lo stesso principio che hai visto in Kanban)
M	Measurement (Misurazione)	Misurare tutto ciò che conta con dati oggettivi: quanto spesso si rilascia, quanto tempo serve per riparare un guasto, quanti errori arrivano in produzione (i KPI DevOps visti nella sezione 8, come deployment frequency e MTTR)
S	Sharing (Condivisione)	Condividere conoscenza, strumenti, successi e insuccessi tra i team, per non ripetere gli stessi errori e non "reinventare la ruota"

 **Esempio pratico:** immagina che il team di **ShopFacile** rilasci ancora manualmente, una volta al mese, seguendo una checklist su carta, senza sapere quanto tempo richieda in media risolvere un guasto. Applicando CALMS:

- **Culture:** **Marco** (developer) e chi si occupa dell'infrastruttura iniziano a partecipare alla stessa retrospettiva, invece di due riunioni separate che non si parlano.
- **Automation:** la checklist manuale viene trasformata in una pipeline che esegue build, test e deploy in automatico, riducendo gli errori.
- **Lean:** il team si rende conto che 3 dei 12 passaggi della vecchia checklist non servivano più a nulla, ed erano solo un'abitudine mai rimessa in discussione.
- **Measurement:** **Luca** (Scrum Master) inizia a far misurare al team la *deployment frequency* (quante volte si rilascia) e l'*MTTR* (quanto tempo serve per risolvere un guasto), passando da decisioni "a sensazione" a decisioni basate su dati concreti.
- **Sharing:** la pipeline creata per il catalogo prodotti di ShopFacile viene documentata da **Giulia** e riutilizzata anche per il servizio ordini, invece di essere reinventata da zero.

Vale la pena notare che **la Cultura è la prima lettera, non l'ultima**: non è un caso. Molte aziende provano a “fare DevOps” partendo dall'automazione o dagli strumenti, saltando il lavoro culturale — e spesso falliscono, perché installano pipeline sofisticate sopra un'organizzazione che continua a comportarsi con la vecchia logica del “muro della confusione”. Gli strumenti amplificano una buona cultura; non la creano da soli.

Aver visto CALMS ci porta dritti a una delle sue lettere in particolare: la **A** di Automation. È il pilastro più “tecnico” e visibile dei cinque, ed è talmente centrale nel lavoro quotidiano del team di ShopFacile che merita un paragrafo tutto suo.

9.5 Automazione: il motore pratico di DevOps

Se la cultura è il “perché” di DevOps, l'**automazione** è il “come” con cui quella cultura si traduce in pratica quotidiana. L'idea di base è semplice: **ogni compito ripetitivo, manuale e prevedibile che un umano farebbe sempre nello stesso modo è un candidato ideale per essere automatizzato**.

Perché è così importante automatizzare? Per tre ragioni concrete:

1. **Riduce gli errori umani.** Un essere umano che ripete lo stesso compito manuale centinaia di volte, prima o poi si distrae, salta un passaggio, digita qualcosa di sbagliato. Una macchina, se lo script è scritto correttamente, esegue lo stesso identico procedimento ogni singola volta, senza stanchezza e senza distrazioni.
2. **Velocizza il lavoro.** Un compito che a mano richiede 40 minuti (con passaggi da ricordare, comandi da digitare, controlli da fare manualmente) può essere eseguito automaticamente in pochi minuti, o anche secondi, liberando le persone per lavoro che richiede davvero giudizio umano.
3. **Rende il lavoro ripetibile e verificabile.** Uno script di automazione è, esso stesso, codice: può essere letto, revisionato, versionato con Git (esattamente come hai visto nella sezione 4), e quindi **si sa esattamente cosa fa**, senza doversi fidare della memoria di una persona su “come si faceva quella cosa”.

 **Analogia:** pensa alla differenza tra lavare i piatti a mano, uno per uno, ogni giorno, e usare una lavastoviglie. La lavastoviglie non è “più intelligente” di una persona: esegue sempre la stessa sequenza di passaggi (acqua, detersivo, risciacquo, asciugatura), ma lo fa in modo costante, senza mai saltare il risciacquo per stanchezza dopo una lunga giornata, e libera le persone per fare altro nel frattempo. L'automazione nello sviluppo software è la lavastoviglie del team: non sostituisce il giudizio umano dove serve davvero (decidere cosa costruire, valutare un compromesso tra soluzioni), ma elimina la fatica ripetitiva dove non serve giudizio, solo costanza.

Esempi molto concreti di automazione che incontrerai nel team di ShopFacile, e che approfondiremo nelle prossime sezioni (specialmente nella 11, dedicata proprio a CI/CD):

- **Build automatiche:** ogni volta che **Ahmed** propone un cambiamento al codice del carrello, un sistema compila automaticamente il progetto, senza che nessuno debba farlo manualmente sul proprio computer.
- **Deploy automatici:** il pacchetto software viene installato automaticamente nell'ambiente giusto (test, staging, produzione), senza che una persona debba collegarsi manualmente a un server e copiare file a mano.

Altre forme altrettanto comuni — test automatici e provisioning automatico dell'infrastruttura — le vedrai nel dettaglio più avanti in questa stessa sezione e nella prossima, dedicata proprio a CI/CD.

Prima di andare avanti, però, vale la pena smontare un'illusione comune: **automatizzare non è a costo zero, fatto una volta e buono per sempre**. Uno script o una pipeline sono, essi stessi, software: vanno aggiornati quando cambia il codice che orchestrano. Un'automazione scritta due anni fa e mai più toccata è spesso peggio di nessuna automazione: dà l'illusione di un controllo che non verifica più nulla di rilevante — un falso senso di sicurezza. Per un futuro Project Manager, questo significa allocare esplicitamente anche il tempo di **mantenere** script e pipeline, non solo quello di scriverli la prima volta.

Tra tutte queste forme di automazione, una in particolare merita un paragrafo dedicato, perché è quella con cui uno sviluppatore di ShopFacile ha a che fare più spesso di tutte: l'integrazione automatica del codice di più persone che lavorano insieme sullo stesso progetto.

9.6 Continuous Integration (CI): integrare spesso, non alla fine

Continuous Integration, spesso abbreviata **CI**, è la pratica di **integrare il codice di più sviluppatori nel ramo principale del progetto molto frequentemente** (più volte al giorno, tipicamente), invece di lavorare separati per settimane e unire tutto solo alla fine.

Ricorda cosa hai visto nella sezione su Git e GitHub: quando più persone lavorano sullo stesso codice in branch diversi, unire i cambiamenti (il *merge*) può generare conflitti. Più tempo passa tra un'integrazione e la successiva, più il codice dei diversi sviluppatori si allontana l'uno dall'altro, e più i conflitti che emergono al momento dell'unione sono grandi, complicati da risolvere e rischiosi.

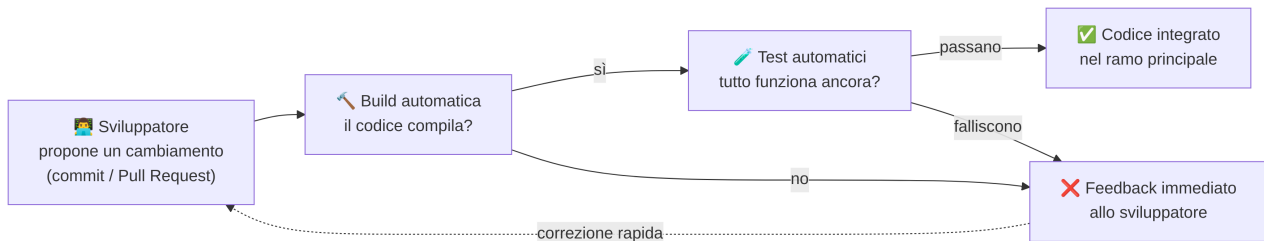
CI risolve questo problema imponendo una disciplina precisa: **ogni volta che uno sviluppatore propone un cambiamento (tipicamente con un commit o una Pull Request), un sistema automatico esegue immediatamente la build del progetto e i test automatici**, per verificare che quel cambiamento si integri correttamente con tutto il resto del codice, senza romperlo.



Analogia: pensa alla differenza tra scrivere un libro a più mani aggiornando ogni giorno un documento condiviso online (dove ogni piccola modifica di ciascun autore viene subito vista dagli altri, e le incoerenze si notano immediatamente), e scrivere ciascuno il proprio capitolo separatamente per sei mesi, per poi provare a incollare tutto insieme alla fine. Nel secondo caso,

è quasi garantito che i personaggi abbiano nomi diversi in capitoli diversi, che la timeline non torni, che il tono sia incoerente — e correggere tutto questo alla fine è molto più difficile che correggere piccole incoerenze giorno per giorno.

Il ciclo tipico di CI, ogni volta che qualcuno propone un cambiamento:



 **Esempio pratico:** Ahmed apre una Pull Request che modifica la funzione di calcolo dello sconto su un ordine di ShopFacile. Ecco cosa succede, passo per passo, in una pipeline di CI tipica (li vedrai nel dettaglio nella sezione 10):

1. Il codice modificato viene “caricato” (push) sul repository condiviso.
2. La pipeline si attiva automaticamente (il cosiddetto *trigger*), senza che nessuno debba lanciarla a mano.
3. Viene eseguita la **build**: il codice viene compilato/assemblato. Se c'è un errore di sintassi, la pipeline si ferma già qui.
4. Vengono eseguiti i **test automatici**: ad esempio, un test verifica che “uno sconto del 10% su un ordine da 100€ dia esattamente 90€”.
5. Se un test fallisce (magari Ahmed ha sbagliato un calcolo), la pipeline segnala l'errore in pochi minuti, direttamente nella Pull Request, prima che il codice venga integrato. Sarà **Giulia**, che revisiona spesso queste Pull Request, ad aiutarlo a individuare il problema.
6. Solo se build e test passano, il codice viene integrato nel ramo principale — pronto per le fasi successive (Delivery o Deployment).

Il beneficio più importante di CI è il **feedback rapido**: se qualcosa non funziona, lo sviluppatore lo scopre in pochi minuti, mentre ha ancora ben in mente cosa ha appena scritto — non settimane dopo, quando ha già dimenticato i dettagli di quella modifica e deve “ricordarsi” cosa aveva fatto.

Sapere che il codice è stato integrato correttamente, però, non dice ancora nulla su **quando e come** quella modifica arriverà davanti agli utenti di ShopFacile: è qui che entrano in gioco Delivery e Deployment, i due concetti che completiamo nel prossimo paragrafo.

9.7 Continuous Delivery e Continuous Deployment: la differenza che confonde tutti

Arriviamo a uno dei punti più importanti — e più spesso confusi da chi è alle prime armi — di tutto il vocabolario DevOps: la differenza tra **Continuous Delivery** e **Continuous Deployment**. Le due espressioni si abbreviano entrambe con **CD**, iniziano con le stesse lettere, e vengono spesso usate (erroneamente) come sinonimi. Non lo sono. Vale la pena fissare bene questa differenza, perché la incontrerai continuamente nel lavoro quotidiano e nella prossima sezione dedicata proprio a CI/CD.

Continuous Delivery: sempre pronto, ma il rilascio è una decisione manuale

Continuous Delivery significa che, grazie all'automazione (build, test, e tutto ciò che serve per preparare il pacchetto software), **il software è sempre in uno stato pronto per essere rilasciato in produzione in qualsiasi momento** — ma il rilascio effettivo in produzione richiede **un'azione manuale, di solito un'approvazione o un click da parte di una persona**.

Continuous Deployment: il rilascio è automatico, senza intervento umano

Continuous Deployment va un passo oltre: qui **anche il rilascio in produzione avviene automaticamente**, senza bisogno di un'approvazione manuale, non appena il codice ha superato con successo tutti i controlli automatici (build, test, eventuali controlli di sicurezza). Nessuna persona deve premere un bottone: se i controlli automatici passano, il codice va in produzione da solo.



Analogia molto concreta: pensa a una catena di montaggio che produce automaticamente dei pacchi pronti per la spedizione, testati e verificati.

- Con **Continuous Delivery**, ogni pacco pronto viene messo su uno scaffale, etichettato "pronto per la spedizione" — ma è sempre una persona che decide quando e quale pacco spedire effettivamente al cliente, magari aspettando il momento giusto, o un'ultima verifica manuale.
- Con **Continuous Deployment**, non c'è nessuno scaffale di attesa: non appena un pacco supera i controlli di qualità automatici, un corriere lo spedisce **automaticamente** al cliente, senza che nessuno debba decidere nulla in quel momento.

Un'altra analogia, forse ancora più diretta per il contesto software: pensa a un aereo pronto al decollo. Con Continuous Delivery, l'aereo è rifornito, controllato, con l'equipaggio a bordo e pronto a partire in qualsiasi momento — ma serve sempre il comando esplicito della torre di controllo ("via libera al decollo") prima che parta davvero. Con Continuous Deployment, non appena tutti i controlli pre-volo automatici sono superati, l'aereo decolla da solo, senza aspettare quel comando umano.

 **Esempio pratico:** immagina che in ShopFacile una nuova funzionalità (“aggiungi un filtro di ricerca per data nel catalogo”) abbia appena superato build e test automatici.

- Con **Continuous Delivery**, il pacchetto pronto resta “in attesa” e **Sara** (Product Owner), insieme a un responsabile del rilascio, apre la pipeline, guarda che tutto sia verde, e clicca manualmente su “Deploy in produzione” — magari aspettando il momento di minor traffico sul sito.
- Con **Continuous Deployment**, non appena i test automatici passano, la stessa funzionalità viene installata in produzione da sola, nel giro di pochi minuti dal commit originale, senza che nessuno clicchi nulla.

La differenza pratica che noterai osservando una pipeline reale (lo farai nella sezione 10): nel primo caso c'è uno step chiamato tipicamente “Approvazione” o “Gate” che aspetta un umano; nel secondo caso quello step semplicemente non esiste.

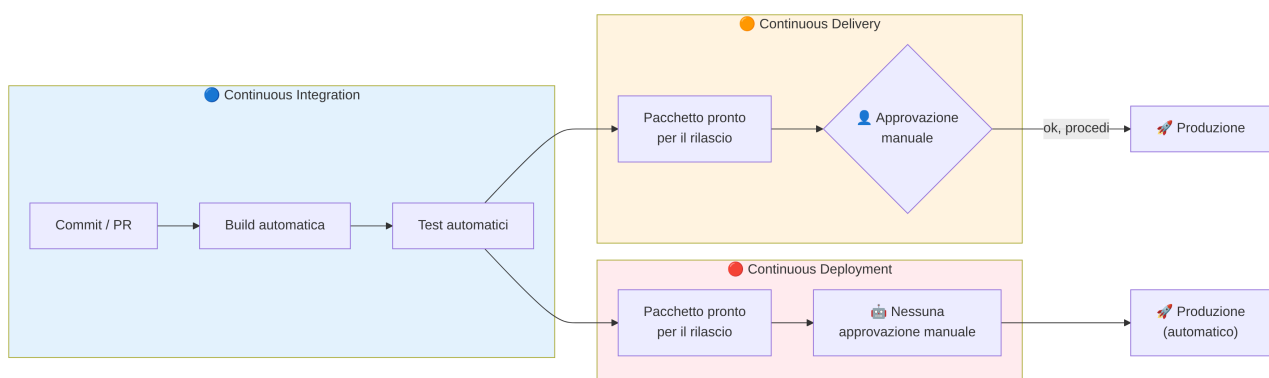
Perché questa distinzione conta davvero

La scelta tra Continuous Delivery e Continuous Deployment non è solo tecnica: è una **decisione di business e di rischio**. Alcuni contesti (sistemi bancari, sanitari, o semplicemente prodotti dove un errore in produzione ha un costo molto alto) preferiscono quasi sempre Continuous Delivery, per mantenere un punto di controllo umano finale prima di esporre gli utenti a un cambiamento. Altri contesti (ad esempio prodotti web con rilasci molto frequenti e un basso costo di un eventuale rollback rapido) possono permettersi Continuous Deployment, guadagnando velocità.

Nota terminologica importante: quando qualcuno usa genericamente l'espressione “**CI/CD**”, di solito si riferisce all'intero flusso automatizzato che va dall'integrazione del codice fino alla sua consegna o al suo rilascio — e la “CD” in questo contesto può voler dire Delivery *oppure* Deployment, a seconda del team e del progetto specifico. Per questo, quando la precisione conta (ad esempio in una discussione tecnica o in un documento di progetto), è sempre meglio specificare esplicitamente quale delle due si intende.

Uno sguardo d'insieme: CI vs Continuous Delivery vs Continuous Deployment

	Continuous Integration (CI)	Continuous Delivery	Continuous Deployment
Cosa succede	Il codice viene integrato, compilato e testato automaticamente ad ogni commit/PR	Il pacchetto software è sempre pronto per il rilascio, dopo build e test automatici	Il pacchetto software viene rilasciato automaticamente in produzione
Il rilascio in produzione è...	Non è ancora in discussione a questo livello	Manuale (richiede un'azione/approvazione umana)	Automatico (nessun intervento umano)
Dove avviene	Ambiente di build/test, prima della produzione	Fino "alla porta" della produzione	Direttamente in produzione
Livello di rischio percepito	Basso: non si tocca ancora la produzione	Medio: c'è un controllo umano finale	Più alto: richiede grande fiducia nei test automatici e nel processo
Frase riassuntiva	"Ci assicuriamo che il codice di tutti funzioni insieme"	"Il software è sempre pronto, ma decidiamo noi quando"	"Il software si rilascia da solo, se supera i controlli"



Nota come, nel diagramma, **CI è sempre presente** in entrambi gli scenari: è la base comune. La differenza tra Continuous Delivery e Continuous Deployment si gioca esclusivamente sull'ultimo passaggio, quello dell'ingresso in produzione: **presenza o assenza di un'approvazione umana**. Se ricordi solo una cosa di questa sezione, ricorda questa: CI riguarda l'integrazione del codice; Delivery e Deployment riguardano entrambi il rilascio, e si distinguono solo per **chi (o cosa) preme l'ultimo bottone**.

Sia con Delivery che con Deployment, però, il codice di ShopFacile deve comunque "atterrare" da qualche parte: su server, reti e database che qualcuno deve aver preparato in anticipo. Il prossimo paragrafo racconta come anche questa parte — l'infrastruttura, non solo il codice applicativo — sia diventata "automazione", esattamente come build e test.

9.8 Infrastructure as Code (IaC): l'infrastruttura scritta come codice

Fino a qui abbiamo parlato soprattutto di codice applicativo (quello che implementa le funzionalità). Ma un software, per funzionare, ha bisogno anche di **infrastruttura**: server su cui essere eseguito, reti che permettano le comunicazioni, database per salvare i dati, regole di sicurezza che decidano chi può accedere a cosa.

Immagina questa scena, capitata più di una volta in team come quello di ShopFacile: bisogna ricreare l'ambiente di test dopo un guasto, e qualcuno lo configura di nuovo **a mano**, a memoria. Ore dopo, un bug compare solo in quell'ambiente: la causa non è nel codice, ma in un parametro di configurazione dimenticato durante la ricostruzione manuale, diverso da quello di produzione. Questo è, storicamente, il problema dell'infrastruttura creata a mano: lenta, difficile da ripetere in modo identico, soggetta a errori umani (i problemi che l'automazione, vista nella sezione 9.5, cerca di risolvere).

Infrastructure as Code (IaC) applica esattamente la stessa logica del controllo di versione del codice (Git, che hai visto nella sezione 4) all'infrastruttura: invece di configurare server e reti manualmente, **si scrivono file di configurazione che descrivono esattamente l'infrastruttura desiderata**, e uno strumento automatico legge quei file e crea (o modifica) l'infrastruttura reale in modo coerente con quanto descritto.

 **Analogia:** pensa alla differenza tra costruire una casa “a memoria”, decidendo ogni misura sul momento parlando a voce con gli operai, e costruirla seguendo un progetto architettonico scritto, preciso, misurabile, che può essere consultato, corretto, e — soprattutto — **rieseguito identico** per costruire una seconda casa uguale alla prima in un altro terreno. Il progetto scritto (il “codice” dell'infrastruttura) permette a chiunque di capire esattamente cosa è stato costruito, di modificarlo in modo controllato, e di ricostruirlo da zero in caso di disastro, senza dover “ricordare a memoria” cosa era stato fatto.

 **Esempio pratico:** ecco un estratto semplificato (sintassi approssimativa, solo per farti capire l'idea, non un file da eseguire davvero) di come potrebbe apparire un file IaC che descrive il server web e il database di ShopFacile:

```

resource "server_web" {
  nome          = "server-shopfacile-produzione"
  dimensione    = "media" # 2 CPU, 4 GB RAM
  sistema       = "Linux"
  porta_aperta  = 443      # HTTPS
}

resource "database" {
  nome          = "db-ordini-shopfacile-produzione"
  tipo          = "PostgreSQL"
  backup_auto   = true
  accesso       = "solo da server_web"
}

```

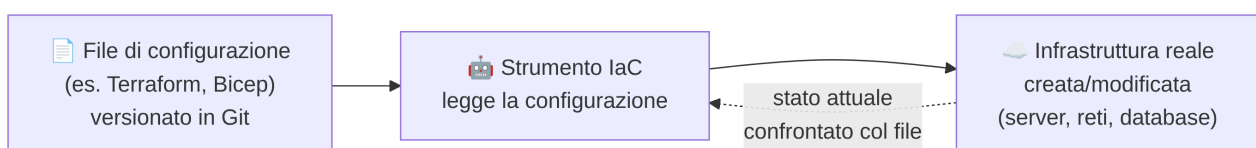
Questo file descrive **cosa deve esistere**, non i singoli comandi per crearlo. Uno strumento IaC (es. Terraform) legge questo file e si occupa lui di creare davvero il server e il database nel cloud, con quelle caratteristiche esatte. Se domani serve un secondo ambiente identico per il collaudo, basta eseguire lo stesso file puntandolo a un ambiente diverso: niente configurazione manuale ripetuta a mano, niente rischio di “dimenticare un dettaglio” rispetto all’ambiente di produzione.

I vantaggi concreti di questo approccio:

- **Ripetibilità:** lo stesso file di configurazione produce sempre la stessa infrastruttura, sia che lo esegua un ingegnere esperto sia che lo esegua un collega alle prime armi.
- **Versionamento:** i file IaC si salvano in Git, come il codice applicativo — si può vedere la storia di ogni modifica, chi l’ha fatta, perché, e tornare indietro se necessario.
- **Revisione:** una modifica all’infrastruttura può passare per una Pull Request e una code review, esattamente come una modifica al codice — invece di essere una modifica manuale invisibile fatta da una persona su un server, di cui nessun altro sa nulla.
- **Velocità e scalabilità:** creare dieci ambienti di test identici, o ricreare rapidamente un intero ambiente dopo un disastro, diventa una questione di minuti, eseguendo lo stesso file, invece di settimane di lavoro manuale ripetitivo.

Alcuni strumenti diffusi di IaC che potresti incontrare (li vedrai citati anche nella sezione 12 sul Cloud):

- **Terraform:** uno degli strumenti IaC più diffusi in assoluto, utilizzabile con diversi fornitori cloud (AWS, Azure, Google Cloud e altri) con un linguaggio di configurazione comune.
- **ARM Template / Bicep:** gli strumenti IaC “nativi” di Microsoft Azure, con Bicep pensato come evoluzione più semplice da scrivere e leggere rispetto agli ARM Template originali (basati su JSON).



Anche l'laC, però, non è un vantaggio senza contropartita. Scrivere e leggere questi file richiede una curva di apprendimento non banale. Inoltre, se qualcuno modifica l'infrastruttura reale a mano “per fare presto” senza aggiornare il file, i due **divergono**: il file descrive uno stato che non corrisponde più a quello reale, e la divergenza è difficile da scoprire finché non causa un problema. Infine, il file che può creare infrastruttura può anche **distruggerla**: una modifica sbagliata e non rivista può eliminare server o database reali in pochi secondi — un motivo in più per passare sempre da Pull Request e code review.

Una volta che server e database di ShopFacile sono stati creati — a mano o, meglio, tramite laC — il lavoro non finisce lì: bisogna sapere, giorno per giorno, se quell'infrastruttura e il software che ci gira sopra stanno funzionando bene. È il compito del monitoring, che vediamo subito.

9.9 Monitoring: sapere se le cose stanno andando bene

Una volta che il software è in produzione (fase “Operate” del ciclo DevOps), non basta “sperare che vada tutto bene”: bisogna **osservarlo attivamente**. Questo è il compito del **monitoring** (monitoraggio).

Il monitoring è l'attività di **raccogliere e osservare dati sullo stato di un sistema in tempo reale**, per rispondere a una domanda semplice ma fondamentale: “**è tutto ok, o c'è un problema?**”.

Concretamente, significa tenere sotto controllo indicatori come: il sito web risponde? Quanto tempo impiega a rispondere? Il server ha ancora memoria disponibile? Ci sono errori nei log?



Analogia: il monitoring è come il cruscotto di un'automobile mentre guidi: la spia della temperatura del motore, l'indicatore di velocità, la spia della pressione delle gomme. Ti dicono, in modo semplice e immediato, **se qualcosa è fuori dai parametri normali** — ma non ti spiegano *perché* il motore si sta scaldando troppo. Per quello serve altro (lo vedremo con l'observability, nella prossima sezione).

Strumenti tipici di monitoring includono **dashboard** con grafici in tempo reale, e **alert** (avvisi automatici, spesso via email, SMS o strumenti come Slack/Teams) che si attivano quando un indicatore supera una soglia critica — ad esempio, “il tempo di risposta del sito ha superato i 3 secondi” o “meno del 5% di spazio disco disponibile”. In questo modo, il team di ShopFacile viene avvisato **prima** che un problema diventi visibile agli utenti finali, o quasi contemporaneamente al primo utente che lo nota — non ore o giorni dopo.

Il monitoring, però, ha un limite che vale la pena anticipare subito: ti dice *che* qualcosa non va (il sito è lento), ma non sempre ti dice *perché*. Per quel “perché” serve un concetto più ampio, che vediamo nel prossimo paragrafo: l'observability.

9.10 Observability: capire non solo il “cosa”, ma il “perché”

Observability (osservabilità) è un concetto più ampio del monitoring, ed è importante distinguerlo bene, perché nel lavoro quotidiano i due termini vengono spesso confusi.

Il monitoring, come abbiamo visto, risponde principalmente alla domanda **“cosa sta succedendo”**: il sistema è su o giù, la CPU è alta o bassa, il tempo di risposta è nella norma o no. È molto utile, ma ha un limite: di solito richiede di aver **già previsto in anticipo** cosa monitorare (hai bisogno di sapere già quale spia mettere sul cruscotto).

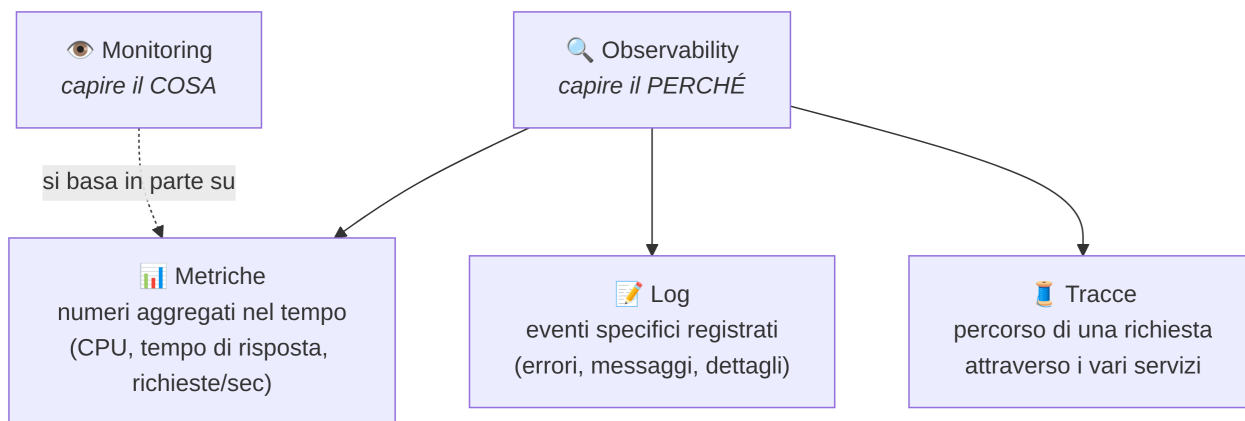
L'**observability** va oltre: è la **capacità di capire perché un sistema si comporta in un certo modo**, anche di fronte a un problema che non era stato previsto in anticipo, senza dover aggiungere nuovo codice o nuovi strumenti di misurazione *dopo* che il problema si è già verificato. Un sistema è “osservabile” quando, guardando i dati che già produce, un ingegnere è in grado di **ricostruire e diagnosticare** un comportamento anomalo, anche complesso o mai visto prima.

 **Analogia:** se il monitoring è il cruscotto dell'auto (spie predefinite su parametri già noti), l'observability è come avere a disposizione **tutti i dati del motore, del sistema elettrico e dei sensori**, più uno strumento diagnostico da meccanico, che ti permette — anche di fronte a un problema mai visto prima, per cui non esisteva una spia dedicata — di indagare a fondo e capire esattamente cosa sta succedendo e perché, ricostruendo la catena di eventi. Non ti basta sapere che “il motore scalda troppo” (questo te lo dice già la spia): ti serve poter scoprire *perché* scalda, magari per la prima volta in assoluto, senza dover smontare il motore alla cieca.

I tre pilastri dell'observability

L'observability si costruisce tipicamente su tre tipi di dati complementari, spesso chiamati i suoi “tre pilastri”:

1. **Metriche (metrics):** numeri aggregati nel tempo, come il numero di richieste al secondo, il tempo medio di risposta, la percentuale di CPU utilizzata. Sono leggere da raccogliere e ottime per grafici e alert (il “cosa” del monitoring).
2. **Log:** registrazioni testuali di eventi specifici avvenuti nel sistema (li vedremo nel dettaglio nella prossima sezione). Sono più dettagliati delle metriche: raccontano *cosa è successo esattamente*, in un dato momento, spesso con informazioni di contesto (un errore, un messaggio, un identificativo di richiesta).
3. **Tracce (traces):** seguono il percorso completo di una singola richiesta di un utente **attraverso i diversi componenti di un sistema** (ad esempio: il sito web, poi un servizio di autenticazione, poi un database, poi un servizio di pagamento). Sono particolarmente utili nei sistemi complessi, composti da tanti servizi diversi che collaborano tra loro, per capire **in quale punto esatto della catena** si è verificato un ritardo o un errore.



Un modo semplice per ricordare la relazione tra i due concetti: **il monitoring è un'attività (osservare dashboard e alert); l'observability è una proprietà del sistema** (la sua capacità intrinseca di essere comprensibile e diagnosticabile quando serve). Puoi fare ottimo monitoring di un sistema poco osservabile (hai le spie giuste, ma non riesci a capire il “perché” quando succede qualcosa di nuovo e inatteso); un sistema davvero osservabile, invece, ti dà gli strumenti per indagare in profondità anche su problemi che nessuno aveva previsto in anticipo.

Tra i tre pilastri dell'observability appena visti — metriche, log e tracce — uno in particolare è così radicato nel lavoro quotidiano di un team come quello di ShopFacile da meritare un paragrafo a sé: i log. Vediamoli più da vicino.

9.11 Logging: la memoria degli eventi del sistema

Abbiamo già citato i log come uno dei tre pilastri dell'observability; vale la pena approfondirli separatamente, perché il **logging** (la pratica di produrre e gestire log) è probabilmente lo strumento di diagnosi più antico, più diffuso e più immediato nel lavoro quotidiano di un team tecnico.

Un **log** è, molto semplicemente, **una registrazione testuale di un evento accaduto in un sistema**, con solitamente un timestamp (quando è successo), un livello di gravità (informazione, avviso, errore, errore critico) e un messaggio che descrive cosa è successo.

💡 **Analogia:** pensa al log come al **diario di bordo di una nave**. Il capitano non scrive solo “oggi tutto bene”: annota eventi specifici, con data e ora precisa — “14:32, avvistata tempesta a nord-ovest”, “18:05, motore sinistro rallentato per manutenzione”, “22:10, rotta corretta di 5 gradi”. Se qualcosa va storto durante il viaggio, il diario di bordo è il primo posto dove si va a cercare *cosa è successo, esattamente, e quando* — non a memoria, ma con un record scritto e preciso di ogni evento rilevante.

Esempio semplificato di alcune righe di log di un'applicazione web, per darti un'idea concreta del formato:

```
2026-07-25 09:14:02 [INFO] Utente 4821 ha effettuato il login
2026-07-25 09:14:05 [INFO] Richiesta GET /catalogo completata in 120ms
2026-07-25 09:15:33 [WARN] Tempo di risposta al database superiore a 500ms
2026-07-25 09:16:01 [ERROR] Pagamento fallito per ordine #98213: timeout verso il
servizio esterno
2026-07-25 09:16:02 [ERROR] Impossibile inviare email di conferma: servizio SMTP non
risponde
```

Da questo semplice estratto, un ingegnere può già iniziare a ricostruire una storia: qualcosa ha iniziato a rallentare (il WARN sul database), poco dopo un pagamento è fallito per timeout verso un servizio esterno, e subito dopo anche l'invio email ha iniziato a fallire — indizi che potrebbero indicare un problema di rete o di un servizio esterno condiviso, non necessariamente un bug nel codice dell'applicazione stessa.

 **Esempio pratico: monitoring, observability e logging insieme in un incidente reale su ShopFacile.** Per capire davvero come questi tre concetti lavorano insieme (e non sono tre cose scollegate), segui questo scenario passo per passo:

1. **Monitoring** (il “cosa”): alle 09:16 un **alert** automatico avvisa il team su Slack/Teams: “tempo di risposta medio del sito superiore a 3 secondi”. La dashboard mostra una linea che sale bruscamente. Il team sa *che* qualcosa non va, ma non ancora *perché*.
2. **Observability** (il “perché”): **Marco** guarda le **tracce** delle richieste più lente e nota che il ritardo si concentra sempre nello stesso punto della catena: la chiamata dal servizio ordini verso un servizio esterno di pagamento. Le **metriche** confermano che è proprio quel servizio esterno ad avere un tempo di risposta anomalo, non il database o il resto dell'applicazione.
3. **Logging** (il dettaglio esatto): Marco apre i **log** dello stesso intervallo di tempo — proprio quelli visti sopra — e trova la riga **[ERROR] Pagamento fallito per ordine #98213: timeout verso il servizio esterno**, che conferma con precisione l'orario, l'ordine coinvolto e la causa tecnica esatta (un timeout).

Risultato: in pochi minuti il team capisce non solo che c'era un problema (monitoring), ma anche dove si trovava nel sistema (observability) e qual era il dettaglio preciso da riportare a chi gestisce quel servizio esterno (logging) — senza dover indovinare nulla.

Perché il logging è così importante nel contesto DevOps:

- **Diagnosi post-incidente:** quando qualcosa va storto (collegandoci direttamente alla blameless culture vista nella sezione 9.4), i log sono spesso la prima fonte di informazione per ricostruire la sequenza esatta degli eventi, senza dover indovinare o fidarsi solo della memoria delle persone.
- **Tracciabilità:** sapere non solo *che* qualcosa è successo, ma *quando* esattamente, e in quale ordine rispetto ad altri eventi.
- **Base per l'automazione:** molti sistemi di alert (visti nel monitoring) si basano proprio sull'analisi automatica dei log — ad esempio, un alert che si attiva se il numero di righe con livello **ERROR** supera una certa soglia in pochi minuti.

Un aspetto pratico da conoscere: nei sistemi moderni, composti da tanti servizi diversi (specialmente nei sistemi che vedrai nella sezione 11 sulle architetture software), i log di ciascun servizio vengono spesso raccolti in un **unico sistema centralizzato**, per poterli cercare e correlare tutti insieme, invece di dover collegarsi manualmente a decine di server diversi per leggere ciascuno il proprio file di log locale.

Monitoring, observability e logging ti dicono, con sempre maggiore dettaglio, *che cosa* sta succedendo e *perché*. Ma sapere che qualcosa non va non è la stessa cosa che sapere **chi deve alzarsi dalla scrivania**, chi ha l'autorità di bloccare un rilascio, o chi deve chiamare il servizio clienti. È il tema del prossimo paragrafo.

9.12 Service Management: cosa succede quando il software è in produzione

Martedì, 9:40. Sul sito di ShopFacile i clienti non riescono a completare il pagamento: il checkout si blocca all'ultimo passaggio. L>alert del monitoring (visto al paragrafo 9.9) è scattato puntuale alle 9:16, come doveva. Il problema è tutto quello che succede *dopo* quell>alert. **Marco** vede l>alert e inizia a indagare sul servizio ordini. In parallelo, senza saperlo, anche **Ahmed** ha visto lo stesso alert e ha iniziato a indagare sul database — nessuno dei due sa che l'altro è già al lavoro sullo stesso incidente. **Giulia**, che nel frattempo stava guardando le tracce viste al paragrafo 9.10, ha già capito che il problema è un timeout verso il servizio di pagamento esterno: lo scrive in una chat di canale tecnico che Marco e Ahmed non leggono, perché lavorano in un altro canale. Il servizio clienti, che riceve le prime chiamate dei clienti già alle 9:20, non è stato avvisato da nessuno: continua a rispondere “controlliamo e la ricontattiamo a breve” senza sapere che il team lo sa già da quattro minuti. Alle 10:05, quando finalmente qualcuno mette insieme i tre pezzi, il problema è identico a quello che Giulia aveva già individuato un'ora prima.

Nessuno, in questa scena, è incompetente. Marco, Ahmed e Giulia hanno fatto esattamente il lavoro tecnico giusto. Quello che è mancato non è competenza: è un **processo** che dicesse, prima che il problema succedesse, chi coordina, dove si comunica, e chi parla con il cliente. È esattamente da questa mancanza che nasce l'**ITSM**.

ITSM: da “consegniamo progetti” a “gestiamo servizi”

ITSM (IT Service Management) è l'insieme di processi con cui un'organizzazione IT gestisce i servizi che fornisce, in modo che funzionino in modo prevedibile **tutti i giorni**, non solo il giorno del rilascio. È un cambio di prospettiva rispetto a tutto quello che hai visto nelle sezioni su Agile e Project Management (sezioni 5-8): lì il focus era consegnare un progetto, con un inizio e una fine; qui il focus è **gestire un servizio che deve restare in piedi indefinitamente**, molto dopo che il progetto che l'ha creato è stato chiuso.

Il framework di riferimento più diffuso per l'ITSM è **ITIL** (attualmente alla versione **ITIL 4**): una raccolta di buone pratiche, non uno standard obbligatorio, che moltissime organizzazioni enterprise — comprese le compagnie assicurative — usano come vocabolario comune per organizzare questi processi. Non serve conoscerlo in dettaglio: serve riconoscere i processi che contiene, perché li sentirai nominare quasi ogni settimana.

Incident management: ripristinare, non capire

L'**incident management** si occupa di una cosa sola: **ripristinare il servizio il più presto possibile**. Non gli interessa, in questa fase, *perché* il checkout si è rotto: gli interessa farlo funzionare di nuovo. Per questo un incidente viene classificato per **severità/priorità** (un checkout bloccato per tutti i clienti non è la stessa cosa di un pulsante mal allineato per un browser di nicchia) e, se chi lo gestisce non riesce a risolverlo in tempi accettabili, si attiva l'**escalation**: il problema sale a qualcuno con più competenza, più autorità, o entrambe. Nello scenario di ShopFacile sopra, un incident management funzionante avrebbe fatto esattamente due cose che sono mancate: un solo punto di coordinamento (non tre indagini parallele) e un avviso tempestivo al servizio clienti.

Problem management: capire la causa, dopo

Il **problem management** è un processo diverso, e separato apposta: si occupa di trovare e rimuovere la **causa radice**, in modo che quell'incidente non si ripeta. È deliberatamente disaccoppiato dall'incident management per un motivo molto concreto: se li confondi, hai due esiti possibili, entrambi cattivi. O tieni il servizio bloccato mentre indagli a fondo sulla causa (il cliente aspetta mentre tu capisci), oppure ripristini in fretta senza mai indagare, e lo stesso timeout verso il servizio di pagamento tornerà a bloccare il checkout la settimana successiva. Nello scenario sopra: ripristinare il checkout è incident management; capire perché il servizio di pagamento esterno vada in timeout sotto un certo carico, e magari introdurre un timeout più aggressivo o un circuito di failover, è problem management — un lavoro che può, e spesso deve, continuare anche dopo che il servizio è di nuovo su.

Change management e change request: chi autorizza una modifica

Il **change management** decide **chi autorizza** una modifica in produzione e con che criterio, tipicamente attraverso una **change request**: una richiesta formale che descrive cosa si vuole cambiare, quale rischio comporta, e chi deve approvarla prima che venga eseguita.

Qui si incontra il collegamento più interessante di questa sezione. Il change management “classico” nasce per ridurre il rischio di un cambiamento **rallentandolo**: più persone lo guardano, più tempo passa prima che venga approvato, minore la probabilità che qualcosa vada storto senza che nessuno se ne accorga in anticipo. Il **CI/CD** che vedrai nella sezione 10 riduce lo stesso rischio nel modo esattamente opposto: **automatizzando e rimpicciolendo** i cambiamenti, così che ciascuno sia troppo piccolo e troppo testato per fare danni seri. Non sono nemici, ma **convivono male se nessuno lo dice esplicitamente**: un team che rilascia dieci volte al giorno e deve comunque aspettare una riunione settimanale di change advisory board per ogni singolo rilascio non sta facendo CI/CD, sta solo rallentando entrambi i processi. Il punto d'incontro più comune è il **change standard** (o pre-autorizzato): una categoria di cambiamenti a basso rischio, già valutata una volta per tutte, che non richiede una nuova approvazione umana ogni volta — esattamente il tipo di cambiamento che una pipeline CI/CD ben testata produce in continuazione.

💡 Il dato che sorprende chi viene da un mondo change-management tradizionale: i report **DORA** (DevOps Research and Assessment) mostrano da anni che i team che rilasciano **più spesso** tendono ad avere **meno** incidenti in produzione, non di più. Rilasci piccoli e frequenti sono più facili da testare, da capire e — se qualcosa va storto — da annullare, rispetto a un grande rilascio trimestrale.

Service Request e Service Desk

Non tutto quello che arriva al team è un guasto. Una **service request** è una richiesta di servizio — “ho bisogno di un accesso al database di staging”, “vorrei una nuova licenza software” — che non indica che qualcosa si sia rotto, ma che qualcuno ha bisogno di qualcosa. Trattarla come un incidente (con la stessa urgenza, la stessa escalation) confonde le priorità e intasa i canali pensati per le vere emergenze. Il **Service Desk** è il punto di contatto unico per entrambe le cose — incidenti e richieste di servizio — così che chi ha un problema non debba sapere in anticipo a quale team specifico rivolgersi.

SLA e SLO

Gli **SLA (Service Level Agreement)**, che ritroverai nella sezione 13 come accordi contrattuali sul livello di servizio garantito, e il concetto di **error budget**, già introdotto al paragrafo 9.3 a proposito di SRE, sono gli strumenti con cui questi processi vengono misurati: non “abbiamo risposto in fretta”, ma “abbiamo rispettato il tempo di ripristino concordato in contratto”.

I processi in sintesi

Processo	Cosa lo attiva	Chi è coinvolto	“Successo” significa
Incident management	Il servizio è degradato o fermo	Chi è di turno/on-call, escalation se serve	Servizio ripristinato, il più in fretta possibile
Problem management	Un incidente (spesso ricorrente) da capire a fondo	Chi ha competenza tecnica sul componente coinvolto	Causa radice trovata e rimossa, l'incidente non si ripete
Change management	Una modifica pianificata in produzione	Chi propone il cambio, chi approva (change advisory)	Il cambiamento è avvenuto senza sorprese, o è stato bloccato prima che causasse danni
Service Request	Una richiesta legittima, non un guasto	Service Desk, team che eroga la risorsa richiesta	Richiesta soddisfatta nei tempi previsti

Il trade-off: un processo aggirato è peggio di nessun processo

Anche qui, come ovunque in questa sezione, non c'è una scelta senza costo. Un processo di change management pensato per proteggere la produzione, se pesa troppo — un modulo da otto giorni per autorizzare una modifica di due righe — spinge le persone a **aggirarlo**: la modifica “veloce” fatta di

nascosto, senza change request, perché “tanto è una cosa piccola e passare dal processo ufficiale richiede troppo”. Questo è più pericoloso di non avere affatto un processo di change management, perché **dà l'illusione del controllo**: chi guarda i numeri crede che ogni modifica sia tracciata e autorizzata, mentre in realtà un canale informale e invisibile sta bypassando ogni controllo. Un processo troppo leggero fallisce in modo visibile; un processo troppo pesante fallisce in silenzio, ed è per questo più difficile da correggere in tempo.

Chiudiamo con una nota sui ruoli, perché la troverai utile quanto quella sulle figure DevOps vista al paragrafo 9.1: in molte organizzazioni enterprise esiste una funzione dedicata — spesso chiamata proprio “ITSM” o “Service Management” — con persone che gestiscono incident, change e service desk come lavoro a tempo pieno, distinto da chi scrive il codice. È una delle figure con cui, da Project Manager, dialogherai più spesso: ogni volta che un tuo progetto deve rilasciare qualcosa in produzione, è a questa funzione — non solo al tuo team di sviluppo — che dovrai rendere conto.

9.13 Farm batch e processi schedulati: il lavoro che gira di notte

Non tutto il software di un'assicurazione risponde a un clic di un utente. Una quantità enorme di lavoro gira **di notte, quando nessuno guarda**: il calcolo dei premi su migliaia di polizze, la produzione dei documenti contrattuali da spedire, le riconciliazioni contabili tra i sistemi, gli invii dei dati verso enti esterni (l'IVASS, il fisco, un partner riassicurativo) che hanno una scadenza fissa e non aspettano nessuno.

Questo lavoro si chiama **elaborazione batch**: elaborare grandi volumi di dati **a lotti**, tutti insieme, invece che uno alla volta in risposta a una richiesta. La **schedulazione** è il meccanismo che decide *quando* questo lavoro parte — a un orario fisso (le 2:00 di notte) o dopo un evento (appena il file del giorno precedente è arrivato) — senza che una persona debba avviarlo a mano. La **farm batch** è l'insieme delle macchine dedicate a farli girare: spesso separate dai server che servono il sito o l'app, proprio perché lavorano su orari e con carichi diversi.

Il dettaglio che rende questo mondo interessante per un PM è l'effetto domino delle **dipendenze tra job**: il calcolo dei premi deve girare prima della produzione dei documenti, che deve girare prima degli invii esterni. Se il primo job slitta di due ore — un file di input arrivato tardi, un guasto sulla farm — ogni job successivo slitta a cascata, e la **finestra notturna** (le poche ore in cui questi job possono girare senza intralciare gli utenti del giorno dopo) può non bastare più.



Perché interessa a un PM: la finestra batch è un vincolo di progetto tanto reale quanto una dipendenza in un Gantt. Un rilascio che allunga anche di pochi minuti il tempo di esecuzione di un job batch critico può far mancare una scadenza normativa — l'invio dei dati all'IVASS entro una certa ora, per esempio — con conseguenze molto più concrete di un ritardo interno. Questo mondo si tocca da vicino con il **disaster recovery** (se la farm batch è ferma per un guasto, chi recupera il lavoro non fatto, e in quanto tempo?) e con gli **SLA**: un “il servizio deve essere disponibile entro le 6:00” nasconde quasi sempre un batch notturno che deve finire prima di quell'ora.

9.14 Riepilogo

Questa sezione ha introdotto DevOps non come uno strumento, ma come una **cultura e un insieme di pratiche** che nascono per risolvere un problema storico molto concreto: la separazione, spesso conflittuale, tra chi sviluppa il software e chi lo gestisce in produzione.

- **DevOps** è l'unione (culturale, prima ancora che tecnica) di Development e Operations, per superare “il muro della confusione” tra i due mondi.
- Il **ciclo DevOps** (Plan-Code-Build-Test-Release-Deploy-Operate-Monitor) è rappresentato come un infinito, perché è un flusso continuo, non un progetto con inizio e fine.
- La **cultura DevOps** si basa su collaborazione, responsabilità condivisa (“you build it, you run it”) e **blameless culture**; il modello **CALMS** (Culture, Automation, Lean, Measurement, Sharing) riassume i suoi pilastri fondamentali, con la cultura sempre al primo posto.
- L'**automazione** riduce errori umani e velocizza il lavoro ripetitivo: build, test e deploy automatici sono gli esempi più concreti.
- **Continuous Integration (CI)** integra il codice frequentemente, con build e test automatici ad ogni commit/PR.
- **Continuous Delivery** rende il software sempre pronto al rilascio, ma richiede un'approvazione umana finale; **Continuous Deployment** rilascia in produzione in modo completamente automatico — la differenza sta esattamente in quell'ultimo passaggio.
- **Infrastructure as Code** applica la logica del controllo di versione (Git) all'infrastruttura, rendendola ripetibile, versionata e revisionabile.
- **Monitoring** risponde al “cosa” (è tutto ok?); **observability** va oltre e permette di capire il “perché”, basandosi su metriche, log e tracce; il **logging** è la registrazione dettagliata degli eventi, fondamentale per la diagnosi e la tracciabilità.
- L'**ITSM** (framework di riferimento: **ITIL**) governa cosa succede *dopo* che il software è in produzione: **incident management** (ripristinare in fretta), **problem management** (capire la causa, separatamente), **change management** e **change request** (chi autorizza una modifica, e come convive con il CI/CD), **service request** e **Service Desk** (una richiesta non è un guasto).
- Il mondo **batch**: elaborazioni notturne a grandi volumi, con **schedulazione**, **farm batch** dedicate e **dipendenze tra job** che possono far slittare a cascata un'intera finestra notturna.

Nella prossima sezione vedrai come tutti questi concetti — in particolare la pipeline CI/CD — si traducano in un meccanismo concreto e verificabile, fatto di fasi precise, trigger e controlli automatici, che il team usa ogni giorno per metterli in pratica.



Esercizi pratici

Questa è una delle sezioni più dense del corso: la teoria da sola non basta a fissarla davvero. Prova a completare questi esercizi con calma, magari uno al giorno, prendendoti il tempo per collegarli a esempi reali del progetto ogni volta che puoi.

1. **Racconta il “muro della confusione” con parole tue.** Scrivi (o registra a voce, se preferisci) una spiegazione di 5-6 righe di cosa fosse il “muro della confusione” tra Dev e Ops, come se dovessi spiegarlo a un amico che non ha mai lavorato in ambito tecnico, usando un esempio

diverso da quello del ristorante già visto in questa sezione (puoi ispirarti a qualsiasi altro contesto: un ufficio postale, una catena di montaggio, uno studio medico).  **Come verificare:** fatti ascoltare/leggere la spiegazione da un collega non tecnico (o da un familiare): se la capisce senza fare domande di chiarimento, hai centrato il punto.

2. **Applica CALMS al progetto reale.** Per ciascuna delle 5 lettere di CALMS (Culture, Automation, Lean, Measurement, Sharing), trova un esempio concreto nel progetto su cui lavori — anche piccolo — che dimostri quel pilastro in azione. Se per una lettera non trovi nessun esempio, annotalo: potrebbe essere un'area di miglioramento da discutere con la tua collega.  **Come verificare:** hai scritto 5 esempi (uno per lettera), ciascuno con un fatto specifico del progetto, non una definizione generica presa dal libro.
3. **Ricostruisci una pipeline CI su carta.** Prendi carta e penna e disegna, passo per passo (come nel diagramma della sezione 9.6), cosa succede da quando uno sviluppatore propone un cambiamento a quando il codice viene integrato nel ramo principale. Poi aggiungi, con un colore diverso, cosa cambierebbe se il team usasse Continuous Delivery invece che Continuous Deployment nello step finale.  **Come verificare:** nel tuo schema devi avere almeno 5 step distinti (proposta del cambiamento, build, test, integrazione, e lo step finale di rilascio) e riuscire a indicare con precisione dove si trova “l'ultimo bottone” premuto da un umano nel caso di Continuous Delivery.
4. **Trova (o fatti mostrare) un file di Infrastructure as Code reale.** Chiedi a un collega developer o operations se il progetto usa IaC (ad esempio Terraform, Bicep o ARM Template) e fatti mostrare un file reale, anche solo per pochi minuti sullo schermo. Non serve capire ogni riga: cerca di individuare almeno una “risorsa” descritta nel file (un server, una rete, un database) e confrontala con l'esempio semplificato visto nella sezione 9.8.  **Come verificare:** sai indicare a voce, guardando il file reale, almeno un pezzo di infrastruttura che descrive e a cosa serve nel progetto — anche con parole semplici e imprecise dal punto di vista tecnico.
5. **Scrivi tre righe di log realistiche.** Immagina una funzionalità qualsiasi del progetto (ad esempio “un utente carica un documento”) e scrivi tre righe di log plausibili che quella funzionalità potrebbe generare: una di livello **INFO** per il caso normale, una di livello **WARN** per un caso limite (es. file grande, connessione lenta), e una di livello **ERROR** per un caso di fallimento. Segui il formato visto nella sezione 9.11 (timestamp, livello, messaggio).  **Come verificare:** le tue tre righe hanno timestamp, livello di gravità corretto rispetto alla situazione descritta, e un messaggio che un collega — leggendolo senza altro contesto — capirebbe cosa è successo.
6. **Distingui monitoring da observability su un caso reale.** Chiedi a un collega di raccontarti un incidente reale (anche piccolo) capitato sul progetto. Per quell'episodio, identifica separatamente: cosa ha fatto scattare l'allarme iniziale (monitoring — la spia che si è accesa) e cosa ha permesso di capire la causa reale (observability — metriche, log o tracce usate per indagare). Se il collega non ricorda una distinzione così netta, va benissimo: è normale, e ti dà comunque materiale utile.  **Come verificare:** riesci a scrivere due frasi separate — una che inizia con “il team si è accorto del problema perché...” (monitoring) e una che inizia con “il team ha capito la causa guardando...” (observability) — riferite allo stesso episodio reale.

7. **Simula un mini post-mortem blameless.** Pensa a un piccolo errore che hai fatto tu stesso di recente, anche non legato al lavoro (es. hai dimenticato un appuntamento, hai fatto un acquisto sbagliato online). Scrivi un breve “post-mortem” in stile blameless culture: cosa è successo (fatti, senza giudizi), quale causa reale ha portato all'errore (una procedura poco chiara? un'informazione mancante?), e un'azione concreta per evitare che si ripeta. Evita ogni frase del tipo “ho sbagliato perché sono stato distratto”: cerca la causa nel “sistema” (promemoria assenti, processo poco chiaro), non solo nella persona.  **Come verificare:** il tuo testo non contiene nessuna frase colpevolizzante verso te stesso o altri, e termina con un'azione concreta e verificabile (non un generico “starò più attento”).
8. **Incident o problem?** Racconta a parole tue un guasto qualsiasi (reale o inventato, es. “il sito è lento ogni lunedì mattina”). Scrivi separate l'azione di **incident management** (cosa faresti per far tornare il servizio normale subito) e l'azione di **problem management** (cosa indagheresti per capire perché succede, e magari evitare che si ripeta).  **Come verificare:** le due azioni che hai scritto sono diverse tra loro, e la prima non richiede di aver già capito la causa per essere eseguita.
9. **Change request o via libera al CI/CD?** Immagina due modifiche diverse su ShopFacile: (a) cambiare il colore di un pulsante nel catalogo, (b) modificare la logica di calcolo di uno sconto che tocca tutti gli ordini. Per ciascuna, decidi se ti aspetteresti che passi per una change request formale con approvazione umana, o che sia un “change standard” già pre-autorizzato che una pipeline CI/CD gestisce da sola, motivando la scelta con il rischio del cambiamento, non con la sua dimensione in righe di codice.  **Come verificare:** la tua motivazione parla di rischio e impatto sugli utenti, non solo di “quanto codice cambia”.
10. **Trova la finestra batch del tuo progetto reale.** Chiedi a un collega developer o operations se il progetto su cui lavori ha elaborazioni notturne o schedate (calcoli, invii, riconciliazioni) e, se sì, quanto dura la finestra disponibile e cosa succede se un job slitta.  **Come verificare:** sai rispondere in una frase alla domanda “cosa succederebbe a un rilascio del venerdì se allungasse il batch della notte successiva di 30 minuti?” — anche solo con un'ipotesi plausibile.

Collegamenti

- [10. CI/CD](#) — approfondimento tecnico su pipeline, Continuous Integration, Delivery e Deployment, e su come il change management convive con i rilasci frequenti
- [12. Cloud](#) — dove gira davvero l'infrastruttura gestita con Infrastructure as Code, e dove trovi Platform Engineering e Internal Developer Platform
- [13. Sicurezza](#) — SLA, backup e disaster recovery, richiamati a proposito di Service Management e farm batch

Risorse

- [AWS](#) — Cos'è DevOps
- [Atlassian](#) — DevOps: guida completa
- [Microsoft Learn](#) — Introduzione a DevOps

- [Microsoft Learn — Infrastructure as Code](#)
- [Atlassian — CI/CD: guida completa](#)
- [Axelos/ITIL — Cos'è ITIL 4](#)
- [Atlassian — ITSM: guida completa](#)
- [Google Cloud — DORA: DevOps Research and Assessment](#)

10. CI/CD

Nella sezione DevOps hai già incontrato i concetti di **Continuous Integration** e **Continuous Delivery/Deployment**, applicati al lavoro quotidiano del team di **ShopFacile**: sai che servono a integrare e rilasciare il codice in modo frequente e automatizzato, invece di accumulare mesi di lavoro in un unico rilascio rischioso. Questa sezione resta sullo stesso progetto, ma va un livello più in profondità: non “cosa sono CI/CD e perché servono” (quello lo trovi in [9. DevOps](#)), ma **come funziona davvero, passo per passo, la pipeline** che Marco, Giulia e Ahmed usano ogni giorno per far arrivare le loro modifiche in produzione — cioè lo strumento concreto che rende possibile la CI/CD.

Alla fine di questa sezione, quando in una daily standup qualcuno dirà “la pipeline è rossa sul quality gate della coverage” o “il deploy in staging è partito ma serve l’approvazione manuale per la produzione”, capirai esattamente di cosa si sta parlando.

Obiettivi della sezione

Alla fine di questa sezione saprai:

- ripassare rapidamente la differenza tra Continuous Integration e Continuous Delivery/Deployment;
 - descrivere le **fasi tipiche** di una pipeline, dal checkout del codice al deploy in produzione;
 - riconoscere i diversi **trigger** che possono far scattare una pipeline;
 - distinguere gli **ambienti** (Dev, Test/QA, Staging, Produzione) e capire cosa significa “promuovere” il codice tra di essi;
 - capire cos’è un **artifact** e perché viaggia da una fase all’altra della pipeline senza essere ricostruito ogni volta;
 - distinguere una **builder image** da una **runtime image**, e capire a cosa serve un **multi-stage build**;
 - spiegare perché si scrive una pipeline come **codice versionato** (Pipeline as Code) invece di configurarla a mano da un’interfaccia grafica;
 - leggere un semplice file YAML che definisce una pipeline;
 - capire cos’è un **quality gate** e come blocca un rilascio problematico;
 - descrivere cosa succede — e cosa dovrebbe succedere — quando un deploy va male (**rollback**);
 - capire cosa sono i **feature flag** e perché separano il rilascio tecnico dalla decisione di business;
 - riconoscere alcuni **strumenti concreti** che implementano queste fasi nella realtà (Jenkins, Docker, SonarQube, Dynatrace, Jira Software).
-

10.1 Ripasso: cosa sono CI e CD

Giusto due righe di richiamo, perché il resto della sezione si basa su questi due concetti.

- **Continuous Integration (CI)**: ogni volta che qualcuno modifica il codice, quella modifica viene automaticamente **integrata** con il resto del progetto, **compilata** e **testata**. Lo scopo è scoprire i problemi di integrazione (il classico “sul mio computer funzionava”) nel giro di minuti, non di settimane.
- **Continuous Delivery / Continuous Deployment (CD)**: una volta che il codice ha superato la CI, viene automaticamente **preparato per il rilascio** (Delivery, con un click finale umano) oppure **rilasciato direttamente** (Deployment, senza intervento umano), fino a produzione.



Se questi concetti non ti sono ancora chiari al 100%, torna un momento alla sezione 9.

DevOps, dove sono introdotti insieme al perché culturale che c'è dietro. Qui li diamo per acquisiti e ci concentriamo sul **meccanismo pratico**.

Il meccanismo che rende concreti questi due concetti si chiama **pipeline**: una sequenza automatizzata di fasi che il codice attraversa, una dopo l'altra, dal momento in cui viene scritto al momento in cui gira davanti agli utenti reali di ShopFacile. Vediamo ora, fase per fase, come è fatta davvero.

10.2 Anatomia di una pipeline: le fasi tipiche

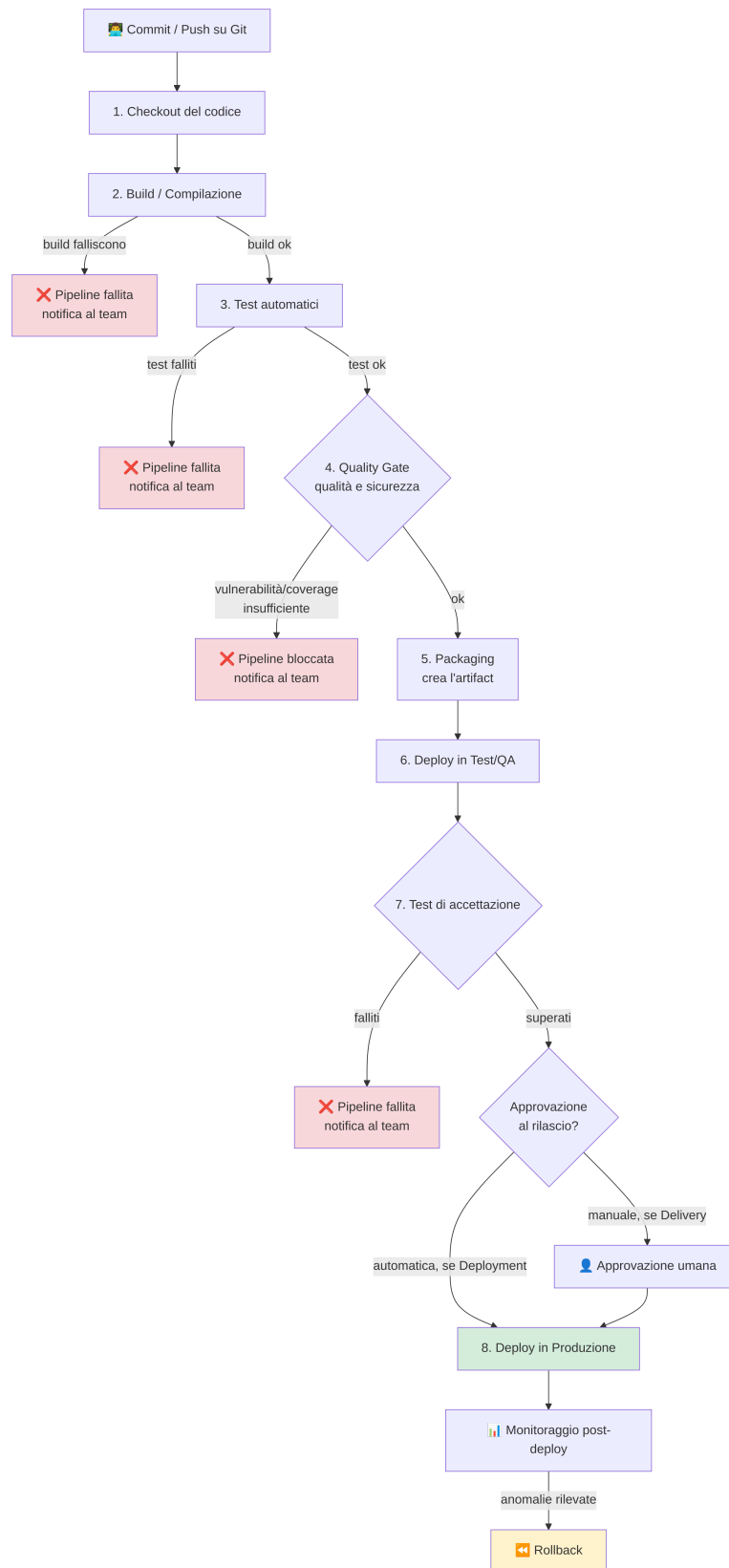
Prima di guardare come funziona una pipeline, vale la pena ricordare come rilasciava ShopFacile prima di averne una. Si andava in produzione **una volta al mese**: Marco, Giulia e Ahmed accumulavano in media 40 modifiche diverse, che finivano tutte in produzione **nella stessa serata**. Tre settimane dopo l'ultimo rilascio, un cliente segnala che il totale del carrello è sbagliato quando si applica un codice sconto. Giulia passa una giornata intera a cercare quale, delle 40 modifiche, abbia causato il problema: sono troppe, mescolate insieme, e i dettagli di ciò che ciascuno aveva cambiato tre settimane prima sono già sfumati nella memoria di tutti. Per ridurre il rischio, Luca aveva proposto di spostare il rilascio al venerdì sera, “quando ci sono pochi utenti”: il risultato era che, quando qualcosa andava storto, era il fine settimana del team ad andarsene, rincorrendo i problemi.

Da qui nasce l'idea centrale di questa sezione, l'esatto opposto dell'istinto di “accumulare tutto e rilasciare quando siamo sicuri”: **integrare e rilasciare pezzi piccoli e frequenti riduce il rischio per singola modifica** — un rilascio con 40 modifiche ha 40 possibili colpevoli, uno con una sola modifica ne ha uno solo. È esattamente questo principio, già visto in generale nella sezione DevOps, che una pipeline concreta rende possibile ogni giorno.

Immagina una pipeline come una **catena di montaggio in una fabbrica**: il “pezzo” che entra da un lato è il codice appena scritto da uno sviluppatore, e ogni stazione della catena lo controlla, lo trasforma o lo sposta, finché non esce dall'altra parte come un prodotto pronto all'uso, installato e funzionante per l'utente finale. Se una stazione rileva un difetto, il pezzo **non passa alla stazione successiva**: si ferma lì, e qualcuno viene avvisato.

Le fasi (in inglese *stage*) più comuni, nell'ordine in cui compaiono quasi sempre:

1. **Checkout del codice:** la pipeline scarica dal repository Git (GitHub e simili) esattamente la versione di codice che ha fatto scattare la pipeline — un commit specifico, un branch specifico.
2. **Build / Compilazione:** il codice sorgente viene trasformato in qualcosa di eseguibile (per i linguaggi compilati) o comunque preparato per l'esecuzione (installazione delle dipendenze, per linguaggi interpretati). Se il codice ha errori di sintassi o non compila, la pipeline si ferma già qui — è il controllo più veloce e più economico da fare.
3. **Test automatici:** vengono eseguiti i test scritti dal team (unit test, test di integrazione) per verificare che il comportamento del software sia quello atteso. Se anche un solo test fallisce, la pipeline normalmente si ferma.
4. **Analisi di qualità e sicurezza del codice:** strumenti automatici analizzano il codice sorgente (senza eseguirlo) per individuare problemi di leggibilità, complessità eccessiva, duplicazioni, e soprattutto **vulnerabilità di sicurezza** note (es. uso di librerie con falle documentate, pattern di codice pericolosi). Questa fase è spesso chiamata *static analysis* o *code scanning*.
5. **Packaging:** il risultato della build viene “impacchettato” in una forma pronta per essere distribuita e installata — un file eseguibile, un pacchetto installabile, un'immagine Docker (ne parliamo più avanti come *artifact*).
6. **Deploy in ambiente di test:** il pacchetto viene installato in un ambiente dedicato ai test (Test/QA), separato da quello che usano gli utenti reali.
7. **Test di accettazione:** test più vicini al punto di vista dell'utente finale (test end-to-end, test manuali di un QA, a volte test di performance), eseguiti sull'ambiente di test appena aggiornato, per confermare che il software si comporti bene in un ambiente “vero”, non solo nella teoria dei test automatici della fase
1.
8. **Deploy in produzione:** solo se tutte le fasi precedenti sono state superate, lo stesso identico pacchetto viene installato nell'ambiente che usano davvero gli utenti finali.



Un punto importante: **non tutte le pipeline hanno esattamente queste otto fasi**. Un progetto piccolo può avere una pipeline con solo build e test; un progetto più maturo, con requisiti di sicurezza stringenti, può avere fasi aggiuntive (scansione delle dipendenze, test di carico, firma digitale del pacchetto). Il principio, però, resta lo stesso: **fasi in sequenza, ciascuna con la possibilità di fermare tutto se qualcosa non va bene**.

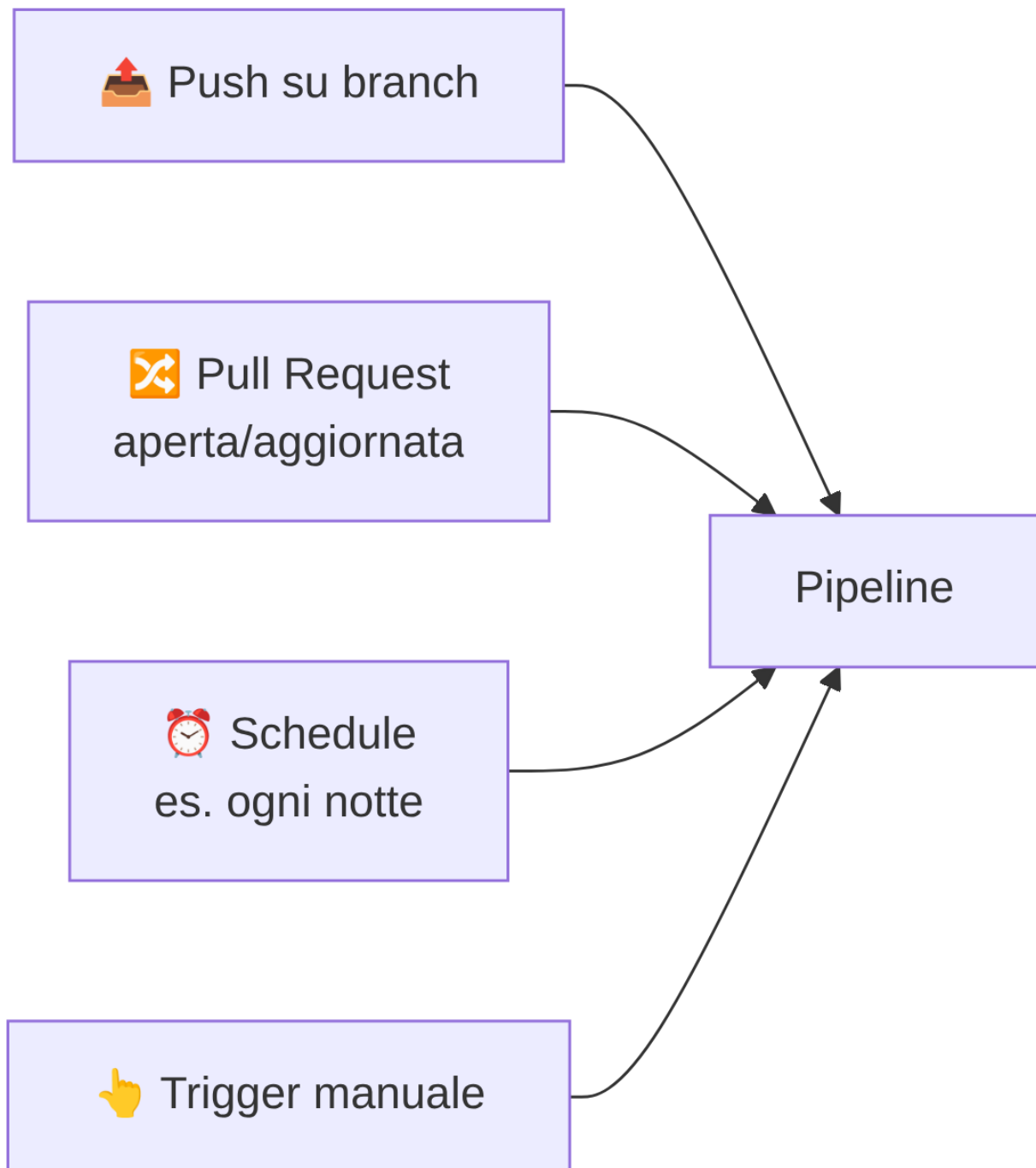
 **Esempio pratico:** Giulia apre alle 10:03 una Pull Request con una modifica al calcolo di uno sconto sul carrello di ShopFacile. Alle 10:04 la pipeline parte da sola: il checkout del codice impiega 5 secondi, la build 40 secondi. Al minuto 2 la fase di test automatici fallisce, perché uno dei test esistenti si aspettava un risultato diverso da quello prodotto dalla sua modifica. La pipeline si ferma **esattamente lì**: non arriva né alla fase di analisi di sicurezza né, ovviamente, al deploy. Giulia riceve una notifica automatica (email o messaggio nella chat del team) con il link al log del test fallito, corregge il codice, fa un nuovo commit sulla stessa MR e la pipeline riparte da capo, dal checkout.

Nell'esempio la pipeline è partita da sola, senza che Giulia dovesse lanciarla a mano: è successo perché ha aperto la Pull Request, uno degli eventi che possono far scattare una pipeline. Vediamo ora quali sono questi eventi, chiamati **trigger**.

10.3 Trigger: cosa fa scattare una pipeline

Una pipeline non gira “sempre”: scatta quando succede un evento specifico, chiamato **trigger**. I trigger più comuni sono:

- **Push su un branch:** ogni volta che qualcuno invia (push) nuovi commit su un branch (tipicamente `main`, o un branch di feature), la pipeline di CI parte automaticamente per validare quel codice.
- **Apertura (o aggiornamento) di una Pull Request:** quando si apre una MR per proporre l'unione di un branch in un altro, la pipeline gira sul codice della MR *prima* che venga approvata e unita — è uno dei modi più efficaci per impedire che codice difettoso entri nel branch principale. Se aggiungi altri commit alla MR, la pipeline riparte.
- **Pianificazione a orario (schedule):** la pipeline parte a intervalli regolari indipendentemente da modifiche al codice — ad esempio ogni notte alle 2:00, per eseguire una suite di test più lunga e completa che non ha senso far girare a ogni singolo commit, o per verificare che tutto continui a funzionare anche senza modifiche (es. dopo un aggiornamento di una libreria esterna).
- **Trigger manuale:** una persona avvia la pipeline a mano, cliccando un bottone “Run workflow” nell'interfaccia (es. GitHub Actions). È tipico per il deploy in produzione, dove spesso si preferisce un'azione deliberata di una persona autorizzata, invece che un automatismo completo.



Esempio pratico: Marco fa un push di alcuni commit sul branch della sua feature per il servizio ordini di ShopFacile. Non deve avvisare nessuno né lanciare nulla a mano: il push stesso è il trigger, e la pipeline di CI (build + test) parte automaticamente, per dare a Marco un feedback rapido su quel codice. Il deploy in produzione, invece, ha un trigger manuale: anche se tutte le fasi precedenti sono andate a buon fine, è una persona (tipicamente il Tech Lead o un ruolo di Release Manager) che clicca “Deploy in produzione” in un momento concordato — ad esempio non di venerdì pomeriggio, per non rischiare un weekend di incidenti con poche persone disponibili a intervenire.

Una volta scattata la pipeline, però, il codice di Marco non resta fermo in un solo posto: attraversa una serie di ambienti diversi, ciascuno con un ruolo preciso. È il prossimo argomento.

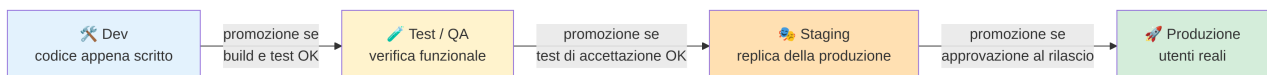
10.4 Ambienti: dove “vive” il codice, fase per fase

Il codice, durante il suo percorso, non gira sempre nello stesso posto. Attraversa diversi **ambienti**, ciascuno con uno scopo preciso:

- **Dev (Sviluppo)**: l'ambiente dove uno sviluppatore lavora e verifica le prime modifiche, spesso sul proprio computer o in un ambiente condiviso ma instabile. Qui è normale che qualcosa non funzioni: è lavoro in corso.
- **Test / QA**: un ambiente dedicato a verificare che il software funzioni correttamente, con dati simili (ma non identici) a quelli reali. Qui lavorano i tester (automatici e umani) prima che il codice vada oltre.
- **Staging**: un ambiente che riproduce **quanto più fedelmente possibile** la produzione (stessa configurazione, volumi di dati simili, stessa infrastruttura), usato come ultimo controllo prima del rilascio reale. Non tutti i progetti hanno uno staging separato dal Test/QA, ma nei progetti più maturi è una tappa distinta.
- **Produzione**: l'ambiente reale, usato dagli utenti finali. Qui ogni errore ha un impatto reale su persone vere.

Il concetto chiave è quello di **promozione**: lo stesso pacchetto di codice (lo stesso artifact, vedi paragrafo successivo) viene spostato in avanti, ambiente dopo ambiente, solo se supera i controlli previsti in quello attuale. Non si “ricostruisce” il software per ogni ambiente: si promuove **la stessa cosa**, testata identica in ogni tappa, per essere sicuri che quello che finisce in produzione sia esattamente ciò che è stato validato prima.

💡 **Analogia**: pensa alla selezione per una squadra sportiva nazionale. Un giocatore passa dalle squadre giovanili (Dev), a un torneo regionale di qualificazione (Test/QA), a un'amichevole contro una nazionale di alto livello in condizioni quasi reali (Staging), fino alla partita ufficiale (Produzione). Non cambi giocatore a ogni tappa: **lo stesso giocatore** viene promosso, se supera ogni prova, fino al campo che conta davvero.



Vale la pena notare, però, che avere più ambienti non è gratis: ognuno è infrastruttura reale da creare e mantenere, con un costo che si somma a quello della produzione. Se lo staging non riceve la stessa cura della produzione, i due **divergono silenziosamente** nel tempo — una libreria aggiornata solo in uno dei due — e da lì nasce il classico lamento “in staging funzionava”.

Approfondirai la configurazione pratica di questi ambienti — chi ci accede, come si isolano, come si gestiscono le credenziali diverse per ciascuno — nella sezione [14. Ambienti di sviluppo](#).

Abbiamo detto che il codice viene “promosso” da un ambiente all'altro **senza essere ricostruito**: ma cos'è, esattamente, questa cosa che si sposta identica da Dev a Test/QA a Staging a Produzione? È il momento di dare un nome preciso a quel “pacco”: l'artifact.

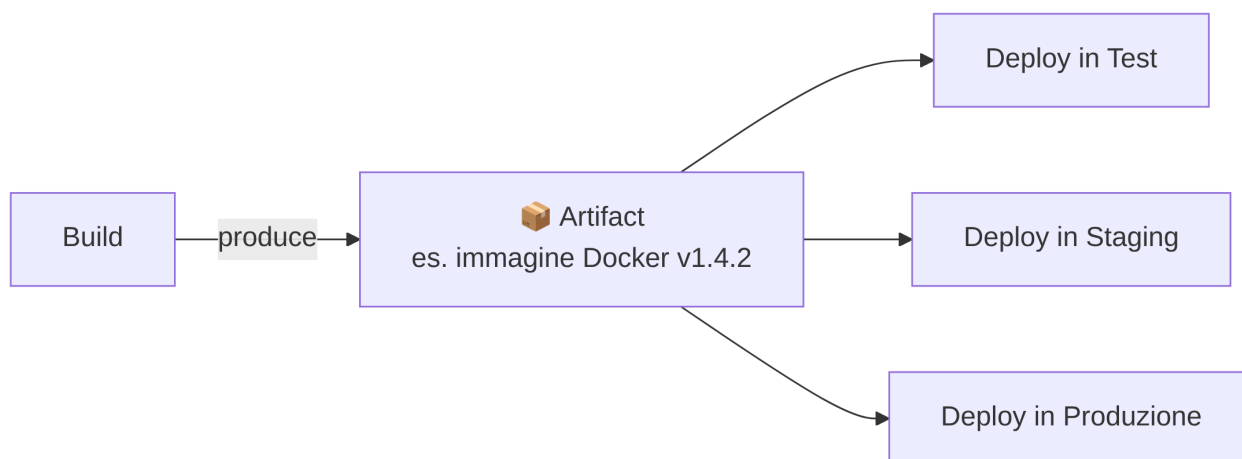
10.5 Artifact: il “pacco” che viaggia lungo la pipeline

Un **artifact** è l'output concreto e riutilizzabile prodotto da una fase della pipeline (tipicamente dalla build/packaging), che viene poi passato alle fasi successive **senza doverlo ricostruire ogni volta**.

Esempi concreti di artifact:

- un file eseguibile o una libreria compilata;
- un pacchetto installabile (es. uno `.zip`, un pacchetto per un gestore di dipendenze);
- un'**immagine Docker**, cioè un “contenitore” con dentro applicazione e tutto ciò che serve per farla girare, pronto per essere avviato identico su qualsiasi macchina;
- un report di test o di analisi di sicurezza, allegato come documentazione del rilascio.

Perché è importante non ricostruire il software a ogni fase? Perché se ogni ambiente compilasse il codice da zero, potresti finire per testare una cosa e rilasciarne un'altra leggermente diversa (magari compilata con una versione diversa di una libreria, scaricata in un momento diverso). L'artifact garantisce che **quello che hai testato in Test/QA è esattamente, byte per byte, quello che finisce in produzione** — questo principio si chiama a volte “build once, deploy many” (compila una volta sola, rilascia molte volte).



Builder image vs runtime image: cosa finisce davvero in produzione

C'è un dettaglio che riguarda proprio la costruzione di un artifact Docker, ed è facile trascurarlo finché qualcuno non se ne accorge nel modo sbagliato: l'immagine che finisce in produzione pesa **1,2 GB**, e dentro ci sono il compilatore, gli strumenti di test, il codice sorgente completo (con tutta la cronologia di Git), e le credenziali che servivano per scaricare le librerie durante la build. Tre problemi in uno, non uno solo: si trascina in produzione roba che non serve a farla funzionare, solo ad attaccarla (più superficie di attacco, un tema che approfondisci nella sezione 13); l'immagine è pesante da trasferire a ogni singolo deploy, anche quando il codice applicativo cambiato è minimo; e il codice sorgente si trova dentro qualcosa che gira in un ambiente esposto, invece di restare solo dove è stato compilato.

La soluzione è distinguere due immagini con scopi diversi:

- la **builder image** è l'immagine che contiene **tutto ciò che serve per costruire** l'applicazione: compilatore, strumenti di build, cache delle dipendenze, a volte anche il codice sorgente e gli strumenti di test;
- la **runtime image** è l'immagine **minima** che contiene solo ciò che serve per **eseguire** l'applicazione già costruita: il codice compilato (o interpretato) e le sue dipendenze effettive, nient'altro.

Il modo standard per ottenere la seconda dalla prima si chiama **multi-stage build**: un unico Dockerfile diviso in più “stadi”, dove il primo stadio compila tutto con tutti gli strumenti necessari, e solo il risultato della compilazione viene **copiato** nello stadio successivo, molto più leggero, che è quello che finisce davvero in produzione.

```
# --- Stadio 1: builder ---
# Immagine "pesante", con tutto il necessario per compilare
FROM node:20 AS builder
WORKDIR /app
COPY package*.json ./
RUN npm install                # installa anche le dipendenze di sviluppo
COPY . .
RUN npm run build              # produce i file compilati in ./dist

# --- Stadio 2: runtime ---
# Immagine minima, contiene solo ciò che serve per eseguire
FROM node:20-slim
WORKDIR /app
COPY --from=builder /app/dist ./dist      # copia solo il risultato
COPY --from=builder /app/package*.json ./
RUN npm install --omit=dev                # solo le dipendenze di produzione
CMD ["node", "dist/main.js"]
```

Il compilatore, il codice sorgente non compilato, la cronologia Git e le credenziali di build **restano nel primo stadio**, che non viene mai distribuito da nessuna parte: solo lo stadio finale diventa l'artifact che viaggia verso Test, Staging e Produzione.

Vale la pena notare che anche l'immagine di build **va versionata e mantenuta**, esattamente come qualsiasi altro artifact: non è un dettaglio da configurare una volta e dimenticare. Ed è proprio per questo che avere **un'immagine di build standard aziendale** — pensata, testata e mantenuta centralmente — rende ripetibili le build di tutti i team, invece di lasciare che ciascuno costruisca la propria a modo suo: è una delle cose che una piattaforma interna offre, tema che approfondisci nella sezione 12.

 **Trade-off:** un Dockerfile a due stadi è meno immediato da leggere di uno a stadio unico, e la cache di build diventa più delicata da gestire (un cambiamento nello stadio “builder” può invalidare cache che si vorrebbero mantenere nello stadio “runtime”). C'è poi un compromesso più sottile:

un'immagine runtime troppo spogliata, priva di qualsiasi strumento da riga di comando, rende più difficile **diagnosticare** un problema quando serve entrarci per capire cosa sta succedendo — è un compromesso reale tra sicurezza (meno superficie di attacco) e diagnosticabilità (più strumenti a disposizione in caso di incidente), non una scelta a costo zero.

Sappiamo ora **cosa** attraversa la pipeline (l'artifact) e **dove** (gli ambienti). Manca un pezzo: **chi decide** l'ordine esatto di queste fasi, e come quella decisione viene scritta e mantenuta nel tempo dal team di ShopFacile. È qui che entra in gioco il concetto di Pipeline as Code.

10.6 Pipeline as Code: la pipeline è codice, non un click di troppo

Nei primi anni degli strumenti di automazione, le pipeline si configuravano quasi sempre **a mano**, da un'interfaccia grafica: si cliccava “aggiungi fase”, si compilavano campi, si salvava. Funzionava, ma aveva problemi seri:

- **non c'era storia delle modifiche**: se qualcuno cambiava una configurazione e qualcosa si rompeva, non c'era modo semplice di vedere “chi ha cambiato cosa e quando” (lo stesso problema che risolve Git per il codice, visto nella sezione 4);
- **non era riutilizzabile**: replicare la stessa pipeline su un altro progetto significava ricliccare tutto daccapo;
- **non era rivedibile in Pull Request**: un collega non poteva fare code review di una configurazione grafica come farebbe su del codice.

La soluzione, oggi lo standard del settore, si chiama **Pipeline as Code**: la definizione della pipeline (quali fasi, in che ordine, con quali condizioni) è scritta in un **file di testo** (quasi sempre in formato YAML), **versionato in Git insieme al codice dell'applicazione**.

I vantaggi pratici di questo approccio:

- la pipeline ha una **cronologia di modifiche** come qualsiasi altro file di codice (chi, quando, perché);
- le modifiche alla pipeline passano per **Pull Request e code review**, come qualsiasi altra modifica importante;
- la pipeline può essere **copiata e riutilizzata** facilmente su altri progetti simili;
- la pipeline **vive insieme al codice che testa**: se un branch richiede una fase diversa, quella definizione sta proprio in quel branch, non in una configurazione globale scollegata dal codice.

Parlare di “file YAML versionato in Git” resta astratto finché non lo si vede scritto per intero: ecco quindi come potrebbe apparire davvero il file che definisce la pipeline di ShopFacile.

10.7 Un esempio di pipeline in YAML

Ecco un esempio semplificato di come potrebbe apparire davvero il file che definisce una pipeline di **GitHub Actions** con le fasi principali viste sopra. Un file di questo tipo si chiama in genere `.github/workflows/ci-cd.yml` e vive in quella cartella del repository (GitHub Actions legge automaticamente ogni file YAML presente lì dentro).

```

# .github/workflows/ci-cd.yml (esempio semplificato)

name: ShopFacile CI/CD

on:
  push:
    branches:
      - main
      - "feature/**"
  pull_request:
    branches:
      - main

jobs:

  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4          # 1. Checkout del codice
      - name: Installa dipendenze
        run: npm install
      - name: Compilazione
        run: npm run build                # 2. Build

  test:
    needs: build
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Unit test
        run: npm run test:unit           # 3. Test automatici
      - name: Analisi statica del codice
        run: npm run lint                # 4. Analisi qualità
      - name: Scansione vulnerabilità
        uses: security-scan/action@v1    # 4. Analisi sicurezza

  package:
    needs: test
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Crea immagine Docker
        run: docker build -t app:${{ github.run_id }} . # 5. Packaging
      - name: Pubblica artifact
        uses: actions/upload-artifact@v4

```

```

    with:
      name: app-image
      path: ./dist

deploy-test:
  needs: package
  runs-on: ubuntu-latest
  environment: test # 6. Deploy in Test/QA
  steps:
    - name: Deploy in Test/QA
      run: ./deploy.sh test ${github.run_id}

deploy-produzione:
  needs: deploy-test
  runs-on: ubuntu-latest
  environment: produzione # richiede approvazione manuale
  steps:
    - name: Deploy in Produzione
      run: ./deploy.sh produzione ${github.run_id} # 8. Deploy in Produzione

```

Non preoccuparti di capire ogni singola parola chiave: quello che conta è la **struttura**. Nota come:

- il blocco `on` in cima definisce **quando** la pipeline parte (i trigger visti al paragrafo 10.3): un push su `main` o su un branch di feature, oppure l'apertura/aggiornamento di una Pull Request verso `main`;
- ogni voce dentro `jobs` corrisponde a una delle fasi della catena di montaggio vista al paragrafo 10.2;
- `needs` collega le fasi in sequenza: un job non parte se quello da cui dipende non è andato a buon fine;
- l'`environment: produzione` è configurato (fuori da questo file, nelle impostazioni del repository su GitHub) per richiedere un'**approvazione manuale** prima di procedere — il trigger manuale di cui parlavamo sopra, applicato a un singolo job invece che a tutta la pipeline.

Guardando questo file, nota il job `test`: contiene non solo gli unit test, ma anche un'analisi di qualità e sicurezza. È proprio lì, in quel punto della pipeline, che entra in gioco uno dei controlli più importanti per un team come quello di ShopFacile: il quality gate.

10.8 Quality Gate: i controlli che possono bloccare tutto

Un **quality gate** (letteralmente “cancello di qualità”) è un punto della pipeline in cui viene verificata una condizione precisa e misurabile: se la condizione non è soddisfatta, la pipeline **si ferma**, e il codice non avanza alla fase successiva — a prescindere da quanta fretta ci sia di rilasciare.

Esempi tipici di quality gate:

Controllo	Condizione tipica	Cosa succede se non è soddisfatta
Test falliti	Tutti i test automatici devono passare	La pipeline si ferma, il team viene notificato, il codice non prosegue
Code coverage	Es. almeno il 70-80% del codice deve essere coperto da test automatici	Se la coverage scende sotto la soglia, la pipeline si blocca (anche se i test esistenti passano)
Vulnerabilità di sicurezza	Nessuna vulnerabilità di livello “alto” o “critico” nelle dipendenze usate	Il rilascio viene bloccato finché la libreria vulnerabile non viene aggiornata o sostituita
Qualità del codice	Es. nessun nuovo problema critico introdotto (complessità eccessiva, duplicazioni)	Il rilascio viene bloccato o segnalato per revisione, secondo le regole del progetto

 **Esempio pratico:** in ShopFacile, il quality gate sulla coverage è impostato all'80%. **Ahmed** aggiunge una nuova funzionalità di 120 righe per il carrello ma scrive test solo per 60 di quelle righe: la coverage complessiva del progetto scende sotto la soglia, e la pipeline si ferma con un messaggio tipo **Quality gate failed: coverage 76% (soglia 80%)**. Ahmed non può “convincere” la pipeline con una buona motivazione: deve aggiungere altri test finché la percentuale non torna sopra soglia. **Giulia**, che segue spesso queste segnalazioni, lo aiuta a capire quali casi mancano di copertura — un'occasione di apprendimento più che un rimprovero, in linea con la blameless culture vista nella sezione DevOps. Solo se davvero giustificato e con l'approvazione di un ruolo autorizzato, il team può decidere consapevolmente di alzare temporaneamente l'eccezione per quel singolo commit, lasciando comunque una traccia scritta del perché.

 **Analogia:** un quality gate funziona come i controlli di sicurezza in aeroporto. Non importa quanta fretta hai di prendere il volo (di rilasciare in produzione): se il metal detector suona (un test fallisce, una vulnerabilità viene trovata), **non passi**, punto. Il gate non è lì per essere scortese: è lì perché le conseguenze di far salire qualcuno con un oggetto pericoloso a bordo (un bug critico in produzione) sono molto peggiori del ritardo di qualche minuto per il controllo.

Il valore dei quality gate non è tecnico in senso stretto: è **organizzativo**. Sostituiscono il giudizio soggettivo (“secondo me va bene così”) con una **regola oggettiva e uguale per tutti**, applicata automaticamente a ogni singolo rilascio, senza eccezioni “perché stavolta ho fretta”. Questo è uno dei motivi per cui i team DevOps maturi riescono a rilasciare spesso e con affidabilità: i quality gate tolgono dalle spalle delle persone la responsabilità di “ricordarsi di controllare”.

Un quality gate, però, misura solo una **traccia** del valore che vuole proteggere, non il valore stesso. Una soglia mal tarata blocca rilasci per problemi irrilevanti; un gate che scatta troppo spesso, a torto, produce **assuefazione** — il team smette di prendere sul serio gli allarmi, l'esatto opposto dell'obiettivo.

C'è poi un rischio più sottile: la metrica può essere “aggirata” — scrivendo test banali solo per far salire la percentuale di coverage. Un quality gate ben progettato richiede quindi soglie scelte con criterio e una revisione periodica, non un numero fissato una volta per tutte.

Ma anche superando ogni quality gate immaginabile, resta un caso che nessun controllo automatico può eliminare del tutto: un problema che emerge solo quando il codice è già davanti agli utenti reali di ShopFacile. Cosa fa il team in quel momento è l'ultimo argomento di questa parte più “operativa” della pipeline.

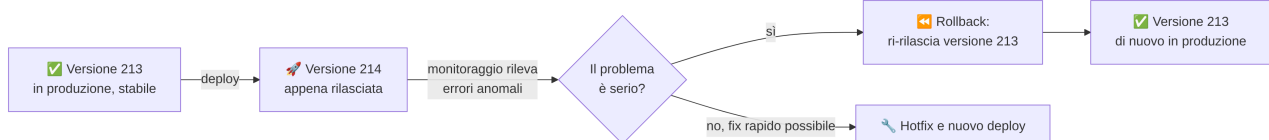
10.9 Rollback: cosa succede se un deploy va male comunque

Anche con tutti i quality gate del mondo, un rilascio in produzione può comunque rivelare un problema che nessun test aveva previsto — magari per un comportamento reale degli utenti diverso da quello simulato nei test, o per un'interazione con un sistema esterno non riproducibile in Staging.

Il **rollback** è l'operazione di **tornare alla versione precedente**, funzionante, il più rapidamente possibile, quando ci si accorge che il nuovo rilascio ha introdotto un problema serio in produzione.

Alcune strategie comuni per rendere il rollback rapido e poco rischioso:

- **Mantenere l'artifact della versione precedente pronto:** proprio perché ogni deploy usa un artifact versionato (vedi paragrafo 10.5), tornare indietro spesso significa semplicemente “rilascia di nuovo l'artifact della versione N-1”, un'operazione rapida quanto il deploy stesso. Ad esempio, per lo storico di rilasci `deploy.sh produzione 214` (versione attuale, con il problema), il rollback potrebbe essere semplicemente `deploy.sh produzione 213` (versione precedente, nota funzionante).
- **Deploy graduali (blue-green, canary):** invece di sostituire l'intero ambiente di produzione in un colpo, si rilascia la nuova versione solo a una piccola parte del traffico/utenti (canary) o si mantengono due ambienti identici e si sposta il traffico dall'uno all'altro solo dopo aver verificato che tutto funzioni (blue-green). Se qualcosa va storto, si torna a instradare il traffico sulla versione precedente, con impatto minimo sugli utenti. Il prezzo di questa sicurezza aggiuntiva è concreto: blue-green richiede di **mantenere due ambienti di produzione** invece di uno (il doppio dei server attivi, anche se solo uno riceve traffico reale in un dato momento), e sia blue-green che canary richiedono un sistema di instradamento del traffico più complesso da configurare e da mantenere di un semplice “sostituisci tutto”.
- **Monitoraggio post-deploy:** metriche e allarmi automatici (es. tasso di errore, tempi di risposta) subito dopo un deploy, per accorgersi di un problema in minuti e non quando arrivano le segnalazioni degli utenti.



 **Esempio pratico:** pochi minuti dopo il deploy della versione 214 del servizio pagamenti di ShopFacile, gli alert segnalano un tasso di errore anomalo sugli acquisti. **Luca**, come Scrum Master, convoca al volo Marco e Giulia per decidere insieme: il problema è abbastanza serio da giustificare un rollback immediato, oppure basta un hotfix rapido? In questo caso decidono per il rollback: si ri-rilascia la versione 213, nota e stabile, mentre Marco indaga con calma sulla causa reale, senza la pressione di dover risolvere tutto a caldo in produzione.

Un punto culturale importante da portare con te come futuro Project Manager: **un rollback non è un fallimento del team, è un successo del processo**. Un'organizzazione che rilascia frequentemente e ha un rollback rapido e testato è molto più sana di una che rilascia raramente sperando che “vada tutto bene”, perché quest'ultima, quando qualcosa va storto (e prima o poi succede), non ha un piano B pronto.

Rollback e deploy gradualali riguardano cosa fare **dopo** che il codice è già in produzione. C'è però anche un modo per disinnescare il rischio **prima** ancora che serva un rollback, separando due decisioni che finora abbiamo trattato come una sola: quando il codice arriva in produzione, e quando la funzionalità diventa visibile agli utenti. È il compito dei feature flag.

10.10 Feature flag: disaccoppiare il rilascio dalla decisione di business

Immagina che **Sara**, Product Owner di ShopFacile, voglia lanciare un nuovo filtro di ricerca solo dal lunedì prossimo, magari prima per il 10% degli utenti. Se “rilasciare il codice” e “attivare la funzionalità” fossero la stessa cosa, questa decisione di business finirebbe nelle mani del team tecnico al momento del deploy.

Un **feature flag** (*feature toggle*) è un interruttore da configurazione che decide se una funzionalità è attiva o spenta, e per chi. Il codice può essere rilasciato già “spento”, superando la pipeline come qualsiasi altra modifica; **Sara**, senza un nuovo deploy, decide poi quando accenderlo e per quale percentuale di utenti. Il flag **disaccoppia** così il rilascio tecnico dalla decisione di business.

 Anche i feature flag costano, se non gestiti con disciplina: un flag introdotto per un test e mai rimosso resta nel codice a tempo indefinito, diventando esso stesso **debito tecnico** — va tracciato e rimosso appena la decisione che rappresentava è definitiva.

Fin qui abbiamo visto ogni fase, trigger e controllo della pipeline in modo concettuale, senza legarli a un prodotto specifico. Nella pratica, però, il team di ShopFacile usa strumenti reali per implementare ciascuna di queste fasi: è il momento di vederli con i loro nomi.

10.11 Strumenti concreti della pipeline

Finora abbiamo parlato di fasi, trigger, ambienti e quality gate in modo concettuale, senza legarci a un prodotto specifico. Nella pratica, ogni fase della pipeline è quasi sempre implementata da uno strumento reale, e sentirai questi nomi citati costantemente nelle conversazioni tecniche del team di ShopFacile. Vediamone cinque tra i più diffusi, seguendo lo stesso ordine in cui si passano il testimone lungo la pipeline.

Jenkins: l'orchestratore storico della pipeline

Jenkins è un motore di automazione CI/CD **open source**, tra i più storici e diffusi al mondo: orchestra l'esecuzione delle varie fasi della pipeline (build, test, packaging, deploy) tramite **job configurabili**, spesso definiti in un file chiamato **Jenkinsfile** — lo stesso principio di “Pipeline as Code” visto al paragrafo 10.6, applicato a Jenkins.

È un'alternativa **self-hosted** a strumenti di CI/CD nativi come GitHub Actions: molte aziende lo scelgono quando vogliono mantenere il pieno controllo sull'infrastruttura di automazione, o quando hanno esigenze di integrazione con strumenti legacy che uno strumento nativo non copre.



Analogia: se GitHub Actions è la catena di montaggio già integrata nella fabbrica (lo stesso edificio che ospita anche il magazzino del codice), Jenkins è un impianto di automazione **indipendente**, che puoi installare dove vuoi e collegare a qualsiasi “magazzino” di codice (GitHub, ma anche altri), con la massima flessibilità di configurazione.

Una volta che Jenkins ha completato con successo build e test del servizio ordini di ShopFacile, il risultato non può restare un file sparso sul disco di un agente di build: serve un formato pronto per viaggiare identico fino in produzione. È qui che il testimone passa a Docker.

Docker: il packaging portatile

Abbiamo già incontrato **Docker** al paragrafo 10.5, parlando di artifact: è la tecnologia di **containerizzazione** più diffusa, che permette di “impacchettare” un'applicazione con tutte le sue dipendenze in un'**immagine** eseguibile identica ovunque venga avviata. Nella pipeline, Docker interviene tipicamente nella fase di **packaging**: il codice compilato e testato viene trasformato in un'immagine versionata (es. **app:1.4.2**), pronta per essere distribuita e avviata identica in Test, Staging e Produzione — lo stesso principio “build once, deploy many” già visto.

Avere un'immagine Docker pronta, però, non significa ancora che sia **sicuro** farla avanzare verso il deploy: prima bisogna verificare che il codice al suo interno superi le soglie di qualità e sicurezza del progetto. È il compito di SonarQube.

SonarQube: il quality gate concreto

SonarQube è uno strumento di **analisi statica del codice** (*static analysis*): esamina il codice sorgente senza eseguirlo, misurando copertura dei test (coverage), complessità, duplicazioni e vulnerabilità di sicurezza note. È, in pratica, **l'implementazione concreta del quality gate** descritto al paragrafo 10.8:

se il codice non supera le soglie configurate (es. coverage minima, zero vulnerabilità critiche), SonarQube segnala il fallimento e la pipeline si ferma, esattamente come nell'esempio della coverage all'80% visto in quel paragrafo.

Superato il quality gate di SonarQube e completato il deploy, il lavoro della pipeline non finisce lì: bisogna sapere come si comporta l'applicazione una volta davanti agli utenti reali di ShopFacile. Questo compito spetta a Dynatrace.

Dynatrace: il monitoraggio che informa il rollback

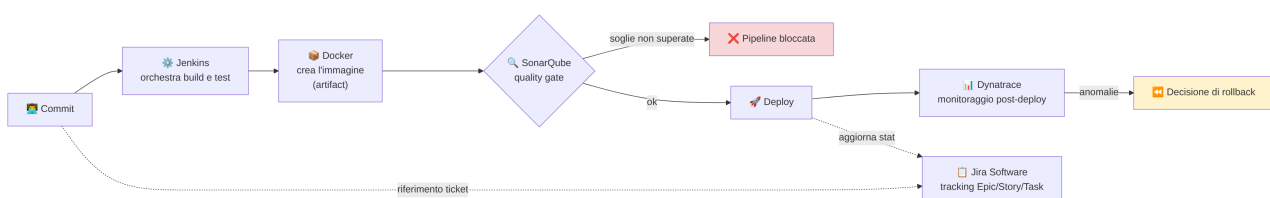
Dynatrace è una piattaforma di **observability e monitoring**: tiene sotto controllo l'applicazione **dopo** il deploy, in tempo reale, rilevando anomalie di performance, errori e comportamenti sospetti in produzione. È esattamente il tipo di “monitoraggio post-deploy” citato al paragrafo 10.9 sul rollback: quando Dynatrace rileva un tasso di errore anomalo dopo un rilascio, quel segnale è spesso l'innesco che porta il team a decidere un rollback, prima ancora che arrivino segnalazioni dagli utenti.

Tutto questo percorso — build, packaging, quality gate, monitoraggio — risolve il problema di portare il codice in produzione in modo sicuro. Ma resta un problema diverso, tipicamente di **Sara**, la Product Owner: una volta approvata una richiesta di business, come risalire, settimane dopo, a **quale codice** l'ha implementata e **quale rilascio** l'ha portata in produzione, senza chiedere a Marco o Giulia ogni volta? Senza un collegamento esplicito tra ticket di business e lavoro tecnico, questa domanda diventa una caccia al tesoro. È il ruolo di Jira Software.

Jira Software: il tracking del lavoro, collegato alla pipeline

Jira Software è uno strumento di **work e project tracking**, concettualmente equivalente a **GitHub Issues/Projects** (Epic, Story, Task che avanzano su una board). Molti team che usano GitHub per il codice continuano a usare Jira, invece delle Issue native di GitHub, per gestire il backlog: in questi casi è comune integrare i due strumenti, referenziando l'ID del ticket Jira (es. **PROJ-123**) nel nome del branch o nel messaggio di commit, così che Jira mostri automaticamente il collegamento alla Pull Request GitHub corrispondente, e chi guarda il ticket veda subito a che punto è il lavoro tecnico collegato — risolvendo esattamente il problema di tracciabilità di cui parlavamo sopra.

Come si incastrano nella pipeline



Tool	A cosa serve	Fase della pipeline
Jenkins	Motore di automazione CI/CD, orchestra l'intera pipeline tramite job/Jenkinsfile	Orchestrazione end-to-end
Docker	Containerizzazione: impacchetta l'app e le dipendenze in un'immagine portatile	Packaging / artifact
SonarQube	Analisi statica del codice: coverage, complessità, vulnerabilità	Quality gate
Dynatrace	Observability/monitoring in tempo reale dell'applicazione in produzione	Monitoraggio post-deploy / rollback
Jira Software	Work e project tracking (Epic/Story/Task), integrato con MR e commit	Trasversale, collegato al codice

Non serve che tu sappia configurare nessuno di questi strumenti: quello che conta, nel tuo ruolo, è riconoscerli quando li senti nominare e sapere a quale fase della pipeline (o del lavoro del team) corrispondono, per seguire con consapevolezza le conversazioni tecniche e capire, ad esempio, perché “il rilascio è bloccato dal quality gate di SonarQube” o “Dynatrace ha segnalato un'anomalia, stiamo valutando il rollback”.

Un'ultima cosa da non dare per scontata: questa catena di strumenti non si mantiene da sola. Qualcuno deve tenere aggiornato il file YAML della pipeline, decidere se alzare o abbassare una soglia di SonarQube, aggiornare le regole di Dynatrace. È lavoro vero, non manutenzione invisibile: va stimato e inserito nel backlog, come qualsiasi altra attività di sviluppo.

10.12 Riepilogo

In questa sezione hai visto, con più dettaglio pratico rispetto alla sezione DevOps, come funziona davvero una pipeline di CI/CD:

- una pipeline è una **sequenza di fasi** (checkout, build, test, analisi qualità/sicurezza, packaging, deploy in test, test di accettazione, deploy in produzione), ciascuna in grado di fermare tutto se qualcosa non va;
- scatta grazie a **trigger** diversi: push, Pull Request, pianificazione a orario, o avvio manuale;
- il codice si muove attraverso **ambienti** (Dev, Test/QA, Staging, Produzione) tramite un processo di **promozione**, senza essere ricostruito a ogni passaggio;
- l'**artifact** è il pacchetto concreto (un eseguibile, un'immagine Docker...) che viaggia identico da una fase all'altra; distinguere **builder image** (tutto ciò che serve per costruire) da **runtime image** (solo ciò che serve per eseguire), tramite un **multi-stage build**, riduce peso e superficie di attacco dell'immagine che finisce in produzione;

- scrivere la pipeline come **Pipeline as Code** (un file YAML versionato) porta gli stessi benefici che Git porta al codice: storia, revisione, riuso;
- i **quality gate** trasformano il controllo di qualità da giudizio soggettivo a regola automatica e oggettiva;
- il **rollback** è il piano B necessario quando, nonostante tutti i controlli, qualcosa va comunque storto in produzione;
- i **feature flag** separano il momento in cui il codice arriva in produzione da quello in cui una funzionalità diventa visibile agli utenti, spostando quella decisione verso il business;
- strumenti concreti come **Jenkins**, **Docker**, **SonarQube**, **Dynatrace** e **Jira Software** implementano queste fasi nella realtà — ma non si mantengono da sole: la manutenzione di pipeline e quality gate è lavoro da pianificare, non magia.

Nelle prossime sezioni vedrai come questi concetti si intreccino con le scelte di **architettura software** (sezione 11) e con l'infrastruttura **cloud** (sezione 12) su cui le pipeline effettivamente rilasciano il codice.

Esercizi pratici

Gli esercizi che segui qui sotto ti chiedono di **osservare, leggere e provare a ragionare** su una pipeline reale, non di scriverne una da zero: l'obiettivo di questa fase è riconoscere sul campo i concetti visti in questa sezione, non ancora configurare strumenti in autonomia.

1. **Disegna lo schema di una pipeline reale.** Chiedi a un developer o al Tech Lead di mostrarti (anche solo a schermo) l'ultima esecuzione di una pipeline nel progetto. Disegna su un foglio le fasi che vedi davvero (non quelle “da manuale”): quali ci sono, quali non ci sono, in che ordine.
 ✓ **Come verificare:** mostra il tuo schizzo a chi te l'ha fatta vedere e chiedi “manca qualcosa o ho capito bene l'ordine?” — se la risposta è “esatto”, hai capito la struttura.
2. **Trova i trigger reali del progetto.** Fatti mostrare (o trova da solo, se hai accesso) il file YAML della pipeline principale e individua il blocco che definisce i trigger (push, PR, schedule, manuale). Scrivi in una riga, con parole tue, quando scatta la pipeline di CI e quando invece scatta il deploy in produzione. ✓ **Come verificare:** chiedi a un collega di confermare che la tua descrizione corrisponde al comportamento reale che ha osservato lui nei giorni scorsi.
3. **Leggi un file YAML di pipeline senza aiuto.** Prendi l'esempio YAML del paragrafo 10.7 (o, meglio, un estratto reale e semplice del progetto, se riesci a farti mostrare i primi 20-30 righe di un file di pipeline) e, senza guardare le spiegazioni, prova a rispondere a voce alta: “quanti job ci sono? qual è la dipendenza tra il primo e il secondo? cosa fa scattare l'intero workflow?”. ✓ **Come verificare:** rileggi la tua risposta confrontandola con il file — se hai individuato correttamente il blocco `on`, il numero di `jobs` e almeno un `needs`, hai capito la struttura di base.
4. **Simula un quality gate che blocca un rilascio.** Immagina (per iscritto, in 4-5 righe) uno scenario concreto — diverso da quelli già visti in questa sezione — in cui un quality gate blocca una pipeline: scegli tu il controllo (test falliti, coverage, vulnerabilità, qualità del codice), descrivi cosa succede e chi dovrebbe essere avvisato. ✓ **Come verificare:** fai leggere il tuo scenario a un collega developer o QA e chiedigli se, secondo la sua esperienza, è realistico e se la pipeline nel progetto reagirebbe davvero così.

5. **Ricostruisci un rollback a parole.** Chiedi a un collega se nel progetto è mai capitato un rollback (anche piccolo) e fatti raccontare cosa è successo: cosa ha fatto scattare l'allarme, quanto tempo è passato tra il deploy e l'accorgersi del problema, come si è tornati alla versione precedente. Scrivi la sequenza in 4-6 passaggi.  **Come verificare:** la tua sequenza deve contenere almeno un momento di “rilevazione del problema” e un momento di “ripristino della versione precedente” — se manca uno dei due, richiedi il dettaglio mancante e completa lo schema.
6. **Distingui Delivery da Deployment nel progetto.** Osservando (o chiedendo) come funziona davvero il rilascio in produzione nel progetto, stabilisci se si tratta di Continuous Delivery (con approvazione manuale finale) o Continuous Deployment (automatico fino in produzione), e spiega in 2-3 righe **perché**, secondo te, il team ha scelto quell'approccio per quell'ambiente specifico.  **Come verificare:** confronta la tua conclusione con quanto ti conferma la tua collega Scrum Master/PM o un Tech Lead — se coincide, hai capito la differenza pratica tra i due concetti, non solo la definizione teorica.
7. **Riconosci gli strumenti concreti del progetto.** Chiedi a un developer o al Tech Lead quali strumenti concreti, tra quelli visti al paragrafo 10.11 (o eventuali equivalenti), usa davvero il progetto per orchestrare la pipeline, fare code quality e monitorare la produzione. Per ciascuno che ti viene citato, scrivi a quale fase della pipeline corrisponde.  **Come verificare:** confronta il tuo elenco con la tabella del paragrafo 10.11 — se hai assegnato correttamente ogni strumento citato alla fase giusta, hai capito il collegamento tra teoria e strumenti reali.
8. **Trova builder e runtime image nel progetto reale.** Chiedi a un developer o DevOps engineer se il progetto usa un Dockerfile multi-stage, e fatti indicare quanto pesa l'immagine finale che va in produzione rispetto a quella usata per compilare. Chiedi anche se è mai capitato di dover “entrare” in un container di produzione per diagnosticare un problema, e quanto è stato facile o difficile farlo con un'immagine runtime minimale.  **Come verificare:** sai spiegare a un collega, con parole tue, cosa contiene la builder image che non contiene la runtime image, e perché quella differenza non è un dettaglio ma una scelta di sicurezza.

Collegamenti

- [11. Architetture software](#) — come è organizzato il software che la pipeline compila, testa e rilascia
- [12. Cloud](#) — dove “vivono” fisicamente gli ambienti di Dev, Test, Staging e Produzione su cui la pipeline effettua il deploy
- [13. Sicurezza](#) — perché ridurre la superficie di attacco di un'immagine runtime è parte della sicurezza, non solo delle performance

Risorse

- [Microsoft Learn — Continuous integration e continuous delivery](#)
- [GitHub Docs — Documentazione su GitHub Actions](#)
- [GitHub Docs — Sintassi del workflow per GitHub Actions](#)
- [Jenkins — Documentazione ufficiale](#)
- [Docker Docs — What is a container image](#)

- [Docker Docs — Multi-stage builds](#) — guida ufficiale alla tecnica builder image / runtime image
- [SonarQube — Documentazione ufficiale](#)
- [Dynatrace — Cos'è l'observability](#)
- [Atlassian — Jira Software, panoramica](#)
- [Atlassian — Continuous integration vs continuous delivery vs continuous deployment](#)
- [Martin Fowler — Continuous Delivery](#)

11. Architetture software

Nella sezione 2 (Fondamenti di informatica) hai già incontrato i mattoncini di base: API, REST, database, container. In questa sezione facciamo un passo in più: vediamo come questi mattoncini vengono **combinati insieme** per costruire il software di cui il tuo team si occupa — e lo facciamo seguendo di nuovo **ShopFacile**, la piattaforma e-commerce già incontrata nelle sezioni DevOps e CI/CD, con **Marco** nel ruolo di chi discute spesso le scelte infrastrutturali e architetturali del progetto.

Non ti serve saper progettare un'architettura software — è il lavoro degli sviluppatori e degli architetti. Ti serve invece **capire a grandi linee come è “fatto” il software**, per poter:

- capire perché certe attività richiedono più tempo di altre (“aggiungere questa funzionalità è complicato perché tocca tre servizi diversi”);
- dialogare con il team usando le parole giuste (“frontend”, “backend”, “microservizio”, “database”);
- capire meglio certe scelte tecniche quando vengono discusse in retrospettiva o in fase di stima.

Obiettivi della sezione

Al termine di questa sezione saprai:

- distinguere un'architettura monolitica da un'architettura a microservizi, e spiegarne pro e contro;
- capire la differenza tra frontend e backend;
- capire cos'è il modello client-server e l'architettura a 3 livelli;
- avere un'idea di base di come comunicano i vari “pezzi” di un software (API/REST e code di messaggi);
- capire cos'è un **Event-Driven Architecture** e distinguere un evento da un comando;
- sapere cos'è il “serverless” e quando viene usato.

11.1 Cos'è un'architettura software

Quando si costruisce una casa, prima di posare un mattone si disegna una **pianta**: dove va la cucina, dove il bagno, come sono collegate le stanze, dove passano i tubi dell'acqua e i cavi elettrici. Quella pianta è l'“architettura” della casa: definisce come sono organizzati e collegati i suoi pezzi.

L'**architettura software** è la stessa cosa applicata a un programma: è il modo in cui le sue diverse parti (chi mostra le pagine, chi elabora i dati, chi li salva) sono organizzate e come comunicano tra loro.

Come per una casa, non esiste un'unica pianta giusta per tutti i casi: una villetta e un grattacielo hanno esigenze diverse e richiedono progetti diversi. Lo stesso vale per il software: la scelta dell'architettura dipende dalle dimensioni del progetto, dal numero di utenti, dal team disponibile e da quanto velocemente deve poter crescere.

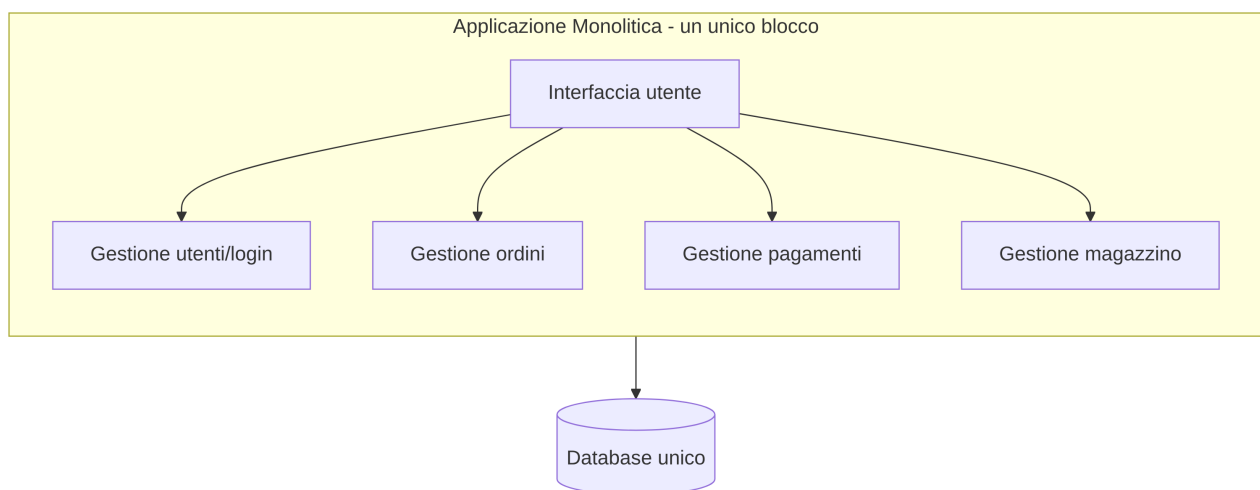
Anche ShopFacile, come ogni software, ha dovuto affrontare questa scelta fin dall'inizio. Vediamo le due “piante” più comuni tra cui il team ha dovuto decidere: il monolite e i microservizi.

11.2 Architettura Monolitica

Un'architettura **monolitica** (dal greco “un unico blocco di pietra”) è quella in cui **tutto il software è scritto e distribuito come un unico blocco unito**: la parte che gestisce gli utenti, quella che gestisce gli ordini, quella che gestisce i pagamenti, quella che parla con il database... tutto vive nello stesso progetto, viene compilato insieme e viene eseguito come un unico programma.

Analogia: il grande magazzino unico

Pensa a un **grande magazzino** che vende di tutto sotto lo stesso tetto: abbigliamento, elettronica, alimentari, arredamento. C'è un'unica cassa centrale, un unico sistema di gestione del magazzino, un'unica squadra di addetti che, in teoria, potrebbe occuparsi di qualsiasi reparto. È comodo da costruire all'inizio (un solo edificio, una sola gestione), ma se un giorno vuoi ampliare solo il reparto elettronica, devi comunque intervenire sull'intero edificio, e se va in tilt il sistema di cassa centrale, si ferma la vendita in **tutti** i reparti contemporaneamente.



Pro del monolite

- **Semplice da avviare:** all'inizio di un progetto, con un team piccolo, è molto più rapido costruire e mandare in produzione un blocco unico.
- **Meno complessità operativa:** un solo “pacchetto” da distribuire, un solo ambiente da monitorare, meno pezzi che possono comunicare male tra loro.
- **Più facile da testare end-to-end,** perché tutto il flusso vive nello stesso posto.

Contro del monolite

- **Difficile da scalare in modo selettivo:** se solo una parte del software è sotto carico (es. tanti utenti che fanno ricerche), non puoi “potenziare” solo quella parte: devi duplicare l'intero blocco, anche le parti che non ne hanno bisogno.
- **Un problema in una parte può bloccare tutto:** un bug o un sovraccarico in un singolo componente rischia di far cadere l'intera applicazione, non solo quella funzionalità.
- **Rilasci più lenti e rischiosi al crescere delle dimensioni:** con tanti sviluppatori che lavorano sullo stesso blocco di codice, ogni rilascio richiede di testare e distribuire **tutto** insieme, anche se hai modificato solo un piccolo pezzo.

- **Difficile far crescere tanti team in parallelo** senza che si “pestino i piedi” a vicenda sullo stesso codice.

Esempio pratico: un lunedì, **Marco, Giulia e Ahmed** arrivano tutti e tre con una modifica pronta e indipendente — ma ShopFacile è un unico blocco, quindi esiste **un solo rilascio possibile** che le contiene tutte insieme. Se il test automatico sulla modifica di Giulia fallisce all'ultimo momento, il rilascio si ferma: anche le modifiche di Marco e Ahmed, già pronte da ore, restano bloccate con lei, benché il loro lavoro non abbia nulla in comune col suo. Il costo del monolite non è solo “ricompilare e rilasciare **l'intera applicazione**”: è soprattutto **far aspettare le persone** l'una per colpa delle altre — un costo che, all'inizio, il team accettava volentieri in cambio della semplicità.



Per confronto: non tutti i software devono necessariamente evolvere oltre il monolite. Un gestionale interno per l'ufficio commerciale, usato da poche decine di persone e senza picchi di traffico, può restare un monolite per anni senza alcun problema: la scelta dipende dal contesto, non è “il monolite è superato”. Lo vedremo tra poco parlando di quando ha davvero senso cambiare.

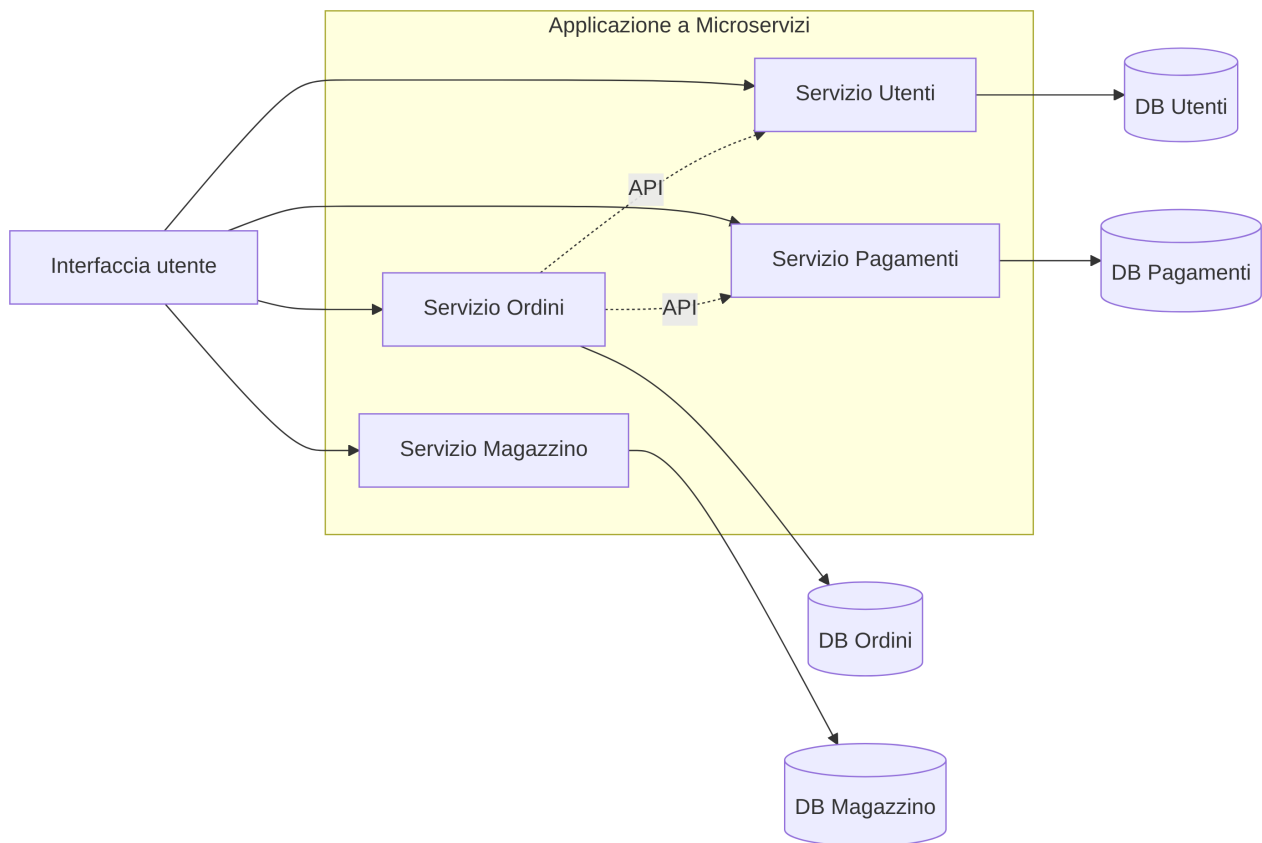
Con il tempo, però, ShopFacile è cresciuto: più utenti, più developer nel team, più traffico concentrato su alcune funzionalità (il catalogo durante i saldi, i pagamenti durante il Black Friday). È qui che il monolite ha iniziato a mostrare i suoi limiti, ed è qui che entra in scena l'architettura alternativa: i microservizi.

11.3 Architettura a Microservizi

Un'architettura a **microservizi** divide il software in tanti **piccoli servizi indipendenti**, ciascuno responsabile di **una sola area** (es. “il servizio che gestisce gli utenti”, “il servizio che gestisce i pagamenti”, “il servizio che gestisce il magazzino”). Ogni servizio ha il suo codice, a volte il suo database, e viene distribuito e aggiornato **separatamente** dagli altri. I servizi comunicano tra loro tramite API (esattamente quelle che hai visto nella sezione 2) o tramite code di messaggi (le vediamo a breve).

Analogia: tanti piccoli negozi specializzati

Rispetto al grande magazzino unico, immagina ora una **via dello shopping** con tanti piccoli negozi specializzati: la libreria, il negozio di elettronica, il fruttivendolo. Ogni negozio ha la sua cassa, il suo personale, i suoi orari, la sua gestione del magazzino, ed è **indipendente** dagli altri: se il fruttivendolo chiude per ferie, la libreria continua a vendere libri senza problemi. Se un giorno la libreria diventa particolarmente famosa e affollata, il proprietario può assumere più commessi solo lì, senza dover potenziare anche gli altri negozi. In compenso, se un cliente vuole comprare un libro E della frutta, deve andare in due negozi diversi (cioè: serve più “coordinamento” tra i pezzi).



Pro dei microservizi

- **Scalabilità selettiva:** puoi potenziare solo il servizio sotto carico (es. più istanze del servizio Ordini durante il Black Friday), senza toccare gli altri.
- **Rilasci indipendenti:** puoi aggiornare il servizio Pagamenti senza dover rilasciare (e rischiare di rompere) il servizio Magazzino.
- **Team più autonomi:** team diversi possono lavorare su servizi diversi in parallelo, con meno conflitti e meno dipendenze reciproche.
- **Guasto isolato:** se un servizio ha un problema, in teoria gli altri possono continuare a funzionare (non sempre in pratica, ma è l'obiettivo).
- **Libertà tecnologica:** ogni servizio può, in teoria, usare il linguaggio o il database più adatto al suo compito specifico.

Contro dei microservizi

- **Complessità operativa molto più alta:** ora ci sono tanti "pezzi" da distribuire, monitorare, far comunicare correttamente tra loro. Serve un'infrastruttura DevOps solida (vedi sezione 9) per gestirla bene.
- **Comunicazione tra servizi da progettare con cura:** se il servizio A chiama il servizio B che chiama il servizio C, e uno di questi è lento o irraggiungibile, il problema si propaga.
- **Più difficile da debuggare:** un errore può nascere dall'interazione tra più servizi, ed è un costo aggiuntivo che richiede di investire proprio negli strumenti di monitoraggio già visti nella sezione 9 (log centralizzati, dashboard, allarmi) — senza di essi, capire quale servizio ha causato il problema può richiedere ore anziché minuti.

- **Richiede un team più maturo** su temi come CI/CD, containerizzazione e monitoraggio, perché la complessità operativa cresce parecchio.

Esempio pratico: Marco e il resto del team decidono di scomporre ShopFacile in microservizi: il catalogo prodotti diventa un servizio separato, con il suo repository e la sua pipeline di rilascio (quella vista nella sezione 10). Se il team vuole aggiungere un nuovo filtro di ricerca nel catalogo, rilascia **solo quel servizio**: i pagamenti e la gestione ordini continuano a funzionare esattamente come prima, senza bisogno di essere ritestati o rilasciati di nuovo.

Il prezzo pagato: quando la rete si mette in mezzo

Qualche settimana dopo la scomposizione, un cliente segnala un ordine risultato pagato ma mai registrato come completato. Giulia indaga: il servizio Pagamenti ha confermato l'incasso, ma il messaggio verso il servizio Ordini si è perso per un timeout di rete. Nel vecchio monolite questo **non poteva accadere**: pagamento e ordine erano un'unica operazione sullo stesso database, o riuscivano entrambe o fallivano entrambe. Ora, con due servizi che si parlano via rete, una delle due operazioni può andare a buon fine e l'altra no — un problema chiamato **transazione distribuita**: un'operazione che deve avere successo su più servizi insieme, ma che la rete può “spezzare” a metà.

I microservizi non eliminano la complessità: la **spostano** dal codice (dove un unico database garantiva tutto o niente) alla rete (dove serve gestire esplicitamente messaggi persi, in ritardo o duplicati). Per il tuo ruolo, la conseguenza è concreta: una modifica che coinvolge la comunicazione tra servizi può costare più del previsto in stima, per la gestione di errori e casi limite che nel monolite non esistevano.

Quando ha senso passare da monolite a microservizi

Non è vero che i microservizi sono “sempre meglio”. Anzi: molti progetti, soprattutto agli inizi, **partono volutamente come monolite**, proprio come ha fatto ShopFacile, perché è più rapido e il team è piccolo. Il passaggio a microservizi ha senso quando succede una o più di queste cose — ed è esattamente il ragionamento che Marco ha portato al team di ShopFacile:

- il team è cresciuto molto e più squadre lavorano sullo stesso codice, intralciandosi a vicenda;
- alcune parti del sistema hanno bisogno di scalare molto più di altre (es. il catalogo prodotti di ShopFacile riceve 100 volte più traffico della gestione fatture);
- i rilasci del monolite sono diventati lenti, rischiosi e “tutto o nulla”;
- il codice è diventato talmente grande e intrecciato che anche piccole modifiche richiedono di toccare (e ritestare) mezza applicazione.

Una buona regola pratica che sentirai spesso: **“non iniziare con i microservizi”**. Molte aziende, incluse alcune molto famose nel settore tech, sono partite da un monolite e sono passate ai microservizi solo quando la crescita lo ha reso necessario — non il contrario. Un gestionale interno per l'ufficio commerciale, usato da poche decine di persone senza picchi di traffico, è un controesempio utile: probabilmente non incontrerà mai nessuna di queste quattro condizioni, e restare monolite per lui è la scelta giusta, non un ritardo da recuperare.

Monolite vs Microservizi: la tabella di confronto

Aspetto	Monolite	Microservizi
Velocità di partenza di un progetto	Alta: si parte rapidamente	Più bassa: serve progettare i confini tra servizi e l'infrastruttura
Scalabilità	Si scala tutto insieme, anche le parti che non servono	Si scala solo il servizio che ne ha bisogno
Complessità operativa (deploy, monitoraggio)	Bassa: un solo blocco da gestire	Alta: tanti servizi da distribuire, far comunicare, monitorare
Rilasci	Un rilascio unico, tocca tutto il sistema	Rilasci indipendenti per singolo servizio
Isolamento dei guasti	Basso: un problema può bloccare tutto	Più alto: un servizio può fallire senza fermare tutti gli altri
Autonomia dei team	Bassa: più team sullo stesso codice si intralciano	Alta: ogni team può gestire i propri servizi
Adatto a	Progetti piccoli/medi, team piccoli, fasi iniziali (MVP)	Progetti grandi, molti team, esigenze di scalabilità differenziate

Un effetto collaterale: la Legge di Conway

Un'osservazione nota come **Legge di Conway**: la struttura del software che un'azienda costruisce tende a **rispecchiare la struttura del team** che lo costruisce — e vale anche il contrario. Quando il team di ShopFacile decide di scomporre il software per dominio (catalogo, ordini, pagamenti), prima o poi sente il bisogno di riorganizzare anche le **persone** in squadre allineate a quei domini. Per un PM o uno Scrum Master, una scelta architetturale non resta quindi confinata al codice: può tradursi in nuovi ruoli, responsabilità e meccanismi di coordinamento tra team che prima erano un tutt'uno.

11.4 Come comunicano i componenti

Nel momento in cui ShopFacile è diventato un insieme di servizi separati (catalogo, ordini, pagamenti, magazzino), è emersa una domanda nuova: come fanno questi servizi a scambiarsi informazioni tra loro? Sia nel monolite (tra i suoi moduli interni) che, soprattutto, nei microservizi (tra servizi separati), i vari “pezzi” del software devono comunicare. Ci sono principalmente due modalità.

Comunicazione sincrona: API e REST

Il modo più comune è quello che hai già visto nella sezione 2: un componente fa una **richiesta** a un altro tramite un'**API**, tipicamente in stile **REST**, e resta in attesa della **risposta** prima di andare avanti — un po' come una telefonata: fai la domanda e resti in linea aspettando la risposta, prima di riprendere quello che stavi facendo.

Questo funziona benissimo per molti casi (“dammi i dati di questo cliente”, “verifica se questo pagamento è andato a buon fine”), ma ha un limite: se il servizio che deve rispondere è lento o irraggiungibile, chi ha fatto la richiesta resta bloccato ad aspettare.

Esempio pratico: il frontend di ShopFacile vuole mostrare i dettagli di un ordine al cliente. Fa una richiesta HTTP di tipo GET a un’API REST del backend, e resta in attesa della risposta:

Richiesta:

```
GET /api/ordini/42
```

Risposta (dopo pochi millisecondi):

```
{
  "id": 42,
  "cliente": "ACME S.r.l.",
  "stato": "in consegna",
  "totale": 129.90
}
```

Finché il backend non risponde, il frontend resta “in attesa” — esattamente come nell’analogia della telefonata — prima di poter mostrare questi dati a schermo.

Questo tipo di comunicazione, però, non è adatto a tutto: se il servizio Ordini di ShopFacile dovesse restare “in attesa” ogni volta che avvisa il servizio Email di inviare una conferma, un rallentamento della posta elettronica bloccherebbe anche la conferma dell’ordine. Per questi casi serve un modo diverso di comunicare: la comunicazione asincrona.

Comunicazione asincrona: le code di messaggi

Per certi casi è più comodo **non restare in attesa** di una risposta immediata, ma semplicemente “lasciare un messaggio” che qualcun altro leggerà quando potrà. È qui che entrano in gioco le **code di messaggi** (*message queue*), uno strumento tipico della comunicazione **asincrona**.

Analogia: la cassetta della posta

Pensa alla differenza tra una telefonata e una **cassetta della posta**:

- con la telefonata (comunicazione sincrona/API) chiami qualcuno e resti in attesa che risponda subito, in tempo reale;
- con una lettera lasciata nella cassetta della posta (comunicazione asincrona/coda di messaggi) **scrivi il messaggio e vai avanti con le tue cose**: non resti lì ad aspettare. Il destinatario, quando ha tempo, apre la cassetta, legge il messaggio e agisce di conseguenza. Se il destinatario in questo momento non è in casa, il messaggio resta comunque nella cassetta ad aspettarlo, senza perdersi.

Esempio pratico: quando un cliente completa un ordine su ShopFacile, il servizio Ordini non chiama direttamente e “in diretta” il servizio Email per inviare la conferma. Invece, scrive un messaggio del tipo “ordine 42 completato” in una coda; il servizio Email (quando è pronto, magari qualche secondo dopo) legge quel messaggio dalla coda e invia l’email. Se il servizio Email in quel momento è sovraccarico o temporaneamente fermo, il messaggio resta comunque in coda, e verrà elaborato appena il servizio torna disponibile — non si perde nulla.



Questo approccio è molto usato quando un evento (es. “ordine completato”) deve essere notificato a **più** servizi interessati, senza che il servizio che genera l’evento debba conoscerli tutti o aspettare che rispondano tutti.

Perché serve uno strumento dedicato invece di scrivere i messaggi in una tabella del database? Perché una coda vera e propria garantisce la **consegna** anche se il destinatario era offline, **ritenta automaticamente** se il primo tentativo fallisce, e permette a ciascun servizio di leggere al proprio ritmo, senza che un consumatore lento accumuli lavoro per gli altri. Esempi: **RabbitMQ**, **Azure Service Bus**, **Kafka**.

	Comunicazione sincrona (API/ REST)	Comunicazione asincrona (coda di messaggi)
Analogia	Telefonata	Cassetta della posta
Chi chiama resta in attesa?	Sì, della risposta	No, va avanti
Adatta a	Richieste che servono “subito” (es. leggere dati)	Notifiche/eventi che possono essere elaborati “quando capita”
Rischio se il destinatario è lento/ offline	Chi chiama resta bloccato o riceve un errore	Il messaggio resta in coda, nessuna perdita

11.5 Event-Driven Architecture: reagire invece di chiedere

Cambiamo per un momento esempio, restando nel contesto assicurativo in cui lavori davvero: quando un cliente sottoscrive una polizza, non succede solo una cosa: ne succedono sette. Va emesso il documento di polizza, va incassato il primo premio, va aggiornata l’anagrafica del cliente, va avvisato l’agente che ha curato la vendita, va alimentato il datawarehouse per le statistiche, va spedita l’email di benvenuto, e va comunicata la sottoscrizione a un ente esterno per gli obblighi di legge. Il servizio che registra la polizza le chiama tutte e sette, una dopo l’altra, come nella comunicazione sincrona vista al paragrafo precedente: **Ahmed** ci ha messo tre giorni a scrivere quelle sette chiamate, con tutta la gestione degli errori che ciascuna richiede.

Il problema arriva dopo, quando il codice è già in produzione. Se il sistema del datawarehouse è lento, il cliente resta davanti a una rotellina di caricamento per una cosa che, dal suo punto di vista, non gli serve a niente: sta solo aspettando che una statistica interna venga aggiornata. Se l'invio all'ente esterno è temporaneamente giù, la sottoscrizione **fallisce del tutto** — anche se la polizza, in realtà, è perfettamente valida e il premio è stato incassato. E ogni volta che il business chiede di aggiungere un ottavo destinatario (un nuovo report, un nuovo partner da avvisare), bisogna modificare e ritestare il servizio più critico dell'intera azienda: quello che vende le polizze.

Marco lo riassume a **Luca** in una frase che vale l'intera sottosezione: *“il problema non è quello che facciamo, è chi deve saperlo”*. Il servizio Polizze non dovrebbe essere lui a doversi ricordare di tutti e sette gli interessati, uno per uno, e a restare in attesa che ciascuno gli risponda.

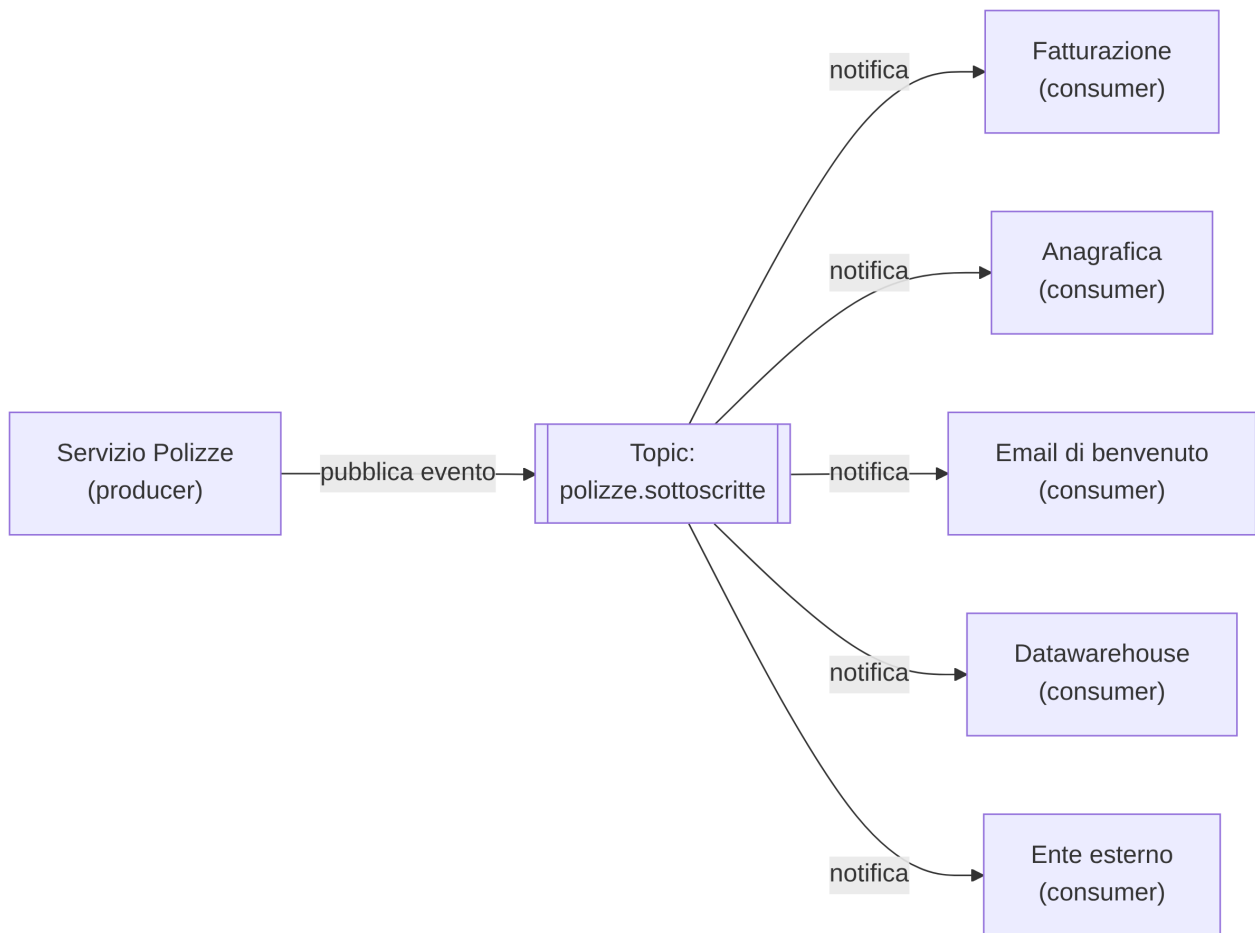
Il capovolgimento: dichiarare un fatto, non impartire ordini

L'architettura **event-driven** (guidata dagli eventi) capovolge il problema. Invece che il servizio Polizze chieda a sette sistemi di fare qualcosa, dichiara semplicemente che **è accaduto un fatto**: “polizza 88471 sottoscritta”. Chi è interessato a quel fatto **reagisce** per conto proprio, senza che il servizio Polizze debba saperlo, aspettarlo o richiamarlo.

Vale la pena fermarsi su questa distinzione, perché è il cuore di tutto il resto:

- un **comando** è un ordine diretto al futuro, verso un destinatario preciso: “Servizio Email, invia questa conferma”. Chi lo riceve può rifiutarlo o fallirlo, e chi lo manda normalmente se ne aspetta un esito.
- un **evento** è un fatto accaduto al passato, e per definizione **immutabile**: “la polizza è stata sottoscritta”. Non è indirizzato a nessuno in particolare, non richiede una risposta, e non può essere “rifiutato” — è già successo.

Chi pubblica un evento si chiama **producer** (o *publisher*); chi lo riceve e reagisce si chiama **consumer** (o *subscriber*). Il meccanismo con cui i due si incontrano si chiama **pub/sub** (*publish/subscribe*, pubblica/sottoscrivi): il producer pubblica l'evento su un canale dedicato, chiamato **topic** (“polizze.sottoscritte”), e ogni consumer interessato si sottoscrive a quel topic per riceverne una copia. A fare da intermediario tra producer e consumer c'è un **broker**: il software che riceve gli eventi pubblicati e li distribuisce a tutti i sottoscrittori. Non è un componente nuovo da imparare: l'infrastruttura è esattamente quella già vista al paragrafo 11.4 a proposito delle code di messaggi — RabbitMQ, Azure Service Bus, Kafka. Cambia il modo di usarla: non un messaggio per un singolo destinatario noto, ma un evento per un numero qualsiasi di sottoscrittori, anche futuri.



Il beneficio: disaccoppiamento, prima organizzativo che tecnico

Con questo schema, aggiungere un ottavo consumatore (un nuovo report per un nuovo partner commerciale) significa scrivere un nuovo consumer che si sottoscrive al topic già esistente — **senza toccare** il servizio Polizze, senza rimetterlo in test, e senza dover chiedere una finestra di rilascio sul sistema più critico dell'azienda. Questo si chiama **disaccoppiamento**, ed è il vantaggio principale del pattern: per un PM non è (solo) un beneficio tecnico, è un beneficio **organizzativo**. Il team che gestisce le polizze non deve più coordinarsi con ogni altro team ogni volta che qualcuno ha bisogno di sapere qualcosa in più — si aggancia qui la Legge di Conway vista al paragrafo 11.3: se i team restano indipendenti anche nel modo in cui i loro sistemi comunicano, l'architettura a eventi è spesso ciò che rende quell'indipendenza possibile davvero, invece che solo desiderata sulla carta.

💡 Sentirai talvolta due nomi più avanzati legati a questo mondo: **event sourcing** (salvare la storia di un'entità come sequenza di eventi accaduti, invece che il suo solo stato attuale) e **CQRS** (*Command Query Responsibility Segregation*, separare i percorsi di scrittura e lettura dei dati). Non ti servono a questo livello: è sufficiente riconoscere i nomi quando li senti nominare, per non confonderli con l'event-driven "di base" appena visto.

Il prezzo: cosa si perde rispetto a una catena di chiamate

Il disaccoppiamento non è gratuito. Quattro conseguenze concrete, e reali:

- **Il flusso non si legge più in un posto solo.** Nessuno può aprire il codice del servizio Polizze e vedere, in ordine, “cosa succede dopo una sottoscrizione”: succede in cinque punti diversi del sistema, ciascuno scritto da un team diverso. Per ricostruire la sequenza serve il **tracciamento distribuito** visto nella sezione 9 (le tracce che seguono una richiesta attraverso più servizi) — senza quello strumento, capire “chi ha reagito e quando” a un singolo evento richiede una caccia al tesoro tra i log di sistemi diversi.
- **L'ordine di arrivo non è garantito, e un evento può arrivare due volte.** Se la rete o il broker hanno un intoppo, un consumer può ricevere lo stesso evento due volte. Se il consumer che incassa il premio non se ne accorge, il cliente si trova il premio scalato due volte dal conto. La protezione si chiama **idempotenza**: progettare il consumer in modo che elaborare lo stesso evento due volte produca lo stesso risultato di elaborarlo una volta sola (es. controllare prima “ho già incassato questo premio?” invece di incassarlo incondizionatamente).
- **La coerenza è eventuale, non immediata.** Subito dopo la sottoscrizione, il cliente può già vedere la sua polizza attiva nell'area clienti, mentre l'app dell'agente non l'ha ancora ricevuta, perché il suo consumer non ha ancora elaborato l'evento — magari sono passati solo due secondi, ma in quei due secondi i due sistemi “raccontano” cose diverse. Questa **coerenza eventuale** (*eventual consistency*) non è un bug da correggere: è una caratteristica del pattern che il business deve **accettare consapevolmente**, decidendo per quali processi è tollerabile e per quali no.
- **Il debug richiede strumenti diversi.** Non basta più riprodurre una chiamata e guardare la risposta: serve poter ricostruire, evento per evento, chi ha pubblicato cosa e chi ha reagito come — di nuovo, observability e tracciamento distribuito, non un debugger tradizionale.

Il criterio per scegliere tra i due mondi resta quello visto al paragrafo 11.4, solo più affilato: comunicazione **sincrona** quando chi chiama ha bisogno della risposta per decidere cosa fare subito dopo (es. “il pagamento è andato a buon fine? allora mostro la conferma”); comunicazione **a eventi** quando chi pubblica sta semplicemente informando che qualcosa è successo, senza bisogno di sapere chi reagirà né come.

11.6 Frontend vs Backend

Finora abbiamo parlato di come i servizi di ShopFacile comunicano **tra loro**. Ma c'è un'altra comunicazione altrettanto importante: quella tra il sistema e la persona che lo usa, cioè il cliente che naviga il sito. Per capirla, serve distinguere due grandi parti presenti in ogni applicazione web o mobile (un sito di e-commerce come ShopFacile, un'app bancaria, un gestionale aziendale): **frontend** e **backend**.

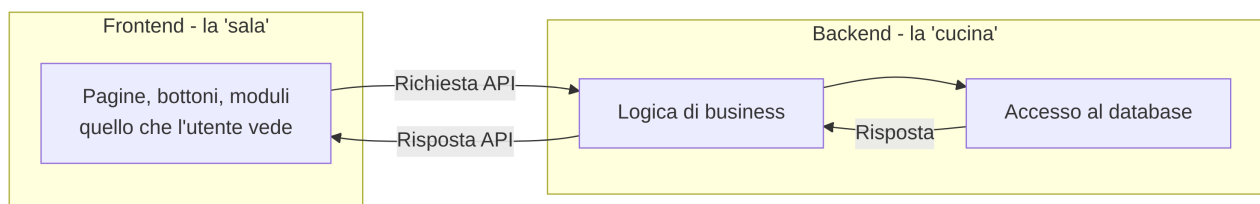
- il **frontend** è tutto ciò che l'utente vede e con cui interagisce direttamente: pagine, bottoni, moduli da compilare, grafici. È la parte che “gira” nel browser o nell'app sul telefono dell'utente;
- il **backend** è la parte che l'utente non vede: elabora la logica di business (“questo sconto si applica solo se il carrello supera 50€”), gestisce l'accesso al database, si occupa della sicurezza, e risponde alle richieste che arrivano dal frontend.

Analogia: il ristorante, sala vs cucina

Pensa a un ristorante:

- la **sala** (i tavoli, i camerieri, il menù che vedi) è il **frontend**: è quello che tu, cliente, vedi e con cui interagisci direttamente. Ordini un piatto guardando il menù, il cameriere prende nota;
- la **cucina** è il **backend**: è dove avviene il vero lavoro (cucinare il piatto, gestire le scorte, applicare la ricetta), ma tu, seduto al tavolo, non la vedi. Interagisci con essa solo indirettamente, tramite il cameriere (che, come abbiamo visto nella sezione 2, è un po' come l'API).

Un frontend curato ma con un backend debole è come un ristorante con una sala elegantissima ma una cucina che sbaglia sempre gli ordini: l'esperienza finale ne risente comunque. Vale anche il contrario: un backend eccellente con un frontend confuso farà comunque fatica a piacere ai clienti.



Esempio pratico: quando clicchi il pulsante “Conferma ordine” su ShopFacile, il **frontend** (il bottone che hai cliccato) invia una richiesta al **backend** con i dati del carrello; il backend verifica che i prodotti siano disponibili, calcola il totale applicando eventuali sconti, salva l'ordine nel database e restituisce al frontend una risposta (“ordine confermato, numero 42”), che il frontend traduce in un messaggio di conferma a schermo. Tu, da cliente, vedi solo l'ultimo passaggio: tutto il resto avviene “in cucina”.

Nel team sentirai spesso parlare di “sviluppatori frontend” e “sviluppatori backend” (o “full-stack”, chi lavora su entrambi): sono specializzazioni diverse, con linguaggi e strumenti spesso diversi, anche se lavorano sullo stesso prodotto finale.

Detto così, “frontend” e “backend” sembrano semplicemente due metà dello stesso programma. In realtà, quasi sempre, girano fisicamente in due posti diversi e comunicano attraverso una rete: è il momento di guardare a questo rapporto con il nome più tecnico che porta, il modello client-server.

11.7 Client-Server: il concetto base

Hai già incontrato questo concetto nella sezione 2 parlando di HTTP: il modello **client-server** è l'idea di base secondo cui esiste un **client** (chi fa richieste, es. il tuo browser) e un **server** (chi riceve la richiesta, la elabora e risponde, es. il computer che ospita l'applicazione).

È utile ricordarlo qui perché frontend e backend, di fatto, **spesso corrispondono** a client e server: il frontend che gira nel tuo browser è il client, il backend che gira su un server (fisico, virtuale, o un container in cloud) è il server. Non è una regola matematica assoluta — esistono architetture più sofisticate — ma per il 90% dei casi che incontrerai questa corrispondenza è un buon punto di partenza mentale.

Esempio pratico: quando apri ShopFacile sul telefono, l'app stessa (che gira sul tuo telefono) è il **client**: fa una richiesta (“dammi i dettagli del mio ultimo ordine”) a un **server** remoto di ShopFacile, che elabora la richiesta e restituisce il dato. Se chiudi l'app, il server continua a funzionare tranquillamente per tutti gli altri clienti: il client è “usa e getta”, il server resta sempre lì, pronto a rispondere a nuove richieste.



Per confronto: lo stesso schema vale, identico, per un'app di home banking: l'app sul telefono è il client, il server remoto della banca è il server. Cambia il contesto (acquisti contro conto corrente), non la struttura client-server sottostante.

Sappiamo quindi che un client parla con un server, e che dentro quel server convivono frontend e backend. Ma **dentro il backend stesso**, come sono organizzati presentazione, logica di business e dati? Un modo diffuso di rispondere a questa domanda è l'architettura a 3 livelli.

11.8 Architettura a 3 livelli

Un modo molto comune, semplice e diffuso di organizzare un'applicazione, soprattutto nei sistemi gestionali “classici”, è l'**architettura a 3 livelli** (in inglese *3-tier architecture*). Divide il software in tre strati, ciascuno con una responsabilità precisa:

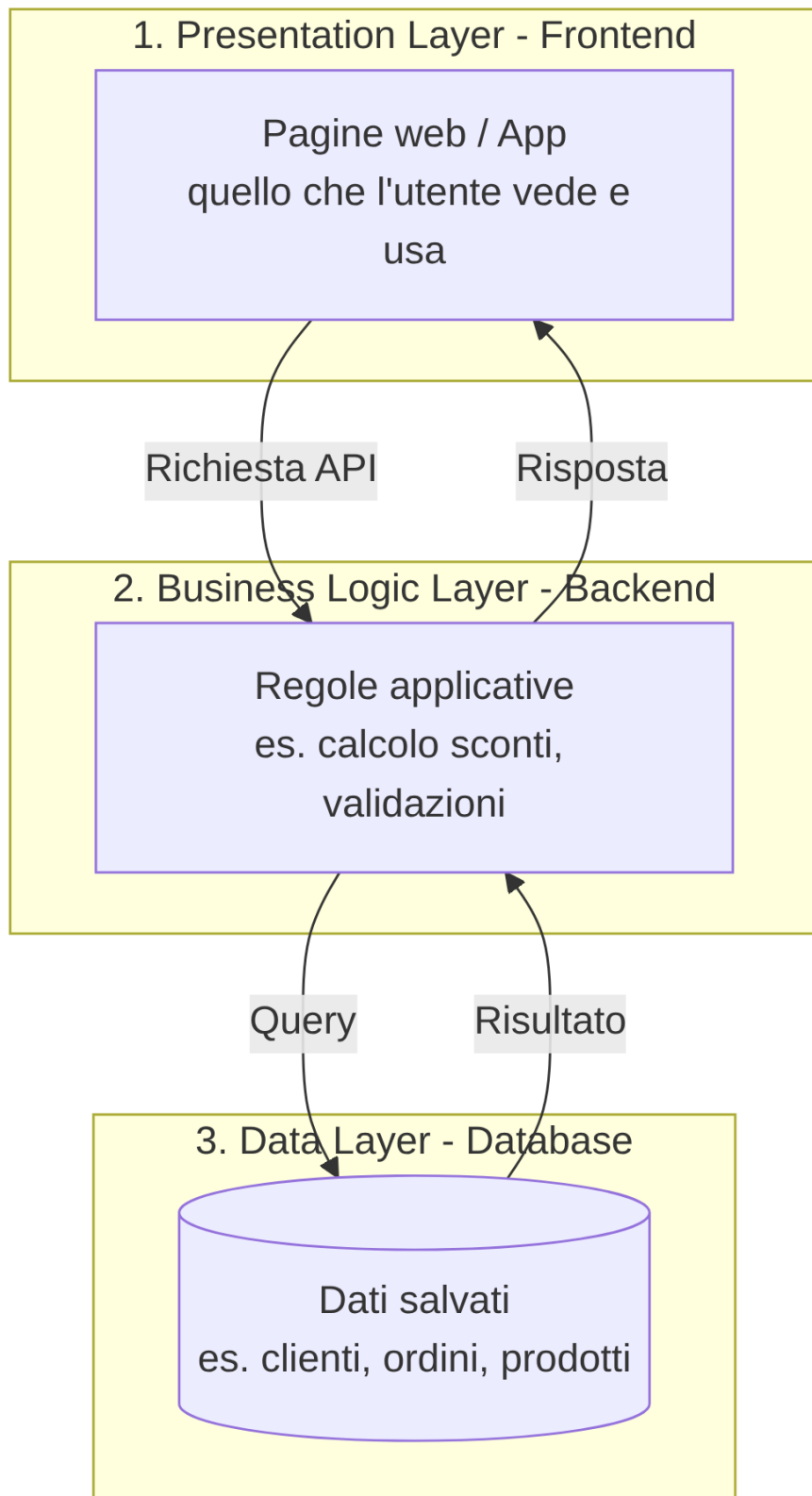
1. **Presentation layer** (livello di presentazione): è il frontend, quello che l'utente vede e usa;
2. **Business logic layer** (livello di logica di business): è il backend, dove vengono applicate le regole (“uno sconto si applica solo se...”, “un ordine non può essere spedito se non è stato pagato...”);
3. **Data layer** (livello dei dati): è il database, dove i dati vengono effettivamente salvati e recuperati.

L'idea chiave è che **ogni livello parla solo con quello adiacente**: il livello di presentazione non accede mai direttamente al database, ma passa sempre dal livello di logica di business, che fa da “filtro” e applica le regole necessarie prima di leggere o scrivere i dati.

Questa regola evita un problema concreto: se l'interfaccia accedesse direttamente al database, le regole di business (“lo sconto si applica solo se...”) finirebbero duplicate in più punti (pagina web, app mobile, un'esportazione dati) — e il giorno in cui una regola cambia, il rischio è che qualcuno aggiorni una copia e si scordi le altre, senza che nessuno sappia più **dove sta la verità**.

Analogia

Pensa di nuovo al ristorante, ma con un passaggio in più: tu (client) parli solo con il cameriere (presentation layer), il cameriere porta l'ordine allo chef (business logic layer), che decide come preparare il piatto secondo la ricetta e le regole della cucina, e solo lo chef accede alla **dispensa** (data layer) per prendere gli ingredienti. Tu non entri mai direttamente nella dispensa: passi sempre dallo chef, che sa quali regole rispettare (es. “questo piatto non si può preparare se manca un ingrediente fondamentale”).



Questo schema a 3 livelli è alla base di moltissime applicazioni aziendali “tradizionali” (gestionali, portali interni, applicazioni web classiche), e puoi ritrovarlo sia dentro un monolite (i tre livelli vivono nello stesso blocco di codice) sia distribuito su più microservizi (ogni servizio ha comunque, internamente, una sua piccola struttura a livelli) — come nel caso del servizio Ordini di ShopFacile.

Esempio pratico: nel servizio Ordini di ShopFacile, che applica uno sconto del 10% agli ordini superiori a 100€, il livello di presentazione mostra semplicemente il totale finale nel carrello; il livello di logica di business è quello che **decide** se lo sconto si applica o no, facendo il calcolo; il livello dati è quello che, a fine ordine, salva il totale scontato nel database. Se un giorno la regola cambia (es. lo sconto diventa 15%), basta modificare il livello di logica di business: la presentazione e il database non vengono toccati. Lo stesso identico schema, del resto, è alla base anche di un gestionale aziendale “classico” per l'ufficio commerciale: cambia il dominio (sconti sugli ordini contro preventivi e fatture), non la struttura a tre livelli.

Dopo monolite, microservizi, comunicazione e 3 livelli, resta un ultimo modo di organizzare il codice, agli antipodi rispetto al server “sempre acceso” che abbiamo dato per scontato finora: il serverless.

11.9 Un accenno al Serverless

Finora abbiamo parlato di software che gira su server (fisici, virtuali, o container) che devono restare **sempre attivi**, pronti a rispondere in qualsiasi momento. Il modello **serverless** (“senza server”, anche se in realtà un server c'è sempre, solo che non te ne devi occupare tu) propone un'idea diversa: scrivi piccole **funzioni** che vengono eseguite dal fornitore cloud **solo quando servono**, e vengono “spente” subito dopo.

Analogia: il taxi vs l'auto di proprietà

Gestire un server tradizionale è un po' come avere un'**auto di proprietà**: la paghi (e la mantieni) sempre, anche quando è parcheggiata e non la usi. Il serverless è più come prendere un **taxi**: lo chiami solo quando ti serve uno spostamento, paghi solo per quella corsa, e non devi preoccuparti di manutenzione, parcheggio o assicurazione quando non lo stai usando.

Esempio pratico: in ShopFacile, quando un cliente carica la foto di un prodotto reso da segnalare, una funzione serverless si attiva solo in quel momento, per ridimensionare l'immagine automaticamente in diverse dimensioni, e poi si “spegne” fino al prossimo caricamento. Non serve tenere un server acceso 24 ore su 24 in attesa che qualcuno carichi una foto — a differenza del servizio Ordini o Pagamenti, che devono restare sempre pronti a rispondere.

Vantaggi principali: si paga solo per l'uso effettivo (nessun costo per il tempo “di inattività”), e non serve gestire manualmente l'infrastruttura sottostante (il fornitore cloud se ne occupa). Lo svantaggio principale è che non è adatto a tutti i casi: per applicazioni che devono rispondere istantaneamente e in modo continuo, o che richiedono elaborazioni molto lunghe, il modello serverless può introdurre piccoli ritardi (“tempo di avvio a freddo”) o limiti di durata.

Esempi di servizi serverless che sentirai citare: **Azure Functions**, **AWS Lambda**. Ne ripareremo con più dettaglio nella sezione 12 (Cloud).

Con questo abbiamo attraversato tutte le scelte architetturali principali che il team di ShopFacile ha dovuto affrontare, da “un blocco unico o tanti servizi?” fino a “serve un server sempre acceso o basta una funzione al bisogno?”. Prima di passare al Cloud, fermiamoci un momento a riepilogare.

11.10 Riepilogo: cosa ti serve ricordare

Non devi memorizzare ogni dettaglio di questa sezione. Ti basta portarti via questi concetti chiave, che ti aiuteranno a capire meglio le conversazioni tecniche del team:

- **Monolite**: un unico blocco di codice. Semplice all'inizio, difficile da scalare selettivamente e da far crescere con tanti team.
- **Microservizi**: tanti servizi piccoli e indipendenti. Più scalabili e flessibili, ma più complessi da gestire operativamente.
- **API/REST** (comunicazione sincrona, “telefonata”) e **code di messaggi** (comunicazione asincrona, “cassetta della posta”) sono i due modi principali con cui i componenti di un software comunicano tra loro.
- **Event-Driven Architecture**: invece di chiamare direttamente chi deve fare qualcosa, un servizio pubblica un **evento** (un fatto accaduto) su un **topic**, e chi è interessato (**consumer**) reagisce in autonomia, senza che chi pubblica (**producer**) debba conoscerlo — un **disaccoppiamento** organizzativo, non solo tecnico, che ha un prezzo: coerenza eventuale, idempotenza da gestire, debug più complesso.
- **Frontend** (sala) e **Backend** (cucina): la parte visibile all'utente e quella che elabora la logica e i dati.
- **Client-Server**: chi fa la richiesta (client) e chi la elabora (server).
- **Architettura a 3 livelli**: Presentazione → Logica di business → Dati, ognuno responsabile del proprio compito.
- **Serverless**: funzioni eseguite solo quando servono, senza gestire un server sempre attivo — come un taxi rispetto a un'auto di proprietà.

Quando il team discute di “spacchettare il monolite”, “aggiungere un servizio”, “usare una coda per questo evento” o “spostare questa funzione su serverless”, ora avrai gli strumenti concettuali per seguire la conversazione e fare le domande giuste.

Checklist di autoverifica

- Sapresti spiegare la differenza tra monolite e microservizi con un'analogia tua?
 - Sai indicare almeno due vantaggi e due svantaggi dei microservizi?
 - Sapresti spiegare quando ha senso passare da monolite a microservizi?
 - Sai spiegare la differenza tra comunicazione sincrona e asincrona, con l'analogia della telefonata e della cassetta della posta?
 - Sai spiegare la differenza tra un comando e un evento, e perché è il cuore dell'architettura event-driven?
 - Sai indicare almeno due trade-off concreti dell'architettura a eventi (es. coerenza eventuale, idempotenza)?
 - Sai spiegare la differenza tra frontend e backend con l'analogia del ristorante?
 - Sai descrivere i tre livelli dell'architettura a 3 livelli?
 - Sai spiegare, a grandi linee, cos'è il serverless e quando può essere utile?
-



Esercizi pratici

1. **Disegna l'architettura di un'app che usi tutti i giorni.** Scegli un'app che conosci bene (es. un'app di food delivery, di messaggistica o di home banking) e disegna a mano uno schema con frontend, backend e database, provando a immaginare se sia più vicina a un monolite o a un sistema a microservizi.  **Come verificare:** se riesci a indicare almeno tre “pezzi” separati (es. “app sul telefono”, “server che elabora i pagamenti”, “database degli utenti”) e a spiegare a voce come comunicano tra loro, hai centrato l'esercizio.
2. **Scrivi una richiesta e una risposta API “a mano”.** Scegli una funzionalità semplice (es. “ottieni il profilo di un utente”) e scrivi tu stesso, come nell'esempio pratico di questa sezione, una richiesta GET/POST semplificata e la relativa risposta in formato JSON, con almeno 3-4 campi.  **Come verificare:** fai leggere quello che hai scritto a un developer del team e chiedi se il formato “assomiglia” a una vera richiesta/risposta API che usano nel progetto.
3. **Trasforma un'operazione sincrona in asincrona.** Pensa a un'operazione che oggi, nella tua esperienza quotidiana (non necessariamente software), richiede un'attesa “in linea” (es. una telefonata al call center) e descrivi come diventerebbe se fosse gestita con una coda di messaggi (es. lasciare un messaggio in segreteria che verrà richiamato quando c'è disponibilità).  **Come verificare:** la tua descrizione deve contenere chiaramente chi “scrive” il messaggio, dove viene “depositato” e chi lo “legge” più tardi, senza che nessuno resti bloccato ad aspettare.
4. **Distingui un comando da un evento.** Prendi tre frasi che potresti sentire in una daily standup del tuo progetto (es. “invia la mail di conferma”, “l'ordine è stato pagato”, “aggiorna il saldo del cliente”) e classifica ciascuna come comando o come evento, spiegando perché.  **Come verificare:** per ogni frase, sai indicare se è diretta a un destinatario preciso e richiede un'azione futura (comando) o descrive un fatto già accaduto che qualcuno può scegliere di ignorare o reagire (evento) — se confondi le due categorie su più di una frase, rileggi la distinzione al paragrafo 11.5.
5. **Intervista un developer sull'architettura del progetto.** Chiedi a un collega developer se il progetto su cui lavori è organizzato come monolite, come microservizi, o come una via di mezzo, e fatti indicare un esempio concreto di comunicazione tra due componenti (API, coda di messaggi o evento pub/sub).  **Come verificare:** dopo la chiacchierata, sapresti riassumere in 3-4 frasi, senza guardare appunti, “come è fatto” il software del progetto e quali pezzi lo compongono.
6. **Compila la tabella di confronto con un esempio reale.** Riprendi la tabella “Monolite vs Microservizi” della sezione 11.3 e, per ogni riga, scrivi a fianco un esempio concreto (reale o immaginario) legato al contesto del tuo progetto.  **Come verificare:** condividi la tabella compilata con la tua collega Scrum Master/PM e chiedile se gli esempi che hai scelto sono plausibili per un progetto reale.
7. **Individua un caso d'uso serverless.** Pensa a una funzionalità che, nel progetto o in un'app che usi, viene eseguita “una tantum” e su richiesta (es. generazione di un PDF, invio di una notifica, ridimensionamento di un'immagine) e spiega perché potrebbe essere un buon candidato per il

modello serverless, oppure perché no.  **Come verificare:** la tua spiegazione deve menzionare esplicitamente il criterio “si attiva solo quando serve, poi si spegne” e collegarlo al caso d'uso scelto.

Collegamenti

- [9. DevOps](#) — dove trovi observability, tracciamento distribuito e logging, gli strumenti che servono a seguire un flusso event-driven distribuito su più servizi
- [12. Cloud](#) — dove vedremo come queste architetture vengono effettivamente eseguite su infrastrutture cloud come Azure o AWS
- [13. Sicurezza](#) — dove vedremo come proteggere questi componenti e le comunicazioni tra di essi

Risorse

- [Microsoft Learn – Architetture dei microservizi](#) — guida approfondita (in italiano) su monolite vs microservizi
- [Martin Fowler – Microservices](#) — l'articolo di riferimento sul tema, uno dei più citati nel settore
- [Martin Fowler – MonolithFirst](#) — perché spesso ha senso partire da un monolite prima di passare ai microservizi
- [Microsoft Learn – Event-driven architecture style](#) — introduzione al pattern event-driven e ai suoi trade-off
- [Martin Fowler – What do you mean by Event-Driven?](#) — chiarisce le diverse sfumature del termine “event-driven”, incluso il confronto con event sourcing e CQRS
- [Microsoft Learn – Cos'è il serverless computing](#) — introduzione al modello serverless
- [Microsoft Learn – Message queue e comunicazione asincrona](#) — approfondimento sui pattern di comunicazione asincrona
- [Documentazione ufficiale RabbitMQ – Concetti base](#) — introduzione pratica alle code di messaggi

12. Cloud

Nella sezione precedente hai visto come è “fatto” un software: monolite o microservizi, frontend e backend, database, code di messaggi. Ora facciamo un passo ulteriore e ci chiediamo: **dove vive fisicamente tutto questo?** Su quali macchine viene eseguito il codice? Dove sono salvati i dati?

La risposta, oggi, nella grande maggioranza dei casi è: **nel cloud**. In questa sezione capirai cosa significa davvero questa parola (che spesso viene usata in modo vago, quasi magico), quali sono i principali modelli con cui si “affitta” il cloud, e come si chiamano i pezzi principali di Azure e AWS, i due grandi fornitori che incontrerai più spesso lavorando in ambito DevOps.

Obiettivi della sezione

Al termine di questa sezione saprai:

- spiegare cos'è il cloud computing con parole tue, senza tecnicismi;
- distinguere IaaS, PaaS e SaaS e sapere chi gestisce cosa in ciascun modello;
- riconoscere i nomi dei principali servizi Azure e i loro equivalenti AWS;
- capire cosa significano scalabilità verticale/orizzontale, pay-as-you-go, regioni e alta disponibilità;
- capire cos'è il **Platform Engineering** e perché nasce come risposta a un problema che DevOps, da solo, lascia scoperto;
- distinguere la **piattaforma DevOps** dalla **piattaforma di integrazione**, due significati diversi che condividono il nome “piattaforma”.

12.1 Cos'è il cloud computing

Immagina di dover organizzare una cena per 50 persone. Hai due opzioni:

1. Comprare pentole, forni, piatti, tavoli e sedie per 50 persone, tenerli in un magazzino tutto l'anno e usarli una volta ogni tanto.
2. Affittare una sala da un catering: paghi solo per l'evento, usi le loro attrezzature, e quando hai finito non devi più pensarci.

Per decenni, le aziende che avevano bisogno di far girare un software hanno fatto la scelta (1) — ed è esattamente quello che ha fatto per anni **ShopFacile**, la piattaforma di e-commerce (catalogo prodotti, carrello, ordini, pagamenti) che useremo come filo conduttore di questa sezione: compravano server fisici, li installavano in una stanza climatizzata (la “sala server”), pagavano tecnici per mantenerli, e se il software cresceva dovevano comprare altri server. Se il software calava di utilizzo, quei server restavano lì, comprati e pagati, a fare poco.

Il momento in cui questo problema si è sentito davvero, in ShopFacile, è stato un dicembre di qualche anno fa. Le vendite di Natale erano già partite, il traffico sul sito cresceva di giorno in giorno, e il server esistente faticava a starci dietro: le pagine si caricavano lente, il checkout a volte si bloccava per qualche secondo nei momenti di punta. Il team ha ordinato subito un server aggiuntivo — ma il fornitore

ha risposto che sarebbe arrivato **tra sei settimane**. Sei settimane dopo, i saldi di Natale erano finiti da un pezzo: il nuovo server sarebbe arrivato giusto in tempo per l'unico periodo dell'anno in cui il traffico, e quindi la capacità di calcolo in più, valeva davvero qualcosa. È esattamente il problema che la scelta (2) — pagare solo per quello che serve, quando serve — avrebbe potuto evitare.

Il **cloud computing** è la scelta (2) applicata all'informatica: invece di possedere e gestire tu i server fisici, **usi via internet i server e le risorse informatiche di qualcun altro** (un “provider cloud” come Microsoft, Amazon o Google), pagando solo per quello che usi e quando lo usi.

Analogia: possedere un data center è come costruirsi una casa da zero — devi comprare il terreno, costruire le fondamenta, tirare su i muri, allacciare le utenze. Usare il cloud è come **affittare un appartamento già arredato**: entri, usi quello che c'è, e quando ti serve una stanza in più la aggiungi (o la togli) senza dover ristrutturare un edificio.

In pratica, quando ShopFacile “mette il software sul cloud” significa che il codice, il database degli ordini e i file (immagini dei prodotti, fatture) non girano più su un server fisico di proprietà dell'azienda dentro il suo ufficio, ma su server che appartengono a un provider cloud, distribuiti in enormi data center in giro per il mondo, ai quali si accede via internet.

Questo cambia radicalmente il modo di lavorare:

- non serve comprare hardware in anticipo “per sicurezza”: si aggiungono risorse quando servono, in pochi minuti;
- non serve occuparsi di manutenzione fisica (cambiare un disco rotto, sostituire un alimentatore): è il provider che ci pensa;
- si paga in base al consumo, non un costo fisso indipendentemente dall'uso.

Esempio pratico: Marco, che in ShopFacile si occupa spesso di infrastruttura, guida la migrazione: il vecchio server fisico che ospitava il catalogo prodotti e il database degli ordini viene spento, e lo stesso software torna a girare — senza essere riscritto — su risorse affittate da un provider cloud. Il giorno dei saldi, quando il traffico su ShopFacile quadruplica, il team non deve più sperare che il server fisico regga: basta aggiungere risorse a richiesta e poi rilasciarle.

12.2 I tre modelli principali: IaaS, PaaS, SaaS

Migrare ShopFacile sul cloud non è stata una scelta “tutto o niente”: Marco ha dovuto decidere, pezzo per pezzo dell'infrastruttura, **quanta responsabilità di gestione lasciare al provider e quanta tenerla**. Non tutto il cloud è uguale: quando “affitti” qualcosa dal cloud, puoi affittare **più o meno responsabilità**. Il modo più semplice per capirlo è pensare a come si può abitare in un posto: puoi comprare un terreno e costruire tu la casa, puoi affittare un appartamento arredato, oppure puoi prenotare una stanza d'hotel con servizio completo.

Nel mondo cloud questi tre livelli si chiamano **IaaS, PaaS e SaaS**.

12.2.1 IaaS — Infrastructure as Service

IaaS significa “infrastruttura come servizio”. Il provider ti affitta l'**infrastruttura grezza**: server virtuali, spazio di archiviazione (dischi virtuali) e rete. Tutto il resto — sistema operativo, aggiornamenti, programmi installati, sicurezza a livello di software — è **compito tuo**.

Analogia: è come affittare un **terreno con già lo scheletro di un edificio** (fondamenta, struttura portante, allacci alla rete elettrica e idrica). Il proprietario ti garantisce che lo scheletro sta in piedi e che l'elettricità arriva. Ma le pareti interne, gli impianti, l'arredamento e le tinteggiature te li fai tu.

Esempio concreto: una **macchina virtuale** su Azure o AWS. Il provider ti dà un computer virtuale con una certa potenza di calcolo, ma sei tu a installare il sistema operativo, gli aggiornamenti di sicurezza, il software applicativo e a occuparti della configurazione.

Chi usa IaaS: chi ha bisogno di massimo controllo e flessibilità, o deve migrare software esistente pensato per girare su server “tradizionali” senza riscriverlo.

Esempio pratico: Marco deve spostare sul cloud il vecchio motore di generazione delle fatture di ShopFacile, pensato per girare su un server Windows in ufficio, senza riscriverlo. La soluzione è creare una macchina virtuale IaaS: si installa lo stesso sistema operativo, si copiano gli stessi file, si configurano le stesse porte di rete. Non cambia una riga di codice. Il vantaggio non è “meno lavoro da fare” ma “niente hardware da comprare e mantenere”: se la macchina virtuale si rivela sottodimensionata durante un picco di ordini, in pochi minuti Marco ne aumenta la potenza (CPU, RAM) dal pannello di controllo del provider, invece di aspettare settimane per un nuovo server fisico.

12.2.2 PaaS — Platform as Service

Se con la fattura Marco ha scelto di portarsi dietro anche il sistema operativo da gestire, per il catalogo prodotti di ShopFacile — la parte dell'applicazione che cambia più spesso — il team preferisce liberarsi anche di quel livello: entra in scena il PaaS.

PaaS significa “piattaforma come servizio”. Il provider ti affitta anche **la piattaforma** su cui far girare il software: sistema operativo, ambiente di esecuzione (runtime), database già pronti e gestiti. Tu ti devi occupare solo di scrivere e distribuire il tuo codice.

Analogia: è l'**appartamento già arredato**. Non devi costruire i muri né comprare i mobili: trovi tutto pronto, incluse le utenze allacciate. Ti devi solo preoccupare di viverci, cioè — nel caso del software — di scrivere e pubblicare il codice della tua applicazione.

Esempio concreto: **Azure App Service**, dove carichi il codice della tua applicazione web e il provider si occupa di far girare il sistema operativo, il runtime (per esempio .NET o Node.js), gli aggiornamenti di sicurezza e la scalabilità di base. Oppure un database gestito come **Azure SQL Database**, dove non devi installare né amministrare tu il motore del database: esiste già, pronto all'uso, e il provider si occupa di backup, patch e affidabilità.

Chi usa PaaS: la maggior parte dei team che sviluppano applicazioni moderne, perché permette di concentrarsi sul codice senza perdere tempo a gestire server e sistemi operativi.

Esempio pratico: Ahmed finisce di scrivere una nuova funzionalità per il catalogo prodotti di ShopFacile (i filtri di ricerca per categoria). Con un servizio PaaS, pubblica il codice (spesso con un comando o con la pipeline vista nella sezione 10) e in pochi minuti la nuova versione è online, raggiungibile via browser, senza che nessuno debba configurare un sistema operativo, aprire porte di rete o installare un runtime a mano. In parallelo, i dati del catalogo vivono in un database gestito: nessuno del team deve occuparsi di installare il motore del database, programmare i backup notturni o applicare le patch di sicurezza — il provider lo fa in automatico, e se qualcosa va storto è possibile ripristinare un backup recente con pochi clic.

12.2.3 SaaS — Software as Service

Con IaaS e PaaS, il team di ShopFacile ha comunque scritto e distribuito codice proprio. C'è però un terzo livello, ancora più delegato, che il team usa ogni giorno senza nemmeno pensarci: quello del software già pronto.

SaaS significa “software come servizio”. Qui non affitti infrastruttura né piattaforma: **usi direttamente un software già finito**, di solito via browser, senza installare nulla e senza sapere (né doverti preoccupare di) dove e come girano i server dietro.

Analogia: è l'**hotel a servizio completo**. Non ti preoccupi di struttura, arredamento, pulizie o lenzuola: prenoti la stanza, entri, e tutto è già pronto per essere usato. Il tuo unico compito è vivere la tua vacanza (cioè usare il software per il tuo lavoro).

Esempi concreti che usi già ogni giorno: **Gmail** o **Outlook** (posta elettronica), **Office 365** o **Google Workspace** (documenti e foglio di calcolo), **Salesforce** (gestione clienti), lo stesso **GitHub** che usi per il codice e le pipeline del progetto, o **Slack/Teams** per la comunicazione. In tutti questi casi apri un browser, accedi con le tue credenziali, e il software è già lì, funzionante, aggiornato, senza che tu debba installare o gestire nulla.

Chi usa SaaS: chiunque abbia bisogno di una funzionalità (email, CRM, gestione progetti) senza voler sviluppare o mantenere il software che la fornisce.

Esempio pratico: il team di ShopFacile ha bisogno di gestire il backlog, le pipeline e i repository del progetto. Nessuno “installa” GitHub su un server: Sara e Luca si registrano, aprono il browser, accedono con le proprie credenziali e il servizio è già lì, pronto all'uso e aggiornato dal provider senza che il team debba fare nulla. Lo stesso vale per la posta elettronica aziendale o per lo strumento di videochiamata usato nei daily: sono tutti software “già finiti”, usati così come sono, senza che qualcuno in ShopFacile ne debba gestire l'infrastruttura sottostante.

12.3 Confronto: chi gestisce cosa

Ora che Marco ha usato tutti e tre i livelli su pezzi diversi dell'infrastruttura di ShopFacile, conviene vedere in un colpo d'occhio come si posizionano l'uno rispetto all'altro. La differenza chiave tra IaaS, PaaS e SaaS è **quanta responsabilità di gestione resta al cliente e quanta passa al provider cloud**. Più si sale da IaaS verso SaaS, meno cose deve gestire il cliente (ma meno controllo ha).

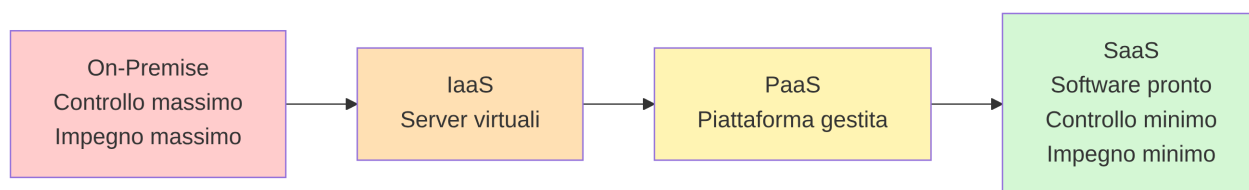
La tabella seguente è il modo più chiaro per vedere la “torta delle responsabilità”: ogni riga è uno strato tecnico, e la cella indica **chi** se ne occupa.

Livello / Strato	On-Premise (server proprio)	IaaS	PaaS	SaaS
Applicazioni	 Cliente	 Cliente	 Cliente	 Provider
Dati	 Cliente	 Cliente	 Cliente	 Cliente*
Runtime (linguaggio, librerie)	 Cliente	 Cliente	 Provider	 Provider
Middleware	 Cliente	 Cliente	 Provider	 Provider
Sistema operativo	 Cliente	 Cliente	 Provider	 Provider
Virtualizzazione	 Cliente	 Provider	 Provider	 Provider
Server (hardware fisico)	 Cliente	 Provider	 Provider	 Provider
Storage (fisico)	 Cliente	 Provider	 Provider	 Provider
Rete (fisica)	 Cliente	 Provider	 Provider	 Provider

** anche in SaaS i dati che inserisci nel software (es. le tue email, i tuoi documenti) restano “tuo”: il provider li custodisce e li protegge, ma il contenuto è di tua responsabilità (es. non condividere per errore un file sensibile).*

Questo “meno controllo diretto” ha un rovescio concreto. Se un servizio PaaS o SaaS ha un malfunzionamento, o cambia comportamento con un aggiornamento non richiesto, il team di ShopFacile **non può intervenire direttamente**: può solo aprire una segnalazione al provider, aspettare che risolva dal suo lato, e comunicare la situazione al cliente. È il prezzo di non doversi occupare della gestione quotidiana.

In sintesi, salendo da IaaS a SaaS **deleghi progressivamente più lavoro operativo al provider**, in cambio di **meno controllo diretto** su come le cose sono configurate. Non esiste un livello “migliore” in assoluto: la scelta dipende da quanto controllo serve rispetto a quanta comodità si vuole.



12.4 Azure: panoramica generale

Visti i tre modelli in astratto, è utile sapere anche su quale provider concreto ShopFacile li ha effettivamente implementati. **Microsoft Azure** è la piattaforma cloud di Microsoft, ed è quella che incontrerai più spesso in questo corso e nel lavoro quotidiano, dato che il team vi ha migrato la propria infrastruttura. Azure offre centinaia di servizi; qui vediamo solo i concetti principali, giusto per riconoscerli quando li sentirai nominare.

- **Macchine virtuali (Virtual Machines)** — il servizio IaaS di base: un server virtuale su cui installare qualsiasi sistema operativo e software, esattamente come faresti su un computer fisico.
- **Azure App Service** — il servizio PaaS per pubblicare applicazioni web e API senza gestire server o sistemi operativi: carichi il codice, Azure fa girare tutto il resto.
- **Database gestiti** (es. **Azure SQL Database**, **Cosmos DB**) — database pronti all'uso, con backup automatici, aggiornamenti e scalabilità gestiti dal provider: non devi installare né amministrare tu il motore del database.
- **Storage** (es. **Azure Blob Storage**) — spazio per salvare file, backup, immagini, documenti: una specie di “grande armadio” accessibile via internet, pensato per grandi quantità di dati.
- **Azure Kubernetes Service (AKS)** — servizio gestito per eseguire container orchestrati con Kubernetes (concetto già visto nella sezione 2), senza dover installare e configurare Kubernetes da zero.

Non serve memorizzare questi nomi: basta sapere che esistono e a cosa servono a grandi linee, così quando il team dice “l’abbiamo messo su App Service” o “i dati sono su Blob Storage” saprai di cosa si sta parlando.

12.5 AWS: la principale alternativa

ShopFacile ha scelto Azure, ma non è l'unico provider possibile: se un giorno il progetto dovesse integrarsi con un partner che usa un altro fornitore, o valutare un cambio, è utile riconoscere subito i nomi equivalenti. **Amazon Web Services (AWS)** è il principale concorrente di Azure (ed è storicamente il pioniere del cloud computing su larga scala). Alcuni progetti o clienti usano AWS invece di (o insieme ad) Azure, quindi è utile riconoscere i nomi equivalenti:

Concetto	Azure	AWS
Macchine virtuali (IaaS)	Virtual Machines	EC2
Piattaforma per applicazioni web (PaaS)	App Service	Elastic Beanstalk
Storage di file/oggetti	Blob Storage	S3
Database relazionale gestito	Azure SQL Database	RDS
Container orchestrati	AKS	EKS
Funzioni serverless	Azure Functions	Lambda

Il concetto importante da portarti via non è il nome preciso di ogni servizio, ma il fatto che **i grandi provider cloud offrono tutti gli stessi tipi di servizio**, solo con nomi diversi. Una volta capito il concetto (es. “storage per file”), riconoscere l'equivalente su un altro provider è semplice.

C'è però un motivo per cui “cambiare provider” è più difficile di quanto sembri: si chiama **lock-in**. Più un progetto usa servizi **specifici** di un provider (non solo una macchina virtuale generica, ma funzionalità disponibili solo su Azure), più costa — in tempo, denaro, rischio — migrare altrove in futuro. Non è un divieto: è un costo da metter sul tavolo consapevolmente prima di scegliere, non da scoprire dopo. Alcune aziende usano **più provider contemporaneamente** (multi-cloud) per ridurre questa dipendenza, ma il prezzo è una complessità operativa che si moltiplica.

12.6 Scalabilità: verticale vs orizzontale

Che si scelga Azure o AWS, il motivo per cui Marco ha spinto per il cloud fin dall'inizio ha un nome preciso: la scalabilità. Uno dei grandi vantaggi del cloud è la **scalabilità**: la capacità di adattare le risorse informatiche disponibili in base a quanto ne serve in un dato momento, in modo rapido.

Esistono due modi per scalare:

- **Scalabilità verticale** (“scale up”): dare **più risorse alla stessa macchina** (più potenza di calcolo, più memoria). È come sostituire il motore di un'auto con uno più potente: la stessa auto va più veloce, ma c'è un limite a quanto puoi potenziarla.
- **Scalabilità orizzontale** (“scale out”): aggiungere **più macchine** che lavorano in parallelo, distribuendo il carico tra loro. È come aggiungere più corsie a un'autostrada: non rendi ogni auto più veloce, ma fai passare più traffico complessivamente.



Nel cloud, la scalabilità orizzontale è particolarmente potente perché si possono aggiungere (o togliere) macchine in pochi minuti, spesso in modo **automatico** in base al traffico reale — esattamente il caso di ShopFacile durante i saldi, con più macchine durante il picco e meno durante la notte. Questo sarebbe quasi impossibile con server fisici di proprietà, dove comprare un nuovo server richiede settimane.

12.7 Pay-as-you-go: paghi solo quello che usi

Poter scalare su e giù in pochi minuti sarebbe un vantaggio a metà se si continuasse comunque a pagare come prima, a prescindere dall'uso: il pezzo che completa il quadro è il modo in cui il cloud fa pagare quelle risorse. Il modello di pagamento tipico del cloud si chiama **pay-as-you-go** ("paga in base a quanto usi"), ed è probabilmente il cambiamento più concreto rispetto al vecchio modo di gestire l'infrastruttura.

Analogia: è come la bolletta della luce o dell'acqua: non paghi una cifra fissa indipendentemente da quanto consumi, ma in base ai consumi effettivi (kWh usati, litri consumati). Se un mese consumi meno, paghi meno.

Nel cloud questo significa, per esempio, che se hai una macchina virtuale attiva 3 ore al giorno, paghi solo per quelle 3 ore (e non per le 24). Se il sito di ShopFacile ha un picco di traffico per una settimana di saldi e poi torna normale, il team può scalare su e poi tornare giù, pagando solo per il periodo di picco.

Questo modello è molto diverso dall'acquisto di un server fisico, dove il costo è quasi tutto "anticipato" (compri il server, che poi resta tuo per anni, usato o no).

Per confronto: il pay-as-you-go non è automaticamente più economico — dipende da quanto il carico **varia**. Con un carico costante e prevedibile tutto l'anno, senza picchi, una macchina di proprietà usata al massimo può costare **meno**, nel tempo, di una macchina cloud sempre accesa e pagata a consumo. Il cloud conviene davvero quando il carico **varia** (scali su e giù pagando solo i picchi, come nei saldi di ShopFacile) o quando serve **partire subito** senza investimento iniziale. È per questo che molte aziende adottano una scelta **ibrida**: carichi stabili su infrastruttura propria, carichi variabili sul cloud.

Il costo cloud come voce di budget da monitorare

Un costo che **varia** in base all'uso è più difficile da controllare di un costo fisso. Con un budget di progetto “classico” (sezione 8) la cifra è nota in anticipo; con il cloud, la fattura di un mese può crescere silenziosamente — una macchina dimenticata accesa, un picco non previsto — senza che nessuno se ne accorga finché non arriva il conto. Per questo un team maturo imposta **allarmi di spesa e attribuisce i costi ai progetti giusti**. La disciplina che si occupa di gestire e ottimizzare la spesa cloud si chiama **FinOps** (Finance + Operations): non serve diventarne esperti, ma sapere che esiste spiega perché il costo cloud compare come voce di controllo ricorrente al pari degli altri indicatori.

12.8 Regioni, data center e alta disponibilità

Pagare solo per quello che si usa presuppone che ci sia sempre qualcosa da usare: se ShopFacile fosse ospitata in un solo data center e quello avesse un guasto, tutto il pay-as-you-go del mondo non servirebbe a nulla mentre il sito è offline. È qui che entrano in gioco regioni e data center multipli. I provider cloud non hanno un solo, enorme data center: ne hanno **decine**, sparsi in tutto il mondo, organizzati in **regioni** (per esempio “Europa occidentale”, “Stati Uniti orientali”, “Asia sud-orientale”). Ogni regione contiene a sua volta più data center fisicamente separati.

Questo ha due vantaggi principali:

- **Vicinanza geografica:** puoi far girare il tuo software in una regione vicina ai tuoi utenti, così le pagine si caricano più velocemente (meno distanza = meno tempo di viaggio dei dati) — per questo ShopFacile, che vende soprattutto in Italia, gira su una regione Azure in Europa occidentale.
- **Alta disponibilità:** se un data center ha un problema (es. un guasto elettrico), il software può continuare a funzionare perché è distribuito su più data center o più regioni. È come avere due farmacie in città invece di una sola: se una chiude per un imprevisto, l'altra resta aperta e il servizio continua.

Non serve approfondire i dettagli tecnici dell'alta disponibilità in questa fase: basta sapere che è uno dei motivi per cui le grandi aziende scelgono il cloud invece di un singolo server fisico in un singolo posto — meno rischio di interruzioni del servizio.

12.9 Platform Engineering: rendere semplice la strada giusta

La prima volta che **Ahmed** ha dovuto mettere in produzione un microservizio tutto suo, è andata così. Nessuno gli nega nulla: semplicemente, deve capire da solo come si fa. Apre un ticket per farsi creare l'ambiente di test, e aspetta. Chiede a **Giulia** il file della pipeline di un altro servizio, “tanto è uguale a quello che serve a te” — scopre dopo un giorno che non era uguale: usava un registro artefatti diverso, con permessi diversi. Scopre al terzo giorno che per pubblicare l'immagine del suo servizio sul registry serve un'autorizzazione che nessuno gli ha detto di richiedere. Chiede a **Marco**, che ha attraversato lo stesso percorso tempo prima: Marco si ricorda che c'era un problema simile, ma non ricorda più come

l'aveva risolto, e deve rimettersi a cercare tra i suoi vecchi commit. **Due settimane e mezzo** dopo il primo commit, la prima riga di codice di Ahmed arriva finalmente in produzione — non per un problema nel suo codice, che era pronto dal secondo giorno, ma per tutto quello che stava intorno.

Il rovescio della medaglia si vede dal lato di chi mantiene, non di chi arriva. In ShopFacile, col tempo, sono nati sette team diversi, e ciascuno ha risolto **lo stesso problema** di Ahmed a modo suo: sette pipeline diverse, quattro modi diversi di gestire i segreti (password, chiavi API), tre modi diversi di scrivere i log. Il giorno in cui esce una vulnerabilità grave in una libreria usata quasi ovunque, la domanda “in quanti posti dobbiamo intervenire?” non ha una risposta rapida — bisogna chiederlo, team per team, e sperare che nessuno risponda in ritardo.

Luca vede lo stesso problema da una terza angolazione, quella delle stime: ogni volta che un team stima un nuovo servizio, dentro la stima c'è un margine per “il tempo che serve per mettere in piedi tutto il resto” — ambiente, pipeline, permessi — e quel margine varia enormemente da team a team, senza che nessuno sappia spiegare bene perché uno ci metta tre giorni e un altro tre settimane.

Il punto di svolta è culturale, prima che tecnico. DevOps (sezione 9) ha detto: **ogni team è responsabile del proprio software fino alla produzione**. Preso alla lettera, questo significa che ogni team deve diventare esperto di Kubernetes, di reti, di sicurezza, di observability — tutto quello che hai visto nelle sezioni precedenti, moltiplicato per sette team che lo imparano ciascuno per la propria strada. **Non è realistico, e quando DevOps è nato nessuno l'aveva detto esplicitamente.** Il **Platform Engineering** nasce esattamente da questa contraddizione: non è una moda del momento, è la risposta a un debito che il modello DevOps, preso alla lettera, ha lasciato scoperto.

Cos'è il Platform Engineering

Il **Platform Engineering** è la disciplina di costruire e mantenere una **piattaforma interna** per gli sviluppatori — non un decreto, un **prodotto**: con dei clienti (gli sviluppatori dei vari team) che possono, in linea di principio, scegliere di non usarla. Questo è il criterio che distingue davvero una piattaforma da un ufficio che impone strumenti: **se l'adozione è obbligatoria per decreto, non hai una piattaforma, hai una nuova burocrazia** con un nome più moderno.

A occuparsene è un **team di piattaforma**: un team come gli altri, con il proprio backlog, che però non consegna funzionalità per il cliente finale di ShopFacile, ma funzionalità per gli **altri team di sviluppo**. Il suo successo si misura con indicatori concreti, non con “quanti strumenti abbiamo costruito”: quanto tempo passa dal codice scritto alla produzione, quanto tempo serve per creare un nuovo servizio da zero, e — l'indicatore più onesto di tutti — **quale percentuale di team usa la piattaforma per scelta**, non per obbligo.

Internal Developer Platform (IDP): il self-service sopra l'infrastruttura

L'**Internal Developer Platform (IDP)** è ciò che il team di piattaforma costruisce concretamente: il livello di **self-service** che si mette sopra l'infrastruttura (il cloud visto in questa sezione, Kubernetes visto nella sezione 2, le pipeline della sezione 10), così che un team di sviluppo possa ottenerne i pezzi che gli servono senza passare ogni volta da zero. Il syllabus del settore individua tre pilastri:

- **Standardizzazione:** gli stessi mattoni per tutti — un unico modo di gestire i segreti, un unico formato di log, una struttura comune per le pipeline. Non impedisce le eccezioni, ma fa in modo che l'eccezione sia una scelta consapevole, non l'esito di sette invenzioni indipendenti.
- **Automazione:** creare un ambiente, un database, una pipeline non richiede più una sequenza di passaggi manuali fatti a mano da un ingegnere esperto, ma un processo automatico, ripetibile, uguale ogni volta.
- **Developer Experience:** quanto è facile, per chi scrive codice, capire cosa fare e farlo — documentazione chiara, messaggi d'errore utili, strumenti che non richiedono di leggere un manuale di cinquanta pagine prima di poterli usare.

Due parole del vocabolario tornano continuamente parlando di IDP. Il **golden path** è il percorso raccomandato e già pronto, che copre l'80% dei casi tipici — non l'unico percorso possibile: un team con esigenze particolari può ancora uscirne, ma deve saperlo e deciderlo, non scoprirlo per caso mesi dopo. Il **self-service** è la possibilità di ottenere un ambiente, un database o una pipeline **senza aprire un ticket e senza aspettare una persona**: un modulo, un comando, un catalogo di opzioni pre-approvate, non un'email a chi "se ne occupa".

Torniamo ad Ahmed, con una piattaforma di questo tipo in funzione. Apre il catalogo interno, sceglie il modello "microservizio REST standard", il sistema gli crea ambiente, pipeline e permessi già configurati secondo il golden path. La sua prima riga di codice arriva in produzione **in mezza giornata**, non in due settimane e mezzo. Ma vale la pena essere onesti su cosa questo **non** risolve: se Ahmed ha bisogno di qualcosa fuori dal golden path (un tipo di database che nessun altro usa, un requisito di sicurezza particolare), la piattaforma non lo aiuta più di prima — anzi, può disorientarlo, perché si aspetta che "funzioni tutto in automatico" anche dove il golden path semplicemente non arriva. E la piattaforma non rende Ahmed esperto di quello che sta usando: gli permette di non doverlo essere subito, il che è diverso.

Un prodotto come **OpenShift**, che hai visto nella sezione 2 come distribuzione di Kubernetes con molti pezzi già integrati, è un **mattone** su cui si costruisce una piattaforma interna — non è già la piattaforma, perché la piattaforma include anche le convenzioni, i modelli di partenza e il supporto alle persone che OpenShift da solo non fornisce.

Trade-off: una piattaforma non è mai gratis

Come ogni soluzione vista in questo corso, anche il Platform Engineering ha un costo, e vale la pena elencarlo con la stessa onestà con cui abbiamo raccontato il beneficio:

- È un **investimento che non produce valore visibile per il cliente finale nel breve periodo**: il team di piattaforma non consegna una funzionalità che il cliente di ShopFacile vede, consegna capacità agli altri team — un beneficio che si vede a distanza di mesi, non di sprint.

- **Sotto una certa dimensione — indicativamente meno di 3-4 team — il costo di costruire e mantenere una piattaforma supera quello che rende:** gli stessi problemi si possono risolvere con convenzioni condivise e poca disciplina, senza bisogno di un team dedicato.
- La piattaforma diventa un **collo di bottiglia** se il team di piattaforma resta l'unico che può fare certe cose: si torna, con un nome diverso, al vecchio ticket verso l'infrastruttura che il Platform Engineering voleva eliminare.
- Il **golden path invecchia**: se nessuno lo mantiene, diventa la strada che tutti aggirano perché non copre più i casi reali — e un golden path aggirato da tutti è peggio che non averlo mai avuto, perché nasconde la frammentazione invece di prevenirla.
- Il caso peggiore, quello più comune da vedere davvero: una piattaforma costruita *per* gli sviluppatori ma *senza* gli sviluppatori — decisa a tavolino dal team di piattaforma, senza ascoltare chi la dovrebbe usare ogni giorno — che finisce per non essere adottata da nessuno, con il budget e il tempo già spesi.

💡 Il concetto di “platform team” è documentato in modo approfondito in **Team Topologies** di Matthew Skelton e Manuel Pais, che lo descrive come uno dei quattro tipi fondamentali di team in un'organizzazione tecnologica; i report **DORA** (DevOps Research and Assessment) discutono da anni la relazione tra piattaforme interne, produttività degli sviluppatori e velocità di rilascio.

12.10 DevOps Platform e Integration Platform: due mondi che si confondono

Parlando di “piattaforma” in un contesto enterprise, il syllabus distingue due cose che condividono il nome ma risolvono problemi diversi, e vale la pena non confonderle.

La **piattaforma DevOps** è quella che hai già incontrato pezzo per pezzo in questo corso: la catena di strumenti che porta il codice dallo sviluppo alla produzione — repository (sezione 4), pipeline (sezione 10), artifact e registry, ambienti (sezione 14). Risolve il problema di “come faccio arrivare il mio codice in produzione in modo ripetibile”.

La **piattaforma di integrazione** risolve un problema diverso: far parlare tra loro **sistemi che non sono nati per parlarsi**. Include componenti come l'**API gateway** e l'**API management** (il punto unico da cui passano le chiamate verso i servizi, con autenticazione e limiti di traffico centralizzati), il **middleware**, a volte ancora chiamato **ESB** (Enterprise Service Bus, il “bus” software su cui i sistemi si scambiano messaggi), le **code e gli stream di eventi** — lo stesso meccanismo pub/sub visto nella sezione 11 a proposito dell'Event-Driven Architecture, qui applicato non tra microservizi dello stesso sistema ma tra piattaforme ed epoche diverse — e i **connettori verso i sistemi legacy**, come il mainframe.

In un'assicurazione questo pesa più che altrove: i sistemi che tengono le polizze hanno spesso **trent'anni**, funzionano, gestiscono dati critici, e **non si riscrivono** — riscriverli da zero sarebbe un progetto pluriennale a rischio altissimo, per un beneficio incerto. La piattaforma di integrazione è ciò che permette al software nuovo (un sito ShopFacile, una app, un servizio moderno a microservizi) di **usare** quel sistema vecchio, leggendo e scrivendo dati, **senza doverlo toccare**.

	Piattaforma DevOps	Piattaforma di integrazione
Problema che risolve	Portare il codice dallo sviluppo alla produzione in modo ripetibile	Far comunicare sistemi diversi, spesso di età ed epoche diverse
Chi la usa	Sviluppatori, team di piattaforma, chi rilascia	Team applicativi che devono leggere/scrivere dati su altri sistemi, incluso il legacy
Esempi di componenti	Repository, pipeline CI/CD, artifact, container registry, ambienti	API gateway, API management, middleware/ESB, code/stream di eventi, connettori legacy/mainframe
Domanda tipica a cui risponde	“Come faccio arrivare questo codice in produzione?”	“Come faccio leggere il dato di una polizza che vive su un sistema di trent’anni fa?”

Le due piattaforme, nella pratica di un’azienda enterprise come quella assicurativa di questo corso, convivono e si sovrappongono: una pipeline DevOps può rilasciare un servizio che, una volta in produzione, comunica con il resto del mondo passando esattamente dalla piattaforma di integrazione. Non sono la stessa cosa, ma il software moderno di ShopFacile ha bisogno di entrambe per funzionare davvero in un contesto con sistemi vecchi e nuovi che devono convivere.

Con questo, il percorso di ShopFacile dal server in ufficio a un’infrastruttura cloud distribuita su più data center, con le sue piattaforme interne e di integrazione, è completo: riassumiamo i concetti principali visti lungo la strada.

12.11 Riepilogo: cosa ti serve ricordare

- **Cloud computing:** usare risorse informatiche (server, storage, rete) di qualcun altro via internet, invece di possedere e gestire hardware proprio — come affittare un appartamento invece di costruirsi una casa.
- **IaaS:** affitti l’infrastruttura grezza (server virtuali, storage, rete); gestisci tu sistema operativo e software.
- **PaaS:** affitti anche la piattaforma (OS, runtime, database gestiti); ti concentri solo sul codice.
- **SaaS:** usi un software finito via browser (Gmail, Office 365, Salesforce); non gestisci nulla sotto.
- Più sali da IaaS a SaaS, meno responsabilità di gestione hai (e meno controllo).
- **Azure** (Microsoft) e **AWS** (Amazon) sono i due principali provider cloud: offrono gli stessi tipi di servizi con nomi diversi (es. Virtual Machines/EC2, App Service/Elastic Beanstalk, Blob Storage/S3).
- **Scalabilità verticale:** più risorse alla stessa macchina. **Scalabilità orizzontale:** più macchine in parallelo.
- **Pay-as-you-go:** paghi in base al consumo reale, come una bolletta.
- **Regioni e data center:** i provider distribuiscono le loro infrastrutture in tutto il mondo, garantendo vicinanza agli utenti e **alta disponibilità** (continuità del servizio anche in caso di guasti locali).

- **Platform Engineering**: costruire e mantenere una **piattaforma interna** come un prodotto, con un **team di piattaforma** e un'**Internal Developer Platform (IDP)** basata su **standardizzazione**, **automazione** e **Developer Experience**, con un **golden path** raccomandato e un accesso **self-service** senza ticket — ma solo se il team di piattaforma resta al servizio degli sviluppatori, non il contrario.
- **Piattaforma DevOps vs piattaforma di integrazione**: la prima porta il codice dallo sviluppo alla produzione; la seconda fa parlare tra loro sistemi diversi (API gateway, middleware/ESB, code di eventi, connettori legacy) — cruciale in un contesto enterprise con sistemi vecchi che non si riscrivono.

Checklist di autoverifica

- Sapresti spiegare cos'è il cloud computing con un'analogia tua?
- Sai distinguere IaaS, PaaS e SaaS e dire, per ciascuno, cosa gestisce il cliente e cosa il provider?
- Sapresti nominare almeno due servizi Azure e i loro equivalenti AWS?
- Sai spiegare la differenza tra scalabilità verticale e orizzontale?
- Sai spiegare cosa significa "pay-as-you-go" con un esempio concreto?
- Sai spiegare perché avere più regioni/data center aiuta l'alta disponibilità?
- Sai spiegare la differenza tra Platform Engineering e un semplice "ufficio che impone strumenti"?
- Sai indicare almeno due componenti di una piattaforma di integrazione e perché servono di più in un contesto enterprise che in una startup?

Esercizi pratici

1. **Mappa il progetto sul modello IaaS/PaaS/SaaS**. Chiedi a un collega developer o operations quali servizi cloud usa il progetto e prova a classificarli come IaaS, PaaS o SaaS (es. "usiamo una macchina virtuale per X" è IaaS, "il sito web gira su un servizio di hosting gestito" è PaaS). Scrivi la lista su un foglio o in un documento.  **Come verificare**: mostra la tua classificazione alla tua collega Scrum Master/PM o al developer che hai intervistato e fatti confermare se hai classificato correttamente almeno 3 servizi su 4.
2. **Costruisci la tua "torta delle responsabilità"**. Senza guardare la tabella della sezione 12.3, ridisegna a mano su un foglio le righe Applicazioni/Dati/Runtime/Sistema operativo/Server e prova a compilare le colonne On-Premise, IaaS, PaaS, SaaS indicando chi gestisce cosa.  **Come verificare**: confronta il tuo schema con la tabella originale nella sezione 12.3; se hai più di due celle diverse, rileggi la sezione 12.3 prima di andare avanti.
3. **Traduci Azure in AWS (e viceversa)**. Prendi 4 servizi Azure a caso tra quelli citati nella sezione 12.4 (es. Virtual Machines, App Service, Blob Storage, AKS) e scrivi a memoria il loro equivalente AWS, senza guardare la tabella della sezione 12.5. Poi controlla.  **Come verificare**: hai indovinato almeno 3 corrispondenze su 4 senza guardare la tabella.

4. **Scalabilità verticale o orizzontale?** Pensa a tre scenari reali (es. “il sito del progetto ha un picco di traffico durante una campagna marketing”, “un singolo processo di calcolo è troppo lento”, “un servizio deve restare disponibile anche se una macchina si guasta”) e per ciascuno decidi se serve scalabilità verticale, orizzontale, o entrambe, motivando la scelta a voce alta o per scritto.  **Come verificare:** chiedi a un collega tecnico di ascoltare/leggere le tue tre risposte e dirti se il ragionamento (non necessariamente la soluzione tecnica esatta) è sensato.
5. **Calcola un pay-as-you-go semplificato.** Immagina una macchina virtuale che costa 0,10 € all'ora. Calcola quanto costerebbe tenerla attiva 24 ore su 24 per un mese, e quanto costerebbe invece tenerla attiva solo 8 ore al giorno nei giorni lavorativi. Confronta i due numeri.  **Come verificare:** il secondo scenario deve costare meno di un terzo del primo; se il conto non torna, rileggi la sezione 12.7 sul concetto di pagamento a consumo.
6. **Individua le regioni del progetto reale.** Chiedi a un collega developer o a chi si occupa dell'infrastruttura in quale regione (o regioni) cloud gira il progetto, e se è previsto un meccanismo di ripristino in caso di guasto di un data center (anche solo “sappiamo che esiste un piano B” è una risposta valida a questo livello).  **Come verificare:** sai rispondere, in una frase, alla domanda “cosa succederebbe se il data center principale del progetto avesse un problema per un giorno?” basandoti su quello che hai scoperto.
7. **Piattaforma o burocrazia?** Pensa a uno strumento o processo interno che il tuo team è “obbligato” a usare (anche fuori dall'IT, se non ti viene in mente un esempio tecnico). Applica il criterio visto nella sezione 12.9: se nessuno può scegliere di non usarlo, e non ha “clienti” che ne guidano il miglioramento, è più vicino a una burocrazia che a una piattaforma. Scrivi due righe che spiegano la tua conclusione.  **Come verificare:** la tua risposta cita esplicitamente il criterio dell'adozione volontaria, non solo “è comodo/scomodo da usare”.
8. **DevOps o Integration Platform?** Per ciascuno di questi tre problemi, decidi quale delle due piattaforme viste nella sezione 12.10 useresti per risolverlo, motivando la scelta: (a) “il nuovo servizio di pagamento deve arrivare in produzione ogni giorno, non ogni mese”; (b) “un nuovo servizio deve leggere il numero di polizza da un sistema gestionale assicurativo scritto trent'anni fa, che nessuno vuole riscrivere”; (c) “due team diversi vogliono ricevere una notifica ogni volta che un ordine cambia stato, senza doversi chiamare a interfaccia diretta”.  **Come verificare:** hai assegnato (a) alla piattaforma DevOps e (b) e (c) alla piattaforma di integrazione, con una frase di motivazione per ciascuna.

Collegamenti

- [9. DevOps](#) — la cultura “you build it, you run it” da cui nasce il debito che il Platform Engineering colma, e Service Management/ITSM per quello che succede dopo il rilascio
- [10. CI/CD](#) — la piattaforma DevOps vista qui in concreto: repository, pipeline, artifact, registry
- [13. Sicurezza](#) — dove vedremo come proteggere i dati e i sistemi che girano su queste infrastrutture cloud
- [14. Ambienti di sviluppo](#) — dove vedremo come sviluppo, test e produzione si organizzano concretamente, spesso proprio su infrastrutture cloud come quelle viste qui

- [Microsoft Azure – Cos'è il cloud computing](#) — introduzione ufficiale ai concetti base del cloud
- [Microsoft Learn – Modelli di cloud computing: IaaS, PaaS, SaaS](#) — panoramica dei modelli di servizio cloud
- [AWS – What is Cloud Computing?](#) — la spiegazione ufficiale di Amazon Web Services
- [AWS – Tipi di cloud computing](#) — approfondimento su IaaS, PaaS e SaaS lato AWS
- [Microsoft Learn – Panoramica dei servizi Azure](#) — elenco dei servizi Azure più usati
- [Team Topologies – Platform team](#) — Skelton & Pais sui quattro tipi fondamentali di team, incluso il platform team
- [Google Cloud – DORA: DevOps Research and Assessment](#) — i report annuali citati a proposito di piattaforme interne e produttività
- [Red Hat – Cos'è Platform Engineering](#) — introduzione a Platform Engineering e Internal Developer Platform

13. Sicurezza

Fin qui hai visto come il codice viene scritto, integrato, testato e rilasciato attraverso una pipeline (sezione 10), come viene organizzato in architetture e servizi (sezione 11), e dove “vive” fisicamente nell’infrastruttura cloud (sezione 12). Manca un ultimo ingrediente, che non è un passaggio in più nel processo ma una **lente che attraversa tutti i passaggi precedenti**: la sicurezza informatica.

Non ti aspettare in questa sezione un corso da esperto di cybersecurity: non è il tuo ruolo, e non deve diventarlo. L’obiettivo è più modesto e insieme molto concreto: darti il vocabolario e i concetti di base per **capire di cosa parla il team** quando dice “abbiamo trovato una vulnerabilità critica, il rilascio è bloccato” o “dobbiamo attivare l’MFA per l’accesso all’ambiente di produzione”, e per **valutare correttamente il peso** di questi temi quando pianifichi il lavoro, parli con il cliente o scrivi un contratto.

Obiettivi della sezione

Al termine di questa sezione saprai:

- spiegare perché la sicurezza informatica non è “un problema tecnico” ma riguarda anche costi, reputazione, contratti e conformità legale;
- distinguere **autenticazione** e **autorizzazione**;
- riconoscere il ruolo di **Active Directory**, **Single Sign-On**, **token** e **OAuth/OpenID Connect** nella gestione delle identità aziendali;
- descrivere le buone pratiche di base su password e **autenticazione a più fattori (MFA)**;
- ricollegare HTTPS e crittografia a quanto già visto nei Fondamenti di informatica, con un’idea semplice di cosa significhi “dato cifrato”;
- riconoscere, a grandi linee, cosa sono vulnerabilità comuni come SQL Injection e Cross-Site Scripting (XSS), e perché esistono;
- sapere cos’è la **OWASP Top 10** e a cosa serve come riferimento di settore;
- capire cos’è il **DevSecOps** e il principio dello “**shift left**” della sicurezza;
- capire come gli scan di sicurezza automatici si inseriscono nella pipeline CI/CD già vista nella sezione 10;
- spiegare, a livello base, perché il **GDPR** e la protezione dei dati personali riguardano anche il tuo lavoro di PM;
- capire perché **backup e disaster recovery** sono parte della sicurezza, non un tema separato;
- distinguere il **Disaster Recovery** dalla **Business Continuity**, e capire perché la **Business Impact Analysis** rende RTO e RPO una decisione di business.

13.1 Perché la sicurezza riguarda anche un PM, non solo i tecnici

Qualche mese prima che ShopFacile introducesse l’MFA anche in produzione, un collaboratore esterno aveva riusato la sua password aziendale su un servizio online diverso, non collegato al progetto. Quel servizio viene violato, e le sue credenziali finiscono in una lista di password rubate che circola online. Qualcuno le prova, per tentativo, sugli accessi di ShopFacile — chi ruba credenziali le prova ovunque,

sperando che siano state riusate. Il tentativo si ferma sull'ambiente di test, perché lì l'MFA era già obbligatoria; in produzione, in quel momento, non lo era ancora. Ad accorgersene è Marco, il mattino dopo, leggendo per altri motivi il registro degli accessi: dodici tentativi falliti con una password **corretta**, bloccati solo dal secondo fattore. Quando lo porta a Luca, la domanda che si fanno non è “chi è stato”, ma un'altra: perché una protezione che c'era in test non c'era anche dove stanno i dati veri dei clienti?

È bastato questo episodio a rendere concreta un'idea altrimenti astratta: la sicurezza informatica è come **l'impianto antincendio di un edificio**. Non lo vedi quasi mai in azione, e nessuno pianifica una giornata di lavoro pensando all'impianto antincendio — ma se manca, o è mal progettato, il giorno in cui succede qualcosa **le conseguenze sono enormemente più gravi** di quanto sarebbe costato prevenirle.

Per un Project Manager o uno Scrum Master, un incidente di sicurezza non è “un bug in più da segnare sul backlog”: ha impatti che riconoscerai immediatamente come “roba tua”:

- **Costi diretti**: fermare un servizio compromesso, investigare cosa è successo, correggere la falla, spesso richiede ore o giorni di lavoro imprevisto di più persone del team, sottratte a quanto era pianificato nello sprint.
- **Reputazione**: un cliente che scopre che i dati dei suoi utenti sono stati esposti perde fiducia nel progetto e in chi lo ha realizzato — una fiducia che si ricostruisce molto più lentamente di quanto si perda.
- **Contratti**: molti contratti con i clienti includono clausole precise su sicurezza e gestione dei dati (a volte chiamate SLA di sicurezza o requisiti di compliance). Non rispettarle può voler dire penali economiche o, nei casi più seri, la fine della collaborazione.
- **Conformità legale**: normative come il GDPR (che vedremo al paragrafo 13.9) impongono obblighi precisi sulla protezione dei dati personali, con sanzioni concrete in caso di violazione.

Il punto chiave da portare con te: **la sicurezza è un requisito del progetto, esattamente come una funzionalità richiesta dal cliente** — non un “optional” che si aggiunge se resta tempo a fine sprint. Un PM che tratta la sicurezza come priorità fin dalla pianificazione fa risparmiare tempo, denaro e problemi seri più avanti.

Esempio pratico: durante la pianificazione di una nuova funzionalità di ShopFacile che permette ai clienti di caricare la foto di un prodotto da restituire, il team si accorge che serve anche configurare i permessi di accesso a quei file e cifrare il canale di upload. Se Sara, come PO, inserisce questo lavoro nella stima dello sprint fin dall'inizio (come farebbe con qualsiasi altro requisito), il rilascio resta nei tempi. Se invece la sicurezza viene “ricordata” solo a fine sprint, il rilascio slitta e il team lavora sotto pressione per rimettersi in pari — lo stesso lavoro, fatto più tardi, costa di più.

13.2 Autenticazione vs Autorizzazione

Il caricamento delle foto di resa solleva subito una domanda concreta: chi può fare cosa, su quei file e su ShopFacile in generale? È il momento di distinguere con precisione due concetti che si sentono nominare quasi sempre insieme, e che vengono facilmente confusi — ma rispondono a due domande diverse:

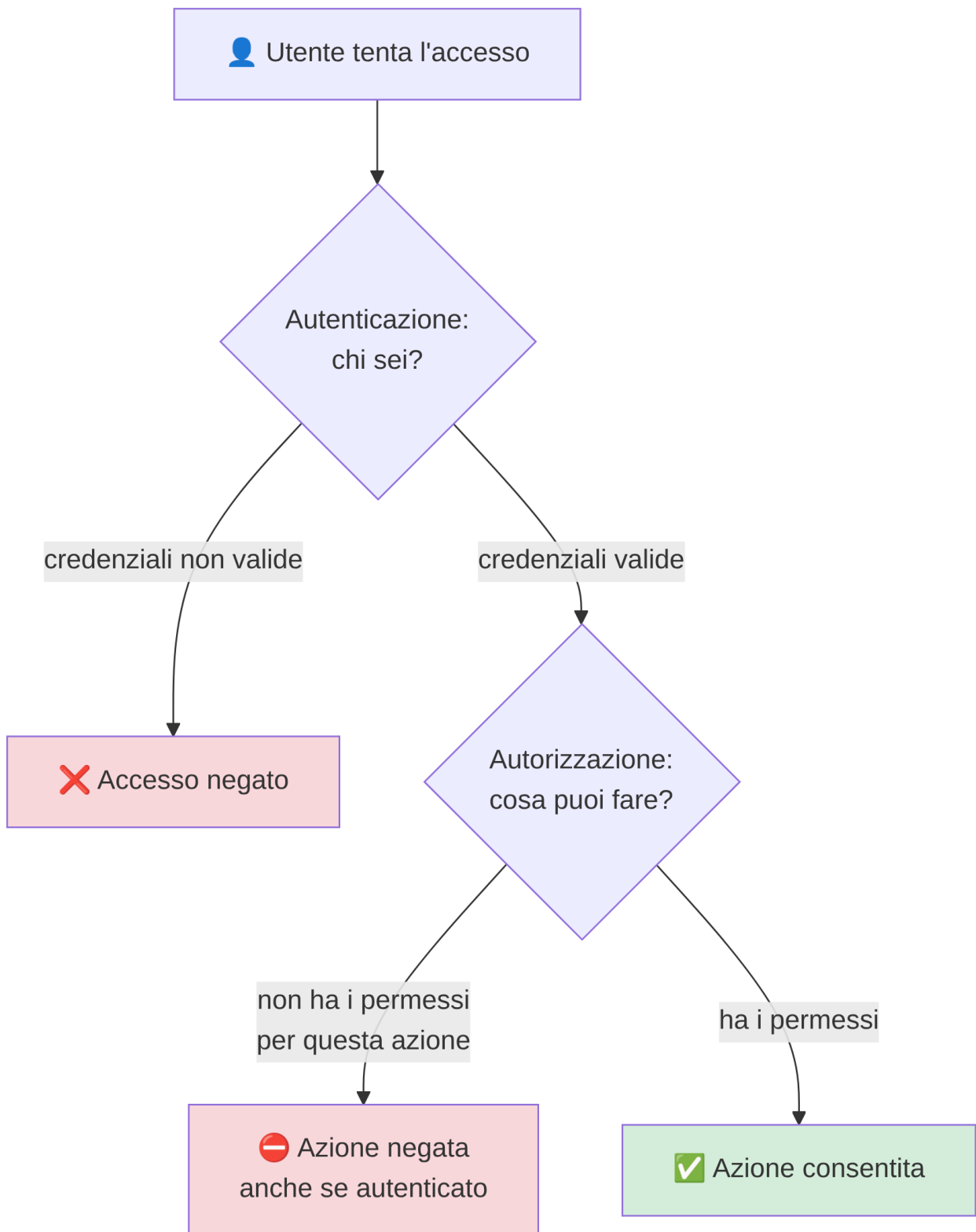
- **Autenticazione** risponde alla domanda: “**Chi sei?**”. È il processo con cui un sistema verifica che tu sia davvero chi dici di essere (tipicamente inserendo username e password, o mostrando un altro tipo di “prova d’identità”).
- **Autorizzazione** risponde alla domanda: “**Cosa puoi fare, ora che sappiamo chi sei?**”. È il processo con cui il sistema decide a quali risorse o funzionalità hai accesso.

Analogia: il cinema

Immagina di andare al cinema:

- Alla biglietteria mostri il tuo **biglietto**: la persona alla cassa verifica che sia un biglietto valido per lo spettacolo di oggi. Questa è l'**autenticazione**: hai dimostrato di essere una persona con diritto a entrare.
- Una volta dentro, il biglietto indica anche **quale posto** puoi occupare — Sala 3, fila D, posto 12. Non puoi sederti dove vuoi, né entrare nella sala riservata a un altro film. Questa è l'**autorizzazione**: definisce esattamente cosa puoi fare, anche se sei già stato ammesso.

Puoi essere autenticato correttamente (il tuo biglietto è vero) ma non autorizzato a fare una certa cosa (entrare nella Sala 5, dove è in programmazione un altro film). Sono due controlli **distinti e sequenziali**: prima si stabilisce chi sei, poi cosa puoi fare.



Esempio pratico nel progetto: Ahmed può autenticarsi correttamente su GitHub, la piattaforma del progetto ShopFacile (sa la password, entra), ma essere autorizzato solo a **leggere** il codice del repository del catalogo prodotti e non a **modificare** le impostazioni della pipeline di produzione — quel permesso è riservato, ad esempio, a Marco o a Luca. Due sviluppatori autenticati sulla stessa piattaforma possono avere autorizzazioni completamente diverse.

Chi gestisce davvero autenticazione e autorizzazione, in un'azienda vera

Nella scena sopra, Ahmed si autentica su un solo sistema. Nella realtà di un'azienda enterprise, però, quando Ahmed è entrato nel team ha ricevuto **sei credenziali diverse su sei canali diversi**: una per la posta, una per GitHub, una per la VPN, una per il gestionale interno, una per lo strumento di ticketing, una per l'ambiente cloud. Quando **Giulia**, qualche mese dopo, lascia il progetto, ha **cinque accessi** che nessuno sa elencare con certezza — e disattivarli tutti significa ricordarsi di cinque sistemi diversi, sperando di non scordarne uno. Nel frattempo, ogni nuova applicazione che il team costruisce si trova a dover reinventare da zero la sua schermata di login, cioè a reimplementare, applicazione per applicazione, la parte più delicata dell'intero sistema. **Luca** la sintetizza così: l'unica lista completa di “chi può fare cosa” **non esiste da nessuna parte**.

Directory e Active Directory. La soluzione a questo problema è avere una **fonte unica delle identità**: una directory che contiene utenti, gruppi, computer e politiche di sicurezza dell'intera azienda, invece di farle gestire a ogni applicazione per conto proprio. **Active Directory** (spesso abbreviato AD) è l'implementazione più diffusa nelle aziende enterprise, e parla tipicamente il protocollo **LDAP** (*Lightweight Directory Access Protocol*) per farsi interrogare dalle altre applicazioni; **Entra ID** (il nome attuale di quello che si chiamava Azure AD) è la sua versione pensata per il cloud. Il dettaglio che riguarda direttamente il tuo ruolo: l'appartenenza a un **gruppo** nella directory diventa un'**autorizzazione** — “chi è nel gruppo Developer Catalogo può modificare quel repository” — e quindi **gestire gli accessi diventa, in pratica, gestire i gruppi**. È il motivo per cui, quando arriva una persona nuova nel team, è comune che un PM venga coinvolto: non per decidere la tecnologia, ma per confermare a quali gruppi quella persona deve appartenere.

SSO (Single Sign-On). Una volta che esiste una directory unica, ha senso costruirci sopra un'unica autenticazione valida per molte applicazioni: è il **Single Sign-On (SSO)**. Con l'SSO attivo, Ahmed si autentica **una sola volta** e da lì accede a email, GitHub, gestionale e strumenti cloud senza rifare il login su ciascuno — le sue sei credenziali diventano una sola identità; e quando Giulia lascia il progetto, i suoi cinque accessi si spengono **con un solo interruttore**, quello della sua identità centrale, invece di dover essere disattivati sistema per sistema. Il rovescio della medaglia è che l'SSO **concentra** il rischio esattamente dove concentra la comodità: se il sistema di SSO cade, cade l'accesso a tutto contemporaneamente; se una credenziale viene rubata, quella singola credenziale apre tutto, non solo un'applicazione. Ed è **per questo**, non per policy generica, che l'MFA — che vedremo nel dettaglio al paragrafo 13.3 — non è opzionale proprio sull'identità centrale: più un accesso è “la chiave di tutto”, più quella chiave deve avere una seconda verifica.

Token. Una volta autenticato tramite SSO, il sistema non manda più la password a ogni singola richiesta — sarebbe come dover riesibire il biglietto del cinema (l'analogia vista sopra) a ogni passo del corridoio. Manda invece un **token**: un biglietto temporaneo che dice chi sei e cosa puoi fare, con una **scadenza**. La scadenza breve non è un fastidio, è una protezione: se un token viene intercettato o rubato, il danno che può fare è limitato alla finestra di tempo in cui è ancora valido, non a tempo indefinito. Per lo stesso motivo, un token va trattato **esattamente come una password**: se finisce per errore in un file del repository — lo stesso errore che vedremo tra poco per le chiavi di accesso, al paragrafo 13.3 — va considerato compromesso e sostituito, non semplicemente rimosso dal commit successivo.

OAuth e OpenID Connect. C'è una distinzione che quasi nessuno spiega bene, ed è proprio quella che separa OAuth da tutto il resto: **OAuth** serve a **delegare un'autorizzazione limitata**, senza mai cedere la password. Immagina un'applicazione di terzi che deve leggere i documenti di un cliente di ShopFacile per conto suo (es. per compilare automaticamente un modulo): il cliente non le consegna la sua password, le concede un **permesso limitato e revocabile** — “puoi leggere solo i documenti, solo per i prossimi 30 giorni”. Il pulsante “accedi con...” che vedi su tanti siti è l'esempio quotidiano di questo meccanismo. **OpenID Connect (OIDC)** costruisce sopra OAuth un livello ulteriore: non si limita a delegare un permesso, aggiunge l'**autenticazione** vera e propria — “dimmi chi è questa persona”, non solo “dammi il permesso di fare questa cosa specifica”. Il punto in cui un PM deve fare attenzione è il momento del **consenso**: quando un'applicazione chiede “vuoi autorizzare X a leggere i tuoi documenti?”, la domanda da porsi non è solo “mi fido di X”, ma **cosa** sto autorizzando esattamente a leggere, e **per quanto tempo** — un permesso concesso una volta e poi dimenticato resta attivo silenziosamente, anche quando non serve più.

Termine	Risponde alla domanda
Autenticazione	Chi sei?
Autorizzazione	Cosa puoi fare, ora che sappiamo chi sei?
SSO	Posso autenticarmi una volta sola per più applicazioni?
Token	Come dimostro chi sono e cosa posso fare, senza rimandare la password ogni volta?
OAuth	Come do a un'app un permesso limitato, senza darle la mia password?
OpenID Connect	Come uso lo stesso meccanismo di OAuth anche per dire “chi sono”?
Active Directory / LDAP	Dove vive, in un unico posto, l'elenco di chi esiste e a cosa appartiene?

I trade-off da portare con te: un'identità centralizzata (directory + SSO) è comoda e, allo stesso tempo, fragile, perché concentra in un solo punto un rischio che prima era distribuito su tanti sistemi diversi; un token a vita lunga è comodo (non devi rifare login spesso) ed è proprio per questo pericoloso, perché resta utilizzabile a lungo se rubato; la delega OAuth è potente ma i permessi concessi vengono spesso **dimenticati** — nessuno torna a controllare, mesi dopo, se quell'app di terzi ha ancora bisogno di leggere i documenti del cliente. È esattamente per questo che la **revisione periodica degli accessi**, già citata come pratica di governance della sicurezza al paragrafo 13.7, non riguarda solo chi ha accesso a un repository: riguarda anche quali applicazioni terze hanno ancora un permesso OAuth attivo che nessuno usa più.

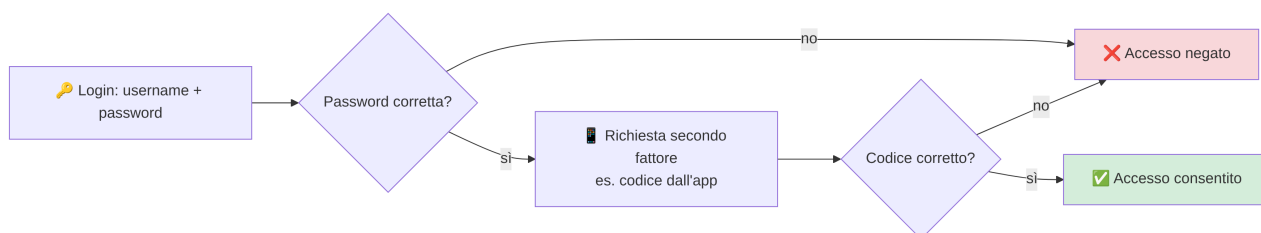
13.3 Password, credenziali e MFA: le buone pratiche di base

Autenticazione e autorizzazione presuppongono entrambe una cosa: che sia davvero difficile “spacciarsi” per qualcun altro. Ed è proprio qui, sulle credenziali usate per autenticarsi, che si concentra la maggior parte degli incidenti reali. La stragrande maggioranza degli incidenti di sicurezza non nasce da tecniche sofisticate da film di spionaggio, ma da **credenziali debitamente rubate o indovinate**: password troppo semplici, riutilizzate ovunque, o condivise con troppa disinvoltura.

Qualche buona pratica base, valida per te e per chiunque nel team:

- **Password forti e uniche**: una password lunga (idealmente 12+ caratteri), non riconducibile a informazioni personali facilmente reperibili (data di nascita, nome del cane), e **diversa per ogni servizio**. Se un servizio viene compromesso e la password è riusata altrove, il danno si moltiplica.
- **Gestori di password (password manager)**: strumenti come quelli aziendali forniti dall'IT (o soluzioni diffuse come Bitwarden, 1Password) generano e ricordano password complesse per ogni servizio, così non devi memorizzarle tu né riusarle. È molto più sicuro affidarsi a uno di questi strumenti che scrivere le password su un file di testo o, peggio, su un post-it.
- **Autenticazione a più fattori (MFA - Multi-Factor Authentication)**: oltre alla password (qualcosa che sai), il sistema richiede una seconda “prova”, ad esempio un codice generato da un'app sul telefono o un'impronta digitale (qualcosa che hai o che sei). Anche se qualcuno scoprisse la tua password, senza il secondo fattore non potrebbe entrare.

💡 **Analogia**: la password è come la chiave di casa. L'MFA è come avere, oltre alla chiave, anche un citofono con telecamera che chiede conferma prima di far entrare qualcuno — anche se ha copiato la chiave, senza quella seconda verifica non entra.



Nel progetto, è molto probabile che l'accesso a strumenti sensibili (GitHub, ambiente cloud, VPN aziendale) richieda l'MFA per policy aziendale: se ti viene chiesto di installare un'app di autenticazione sul telefono per lavoro, ora sai perché.

Esempio pratico: immagina che la password di un membro del team venga scoperta perché usata anche su un altro servizio online violato in passato (è più comune di quanto sembri: succede quando la stessa password viene riusata su più siti). Chi ha ottenuto quella password prova ad accedere a GitHub con quelle credenziali. Senza MFA, entrerebbe subito. Con l'MFA attivo, il sistema chiede anche il codice generato dall'app sul telefono di quella persona: l'attaccante non lo ha, e l'accesso viene bloccato. Questo è esattamente il motivo per cui, dopo un data breach di un servizio terzo, la prima raccomandazione è sempre “cambia la password e verifica che l'MFA sia attivo”, non solo la prima.

Un errore frequente: le credenziali salvate nel repository

Un modo comune tra i developer alle prime armi di vanificare queste precauzioni: salvare per errore una chiave di accesso (*API key*) **direttamente nei file del repository**. Come visto nella sezione 4, la storia dei commit **non si dimentica**: cancellarla nel commit successivo non basta, resta recuperabile nella cronologia.

Esempio pratico: Ahmed sta per committare, per sbaglio, la chiave email di ShopFacile in un file di configurazione. Uno scanner di segreti nella pipeline (sezione 10) la intercetta e blocca il commit. Se non ci fosse stato, la regola sarebbe: quella credenziale va considerata **compromessa e sostituita**, non solo rimossa — resterebbe visibile nella cronologia. Un `.gitignore` ben configurato (sezione 4) previene il problema a monte.

13.4 HTTPS e crittografia: un richiamo veloce

Password e MFA proteggono l'accesso a un sistema, ma non bastano da sole: anche quando i dati viaggiano in rete o restano salvati da qualche parte, serve un'altra forma di protezione. Nella sezione [2. Fondamenti di informatica](#) hai già visto la differenza tra HTTP e HTTPS: HTTPS **cifra** i dati scambiati tra client e server, in modo che chi li intercettasse lungo il tragitto non possa leggerli.

Qui aggiungiamo solo un tassello: cosa significa concretamente “dato cifrato”?

Cifrare un dato significa trasformarlo, tramite un algoritmo matematico e una “chiave” segreta, in una sequenza illeggibile per chiunque non possieda la chiave giusta per invertire la trasformazione (**decifrare**). Un esempio semplicissimo, giocattolo, solo per l'idea:

```
Testo originale:   CIAO
Regola di cifratura: sposta ogni lettera di 3 posizioni nell'alfabeto
Testo cifrato:     FLDR
```

Chi intercetta “FLDR” senza conoscere la regola (“sposta di 3 posizioni”) non ha modo semplice di capire che il messaggio originale era “CIAO”. Gli algoritmi usati davvero da HTTPS sono incomparabilmente più complessi e robusti di questo esempio — è solo per rendere intuitiva l'idea di “trasformare un dato leggibile in uno illeggibile, con una chiave che serve per tornare indietro”.

Due momenti in cui i dati vanno protetti, entrambi importanti:

- **Dati “in transito”:** mentre viaggiano in rete tra client e server — è il ruolo di HTTPS, già visto.
- **Dati “a riposo”** (*at rest*): mentre sono salvati su un disco, in un database, in un backup. Anche questi vanno spesso cifrati, perché se qualcuno accede fisicamente o illegittimamente a quel disco, senza la chiave i dati restano illeggibili.

Non avrai bisogno di implementare crittografia, ma è utile sapere che quando un cliente chiede “i nostri dati sono cifrati?”, la risposta completa riguarda **sia** il transito (HTTPS) **sia** il salvataggio (a riposo).

Esempio pratico: un cliente compila il modulo di pagamento al checkout di ShopFacile e invia i propri dati (indirizzo di spedizione, dati della carta). Se il sito usa HTTPS, quei dati viaggiano cifrati dal browser del cliente al server — anche se qualcuno intercettasse il traffico di rete (ad esempio su un Wi-Fi pubblico non sicuro), leggerebbe solo una sequenza illeggibile. Una volta arrivati al server, quei dati vengono salvati nel database ordini: se anche il database è configurato per cifrare i dati “a riposo”, chi eventualmente accedesse senza autorizzazione al disco fisico del server (un furto, una copia illegittima del backup) troverebbe comunque dati illeggibili senza la chiave giusta. Le due protezioni sono indipendenti: un sito può avere HTTPS ma un database non cifrato, o viceversa — un cliente attento chiede entrambe.

13.5 Vulnerabilità comuni: perché il codice va “testato” anche per la sicurezza

Cifrare i dati non serve a nulla se il codice stesso ha una falla che permette di aggirare i controlli a monte: è il momento di guardare da dove nascono davvero questi problemi. Una **vulnerabilità** è un errore o una debolezza nel software che qualcuno con intenzioni malevole (un *attaccante*) può sfruttare per fare qualcosa che non dovrebbe poter fare: leggere dati che non gli appartengono, modificarli, bloccare un servizio.

Non serve conoscere i dettagli tecnici di come si sfruttano, ma è utile sapere che **esistono**, che hanno nomi ricorrenti, e che è per questo che il codice va controllato anche dal punto di vista della sicurezza, non solo della correttezza funzionale. Due esempi molto citati:

- **SQL Injection:** succede quando un'applicazione inserisce, senza i controlli giusti, un testo scritto dall'utente direttamente dentro una query verso il database (quelle query SQL viste nella sezione 2). Se l'utente scrive, in un campo di ricerca, non un nome ma un pezzo di comando SQL “camuffato”, e l'applicazione non lo riconosce come pericoloso, quel comando può finire per essere eseguito sul database — ad esempio per leggere dati che non dovrebbe vedere.
- **Cross-Site Scripting (XSS):** succede quando un'applicazione web mostra, senza i controlli giusti, un testo scritto da un utente (es. un commento, un messaggio) direttamente dentro la pagina vista da altri utenti. Se quel testo contiene in realtà un piccolo “programma” camuffato da testo normale, il browser di chi legge quella pagina può finire per eseguirlo — ad esempio per rubare dati della sessione di quell'altro utente.



Analogia semplice: pensa a un modulo di iscrizione dove il campo “Nome” dovrebbe contenere solo... un nome. Se il sistema si fida ciecamente di quello che scrivi e non controlla mai cosa c'è davvero scritto lì dentro, qualcuno potrebbe scrivere, invece del suo nome, un'istruzione nascosta — e se il sistema la esegue senza accorgersene, il “modulo” è diventato una porta d'accesso indesiderata.

Il punto per te, come PM, non è saper riconoscere queste vulnerabilità nel codice — è sapere che **esistono categorie note e ricorrenti** di errori di questo tipo, e che per questo il codice va **testato anche per la sicurezza**, con strumenti dedicati, esattamente come si testa la correttezza funzionale con i test automatici visti nella sezione 10.

Esempio pratico: Ahmed, ancora alle prime armi, scrive una nuova funzionalità per applicare i codici promozionali al carrello e, per fare in fretta, costruisce la query verso il database concatenando direttamente il testo digitato dal cliente nel campo “codice promozionale”, senza validarlo. Durante la code review, Giulia — che segue da vicino test e qualità del codice — nota il problema e segnala la Pull Request come da correggere prima ancora che arrivi a un test funzionale: è esattamente il tipo di errore (un classico caso di SQL Injection) che un developer alle prime esperienze può commettere senza malizia, e che una revisione attenta deve intercettare prima del rilascio, non dopo.

13.6 OWASP Top 10: il riferimento standard del settore

Riconoscere una singola query pericolosa in una code review è utile, ma il team non può fare affidamento solo sulla memoria di chi legge il codice quel giorno: serve un riferimento condiviso e sistematico. Con il tempo, la community internazionale della sicurezza informatica ha raccolto e catalogato le vulnerabilità più comuni e più pericolose che colpiscono le applicazioni web, mantenendo una lista aggiornata periodicamente: la **OWASP Top 10**.

OWASP (Open Worldwide Application Security Project) è un’organizzazione internazionale, non a scopo di lucro, dedicata a migliorare la sicurezza del software. La sua **Top 10** è probabilmente il documento più citato al mondo in questo ambito: una classifica delle categorie di vulnerabilità più critiche e più frequenti nelle applicazioni web (di cui SQL Injection e XSS, visti sopra, sono esempi storici).

Non devi memorizzare l’elenco né conoscerne i dettagli tecnici. Quello che ti serve sapere è:

- è un **riferimento riconosciuto a livello mondiale**: quando un cliente, in un contratto o in un audit, chiede che l’applicazione sia “conforme alla OWASP Top 10” o che sia stato eseguito un “test basato sulla OWASP Top 10”, sta chiedendo che il software sia stato verificato contro questo elenco di rischi noti;
- gli strumenti automatici di analisi di sicurezza che vedrai nella pipeline (paragrafo 13.8) fanno spesso riferimento esplicito a queste categorie;
- è aggiornata periodicamente, perché le tecniche di attacco e le tecnologie usate nel software cambiano nel tempo.

Sapere che questo standard esiste ti permette di seguire una conversazione tecnica sulla sicurezza senza perderti, e di capire perché in una gara d’appalto o in un capitolato può comparire esplicitamente il riferimento a OWASP.

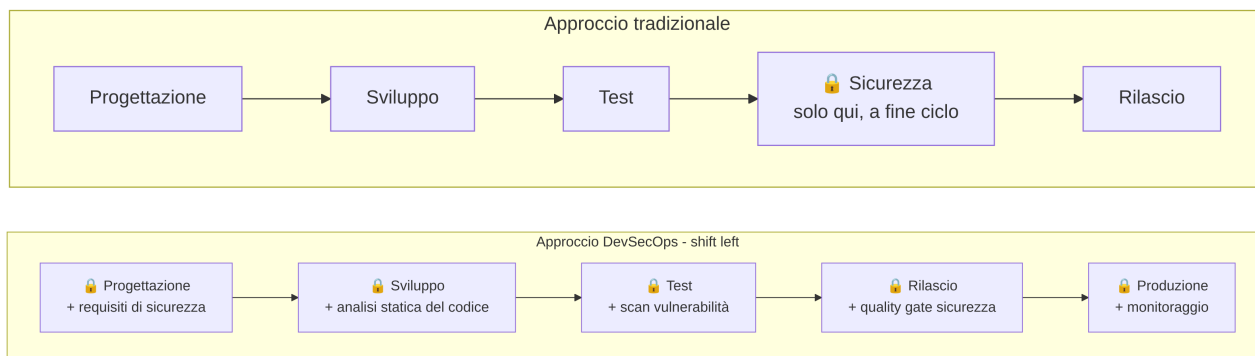
Esempio pratico: il cliente di un progetto chiede, in fase di capitolato, che prima del rilascio in produzione venga eseguito un “penetration test basato sulla OWASP Top 10”. In pratica, un team di sicurezza (interno o esterno) prova a verificare, categoria per categoria di quella lista, se l’applicazione

presenta debolezze note — ad esempio, controlla se è vulnerabile a SQL Injection o XSS (visti sopra), se gestisce correttamente l'autenticazione, se espone informazioni che non dovrebbe. Il risultato è un report con eventuali criticità trovate, classificate per gravità: è esattamente il tipo di documento che un PM può dover presentare al cliente come prova che il software è stato verificato secondo uno standard riconosciuto, non “a occhio”.

13.7 DevSecOps: la sicurezza fin dall'inizio (“shift left”)

La OWASP Top 10 dice cosa controllare; resta da capire *quando* farlo nel ciclo di vita del progetto — ed è qui che entra in gioco un principio già visto altrove in una forma diversa. Nella sezione 9. DevOps hai visto come la cultura DevOps abbatta il muro tra chi scrive il software e chi lo mette in produzione, integrando i due mondi in un unico flusso continuo. Il **DevSecOps** estende esattamente la stessa logica alla sicurezza: la “Sec” (Security) non è più una fase separata, affidata a un team diverso, che interviene solo alla fine, poco prima del rilascio — è **integrata in ogni fase** del processo di sviluppo, fin dall'inizio.

Questo principio si chiama “**shift left**” (letteralmente “spostare a sinistra”): se disegni il processo di sviluppo come una linea temporale che va da sinistra (progettazione) a destra (produzione), storicamente i controlli di sicurezza avvenivano quasi solo a destra, poco prima del rilascio — quando un problema trovato è già molto costoso da correggere, perché magari richiede di rivedere scelte di progettazione fatte mesi prima. Il DevSecOps **sposta quei controlli verso sinistra**, il più presto possibile nel processo.



Perché conviene? Perché **correggere un problema di sicurezza costa sempre meno, più presto lo si scopre** — esattamente come per i bug funzionali: un errore di progettazione trovato durante la stesura dei requisiti costa una discussione; lo stesso errore scoperto in produzione, con dati reali di utenti reali già esposti, può costare giorni di lavoro d'emergenza, comunicazioni al cliente e, potenzialmente, obblighi legali di notifica (ne parliamo al paragrafo 13.9).

Per un PM/Scrum Master, il DevSecOps si traduce concretamente in domande da porsi già in fase di pianificazione, non solo a rilascio imminente: “questa nuova funzionalità gestisce dati sensibili?”, “abbiamo previsto tempo per la revisione di sicurezza?”, “il quality gate di sicurezza è configurato sulla pipeline di questo progetto?”.

Il prezzo reale: sicurezza e velocità sono in tensione

Più controlli non sono “gratis”: ogni scan aggiuntivo allunga i tempi di rilascio, e alcune segnalazioni sono **falsi positivi** che qualcuno deve comunque analizzare e scartare. Un conflitto tipico: un quality gate blocca un rilascio per una vulnerabilità “media” in una libreria usata **solo nei test**, mai raggiungibile da un utente reale. Applicare sempre la regola più severa sembra prudente, ma può fermare un rilascio urgente per un rischio quasi nullo. Chi facilita il processo (spesso lo Scrum Master) deve **arbitrare in base al rischio reale** — dove si trova la vulnerabilità, chi potrebbe sfruttarla — invece di applicare automaticamente la regola più rigida.

Chi decide cosa: la governance della sicurezza

Questo compromesso funziona solo se è chiaro **chi ha l'autorità di decidere**. Come per la matrice RACI vista nella sezione 8, per ShopFacile vanno resi espliciti: chi può **bloccare un rilascio** per motivi di sicurezza; chi **approva un'eccezione** al quality gate (non la stessa persona che ha scritto il codice coinvolto); chi **revoca gli accessi** quando una persona lascia il progetto, prima che un accesso dimenticato diventi un rischio silenzioso; chi fa la **revisione periodica dei permessi**, verificando che le autorizzazioni (paragrafo 13.2) siano ancora giustificate. Metterli per iscritto nella RACI, prima dell'emergenza, è lavoro di processo.

Sicurezza come voce di backlog, non come intenzione generica

Un elemento come “rendere sicura l'applicazione” non è utilizzabile: non si stima, non si testa, e nessuno sa quando è “fatto” — viola la Definition of Done della sezione 6. Un elemento scritto bene ha un **criterio di accettazione verificabile**: *“Come cliente, voglio che l'accesso al mio account richieda un secondo fattore, così che una password rubata non basti. Criterio di accettazione: login con password corretta ma senza MFA valido restituisce accesso negato, verificabile con un test automatico.”*

13.8 Scan di sicurezza nella pipeline CI/CD

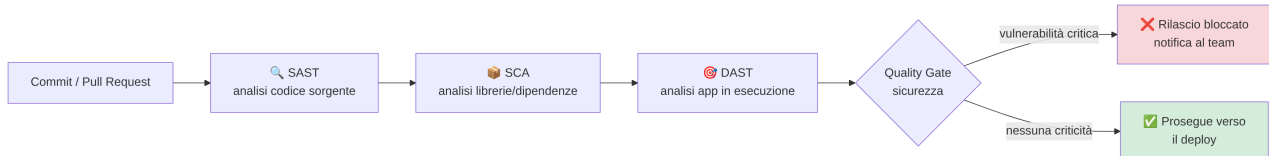
Il “shift left” resta un principio astratto finché non si traduce in qualcosa che gira automaticamente a ogni commit: vediamo concretamente come. Nella sezione 10. CI/CD hai già incontrato, nella fase “Analisi di qualità e sicurezza del codice” e nella tabella dei quality gate, l'idea che la pipeline includa controlli automatici di sicurezza. Qui rendiamo quel concetto un po' più concreto.

Gli strumenti di scan automatico più comuni, integrati direttamente nella pipeline, verificano cose diverse:

- **Analisi statica del codice (SAST - Static Application Security Testing)**: analizza il codice sorgente, senza eseguirlo, cercando pattern noti di vulnerabilità (es. un punto dove un dato dell'utente finisce, senza controlli, dentro una query SQL — il rischio di SQL Injection visto al paragrafo 13.5).
- **Analisi delle dipendenze (SCA - Software Composition Analysis)**: quasi nessun software è scritto completamente da zero — si usano librerie esterne (open source o commerciali). Questi strumenti controllano se una delle librerie usate ha vulnerabilità **note e pubblicamente documentate**, e segnalano se serve aggiornarla.

- **Analisi dinamica (DAST - Dynamic Application Security Testing):** a differenza del SAST, prova ad “attaccare” l'applicazione realmente in esecuzione (tipicamente in un ambiente di test), simulando alcune delle tecniche di un vero attaccante, per vedere come si comporta davvero.

Il risultato di questi scan si traduce, come già visto nella sezione 10, in un **quality gate**: se viene trovata una vulnerabilità sopra una certa soglia di gravità (tipicamente “alta” o “critica”), la pipeline si ferma e il rilascio viene bloccato finché il problema non viene risolto — non è diverso, nel meccanismo, da un test funzionale che fallisce.



Il vantaggio di questi scan automatici, rispetto a una revisione manuale occasionale, è lo stesso visto per i test automatici in generale: **ripetibilità e velocità**. Ogni singolo commit viene controllato, sempre, senza dipendere dalla memoria o dalla disponibilità di una persona che “se ne ricordi”.

Esempio pratico: Marco, che segue spesso la parte infrastrutturale di ShopFacile, configura lo scan SCA sulla pipeline del servizio pagamenti e scopre che una libreria usata per generare i PDF delle fatture ha una vulnerabilità nota, già pubblicata. Il quality gate blocca il rilascio finché la libreria non viene aggiornata: senza quello scan automatico, la falla sarebbe rimasta in produzione fino a quando (e se) qualcuno se ne fosse accorto da solo, magari troppo tardi.

13.9 GDPR e privacy dei dati: un accenno essenziale

Gli scan automatici proteggono il codice, ma c'è un'altra dimensione della sicurezza che riguarda direttamente le persone i cui dati ShopFacile raccoglie ogni giorno — clienti che comprano, si registrano, chiedono un rimborso. Il **GDPR** (General Data Protection Regulation, “Regolamento Generale sulla Protezione dei Dati”) è la normativa europea che disciplina come le aziende devono trattare i **dati personali** delle persone (nomi, email, indirizzi, dati di navigazione, e molto altro).

Non è un tema puramente etico (“è giusto proteggere i dati delle persone”), è un **obbligo legale**, con conseguenze concrete in caso di violazione: sanzioni economiche (che possono essere molto rilevanti), obbligo di comunicare la violazione alle autorità competenti e, spesso, agli stessi utenti coinvolti entro tempi stretti, oltre al danno reputazionale già visto al paragrafo 13.1.

Per il tuo ruolo, alcuni concetti base sufficienti per orientarti:

- se un progetto raccoglie o gestisce dati di persone (utenti, dipendenti, clienti), quei dati vanno protetti con misure tecniche adeguate (accessi controllati, cifratura, backup sicuri — tutti concetti già visti in questa sezione);
- va raccolto solo il dato **necessario** allo scopo dichiarato (principio di “minimizzazione”), non “tutto quello che si può raccogliere”;
- una violazione di dati personali (un cosiddetto **data breach**) ha obblighi di notifica precisi, con scadenze temporali stringenti;

- molti progetti prevedono un ruolo o un referente per la protezione dei dati (in alcuni contesti, un DPO - Data Protection Officer) da coinvolgere quando una funzionalità tratta dati personali in modo nuovo.

Non ti verrà chiesto di diventare un esperto legale di GDPR, ma è utile riconoscere quando una richiesta del cliente o una funzionalità del progetto **tocca dati personali**, per sapere che in quei casi il tema sicurezza/privacy va coinvolto fin dalla pianificazione, non aggiunto a posteriori.

Esempio pratico: Sara, come PO, riceve dal cliente una richiesta di funzionalità che prevede di salvare, per ogni cliente di ShopFacile, anche il numero di telefono e la posizione geografica approssimativa (per proporre le consegne più rapide vicino a lui). Sono entrambi dati personali secondo il GDPR. Sara, in fase di pianificazione, si pone (e pone al team) alcune domande prima di stimare lo sprint: “questo dato è davvero necessario per la funzionalità, o stiamo raccogliendo più del dovuto?”, “chi avrà accesso a questo dato una volta salvato?”, “per quanto tempo lo conserviamo?”. Se invece la funzionalità viene sviluppata e rilasciata senza porsi queste domande, e in seguito si scopre un accesso non autorizzato ai dati dei clienti, ShopFacile si trova davanti a un data breach da notificare formalmente, con tempi stretti e conseguenze reali — non un semplice bug da correggere alla prossima release.

13.10 Backup e disaster recovery: il piano B quando qualcosa va storto

Anche il progetto più attento a GDPR, scan e code review non è immune da un imprevisto puro e semplice: un tema che a volte viene percepito come “amministrativo” ma è parte integrante della sicurezza. Cosa succede se, nonostante tutte le precauzioni, qualcosa va comunque storto — un guasto hardware, un errore umano che cancella dati per sbaglio, o un attacco che compromette un sistema.

- **Backup:** una copia dei dati, salvata separatamente (spesso in un luogo fisicamente diverso), che permette di recuperare le informazioni se l'originale viene perso o danneggiato. Un backup che non viene mai testato — cioè non si è mai provato davvero a ripristinarlo — è un falso senso di sicurezza: l'unico backup di cui ci si può davvero fidare è quello che si è verificato essere effettivamente ripristinabile.
- **Disaster Recovery (DR):** il piano più ampio, non limitato ai soli dati, per far ripartire un intero sistema/servizio dopo un evento grave (un guasto importante, un disastro naturale che colpisce un data center, un attacco su larga scala), nel modo più rapido possibile.

 **Analogia:** il backup è come la copia delle chiavi di casa lasciata a un vicino di fiducia: se perdi le tue, non resti fuori. Il disaster recovery è come avere un piano intero per la tua famiglia in caso la casa diventi inabitabile — dove andare, cosa portare, chi contattare — non basta la copia delle chiavi se la casa stessa non esiste più.

Due metriche che il team può citare parlando di DR, utili da riconoscere:

- **RTO (Recovery Time Objective):** quanto tempo, al massimo, ci si può permettere che il servizio resti fermo prima di essere ripristinato.
- **RPO (Recovery Point Objective):** quanti dati, al massimo, ci si può permettere di perdere (misurato in tempo — es. “non più di 1 ora di dati persi”) in caso di ripristino da backup.

Per un PM, sapere che backup e DR non sono un dettaglio tecnico invisibile ma **una scelta con un costo e un livello di rischio accettato** è importante: un cliente che chiede “in quanto tempo torniamo operativi se tutto si ferma?” sta facendo una domanda di business legittima, a cui il team tecnico deve poter rispondere con numeri concreti (gli RTO/RPO concordati), non con un generico “dovremmo farcela”.

Esempio pratico: il database ordini di ShopFacile subisce un guasto e diventa inaccessibile alle 14:00 di un giorno lavorativo, proprio durante i saldi. Luca, come Scrum Master, coordina la risposta all'incidente: raccoglie chi serve, tiene aggiornato il resto del team e fa da punto di contatto con Sara per le comunicazioni verso il cliente, mentre Marco lavora al ripristino tecnico. ShopFacile ha concordato un RTO di 2 ore, quindi il team ha l'obiettivo di ripristinare il servizio entro le 16:00. Se l'RPO concordato è di 1 ora, e l'ultimo backup automatico risale alle 13:30, il team sa già, prima ancora di iniziare il ripristino, che andranno persi al massimo 30 minuti di ordini recenti — un'informazione che Luca può comunicare subito, invece di scoprirla a ripristino concluso. Senza questi numeri concordati in anticipo, la stessa situazione genera solo incertezza e domande a cui nessuno sa rispondere con precisione.

13.11 Business Continuity: cosa fa l'azienda mentre i sistemi sono giù

Il piano di Disaster Recovery di ShopFacile, alla prova dei fatti, **funziona**: il database ordini viene ripristinato entro le due ore concordate, esattamente come previsto. Il problema è quello che succede **prima** che il ripristino sia completo. Per sei ore — perché stavolta il guasto è più serio del solito, e il ripristino richiede più tempo del solito — i venditori che lavorano con ShopFacile non possono confermare un ordine, e il call center riceve chiamate di clienti che chiedono aiuto senza che nessuno abbia detto loro cosa rispondere. Nessun sistema, in queste sei ore, è “in errore” nel senso tecnico: il DR sta facendo esattamente il suo lavoro. Quello che manca è un piano per le **persone** — per i venditori, per il call center, per i clienti — mentre i sistemi sono fermi. È questo il confine tra Disaster Recovery e **Business Continuity (BC)**: il DR risponde alla domanda “come rimetto in piedi i sistemi?”; la Business Continuity risponde a una domanda diversa, “come continua a funzionare **l'azienda** mentre i sistemi non ci sono?”.

Gli ingredienti principali di un piano di Business Continuity:

- **BIA (Business Impact Analysis):** l'analisi che stima, processo per processo, quanto costa un'ora di fermo — non in senso tecnico, in senso di business: quanti ordini non confermati, quanti clienti persi, quale obbligo normativo non rispettato. È questo numero, e non una preferenza tecnica, a determinare quale RTO e quale RPO ha senso concordare per ciascun sistema (visti al paragrafo 13.10): un processo che costa poco stare fermo un giorno può

accettare un RTO di ore; un processo che costa moltissimo ogni minuto di fermo giustifica un investimento ben più alto in ridondanza. È il collegamento più importante di questa sottosezione: **i numeri di RTO e RPO sono una decisione di business**, presa sulla base della BIA, non una scelta tecnica lasciata al team IT.

- **Processi critici e procedure alternative:** cosa fa il call center quando il sistema è giù — un copione da seguire, un modulo cartaceo per registrare a mano una richiesta urgente, un numero di emergenza da dare al cliente. Sono procedure manuali, spesso “vecchio stile” apposta, pensate proprio per i momenti in cui la tecnologia non è disponibile.
- **Comunicazione di crisi e catena di comando:** chi decide cosa dire ai clienti, chi lo dice, e chi ha l'autorità di prendere decisioni senza aspettare l'approvazione di qualcuno che magari, in quel momento, non è raggiungibile.
- **Il test**, con una verità scomoda: un piano di Business Continuity mai provato è un **documento**, non un piano. E i piani, quando vengono finalmente testati (o messi alla prova da un incidente vero), falliscono quasi sempre su dettagli banali: la lista dei numeri di telefono del call center non è aggiornata da un anno, l'unica persona che conosce a memoria la procedura manuale è in ferie quella settimana, la password del sistema di emergenza è scaduta e nessuno se n'era accorto perché non la usa mai nessuno.

	Disaster Recovery	Business Continuity
Obiettivo	Ripristinare sistemi e dati	Far continuare a operare l'azienda
Ambito	Infrastruttura tecnica (server, database, rete)	Processi, persone, comunicazione
Chi lo redige	Team IT/infrastruttura	Business, con il supporto dell'IT
Chi lo attiva	Team tecnico, al momento del guasto	Management/crisis team, insieme al tecnico
Come si misura	RTO, RPO	Tempo di continuità operativa accettabile per processo, definito dalla BIA
Esempio	Ripristinare il database ordini in 2 ore	Il call center segue un copione manuale mentre il database è giù

Come per il DR, anche la continuità operativa **costa in proporzione a quanto la si vuole**: una ridondanza completa (uffici alternativi, sistemi di emergenza sempre pronti, personale duplicato) raddoppia i costi e va giustificata processo per processo con la BIA, non applicata indiscriminatamente a tutto. C'è poi un costo nascosto, meno visibile ma altrettanto reale: il piano va **mantenuto** nel tempo — numeri di telefono, procedure, responsabili — altrimenti, il giorno in cui serve, descrive un'azienda che non esiste più.

13.12 Riepilogo

In questa sezione hai visto i concetti di base della sicurezza informatica utili al tuo ruolo di PM/Scrum Master:

- la sicurezza non è solo un tema tecnico: ha impatti diretti su costi, reputazione, contratti e conformità legale;
- **autenticazione** (chi sei) e **autorizzazione** (cosa puoi fare) sono due controlli distinti e sequenziali;
- un' **Active Directory** (o il suo equivalente cloud, Entra ID) è la fonte unica delle identità aziendali; il **Single Sign-On** unifica l'autenticazione su più applicazioni; il **token** è il "biglietto" temporaneo che sostituisce la password ad ogni richiesta; **OAuth** delega un permesso limitato senza cedere la password, e **OpenID Connect** ci aggiunge sopra l'autenticazione;
- password forti, gestori di password e **MFA** sono le difese di base contro il tipo di incidente più comune: il furto di credenziali;
- **HTTPS** cifra i dati in transito; i dati "a riposo" (salvati) vanno cifrati a loro volta;
- vulnerabilità come **SQL Injection** e **XSS** sono categorie note e ricorrenti di errori nel codice, motivo per cui il software va testato anche dal punto di vista della sicurezza;
- la **OWASP Top 10** è il riferimento standard del settore per le vulnerabilità più critiche nelle applicazioni web;
- il **DevSecOps** integra la sicurezza fin dall'inizio del processo di sviluppo, con il principio dello "shift left";
- gli **scan di sicurezza automatici** (SAST, SCA, DAST) si inseriscono nella pipeline CI/CD come quality gate, esattamente come i test funzionali;
- il **GDPR** impone obblighi legali precisi sulla protezione dei dati personali, con conseguenze concrete in caso di violazione;
- **backup e disaster recovery** sono il piano B necessario quando, nonostante tutte le precauzioni, qualcosa va comunque storto;
- la **Business Continuity** è diversa dal Disaster Recovery: il DR rimette in piedi i sistemi, la BC fa continuare a funzionare l'azienda mentre i sistemi non ci sono — e la **Business Impact Analysis** è ciò che trasforma RTO e RPO da numeri tecnici a decisioni di business.

Nella prossima sezione vedrai come questi concetti si traducono in scelte concrete di configurazione degli ambienti di sviluppo, test, staging e produzione.

Checklist di autoverifica

Prima di passare alla sezione successiva, prova a rispondere (anche solo a voce, o scrivendo due righe) a queste domande:

- Sai spiegare la differenza tra autenticazione e autorizzazione con l'analogia del cinema?
- Sai spiegare cos'è un token e perché la sua scadenza breve è una protezione, non un fastidio?
- Sai spiegare la differenza tra OAuth e OpenID Connect con un esempio concreto (un'app che deve leggere i tuoi documenti per conto tuo)?
- Sai spiegare perché il Single Sign-On è insieme comodo e rischioso?

- Sai spiegare a cosa serve l'MFA e perché rafforza la sola password?
- Sai spiegare, con parole tue, cosa significa “dato cifrato”?
- Sapresti spiegare a un collega non tecnico perché il codice va testato anche per la sicurezza, non solo per la correttezza funzionale?
- Sai spiegare cos'è la OWASP Top 10 e perché viene citata nei contratti?
- Sai spiegare il principio dello “shift left” nel DevSecOps?
- Sai collegare gli scan SAST/SCA/DAST al concetto di quality gate visto nella sezione CI/CD?
- Sai spiegare perché il GDPR riguarda anche le scelte di un PM, non solo i legali?
- Sai spiegare la differenza tra backup e disaster recovery?
- Sai spiegare la differenza tra Disaster Recovery e Business Continuity, e perché la Business Impact Analysis determina RTO e RPO?



Esercizi pratici

1. **Mappa autenticazione/autorizzazione sul progetto:** chiedi a un collega developer come funziona il login nell'applicazione del progetto e quali ruoli/permessi esistono (es. “utente base”, “amministratore”). Disegna uno schema semplice (anche a mano) con almeno due ruoli diversi e cosa ciascuno può o non può fare. **Come verificare:** sei in grado di indicare almeno un'azione che un ruolo può fare e un altro no, usando esempi reali del progetto (non inventati) e senza confondere “chi è” con “cosa può fare”.
2. **Controlla l'MFA sui tuoi strumenti di lavoro:** verifica se hai l'autenticazione a più fattori attiva su tutti gli strumenti che usi per lavoro (email aziendale, GitHub, VPN). Se manca su qualcuno, attivala (o chiedi all'IT come fare). **Come verificare:** puoi elencare, per ogni strumento di lavoro che usi, se l'MFA è attiva sì/no, e sai spiegare a un collega perché è importante che lo sia anche se la password è “forte”.
3. **Mappa le tue identità digitali di lavoro:** elenca tutti gli strumenti a cui accedi per lavoro (email, GitHub, VPN, gestionale, cloud...) e verifica quali passano da un Single Sign-On aziendale e quali richiedono ancora un login separato. Per almeno un'applicazione che usa il pulsante “accedi con...”, individua quale permesso OAuth le hai concesso. **Come verificare:** sai indicare quanti “punti di accesso separati” avresti se il SSO non esistesse, e sai spiegare a un collega cosa succederebbe ai tuoi accessi se la tua identità centrale venisse disattivata.
4. **Riconosci una vulnerabilità in un caso pratico:** leggi la descrizione di una vulnerabilità reale (puoi cercarne una recente sul sito OWASP o in una news di settore, senza bisogno di capirne i dettagli tecnici) e prova a classificarla: assomiglia più a una SQL Injection, a una XSS, o a un'altra categoria? Scrivi due righe su cosa avrebbe potuto fare un attaccante se quella vulnerabilità non fosse stata corretta. **Come verificare:** sai spiegare, con parole tue e senza gergo tecnico eccessivo, cosa rischiava di succedere e perché “testare anche per la sicurezza” avrebbe potuto prevenirlo.
5. **Trova gli scan di sicurezza nella pipeline reale:** chiedi a un collega (developer o DevOps engineer) di farti vedere, nella pipeline CI/CD del progetto, dove sono configurati gli scan di sicurezza (SAST, SCA, o eventualmente DAST) e cosa succede quando trovano una

vulnerabilità critica.  **Come verificare:** sai indicare in quale fase della pipeline si trova ciascun tipo di scan, e sai descrivere concretamente cosa vede il team quando un quality gate di sicurezza blocca un rilascio (una notifica? un'email? il rilascio che semplicemente non parte?).

6. **Simula una richiesta del cliente sul GDPR:** immagina che il cliente chieda “per quanto tempo conservate i dati personali degli utenti dopo che chiudono l'account, e chi può accedervi?”. Prova a scrivere, in 5-6 righe, come strutturaresti la risposta (anche senza conoscere i dettagli tecnici esatti del progetto — l'obiettivo è la struttura del ragionamento, non il dato preciso).  **Come verificare:** la tua risposta tocca sia l'aspetto tecnico (dove e come sono protetti i dati) sia l'aspetto di processo (chi decide i tempi di conservazione, chi ha accesso), e non promette nulla che non puoi verificare con il team.
7. **Chiedi RTO e RPO reali del progetto:** chiedi alla tua collega Scrum Master/PM o a un membro del team operations quali sono, se definiti, l'RTO e l'RPO concordati per l'ambiente di produzione del progetto (anche se la risposta è “non sono ancora stati definiti formalmente” — è comunque un'informazione utile).  **Come verificare:** sai riportare a un collega, con le tue parole, cosa significano quei due numeri specifici (non la definizione generica del libro) applicati al contesto reale del progetto.
8. **Distingui DR e BC su uno scenario reale:** pensa a un possibile incidente nel tuo progetto (o immaginane uno plausibile) e descrivi, in due elenchi separati, cosa farebbe il piano di Disaster Recovery (sistemi) e cosa dovrebbe fare un piano di Business Continuity (persone e processi) nelle stesse ore.  **Come verificare:** il tuo elenco “Business Continuity” non contiene nessuna azione tecnica sui sistemi (ripristini, failover) — se ne trovi una, appartiene al DR, non alla BC.

Collegamenti

- [14. Ambienti di sviluppo](#) — come si configurano in pratica accessi, credenziali e isolamento tra Dev, Test, Staging e Produzione
- [16. Glossario](#) — per ripassare rapidamente ogni termine visto in questa sezione

Risorse

- [OWASP Top 10](#) — la classifica ufficiale delle vulnerabilità più critiche per le applicazioni web
- [OWASP Foundation — sito ufficiale](#) — panoramica su progetti, guide e risorse gratuite in ambito sicurezza applicativa
- [NCSC \(National Cyber Security Centre, UK\) — Cyber security basics](#) — guide pratiche e accessibili su password, MFA e igiene informatica di base
- [Microsoft Learn — Security concepts](#) — introduzione ai principi moderni di sicurezza (Zero Trust) applicati al cloud
- [Microsoft Learn — Cos'è il DevSecOps](#) — approfondimento su come la sicurezza si integra nel ciclo DevOps
- [Garante per la protezione dei dati personali — Cos'è il GDPR](#) — riferimento istituzionale italiano sul regolamento europeo

- [Microsoft Learn — Cos'è Microsoft Entra ID \(Azure AD\)](#) — introduzione a directory, SSO e identità nel cloud
- [OAuth.net — An Introduction to OAuth 2.0](#) — panoramica non tecnica sul funzionamento della delega OAuth
- [OpenID Foundation — What is OpenID Connect](#) — chiarisce la differenza tra OAuth (autorizzazione) e OIDC (autenticazione)
- [Ready.gov — Business Continuity Planning](#) — guida pratica e istituzionale (in inglese) alla pianificazione della continuità operativa

14. Ambienti di sviluppo

Nella sezione [10. CI/CD](#) hai già incontrato il concetto di **ambiente**: sai che il codice non passa direttamente dal computer di uno sviluppatore agli utenti finali, ma attraversa diverse “tappe” (Dev, Test/QA, Staging, Produzione) prima di arrivare al traguardo. In questa sezione ci fermiamo su ciascuna di queste tappe, una per una, per capire **cosa succede davvero dentro ognuna di esse**, chi ci lavora, che dati si usano e perché la disciplina attorno a questo tema è tutt'altro che un dettaglio tecnico secondario. Seguiremo, tappa per tappa, un unico bug reale del carrello di ShopFacile: vedrai lo stesso identico problema attraversare le quattro tappe, dalla scrivania di Marco fino alla produzione.

Obiettivi della sezione

Alla fine di questa sezione saprai:

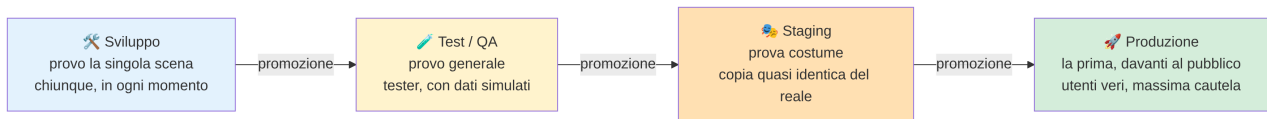
- spiegare perché non si testa mai direttamente in produzione;
- descrivere lo scopo specifico di ognuno dei quattro ambienti tipici (Dev, Test/QA, Staging, Produzione);
- capire perché gli ambienti devono essere quanto più simili possibile tra loro, ed evitare il classico “funziona sul mio computer”;
- distinguere dati di test da dati reali, e sapere perché non si usano mai dati sensibili reali fuori da produzione;
- capire cos'è una configurazione per ambiente (es. l'indirizzo di un database diverso per ogni ambiente);
- descrivere perché l'accesso alla produzione è, di norma, molto più restrittivo di quello agli altri ambienti.

14.1 Perché servire diversi ambienti: la “prova costume”

Immagina uno spettacolo teatrale che debutta davanti al pubblico pagante la prima serata, senza mai aver fatto una prova. Nessun regista serio lo farebbe: prima ci sono le **provo generali**, poi la **prova costume** (con luci, scenografia e costumi veri, ma senza pubblico), e solo alla fine la **prima** davanti agli spettatori. Ogni tappa serve a scoprire un problema diverso — un attore che sbaglia una battuta, un cambio scena troppo lento, un microfono che non funziona — in un contesto dove sbagliare **non costa nulla**, invece di scoprirlo in diretta con il pubblico in sala.

Il software funziona esattamente allo stesso modo. **Non si testa mai direttamente in produzione** per un motivo semplicissimo: in produzione ci sono utenti reali, che stanno usando il software per lavorare, comprare, prenotare, pagare. Se un test fallisce lì, l’“errore” non è un rigo rosso in un log: è un utente che perde dei dati, una transazione che non va a buon fine, un cliente che chiama il servizio assistenza arrabbiato.

Per questo si costruisce una **catena di ambienti**, ciascuno con un livello crescente di somiglianza alla produzione e un livello crescente di cautela richiesta, esattamente come le prove di uno spettacolo prima del debutto.



14.2 Ambiente di Sviluppo (Dev)

L'ambiente di **Sviluppo** è dove nasce il codice. Ogni sviluppatore scrive e prova le proprie modifiche, quasi sempre in due modalità complementari:

- **In locale**, sul proprio computer: l'applicazione gira dentro il suo portatile, con dati finti creati al momento, senza toccare nulla di condiviso con il resto del team.
- **In un ambiente Dev condiviso**, in cloud: una versione dell'applicazione sempre aggiornata con l'ultimo codice del branch principale, usata per verificare che le proprie modifiche funzionino anche insieme a quelle degli altri, prima ancora di arrivare al Test/QA.

Chi ci accede: gli sviluppatori, in modo ampio e informale.

Cosa succede qui: è normale — anzi previsto — che qualcosa non funzioni. È lavoro in corso, come una scena provata da capo più volte prima di essere pronta. Non c'è alcun tipo di garanzia di stabilità: l'ambiente Dev può "rompersi" più volte al giorno senza che questo sia un problema per nessuno fuori dal team di sviluppo.

 **Esempio pratico:** Marco deve correggere un bug del carrello di ShopFacile: quando un cliente applica un codice sconto, il totale mostrato a video non tiene conto dell'IVA ricalcolata, e risulta più basso di quanto dovrebbe. Scrive il codice della correzione e lo prova prima **in locale**, sul suo computer, con dati completamente inventati (un carrello finto con prodotti fittizi e un codice sconto di prova). Quando è abbastanza sicuro del risultato, pusha il proprio branch: la stessa modifica viene automaticamente distribuita anche nell'**ambiente Dev condiviso** in cloud, dove i colleghi possono verificare che funzioni insieme al resto dell'applicazione, ancora prima che Marco apra la Pull Request.

14.3 Ambiente di Test / QA

Una volta che la correzione di Marco supera i controlli automatici della pipeline (build, unit test, analisi di qualità — visti nella sezione CI/CD), viene **promossa** nell'ambiente di **Test/QA** (Quality Assurance, ovvero "garanzia di qualità").

Qui il software viene verificato in modo più realistico:

- test automatici più ampi (test di integrazione, test end-to-end);
- verifiche manuali da parte del team di test, che prova il software come farebbe un utente, cercando deliberatamente di romperlo;

- prime verifiche di casi limite: cosa succede se inserisco un valore strano, se clicco due volte sullo stesso bottone, se la connessione si interrompe a metà di un'operazione.

Chi ci accede: il team di test/QA, gli sviluppatori per investigare eventuali segnalazioni, occasionalmente il Project Manager per una demo interna.

Cosa succede qui: si scopre e si corregge la maggior parte dei problemi funzionali, **prima** che possano avvicinarsi anche solo lontanamente agli utenti reali. Se qualcosa non funziona, il codice **non viene promosso** oltre: torna in Sviluppo per essere corretto.

 **Esempio pratico:** la correzione di Marco al totale del carrello supera la build e i test automatici della pipeline, e viene promossa in Test/QA. Qui Giulia prova deliberatamente a “romperla”: applica due codici sconto diversi allo stesso carrello, prova uno sconto su un prodotto già scontato di suo, svuota il carrello a metà del ricalcolo. Scopre che, combinando due sconti insieme, il totale torna a essere sbagliato in un modo nuovo: apre una segnalazione, e il codice **torna** in Sviluppo per la correzione, prima di poter proseguire oltre.

14.4 Ambiente di Staging (Pre-produzione)

Superato anche il secondo giro di test, la correzione del carrello è pronta per un controllo ancora più realistico, in un ambiente che assomiglia quanto più possibile alla produzione vera. Lo **Staging** (a volte chiamato Pre-produzione) è l'ambiente pensato per essere **quanto più fedele possibile alla produzione**: stessa configurazione di infrastruttura, stesso tipo di database, volumi di dati paragonabili, stesse integrazioni con sistemi esterni (per quanto possibile in modo sicuro). È la “prova costume”: tutto è allestito come sarà la sera del debutto, ma senza pubblico pagante in sala.

Qui si eseguono gli ultimi controlli prima del rilascio reale:

- test di accettazione finali, spesso con il coinvolgimento di chi ha richiesto la funzionalità (il Project Manager, un referente del progetto);
- a volte test di performance/carico, per verificare che l'applicazione regga un numero di utenti realistico;
- verifica che il processo di deploy stesso funzioni senza sorprese, proprio perché l'ambiente è configurato come la produzione.

Chi ci accede: team di test/QA, sviluppatori senior o Tech Lead, Project Manager per l'ultima validazione, in modo più controllato rispetto al Test/QA.

Cosa succede qui: se qualcosa emerge in Staging che non era emerso prima, è un segnale importante — spesso significa che Test/QA non era sufficientemente simile alla produzione, ed è un'occasione per migliorare quell'ambiente. Non tutti i progetti hanno uno Staging separato dal Test/QA: nei progetti più piccoli le due cose a volte coincidono, ma nei progetti maturi restano due tappe distinte, con scopi diversi.

Il motivo per cui non tutti i progetti hanno quattro ambienti separati è il costo: più ambienti significano più infrastruttura (risorse cloud proprie per ciascuno, viste nella sezione 12), più licenze e più tempo del team per mantenerli allineati. Un progetto piccolo può scegliere consapevolmente di far coincidere Test/QA e Staging. È una decisione che un PM supporta spesso in prima persona: quando arriva una correzione urgente e qualcuno chiede “possiamo saltare lo Staging per questa volta?”, la risposta giusta non è un automatico sì né un automatico no, ma un compromesso tempo/rischio da valutare caso per caso.

 **Esempio pratico:** la correzione al totale del carrello, dopo aver superato il Test/QA, viene promossa in Staging. Qui il volume di dati è paragonabile a quello reale (migliaia di ordini storici anonimizzati, non le tre righe di prova usate in Test), e l'infrastruttura è configurata esattamente come in produzione. Sara, come Product Owner che aveva segnalato il problema arrivato dai clienti, prova il flusso completo end-to-end — carrello, sconto, checkout — prima di dare l'ultima validazione. Ahmed segue il test insieme a lei: è il primo bug che vede attraversare tutte le tappe fino a qui, ed è un buon modo per imparare cosa cambia da un ambiente all'altro. Solo a questo punto il rilascio in produzione può essere pianificato.

14.5 Ambiente di Produzione

Dopo Dev, Test/QA e Staging, resta un'ultima tappa, quella in cui il bug del carrello incontra finalmente i clienti veri di ShopFacile. La **Produzione** è l'ambiente reale, quello che usano davvero gli utenti finali — che siano dipendenti del cliente, clienti finali di un servizio, o cittadini che usano un servizio pubblico. È la “prima” davanti al pubblico pagante.

Qui la parola d'ordine è **massima cautela**:

- ogni deploy segue i quality gate e, tipicamente, un'approvazione manuale (vista nella sezione CI/CD) — spesso data da un **Release Manager**, cioè la persona (o il ruolo) responsabile di dare il via libera finale a un rilascio in produzione, verificando che tutti i controlli precedenti siano stati superati prima di autorizzarlo;
- ogni modifica è monitorata attentamente subito dopo il rilascio (metriche di errore, tempi di risposta);
- esiste sempre un piano di rollback pronto, nel caso qualcosa vada storto nonostante tutti i controlli precedenti.

Chi ci accede: un numero molto ristretto di persone, quasi sempre solo tramite strumenti e processi automatizzati (la pipeline stessa), raramente con accesso manuale diretto — approfondiremo il perché nel paragrafo 14.10.

Cosa succede qui: ogni errore ha un impatto reale su persone vere. Non è più “lavoro in corso”: è il prodotto finito, in uso.

Rollback: tornare indietro, ma non sempre su tutto

Vale la pena capire cosa significa davvero “un piano di rollback pronto”, un’informazione utile se un giorno dovrai comunicare tempi durante un incidente. Tornare alla versione precedente del **codice** è di norma semplice: grazie a “build once, deploy many” (che vedrai tra poco), basta rimettere in esecuzione l’artifact precedente, già verificato in tutte le tappe.

Tornare indietro sui **dati**, invece, spesso non lo è: se il rilascio ha anche modificato la struttura del database, il rollback del codice da solo non basta, e certe modifiche possono essere **irreversibili**. Per questo, davanti a un incidente, la domanda giusta non è solo “quanto ci vuole per tornare al codice precedente?” ma anche “il rollback coinvolge anche i dati?” — le due risposte possono avere tempi molto diversi.

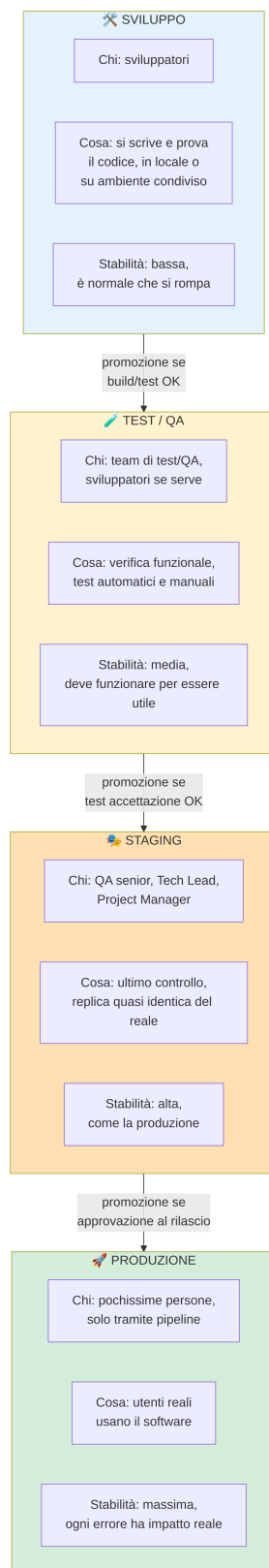
 **Esempio pratico — il percorso completo di una modifica:** seguiamo l’intero viaggio della correzione al totale del carrello, dal commit alla produzione:

1. **Dev:** Marco corregge il calcolo dell’IVA con lo sconto applicato e lo prova in locale, poi pusha il branch: la pipeline fa build e unit test con esito positivo.
2. **Test/QA:** la modifica viene promossa automaticamente. Giulia riprova esattamente lo scenario dei due sconti combinati, più altri casi limite: questa volta tutto funziona come previsto. Promozione approvata.
3. **Staging:** la stessa modifica (lo stesso identico pacchetto, non una ricompilazione) viene distribuita in un ambiente identico alla produzione. Sara verifica il flusso end-to-end con dati realistici e dà l’ultima validazione.
4. **Produzione:** dopo un’approvazione manuale del Release Manager, la pipeline distribuisce la modifica ai clienti reali, fuori dall’orario di punta. Nei minuti successivi, il team monitora le metriche di errore: se qualcosa andasse storto, è pronto un piano di rollback.

Da notare: è **lo stesso pacchetto** che attraversa tutte le tappe, non una versione “ricostruita” ambiente per ambiente — esattamente il principio “build once, deploy many” che vedrai nel paragrafo 15.7.

14.6 I quattro ambienti, fase per fase

Il viaggio della correzione al carrello, visto passo per passo nel box precedente, si può anche osservare “da fermo”, guardando in un unico schema chi lavora in ciascuna tappa e cosa cambia tra l’una e l’altra. Il diagramma seguente riprende e amplia quello già visto nella sezione CI/CD, aggiungendo per ciascun ambiente **chi vi accede** e **cosa succede concretamente**.



14.7 “Funziona sul mio computer”: perché gli ambienti devono somigliarsi

Il percorso della correzione al carrello ha funzionato senza sorprese proprio perché Dev, Test/QA, Staging e Produzione si somigliano il più possibile. Vediamo perché questa somiglianza non è un dettaglio, ma il punto centrale di tutta la catena. Conosci già, dalla sezione 2, il problema del “funziona sul mio computer” (una versione diversa di una libreria, un sistema operativo diverso, dati o configurazioni differenti) e, dalla sezione 10, la soluzione — **container** e principio “build once, deploy many”: si pacchettizza il software una sola volta, e lo stesso identico artifact viene promosso, senza modifiche, da un ambiente all'altro.

Quello che aggiungiamo qui è come questo stesso principio si applica proprio al **passaggio tra ambienti**: se qualcosa funziona in Staging, è ragionevole aspettarsi che funzioni anche in Produzione, perché tecnicamente **è lo stesso identico pacchetto**, non una ricostruzione “simile” costruita ambiente per ambiente. Più gli ambienti sono simili tra loro (stessa infrastruttura, stesso artifact, differenze solo nella configurazione che vedrai al paragrafo 14.9), meno sorprese si trovano man mano che si avanza nella catena — ed è per questo che lo Staging, in particolare, è tenuto quanto più possibile identico alla Produzione.

 **Esempio pratico:** in ShopFacile, ogni volta che una modifica passa la fase di build in pipeline, viene generato un **unico** artifact (es. un'immagine container con un numero di versione, tipo `shopfacile-carrello:1.42.0`). Quello stesso artifact — non uno ricostruito da capo — viene distribuito in Test/QA, poi in Staging, poi in Produzione. Se in Staging l'immagine `1.42.0` funziona correttamente, il team sa che in Produzione girerà **esattamente lo stesso codice, con le stesse dipendenze**: non resta da chiedersi “avranno usato la stessa versione di libreria X?”, perché la domanda non ha nemmeno senso — è letteralmente lo stesso file.

14.8 Dati di test vs dati reali

Sapere che l'artifact è sempre lo stesso risolve il problema del codice, ma resta un'altra domanda: se il codice è identico, i dati che usa devono esserlo altrettanto? Qui la risposta cambia radicalmente. Un punto spesso sottovalutato da chi arriva da fuori dal settore: **gli ambienti di Sviluppo e Test/QA non usano mai dati reali degli utenti**.

Perché? Perché i dati reali possono contenere informazioni sensibili: nomi, indirizzi, numeri di documento, dati di pagamento, informazioni sanitarie. Usarli fuori da un ambiente protetto come la Produzione significherebbe esporli a un numero molto più ampio di persone (tutto il team di sviluppo e test) e a controlli di sicurezza tipicamente meno stringenti — un rischio enorme di violazione della privacy, oltre che, in molti contesti, una vera e propria violazione di legge.

Per questo si usano invece:

- **dati sintetici**: generati appositamente, plausibili ma inventati (nomi, indirizzi ed email finti ma verosimili);

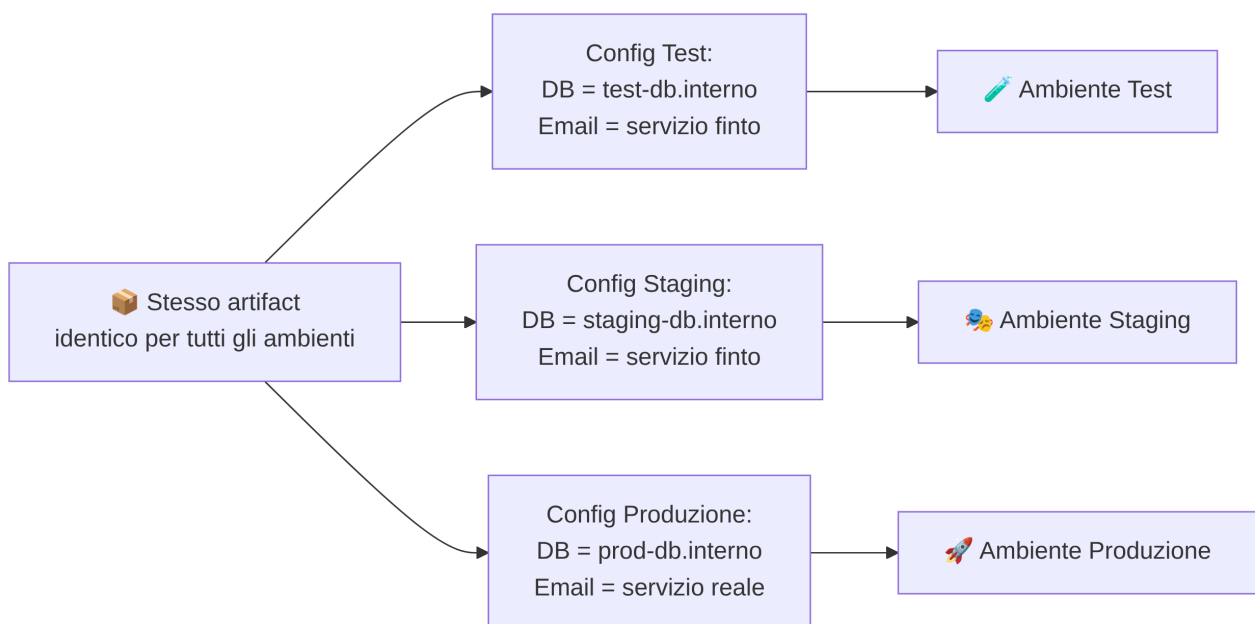
- **dati anonimizzati o pseudonimizzati**: partono da dati reali, ma privati di ogni elemento che permetterebbe di risalire a una persona specifica.

Questo tema è collegato a doppio filo con quanto visto nella sezione [13. Sicurezza](#): la protezione dei dati personali non è solo una questione di “chi può leggerli in produzione”, ma anche di “dove questi dati non devono mai nemmeno arrivare”.

14.9 Configurazioni per ambiente

Dati diversi non bastano da soli: se lo stesso artifact deve girare su dati e infrastrutture diverse in ogni ambiente, deve anche sapere in quale ambiente si trova in un dato momento. Lo stesso identico artifact (lo stesso pacchetto di codice) deve comportarsi in modo leggermente diverso a seconda dell'ambiente in cui gira: in Test deve collegarsi al database di test, in Produzione al database di produzione; in Test può usare un servizio di invio email “finto” che non spedisce nulla, in Produzione deve usare il servizio vero.

La soluzione **non è modificare il codice** per ogni ambiente (violerebbe il principio “build once, deploy many” visto sopra), ma usare delle **variabili di configurazione** (o variabili d'ambiente): valori esterni al codice, forniti al momento dell'avvio, che dicono all'applicazione “in questo momento stai girando in Test, quindi usa questo indirizzo di database” oppure “stai girando in Produzione, quindi usa quest'altro”.



Le credenziali (password, chiavi di accesso) che fanno parte di queste configurazioni non vengono mai scritte nel codice: sono gestite tramite strumenti dedicati, spesso chiamati **vault** (letteralmente “caveau”) di segreti — sistemi pensati apposta per custodire password e chiavi in modo cifrato, concedendo l'accesso solo a chi (o a cosa, es. la pipeline stessa) ne ha davvero bisogno, collegandosi ancora al tema della sicurezza approfondito nella sezione dedicata.

 **Esempio pratico:** ShopFacile deve inviare un'email di conferma quando un ordine viene completato. In Test/QA, la variabile di configurazione `EMAIL_SERVICE_URL` punta a un servizio "finto" che si limita a registrare in un log "avrei inviato questa email", senza spedire nulla per davvero — utile per verificare che il codice tenti l'invio, senza intasare le caselle di posta di nessuno con conferme d'ordine finte. In Produzione, la stessa identica variabile punta invece al servizio email reale, quello che avvisa davvero il cliente. Il codice che gestisce l'invio non cambia di una riga tra i due ambienti: cambia solo il valore di quella variabile.

Deriva di configurazione: quando gli ambienti si disallineano in silenzio

Le configurazioni per ambiente funzionano solo finché qualcuno le mantiene allineate. Qualche tempo dopo aver cambiato fornitore per l'invio email in produzione, il team di ShopFacile scopre per caso che in Staging la variabile `EMAIL_SERVICE_URL` punta ancora al **vecchio** fornitore: qualcuno aveva aggiornato la produzione scordandosi di Staging. Nel frattempo, ogni test in Staging che coinvolgeva l'email continuava a "passare" — senza verificare nulla sul comportamento reale.

Questo fenomeno si chiama **deriva di configurazione** (*configuration drift*): gli ambienti tendono a **divergere silenziosamente** nel tempo se nessuno se ne occupa attivamente. Riallineare gli ambienti richiede un lavoro esplicito: accorgersi di una deriva dovrebbe tradursi in un elemento di backlog concreto, non restare un problema noto che si riscopre per caso ogni volta.

14.10 Chi ha accesso a quali ambienti

La regola che stai per leggere, in ShopFacile, non è nata da un manuale. Marco — la persona più esperta del team, non un principiante — teneva aperte contemporaneamente due connessioni al database: una su Test, una su Produzione. Esegue un comando per svuotare una tabella di prova, un comando che aveva già lanciato decine di volte senza conseguenze. Quel giorno, per distrazione, la finestra attiva era quella di **Produzione**: la tabella svuotata conteneva dati reali, e il ripristino dal backup ha richiesto ore, sotto pressione, con il sito in parte inutilizzabile. Nessuno aveva fatto niente di stupido: è esattamente questo il punto. Un errore banale e alla portata di chiunque non si previene chiedendo alle persone di stare più attente, ma **togliendo la possibilità di commetterlo**.

Hai visto chi lavora in ciascun ambiente lungo il percorso del bug del carrello: Marco in Dev, Giulia in Test/QA, Sara e Ahmed in Staging, un numero ristretto di persone in Produzione. Questo schema segue una regola precisa, nota come principio del **privilegio minimo**: ogni persona (o sistema) dovrebbe avere accesso solo a ciò che le serve davvero, non un accesso più ampio "per comodità". Applicato agli ambienti: **più un ambiente è vicino agli utenti reali, più l'accesso è restrittivo** — la stessa logica per cui non chiunque entra backstage la sera della prima. Se l'accesso a Produzione fosse limitato a poche persone tramite la sola pipeline, l'incidente sopra avrebbe avuto molte meno occasioni di verificarsi.

- **Sviluppo**: accesso ampio, quasi tutto il team tecnico.
- **Test/QA**: accesso al team di test/QA e agli sviluppatori, più limitato di Dev.

- **Staging**: accesso più ristretto, tipicamente ruoli senior (Tech Lead, QA lead) e il Project Manager per le validazioni finali.
- **Produzione**: accesso **molto** ristretto — spesso solo tramite la pipeline automatizzata stessa, con un numero minimo di persone autorizzate ad accedere manualmente in casi eccezionali (es. un'emergenza), e ogni accesso manuale tipicamente registrato e tracciato.



Confronto tra i quattro ambienti

Ambiente	Scopo	Chi lo usa	Tipo di dati	Stabilità richiesta	Frequenza di aggiornamento
Sviluppo (Dev)	Scrivere e provare codice nuovo	Sviluppatori	Dati finti, creati al momento	Bassa: è normale che si rompa	Molto alta, più volte al giorno
Test / QA	Verificare funzionalmente il software	Team di test/ QA, sviluppatori	Dati sintetici o anonimizzati, simili ai reali	Media: deve funzionare per essere utile	Alta, ad ogni promozione dalla CI
Staging (Pre-produzione)	Ultimo controllo prima del rilascio reale	QA senior, Tech Lead, Project Manager	Dati anonimizzati, volumi simili al reale	Alta: quasi come la produzione	Media, prima di ogni rilascio
Produzione	Uso reale da parte degli utenti finali	Utenti finali; accesso tecnico molto limitato	Dati reali e sensibili	Massima: ogni errore ha impatto reale	Bassa/ controllata, solo rilasci approvati

14.11 Riepilogo

Il bug del carrello di ShopFacile ci ha accompagnato da Marco in Dev fino al rilascio in Produzione: riassumiamo i concetti principali visti lungo il percorso. In questa sezione hai approfondito il concetto di ambiente già incontrato nella sezione CI/CD:

- non si testa mai direttamente in produzione: si passa attraverso Sviluppo, Test/QA e Staging, come le prove prima di uno spettacolo dal vivo;
- ogni ambiente ha un pubblico e uno scopo diversi, con un livello di cautela e stabilità crescente man mano che ci si avvicina alla produzione;
- gli ambienti devono somigliarsi quanto più possibile tra loro, per evitare il problema del “funziona sul mio computer”;
- non si usano mai dati reali e sensibili fuori dall'ambiente di produzione, per motivi di sicurezza e privacy;
- lo stesso pacchetto di codice cambia comportamento tra ambienti grazie a **variabili di configurazione**, non a modifiche del codice stesso;
- l'accesso agli ambienti è tanto più restrittivo quanto più ci si avvicina alla produzione.

Checklist di autoverifica

- Sapresti spiegare, con l'analogia delle prove di uno spettacolo, perché non si testa mai direttamente in produzione?
 - Sai descrivere lo scopo specifico dei quattro ambienti?
 - Sai spiegare “funziona sul mio computer” e come lo risolvono container e “build once, deploy many”?
 - Sai spiegare perché non si usano mai dati reali fuori da produzione?
 - Sai spiegare a cosa servono le variabili di configurazione, con un esempio, e cos'è la deriva di configurazione?
 - Sai spiegare perché l'accesso agli ambienti è tanto più restrittivo quanto più ci si avvicina alla produzione, collegandolo al privilegio minimo?
 - Sai spiegare la differenza tra rollback del codice e dei dati?
-

Esercizi pratici

1. **Mappa gli ambienti del progetto.** Con l'aiuto di un collega developer o del Tech Lead, scopri quanti e quali ambienti esistono davvero nel progetto (Dev, Test, Staging, Prod, o eventuali nomi diversi usati internamente) e scrivi in ordine, con una riga di descrizione per ciascuno.  **Come verificare:** sai disegnare, senza guardare gli appunti, la sequenza di ambienti del progetto con una freccia di “promozione” tra l'uno e l'altro, proprio come nei diagrammi di questa sezione.
2. **Segui una modifica reale attraverso gli ambienti.** Chiedi a uno sviluppatore di mostrarti (anche solo a schermo, senza dover intervenire) l'ultima modifica che ha promosso da Dev fino a Produzione: in quale ambiente si trova adesso, cosa ha superato per arrivare lì, cosa manca prima della produzione (se non è già arrivata).  **Come verificare:** sai raccontare a voce, in meno di due minuti, il percorso di quella specifica modifica citando ogni ambiente attraversato e cosa è stato verificato in ciascuno.
3. **Trova le variabili di configurazione per ambiente.** Chiedi a un developer di farti vedere (anche solo a schermo) dove sono definite le variabili di configurazione che cambiano tra un ambiente e l'altro nel progetto (es. l'indirizzo del database, l'URL di un servizio esterno). Non serve capire ogni dettaglio tecnico: l'obiettivo è vedere con i tuoi occhi che esistono e dove vivono.  **Come verificare:** sai indicare almeno due valori di configurazione che cambiano tra Test/QA e Produzione nel progetto, senza però conoscere (né dover conoscere) le password o i segreti veri.
4. **Distingui i dati usati nei diversi ambienti.** Chiedi al team se e come vengono generati o anonimizzati i dati usati in Test/QA e in Staging nel progetto (dati sintetici generati da uno script? dati reali anonimizzati? un mix?).  **Come verificare:** sai spiegare a un collega non tecnico perché non si può semplicemente “copiare” il database di produzione dentro l'ambiente di Test così com'è, citando almeno un rischio concreto.

5. **Osserva chi può fare cosa, dove.** Confronta la tabella di accesso agli ambienti vista nel paragrafo 14.10 con la situazione reale del progetto: chi ha accesso diretto a Dev? Chi può promuovere codice in Staging? Chi (o cosa, es. solo la pipeline) può rilasciare in Produzione?  **Come verificare:** sai indicare, per ciascuno dei quattro ambienti, almeno un ruolo del progetto che vi ha accesso e uno che invece **non** ce l'ha, spiegando perché.
6. **Simula un rollback.** Chiedi a un collega operations o Tech Lead di raccontarti (anche solo a parole, con un esempio ipotetico o reale se ce n'è uno) cosa succederebbe se, subito dopo un rilascio in Produzione, le metriche di errore iniziassero a salire in modo anomalo: chi se ne accorge, chi decide il rollback, quanto tempo richiede tipicamente tornare alla versione precedente.  **Come verificare:** sai descrivere, passo per passo, la sequenza di azioni tra “qualcosa va storto in produzione” e “il sistema torna stabile”, indicando almeno un ruolo coinvolto in ogni passo.
-

Collegamenti

- [10. CI/CD](#) — come la pipeline promuove il codice tra questi stessi ambienti
- [16. Glossario](#) — definizioni rapide di Dev, Staging, Produzione e termini correlati

Risorse

- [GitHub Docs](#) — Usare gli ambienti per il deployment
- [Atlassian](#) — What is a staging environment
- [Martin Fowler](#) — Bliki: DeploymentEnvironment
- [OWASP](#) — Data Protection e ambienti non di produzione
- [12 Factor App](#) — Config

15. Intelligenza artificiale

Nella sezione [14. Ambienti di sviluppo](#) hai seguito un bug del carrello di ShopFacile passo per passo, da Dev fino a Produzione, e hai visto quante persone e quanti controlli intervengono lungo il percorso: Marco che scrive il codice, Giulia che lo revisiona e lo testa, la pipeline che lo promuove da un ambiente all'altro. Negli ultimi anni, in quello stesso percorso, si è affacciato un nuovo tipo di “collaboratore”: non una persona, non uno strumento che esegue regole fisse, ma un software che **suggerisce** codice, **riformula** testi, **riassume** riunioni — a volte con risultati sorprendenti, a volte sbagliando con altrettanta sicurezza. Questa sezione ti dà il vocabolario e lo spirito critico per lavorare bene al fianco di questi strumenti, senza subirli né idealizzarli.

Non troverai qui un corso di data science: niente formule, niente matematica. L'obiettivo è che tu, come Junior Project Manager/Scrum Master, sappia **di cosa parla il team** quando dice “ho usato Copilot per generare i test” o “il modello di rilevamento anomalie ha segnalato un picco strano”, e che tu stessa o tu stesso sappia usare questi strumenti nel tuo lavoro quotidiano con consapevolezza dei loro limiti.

Obiettivi della sezione

Alla fine di questa sezione saprai:

- distinguere **intelligenza artificiale (AI)**, **machine learning (ML)** e **deep learning** come cerchi via via più piccoli, non come sinonimi intercambiabili;
- spiegare la differenza tra un software tradizionale (regole scritte da un umano) e un modello addestrato (che impara pattern dai dati);
- descrivere cosa significano **training** e **inferenza**, e la differenza tra apprendimento **supervisionato** e **non supervisionato**;
- riconoscere il concetto di **overfitting** con un'analogia semplice;
- spiegare cos'è un **LLM (Large Language Model)**, cosa vuol dire “AI generativa” e perché un modello può **“allucinare”** (inventare informazioni false in modo plausibile);
- spiegare cos'è il **RAG (Retrieval-Augmented Generation)** e perché è la tecnica che permette a un assistente AI di rispondere basandosi su documenti reali dell'azienda, invece di inventare o affidarsi solo a quanto imparato durante il training;
- spiegare cos'è **MCP (Model Context Protocol)** e la differenza tra un assistente che “conversa” e un assistente che può anche **usare strumenti e collegarsi a sistemi** (repository, ticket, database);
- riconoscere come l'AI si inserisce negli strumenti di sviluppo (es. **GitHub Copilot**) e perché il codice generato va comunque revisionato;
- riconoscere il termine **AIOps** e capire dove l'AI si inserisce nel ciclo di vita DevOps già visto nella sezione 9 e nella sezione 10;
- individuare almeno cinque usi concreti dell'AI nel lavoro quotidiano di un PM/Scrum Master, e i relativi limiti;
- riconoscere i principali **rischi** (allucinazioni, bias, privacy, proprietà intellettuale, dipendenza eccessiva) e collegarli a quanto già visto nella sezione 13 sulla sicurezza;
- avere un'idea di base di cosa sia **MLOps** e perché un modello, a differenza del software tradizionale, può **degradare nel tempo**;

- applicare qualche regola pratica di buon senso quando usi uno strumento di AI generativa nel tuo lavoro.

15.1 Cos'è l'intelligenza artificiale, in parole semplici

Il codice che hai seguito nella sezione 14 era scritto da una persona: ogni istruzione “se succede X allora fai Y” pensata e digitata da un umano — il modo in cui si è sempre costruito software. Ma il team di ShopFacile si è scontrato con un compito che questo approccio non riusciva più a gestire: riconoscere un ordine potenzialmente fraudolento.

Il primo tentativo è stato scrivere regole esplicite: “se l'importo supera 1000 euro e l'indirizzo di spedizione è diverso da quello di fatturazione, segnala come sospetto”. Funziona, per un po'. Poi arriva l'ordine legittimo di un cliente che fa un regalo: falso allarme, si aggiunge un'eccezione. Poi un nuovo schema di frode che nessuna regola copre, perché gli schemi cambiano ogni pochi mesi per eludere esattamente le regole scritte. Le eccezioni crescono senza fine, e il sistema continua a sbagliare — troppi criteri, troppo sfumati, che cambiano troppo per un elenco fisso di regole.

È per problemi così che è nato un approccio radicalmente diverso: invece di far scrivere le regole a un programmatore, si mostrano al sistema migliaia di esempi (ordini fraudolenti o no) e si lascia che sia lui a **ricavare gli schemi dagli esempi**, individuando da solo le caratteristiche che distinguono un ordine sospetto da uno legittimo. Questa idea — imparare dai dati invece di seguire istruzioni scritte a mano — è il cuore di ciò che va sotto il nome, molto generale, di intelligenza artificiale.

Intelligenza Artificiale (AI - Artificial Intelligence) è un termine molto ampio: indica qualsiasi sistema informatico capace di svolgere compiti che, se li facesse una persona, diremmo che richiedono “intelligenza” — riconoscere un'immagine, capire una frase, prendere una decisione in base a dati incerti. Dentro questo termine ampio esistono approcci molto diversi tra loro, e per orientarti ti servono soprattutto due sotto-categorie:

- **Machine Learning (ML)**, in italiano “apprendimento automatico”: è l'approccio appena descritto per il caso delle frodi — il sistema non segue regole scritte a mano, ma **impara pattern a partire da grandi quantità di dati**.
- **Deep Learning**: un sotto-insieme del machine learning che usa un tipo particolare di modello, le **reti neurali**, organizzate in molti “strati” (da cui *deep*, profondo). È l'approccio dietro ai risultati più sorprendenti degli ultimi anni, incluso il riconoscimento di immagini e i modelli di linguaggio che vedremo al paragrafo 15.3.

Il punto da fissare bene: **AI, ML e deep learning non sono sinonimi**, sono **cerchi concentrici**, uno dentro l'altro. Ogni rete neurale di deep learning è anche machine learning, ed è anche AI — ma non ogni sistema di AI è machine learning (esistono sistemi di AI più vecchi, basati su regole logiche scritte a mano, che nessuno chiamerebbe machine learning), e non ogni modello di machine learning è deep learning (esistono tecniche di ML più semplici, che non usano reti neurali).



💡 **Analogia:** è come “veicolo”, “automobile” e “auto elettrica” — ogni auto elettrica è un’automobile ed è anche un veicolo, ma non vale il contrario (una bicicletta è un veicolo, non un’automobile).

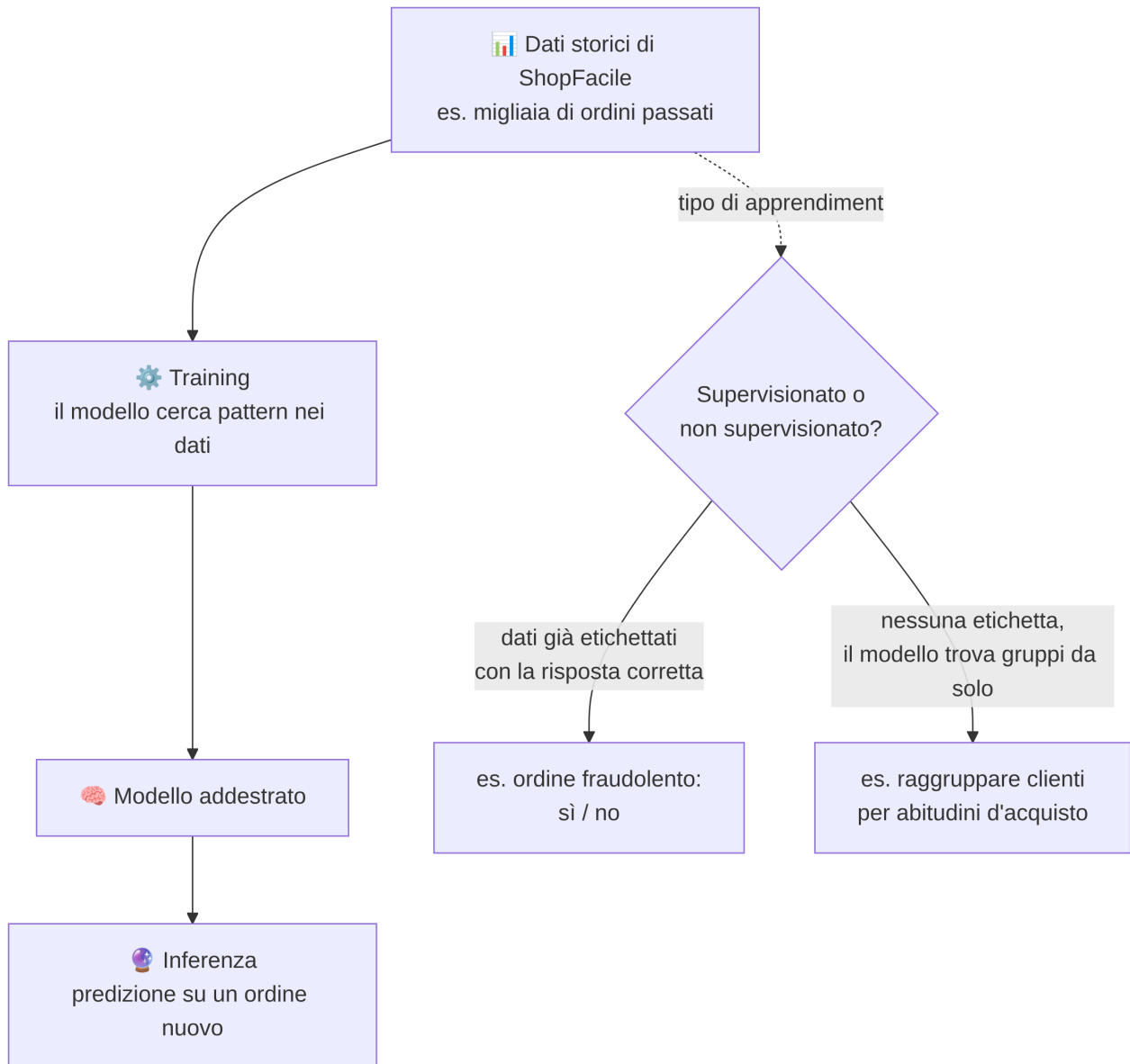
La differenza di fondo con il software “tradizionale” visto nelle sezioni precedenti del corso è quindi questa: un programma tradizionale esegue **esattamente** le regole che un umano ha scritto, senza modificarle mai da solo; un modello di machine learning **cambia il proprio comportamento in base ai dati** che gli vengono mostrati durante una fase chiamata training — il vero cuore del machine learning, ed è il prossimo argomento.

15.2 Machine learning: come “impara” un modello

Abbiamo detto che un modello di machine learning impara pattern dai dati invece di seguire regole scritte a mano. Ma cosa significa, in pratica, “imparare”? Il processo si divide in due fasi ben distinte, che è utile imparare a riconoscere per nome.

- **Training (addestramento):** si mostrano al modello grandi quantità di **dati di addestramento** (in inglese *training data*), e un algoritmo aggiusta progressivamente il modello finché non riconosce bene i pattern presenti in quei dati. È una fase che richiede tempo e potenza di calcolo, e avviene **prima** che il modello venga usato davvero.

- **Inferenza (o predizione):** una volta addestrato, il modello viene utilizzato su **dati nuovi**, mai visti durante il training, per produrre un risultato — una predizione, una classificazione, un testo generato. L'inferenza è rapida e avviene ogni volta che il modello viene usato “sul campo”.



Dentro il training, esistono due modalità principali, entrambe utili da riconoscere per nome:

- **Apprendimento supervisionato:** i dati di addestramento sono già **etichettati** con la risposta corretta. Esempio ShopFacile: si mostrano al modello migliaia di ordini passati, ciascuno già marcato come “fraudolento” o “regolare” da un’analisi umana precedente; il modello impara quali caratteristiche (importo, orario, indirizzo di spedizione diverso da quello di fatturazione...) sono associate a un ordine fraudolento, per poi valutare autonomamente un ordine nuovo, mai visto prima.
- **Apprendimento non supervisionato:** i dati **non** hanno un’etichetta corretta predefinita; il modello cerca da solo strutture o somiglianze. Esempio ShopFacile: si forniscono al modello i dati di acquisto di tutti i clienti, senza dire in anticipo “questo cliente appartiene al gruppo A”, e il modello raggruppa da solo i clienti con abitudini simili (chi compra spesso articoli scontati, chi

acquista solo prodotti premium, chi acquista raramente ma con carrelli molto pieni) — un raggruppamento che il team marketing di ShopFacile può poi usare per proporre promozioni diverse a gruppi diversi.

Un problema molto comune, e importante da riconoscere per nome, è l'**overfitting**: succede quando un modello, durante il training, impara “a memoria” i dati di addestramento nei minimi dettagli, invece di imparare i pattern generali che servono davvero. Il risultato è un modello che funziona in modo impressionante sui dati che ha già visto, ma va male su dati nuovi, mai incontrati prima.

 **Analogia:** pensa a due studenti che si preparano per un esame di matematica usando gli stessi identici esercizi svolti in classe. Il primo studente **capisce il metodo** per risolvere quel tipo di problema, e all'esame, davanti a un esercizio leggermente diverso, se la cava bene. Il secondo studente **memorizza a memoria** i numeri e i passaggi esatti degli esercizi visti, senza capire il perché: se l'esame contiene esercizi identici a quelli di classe, sembra bravissimo, ma davanti a un problema anche solo leggermente diverso non sa cosa fare. Il secondo studente ha fatto “overfitting” sugli esercizi di classe: ha imparato quei dati specifici, non il metodo generale.

Sapere che questo problema esiste ti è utile soprattutto per un motivo pratico: quando un collega dice “il modello va benissimo sui dati di test ma in produzione si comporta male”, overfitting è spesso la prima ipotesi da considerare — non un bug nel senso tradizionale, ma un modello che ha imparato le cose sbagliate.

I modelli visti finora imparano un compito **specifico e circoscritto**: prevedere una frode, raggruppare clienti. Negli ultimi anni, però, una categoria di modelli ha preso una strada molto diversa: modelli generici, capaci di generare testo su qualsiasi argomento. È il momento di parlare di AI generativa e dei suoi protagonisti più discussi, gli LLM.

15.3 AI generativa e LLM: cosa sono, e perché “sbagliano” in un modo particolare

A differenza dei modelli visti al paragrafo precedente, pensati per un compito preciso (fraudolento sì/no, gruppo A o B), un **LLM (Large Language Model, “modello linguistico di grandi dimensioni”)** è un modello addestrato su enormi quantità di testo, capace di **generare** testo nuovo, coerente e plausibile, su praticamente qualsiasi argomento. Per questo si parla di **AI generativa**: non classifica o predice un'etichetta, ma **produce** contenuto nuovo — testo, ma anche codice, immagini, audio, secondo lo stesso principio.

Il modo in cui interagisci con un LLM è tramite un **prompt**: il testo che scrivi per chiedere qualcosa al modello (una domanda, un'istruzione, una richiesta di riformulazione). Il modello elabora il tuo prompt insieme

a quello che ha “detto” o “letto” fino a quel momento nella stessa conversazione — il cosiddetto **contesto** — e genera una risposta parola per parola (più precisamente, **token** per token: un token è un pezzetto di testo, a volte una parola intera, a volte solo una parte di parola, l'unità minima con cui il modello lavora davvero).

Questo contesto, però, non è infinito: ogni modello ha una capacità massima di testo che riesce a “tenere a mente” in una conversazione, chiamata **finestra di contesto** (in inglese *context window*). È un po’ come un tavolo di lavoro di dimensioni fisse: se la conversazione si allunga troppo, a un certo punto le parti più vecchie iniziano a “cadere dal tavolo” per fare spazio a quelle nuove, e il modello può sembrare che abbia “dimenticato” cose che gli avevi detto all’inizio — non per distrazione, ma perché quella parte non è più materialmente dentro la finestra che sta elaborando.

Il punto più importante, e più facile da fraintendere, è **come** un LLM genera quella risposta: **non consulta un database di fatti verificati**, e **non “sa” nulla** nel senso in cui lo sa una persona. Un LLM, in modo molto semplificato, **predice quale sequenza di parole sia statisticamente plausibile** come continuazione del testo che ha davanti, sulla base di tutti i pattern linguistici visti durante il training — non predice quale sequenza sia **vera**. Plausibile e vero, spesso, coincidono: un testo corretto tende anche a “suonare” come la continuazione più naturale. Ma sono due proprietà diverse, e nulla garantisce che coincidano sempre — ed è qui che nasce il problema più noto degli LLM: dalla scioltezza e sicurezza di una risposta non si può dedurre che sia corretta.

Le **allucinazioni** sono il fenomeno per cui un LLM genera informazioni **false**, ma espresse con la stessa sicurezza e la stessa fluidità di un’informazione vera: un nome di funzione che non esiste in una libreria reale, una citazione mai scritta, una statistica inventata. Non è un “errore di battitura” del modello: è una conseguenza diretta di **come** funziona un LLM — genera la continuazione più plausibile, non la continuazione verificata.

 **Da ricordare sempre:** un LLM non ha un modo interno di distinguere “questo lo so per certo” da “questo mi sembra plausibile ma non lo sono”. Genera entrambe le cose con lo stesso tono sicuro. La responsabilità di verificare resta sempre di chi legge la risposta.

 **Esempio pratico:** Sara, come Product Owner, chiede a un assistente AI generativa di generare una prima bozza di descrizione della funzionalità “salvataggio indirizzi multipli” di ShopFacile, da usare come punto di partenza per il backlog. Il testo prodotto è chiaro e ben strutturato, e descrive anche un dettaglio specifico: che ShopFacile “supporta già la selezione automatica dell’indirizzo più vicino al magazzino più conveniente”. Sara, prima di inserire quella descrizione nel backlog, la confronta con Marco e Giulia — e scopre che quella funzionalità non esiste affatto nel prodotto reale: il modello l’ha generata perché **plausibile** per un e-commerce, non perché **vera** per ShopFacile. Se Sara l’avesse copiata così com’è nel backlog, il team avrebbe rischiato di considerare come requisito esistente qualcosa che va invece progettato da zero.

Le allucinazioni viste sopra nascono da un limite molto specifico: un LLM conosce solo ciò che ha “visto” durante il training, un processo che si conclude a un certo punto nel tempo e non si aggiorna da solo. Non conosce i documenti interni di ShopFacile, non conosce i ticket dell’ultima settimana, non conosce nulla che sia stato scritto dopo la chiusura del suo training — e se gli chiedi qualcosa che non sa, come visto al paragrafo precedente, il rischio è che lo **inventi** con la stessa sicurezza con cui

risponde a una domanda che conosce davvero. La domanda naturale, a questo punto, è: c'è un modo per far sì che il modello risponda basandosi sui **nostri** documenti reali, invece di affidarsi solo a quello che ha imparato una volta per tutte? È esattamente il problema che risolve la tecnica che vediamo ora.

15.4 RAG: far rispondere il modello sui documenti veri, non su quello che “ricorda”

RAG (Retrieval-Augmented Generation), in italiano “generazione aumentata dal recupero”, è una tecnica che affronta di petto il problema appena descritto: un LLM che non conosce i documenti interni di un'azienda e che, di fronte a una domanda su di essi, rischia di inventare una risposta plausibile ma falsa.

Il problema, in breve

Un LLM impara tutto quello che sa durante il training, guardando enormi quantità di testo pubblico. Non ha mai visto, né potrebbe aver visto, la documentazione interna di ShopFacile, la politica di reso aggiornata la settimana scorsa, o i ticket di supporto degli ultimi mesi. Se qualcuno gli chiede qualcosa su questi contenuti, il modello non ha altra scelta che generare la continuazione più plausibile — con il rischio di allucinazione già visto al paragrafo 15.3, ma questa volta su informazioni che **esistono davvero**, solo che il modello non le ha mai viste.

L'idea del RAG

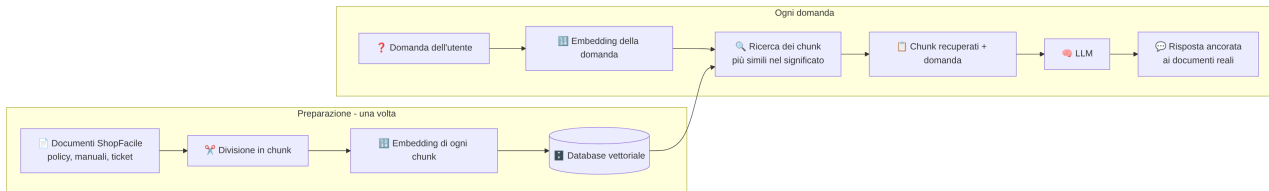
Una prima idea per risolvere il problema potrebbe essere: ri-addestrare il modello anche sui documenti di ShopFacile. Ma il training, come visto al paragrafo 15.2, è un processo lento e costoso, da ripetere ogni volta che un documento cambia — del tutto impraticabile per una politica di reso che viene aggiornata ogni due mesi. Il RAG prende una strada diversa e molto più semplice: **non tocca il modello**. Invece, al momento della domanda, va a **recuperare** i documenti giusti e li **fornisce** al modello insieme alla domanda stessa, chiedendogli di rispondere basandosi su quelli.

Come funziona, in 4 passaggi

1. **Preparazione (una volta, in anticipo)**: i documenti aziendali (policy, manuali, ticket passati...) vengono spezzati in pezzi più piccoli (chiamati **chunk**, letteralmente “pezzi”) e trasformati in **embedding**: rappresentazioni numeriche del *significato* di quel pezzo di testo, non del testo letterale. Pensa a un embedding come a un modo per convertire una frase in una serie di coordinate su una mappa immaginaria, dove testi con un significato simile finiscono in punti vicini tra loro. Questi embedding vengono salvati in un **database vettoriale**: un archivio pensato apposta per trovare rapidamente, dentro migliaia di questi “punti sulla mappa”, quelli più vicini a un punto di partenza.
2. **La domanda dell'utente** viene trasformata nello stesso modo, cioè nel suo embedding — la stessa “mappa”, lo stesso tipo di coordinate.
3. **Ricerca per similarità (retrieval)**: il sistema cerca, dentro il database vettoriale, i pezzi di documento con l'embedding più vicino a quello della domanda — cioè quelli **semanticamente** più simili, non quelli che contengono le stesse identiche parole. È una differenza importante: chi

cerca “posso avere un rimborso?” può ottenere anche i pezzi di documento che parlano di “reso” o di “storno”, perché il significato è vicino anche se le parole usate sono diverse — una ricerca tradizionale per parola esatta non troverebbe quel collegamento.

4. **Generazione aumentata:** i pezzi di documento recuperati vengono inseriti nel prompt insieme alla domanda originale, con un’istruzione del tipo “rispondi basandoti solo su questi testi”. Il modello, a questo punto, non deve più “ricordare” nulla: ha il materiale giusto davanti, nella finestra di contesto vista al paragrafo 15.3, e lo usa per formulare la risposta.



Perché interessa a un PM

Il RAG è la tecnologia dietro molti strumenti che probabilmente incontrerai presto: chatbot che rispondono su una base di documentazione aziendale, assistenti per il servizio clienti che rispondono basandosi sulle policy reali, strumenti di ricerca interna che, invece di restituire un elenco di documenti da leggere, restituiscono direttamente una risposta sintetica citando le fonti.

 **Esempio pratico:** Sara propone al team di costruire un assistente interno che risponda alle domande frequenti sul prodotto ShopFacile pescando dalla documentazione di prodotto e dai ticket di supporto già chiusi, così i nuovi membri del team (e i colleghi del servizio clienti) non debbano più cercare a mano tra decine di pagine. **Marco** la implementa proprio con un RAG: i documenti di ShopFacile e i ticket passati vengono trasformati in embedding e salvati in un database vettoriale; quando qualcuno chiede “qual è la politica di reso per un articolo già usato?”, il sistema recupera i paragrafi giusti della policy reale e li passa al modello, che risponde citando esattamente quella fonte — non una policy generica “plausibile per un e-commerce” come l’esempio delle allucinazioni visto al paragrafo 15.3.

Vantaggi

- Le risposte sono **ancorate a fonti reali**, che si possono anche citare esplicitamente (“questa risposta si basa sul documento X”).
- Aggiornare le informazioni è semplice: basta aggiornare i documenti di partenza (e i relativi embedding), **senza toccare il modello**.
- Non serve rifare nessun processo di training: è molto più rapido ed economico che ri-addestrare un modello da zero.

Limiti e attenzioni — da non sottovalutare

- **Garbage in, garbage out:** se la ricerca recupera il documento sbagliato (o un documento vecchio, mai aggiornato), la risposta sarà costruita su basi sbagliate — con la stessa sicurezza apparente di una risposta corretta.

- Il RAG **riduce** le allucinazioni, ma non le **elimina**: il modello può comunque interpretare male un documento corretto, o mischiarlo con qualcosa che ha “imparato” durante il training.
- La qualità delle risposte dipende **direttamente** dalla qualità e dall'organizzazione della documentazione di partenza — un punto su cui un PM/Scrum Master può avere un impatto reale, ad esempio insistendo perché la documentazione di ShopFacile sia tenuta aggiornata e ben strutturata.
- I **permessi di accesso** ai documenti vanno rispettati anche dentro un sistema RAG: se l'assistente pesca indistintamente anche da documenti riservati, chi lo interroga potrebbe leggere informazioni a cui non dovrebbe avere accesso — lo stesso tema di autorizzazione già visto nella sezione [13. Sicurezza](#).

RAG o fine-tuning? Due strade diverse

Il **fine-tuning** è l'alternativa “storica” al RAG: consiste nel ri-addestrare, almeno parzialmente, un modello sui propri dati, così che le informazioni diventino parte del modello stesso, invece di essere recuperate al momento della domanda.

	RAG	Fine-tuning
Cosa serve	Documenti organizzati + un database vettoriale	Un nuovo ciclo di training su dati propri
Quando conviene	Informazioni che cambiano spesso (policy, ticket, catalogo)	Uno stile o un comportamento molto specifico e stabile nel tempo
Aggiornare le informazioni	Semplice: si aggiornano i documenti	Complesso: richiede un nuovo training
Complessità/costo tipici	Più bassi, più rapidi da mettere in piedi	Più alti, richiedono più tempo e competenze
Cita le fonti	Sì, naturalmente	No, l'informazione è “dentro” il modello

Nella maggior parte dei casi in cui un'azienda vuole che un assistente risponda sui propri documenti aggiornati, il RAG è la strada più diretta; il fine-tuning entra in gioco più raramente, quando serve cambiare in modo stabile “come” un modello si comporta, non solo “cosa” sa.

Il RAG risolve un problema preciso: dare al modello **conoscenza** aggiornata su cui basare le risposte. Ma un assistente che sa solo *rispondere*, per quanto bene informato, resta comunque isolato dal resto del lavoro del team: non può aprire un ticket, non può leggere lo stato di una Pull Request, non può interrogare un sistema aziendale. È il prossimo problema da affrontare.

15.5 MCP: dare al modello mani per agire, non solo occhi per leggere

Il RAG, visto al paragrafo precedente, risolve il problema della **conoscenza**: permette a un modello di rispondere basandosi su documenti veri. Ma un assistente AI, anche ben informato, resta per natura **isolato**: sa conversare, ma da solo non può leggere il repository GitHub di ShopFacile, aprire un ticket, interrogare un database o consultare un sistema aziendale specifico. Serve un modo per collegarlo a questi sistemi — ed è qui che entra in gioco MCP.

Il problema, prima di MCP

Storicamente, ogni volta che si voleva collegare un assistente AI a un sistema esterno (un repository di codice, un sistema di ticket, un database), quell'integrazione andava costruita **su misura**, in modo diverso per ogni combinazione di assistente e sistema. Con N assistenti diversi e M sistemi diversi da collegare, il numero di integrazioni da costruire e mantenere cresce rapidamente — ognuna con le sue regole, i suoi formati, la sua manutenzione separata.

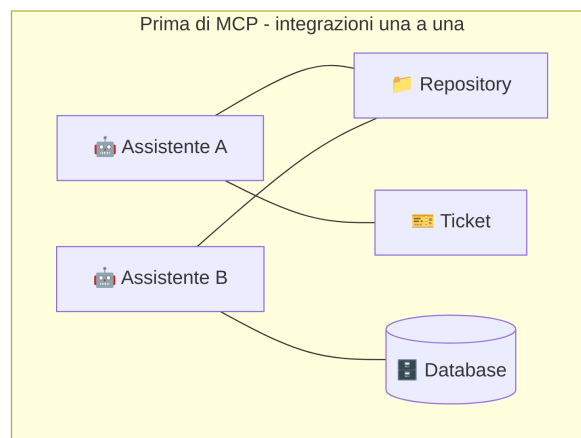
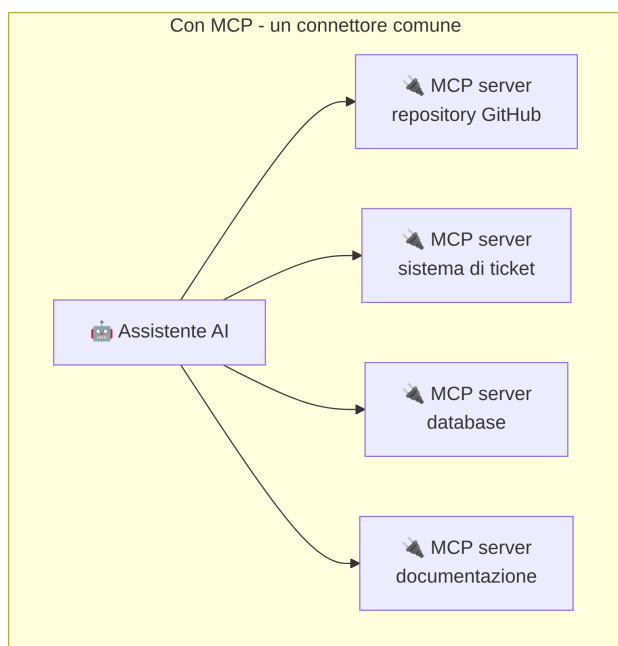
Cos'è MCP

MCP (Model Context Protocol) è uno **standard aperto** — introdotto da Anthropic e con un'adozione crescente nel resto del settore — che definisce un modo **comune** con cui un assistente AI può collegarsi a fonti di dati e strumenti esterni.

 **Analogia:** pensa a MCP come a una **presa standard**, un po' come la porta **USB-C** di un dispositivo elettronico. Prima di uno standard condiviso, ogni dispositivo aveva il suo connettore proprietario, e ogni accessorio andava costruito su misura per quel connettore. Con una presa standard, invece, basta che un sistema la supporti perché qualunque dispositivo compatibile possa collegarsi, senza bisogno di un adattatore costruito apposta per ogni singola combinazione.

Come è fatto, concettualmente

- Un **MCP server** è il componente che “espone” verso l'assistente le capacità di un determinato sistema: leggere file, interrogare un database, chiamare un'API (Application Programming Interface, l'insieme di regole con cui due software si parlano tra loro), oppure eseguire un'azione concreta.
- L'assistente AI si comporta da **client**: si collega a uno o più MCP server e, in base alla richiesta dell'utente, **decide quando e quale strumento usare** tra quelli che il server mette a disposizione.
- Lo stesso MCP server, una volta costruito, può essere usato da qualunque assistente compatibile con lo standard — è proprio questo che elimina l'esplosione di integrazioni su misura vista sopra.



Esempio pratico ShopFacile

 **Esempio pratico:** **Marco** collega l'assistente AI usato dal team a un MCP server per il repository GitHub di ShopFacile e a uno per il sistema di ticket. A quel punto **Ahmed** può chiedere direttamente all'assistente “quali sono le Pull Request aperte sul servizio pagamenti?” e ottenere una risposta basata sui dati **reali** del repository in quel momento, non una risposta plausibile ma inventata come nell'esempio delle allucinazioni al paragrafo 15.3. Allo stesso modo, **Luca** può chiedere una sintesi dei ticket chiusi durante l'ultimo sprint per prepararsi alla Sprint Review, senza doverli scorrere uno per uno a mano.

Perché interessa a un PM

MCP segna la differenza tra un assistente che “**parla**” (risponde a domande, propone testo) e un assistente che “**fa**” (può leggere dati aggiornati da sistemi reali, e in alcuni casi anche eseguire azioni). Questo abilita l'automazione di attività ripetitive e riduce il lavoro di integrazione su misura tra strumenti — un impatto che, come PM/Scrum Master, puoi arrivare a vedere direttamente nelle stime e nei tempi di un progetto che prevede questo tipo di automazione. Allo stesso tempo, dare a un assistente la capacità di collegarsi a sistemi reali apre domande di governance nuove, da non prendere alla leggera.

Rischi e attenzioni — da non sottovalutare

- Dare a un assistente la capacità di **agire** su un sistema (non solo leggerlo) richiede confini chiari: permessi minimi necessari per quel compito specifico, non un accesso totale “per sicurezza”; approvazione umana prima di qualsiasi azione **irreversibile** (es. chiudere un ticket, cancellare dei dati); tracciabilità di cosa l'assistente ha effettivamente fatto.

- Va valutato con attenzione **quali dati** un MCP server espone e **a chi** — lo stesso tema di autenticazione e autorizzazione già visto nella sezione 13. **Sicurezza**: un assistente collegato a troppi sistemi, con permessi troppo ampi, può diventare un punto d'accesso a informazioni che chi lo usa non dovrebbe vedere.
- Un assistente capace di eseguire azioni concrete è, di fatto, una **superficie di attacco in più** da considerare nel quadro complessivo della sicurezza del progetto: le stesse domande poste per qualsiasi altro sistema con accesso a dati o funzionalità sensibili si pongono, allo stesso modo, per un assistente collegato via MCP.

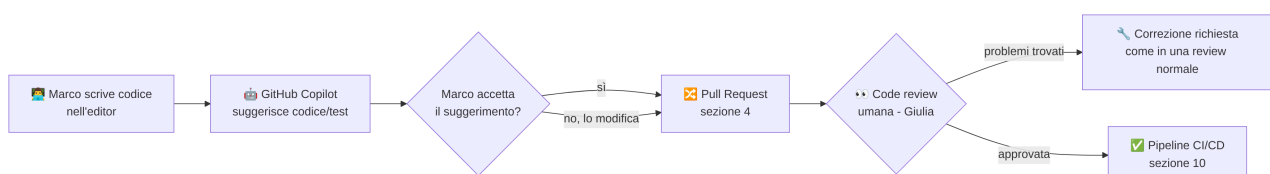
Il RAG e MCP risolvono, insieme, due metà dello stesso problema: il RAG dà al modello **conoscenza** aggiornata su cui basare le risposte, MCP gli dà **mani** per leggere sistemi reali e, quando serve, agire su di essi. Messi insieme, trasformano un assistente generico — capace solo di conversare su argomenti generali — in uno strumento realmente utile nel contesto specifico di un'azienda come ShopFacile.

15.6 AI negli strumenti di sviluppo: cosa cambia per Marco, Giulia e Ahmed

RAG e MCP, visti nei due paragrafi precedenti, riguardano soprattutto assistenti “esterni” collegati a documenti e sistemi. Ma c'è un terreno in cui l'AI generativa si è inserita ancora più a fondo, dentro il lavoro quotidiano di chi scrive codice: se un LLM può generare testo plausibile su qualsiasi argomento, e il codice sorgente è anch'esso un tipo di testo (con una sintassi molto più rigida di una lingua naturale, ma testo), è comprensibile che gli stessi modelli si siano rivelati molto efficaci anche nel generare codice. È così che l'AI è entrata concretamente nell'ambiente di lavoro del team di ShopFacile, dentro lo stesso editor di codice e lo stesso GitHub che hai già incontrato nella sezione 4.

L'esempio più noto e concreto è **GitHub Copilot**: un assistente AI integrato nell'editor di codice, che osserva quello che lo sviluppatore sta scrivendo (e il resto del progetto) e propone in tempo reale:

- **completamento del codice**: mentre Marco scrive una funzione, Copilot suggerisce il resto della riga o dell'intero blocco, sulla base di pattern simili visti durante il proprio addestramento;
- **generazione di test**: a partire da una funzione già scritta, propone una bozza di unit test che ne verifica il comportamento — un aiuto concreto per Ahmed, spesso alle prese con la scrittura di test da zero;
- **spiegazione di codice legacy**: davanti a una porzione di codice vecchia e poco documentata (capita spesso nei progetti maturi come ShopFacile), può proporre una spiegazione in linguaggio naturale di cosa fa quel codice, utile come punto di partenza per orientarsi;
- **supporto alla code review**: può segnalare pattern sospetti o proporre commenti di revisione, come un primo filtro prima che intervenga l'occhio umano di Giulia.



Il punto centrale da portare con te, e che vale la pena ripetere con la stessa fermezza usata per le allucinazioni: **il codice generato o suggerito da un'AI va comunque revisionato**, esattamente come qualsiasi altro codice scritto da una persona. Non perché l'AI sia “cattiva”, ma perché può generare codice che sembra corretto e compila, ma contiene un bug logico, una dimenticanza di un caso limite, o addirittura una vulnerabilità di sicurezza (gli stessi tipi di errore visti nella sezione 13, come una query costruita in modo non sicuro). La Pull Request e la code review viste nella sezione 4, e i quality gate della pipeline CI/CD visti nella sezione 10, restano gli stessi, identici filtri — semplicemente, ora una parte del codice che arriva a quei filtri può essere stata scritta con l'assistenza di un modello, non solo digitata riga per riga da una persona.



Esempio pratico: Ahmed usa Copilot per generare rapidamente una funzione che calcola lo sconto totale su un carrello con più codici promozionali applicati insieme. Il suggerimento compila, i test che Ahmed scrive per i casi “normali” passano. Durante la code review, però, **Giulia** nota che il codice non gestisce il caso di due codici promozionali reciprocamente incompatibili (es. uno sconto percentuale e uno a importo fisso che non dovrebbero sommarsi) — un caso limite che Copilot non aveva “pensato” perché nel suo addestramento non aveva un contesto specifico sulle regole di business di ShopFacile. La Pull Request viene corretta prima di essere unita: la revisione umana ha fatto esattamente il lavoro per cui esiste.

Il codice non è l'unico terreno su cui l'AI si è inserita nel lavoro del team: da un compito puntuale dentro l'editor di uno sviluppatore, gli stessi principi si applicano oggi a un raggio più ampio, quello dell'intero ciclo di vita DevOps.

15.7 AI nel ciclo di vita DevOps: dall'idea al monitoraggio

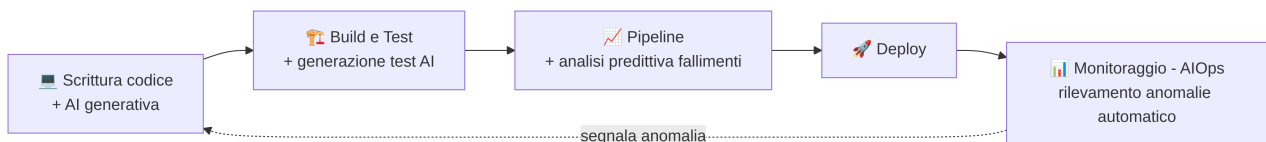
Copilot, visto al paragrafo precedente, interviene in un punto molto preciso del lavoro: mentre il codice viene scritto. Ma il ciclo DevOps visto nella sezione 9 e nella sezione 10 è molto più lungo di quel singolo momento — e l'AI, con logiche simili, si è progressivamente inserita in più punti di quel ciclo, non solo nella scrittura del codice.

Alcuni esempi concreti, seguendo l'ordine con cui li incontreresti lungo la pipeline:

- **Generazione di codice e test:** oltre al completamento visto sopra, strumenti di AI possono generare intere bozze di funzioni o suite di test a partire da una descrizione in linguaggio naturale del comportamento desiderato.
- **Analisi predittiva dei fallimenti della pipeline:** alcuni strumenti, osservando lo storico delle esecuzioni passate di una pipeline, possono segnalare in anticipo che una certa modifica ha una probabilità più alta del solito di far fallire i test, sulla base di pattern simili osservati in passato.
- **Rilevamento di anomalie nel monitoraggio:** hai già incontrato, nella sezione 10, **Dynatrace** come piattaforma di observability che segnala comportamenti anomali dopo un deploy. Molte piattaforme di monitoraggio moderne usano proprio tecniche di machine learning per stabilire, in modo automatico, cosa sia “normale” per una determinata metrica e cosa sia invece un'**anomalia** da segnalare — un compito che, fatto a mano guardando grafici, richiederebbe moltissimo tempo umano e occhio molto allenato.

- **Assistenza nella documentazione e nei ticket:** generazione di bozze di documentazione tecnica, riformulazione di un ticket scritto in modo confuso in una descrizione più chiara, sintesi di log di errore lunghi e difficili da leggere in poche righe comprensibili.

Il termine con cui sentirai riassumere tutto questo, in una parola, è **AIOps** (Artificial Intelligence for IT Operations): l'uso di tecniche di AI e machine learning per rendere più efficienti le attività operative di gestione e monitoraggio dei sistemi IT — la stessa area di cui parlano monitoring, observability e logging visti nella sezione 9, ma con un livello di automazione ulteriore, capace di riconoscere pattern che sarebbe impraticabile individuare guardando i dati a occhio.



Non serve che tu sappia configurare nessuno di questi strumenti: quello che conta è riconoscere l'AIOps come un'evoluzione naturale del monitoring e dell'observability già visti, non come un argomento completamente separato — e riconoscere che l'AI, in questo ambito, lavora più spesso in affiancamento a strumenti già noti (come Dynatrace) che come strumento a sé stante.

Fin qui abbiamo guardato il lavoro tecnico del team. Ma anche il tuo lavoro, come Junior PM/Scrum Master, viene toccato da questi stessi strumenti — ed è un argomento che merita attenzione particolare, perché è quello più vicino al tuo ruolo quotidiano.

15.8 AI nel lavoro di un Project Manager / Scrum Master

Fin qui l'AI ha toccato soprattutto il lavoro tecnico di Marco, Giulia e Ahmed. Ma gli stessi strumenti di AI generativa visti al paragrafo 15.3 sono altrettanto utili — con la stessa cautela sulle allucinazioni — nel lavoro quotidiano di chi, come te, si occuperà di coordinamento, pianificazione e comunicazione. Alcuni usi concreti che è probabile tu stessa o tu stesso sperimenti presto:

- **Stesura e riformulazione di user story:** partire da un'idea grezza di Sara ("i clienti vogliono salvare più indirizzi di spedizione") e chiedere a uno strumento AI di proporre una prima bozza in formato user story ("Come cliente registrato, voglio poter salvare più indirizzi di spedizione, così da scegliere rapidamente quello giusto al checkout"), da rivedere e correggere con il team, non da usare tale e quale.
- **Sintesi di riunioni:** a partire dalla trascrizione (o dagli appunti grezzi) di una retrospettiva o di una daily standup, generare un riassunto con i punti principali e le azioni decise, da verificare comunque contro quanto realmente detto.
- **Supporto alla stima:** analizzare lo storico di user story simili già completate in passato per suggerire un ordine di grandezza di stima — un aiuto, non una sostituzione del giudizio del team durante il Planning Poker visto nella sezione 6.
- **Bozze di documentazione e report:** generare una prima versione di un report di sprint, di una nota di rilascio, di un aggiornamento di stato per il cliente, che tu poi correggi, verifichi e personalizzi con i dati reali del progetto.

- **Analisi di feedback utenti:** a partire da decine o centinaia di recensioni o segnalazioni dei clienti di ShopFacile, individuare i temi ricorrenti (es. “molti utenti si lamentano dei tempi di consegna”) più rapidamente di una lettura manuale una per una.

 **Esempio pratico:** dopo una Sprint Review particolarmente densa, **Luca** usa un assistente AI per generare una prima sintesi degli argomenti discussi a partire dai suoi appunti sparsi. Il risultato è un testo ben organizzato, con punti elenco chiari — ma nella sintesi compare anche una frase su una decisione relativa al budget che in realtà **non è mai stata presa** in quella riunione: probabilmente il modello ha “confuso” un’osservazione informale con una decisione formale. Luca, prima di condividere la sintesi con il team e con Sara, la rilegge confrontandola con i suoi appunti originali, e corregge quel punto. Se l’avesse condivisa senza verificarla, quella falsa decisione avrebbe potuto generare confusione seria nelle settimane successive.

Questi usi sono concretamente utili, ma hanno limiti altrettanto concreti che è importante avere chiari fin da subito:

- **La responsabilità della decisione resta sempre umana.** Un’AI può proporre una stima, un riassunto, una bozza — ma la decisione finale (quale stima accettare, cosa comunicare al cliente, come dare priorità al backlog) resta un giudizio del team e del PM/Scrum Master, non del modello.
- **L’AI non conosce il contesto aziendale specifico di ShopFacile:** non sa quali accordi commerciali esistono con quel cliente, quali sensibilità politiche interne al team vanno gestite con cura, quali promesse sono già state fatte in una riunione precedente non riportata da nessuna parte. Un testo generato può essere linguisticamente perfetto e comunque sbagliato nel merito.
- **Attenzione a delegare la comunicazione con gli stakeholder.** Usare l’AI per una prima bozza di un messaggio a un cliente è utile; inviare quel messaggio senza rileggerlo e senza personalizzarlo con la conoscenza reale della relazione con quel cliente è un rischio concreto — un tono sbagliato o un’imprecisione in una comunicazione ufficiale hanno un costo di reputazione reale, come già visto per gli incidenti di sicurezza nella sezione 13.

Questi limiti — allucinazioni, mancanza di contesto, rischio di delega eccessiva — non sono solo un problema del tuo ruolo: sono parte di un insieme più ampio di rischi che riguardano l’uso dell’AI in qualsiasi contesto professionale, ed è utile vederli insieme, in modo più sistematico.

Cosa cambia, davvero, per stima e revisione

C’è un effetto dell’AI che tocca direttamente due strumenti che conosci bene: la stima (sezione 6, Planning Poker) e la revisione del codice (sezione 4, richiamata al paragrafo 15.6).

La velocity accelera, ma non in modo uniforme. Copilot aiuta moltissimo su compiti ripetitivi e ben definiti — codice “ovvio”, bozze di test per il caso normale. Aiuta molto meno sulla parte più difficile del lavoro: capire **perché** un comportamento è sbagliato, diagnosticare un caso limite (come ha fatto Giulia al paragrafo 15.6 con i codici promozionali incompatibili), incastrare una funzionalità con regole di business complesse. Se il team completa più in fretta le user story “semplici” grazie all’AI, la **velocity** può salire senza che la capacità reale di risolvere problemi complessi sia cambiata in proporzione: i punti storici, da quel momento, **perdono confrontabilità** per chi guida lo sprint planning.

La revisione rischia un eccesso di fiducia. Un codice generato da un modello, quando compila ed è scritto con sintassi impeccabile, **sembra** più affidabile di quanto lo sia — e questo abbassa, senza che chi revisiona se ne accorga, la soglia di attenzione. Un errore incorporato in codice fluente e ben formattato si nasconde meglio di uno scritto goffamente da un principiante.

La responsabilità resta comunque umana, e va tracciata dal processo. Se Ahmed accetta un suggerimento con un difetto e Giulia lo approva senza accorgersene, la responsabilità non “appartiene” al modello che l'ha generato: appartiene alle persone e al processo, esattamente come la matrice RACI (sezione 8) assegna sempre la responsabilità a una persona, mai a uno strumento. Per questo è utile rendere il punto esplicito nella **Definition of Done** (sezione 6): il codice generato da AI si revisiona con gli stessi criteri di quello scritto a mano, senza sconti di scrutinio.

15.9 Rischi, limiti e uso responsabile

Abbiamo incontrato, sezione dopo paragrafo, diversi limiti dell'AI: le allucinazioni (paragrafo 15.3), i limiti del RAG e le questioni di autorizzazione legate a RAG e MCP (paragrafi 15.4 e 15.5), la mancanza di contesto aziendale (paragrafo 15.8), la necessità di revisione umana del codice (paragrafo 15.6). È il momento di raccogliarli in un quadro unico, insieme ad altri rischi che non abbiamo ancora nominato, perché sono proprio quelli su cui un PM/Scrum Master viene più spesso interpellato.

- **Allucinazioni** (già viste al paragrafo 15.3): informazioni false generate con sicurezza. Vanno sempre verificate, specialmente prima di usarle in documenti ufficiali o comunicazioni al cliente.
- **Bias nei dati**: se i dati usati per addestrare un modello contengono squilibri o pregiudizi (ad esempio, dati storici che riflettono decisioni passate non del tutto imparziali), il modello tende a **riprodurre e a volte amplificare** quegli squilibri nelle proprie predizioni, senza “saperlo” e senza intenzione. Un modello che decide, ad esempio, quali candidature dare priorità in un processo di selezione, addestrato su dati storici sbilanciati, può riprodurre quello sbilanciamento nelle nuove decisioni.
- **Privacy e riservatezza**: questo è il punto più delicato e più operativo per il tuo lavoro quotidiano. **Non bisogna mai incollare codice proprietario di ShopFacile, dati dei clienti o informazioni riservate del progetto dentro strumenti di AI pubblici** che non siano esplicitamente approvati e configurati dall'azienda per quell'uso. Quel testo, una volta inviato a un servizio esterno, esce dal perimetro di controllo del progetto — esattamente il tipo di rischio già affrontato nella sezione [13. Sicurezza](#) parlando di **GDPR** (General Data Protection Regulation, “Regolamento Generale sulla Protezione dei Dati”) e protezione dei dati personali: se un prompt contiene per errore il nome, l'email o l'indirizzo di un cliente reale di ShopFacile, quello è un trattamento di dati personali che può violare gli stessi obblighi già visti in quella sezione.
- **Proprietà intellettuale del codice generato**: chi è “l'autore” di una funzione generata da un'AI, addestrata su enormi quantità di codice altrui, spesso pubblicato online con licenze diverse tra loro e non sempre compatibili con un uso commerciale? La domanda si complica ulteriormente chiedendosi chi possieda davvero l'output — chi ha scritto il prompt, chi possiede il modello, o nessuno dei due. È un tema legale ancora in evoluzione e diverso da un paese all'altro, che non spetta a te risolvere da sola o da solo, ma è utile sapere cosa è in gioco: proprio per questa

incertezza, molte aziende adottano **policy interne** che regolano quali strumenti di AI generativa di codice si possono usare e come, invece di lasciare la questione alla sensibilità di ciascun developer.

- **Dipendenza eccessiva:** affidarsi sempre e comunque all'AI, senza mai verificare, allena progressivamente le persone a **non verificare più nulla** — con il rischio concreto, per una persona alle prime esperienze come Ahmed, di **non imparare davvero** un concetto perché ha sempre delegato la parte difficile a un suggerimento generato, senza capire il “perché” dietro quel codice.

 **Nota prudente:** l'Unione Europea ha adottato un regolamento specifico sull'intelligenza artificiale, comunemente chiamato **AI Act**, che introduce obblighi diversi in base al livello di rischio di un determinato uso dell'AI, con un'applicazione che procede in modo progressivo nel tempo. Non ti serve conoscerne gli articoli né le date esatte di entrata in vigore delle diverse parti — quello che conta è sapere che **esiste un quadro normativo europeo specifico sull'AI**, in continua evoluzione, e che se il tuo progetto usa l'AI in modi che toccano dati personali o decisioni che impattano le persone, è un tema da portare all'attenzione di chi si occupa di conformità legale nel progetto, non da gestire “a intuito”.

 **Esempio pratico: Ahmed**, per farsi aiutare a risolvere un bug difficile nel modulo pagamenti, sta per incollare in un assistente AI pubblico un blocco di codice che contiene, tra le righe, anche una chiave di configurazione dell'ambiente di produzione e un frammento di dati reali di un ordine di test. **Marco**, che nota lo screen mentre passa vicino alla scrivania, lo ferma in tempo: quella chiave, una volta inviata a un servizio esterno, non è più sotto il controllo di ShopFacile. Insieme puliscono il codice da qualsiasi informazione sensibile o reale prima di chiedere aiuto all'AI — lo stesso principio di “minimizzazione” già visto nella sezione 13 a proposito del GDPR, applicato qui ai prompt.

Il filo comune di tutti questi rischi è lo stesso: l'AI è uno strumento potente ma **non responsabile** delle proprie conclusioni — la responsabilità resta sempre di chi la usa e di chi decide sulla base di quello che produce.

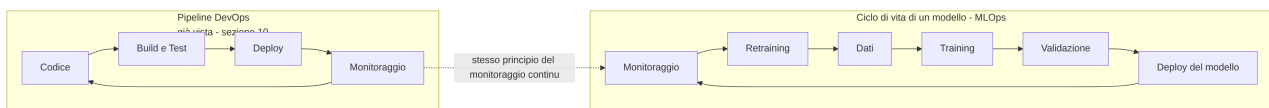
Chi decide quali strumenti si possono usare

L'episodio di Ahmed e Marco mostra perché lasciare la scelta “quale strumento di AI usare, e come” alla sensibilità di ciascuno non basta: qualcuno deve **decidere e mettere per iscritto** una policy, prima che il problema si presenti — chi decide quali strumenti sono ammessi sul progetto, dove finiscono davvero i dati inseriti (se addestrano il modello, se restano visibili al fornitore, per quanto tempo), e come si aggiorna la policy quando cambia lo strumento. Proporarla e mantenerla aggiornata è materia tipica di chi coordina il team — lo Scrum Master o il Tech Lead — non un dettaglio lasciato al buon senso di ciascuno.

15.10 MLOps: come si porta un modello in produzione, in sintesi

I rischi visti al paragrafo precedente riguardano soprattutto l'uso di modelli già pronti, spesso costruiti da altre aziende. Ma se il team di ShopFacile decidesse di costruire un proprio modello (ad esempio, quello per il rilevamento delle frodi visto al paragrafo 15.2), quel modello avrebbe un ciclo di vita tutto suo, diverso da quello del software tradizionale — e vale la pena conoscerne l'esistenza, anche solo a grandi linee.

Il ciclo di vita di un modello di machine learning segue, molto semplificando, questi passaggi: **dati** → **training** → **validazione** → **deploy** → **monitoraggio** → **retraining**. Il nome che il settore ha dato a questa disciplina è **MLOps (Machine Learning Operations)**: il “cugino” del **DevOps** già visto nella sezione 9, applicato non al software tradizionale ma al ciclo di vita dei modelli.



Le somiglianze con il DevOps sono volute: entrambi i cicli puntano all'automazione, alla ripetibilità e al monitoraggio continuo dopo il rilascio. Ma c'è una differenza importante da conoscere, perché non ha un equivalente diretto nel software tradizionale: i modelli **degradano nel tempo**, un fenomeno chiamato **data drift**. Un modello addestrato sui comportamenti d'acquisto dei clienti di ShopFacile in un certo periodo può diventare progressivamente meno accurato se le abitudini dei clienti cambiano — una nuova categoria di prodotti molto richiesta, un cambiamento nei metodi di pagamento preferiti, un evento eccezionale come una promozione molto aggressiva che altera temporaneamente i pattern normali. Il software tradizionale, una volta funzionante e non modificato, continua a fare esattamente la stessa

cosa per sempre; un modello, anche senza che nessuno tocchi una riga di codice, può iniziare a **sbagliare di più** semplicemente perché la realtà che deve prevedere è cambiata rispetto a quando è stato addestrato. Per questo il ciclo MLOps prevede un passaggio di **retraining**: ri-addestrare periodicamente il modello su dati più recenti, per tenerlo aggiornato alla realtà attuale.

Non avrai bisogno di configurare né gestire un ciclo MLOps, ma se in una conversazione tecnica senti dire “le performance del modello anti-frode sono scese, probabilmente serve un retraining”, ora sai esattamente a cosa ci si riferisce e perché non è semplicemente “un bug da correggere”.

15.11 Come usarla bene nel quotidiano

Dopo aver visto potenzialità, usi concreti e rischi, resta una domanda molto pratica: come ti comporti tu, ogni giorno, quando apri uno strumento di AI generativa per un compito reale del tuo lavoro? Qualche regola di buon senso, coerente con tutto quello visto in questa sezione, ti aiuta a usarla bene senza subirne i limiti.

Regole pratiche per un uso responsabile dell'AI generativa

1. **Dai contesto nel prompt.** Un'AI generativa non conosce il progetto ShopFacile, i suoi accordi commerciali o le decisioni prese la settimana scorsa: più contesto utile (e non riservato) fornisci nella richiesta, più la risposta sarà pertinente — ma senza mai includere dati sensibili o proprietari, come visto al paragrafo 15.9. In una conversazione molto lunga, ricorda anche il limite della finestra di contesto visto al paragrafo 15.3: se l'assistente sembra aver “dimenticato” qualcosa detto all'inizio, spesso basta ripeterlo brevemente nel prompt. Se invece l'assistente che usi è collegato a documenti o sistemi reali tramite RAG o MCP (paragrafi 15.4 e 15.5), verifica comunque le risposte prima di darle per buone: aiutano a ridurre le allucinazioni, non le eliminano.
2. **Chiedi di spiegare, non solo di fare.** Chiedere “spiegami perché questa user story è formulata così” invece di limitarti ad accettare il testo generato ti aiuta a imparare e a verificare che il ragionamento dietro la risposta abbia senso, non solo il risultato finale.
3. **Verifica sempre le informazioni fattuali.** Numeri, nomi, citazioni, riferimenti normativi: qualsiasi dato verificabile generato da un'AI va controllato su una fonte affidabile prima di usarlo in un documento ufficiale o in una comunicazione — le allucinazioni del paragrafo 15.3 non sono un caso raro, sono un comportamento possibile in ogni risposta.
4. **Trattala come un collega esperto ma smemorato, che a volte inventa — non come un oracolo.** È un'analogia utile da tenere sempre a mente: un collega con moltissima esperienza generale, capace di darti ottimi spunti e bozze rapide, ma che non conosce la storia specifica del progetto e che, quando non sa una risposta, a volte la inventa con sicurezza invece di dire “non lo so”.
5. **Non smettere di imparare delegando tutto.** Specialmente se sei ancora agli inizi, come Ahmed nel suo percorso di crescita, usa l'AI per accelerare compiti che sai già fare o che stai imparando a fare, non per evitare di imparare a farli.

Queste cinque regole non richiedono competenze tecniche: sono, in fondo, la stessa forma di pensiero critico che hai già praticato in altre sezioni del corso — verificare prima di fidarti ciecamente, capire il “perché” oltre al “cosa”, e ricordare sempre chi ha davvero la responsabilità della decisione finale.

15.12 Riepilogo

In questa sezione hai visto come l'intelligenza artificiale si inserisce nel lavoro quotidiano di un team come quello di ShopFacile, e in particolare nel tuo ruolo di Junior PM/Scrum Master:

- **AI, machine learning e deep learning** sono cerchi concentrici, non sinonimi: un modello di machine learning impara pattern dai dati invece di seguire regole scritte a mano;
- il ciclo **training** → **inferenza** è il cuore del machine learning, con apprendimento **supervisionato** (dati etichettati) e **non supervisionato** (nessuna etichetta); l'**overfitting** è il rischio di un modello che “impara a memoria” invece di generalizzare;
- gli **LLM** e l'**AI generativa** producono testo (o codice) plausibile predicendo la continuazione più probabile, non consultando una base di fatti verificati — da qui il rischio delle **allucinazioni**;

- il **RAG (Retrieval-Augmented Generation)** riduce (senza eliminare) le allucinazioni facendo rispondere il modello sui documenti reali dell'azienda, recuperati al momento della domanda invece che “ricordati” dal training; **MCP (Model Context Protocol)** è lo standard aperto che permette a un assistente AI di collegarsi, in modo comune, a sistemi e strumenti esterni (repository, ticket, database) — dando al modello, rispettivamente, **conoscenza** aggiornata e **mani** per agire;
- strumenti come **GitHub Copilot** assistono Marco, Giulia e Ahmed nella scrittura di codice e test, ma il codice generato **va sempre revisionato** come qualsiasi altro, tramite Pull Request, code review e quality gate;
- l'AI si inserisce in più punti del ciclo DevOps (generazione di codice e test, analisi predittiva dei fallimenti, rilevamento anomalie), un'area spesso riassunta con il termine **AIOps**;
- per un PM/Scrum Master, l'AI è utile per user story, sintesi di riunioni, supporto alla stima e bozze di documentazione — ma la **responsabilità della decisione resta sempre umana**;
- i rischi principali sono allucinazioni, **bias**, **privacy** (mai dati proprietari o dei clienti in strumenti pubblici, in linea con il GDPR visto nella sezione 13), proprietà intellettuale del codice generato e dipendenza eccessiva; l'**AI Act** europeo è il quadro normativo di riferimento, ancora in evoluzione;
- **MLOps** è il ciclo di vita di un modello (dati → training → validazione → deploy → monitoraggio → retraining), il “cugino” del DevOps; i modelli possono **degradare nel tempo** (data drift), a differenza del software tradizionale;
- usare bene l'AI nel quotidiano significa dare contesto, verificare i fatti, chiedere spiegazioni e trattarla come un collega esperto ma smemorato — non come un oracolo infallibile.

Hai ormai incontrato, sezione dopo sezione, un numero considerevole di termini nuovi — da questa sezione in particolare AI, ML, LLM, RAG, MCP, MLOps, AIOps — oltre a tutti quelli delle sezioni precedenti. Nella prossima sezione trovi esattamente lo strumento pensato per non doverli ricordare tutti a memoria: il [16. Glossario](#), un dizionario consultabile in ogni momento del tuo percorso.

Checklist di autoverifica

Prima di passare alla sezione successiva, prova a rispondere (anche solo a voce, o scrivendo due righe) a queste domande:

- Sai spiegare, con l'analogia dei cerchi concentrici, la differenza tra AI, machine learning e deep learning?
- Sai spiegare la differenza tra training e inferenza, e tra apprendimento supervisionato e non supervisionato, con esempi ShopFacile?
- Sai spiegare cos'è l'overfitting con l'analogia dello studente?
- Sai spiegare perché un LLM può “allucinare”, collegandolo a come genera davvero il testo?
- Sai spiegare cos'è il RAG, i suoi 4 passaggi in sintesi, e perché riduce ma non elimina le allucinazioni?
- Sai spiegare cos'è MCP con l'analogia della presa standard/USB-C, e la differenza tra un assistente che “parla” e uno che “fa”?
- Sai spiegare, con parole tue, quando conviene il RAG e quando invece il fine-tuning?

- Sai spiegare perché il codice generato da uno strumento come GitHub Copilot va comunque revisionato, e da chi/come?
- Sai spiegare cos'è l'AIOps e a quale area del DevOps si collega?
- Sai indicare almeno tre usi concreti dell'AI nel lavoro di un PM/Scrum Master e i relativi limiti?
- Sai spiegare perché non si devono incollare dati riservati o dei clienti in strumenti di AI pubblici, collegandolo al GDPR visto nella sezione 13?
- Sai spiegare cos'è MLOps e perché un modello può degradare nel tempo?
- Sapresti elencare almeno tre delle regole pratiche viste al paragrafo 15.11 per un uso responsabile dell'AI?



Esercizi pratici

1. **Distingui AI, ML e deep learning con parole tue.** Scrivi tre frasi brevi, una per termine, usando un esempio diverso da quelli visti in questa sezione (può essere di vita quotidiana, non necessariamente di ShopFacile). **Come verificare:** fai leggere le tue tre frasi a un collega (developer o non) e chiedi se, secondo lui, hai reso chiaro che sono cerchi via via più piccoli, non sinonimi intercambiabili.
2. **Prova un assistente AI generativa e caccia un'allucinazione.** Usa uno strumento di AI generativa che hai già a disposizione (es. quello fornito dalla tua azienda, se esiste una policy che lo consente) e chiedi qualcosa su un argomento che conosci bene, di cui puoi verificare la risposta con certezza. Confronta la risposta con quello che sai per certo. **Come verificare:** se trovi anche un solo piccolo dettaglio non corretto (o se non ne trovi, ma hai comunque verificato con attenzione ogni affermazione controllabile), hai fatto l'esercizio nel modo giusto: l'obiettivo è l'abitudine a verificare, non necessariamente "trovare l'errore".
3. **Rifletti su un caso di privacy.** Immagina (per iscritto, in 4-5 righe) una situazione concreta in cui saresti tentata o tentato di incollare qualcosa di riservato in uno strumento di AI pubblico per farti aiutare più in fretta (un pezzo di codice con una chiave, un'email di un cliente, un contratto). Scrivi come gestiresti la stessa richiesta senza violare la riservatezza. **Come verificare:** la tua alternativa non deve rinunciare all'obiettivo (farti aiutare), ma deve eliminare o anonimizzare l'informazione sensibile prima di condividerla con lo strumento.
4. **Scrivi una user story con e senza AI, e confrontale.** Prendi un'idea grezza (anche inventata, es. "i clienti vogliono poter annullare un ordine entro un'ora dall'acquisto") e scrivi tu stessa/tu stesso una prima versione in formato user story; poi chiedi a un assistente AI di farne una versione sua a partire dalla stessa idea grezza. Confronta le due. **Come verificare:** individua almeno una differenza concreta tra le due versioni (chiarezza, completezza, criteri di accettazione mancanti) e scrivi quale delle due, secondo te, useresti davvero nel backlog e perché.
5. **Riconosci un caso di overfitting o data drift raccontato a parole.** Chiedi a un developer o data scientist del tuo team (se ne avete uno) se è mai capitato un caso in cui un modello si comportava bene "in teoria" ma male "nella realtà", o un modello che ha iniziato a sbagliare di più

con il tempo. Fatti raccontare cosa è successo.  **Come verificare:** sai indicare se il caso raccontato assomiglia più a un overfitting (male su dati nuovi fin da subito) o a un data drift (bene all'inizio, peggio con il tempo), spiegando la differenza con parole tue.

6. **Trova un uso reale di AI nel tuo team o progetto.** Chiedi a un collega se nel progetto viene usato uno strumento di AI (per il codice, per il monitoraggio, per la documentazione) e a quale fase del lavoro si collega tra quelle viste in questa sezione.  **Come verificare:** sai spiegare a un'altra persona, in due frasi, cosa fa concretamente quello strumento nel progetto e quale controllo umano esiste ancora su quel risultato (revisione, approvazione, verifica).
7. **Applica le cinque regole del paragrafo 15.11 a un caso concreto.** Pensa alla prossima volta che useresti (o hai usato) un'AI generativa per un compito reale del tuo lavoro, e ripassa mentalmente le cinque regole pratiche: quali hai già seguito, quali no?  **Come verificare:** individua almeno una regola che, guardando indietro, non avevi seguito, e scrivi in una riga cosa faresti diversamente la prossima volta.
8. **Individua un caso RAG nella tua azienda.** Pensa (o chiedi a un collega) se esiste, o servirebbe, un assistente che risponda a domande frequenti pescando dalla documentazione interna, dai ticket passati o dalle policy aziendali. Elenca quali documenti concreti dovrebbero alimentarlo, e chi dovrebbe poterlo interrogare.  **Come verificare:** la tua lista di documenti è specifica (non solo "la documentazione"), e hai anche pensato a chi **non** dovrebbe poter accedere a quell'assistente, se alcuni documenti fossero riservati.
9. **RAG o fine-tuning?** Per ciascuno dei seguenti casi, decidi (con una riga di motivazione) se ti sembra più adatto un approccio RAG o un fine-tuning: (a) un assistente che deve rispondere sulla policy di reso aggiornata ogni mese; (b) un assistente che deve sempre rispondere con uno stile e un tono aziendale molto specifico, stabile nel tempo.  **Come verificare:** confronta le tue risposte con la tabella del paragrafo 15.4 — il caso (a), che cambia spesso, dovrebbe orientarti verso il RAG; il caso (b), stabile nel tempo, verso il fine-tuning.
10. **Ragiona sui permessi di un assistente collegato via MCP.** Immagina che il team voglia collegare un assistente AI, tramite MCP, al sistema di ticket di supporto di ShopFacile. Scrivi quali permessi minimi gli daresti (es. solo lettura dei ticket aperti? può anche chiuderli? può rispondere ai clienti in autonomia?) e quali azioni richiederebbero invece un'approvazione umana.  **Come verificare:** la tua lista distingue chiaramente tra azioni di sola lettura (più sicure da concedere) e azioni che modificano qualcosa o sono irreversibili (che richiedono più cautela e, idealmente, approvazione umana).

Collegamenti

- [2. Fondamenti di informatica](#) — API, JSON e i concetti tecnici di base su cui si appoggiano molti strumenti di AI
- [4. Git e GitHub](#) — Pull Request e code review, i controlli umani che restano invariati anche quando il codice è generato con assistenza AI
- [9. DevOps](#) — il ciclo DevOps di cui MLOps e AIOps sono, rispettivamente, il "cugino" e l'estensione

- [10. CI/CD](#) — la pipeline e i quality gate che verificano anche il codice generato con assistenza AI
- [13. Sicurezza](#) — autenticazione, autorizzazione, GDPR e protezione dei dati, temi centrali per i permessi di accesso di un sistema RAG e per la governance di un assistente collegato via MCP
- [16. Glossario](#) — per ripassare rapidamente ogni termine visto in questa sezione



Risorse

- [GitHub Copilot](#) — panoramica ufficiale
- [GitHub Docs](#) — Documentazione su GitHub Copilot
- [Model Context Protocol](#) — sito ufficiale del protocollo
- [DeepLearning.AI](#) — corsi introduttivi su AI e machine learning
- [Commissione Europea](#) — Strategia europea per l'intelligenza artificiale
- [OWASP](#) — Top 10 per applicazioni basate su LLM (include rischi di RAG e di integrazioni con strumenti/agenti)

16. Glossario

Questo glossario raccoglie in un unico posto tutti i termini incontrati nel corso, dai concetti base di informatica fino alla sicurezza e agli ambienti di produzione. Le voci sono in **ordine alfabetico rigoroso** (gli acronimi come CI, CPU o KPI sono ordinati per le lettere dell'acronimo stesso, non per il nome per esteso), così puoi usarlo come un vero dizionario: se leggi un termine che non ricordi mentre studi una sezione, torna qui, cercalo e poi eventualmente segui il rimando “→ approfondito nella sezione X” per tornare al capitolo che lo spiega con esempi e contesto completo.

Non serve leggerlo tutto d'un fiato: è pensato per essere consultato, non studiato in ordine. Buona consultazione.



Mappa dei concetti: come si collegano

Questo glossario ha due viste, da usare per scopi diversi: la mappa qui sotto ti mostra **come i termini si concatenano tra loro**, raggruppati per famiglia tematica, mentre l'elenco alfabetico che segue resta il dizionario da consultare voce per voce quando ti serve una definizione precisa. Le definizioni non si ripetono qui: sotto trovi solo i **collegamenti**.

Le basi tecniche

Tutto parte dal **Computer**: un insieme di componenti hardware in cui la **CPU** esegue le istruzioni usando la **RAM** come memoria di lavoro temporanea e il **Disco** come memoria permanente, organizzata in cartelle e file grazie al **File System**. Il **Sistema Operativo** fa da intermediario tra questo hardware e i programmi, e ogni programma in esecuzione diventa un **Processo**, a sua volta eventualmente suddiviso in più **Thread** che lavorano in parallelo. Su questa base tecnica gira tutto il resto: dal browser di un cliente di ShopFacile al server che elabora i suoi ordini.

Come le applicazioni comunicano

Il modello **Client-Server** descrive chi chiede (client) e chi risponde (server); per farlo, i due capi si scambiano dati attraverso una **Rete**, seguendo le regole del **TCP/IP** e, nello specifico del web, del protocollo **HTTP** (o della sua versione cifrata **HTTPS**). Prima ancora di parlarsi, il client deve sapere *dove* si trova il server: è compito del **DNS** tradurre un nome leggibile in un indirizzo. Una volta stabilita la comunicazione, i programmi si scambiano informazioni tramite una **API**, spesso progettata secondo lo stile **REST** e con i dati serializzati in **JSON** (o, più raramente oggi, **XML**).

Come si struttura un'applicazione

Un'applicazione divide le proprie responsabilità tra **Frontend** (quello che l'utente vede) e **Backend** (la logica e i dati che l'utente non vede); quest'ultimo può essere organizzato come un unico **Monolite** oppure spezzato in **Microservizi** indipendenti che comunicano tra loro, a volte tramite una **Message Queue** invece che con chiamate dirette. Un'alternativa a gestire server propri è il modello **Serverless**, dove il codice viene eseguito solo su richiesta.

Dove vivono i dati

I dati di ShopFacile si salvano in un **Database relazionale**, interrogabile con **SQL**, oppure in un **Database NoSQL** quando la struttura dei dati è più flessibile o i volumi molto grandi: la scelta tra i due dipende

dalla forma dei dati (es. il catalogo prodotti rispetto ai log delle sessioni carrello), ma anche dal **CAP Theorem**, che spiega quale compromesso tra coerenza e disponibilità ciascun database privilegia quando la rete ha un problema. Quando un database da solo non basta più, entrano in gioco due strategie complementari: la **Replication** (più copie per scalare le letture) e lo **Sharding** (dati divisi tra più macchine per scalare volumi e scritture). Per evitare di interrogare il database ad ogni richiesta, si usa spesso una **Cache-Aside (lazy loading)**, con un **TTL** che ne scandisce la scadenza.

Come si scrive e si versiona il codice

Tutto il codice vive in un **Repository**, la cui storia è fatta di **Commit** successivi, ciascuno registrato su un **Branch** dedicato per non toccare il codice principale durante lo sviluppo. Quando una modifica è pronta, si apre una **Pull Request**: qui il team fa **Code Review** prima di eseguire il **Merge** che la unisce definitivamente. Un punto del repository può poi essere marcato con un **Tag**, tipicamente per identificare una **Release**, seguendo una convenzione di **Versioning** come il **Semantic Versioning**. Il modo in cui i branch vengono organizzati nel tempo segue un modello come il **Git Flow** oppure, all'opposto, il **Trunk Based Development**; un problema scoperto lungo il percorso viene tracciato come **Issue**.

Come nasce e cresce un prodotto

Tutto comincia dai **Requisiti (funzionali e non funzionali)** che descrivono cosa deve fare il software: da lì nasce una **User Story** che lo racconta dal punto di vista dell'utente, oppure, se il lavoro è troppo grande per uno sprint, un **Epic** che raggruppa più User Story. Ogni User Story diventa una **Feature** una volta realizzata, oppure genera un **Bug** se qualcosa non funziona come previsto. Tutto questo percorso, dalla richiesta iniziale alla manutenzione, è descritto dal **Ciclo di vita del software**.

Come si organizza il lavoro

Il **Manifesto Agile** e il **Mindset Agile** superano il **Waterfall** tradizionale con due framework operativi concreti. Nello Scrum, il **Product Owner** cura il **Product Backlog**, da cui il team seleziona in **Sprint Planning** le voci dello **Sprint Backlog** per ogni **Sprint**, sincronizzandosi nel **Daily Scrum** e chiudendo con la **Sprint Review** e la **Sprint Retrospective**; il risultato è un **Increment** che rispetta la **Definition of Done (DoD)** (ed è entrato in sprint solo se soddisfaceva la **Definition of Ready (DoR)**), mentre lo **Scrum Master** facilita tutto questo. Nel Kanban lo stesso lavoro scorre su una **Board Kanban** con il **WIP (Work In Progress)** limitato per colonna, misurato in **Lead Time** e **Cycle Time** (lo **Scrumban** mescola i due mondi); gli **Story Point** e la **Velocity** quantificano la capacità del team di Luca, mentre **KPI**, **Milestone**, **RACI**, **RAID Log**, il **Rischio** e gli **Stakeholder**, insieme a **Burndown Chart** e **Burnup Chart**, tengono sotto controllo l'andamento del progetto nel suo complesso.

Come si governa un progetto (il vocabolario PMP)

Il **PMI (Project Management Institute)** raccoglie nel **PMBOK Guide** il vocabolario tradizionale del Project Management, alla base della certificazione **PMP (Project Management Professional)**: tutto parte dal **Project Charter**, il documento con cui lo **Sponsor** autorizza formalmente il progetto, da cui si scompone lo **Scope (ambito)** in una **WBS (Work Breakdown Structure)** per capire tutto il lavoro da fare. Da qui si costruisce la schedulazione, individuando il **Critical Path (percorso critico)** e rappresentandolo in un **Diagramma di Gantt**, mentre l'avanzamento viene controllato con l'**EVM (Earned Value Management)** attraverso gli indici **SPI (Schedule Performance Index)** e **CPI (Cost Performance Index)**, sempre nel rispetto del **Triplo vincolo (triple constraint)** tra tempo, costo e ambito. Ogni variazione passa da una **Change Request (richiesta di modifica)** formale, proprio per evitare lo **Scope Creep**; a fine progetto, le **Lessons Learned** chiudono il cerchio, raccogliendo cosa ha funzionato e cosa no. Questo vocabolario più formale — pensato originariamente per progetti pianificati per fasi — si sovrappone e convive con quello Agile visto sopra (**Product Backlog, Sprint Retrospective, Velocity**): nella pratica quotidiana di molti team, incluso quello di ShopFacile, i due mondi si combinano in un **Approccio ibrido (hybrid)**, usando gli strumenti PMP per la visione d'insieme e di lungo periodo e Scrum/Kanban per il lavoro operativo di ogni sprint.

Come il codice arriva agli utenti

Quando una Pull Request supera la Code Review ed entra nel branch principale, un **Trigger** avvia la **Pipeline**: build, test e poi il **Quality Gate**, il controllo automatico che blocca l'avanzamento se la qualità non basta. Se tutto va bene, la pipeline produce un **Artifact (build)** che viene promosso dall'**Ambiente di Sviluppo** all'**Ambiente di Test/QA**, poi allo **Staging** e infine alla **Produzione** tramite il **Deploy/Rilascio**; le pratiche di **CI (Continuous Integration)** e **Continuous Delivery/Continuous Deployment** sono ciò che rende questo percorso frequente e automatico invece che manuale e raro. Se qualcosa va storto dopo il rilascio, il **Rollback** riporta rapidamente alla versione precedente.

Dove gira il software

Un Artifact, per essere eseguito ovunque allo stesso modo, viene spesso pacchettizzato in un **Container** tramite **Docker**, e quando i container sono troppi da gestire a mano entra in gioco **Kubernetes**, che li orchestra automaticamente. Un'alternativa più tradizionale è la **Virtual Machine (VM)**, che simula un intero computer; la capacità di questi ambienti di gestire più carico segue i due modi descritti da **Scalabilità verticale e orizzontale**. Tutto questo gira su infrastrutture cloud offerte secondo tre livelli di responsabilità crescente per il fornitore — **IaaS, PaaS e SaaS** — quasi sempre pagate a consumo con il modello **Pay-as-you-go**.

Come si tiene tutto in piedi e sicuro

Una volta in produzione, il **Monitoring** osserva lo stato del sistema e il **Logging** ne registra gli eventi; l'**Observability** va oltre entrambi, aiutando a capire *perché* qualcosa non funziona, non solo *cosa*. Sul fronte sicurezza, l'**Autenticazione** verifica chi sei e l'**Autorizzazione** stabilisce cosa puoi fare, spesso rafforzate da **MFA (autenticazione a più fattori)**; le vulnerabilità più comuni sono catalogate da **OWASP**, mentre il **GDPR** impone regole precise sul trattamento dei dati personali degli utenti di ShopFacile. Il **DevSecOps** integra questi controlli di sicurezza direttamente nella pipeline, invece di lasciarli come verifica finale isolata.

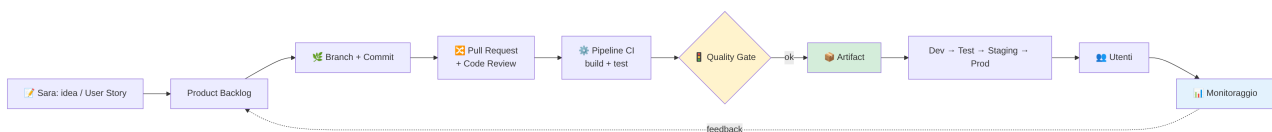
La cultura che tiene insieme tutto

Il **DevOps** è la cornice culturale che tiene insieme tutti i fili visti sopra: il **CALMS** ne riassume i cinque pilastri, mentre la **Cultura DevOps** descrive concretamente cosa cambia nel modo di lavorare quotidiano di un team come quello di ShopFacile, fino a pratiche come l'**IaC (Infrastructure as Code)** che trattano l'infrastruttura come se fosse codice, versionabile e revisionabile come qualsiasi Pull Request.

Come l'AI entra nel lavoro del team

Tutto parte dai dati: attraverso il **Training (addestramento)**, un algoritmo di **Machine Learning** (o, più nello specifico, di **Deep Learning**) impara pattern da grandi quantità di esempi e produce un **Modello (AI)**, che poi viene interrogato in fase di **Inferenza** per generare risposte su casi nuovi. Gli **LLM (Large Language Model)** sono uno dei risultati più visibili di questo processo, e alimentano la categoria dell'**AI generativa**: strumenti concreti come **Copilot (GitHub Copilot)** nascono proprio da qui, trasformando un **Prompt** scritto dallo sviluppatore in suggerimenti di codice. Questa tecnologia ha però dei limiti che il team di ShopFacile deve conoscere prima di fidarsene ciecamente: l'**Allucinazione (AI)**, il **Bias** ereditato dai dati di training, e l'**Overfitting**, quando un modello impara troppo bene gli esempi visti e generalizza male su quelli nuovi. Proprio come il **DevOps** ha portato automazione e collaborazione al rilascio del software, l'**MLOps** ne applica gli stessi principi al ciclo di vita dei modelli AI, mentre l'**AIOps** usa queste stesse tecniche per potenziare il monitoraggio dei sistemi in produzione. Un **LLM (Large Language Model)** da solo, però, non conosce i documenti interni di un'azienda come ShopFacile né può agire sui suoi sistemi: il **RAG (Retrieval-Augmented Generation)** risolve il primo limite recuperando al momento della domanda i pezzi (**Chunk**) di documentazione più pertinenti da un **Database vettoriale**, individuati tramite **Ricerca semantica** sugli **Embedding**, invece di dover riaddestrare il modello con il **Fine-tuning**; il **MCP (Model Context Protocol)**, con i suoi **MCP server**, risolve il secondo, dando allo stesso modello un modo standard per collegarsi a strumenti e sistemi reali. In sintesi: il primo dà al modello la conoscenza, il secondo gli dà la capacità di agire.

Il percorso end-to-end in un unico schema



ACID

L'insieme di quattro proprietà che una transazione di database deve garantire: Atomicity (tutto o nulla), Consistency (il database resta in uno stato valido), Isolation (le transazioni concorrenti non si vedono a metà strada) e Durability (una modifica confermata resta anche dopo un crash). → approfondito nella sezione 20 (Backend Engineering Avanzato). Si collega a: **MVCC** (la tecnica con cui i database moderni implementano l'isolamento senza bloccare tutto) e **Livello di isolamento** (che stabilisce quanto rigorosamente viene garantita l'isolation).

Active Directory (AD)

Il servizio più diffuso nelle aziende enterprise per gestire in un unico posto l'elenco di utenti, gruppi, computer e politiche di sicurezza, invece di farli gestire a ogni applicazione per conto proprio; parla tipicamente il protocollo LDAP per farsi interrogare dalle altre applicazioni. → approfondito nella sezione 13 (Sicurezza). Si collega a: **LDAP** (il protocollo con cui viene interrogato) e **SSO (Single Sign-On)** (il meccanismo che tipicamente si costruisce sopra di esso).

ADR (Architecture Decision Record)

Un documento breve, uno per ogni decisione tecnica rilevante, scritto nel momento in cui la decisione viene presa (non ricostruito a posteriori) e versionato nel repository accanto al codice: serve a non perdere il **perché** di una scelta, non solo il **cosa**. Un ADR non si cancella né si riscrive: se una decisione viene ripensata, si scrive un nuovo ADR che dichiara di superare il precedente. → approfondito nella sezione 3 (Come nasce un software).

AI (Intelligenza Artificiale)

Sigla di Artificial Intelligence: la capacità di un sistema informatico di svolgere compiti che normalmente richiederebbero l'intelligenza umana, come riconoscere immagini, capire il linguaggio o generare testo. → approfondito nella sezione 15 (Intelligenza artificiale).

AI generativa

Un tipo di intelligenza artificiale capace di creare contenuti nuovi (testo, immagini, codice) invece di limitarsi a classificare o prevedere dati esistenti; è la categoria a cui appartengono strumenti come ChatGPT o GitHub Copilot. → approfondito nella sezione 15 (Intelligenza artificiale). Si collega a: **LLM (Large Language Model)** (la tecnologia che più spesso la rende possibile) e **Copilot (GitHub Copilot)** (uno strumento concreto che la usa).

AIOps

L'uso di tecniche di intelligenza artificiale e machine learning per automatizzare il monitoraggio, l'analisi dei log e la diagnosi dei problemi in un sistema IT complesso, individuando anomalie più rapidamente di un controllo puramente manuale. → approfondito nella sezione 15 (Intelligenza artificiale). Si collega a: **Monitoring** (che potenzia con tecniche di AI) e **MLOps** (di cui condivide la logica di automazione, applicata qui alle operations invece che al ciclo di vita del modello).

Allucinazione (AI)

Una risposta di un modello di intelligenza artificiale che sembra sicura e ben scritta ma è in realtà falsa o inventata, ad esempio citare una funzione di libreria che non esiste. È uno dei motivi per cui l'output di uno strumento AI va sempre verificato da una persona, non accettato automaticamente. → approfondito nella sezione 15 (Intelligenza artificiale). Si collega a: **Modello (AI)** (che la produce) e **Training (addestramento)** (i cui limiti e le cui lacune sono spesso all'origine del problema).

Ambiente di Sviluppo

L'ambiente (development environment) in cui gli sviluppatori scrivono e provano il codice per la prima volta, di solito sul proprio computer o in uno spazio dedicato non visibile agli utenti finali. È il primo “gradino” prima che il codice arrivi al Test/QA, allo Staging e infine alla Produzione. → approfondito nella sezione 14 (Ambienti di sviluppo).

Ambiente di Test/QA

Un ambiente separato da quello di sviluppo, usato per verificare (Quality Assurance) che una funzionalità funzioni correttamente prima di mostrarla a chiunque altro. Qui si eseguono i test manuali e automatici in condizioni più simili a quelle reali. → approfondito nella sezione 14 (Ambienti di sviluppo). Si collega a: **Ambiente di Sviluppo** (che lo precede) e **Staging** (che ne consegue, prima della Produzione).

API

Sigla di Application Programming Interface: è un insieme di regole che permette a due programmi di “parlarsi” e scambiarsi informazioni, senza che uno debba conoscere i dettagli interni dell'altro. Pensala come un menu di un ristorante: tu scegli una voce (una richiesta) e la cucina (il sistema) ti restituisce il piatto (la risposta). → approfondito nella sezione 2 (Fondamenti di informatica).

Approccio ibrido (hybrid)

Un modo di gestire un progetto che unisce elementi del Project Management tradizionale (quello descritto da PMP e PMBOK Guide) con le pratiche Agile viste nelle sezioni precedenti, usando lo strumento più adatto a ogni situazione invece di seguire un solo metodo in modo rigido. → approfondito nella sezione 8 (Project Management). Si collega a: **PMBOK Guide** (il vocabolario tradizionale) e **Mindset Agile** (il vocabolario iterativo con cui qui convive).

Artifact (build)

Il “prodotto finito” generato da una pipeline dopo aver compilato e testato il codice: ad esempio un file eseguibile, un pacchetto installabile o un'immagine Docker pronta per essere distribuita. È diverso da un servizio come GitHub Packages, che conserva pacchetti e librerie riutilizzabili invece del singolo prodotto di una pipeline. → approfondito nella sezione 10 (CI/CD). Si collega a: **Pipeline** (che lo produce) e **Deploy/Rilascio** (che lo installa, identico, in ogni ambiente).

Autenticazione

Il processo con cui un sistema verifica che tu sia davvero chi dici di essere, tipicamente chiedendo username e password (authentication). È il primo passo della sicurezza: prima si verifica l'identità, poi si decide cosa quella identità può fare. → approfondito nella sezione 13 (Sicurezza).

Autorizzazione

Il processo che stabilisce, dopo che sei stato riconosciuto (autenticazione), a quali risorse puoi accedere e quali azioni puoi compiere (authorization). Esempio: essere autenticati come dipendenti non significa automaticamente poter modificare i dati contabili dell'azienda. → approfondito nella sezione 13 (Sicurezza).

Backend

La parte “dietro le quinte” di un'applicazione: server, database e logica di business che l'utente non vede direttamente, ma che elabora le richieste e restituisce i risultati al Frontend. → approfondito nella sezione 11 (Architetture software).

Batch (elaborazione batch) / Farm batch

L'elaborazione di grandi volumi di dati “a lotti”, tutti insieme e secondo una schedulazione (a un orario fisso o dopo un evento), invece che uno alla volta in risposta a una richiesta dell'utente: tipica del lavoro notturno di un'assicurazione (calcolo premi, produzione documenti, invii a enti esterni). La farm batch è l'insieme delle macchine dedicate a farlo girare, spesso separate dai server che servono il sito o l'app. → approfondito nella sezione 9 (DevOps). Si collega a: **ITSM** (di cui condivide l'attenzione operativa) e **Disaster Recovery** (rilevante se la farm batch è ferma per un guasto).

BIA (Business Impact Analysis)

L'analisi che stima, processo per processo, quanto costa un'ora di fermo in senso di business (non tecnico): quanti ordini non confermati, quanti clienti persi, quale obbligo normativo non rispettato. È il numero che determina quale RTO e quale RPO ha senso concordare per ciascun sistema: una decisione di business, non una scelta tecnica lasciata al team IT. → approfondito nella sezione 13 (Sicurezza). Si collega a: **Business Continuity** (il piano che la BIA aiuta a costruire) e **Disaster Recovery** (i cui RTO/RPO la BIA determina).

Bias

Una distorsione sistematica nelle risposte di un modello di intelligenza artificiale, che nasce quasi sempre dai dati con cui è stato addestrato (ad esempio se quei dati rappresentano in modo sbilanciato certi gruppi o scenari) e che il team deve conoscere per usare questi strumenti in modo responsabile. → approfondito nella sezione 15 (Intelligenza artificiale). Si collega a: **Training (addestramento)** (la fase in cui più spesso si origina) e **Modello (AI)** (che lo eredita e lo riproduce nelle risposte).

Board Kanban

Una lavagna (fisica o digitale) divisa in colonne — tipicamente “Da fare”, “In corso”, “Fatto” — che rende visibile lo stato di avanzamento di ogni attività del team con dei cartellini (le card). → approfondito nella sezione 7 (Kanban).

Branch

Una “linea di sviluppo” separata all'interno di un repository Git, che permette di lavorare su una modifica senza toccare il codice principale finché non è pronta. → approfondito nella sezione 4 (Git e GitHub). Si collega a: **Repository** (che lo contiene) e **Pull Request** (che ne propone l'unione al codice principale).

Branching

La pratica di creare e gestire più Branch per isolare il lavoro su funzionalità diverse, permettendo a più persone di lavorare in parallelo senza intralciarsi. → approfondito nella sezione 3 (Come nasce un software).

Bug

Un errore nel software che produce un comportamento diverso da quello previsto: dal termine inglese per “insetto”, nato da un aneddoto storico legato a un vero insetto trovato in un computer degli anni '40. → approfondito nella sezione 3 (Come nasce un software).

Builder image

La prima fase di un multi-stage build: un'immagine Docker “temporanea” che contiene tutti gli strumenti necessari a compilare l'applicazione (compilatore, dipendenze di sviluppo), ma che non finisce mai in produzione — solo il risultato della compilazione viene copiato nell'immagine finale, più piccola e senza gli strumenti di build. → approfondito nella sezione 10 (CI/CD). Si collega a: **Multi-stage build** (la tecnica che la usa) e **Artifact (build)** (il prodotto finale, più leggero, che ne deriva).

Burndown Chart

Un grafico che mostra quanto lavoro **resta** da fare in uno Sprint (o in un progetto) man mano che passano i giorni: la linea idealmente “scende” verso lo zero. → approfondito nella sezione 8 (Project Management).

Burnup Chart

Un grafico simile al Burndown Chart, ma che mostra il lavoro **già completato** che “sale” verso il totale previsto, rendendo più facile notare se l'ambito del progetto (lo scope) cresce nel tempo. → approfondito nella sezione 8 (Project Management).

Business Continuity (BC)

La disciplina che risponde alla domanda “come continua a funzionare l'azienda mentre i sistemi non ci sono?”, distinta dal Disaster Recovery che risponde invece a “come rimetto in piedi i sistemi?”. Include procedure alternative per le persone (cosa fa il call center quando il sistema è giù), comunicazione di crisi e test periodici del piano. → approfondito nella sezione 13 (Sicurezza). Si collega a: **Disaster Recovery** (il piano tecnico complementare) e **BIA (Business Impact Analysis)** (l'analisi che ne guida le priorità).

Cache-Aside (lazy loading)

La strategia di caching più diffusa: l'applicazione controlla prima la cache; se il dato c'è (cache hit) lo restituisce subito senza toccare il database, se non c'è (cache miss) lo recupera dal database, lo salva in cache per la prossima volta, e poi risponde. È semplice e “fallisce bene”: se la cache non è disponibile, il sistema continua a funzionare appoggiandosi al database, solo più lentamente. → approfondito nella sezione 20 (Backend Engineering Avanzato). Si collega a: **TTL (Time To Live)** (una delle tecniche più comuni per invalidare i dati salvati con questa strategia).

Cactus (modello di branching)

Un modello di branching Git, noto anche come OneFlow, che si colloca tra Git Flow e Trunk Based Development: un solo tronco principale a lungo termine, branch di feature brevi, e branch di release solo quando serve davvero gestire più versioni in produzione contemporaneamente. → approfondito nella sezione 4 (Git e GitHub). Si collega a: **Git Flow** (il modello più strutturato da cui si differenzia) e **Trunk Based Development** (il modello più semplice all'altro estremo).

CAP Theorem

Un teorema che afferma che, in un sistema distribuito, si possono garantire solo due delle tre proprietà Consistency (tutti i nodi vedono gli stessi dati), Availability (il sistema resta sempre disponibile) e Partition Tolerance (il sistema funziona anche con la rete interrotta tra i nodi): dato che la partition tolerance non è quasi mai opzionale in un sistema reale, la scelta pratica è tra Consistency e Availability. → approfondito nella sezione 20 (Backend Engineering Avanzato). Si collega a: **Database NoSQL** (i cui diversi prodotti fanno scelte diverse su questo teorema) e **Sharding** (un'altra strategia tipica dei sistemi distribuiti a cui il teorema si applica).

CALMS

Un acronimo (Culture, Automation, Lean, Measurement, Sharing) che riassume i cinque pilastri della cultura DevOps: cultura collaborativa, automazione, approccio lean, misurazione dei risultati e condivisione della conoscenza. → approfondito nella sezione 9 (DevOps).

Change Request (richiesta di modifica)

Una proposta formale di modificare qualcosa già deciso nel progetto — ambito, tempi, costi o requisiti — che va valutata e approvata prima di essere applicata, invece di lasciare che il progetto cambi rotta senza controllo. → approfondito nella sezione 8 (Project Management). Si collega a: **Scope Creep** (il rischio che corre un progetto se le richieste di modifica non passano da qui).

Chiave esterna (foreign key)

Una colonna (o gruppo di colonne) di una tabella che contiene lo stesso valore della chiave primaria di un'altra tabella, creando così una relazione tra le due; il database la fa rispettare attivamente, rifiutando di creare righe che puntano a un valore inesistente (integrità referenziale). → approfondito nella sezione 2 (Fondamenti di informatica). Si collega a: **Chiave primaria** (a cui punta) e **JOIN** (l'operazione SQL che la usa per unire le due tabelle).

Chiave primaria (primary key)

Una colonna (o un piccolo gruppo di colonne) che identifica in modo univoco e stabile ogni riga di una tabella, e che il database si impegna a non far duplicare mai: deve essere unica, mai vuota e stabile nel tempo. → approfondito nella sezione 2 (Fondamenti di informatica). Si collega a: **Chiave esterna** (che la riferenzia da un'altra tabella) e **Database relazionale** (di cui è un elemento cardine).

Chunk

Un piccolo pezzo di testo (poche frasi o un paragrafo) in cui viene spezzato un documento lungo prima di elaborarlo con l'intelligenza artificiale, ad esempio nel RAG: dividere in pezzi più piccoli permette di recuperare solo la parte di documento davvero rilevante per una domanda, invece di doverlo leggere per intero ogni volta. → approfondito nella sezione 15 (Intelligenza artificiale).

CI (Continuous Integration)

La pratica di integrare frequentemente (più volte al giorno) il proprio codice con quello del resto del team, verificando automaticamente con dei test che tutto continui a funzionare insieme. → approfondito nella sezione 9 (DevOps) e nella sezione 10 (CI/CD). Si collega a: **Pipeline** (il meccanismo che la rende concreta) e **Quality Gate** (il controllo che ne verifica l'esito).

Ciclo di vita del software

L'insieme delle fasi che un software attraversa dalla sua nascita alla sua “pensione”: analisi dei requisiti, progettazione, sviluppo, test, rilascio e manutenzione (software development lifecycle). → approfondito nella sezione 3 (Come nasce un software).

Client-Server

Un modello architetturale in cui un programma “client” (ad esempio il tuo browser) richiede informazioni o servizi a un programma “server”, che risponde elaborando la richiesta. → approfondito nella sezione 11 (Architetture software).

Code Review

La pratica di far leggere e commentare il proprio codice a un collega prima di unirlo al progetto principale, per individuare errori, migliorare la qualità e condividere conoscenza nel team. → approfondito nella sezione 3 (Come nasce un software). Si collega a: **Pull Request** (che la richiede) e **Merge** (l'operazione che la conclude, una volta approvata).

Commit

Uno “scatto fotografico” delle modifiche fatte al codice in un preciso momento, salvato nella storia del repository con un messaggio che ne descrive il contenuto. → approfondito nella sezione 4 (Git e GitHub). Si collega a: **Branch** (su cui viene registrato) e **Pull Request** (che raccoglie una serie di commit da integrare).

Computer

Una macchina capace di eseguire istruzioni (programmi) per elaborare dati: è composta da componenti hardware (come CPU e RAM) che collaborano seguendo le indicazioni del software. → approfondito nella sezione 2 (Fondamenti di informatica).

Container

Un “pacchetto” leggero e isolato che contiene un’applicazione insieme a tutto ciò che le serve per funzionare (librerie, configurazioni), così da poterla eseguire in modo identico su qualsiasi computer. Docker è la tecnologia più usata per crearli. → approfondito nella sezione 2 (Fondamenti di informatica). Si collega a: **Docker** (lo strumento più diffuso per crearli) e **Kubernetes** (che li orchestra su larga scala).

Continuous Delivery

La pratica che assicura che il software, dopo aver superato tutti i test automatici, sia sempre pronto per essere rilasciato in produzione in qualsiasi momento, anche se il rilascio finale resta una decisione manuale. → approfondito nella sezione 9 (DevOps). Si collega a: **CI (Continuous Integration)** (che la precede nel percorso verso il rilascio) e **Deploy/Rilascio** (l’operazione finale, qui ancora decisa manualmente).

Continuous Deployment

Un’evoluzione della Continuous Delivery in cui ogni modifica che supera i test viene rilasciata automaticamente in produzione, senza intervento manuale. → approfondito nella sezione 9 (DevOps). Si collega a: **Continuous Delivery** (di cui è l’evoluzione automatica) e **Deploy/Rilascio** (che qui avviene senza intervento umano).

Copilot (GitHub Copilot)

Uno strumento di AI generativa integrato nell’editor di codice che suggerisce automaticamente righe o intere funzioni mentre uno sviluppatore scrive, basandosi su un LLM addestrato su enormi quantità di codice pubblico. Non sostituisce lo sviluppatore: velocizza compiti ripetitivi, ma il codice suggerito va sempre letto e verificato in Code Review come qualsiasi altro contributo. → approfondito nella sezione 15 (Intelligenza artificiale). Si collega a: **AI generativa** (la categoria a cui appartiene) e **Code Review** (il controllo che il codice suggerito deve comunque superare).

CPI (Cost Performance Index)

Un indice dell’Earned Value Management che misura quanto efficientemente il progetto sta spendendo il proprio budget: si calcola dividendo il valore del lavoro effettivamente completato (EV) per il costo reale sostenuto (AC). Un CPI sopra 1 significa che il progetto sta spendendo meno di quanto pianificato per il lavoro fatto, sotto 1 significa che sta spendendo di più. → approfondito nella sezione 8 (Project Management). Si collega a: **EVM (Earned Value Management)** (il sistema di cui fa parte) e **SPI (Schedule Performance Index)** (l’indice analogo sui tempi).

CPU

Sigla di Central Processing Unit, il “cervello” del computer: il componente che esegue materialmente le istruzioni dei programmi, facendo calcoli a velocità elevatissima. → approfondito nella sezione 2 (Fondamenti di informatica).

Critical Path (percorso critico)

La sequenza di attività di un progetto che, messe una dopo l'altra, determinano la durata minima totale del progetto: se anche una sola di queste attività si ritarda, l'intero progetto si ritarda. Le attività non sul percorso critico hanno invece un margine di ritardo (la “slack”) che non impatta la data finale. → approfondito nella sezione 8 (Project Management). Si collega a: **Diagramma di Gantt** (lo strumento visivo con cui si rappresenta) e **WBS (Work Breakdown Structure)** (da cui derivano le attività su cui si calcola).

CRUD

Acronimo di Create, Read, Update, Delete (Crea, Leggi, Aggiorna, Elimina): le quattro operazioni fondamentali che si possono fare su un qualsiasi dato salvato da qualche parte, corrispondenti ai verbi HTTP POST/GET/PUT-PATCH/DELETE e ai comandi SQL INSERT/SELECT/UPDATE/DELETE. Quando un tecnico dice “è solo un CRUD” indica una stima semplice, senza logica di business particolare. → approfondito nella sezione 2 (Fondamenti di informatica). Si collega a: **REST** (i verbi HTTP con cui si esprime) e **SQL** (i comandi con cui si realizza sul database).

Cultura DevOps

Un modo di lavorare che abbatte le barriere tra chi sviluppa il software (Dev) e chi lo gestisce in produzione (Ops), promuovendo collaborazione, responsabilità condivisa e automazione. → approfondito nella sezione 9 (DevOps).

Cycle Time

Il tempo che intercorre dal momento in cui il team **inizia effettivamente a lavorare** su un'attività al momento in cui la completa. È una misura più “interna” del Lead Time. → approfondito nella sezione 7 (Kanban).

Daily Scrum

Un breve incontro quotidiano (di solito 15 minuti) in cui il team Scrum sincronizza il lavoro, condivide progressi e segnala eventuali ostacoli. → approfondito nella sezione 6 (Scrum).

Data drift

Il progressivo “invecchiamento” di un modello di intelligenza artificiale che accade quando i dati reali su cui opera cambiano nel tempo rispetto a quelli usati durante il Training, facendo peggiorare silenziosamente le sue previsioni se nessuno se ne accorge. → approfondito nella sezione 15 (Intelligenza artificiale). Si collega a: **Training (addestramento)** (la base che diventa via via meno rappresentativa) e **MLOps** (la disciplina che si occupa di individuarlo e correggerlo).

Database NoSQL

Un database che non organizza i dati in tabelle rigide come quelli relazionali, ma in formati più flessibili (documenti, chiavi-valore, grafi), utile quando i dati cambiano forma spesso o servono grandi volumi. → approfondito nella sezione 2 (Fondamenti di informatica).

Database relazionale

Un database che organizza i dati in tabelle collegate tra loro tramite relazioni, un po' come fogli Excel che si richiamano a vicenda; si interroga con il linguaggio SQL. → approfondito nella sezione 2 (Fondamenti di informatica).

Database vettoriale

Un tipo particolare di database pensato per salvare e ricercare Embedding, cioè rappresentazioni numeriche del significato di un testo: invece di cercare parole esatte, permette di trovare rapidamente i contenuti più simili nel significato a una domanda, anche tra milioni di documenti. → approfondito nella sezione 15 (Intelligenza artificiale).

Debito tecnico

L'insieme dei compromessi tecnici presi per rilasciare più in fretta (codice scritto "alla svelta", scorciatoie di progettazione, test saltati) che, come un debito finanziario, andranno prima o poi "ripagati" con lavoro di manutenzione, e che nel frattempo rendono più lento e rischioso ogni intervento successivo sul codice. → approfondito nella sezione 6 (Scrum).

Deep Learning

Un ramo del Machine Learning basato su reti neurali artificiali organizzate in molti "strati" (deep, cioè profondo), particolarmente efficace per compiti complessi come il riconoscimento di immagini o la comprensione del linguaggio naturale, ed è la tecnologia alla base della maggior parte degli LLM moderni. → approfondito nella sezione 15 (Intelligenza artificiale). Si collega a: **Machine Learning** (il campo più ampio di cui è una tecnica specifica) e **LLM (Large Language Model)** (uno dei risultati più visibili di questa tecnica).

Definition of Done (DoD)

L'elenco condiviso di criteri che un'attività deve soddisfare per essere considerata davvero "completata" (ad esempio: codice scritto, testato, revisionato e documentato), evitando ambiguità nel team. → approfondito nella sezione 6 (Scrum).

Definition of Ready (DoR)

L'elenco di criteri che una User Story deve soddisfare prima di poter essere accettata in uno Sprint (ad esempio: requisiti chiari, stima fatta, dipendenze note). → approfondito nella sezione 6 (Scrum).

Deploy/Rilascio

L'operazione di rendere disponibile una nuova versione del software agli utenti, spostandola dall'ambiente in cui è stata sviluppata e testata a quello di produzione. → approfondito nella sezione 3 (Come nasce un software). Si collega a: **Artifact (build)** (ciò che viene effettivamente distribuito) e **Rollback** (l'operazione da compiere se il rilascio va male).

Deriva di configurazione

Il progressivo disallineamento tra ambienti che dovrebbero somigliarsi (es. Test/QA, Staging, Produzione) quando una modifica di configurazione viene fatta in uno solo di essi e non replicata negli altri, portando col tempo a test che “passano” senza dire nulla sul comportamento reale. → approfondito nella sezione 14 (Ambienti di sviluppo).

Developer Experience (DX)

Quanto è facile, per chi scrive codice, capire cosa fare e farlo: documentazione chiara, messaggi d'errore utili, strumenti che non richiedono di leggere un manuale prima di poterli usare. È uno dei tre pilastri di un Internal Developer Platform, insieme a standardizzazione e automazione. → approfondito nella sezione 12 (Cloud). Si collega a: **IDP (Internal Developer Platform)** (lo strumento che la mette in pratica) e **Golden path** (il percorso raccomandato che la incarna).

DevOps

La fusione tra “Development” (sviluppo) e “Operations” (gestione dei sistemi): un approccio che unisce persone, processi e strumenti per rilasciare software in modo più rapido, frequente e affidabile. → approfondito nella sezione 9 (DevOps). Si collega a: **CALMS** (che ne riassume i pilastri) e **CI (Continuous Integration)** (una delle pratiche concrete che lo realizzano).

DevSecOps

Un'estensione del DevOps che integra la sicurezza (Security) in ogni fase del ciclo di vita del software, invece di considerarla solo un controllo finale prima del rilascio. → approfondito nella sezione 13 (Sicurezza).

Diagramma di Gantt

Un grafico a barre orizzontali che mostra le attività di un progetto lungo una linea del tempo, con inizio, fine e dipendenze reciproche visibili a colpo d'occhio. È lo strumento visivo classico della pianificazione “a cascata” (Waterfall), ma resta utile anche in un contesto Agile per comunicare scadenze e dipendenze a stakeholder che non seguono la board del team giorno per giorno. → approfondito nella sezione 8 (Project Management). Si collega a: **Critical Path** (il percorso che il diagramma rende visibile) e **WBS (Work Breakdown Structure)** (le attività che il diagramma colloca nel tempo).

Disco

Il componente del computer dove i dati vengono salvati in modo permanente (anche da spento), a differenza della RAM che perde i dati quando il computer si spegne. → approfondito nella sezione 2 (Fondamenti di informatica).

DNS

Sigla di Domain Name System: il “elenco telefonico” di internet, che traduce nomi di siti facili da ricordare (come esempio.it) negli indirizzi numerici (IP) che i computer usano davvero per comunicare. → approfondito nella sezione 2 (Fondamenti di informatica).

Docker

Lo strumento più diffuso per creare, eseguire e distribuire Container, cioè applicazioni “pacchettizzate” insieme a tutto ciò che serve loro per funzionare ovunque nello stesso modo. → approfondito nella sezione 2 (Fondamenti di informatica). Si collega a: **Container** (ciò che crea e distribuisce) e **Artifact (build)** (un'immagine Docker è uno dei formati più comuni di artifact).

Embedding

Una rappresentazione numerica (una lista di numeri) del significato di un testo, un'immagine o altro contenuto, calcolata da un modello di intelligenza artificiale: due contenuti con significato simile avranno un embedding simile, anche se usano parole diverse. È la base tecnica che rende possibile la Ricerca semantica. → approfondito nella sezione 15 (Intelligenza artificiale).

Epic

Un'attività molto grande, che raggruppa più User Story legate a un unico obiettivo di ampio respiro, troppo estesa per essere completata in un singolo Sprint. → approfondito nella sezione 6 (Scrum).

Event-Driven Architecture

Un'architettura in cui un servizio, invece di chiamare direttamente chi deve fare qualcosa, pubblica un evento (un fatto accaduto) su un topic dedicato; chi è interessato (consumer) reagisce in autonomia, senza che chi pubblica (producer) debba conoscerlo. Il meccanismo con cui producer e consumer si incontrano si chiama pub/sub. → approfondito nella sezione 11 (Architetture software). Si collega a: **Evento** (l'unità che vi circola) e **Message Queue** (l'infrastruttura sottostante, usata qui in modo diverso).

Evento

Un fatto accaduto al passato, per definizione immutabile (es. “la polizza è stata sottoscritta”): a differenza di un comando, non è indirizzato a nessuno in particolare, non richiede una risposta e non può essere “rifiutato” perché è già successo. → approfondito nella sezione 11 (Architetture software). Si collega a: **Event-Driven Architecture** (il pattern che si basa su di esso).

EVM (Earned Value Management)

Una tecnica di Project Management che confronta tre valori — quanto lavoro era pianificato (PV, Planned Value), quanto è stato effettivamente completato in termini di valore (EV, Earned Value) e quanto è realmente costato (AC, Actual Cost) — per capire in modo oggettivo se un progetto è in linea con tempi e budget, invece di basarsi solo su una sensazione. → approfondito nella sezione 8 (Project Management). Si collega a: **CPI (Cost Performance Index)** e **SPI (Schedule Performance Index)** (i due indici che questa tecnica calcola).

Feature

Una funzionalità del software: una caratteristica o capacità concreta che l'utente può usare (ad esempio “possibilità di esportare un report in PDF”). → approfondito nella sezione 3 (Come nasce un software).

Feature flag

Un interruttore nel codice che permette di attivare o disattivare una funzionalità già distribuita in produzione senza fare un nuovo rilascio, ad esempio per mostrarla solo a un gruppo ristretto di utenti o per disattivarla rapidamente se causa problemi. → approfondito nella sezione 10 (CI/CD).

File System

Il modo in cui un sistema operativo organizza e conserva i file su un disco, tramite cartelle e nomi, permettendo di ritrovarli e gestirli facilmente. → approfondito nella sezione 2 (Fondamenti di informatica).

Fine-tuning

Il processo di riaddestrare ulteriormente un modello di intelligenza artificiale già pronto, usando dati specifici di un'azienda o di un compito, per specializzarlo su quel contesto invece di usarlo “come viene”. È un'alternativa al RAG per dare a un modello conoscenza specifica: più lenta e costosa da aggiornare, ma capace di cambiare più a fondo il comportamento del modello. → approfondito nella sezione 15 (Intelligenza artificiale).

FinOps

La disciplina (dall'unione di “Finance” e “Operations”) che si occupa di gestire, prevedere e ottimizzare la spesa cloud di un'azienda, tramite allarmi di spesa, attribuzione dei costi ai progetti giusti e collaborazione tra team tecnici e finanziari. → approfondito nella sezione 12 (Cloud).

Frontend

La parte di un'applicazione con cui l'utente interagisce direttamente: l'interfaccia visibile (pagine web, schermate, pulsanti), che comunica con il Backend per mostrare e inviare informazioni. → approfondito nella sezione 11 (Architetture software).

GDPR

Il Regolamento Generale sulla Protezione dei Dati (General Data Protection Regulation), la legge europea che stabilisce come le aziende devono raccogliere, trattare e proteggere i dati personali delle persone. → approfondito nella sezione 13 (Sicurezza).

Git Flow

Un modello ben definito di organizzazione dei Branch in Git, con rami dedicati per lo sviluppo, le funzionalità, i rilasci e le correzioni urgenti, utile in progetti con rilasci pianificati e meno frequenti. → approfondito nella sezione 4 (Git e GitHub).

Golden path

Il percorso raccomandato e già pronto per costruire e rilasciare un servizio, che copre la maggior parte dei casi tipici: non l'unico percorso possibile (un team con esigenze particolari può uscirne), ma quello che rende semplice fare la cosa giusta senza dover reinventare tutto da zero. → approfondito nella sezione 12 (Cloud). Si collega a: **IDP (Internal Developer Platform)** (lo strumento che lo rende disponibile) e **Self-service** (il modo in cui viene ottenuto).

HTML

Il linguaggio di marcatura che descrive la struttura di una pagina web (titoli, paragrafi, link, immagini) tramite tag, dicendo al browser cosa mostrare; a differenza di Markdown, gestisce anche la formattazione visiva più fine (tramite CSS) ed è il "compilato" verso cui Markdown viene spesso convertito. → approfondito nella sezione 2 (Fondamenti di informatica). Si collega a: **Markdown** (il linguaggio più semplice da cui viene spesso generato) e **XML** (un altro linguaggio a tag, ma pensato per i dati non per le pagine).

HTTP

Sigla di HyperText Transfer Protocol: il linguaggio con cui il browser e i siti web si scambiano informazioni su internet, ad esempio quando richiedi una pagina web. → approfondito nella sezione 2 (Fondamenti di informatica).

HTTPS

La versione sicura e cifrata di HTTP (la "S" sta per Secure), che protegge le informazioni scambiate tra browser e sito web da occhi indiscreti, oggi usata praticamente ovunque. → approfondito nella sezione 2 (Fondamenti di informatica).

IaaS

Sigla di Infrastructure as a Service: un modello cloud in cui il fornitore ti dà "solo" server, storage e rete virtuali, e sei tu a occuparti di sistema operativo, applicazioni e configurazioni. → approfondito nella sezione 12 (Cloud).

IaC (Infrastructure as Code)

La pratica di descrivere l'infrastruttura informatica (server, reti, configurazioni) tramite file di testo/codice invece che configurandola manualmente, così da poterla creare, versionare e ripetere in modo automatico e affidabile. → approfondito nella sezione 9 (DevOps).

Idempotenza

La proprietà di un'operazione per cui eseguirla più volte con lo stesso input produce lo stesso risultato di eseguirla una sola volta: fondamentale per i consumer di un'architettura event-driven, dove un evento può arrivare duplicato (es. controllare "ho già incassato questo premio?" invece di incassarlo incondizionatamente). → approfondito nella sezione 11 (Architetture software). Si collega a: **Event-Driven Architecture** (il contesto in cui diventa cruciale) e **Evento** (che può arrivare più di una volta allo stesso consumer).

IDP (Internal Developer Platform)

Il livello di self-service che un team di piattaforma costruisce sopra l'infrastruttura (cloud, Kubernetes, pipeline), così che un team di sviluppo possa ottenere ambienti, database o pipeline senza passare ogni volta da zero. Si basa su tre pilastri: standardizzazione, automazione e Developer Experience. → approfondito nella sezione 12 (Cloud). Si collega a: **Platform Engineering** (la disciplina che lo costruisce) e **Golden path** (il percorso che offre).

Increment

Il risultato concreto e utilizzabile prodotto durante uno Sprint: la somma di tutte le User Story completate, che si aggiunge a quanto già costruito nei Sprint precedenti. → approfondito nella sezione 6 (Scrum).

Inferenza

Il momento in cui un modello di intelligenza artificiale già addestrato viene effettivamente usato per produrre una risposta o una previsione su un nuovo dato in ingresso, distinto dalla fase di Training in cui il modello impara. → approfondito nella sezione 15 (Intelligenza artificiale). Si collega a: **Training (addestramento)** (la fase precedente, in cui il modello apprende) e **Modello (AI)** (l'oggetto che, una volta addestrato, viene interrogato in questa fase).

Issue

Una "segnalazione" aperta in un repository (su GitHub Issues) per tracciare un bug, una richiesta di funzionalità o qualsiasi attività da discutere e risolvere. → approfondito nella sezione 4 (Git e GitHub).

ITIL

Il framework di riferimento più diffuso per l'ITSM (attualmente alla versione ITIL 4): una raccolta di buone pratiche, non uno standard obbligatorio, usata da moltissime organizzazioni enterprise come vocabolario comune per organizzare i processi di gestione dei servizi IT. → approfondito nella sezione 9 (DevOps). Si collega a: **ITSM** (la disciplina che descrive).

ITSM (IT Service Management)

L'insieme di processi con cui un'organizzazione IT gestisce i servizi che fornisce, perché funzionino in modo prevedibile tutti i giorni, non solo il giorno del rilascio: incident management (ripristinare il servizio), problem management (trovare la causa radice), change management (autorizzare le modifiche) e service request (richieste che non sono guasti). → approfondito nella sezione 9 (DevOps). Si collega a: **ITIL** (il framework di riferimento) e **Change Request (richiesta di modifica)** (l'equivalente PMP del concetto di change management).

JOIN

L'istruzione SQL che unisce temporaneamente le righe di due tabelle collegate tra loro tramite una chiave esterna, per recuperare insieme informazioni che vivono in tabelle diverse. Una INNER JOIN restituisce solo le righe con corrispondenza in entrambe le tabelle; una LEFT JOIN restituisce anche quelle senza corrispondenza. → approfondito nella sezione 2 (Fondamenti di informatica). Si collega a: **Chiave esterna** (su cui si basa) e **Normalizzazione** (il principio che rende la JOIN necessaria).

JQL (Jira Query Language)

Il linguaggio di interrogazione di Jira che permette di costruire ricerche precise sulle issue (per progetto, assegnatario, stato, data) invece di filtrare a mano una lista lunghissima, ed è alla base della maggior parte dei report e delle dashboard costruite su Jira. → approfondito nella sezione 8 (Project Management).

JSON

Sigla di JavaScript Object Notation: un formato di testo semplice e leggibile usato moltissimo per scambiare dati tra programmi, ad esempio nelle risposte delle API. → approfondito nella sezione 2 (Fondamenti di informatica).

JWT (JSON Web Token)

Un token compatto e firmato, composto da tre parti (header.payload.signature), usato per l'autenticazione stateless: contiene già i dati sull'identità di chi lo possiede, così il server può verificarne l'autenticità senza dover consultare nulla, a differenza di una sessione server-side. Va usato con una scadenza breve, perché non può essere revocato a metà strada come una sessione. → approfondito nella sezione 20 (Backend Engineering Avanzato). Si collega a: **Token** (di cui è un'implementazione concreta molto diffusa) e **OAuth** (che spesso lo usa come formato dell'access token).

Knowledge Area (area di conoscenza)

Nell'impostazione del PMBOK Guide 6ª edizione, una delle 10 categorie in cui viene organizzata la conoscenza necessaria a gestire un progetto (ad esempio i costi, i tempi, la qualità, le comunicazioni). Ogni area raggruppa i processi legati a quel tema specifico, indipendentemente dalla fase del progetto in cui si applicano. → approfondito nella sezione 8 (Project Management). Si collega a: **Process Group (gruppo di processi)** (l'altra dimensione, complementare, con cui il PMBOK 6 organizza gli stessi processi).

KPI

Sigla di Key Performance Indicator: un indicatore numerico che misura quanto bene un progetto o un'attività sta raggiungendo i propri obiettivi (ad esempio il tempo medio di risposta a un cliente). → approfondito nella sezione 8 (Project Management).

Kubernetes

Uno strumento che gestisce automaticamente grandi quantità di Container in produzione: li avvia, li riavvia se si bloccano, li distribuisce su più macchine e ne regola il numero in base al carico. → approfondito nella sezione 2 (Fondamenti di informatica).

LDAP (Lightweight Directory Access Protocol)

Il protocollo con cui le applicazioni interrogano una directory di identità (tipicamente Active Directory) per verificare utenti, gruppi e permessi, senza che ogni applicazione debba reinventare la propria gestione degli accessi. → approfondito nella sezione 13 (Sicurezza). Si collega a: **Active Directory (AD)** (l'implementazione più diffusa che lo parla).

Lead Time

Il tempo totale che intercorre dal momento in cui un'attività viene **richiesta** al momento in cui viene **consegnata**, includendo anche l'attesa prima che il lavoro inizi davvero. → approfondito nella sezione 7 (Kanban).

Legge di Conway

L'osservazione secondo cui la struttura di un software tende a rispecchiare la struttura del team che lo costruisce, e viceversa: scomporre un'applicazione in servizi per dominio porta spesso, prima o poi, a riorganizzare anche le persone in squadre allineate a quegli stessi domini. → approfondito nella sezione 11 (Architetture software).

Lessons Learned

La raccolta strutturata, alla chiusura di un progetto (o di una sua fase), di cosa ha funzionato bene e cosa no, così da non ripetere gli stessi errori nei progetti successivi. → approfondito nella sezione 8 (Project Management). Si collega a: **Sprint Retrospective** (l'equivalente Agile, ma ripetuto a ogni Sprint invece che solo a fine progetto).

Livello di isolamento

Una delle configurazioni (Read Uncommitted, Read Committed, Repeatable Read, Serializable) con cui un database stabilisce quanto rigorosamente proteggere una transazione dagli effetti di altre transazioni concorrenti: livelli più rigorosi prevengono più problemi (dirty read, non-repeatable read, phantom read) ma costano in performance. → approfondito nella sezione 20 (Backend Engineering Avanzato). Si collega a: **ACID** (la proprietà Isolation che questi livelli implementano in pratica) e **MVCC** (la tecnica con cui molti database moderni realizzano l'isolamento senza bloccare le letture).

LLM (Large Language Model)

Sigla di Large Language Model, un modello di intelligenza artificiale addestrato su enormi quantità di testo per prevedere e generare linguaggio naturale in modo fluido; è la tecnologia dietro strumenti come

ChatGPT o GitHub Copilot. → approfondito nella sezione 15 (Intelligenza artificiale). Si collega a: **AI generativa** (la categoria di utilizzo più diffusa per questi modelli) e **Token** (l'unità in cui il modello scompone il testo per elaborarlo).

Load Balancer (bilanciatore di carico)

Un componente che riceve tutte le richieste in arrivo verso un sistema e le distribuisce su più server applicativi identici, invece di farle gestire a un unico server: se un server si guasta il traffico continua a fluire sugli altri, e se il traffico cresce basta aggiungere un altro server dietro lo stesso bilanciatore. → approfondito nella sezione 20 (Backend Engineering Avanzato). Si collega a: **Scalabilità verticale e orizzontale** (lo strumento che rende possibile la scalabilità orizzontale) e **Client-Server** (il modello che, per funzionare con più server dietro un load balancer, richiede che le richieste non dipendano da uno stato salvato su un server specifico).

Lock-in

La dipendenza da un provider cloud (o da un fornitore di software) che si crea usando i suoi servizi specifici: più se ne usano, più diventa costoso, in tempo e denaro, cambiare fornitore in futuro. Non è un divieto ad usare quei servizi, ma un costo da valutare consapevolmente. → approfondito nella sezione 12 (Cloud).

Logging

La pratica di registrare in modo continuo gli eventi che accadono in un sistema (errori, richieste, azioni) in file o strumenti dedicati, utile per capire cosa è successo quando qualcosa va storto. → approfondito nella sezione 9 (DevOps).

Lost Update (aggiornamento perso)

Un problema di concorrenza in cui due transazioni leggono lo stesso valore, lo modificano entrambe e l'ultima sovrascrive silenziosamente il lavoro della prima, senza nessun errore visibile: ad esempio due operatori che aggiornano lo stock dello stesso prodotto contemporaneamente. → approfondito nella sezione 20 (Backend Engineering Avanzato). Si collega a: **Livello di isolamento** (il meccanismo che, se scelto correttamente, previene questo problema) e **Race Condition** (di cui è un caso specifico, applicato a una transazione di database).

Machine Learning

Un ramo dell'intelligenza artificiale in cui un sistema impara a svolgere un compito analizzando esempi (dati) invece di seguire regole scritte a mano da un programmatore, migliorando le proprie previsioni con l'esperienza. → approfondito nella sezione 15 (Intelligenza artificiale). Si collega a: **AI (Intelligenza Artificiale)** (il campo più ampio di cui è una branca) e **Deep Learning** (una delle sue tecniche più diffuse oggi).

Manifesto Agile

Il documento pubblicato nel 2001 da un gruppo di sviluppatori che definisce i valori e i principi alla base dell'approccio Agile, come privilegiare le persone e la collaborazione rispetto a processi rigidi e documentazione eccessiva. → approfondito nella sezione 5 (Agile).

Markdown

Un linguaggio di marcatura leggero per scrivere documenti formattati (titoli, elenchi, grassetto, link) usando pochi simboli semplici (`#`, `*`, `[]()`), pensato per essere leggibile anche senza essere convertito, a differenza dell'HTML. Questo stesso corso è scritto in Markdown. → approfondito nella sezione 2 (Fondamenti di informatica). Si collega a: **HTML** (il linguaggio più ricco verso cui viene spesso convertito).

Maven

Uno strumento di build e gestione delle dipendenze per progetti Java: legge un file di configurazione (il POM) che dichiara le librerie esterne necessarie e le scarica automaticamente, insieme alle loro dipendenze transitive, invece di doverle procurare e aggiornare a mano una per una. → approfondito nella sezione 4 (Git e GitHub). Si collega a: **POM** (il file che lo configura).

MCP (Model Context Protocol)

Sigla di Model Context Protocol: uno standard aperto, introdotto da Anthropic e con adozione crescente, che definisce un modo comune con cui un assistente AI si collega a fonti di dati e strumenti esterni, un po' come una presa USB-C che permette di collegare dispositivi diversi con lo stesso connettore. Prima di MCP, ogni assistente andava collegato "a mano" a ogni sistema con cui doveva interagire; MCP evita questa moltiplicazione di integrazioni su misura. → approfondito nella sezione 15 (Intelligenza artificiale). Si collega a: **LLM (Large Language Model)** (l'assistente che, tramite MCP, si collega agli strumenti esterni) e **API** (il modo con cui, dietro le quinte, un MCP server spesso espone le proprie capacità).

MCP server

Il componente che, seguendo lo standard MCP, espone a un assistente AI le capacità di un sistema — ad esempio leggere dei file, interrogare un database o chiamare un'API — così che l'assistente possa scegliere quando e come usarle per rispondere a una richiesta. → approfondito nella sezione 15 (Intelligenza artificiale).

Merge

L'operazione con cui le modifiche fatte su un Branch vengono riunite (unite) al codice principale o a un altro branch. → approfondito nella sezione 3 (Come nasce un software).

Message Queue

Un sistema che permette a diverse parti di un'applicazione di scambiarsi messaggi senza dover comunicare direttamente e nello stesso istante: un componente "deposita" un messaggio in una coda e un altro lo legge quando è pronto. → approfondito nella sezione 11 (Architetture software).

MFA (autenticazione a più fattori)

Un metodo di sicurezza che richiede più di una prova d'identità per accedere a un sistema (ad esempio password più codice ricevuto sul telefono), rendendo molto più difficile un accesso non autorizzato anche se la password viene scoperta. → approfondito nella sezione 13 (Sicurezza).

Microservizi

Un'architettura software in cui un'applicazione è divisa in tanti piccoli servizi indipendenti, ciascuno responsabile di una funzione specifica, che comunicano tra loro (spesso via API), invece di essere un unico grande blocco di codice. → approfondito nella sezione 11 (Architetture software).

Milestone

Un punto di controllo significativo in un progetto, che segna il completamento di una fase importante o il raggiungimento di un obiettivo intermedio. → approfondito nella sezione 8 (Project Management).

Mindset Agile

L'atteggiamento mentale alla base di Agile: adattarsi al cambiamento, imparare dagli errori, collaborare con il cliente e migliorare continuamente, più che l'applicazione meccanica di regole e strumenti. → approfondito nella sezione 5 (Agile).

MLOps

L'insieme di pratiche che estendono i principi del DevOps (automazione, collaborazione, versionamento) al ciclo di vita dei modelli di Machine Learning: dalla preparazione dei dati al Training, dal deploy in produzione al monitoraggio delle prestazioni nel tempo. → approfondito nella sezione 15 (Intelligenza artificiale). Si collega a: **DevOps** (di cui applica i principi al mondo dei modelli AI) e **Data drift** (uno dei problemi che il monitoraggio MLOps serve a individuare).

Modello (AI)

Il risultato concreto del Training: una sorta di "funzione matematica" che ha imparato a riconoscere pattern nei dati e che, una volta pronta, viene usata in fase di Inferenza per produrre risposte o previsioni su nuovi dati. → approfondito nella sezione 15 (Intelligenza artificiale). Si collega a: **Training (addestramento)** (il processo che lo produce) e **Inferenza** (il momento in cui viene effettivamente usato).

Monitoring

L'attività di osservare costantemente lo stato di un sistema (uso della CPU, tempi di risposta, errori) per accorgersi rapidamente se qualcosa non funziona come dovrebbe. → approfondito nella sezione 9 (DevOps).

Monolite

Un'architettura software in cui tutta l'applicazione è costruita come un unico grande blocco di codice, con tutte le funzionalità interdipendenti tra loro, contrapposta ai Microservizi. → approfondito nella sezione 11 (Architetture software).

Multi-stage build

Una tecnica per scrivere un Dockerfile in più fasi (stage): una "builder image" compila l'applicazione con tutti gli strumenti di sviluppo necessari, poi solo il risultato della compilazione viene copiato in un'immagine finale più piccola e senza quegli strumenti, che è quella che finisce davvero in produzione. → approfondito nella sezione 10 (CI/CD). Si collega a: **Builder image** (la fase intermedia che rende possibile) e **Artifact (build)** (il prodotto finale, più leggero, che ne risulta).

MVCC (Multi-Version Concurrency Control)

Una tecnica usata dai database moderni (PostgreSQL, MySQL InnoDB) per gestire l'isolamento tra transazioni senza bloccare tutto: quando una riga viene modificata, il database mantiene più versioni della stessa riga contemporaneamente, così chi legge continua a vedere una versione coerente mentre chi scrive lavora su una nuova versione — i lettori non bloccano gli scrittori, e viceversa. → approfondito nella sezione 20 (Backend Engineering Avanzato). Si collega a: **Livello di isolamento** (il comportamento che questa tecnica realizza in pratica) e **ACID** (la proprietà Isolation di cui è un'implementazione efficiente).

Normalizzazione

Il principio secondo cui ogni informazione in un database relazionale viene scritta una volta sola, nel posto giusto, e recuperata altrove tramite relazioni e JOIN quando serve, invece di essere ripetuta in più righe (con il rischio che, se cambia, venga aggiornata in un posto e dimenticata in un altro). → approfondito nella sezione 2 (Fondamenti di informatica). Si collega a: **JOIN** (lo strumento che rende possibile recuperare i dati normalizzati) e **Chiave esterna** (su cui si basano le relazioni normalizzate).

OAuth

Un protocollo che permette di delegare un'autorizzazione limitata e revocabile a un'applicazione terza, senza mai cederle la password (es. "puoi leggere solo i miei documenti, per i prossimi 30 giorni"): è il meccanismo dietro al pulsante "accedi con..." di tanti siti. → approfondito nella sezione 13 (Sicurezza). Si collega a: **OpenID Connect** (che aggiunge l'autenticazione allo stesso meccanismo) e **Token** (lo strumento con cui l'autorizzazione viene concessa in pratica).

Observability

La capacità di capire **cosa sta succedendo dentro** un sistema complesso osservandolo dall'esterno, combinando log, metriche e tracce; va oltre il semplice Monitoring perché aiuta a capire anche il “perché” di un problema, non solo il “cosa”. → approfondito nella sezione 9 (DevOps).

OpenAPI / Swagger

OpenAPI è uno standard per descrivere un'API REST (risorse, verbi, campi di richiesta e risposta) in un formato leggibile sia da una persona che da una macchina, trasformandola da “documento di cui ci si fida” a contratto verificabile tra due team. Swagger è il nome della famiglia di strumenti storicamente associata a OpenAPI, che genera documentazione navigabile e codice di base a partire dalla stessa descrizione. → approfondito nella sezione 2 (Fondamenti di informatica). Si collega a: **REST** (lo stile di API che descrive) e **JSON** (uno dei formati in cui la specifica è tipicamente scritta).

OpenID Connect (OIDC)

Un livello costruito sopra OAuth che aggiunge l'autenticazione vera e propria (“dimmi chi è questa persona”), non solo la delega di un permesso limitato: OAuth risponde a “cosa posso fare per conto tuo”, OIDC risponde anche a “chi sono”. → approfondito nella sezione 13 (Sicurezza). Si collega a: **OAuth** (il protocollo su cui si basa) e **SSO (Single Sign-On)** (uno degli usi pratici più comuni di questa combinazione).

OpenShift (OCP)

Il prodotto di Red Hat che racchiude Kubernetes con registry, autenticazione, rete, pipeline e monitoraggio già integrati, oltre a un supporto commerciale e a vincoli di sicurezza più severi per impostazione predefinita (es. niente container con privilegi di root). OpenShift è Kubernetes, non lo sostituisce. → approfondito nella sezione 2 (Fondamenti di informatica). Si collega a: **Kubernetes** (che racchiude) e **Platform Engineering** (di cui è un mattone possibile, non l'intera piattaforma).

Overfitting

Un problema che si verifica quando un modello di intelligenza artificiale “impara a memoria” i dati di Training invece di generalizzare, risultando molto bravo sugli esempi già visti ma poco affidabile su dati nuovi. → approfondito nella sezione 15 (Intelligenza artificiale). Si collega a: **Training (addestramento)** (la fase in cui si verifica) e **Modello (AI)** (che ne risulta compromesso, con previsioni poco affidabili su casi nuovi).

OWASP

Sigla di Open Web Application Security Project: un'organizzazione che pubblica risorse gratuite (come la celebre lista “OWASP Top 10”) sulle vulnerabilità di sicurezza più comuni nelle applicazioni web. → approfondito nella sezione 13 (Sicurezza).

PaaS

Sigla di Platform as a Service: un modello cloud in cui il fornitore gestisce anche il sistema operativo e gli strumenti di base, lasciandoti concentrare solo sullo sviluppo e sulla gestione della tua applicazione. → approfondito nella sezione 12 (Cloud).

Pay-as-you-go

Un modello di costo cloud in cui paghi solo per le risorse effettivamente usate (come una bolletta elettrica), invece di acquistare in anticipo hardware costoso che magari resterà sottoutilizzato. → approfondito nella sezione 12 (Cloud).

Pipeline

Una sequenza automatizzata di passaggi (compilare, testare, rilasciare) che il codice attraversa dal momento in cui viene scritto a quando arriva agli utenti, senza interventi manuali ripetitivi. → approfondito nella sezione 10 (CI/CD). Si collega a: **Trigger** (l'evento che ne fa scattare l'avvio) e **Artifact (build)** (il prodotto finale che genera).

Pipeline as Code

La pratica di definire i passaggi di una pipeline in un file di testo versionato insieme al codice, invece che configurarla a mano tramite un'interfaccia grafica, così da poterla tracciare e riutilizzare facilmente. → approfondito nella sezione 10 (CI/CD).

Platform Engineering

La disciplina di costruire e mantenere una piattaforma interna per gli sviluppatori, trattata come un prodotto con dei clienti (i team di sviluppo) che possono, in linea di principio, scegliere di non usarla — a differenza di un ufficio che impone strumenti per decreto. Nasce come risposta al debito organizzativo lasciato scoperto dal DevOps preso alla lettera. → approfondito nella sezione 12 (Cloud). Si collega a: **IDP (Internal Developer Platform)** (ciò che concretamente costruisce) e **DevOps** (la cultura di cui completa la promessa).

PMBOK Guide

Sigla di Project Management Body of Knowledge: la guida pubblicata dal PMI che raccoglie le conoscenze, i processi e le pratiche di riferimento del Project Management. Attenzione: le edizioni non sono tutte uguali — la 6ª edizione è basata su 5 gruppi di processi e 10 aree di conoscenza, mentre la 7ª edizione (2021) ha cambiato impostazione, basandosi su 12 principi e 8 performance domain. Quando qualcuno cita “il PMBOK” senza specificare l'edizione, vale la pena chiedere a quale delle due si riferisce. → approfondito nella sezione 8 (Project Management). Si collega a: **PMI (Project Management Institute)** (l'organizzazione che la pubblica) e **PMP (Project Management Professional)** (la certificazione che su questa guida si basa).

PMI (Project Management Institute)

L'organizzazione internazionale di riferimento per il Project Management: pubblica il PMBOK Guide, rilascia certificazioni professionali (come il PMP) e promuove standard e buone pratiche condivise nel settore. → approfondito nella sezione 8 (Project Management).

PMP (Project Management Professional)

Una certificazione professionale rilasciata dal PMI, tra le più riconosciute al mondo per chi gestisce progetti: attesta la conoscenza dei processi e delle pratiche descritte nel PMBOK Guide e un certo numero di anni di esperienza sul campo. Per i requisiti aggiornati (esperienza richiesta, costi, modalità d'esame) fai sempre riferimento al sito ufficiale del PMI, perché cambiano nel tempo. → approfondito nella sezione 8 (Project Management).

POM (Project Object Model)

Il file (tipicamente `pom.xml`) con cui Maven configura la build di un progetto Java: dichiara nome e versione del progetto, le dipendenze esterne necessarie con la loro versione precisa, e le fasi della build (compile, test, package). → approfondito nella sezione 4 (Git e GitHub). Si collega a: **Maven** (lo strumento che lo legge ed esegue).

Process Group (gruppo di processi)

Nell'impostazione del PMBOK Guide 6ª edizione, una delle 5 macro-fasi in cui si organizzano i processi di un progetto: Avvio, Pianificazione, Esecuzione, Monitoraggio e Controllo, Chiusura. Non sono fasi rigidamente sequenziali quanto categorie di attività che possono anche sovrapporsi nel tempo. → approfondito nella sezione 8 (Project Management). Si collega a: **Knowledge Area (area di conoscenza)** (l'altra dimensione con cui il PMBOK 6 organizza gli stessi processi).

Processo

Un programma in esecuzione sul computer, con la propria area di memoria dedicata: quando apri un'applicazione, il sistema operativo crea un processo per farla funzionare. → approfondito nella sezione 2 (Fondamenti di informatica).

Procurement (approvvigionamenti)

L'attività di project management che riguarda l'acquisto di beni o servizi esterni necessari al progetto (ad esempio un fornitore, una licenza software, una consulenza specialistica), inclusa la scelta del fornitore e la gestione del contratto. → approfondito nella sezione 8 (Project Management).

Product Backlog

L'elenco ordinato di tutto ciò che potrebbe essere fatto su un prodotto: funzionalità, correzioni, miglioramenti. È gestito dal Product Owner e cambia continuamente in base alle priorità. → approfondito nella sezione 6 (Scrum). Si collega a: **Product Owner** (che lo gestisce) e **Sprint Backlog** (il sottoinsieme selezionato per lo sprint corrente).

Product Owner

La persona responsabile di definire cosa deve essere costruito e in quale ordine di priorità, rappresentando la voce del cliente e degli utenti all'interno del team Scrum. → approfondito nella sezione 6 (Scrum).

Produzione

L'ambiente "vero", quello effettivamente usato dagli utenti finali del software; è l'ultimo gradino dopo Sviluppo, Test/QA e Staging, e qui gli errori hanno un impatto reale. → approfondito nella sezione 14 (Ambienti di sviluppo).

Project Charter

Il documento che autorizza formalmente l'esistenza di un progetto: definisce obiettivi, sponsor, vincoli principali e dà al project manager l'autorità per usare le risorse necessarie. È tipicamente uno dei primi documenti prodotti, nel gruppo di processi di Avvio. → approfondito nella sezione 8 (Project Management). Si collega a: **Sponsor** (chi lo firma e autorizza) e **Product Backlog** (l'equivalente Agile più operativo, che nasce dopo, una volta che il progetto è già autorizzato).

Prompt

Il testo (un'istruzione, una domanda, una richiesta) che un utente fornisce a un modello di AI generativa per ottenere una risposta: la qualità del prompt influenza molto la qualità della risposta ottenuta. → approfondito nella sezione 15 (Intelligenza artificiale). Si collega a: **AI generativa** (il tipo di strumento con cui si usa) e **LLM (Large Language Model)** (il modello che lo interpreta per generare una risposta).

Pub/Sub (publish/subscribe)

Il meccanismo con cui producer e consumer si incontrano in un'architettura event-driven: il producer pubblica un evento su un canale dedicato (topic), e ogni consumer interessato si sottoscrive a quel topic per riceverne una copia, tramite un broker che fa da intermediario. → approfondito nella sezione 11 (Architetture software). Si collega a: **Event-Driven Architecture** (il pattern che lo usa) e **Message Queue** (l'infrastruttura sottostante, usata qui in modo diverso: non un messaggio per un destinatario noto, ma un evento per un numero qualsiasi di sottoscrittori).

Pull Request

Una richiesta formale di unire le proprie modifiche di codice (spesso su un Branch) al codice principale, che di solito viene prima discussa e revisionata dal team (Code Review) prima di essere accettata: è il termine usato da GitHub per questo meccanismo (spesso abbreviata PR). → approfondito nella sezione 3 (Come nasce un software). Si collega a: **Branch** (che la genera) e **Code Review** (che ne consegue, prima dell'approvazione).

Quality Gate

Un controllo automatico inserito in una pipeline che blocca l'avanzamento del rilascio se certi criteri di qualità non sono soddisfatti (ad esempio troppi errori nei test o problemi di sicurezza rilevati). → approfondito nella sezione 10 (CI/CD). Si collega a: **CI (Continuous Integration)** (di cui è parte integrante) e **Artifact (build)** (che viene generato solo se il gate viene superato).

RACI

Una matrice usata in Project Management per chiarire i ruoli in un'attività: chi è Responsible (la esegue), chi è Accountable (ne risponde), chi va Consultato e chi va solo Informato. → approfondito nella sezione 8 (Project Management).

RAG (Retrieval-Augmented Generation)

Sigla di Retrieval-Augmented Generation, cioè “generazione aumentata dal recupero”: una tecnica che, invece di riaddestrare un modello di intelligenza artificiale, gli fornisce al momento della domanda i documenti più rilevanti recuperati da una base documentale aziendale, così che possa rispondere basandosi su fonti verificabili invece che solo su ciò che ha imparato durante il Training. Ha però dei limiti: se recupera il documento sbagliato la risposta sarà sbagliata, e la qualità dipende sempre da quella della documentazione di partenza. → approfondito nella sezione 15 (Intelligenza artificiale). Si collega a: **Allucinazione (AI)** (il rischio che questa tecnica riduce, senza eliminarlo del tutto) e **Fine-tuning** (l'alternativa, più costosa da aggiornare, di riaddestrare il modello invece di fornirgli i documenti al momento della domanda).

RAID Log

Un registro che il project manager tiene per tracciare Risks (rischi), Assumptions (assunzioni), Issues (problemi) e Dependencies (dipendenze) di un progetto, per avere sempre sotto controllo i punti critici. → approfondito nella sezione 8 (Project Management).

Race Condition

Un problema che si verifica quando più thread (o processi) accedono e modificano lo stesso dato condiviso contemporaneamente senza coordinamento, producendo un risultato finale inconsistente e dipendente dall'ordine casuale con cui le operazioni si sono sovrapposte. Si previene con lock (mutex) o strutture dati pensate apposta per la concorrenza. → approfondito nella sezione 20 (Backend Engineering Avanzato). Si collega a: **Thread** (l'unità di esecuzione tra cui il problema si manifesta) e **Lost Update** (un caso specifico di race condition applicato a una transazione di database).

RAM

Sigla di Random Access Memory: la memoria “di lavoro” del computer, veloce ma temporanea, che perde i dati quando il computer viene spento — a differenza del Disco. → approfondito nella sezione 2 (Fondamenti di informatica).

Release

Una versione del software resa disponibile agli utenti, spesso accompagnata da un elenco delle novità e delle correzioni incluse rispetto alla versione precedente. → approfondito nella sezione 4 (Git e GitHub).

Replication

La strategia di scaling che mantiene più copie identiche dello stesso database: un nodo primario (master) gestisce tutte le scritture e le propaga a uno o più nodi replica (slave) dedicati alle letture, distribuendo così il carico di lettura e fornendo alta disponibilità se il primario si guasta. → approfondito nella sezione 20 (Backend Engineering Avanzato). Si collega a: **Sharding** (la strategia complementare, che scala i dati invece delle sole letture) e **Scalabilità verticale e orizzontale** (di cui è una forma pratica di scalabilità orizzontale).

Repository

Lo “spazio” (cartella speciale) dove Git conserva tutto il codice di un progetto insieme alla sua intera storia di modifiche (Commit). → approfondito nella sezione 4 (Git e GitHub). Si collega a: **Commit** (che ne costituisce la storia) e **Branch** (che ne organizza lo sviluppo in parallelo).

Requisiti (funzionali e non funzionali)

Le caratteristiche che un software deve avere: i requisiti funzionali descrivono **cosa** il sistema deve fare (es. “l'utente può resettare la password”), quelli non funzionali descrivono **come** deve farlo (es. velocità, sicurezza, disponibilità). → approfondito nella sezione 3 (Come nasce un software).

REST

Uno stile molto diffuso per progettare API web, basato su regole semplici e standard (come l'uso di HTTP) che rendono i servizi facili da capire, usare e collegare tra loro. → approfondito nella sezione 2 (Fondamenti di informatica).

Rete

Un insieme di computer e dispositivi collegati tra loro per scambiarsi dati e comunicare, dalla piccola rete di un ufficio fino a internet, la rete di reti più grande al mondo. → approfondito nella sezione 2 (Fondamenti di informatica).

Ricerca semantica

Una ricerca che trova i contenuti più simili nel significato a una domanda, confrontando i loro Embedding, invece di cercare le stesse identiche parole scritte: permette ad esempio di trovare un documento utile anche se non contiene nessuna delle parole usate nella domanda. → approfondito nella sezione 15 (Intelligenza artificiale).

Rischio

Un evento incerto che, se si verifica, può avere un impatto positivo o (più spesso) negativo su un progetto; identificarlo e pianificarne la gestione è uno dei compiti principali del project manager. → approfondito nella sezione 8 (Project Management).

Rollback

L'operazione di tornare indietro a una versione precedente e funzionante del software dopo che un rilascio ha causato problemi in produzione; il rollback del codice è di norma semplice, quello dei dati può non esserlo, se una modifica alla struttura del database è irreversibile. → approfondito nella sezione 10 (CI/CD) e nella sezione 14 (Ambienti di sviluppo). Si collega a: **Deploy/Rilascio** (l'operazione precedente, che a volte va corretta) e **Monitoring** (che ne segnala spesso la necessità).

SaaS

Sigla di Software as a Service: un modello cloud in cui usi direttamente un'applicazione già pronta tramite browser (come una webmail), senza doverti preoccupare di server, installazioni o aggiornamenti. → approfondito nella sezione 12 (Cloud).

Scalabilità verticale e orizzontale

La capacità di un sistema di gestire più carico di lavoro: la scalabilità verticale significa potenziare la singola macchina (più CPU, più RAM), quella orizzontale significa aggiungere più macchine che si dividono il lavoro. → approfondito nella sezione 12 (Cloud).

Scope (ambito)

L'insieme di tutto ciò che un progetto deve produrre (e, implicitamente, tutto ciò che invece resta fuori). Definire bene lo scope all'inizio, e proteggerlo con le Change Request, è uno dei compiti più importanti del project management tradizionale. → approfondito nella sezione 8 (Project Management). Si collega a: **WBS (Work Breakdown Structure)** (lo strumento con cui lo scope viene scomposto in attività) e **Scope Creep** (il rischio che corre se non viene controllato).

Scope Creep

L'espansione incontrollata dell'ambito (scope) di un progetto, quando si aggiungono via via piccole richieste "non concordate ufficialmente" senza passare da una Change Request, finché il progetto finisce per fare molto più di quanto pianificato, con tempi e costi che ne risentono. → approfondito nella sezione 8 (Project Management). Si collega a: **Change Request (richiesta di modifica)** (il meccanismo che serve a prevenirlo) e **Scope (ambito)** (ciò che si espande).

Scrum Master

La persona che facilita l'applicazione di Scrum nel team, rimuove gli ostacoli che intralciano il lavoro e protegge il team da interferenze esterne, senza dare ordini diretti su cosa fare. → approfondito nella sezione 6 (Scrum).

Scrumban

Un approccio ibrido che combina elementi di Scrum (come i ruoli e le cerimonie) con elementi di Kanban (come il flusso continuo e i limiti di WIP), utile per team che vogliono più struttura di Kanban ma più flessibilità di Scrum. → approfondito nella sezione 7 (Kanban).

Self-service

La possibilità di ottenere un ambiente, un database o una pipeline senza aprire un ticket e senza aspettare una persona: un modulo, un comando, un catalogo di opzioni pre-approvate, non un'email a chi "se ne occupa". → approfondito nella sezione 12 (Cloud). Si collega a: **Golden path** (il percorso che il self-service rende disponibile senza attriti) e **IDP (Internal Developer Platform)** (lo strumento che lo implementa).

Semantic Versioning

Una convenzione per numerare le versioni del software nel formato MAJOR.MINOR.PATCH (es. 2.4.1), dove ogni numero comunica il tipo di cambiamento avvenuto rispetto alla versione precedente. → approfondito nella sezione 3 (Come nasce un software).

Serverless

Un modello cloud in cui scrivi solo il codice della funzione che ti serve e il fornitore si occupa di tutto il resto (server, scalabilità, manutenzione), facendoti pagare solo per l'effettivo utilizzo. → approfondito nella sezione 11 (Architetture software).

Sharding

La strategia di scaling che divide i dati in porzioni indipendenti (shard) distribuite su macchine diverse, in base a una chiave di sharding (es. user_id o region): a differenza della Replication, ogni shard contiene solo una parte dei dati, non l'intero database, ed è la tecnica usata quando i dati o le scritture sono troppo grandi per una singola macchina. → approfondito nella sezione 20 (Backend Engineering Avanzato). Si collega a: **Replication** (la strategia complementare, per scalare le sole letture) e **CAP Theorem** (il compromesso teorico che governa questi sistemi distribuiti).

Single Sign-On (SSO)

Un'autenticazione unica valida per molte applicazioni: chi si autentica lo fa una sola volta e da lì accede a email, repository, gestionale e strumenti cloud senza rifare il login su ciascuno. Concentra però anche il rischio: se il sistema di SSO cade, cade l'accesso a tutto contemporaneamente. → approfondito nella sezione 13 (Sicurezza). Si collega a: **Active Directory (AD)** (la directory su cui tipicamente si costruisce) e **Token** (lo strumento con cui l'identità viene poi dimostrata a ogni applicazione).

Sistema Operativo

Il software di base che gestisce le risorse del computer (CPU, RAM, Disco) e permette a te e alle altre applicazioni di usarle, facendo da intermediario tra hardware e programmi. Esempi: Windows, macOS, Linux. → approfondito nella sezione 2 (Fondamenti di informatica).

SLA

Sigla di Service Level Agreement: un accordo, spesso contrattuale, che definisce il livello di servizio garantito da un fornitore (ad esempio il tempo massimo di inattività accettabile o il tempo massimo di risposta a una segnalazione), con conseguenze concrete (spesso economiche) se non viene rispettato. → approfondito nella sezione 13 (Sicurezza).

Solution Architect

Il ruolo che tiene insieme una soluzione tra più sistemi e più team, traducendo requisiti di business in scelte tecniche e viceversa, e presidiando i requisiti non funzionali del sistema nel suo complesso — a differenza del Tech Lead, che si occupa di un singolo team, o dell'Enterprise Architect, che si occupa di tutta l'organizzazione. → approfondito nella sezione 8 (Project Management). Si collega a: **ADR (Architecture Decision Record)** (lo strumento con cui documenta le decisioni tra sistemi) e **Requisiti (funzionali e non funzionali)** (quelli non funzionali, che più spesso presidia).

SPI (Schedule Performance Index)

Un indice dell'Earned Value Management che misura quanto il progetto è in linea con i tempi pianificati: si calcola dividendo il valore del lavoro effettivamente completato (EV) per il valore del lavoro che era pianificato a questo punto (PV). Un SPI sopra 1 significa che si è avanti rispetto al piano, sotto 1 significa che si è in ritardo. → approfondito nella sezione 8 (Project Management). Si collega a: **EVM (Earned Value Management)** (il sistema di cui fa parte) e **CPI (Cost Performance Index)** (l'indice analogo sui costi).

Sponsor

La persona (o il ruolo) che finanzia il progetto e ne autorizza formalmente l'avvio, tipicamente firmando il Project Charter; è il principale punto di riferimento a cui il project manager rende conto sull'andamento generale. → approfondito nella sezione 8 (Project Management). Si collega a: **Project Charter** (il documento che firma) e **Stakeholder** (di cui è una figura specifica, con potere e interesse tipicamente molto alti).

Sprint

Un periodo di tempo fisso e breve (di solito 1-4 settimane) durante il quale il team Scrum lavora per completare un insieme di attività scelte dal Product Backlog, prodotto poi come Increment. → approfondito nella sezione 6 (Scrum). Si collega a: **Product Backlog** (da cui trae le attività) e **Increment** (il risultato concreto che produce).

Sprint Backlog

L'elenco delle attività selezionate dal Product Backlog che il team si impegna a completare durante lo Sprint corrente. → approfondito nella sezione 6 (Scrum).

Sprint Planning

L'incontro all'inizio di ogni Sprint in cui il team decide cosa realizzare, selezionando le voci dal Product Backlog e definendo come portarle a termine. → approfondito nella sezione 6 (Scrum).

Sprint Retrospective

L'incontro alla fine di ogni Sprint in cui il team riflette su come ha lavorato (non su cosa ha costruito) per individuare cosa migliorare nel prossimo Sprint. → approfondito nella sezione 6 (Scrum).

Sprint Review

L'incontro alla fine di ogni Sprint in cui il team mostra agli stakeholder ciò che ha completato, raccogliendo feedback utile per i prossimi passi. → approfondito nella sezione 6 (Scrum).

SQL

Sigla di Structured Query Language: il linguaggio usato per "interrogare" un Database relazionale, ad esempio per chiedere, aggiungere o modificare dati nelle tabelle. → approfondito nella sezione 2 (Fondamenti di informatica).

SRE (Site Reliability Engineering)

Un approccio, nato in Google, che applica pratiche e mentalità da ingegneria del software alla gestione dei sistemi in produzione, con l'obiettivo di renderli affidabili in modo misurabile (ad esempio tramite obiettivi di affidabilità concordati) invece che "a sensazione". → approfondito nella sezione 9 (DevOps).

Staging

Un ambiente che replica il più possibile le condizioni reali della Produzione, usato come ultima verifica prima del rilascio definitivo agli utenti. → approfondito nella sezione 14 (Ambienti di sviluppo).

Stakeholder

Qualsiasi persona o gruppo che ha un interesse nel progetto o ne è influenzato: può essere un cliente, un utente finale, un manager o un membro del team. → approfondito nella sezione 8 (Project Management).

Story Point

Un'unità di misura relativa (non temporale) usata per stimare quanto sia complessa una User Story rispetto alle altre, tenendo conto di difficoltà, incertezza e quantità di lavoro. → approfondito nella sezione 6 (Scrum).

Sub-task

In Jira, un pezzo di lavoro dentro una Story o un Task più grande, non stimabile da solo come issue a sé stante: il livello più fine della gerarchia Epic → Story/Task → Sub-task. → approfondito nella sezione 8 (Project Management). Si collega a: **User Story** (il livello sopra, di cui il sub-task è parte) e **WBS (Work Breakdown Structure)** (l'equivalente concettuale nel vocabolario PMP).

Tag

Un'etichetta che Git può assegnare a un Commit specifico, tipicamente per marcare un punto importante come una Release (es. "v2.4.1"). → approfondito nella sezione 4 (Git e GitHub).

TCP/IP

La famiglia di regole (protocolli) che permette ai computer di tutto il mondo di scambiarsi dati su internet in modo ordinato e affidabile, anche passando per reti diverse. → approfondito nella sezione 2 (Fondamenti di informatica).

Thread

Una "sotto-unità" di un Processo che può eseguire istruzioni in modo indipendente; più thread nello stesso processo permettono a un programma di fare più cose contemporaneamente in modo più efficiente. → approfondito nella sezione 2 (Fondamenti di informatica).

Throughput

La quantità di lavoro (attività completate, richieste gestite) che un team o un sistema riesce a portare a termine in un'unità di tempo: per un team è quante attività completa a settimana, per un sistema informatico è quante richieste gestisce al secondo. → approfondito nella sezione 7 (Kanban).

TLS/SSL (handshake)

Il protocollo che rende sicura una connessione HTTPS: prima di scambiare dati, client e server negoziano una chiave di cifratura condivisa (usando crittografia asimmetrica, più lenta ma solo per questa fase iniziale), verificano l'identità del server tramite un certificato firmato da una Autorità di Certificazione, e poi passano a cifrare tutta la comunicazione con crittografia simmetrica, molto più rapida. → approfondito nella sezione 20 (Backend Engineering Avanzato). Si collega a: **HTTPS** (il protocollo che questo handshake rende possibile) e **Token** (che viaggia in sicurezza proprio grazie a questa cifratura).

Token

Il termine ha due significati distinti nel corso. In ambito AI: la più piccola unità di testo (una parola, parte di una parola o un simbolo di punteggiatura) in cui un LLM scompone il testo per elaborarlo; i servizi di AI generativa spesso fatturano l'utilizzo proprio in base al numero di token letti e generati (→ sezione 15, Intelligenza artificiale). In ambito sicurezza: un "biglietto" temporaneo con scadenza breve che un sistema rilascia dopo l'autenticazione (tipicamente via SSO), da presentare a ogni richiesta successiva invece della password; va trattato come una password, e se finisce per errore in un repository va

considerato compromesso e sostituito (→ sezione 13, Sicurezza). Si collega a: **LLM (Large Language Model)** (che elabora il testo nel primo senso) e **SSO (Single Sign-On) / OAuth** (con cui si rilascia e usa nel secondo senso).

Training (addestramento)

Il processo con cui un modello di intelligenza artificiale “impara” analizzando grandi quantità di dati di esempio, aggiustando progressivamente i propri parametri interni finché le sue previsioni non diventano sufficientemente accurate. → approfondito nella sezione 15 (Intelligenza artificiale). Si collega a: **Modello (AI)** (il risultato di questo processo) e **Overfitting** (un rischio tipico se il training non è condotto con attenzione).

Trigger

L'evento che fa scattare automaticamente l'avvio di una Pipeline, ad esempio un nuovo Commit su un branch o l'apertura di una Pull Request. → approfondito nella sezione 10 (CI/CD).

Triplo vincolo (triple constraint)

Il principio secondo cui un progetto è sempre limitato da tre fattori collegati tra loro — tempo, costo e ambito (scope), talvolta con la qualità come quarto vincolo aggiuntivo — e non si può cambiarne uno senza avere un effetto sugli altri: fare di più (scope) senza più tempo o più budget non è quasi mai possibile. → approfondito nella sezione 8 (Project Management). Si collega a: **Scope (ambito)** (uno dei tre vertici del triangolo) e **Change Request (richiesta di modifica)** (lo strumento con cui si negozia consapevolmente uno spostamento tra i tre vincoli).

Trunk Based Development

Una pratica in cui tutti gli sviluppatori integrano il proprio codice molto frequentemente su un unico ramo principale (il “trunk”), evitando Branch di lunga durata e favorendo una forte automazione dei test. → approfondito nella sezione 4 (Git e GitHub).

TTL (Time To Live)

Il tempo dopo il quale un dato salvato in cache scade automaticamente ed è considerato non più valido: semplice da implementare, ma per tutta quella finestra di tempo il dato in cache può non essere più aggiornato rispetto al database. → approfondito nella sezione 20 (Backend Engineering Avanzato). Si collega a: **Cache-Aside (lazy loading)** (la strategia di caching con cui questo meccanismo di scadenza viene usato più spesso).

User Story

Una breve descrizione di una funzionalità scritta dal punto di vista dell'utente, spesso nel formato “Come [ruolo], voglio [obiettivo], per [beneficio]”, usata per catturare i requisiti in modo semplice e centrato sulle persone. → approfondito nella sezione 6 (Scrum). Si collega a: **Requisiti (funzionali e non funzionali)** (da cui spesso nasce) e **Product Backlog** (dove viene inserita in attesa di essere pianificata).

Vault (dei segreti)

Uno strumento dedicato a custodire in modo cifrato password, chiavi di accesso e altre credenziali, concedendone l'uso solo a chi (una persona o un sistema, come una pipeline) ne ha davvero bisogno, invece di scriverle nel codice o passarle a mano tra le persone del team. → approfondito nella sezione 14 (Ambienti di sviluppo).

Velocity

La quantità media di Story Point che un team Scrum riesce a completare in uno Sprint, misurata negli Sprint passati e usata per stimare quanto lavoro pianificare in futuro. → approfondito nella sezione 6 (Scrum).

Versioning

La pratica di assegnare un numero o un'etichetta univoca a ogni versione del software, per poter distinguere, confrontare e tornare a versioni precedenti quando necessario. → approfondito nella sezione 3 (Come nasce un software).

Virtual Machine (VM)

Un "computer dentro il computer": un ambiente simulato dal software che si comporta come una macchina indipendente, con proprio sistema operativo, pur condividendo l'hardware fisico con altre VM. → approfondito nella sezione 2 (Fondamenti di informatica).

Vista (VIEW)

Una query SQL salvata con un nome, che si comporta come se fosse essa stessa una tabella: permette a chi la usa di non dover riscrivere (né conoscere) la JOIN che c'è dietro. Non elimina il costo della query sottostante, che viene comunque eseguita a ogni chiamata. → approfondito nella sezione 2 (Fondamenti di informatica). Si collega a: **JOIN** (l'operazione che tipicamente racchiude) e **Database relazionale** (il contesto in cui vive).

Waterfall

Un approccio tradizionale alla gestione dei progetti in cui le fasi (analisi, progettazione, sviluppo, test, rilascio) si susseguono in modo lineare e sequenziale, una dopo l'altra, a differenza dell'iteratività di Agile. → approfondito nella sezione 5 (Agile).

WBS (Work Breakdown Structure)

La scomposizione gerarchica di tutto il lavoro necessario a completare lo scope di un progetto, dai macro-obiettivi fino a singole attività gestibili e stimabili, spesso rappresentata con codici gerarchici (1, 1.1, 1.1.1...) anche in un semplice foglio Excel. Il livello più basso della scomposizione si chiama **work package**: abbastanza piccolo da poter essere stimato con sicurezza e assegnato a una singola persona. È uno degli strumenti cardine della pianificazione tradizionale, e serve da base per stimare

tempi e costi. → approfondito nella sezione 8 (Project Management). Si collega a: **Scope (ambito)** (ciò che la WBS scompone) e **Product Backlog** (l'equivalente Agile, più flessibile e continuamente ri-prioritizzato invece che fissato all'inizio).

WIP (Work In Progress)

Il numero di attività che sono attualmente “in corso” in una Board Kanban; limitare il WIP (fissando un tetto massimo per colonna) aiuta il team a concentrarsi e a completare il lavoro più rapidamente invece di iniziarne troppo in parallelo. → approfondito nella sezione 7 (Kanban).

XML

Sigla di eXtensible Markup Language: un formato di testo, più “verboso” di JSON, usato per organizzare e scambiare dati tramite tag simili a quelli dell'HTML. → approfondito nella sezione 2 (Fondamenti di informatica).



Collegamenti

- [17. Piano di studio](#) — un percorso suggerito per ripassare tutti questi concetti in modo organico



Risorse

- [Glossario Agile Alliance](#) — glossario dei termini Agile e Scrum (in inglese)
- [GitHub Docs](#) — documentazione ufficiale, utile per verificare la terminologia esatta usata dalla piattaforma (Pull Request, Issue, Actions...)
- [Wikipedia — Glossario di informatica](#) — per approfondire i termini di base

17. Piano di studio

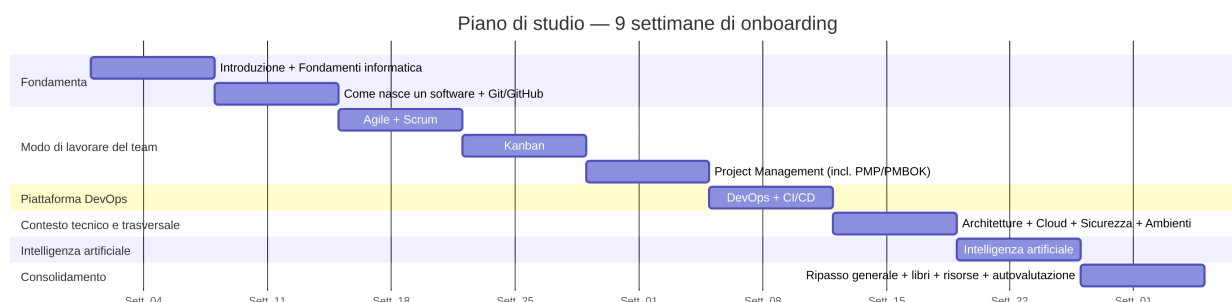
Le sezioni da 1 a 15 di questo corso contengono tutto il materiale teorico e pratico che ti serve. Questa sezione risponde a una domanda diversa e molto concreta: **in che ordine e con quale ritmo studiarlo, in 2-3 mesi, mentre affianchi la tua collega Scrum Master/PM sul progetto?**

Il piano che segue copre **9 settimane**. Non è un vincolo rigido: è una traccia pensata per darti un ritmo sostenibile, evitare di sentirti sopraffatto dalla quantità di argomenti nuovi (specialmente nelle prime due settimane, le più dense dal punto di vista tecnico) e assicurarti che, settimana dopo settimana, tu costruisca le competenze nell'ordine in cui ti serviranno davvero sul campo.

Come usare questo piano

- **È un percorso, non una gara.** Le ore indicate per ogni settimana (in media 5-13 ore) sono una stima per un neolaureato senza background informatico che studia part-time, in parallelo all'affiancamento quotidiano sul progetto. Se una settimana ti serve più tempo — è normalissimo, soprattutto per le sezioni 2, 8 e 9, cresciute con il vocabolario PMP/PMBOK, Jira/Excel e ITSM, e per la settimana 7 che ora copre anche Platform Engineering e identità digitale — **rallenta**. È molto meglio arrivare alla settimana 9 con basi solide che aver “letto tutto” senza averlo capito.
- **Le settimane si possono spostare, non solo comprimere.** Se il progetto ti mette in un contesto reale prima del previsto (es. partecipi a un Sprint Planning già alla settimana 2), va benissimo anticipare la lettura della sezione corrispondente: il piano è una guida, il contesto reale è il miglior maestro che hai.
- **Il check-in con la tua collega è la parte più importante del piano**, non un'aggiunta facoltativa. Ogni settimana prevede almeno un momento di confronto (15-30 minuti bastano) in cui le racconti cosa hai capito, le fai le domande che ti sono rimaste in sospeso e — soprattutto — collega la teoria che hai letto a un esempio concreto del progetto (“quello che ho letto sulle user story, corrisponde a come scriviamo le card sul backlog?”). Senza questo passaggio, il rischio è restare con una conoscenza da manuale che non si aggancia mai alla pratica quotidiana.
- **Gli esercizi pratici non sono opzionali.** Leggere una sezione ti dà il vocabolario; l'esercizio è quello che lo trasforma in competenza. Se salti gli esercizi per fare prima, arriverai alla fine del percorso con parole difficili ma poca sicurezza nell'usarle.
- **Il Glossario (sezione 16) è il tuo compagno silenzioso per tutte le 9 settimane.** Ogni volta che incontri un termine che non ricordi — anche uno già visto — torna lì. Non è un segno di scarsa preparazione: è esattamente lo scopo per cui esiste.
- **Non aspettarti di “sapere tutto” alla settimana 9.** L'obiettivo del piano è darti l'autonomia per affiancare e poi sostituire la tua collega con sicurezza sulle basi, non farti diventare un esperto DevOps in due mesi. Il resto lo imparerai sul campo, con la sezione 19 (Risorse online) e la sezione 18 (Libri consigliati) a disposizione per gli approfondimenti che vorrai fare più avanti.

Vista d'insieme del percorso



Il percorso segue una logica a blocchi progressivi: prima il **linguaggio di base** del software (settimane 1-2), poi il **modo in cui il team si organizza** per lavorare (settimane 3-5, con Kanban e Project Management sdoppiati su due settimane separate per dare più respiro al vocabolario PMP/PMBOK), poi **gli strumenti e i processi specifici** della piattaforma DevOps che gestirai (settimana 6, DevOps e CI/CD insieme), poi il **contesto tecnico e trasversale** che ti serve per capire le decisioni del team (settimana 7), poi un approfondimento su **come l'intelligenza artificiale entra nel lavoro quotidiano del team** (settimana 8), e infine un momento di **consolidamento** (settimana 9) prima di sentirti pronto/a a camminare sempre più da solo/a.

Settimana 1 — Le fondamenta: come funziona un computer, come nasce un software

Argomenti	Sezione 1 — Introduzione · Sezione 2 — Fondamenti di informatica
Tempo stimato	9-11 ore (la sezione 2 è la più densa dell'intero corso: CPU, RAM, rete, database — incluse chiavi, JOIN e normalizzazione —, container, Docker, Kubernetes, OpenShift, e infine Markdown/HTML — non sentirti in colpa se ti serve più tempo)

Esercizi pratici

1. Fatti installare (o installa da solo/a, se hai i permessi) Docker Desktop sul tuo computer e prova a lanciare un container di esempio (es. `docker run hello-world`), anche senza capire ogni dettaglio: l'obiettivo è vedere con i tuoi occhi cos'è un container prima di studiarlo in teoria.
2. Chiedi a un collega developer di mostrarti, per 15 minuti, come si collegano tra loro un'applicazione, un database e un server nel progetto (anche solo a parole, con un disegno su una lavagna o su un foglio): non serve capire il codice, serve vedere la "geografia" del sistema.
3. Disegna a mano (anche su carta) uno schizzo con client, server, database e rete, usando le tue parole per etichettare ogni pezzo.

Obiettivi di apprendimento

Al termine della settimana devi saper spiegare a parole tue, senza usare il manuale:

- cosa fanno CPU e RAM e perché sono diverse;
- cos'è un indirizzo IP e a cosa serve una rete;
- cos'è un database e perché serve rispetto a “un file con dentro i dati”;
- cos'è un container e in cosa differisce da una macchina virtuale;
- a cosa serve, in generale, Kubernetes (senza ancora sapere usarlo).

Verifica finale della settimana

Sai spiegare a un amico non tecnico, in meno di due minuti, la differenza tra un container e una macchina virtuale, usando un'analogia (ad esempio quella degli appartamenti in un condominio vista nella sezione 2)?

Settimana 2 — Come nasce un software e il controllo di versione con Git

Argomenti	Sezione 3 — Come nasce un software · Sezione 4 — Git e GitHub
Tempo stimato	8-10 ore (la sezione 4 richiede pratica al computer, non solo lettura, e copre anche due modelli di branching aggiuntivi — cactus/OneFlow — più Maven/POM per chi lavora su progetti Java)

Esercizi pratici

1. Crea un account GitHub (se non lo hai già) e crea un repository di prova personale, anche vuoto.
2. Clona un repository di esempio (puoi usare uno dei tuoi o uno pubblico semplice) sul tuo computer con `git clone`.
3. Crea un nuovo branch, modifica un file (anche solo il README), fai un commit e apri una **pull request fittizia** verso il branch principale del tuo repository di prova.
4. Chiedi a un developer del team di farti vedere, sullo schermo, una vera pull request aperta sul progetto: osserva come è strutturata la descrizione, chi la revisiona, quali commenti riceve.
5. Prova a spiegare a voce, senza guardare gli appunti, cosa succede “dietro le quinte” quando fai un commit.

Obiettivi di apprendimento

Al termine della settimana devi saper:

- descrivere le fasi principali del ciclo di vita di un software (analisi, progettazione, sviluppo, test, rilascio, manutenzione);
- distinguere un bug da una richiesta di nuova funzionalità;
- spiegare perché una decisione tecnica importante va scritta in un ADR, e cosa significa che un ADR viene “superato”;
- spiegare cosa sono repository, commit, branch e pull request;

- distinguere concettualmente Git Flow, il modello cactus/OneFlow e Trunk Based Development (anche solo a grandi linee, senza padroneggiarli);
- riconoscere a cosa serve Maven e il file `pom.xml`, e cosa sono le dipendenze transitive.

Verifica finale della settimana

Sai disegnare su un foglio il percorso di una modifica al codice, dal branch alla pull request fino al merge sul branch principale, spiegando ogni passaggio?

Settimana 3 — Agile e Scrum: il mindset e il framework che il team usa ogni giorno

Argomenti	Sezione 5 — Agile · Sezione 6 — Scrum
Tempo stimato	8-10 ore (la sezione 6 è la più importante del corso per il tuo ruolo: prendila con calma)

Esercizi pratici

1. Partecipa come **osservatore** a uno Sprint Planning e a una Daily Scrum del team, senza intervenire: prendi appunti su chi parla, cosa viene deciso, quanto dura ogni evento rispetto a quanto previsto dalla teoria.
2. Scrivi 3 user story per una feature immaginaria (ad esempio “un’app per prenotare una sala riunioni”), seguendo il formato “Come [utente], voglio [obiettivo], così che [beneficio]”, con relativi criteri di accettazione.
3. Prova a stimare le 3 user story che hai scritto con la scala di Story Point di Fibonacci (1, 2, 3, 5, 8...), motivando a voce alta perché hai scelto quel valore.
4. Confronta con la tua collega la Definition of Ready e la Definition of Done reali del progetto: sono scritte da qualche parte? Coincidono con quello che hai letto in teoria?

Obiettivi di apprendimento

Al termine della settimana devi saper:

- riassumere i 4 valori e i 12 principi del Manifesto Agile con parole tue;
- elencare e descrivere i tre ruoli di Scrum (Product Owner, Scrum Master, Team di sviluppo);
- descrivere i cinque eventi di Scrum (Sprint, Sprint Planning, Daily Scrum, Sprint Review, Sprint Retrospective) con durata e obiettivo di ciascuno;
- distinguere i tre artefatti (Product Backlog, Sprint Backlog, Increment);
- scrivere una user story ben formata e spiegare a cosa servono Story Point e Definition of Ready/Done.

Verifica finale della settimana

Sai spiegare, senza guardare gli appunti, a cosa serve ciascuno dei cinque eventi di Scrum e cosa succederebbe al team se uno di essi venisse eliminato?

Settimana 4 — Kanban: un altro modo di visualizzare il lavoro

Argomenti	Sezione 7 — Kanban
Tempo stimato	5-6 ore (sezione più breve rispetto alle altre di questo blocco: un buon momento per consolidare anche un ripasso rapido di Scrum)

Esercizi pratici

1. Osserva la board del team (Kanban o Scrum board che sia) e identifica le colonne, i limiti di WIP (Work In Progress) se presenti, e chiedi a un collega di spiegarti come si sposta una card da una colonna all'altra.
2. Crea, anche solo su carta o con un tool gratuito online, una board Kanban di prova con 5 card immaginarie e sposta manualmente una card lungo le colonne, calcolando a mano un ipotetico lead time e cycle time.
3. Confronta con la tua collega quando il team di ShopFacile usa un flusso Kanban puro e quando invece segue la cadenza a Sprint di Scrum: cosa cambia in pratica.

Obiettivi di apprendimento

Al termine della settimana devi saper:

- descrivere la struttura di una board Kanban e a cosa serve il limite di WIP;
- distinguere lead time e cycle time;
- spiegare cos'è lo Scrumban e in quali situazioni un team lo preferisce a Scrum o Kanban puri.

Verifica finale della settimana

Sai spiegare a parole tue la differenza tra Sprint (Scrum) e flusso continuo (Kanban)? In quale scenario useresti l'uno o l'altro?

Settimana 5 — Project Management: come si tiene sotto controllo un progetto

Argomenti	Sezione 8 — Project Management
Tempo stimato	11-13 ore (la sezione è la più lunga del corso: oltre a stakeholder, RAID log, RACI e Solution Architect, copre il vocabolario PMP/PMBOK — charter, WBS, tempi e costi — e la gerarchia Jira/Excel per il tracking quotidiano del lavoro, quindi mettici più tempo del previsto, specialmente se l'ambito PMP è del tutto nuovo per te)

Esercizi pratici

1. Costruisci una tabella RACI di esempio (anche minimale, 4-5 righe) per un piccolo progetto immaginario, assegnando ruoli Responsible, Accountable, Consulted, Informed.
2. Chiedi alla tua collega di farti vedere un report o una dashboard reale del progetto (burndown, RAID log, o simili) e provate insieme a leggerla.
3. Scrivi un Project Charter minimale (anche solo mezza pagina) per un piccolo progetto immaginario: obiettivo, sponsor, vincoli principali.
4. Prova a scomporre lo stesso progetto immaginario in una WBS di 2 livelli (macro-attività e sotto-attività), poi individua a occhio quale sequenza di attività ne determina la durata minima (il critical path).
5. Con numeri inventati e semplici (es. PV = 1000, EV = 800, AC = 900), calcola a mano SPI e CPI e interpreta il risultato: il progetto immaginario è in anticipo o in ritardo? Sta spendendo di più o di meno del previsto?
6. Chiedi a un collega di mostrarti la gerarchia Epic/Story/Task/Sub-task su Jira (o lo strumento equivalente del progetto) e osserva la stessa issue vista da due prospettive diverse: la board e un eventuale Gantt/roadmap.
7. Apri un foglio Excel di esempio con almeno due colonne in comune con un secondo elenco, e prova a usare CERCA.VERT (o CERCA.X) per unirle: è lo stesso principio della JOIN vista nella sezione 2, applicato a due fogli invece che a due tabelle SQL.

Obiettivi di apprendimento

Al termine della settimana devi saper:

- spiegare cos'è un RAID log e a cosa serve una matrice RACI;
- descrivere almeno due KPI tipici di un progetto software e leggere un report/dashboard di base;
- spiegare a cosa serve un Project Charter e chi lo autorizza;
- scomporre l'ambito (scope) di un piccolo progetto in una WBS, e riconoscere cos'è il critical path in un Diagramma di Gantt;
- spiegare cos'è l'Earned Value Management e leggere gli indici SPI e CPI;
- spiegare il triplo vincolo (tempo, costo, ambito) e perché una change request serve a evitare lo scope creep;
- confrontare a grandi linee l'approccio PMP/PMBOK con quello Agile, e capire cosa significa "approccio ibrido";
- leggere la gerarchia Epic/Story/Task/Sub-task di Jira e riconoscere quando Excel resta lo strumento giusto (reporting rapido, WBS) e quando serve invece lo strumento condiviso del team;
- distinguere il ruolo del Solution Architect da quello di Tech Lead, Product Owner ed Enterprise Architect.

Verifica finale della settimana

Sai spiegare, con un esempio, cosa significherebbe per il progetto ShopFacile un "approccio ibrido" che combina Project Charter e WBS con Sprint e Product Backlog?

Settimana 6 — DevOps e CI/CD: la cultura e la pipeline che uniscono sviluppo e operations

Argomenti	Sezione 9 — DevOps · Sezione 10 — CI/CD
Tempo stimato	11-13 ore (settimana densa e centrale: cultura, CI/CD concettuale e pratico, Infrastructure as Code, monitoring/observability, e ora anche ITSM/ITIL — incident, problem e change management — e il mondo dei job batch notturni: vale la pena dedicarle tempo extra)

Esercizi pratici

1. Chiedi a un membro del team (developer o operations) di raccontarti, con parole semplici, un episodio concreto in cui l'automazione (una pipeline, un test automatico) ha evitato un problema in produzione: raccogli l'aneddoto, aiuta più di dieci definizioni.
2. Osserva, anche solo guardando lo schermo di un collega, una dashboard di monitoring reale del progetto: prova a distinguere cosa mostra il “monitoring” (è tutto ok?) da cosa servirebbe per fare “observability” (perché non va?).
3. Scrivi in 5 righe, con parole tue, cosa significa “you build it, you run it” e perché cambia il modo in cui un team lavora rispetto al modello tradizionale sviluppo/operations separati.
4. Prova a spiegare il modello CALMS (Culture, Automation, Lean, Measurement, Sharing) usando un esempio del progetto per ciascuna delle 5 lettere.
5. Osserva una pipeline CI/CD reale del progetto in esecuzione (o la sua ultima esecuzione registrata) e **disegna uno schema** di come funziona: trigger, fasi (build, test, deploy), ambienti coinvolti, eventuali quality gate.
6. Trova nel progetto un file di pipeline (ad esempio un workflow GitHub Actions in [.github/workflows/](#)) e prova a identificare, senza capire ogni riga, dove sono definiti trigger, stage e job.
7. Chiedi cosa succede quando una pipeline “si rompe” (fallisce un quality gate, un test): chi viene notificato, cosa succede al rilascio.
8. Chiedi a chi gestisce la produzione di raccontarti l'ultimo incidente reale gestito dal team: prova a distinguere a voce alta cosa è stato incident management (ripristinare) e cosa problem management (capire la causa).
9. Se il progetto ha job schedulati/batch (calcoli notturni, invii automatici), chiedi a un collega quali dipendenze esistono tra un job e il successivo, e cosa succede se il primo slitta.

Obiettivi di apprendimento

Al termine della settimana devi saper:

- spiegare cos'è la cultura DevOps e perché nasce come risposta al conflitto storico tra Dev e Ops;
- descrivere il modello CALMS;
- distinguere concettualmente Continuous Integration, Continuous Delivery e Continuous Deployment;

- spiegare cos'è l'Infrastructure as Code e perché è utile;
- distinguere monitoring, observability e logging;
- descrivere le fasi tipiche di una pipeline CI/CD (checkout, build, test, pubblicazione artifact, deploy);
- spiegare cosa fa scattare (trigger) una pipeline e cos'è un quality gate;
- distinguere incident management, problem management e change management (ITSM/ITIL), e capire come il change management convive con il CI/CD;
- spiegare cos'è l'elaborazione batch e perché la finestra notturna è un vincolo di progetto, non solo un dettaglio tecnico.

Verifica finale della settimana

Sai descrivere le fasi di una pipeline CI/CD del progetto, dal commit del developer fino al rilascio, indicando dove potrebbe fermarsi se qualcosa va storto, e sai spiegare la differenza tra Continuous Delivery e Continuous Deployment?

Settimana 7 — Il contesto tecnico e trasversale: architetture, cloud, sicurezza, ambienti

Argomenti	Sezione 11 — Architetture software · Sezione 12 — Cloud · Sezione 13 — Sicurezza · Sezione 14 — Ambienti di sviluppo
Tempo stimato	10-12 ore (quattro sezioni trasversali, tutte cresciute: Event-Driven Architecture nella 11, Platform Engineering e Integration Platform nella 12, identità/OAuth/SSO e Business Continuity nella 13 — prova a leggerle collegandole sempre al progetto reale)

Esercizi pratici

1. Fatti spiegare da un developer se l'architettura del progetto è più vicina a un monolite o a dei microservizi, e chiedi un esempio concreto di come frontend e backend comunicano.
2. Individua, con l'aiuto di un collega, quale provider cloud usa il progetto (Azure, AWS o altro) e quali servizi principali (IaaS, PaaS, SaaS) sono in uso — anche solo i nomi, senza approfondire ogni dettaglio tecnico.
3. Chiedi come funzionano autenticazione e autorizzazione nell'applicazione del progetto (login, ruoli, permessi) e prova a mappare i concetti visti nella sezione 13 su questo esempio reale.
4. Ricostruisci, con l'aiuto della tua collega, la sequenza reale degli ambienti del progetto (Dev, Test, Staging, Prod o equivalenti) e chi/cosa decide quando il codice passa da uno all'altro.
5. Chiedi a un developer un esempio concreto di comunicazione a eventi (pub/sub) nel progetto, oppure, se non esiste ancora, discutete insieme quale processo trarrebbe beneficio dal passare da una chiamata sincrona a un evento.
6. Chiedi come è organizzata la piattaforma interna del progetto (self-service, pipeline standard, catalogo di ambienti) e se esiste un "golden path" per creare un nuovo servizio.

7. Fatti raccontare la differenza tra il piano di Disaster Recovery e quello di Business Continuity del progetto: cosa succede ai sistemi, e cosa succede alle persone, se una piattaforma resta giù per alcune ore.

Obiettivi di apprendimento

Al termine della settimana devi saper:

- distinguere monolite e microservizi, e frontend e backend;
- distinguere un comando da un evento, e spiegare a grandi linee cos'è un'architettura event-driven (producer, consumer, topic, pub/sub);
- distinguere IaaS, PaaS e SaaS con un esempio ciascuno;
- spiegare cos'è il Platform Engineering e la differenza tra piattaforma DevOps e piattaforma di integrazione;
- spiegare la differenza tra autenticazione e autorizzazione, cos'è il DevSecOps, e a cosa servono Active Directory/SSO/token/OAuth nella gestione delle identità;
- distinguere Disaster Recovery e Business Continuity;
- descrivere a cosa serve ciascun ambiente (Dev, Test, Staging, Prod) nella catena di rilascio del progetto.

Verifica finale della settimana

Sai spiegare, usando il progetto come esempio, perché il codice passa attraverso più ambienti prima di arrivare in produzione, e cosa si rischierebbe saltando uno di questi passaggi?

Settimana 8 — Intelligenza artificiale: come entra nel lavoro quotidiano del team

Argomenti	Sezione 15 — Intelligenza artificiale
Tempo stimato	7-9 ore (sezione di lunghezza media-alta: vocabolario nuovo — Machine Learning, LLM, AI generativa — più esempi pratici legati a strumenti come GitHub Copilot)

Esercizi pratici

1. Chiedi a un developer del team (ad esempio Marco o Ahmed) di mostrarti, per qualche minuto, come usa uno strumento di AI generativa nel lavoro quotidiano (es. un suggerimento di GitHub Copilot durante la scrittura di una funzione) e osserva come lo verifica prima di accettarlo.
2. Scrivi tre esempi di Prompt che potresti usare per chiedere aiuto a un assistente AI su compiti tipici del tuo ruolo (es. riassumere le note di una Daily Scrum, preparare la scaletta di una Sprint Review), e provali se hai accesso a uno strumento del genere.
3. Discuti con la tua collega un caso reale (anche ipotetico) di Allucinazione o di risposta imprecisa di un assistente AI: come se ne accorge il team, e cosa fa per evitare che un errore di questo tipo arrivi in produzione.

4. Prova a spiegare a parole tue, senza guardare gli appunti, la differenza tra Training e Inferenza usando l'esempio di GitHub Copilot: quando “impara” il modello e quando invece lo stai semplicemente “usando”.

Obiettivi di apprendimento

Al termine della settimana devi saper:

- distinguere AI, Machine Learning e Deep Learning, e spiegare cosa sono Training e Inferenza;
- spiegare cos'è un LLM e cosa si intende per AI generativa, con GitHub Copilot come esempio concreto;
- riconoscere i limiti principali di questi strumenti (Allucinazione, Bias, Overfitting) e perché serve sempre una verifica umana;
- spiegare a grandi linee cosa sono MLOps e AIOps e come si collegano ai principi DevOps già visti nella settimana 6.

Verifica finale della settimana

Sai spiegare a un collega non tecnico perché un suggerimento di GitHub Copilot va sempre passato in Code Review come qualsiasi altro contributo, e non accettato “alla cieca” solo perché generato da uno strumento AI?

Settimana 9 — Consolidamento: ripasso generale, autovalutazione e prossimi passi

Argomenti	Ripasso trasversale delle sezioni 1-15, con il supporto di Sezione 16 — Glossario · Sezione 18 — Libri consigliati · Sezione 19 — Risorse online
Tempo stimato	6-8 ore, distribuite tra ripasso e un colloquio conclusivo con la tua collega

Esercizi pratici

1. Scorri il Glossario (sezione 16) da cima a fondo, senza fretta: segna i termini che non ricordi al 100% e torna alla sezione corrispondente per un rapido ripasso mirato.
2. Prepara una “mappa mentale” (anche a mano, su un foglio) che collega i temi principali del corso: Agile → Scrum/Kanban → Project Management → DevOps → CI/CD → Cloud/Sicurezza → Intelligenza artificiale. Non deve essere perfetta, deve aiutarti a vedere le connessioni.
3. Fissa un colloquio conclusivo di 45-60 minuti con la tua collega Scrum Master/PM in cui **tu** guidi la conversazione: racconta con parole tue come funziona il progetto, dal punto di vista di processo (Scrum/Kanban) e di piattaforma (GitHub, pipeline, ambienti, strumenti AI). Fatti correggere dove serve.
4. Scegli, guardando la sezione 18, un libro che approfondirai nei mesi successivi (non serve leggerlo entro questa settimana: è un impegno per dopo).
5. Salva nei preferiti 3-4 risorse online dalla sezione 19 che userai come riferimento rapido nel lavoro quotidiano.

6. Se il vocabolario PMP/PMBOK visto nella sezione 8 ti ha incuriosito, dai una prima occhiata (senza impegno) alle pagine ufficiali delle certificazioni PMI legate al Project Management (PMP, PMI-ACP) e a quella di Scrum.org/Scrum Alliance per il Professional Scrum Master: sono percorsi che potresti valutare più avanti nella carriera, non un requisito di questo corso. Per requisiti di ammissione, costi e formato d'esame aggiornati, fai sempre riferimento ai siti ufficiali (pmi.org, scrum.org) invece che a fonti terze, perché cambiano nel tempo.

Obiettivi di apprendimento

Al termine della settimana devi saper:

- ricostruire una visione d'insieme di tutto il corso, collegando i temi delle diverse sezioni tra loro;
- usare il Glossario come strumento di consultazione rapida senza più bisogno di “studiarlo”;
- muoverti con sicurezza nella conversazione quotidiana del team, sia sul piano di processo che su quello tecnico di base, incluso l'uso responsabile di strumenti AI;
- riconoscere quali aree richiederanno ancora approfondimento nei mesi successivi (e sapere dove trovare le risorse per farlo).

Verifica finale della settimana

Se dovessi spiegare in 5 minuti a un/una nuovo/a collega, arrivato/a oggi, “come funziona questo progetto” — dal processo di lavoro del team alla piattaforma tecnica che lo supporta — ci riusciresti senza guardare gli appunti? Se la risposta è “quasi”, individua i due punti più deboli e torna sulle sezioni corrispondenti.

Tabella riepilogativa delle 9 settimane

Settimana	Argomenti	Tempo stimato	Focus dell'esercizio pratico
1	Introduzione · Fondamenti di informatica	9-11 ore	Docker in locale, schema client/server/DB
2	Come nasce un software · Git e GitHub	8-10 ore	Repository, branch e pull request fittizia su GitHub
3	Agile · Scrum	8-10 ore	Osservazione Sprint Planning/Daily, scrittura di 3 user story
4	Kanban	5-6 ore	Board Kanban di prova, calcolo di lead time e cycle time
5	Project Management	11-13 ore	Tabella RACI, Project Charter e WBS di esempio, calcolo di SPI/ CPI, gerarchia Jira
6	DevOps · CI/CD	11-13 ore	Osservazione dashboard di monitoring e pipeline reale, disegno dello schema, un incidente reale (ITSM)
7	Architetture software · Cloud · Sicurezza · Ambienti di sviluppo	10-12 ore	Mappatura architettura, cloud provider, piattaforma interna e ambienti reali del progetto
8	Intelligenza artificiale	7-9 ore	Osservazione di GitHub Copilot in azione, scrittura di prompt di esempio
9	Ripasso generale · Glossario · Libri · Risorse online	6-8 ore	Mappa mentale finale e colloquio conclusivo con la collega
Totale		75-92 ore	

Collegamenti

- [18. Libri consigliati](#) — approfondimenti da leggere con calma dopo aver completato le 9 settimane
- [19. Risorse online](#) — materiali di consultazione rapida da usare durante e dopo il piano di studio

Risorse

- [Learning How to Learn \(Coursera, Barbara Oakley\)](#) — corso gratuito molto pratico su tecniche di apprendimento efficace, utile per organizzare meglio lo studio in 9 settimane
- [Cal Newport — Come studiare in modo efficace](#) — riflessioni pratiche su concentrazione, ripasso attivo e gestione del tempo di studio
- [Atlassian — Guida per i nuovi Scrum Master](#) — utile come lettura di rinforzo trasversale durante tutto il percorso, in particolare nelle settimane 3-5

18. Libri consigliati

Le sezioni da 1 a 17 di questo corso ti hanno già dato tutto ciò che ti serve per muovere i primi passi nel ruolo: i fondamenti tecnici, i framework Agile/Scrum, il Project Management (incluso il vocabolario PMP/PMBOK), il mondo DevOps e l'intelligenza artificiale. Questa sezione è diversa dalle altre: non introduce concetti nuovi e obbligatori, ma raccoglie **15 libri e pubblicazioni di riferimento**, indicati direttamente dal responsabile del team, pensati per **approfondire con più calma e più profondità** ciò che nel corso hai visto in forma sintetica.

La logica dei 5 livelli

I libri sono organizzati in **5 livelli di lettura progressiva**, costruiti seguendo la stessa logica “a mattoncini” con cui è organizzato il resto del corso (lo ricordi dalla sezione 1: non puoi costruire il tetto se non hai ancora le fondamenta):

- **Livello 0 – Fondamenti di informatica:** rafforza la sezione 2. Utile soprattutto se, arrivando da un percorso gestionale, senti ancora un po' di terreno instabile sotto i piedi quando si parla di computer, reti o architetture di base.
- **Livello 1 – Sviluppo software:** rafforza la sezione 3 (Come nasce un software) e la sezione 4 (Git e GitHub). Ti aiuta a capire **come lavorano davvero gli sviluppatori** che gestirai e con cui parlerai ogni giorno — non per scrivere codice tu stesso, ma per capire le loro priorità, i loro vincoli e il loro linguaggio.
- **Livello 2 – Agile:** rafforza le sezioni 5 (Agile), 6 (Scrum) e 7 (Kanban). È il livello più direttamente collegato al ruolo che andrai a occupare: Scrum Master.
- **Livello 3 – Project Management:** rafforza la sezione 8 (Project Management) e fa da ponte verso la sezione 9 (DevOps), mostrando cosa succede quando i metodi “leggeri” di Agile incontrano progetti complessi, reali, con tutte le loro frizioni.
- **Livello 4 – DevOps:** rafforza le sezioni 9-13 (parte del Blocco C del corso, quello più tecnico, che comprende anche ambienti di sviluppo e intelligenza artificiale), con un'attenzione particolare a **perché** le pratiche DevOps funzionano, supportata da dati e ricerca, non solo da intuizione.

Non c'è un obbligo di leggerli tutti né di seguire l'ordine alla lettera: sono pensati per essere agganciati **al momento giusto del tuo percorso**, quando l'argomento del livello corrispondente è già “caldo” perché lo hai appena studiato o lo stai osservando dal vivo nel team. Per questo, ogni libro qui sotto include un suggerimento su **quando** leggerlo rispetto al [Piano di studio](#) a 9 settimane.



Come usare questa sezione: non è pensata per essere letta tutta subito. Torna qui ogni volta che completi un blocco di sezioni del corso (ad esempio dopo aver finito la sezione 6 su Scrum) e scegli il libro del livello corrispondente che ti interessa di più. Va benissimo leggerne solo alcuni, o leggerli fuori ordine, se un argomento ti appassiona particolarmente.

Livello 0 – Fondamenti di informatica

“Code” – Charles Petzold

Questo libro racconta, con un linguaggio quasi narrativo e senza dare per scontato nulla, come si arriva dai concetti più elementari (interruttori, codice binario) fino a un computer moderno funzionante. È il libro ideale se, dopo la sezione 2, ti restano ancora dei dubbi su **cosa succede davvero dentro la macchina** quando parliamo di CPU, memoria o istruzioni: qui lo vedrai costruito pezzo per pezzo, in modo che l'intuizione resti solida anche quando in team si parla di argomenti più avanzati come container o cloud.

- **Quando leggerlo:** consigliato durante la settimana 1-2, in parallelo o subito dopo la sezione 2 (Fondamenti di informatica).
- **Difficoltà:** Media (richiede attenzione ma nessuna competenza pregressa).
- **Tempo stimato:** 10-12 ore di lettura complessiva.

“Computer Science Distilled” – Wladston Ferreira Filho

Un libro molto compatto e visuale, pensato per chi vuole avere una mappa d'insieme dei concetti chiave dell'informatica (algoritmi, strutture dati, basi di reti e sistemi) senza addentrarsi nei dettagli tecnici che, nel tuo ruolo, non ti serviranno mai scrivere in prima persona. È un ottimo complemento “riassuntivo” alla sezione 2 del corso: dove il corso spiega con analogie, questo libro offre schemi e diagrammi che aiutano a fissare i concetti in memoria a lungo termine.

- **Quando leggerlo:** consigliato durante la settimana 2, come ripasso rapido dopo la sezione 2.
- **Difficoltà:** Facile (molto sintetico e visuale).
- **Tempo stimato:** 4-6 ore di lettura complessiva.

“Computer Networking: A Top-Down Approach” – Kurose & Ross (capitoli introduttivi)

Questo è il manuale universitario di riferimento per le reti informatiche: non serve leggerlo per intero (è pensato per un intero corso semestrale), ma i **capitoli introduttivi** sulla struttura di internet, sui protocolli TCP/IP e su HTTP/HTTPS offrono un livello di dettaglio superiore a quello della sezione 2, utile se nel team senti spesso discussioni su reti, ambienti e connessioni tra servizi e vuoi capirle a fondo, non solo a livello di analogia.

- **Quando leggerlo:** lettura di consolidamento dopo la settimana 2, oppure più avanti se emergono dubbi specifici su reti durante le sezioni su Cloud (12) o CI/CD (10).
- **Difficoltà:** Impegnativo (è un testo universitario, anche se i capitoli iniziali sono i più accessibili).
- **Tempo stimato:** 6-8 ore per i soli capitoli introduttivi.

Livello 1 – Sviluppo software

“Clean Code” – Robert C. Martin

Non diventerai uno sviluppatore, ma capire **cosa rende il codice “buono” o “scadente”** agli occhi di chi lo scrive ti aiuta moltissimo a comprendere discussioni di code review, stime più lunghe del previsto per “ripulire” una parte di codice (il cosiddetto debito tecnico) o perché il team insiste su determinate pratiche. Questo libro, uno dei più citati nel mondo dello sviluppo software, ti dà il vocabolario e la sensibilità per capire quelle conversazioni dall’interno, collegandosi direttamente a quanto visto nella sezione 3 sul ciclo di vita del software e sulla code review.

- **Quando leggerlo:** consigliato durante la settimana 3, dopo aver completato la sezione 3 (Come nasce un software) e la sezione 4 (Git e GitHub).
- **Difficoltà:** Media (alcuni esempi di codice, ma i concetti generali sono comprensibili anche senza saperlo scrivere).
- **Tempo stimato:** 10-12 ore di lettura complessiva.

“The Pragmatic Programmer” – Andrew Hunt & David Thomas

Un classico che va oltre il “come scrivere codice” per parlare di **come pensano e lavorano gli sviluppatori**: gestione degli errori, automazione, comunicazione con il team, cura dei dettagli. È particolarmente utile per un futuro Scrum Master perché offre uno sguardo diretto sulla mentalità delle persone che faciliterai e coordinerai, aiutandoti a capire le loro priorità quando negozi scadenze o priorità del backlog.

- **Quando leggerlo:** consigliato durante la settimana 3-4, come lettura di accompagnamento a “Clean Code” o subito dopo.
- **Difficoltà:** Media.
- **Tempo stimato:** 8-10 ore di lettura complessiva.

Livello 2 – Agile

“Scrum: The Art of Doing Twice the Work in Half the Time” – Jeff Sutherland

Scritto da uno dei co-creatori di Scrum, questo libro racconta la nascita del framework attraverso storie concrete (compresi progetti falliti prima dell'adozione di Scrum), rendendo tangibile il “perché” dietro le pratiche che hai studiato nella sezione 6 in modo più formale. È una lettura quasi narrativa, perfetta per consolidare la motivazione dietro Sprint, Daily Scrum e Retrospective proprio mentre inizi a osservarle dal vivo nel team, secondo la Fase 1 (Osservazione) del tuo percorso di crescita.

- **Quando leggerlo:** consigliato durante la settimana 3-4, subito dopo la sezione 6 (Scrum), mentre osservi le prime cerimonie del team.
- **Difficoltà:** Facile (scorrevole, orientato agli aneddoti più che alla teoria pura).
- **Tempo stimato:** 6-8 ore di lettura complessiva.

“Essential Scrum” – Kenneth Rubin

Se il libro di Sutherland è la storia e la motivazione, questo è il **manuale di riferimento**: copre in modo sistematico ruoli, eventi e artefatti di Scrum, con diagrammi chiari e casi limite che il corso, per motivi di sintesi, non può trattare in dettaglio (ad esempio come gestire Sprint con più team, o come stimare backlog molto grandi). È il libro da tenere sul comodino durante la Fase 2 (Affiancamento attivo), quando comincerai a condurre tu stesso alcune cerimonie.

- **Quando leggerlo**: lettura di consolidamento dopo la settimana 4, quando inizi a facilitare direttamente le prime attività.
- **Difficoltà**: Media (più denso e sistematico del libro di Sutherland).
- **Tempo stimato**: 12-15 ore di lettura complessiva.

Livello 3 – Project Management

“Making Things Happen” – Scott Berkun

Un libro sul project management che non segue la logica rigida del Waterfall universitario, ma parla di gestione dei progetti nel mondo reale: comunicazione, gestione delle pressioni, decisioni sotto incertezza. Si collega direttamente alla sezione 8, dove hai visto che il project management in un contesto Agile è più simile a “un navigatore che aggiorna la rotta” che a un piano rigido: questo libro approfondisce esattamente quella metafora con esempi pratici su stakeholder, rischi e comunicazione.

- **Quando leggerlo**: consigliato durante la settimana 5, subito dopo la sezione 8 (Project Management).
- **Difficoltà**: Media.
- **Tempo stimato**: 10-12 ore di lettura complessiva.

“A Guide to the Project Management Body of Knowledge (PMBOK Guide)” – Project Management Institute (PMI)

La guida ufficiale del PMI, il testo normativo di riferimento dietro la certificazione PMP e dietro gran parte del vocabolario visto nelle sottosezioni PMP della sezione 8 (Project Charter, WBS, EVM, triplo vincolo...). Attenzione a un dettaglio importante: la 7ª edizione, quella attualmente in vigore, ha cambiato profondamente impostazione rispetto alla 6ª, basandosi su 12 principi e 8 performance domain invece che sui classici 5 gruppi di processi e le 10 aree di conoscenza — se trovi materiale (anche online) che parla ancora di questi ultimi, sappi che si riferisce alla 6ª edizione, tuttora molto citata

nella pratica. Non è un libro da leggere in sequenza come un manuale narrativo: è pensato per la **consultazione**, un po' come un dizionario dei processi da cui pescare quando ti serve un dettaglio specifico.

- **Quando leggerlo**: da consultare dopo le sottosezioni PMP della sezione 8 (Project Management), non da leggere in sequenza dall'inizio alla fine.
- **Difficoltà**: Impegnativo (linguaggio normativo e denso, pensato per la consultazione più che per la lettura continua).

- **Tempo stimato:** non è un tempo di lettura lineare — è un testo da tenere a portata di mano e consultare per argomento, secondo necessità.

“The Standard for Project Management” – Project Management Institute (PMI)

Incluso nello stesso volume della 7ª edizione del PMBOK Guide, questo standard descrive in modo più conciso i principi generali su cui si basa la gestione di un progetto, indipendentemente dal settore o dalla metodologia adottata. È il punto di ingresso più leggero se vuoi farti un'idea dei principi alla base del PMP senza affrontare subito tutto il PMBOK Guide.

- **Quando leggerlo:** consigliato come introduzione ai principi PMP, prima di un'eventuale consultazione più approfondita del PMBOK Guide.
- **Difficoltà:** Media (più sintetico e meno tecnico del PMBOK Guide completo).
- **Tempo stimato:** alcune ore per una prima lettura dei principi, ma resta soprattutto un testo a cui tornare quando serve.

“Agile Practice Guide” – Project Management Institute (PMI) e Agile Alliance

Pubblicato dal PMI in collaborazione con Agile Alliance, questo testo affronta esattamente il tema dell'approccio ibrido visto a chiusura delle sottosezioni PMP della sezione 8: come far convivere il vocabolario tradizionale del Project Management con le pratiche Agile che hai già studiato nelle sezioni 5-7. È probabilmente il libro più utile di questo blocco per il tuo ruolo specifico, perché non chiede di scegliere tra i due mondi ma spiega concretamente come si integrano.

- **Quando leggerlo:** consigliato subito dopo aver completato le sottosezioni PMP della sezione 8, per consolidare il collegamento con Agile/Scrum/Kanban.
- **Difficoltà:** Media.
- **Tempo stimato:** 6-8 ore di lettura complessiva.

“PMP Exam Prep” – Rita Mulcahy

Uno dei manuali di preparazione all'esame PMP più diffusi e citati nel settore, pensato per chi vuole affrontare la certificazione in modo strutturato. Non è necessario per questo corso né per il tuo ruolo attuale, ma se in futuro valuterai la certificazione PMP (vedi i prossimi passi nella sezione 17) è uno dei testi più citati dalla community per prepararsi. Verifica sempre qual è l'edizione più recente disponibile: il PMI aggiorna periodicamente l'esame e i manuali di preparazione seguono questi aggiornamenti.

- **Quando leggerlo:** solo se in futuro deciderai di intraprendere il percorso di certificazione PMP; non è collegato a una settimana specifica di questo piano di studio.
- **Difficoltà:** Impegnativo (pensato per la preparazione a un esame professionale, non per una lettura divulgativa).
- **Tempo stimato:** variabile in base all'edizione e al piano di studio personale per l'esame — non indicato qui per non riportare numeri non verificati.

“The Phoenix Project” – Gene Kim, Kevin Behr, George Spafford

Un romanzo aziendale (sì, è scritto come una storia, non come un manuale) che segue un IT manager alle prese con un progetto in crisi, mostrando in modo molto realistico le tensioni tra sviluppo, operations e business che la cultura DevOps cerca di risolvere. È probabilmente il libro più “riconoscibile” di questa lista per chi lavora nel team: molte delle dinamiche raccontate (rilasci in ritardo, colpe scaricate tra reparti, pressione degli stakeholder) sono facilmente riconducibili a episodi reali che osserverai. Fa da ponte naturale verso la sezione 9 (DevOps).

- **Quando leggerlo:** consigliato durante la settimana 5-6, come transizione tra il blocco Project Management e il blocco DevOps del corso.
 - **Difficoltà:** Facile (si legge come un romanzo, nonostante i temi tecnici).
 - **Tempo stimato:** 8-10 ore di lettura complessiva.
-

Livello 4 – DevOps

“The DevOps Handbook” – Gene Kim, Jez Humble, Patrick Debois, John Willis

Se “The Phoenix Project” racconta la storia in forma di romanzo, questo libro ne è il **manuale operativo**: spiega in modo sistematico i principi e le pratiche DevOps (i tre flussi, l'automazione, il modello CALMS che hai già incontrato nella sezione 9) con esempi reali di aziende che li hanno adottati. È il testo di riferimento per capire in profondità le sezioni 9-13 del corso, collegando cultura, pratiche tecniche e risultati di business in un unico quadro coerente.

- **Quando leggerlo:** consigliato durante la settimana 6, dopo aver completato la sezione 9 (DevOps) e la sezione 10 (CI/CD).
- **Difficoltà:** Impegnativo (denso, ma organizzato in modo molto chiaro).
- **Tempo stimato:** 15-18 ore di lettura complessiva.

“Accelerate” – Nicole Forsgren, Jez Humble, Gene Kim

Un libro diverso dagli altri: qui la cultura DevOps non viene raccontata per aneddoti ma **misurata con dati e ricerca scientifica**, mostrando quali pratiche (deployment frequenti, automazione, cultura blameless) sono statisticamente collegate a team e organizzazioni ad alte prestazioni. È la lettura ideale per chi, come te, viene da un percorso gestionale abituato a ragionare per KPI e metriche: ti dà argomenti solidi per spiegare al cliente o ai responsabili **perché** certe pratiche DevOps valgono l'investimento, andando oltre l'intuizione.

- **Quando leggerlo:** lettura di consolidamento dopo la settimana 6-7, come chiusura del percorso DevOps del corso e apertura verso l'autonomia della Fase 3.
 - **Difficoltà:** Impegnativo (alcuni capitoli hanno un taglio statistico/metodologico).
 - **Tempo stimato:** 10-12 ore di lettura complessiva.
-

#	Livello	Titolo	Autore/i	Difficoltà	Tempo stimato
1	0 – Fondamenti di informatica	Code	Charles Petzold	Media	10-12 ore
2	0 – Fondamenti di informatica	Computer Science Distilled	Wladston Ferreira Filho	Facile	4-6 ore
3	0 – Fondamenti di informatica	Computer Networking: A Top-Down Approach (cap. introduttivi)	Kurose & Ross	Impegnativo	6-8 ore
4	1 – Sviluppo software	Clean Code	Robert C. Martin	Media	10-12 ore
5	1 – Sviluppo software	The Pragmatic Programmer	Andrew Hunt & David Thomas	Media	8-10 ore
6	2 – Agile	Scrum: The Art of Doing Twice the Work in Half the Time	Jeff Sutherland	Facile	6-8 ore
7	2 – Agile	Essential Scrum	Kenneth Rubin	Media	12-15 ore
8	3 – Project Management	Making Things Happen	Scott Berkun	Media	10-12 ore
9	3 – Project Management	A Guide to the Project Management Body of Knowledge (PMBOK Guide)	Project Management Institute (PMI)	Impegnativo	consultazione, non lettura lineare
10	3 – Project Management	The Standard for Project Management	Project Management Institute (PMI)	Media	alcune ore + consultazione
11	3 – Project Management	Agile Practice Guide	Project Management Institute (PMI) e Agile Alliance	Media	6-8 ore

#	Livello	Titolo	Autore/i	Difficoltà	Tempo stimato
12	3 – Project Management	PMP Exam Prep	Rita Mulcahy	Impegnativo	variabile (preparazione esame)
13	3 – Project Management	The Phoenix Project	Gene Kim, Kevin Behr, George Spafford	Facile	8-10 ore
14	4 – DevOps	The DevOps Handbook	Gene Kim, Jez Humble, Patrick Debois, John Willis	Impegnativo	15-18 ore
15	4 – DevOps	Accelerate	Nicole Forsgren, Jez Humble, Gene Kim	Impegnativo	10-12 ore

 **Nota importante:** questi 15 libri e pubblicazioni sono un **arricchimento facoltativo**, non un requisito per completare il corso o per essere operativo nel ruolo — ma sono **fortemente raccomandati** dal responsabile del team, perché offrono una profondità che nessun corso di onboarding può dare in poche settimane. Non c'è alcuna fretta: molti di questi libri richiedono decine di ore di lettura e hanno tutto il senso di essere letti **con calma, anche ben oltre le 9 settimane iniziali** del [Piano di studio](#), magari uno al mese nei primi 6-12 mesi di lavoro. L'importante è tornare a questa lista ogni volta che un argomento del corso ti ha incuriosito e vuoi andare più a fondo. Le pubblicazioni ufficiali del PMI (PMBOK Guide e The Standard for Project Management) sono testi normativi di consultazione: non serve leggerli in sequenza come gli altri, ma tenerli a portata di mano quando serve un dettaglio preciso sul vocabolario PMP.

Collegamenti

- [19. Risorse online](#) — altri materiali gratuiti (articoli, video, documentazione ufficiale) da consultare insieme o in alternativa ai libri
- [17. Piano di studio](#) — il programma delle 9 settimane a cui agganciare le letture suggerite in questa sezione

Risorse

- [Goodreads](#) — per leggere recensioni, valutazioni e trovare edizioni (anche in italiano) di ciascun libro prima di acquistarlo
- [O'Reilly Online Learning](#) — molti di questi titoli (in particolare quelli sullo sviluppo software e su DevOps) sono disponibili in versione e-book con abbonamento, spesso già incluso in piani aziendali

- Tutti i libri elencati sono inoltre disponibili sui principali store online (versione cartacea e e-book) e in molte biblioteche universitarie, essendo testi di riferimento ampiamente diffusi nel settore

19. Risorse online

Come usare questa sezione

Questa non è una sezione da leggere tutta d'un fiato, e non è nemmeno un test: è una **cassetta degli attrezzi**. Raccoglie link a documentazione ufficiale, corsi gratuiti, video e articoli, organizzati per area tematica seguendo lo stesso ordine delle sezioni del corso.

Il modo giusto di usarla è così:

- Mentre studi una sezione (ad esempio la 6, su Scrum), torna qui e apri l'area corrispondente ("Agile, Scrum, Kanban") per trovare materiale di approfondimento se un argomento ti ha incuriosito o non ti è chiaro al 100%.
- Se in una riunione qualcuno cita uno strumento o un concetto che non conosci, puoi tornare qui in un secondo momento (non durante la riunione!) e cercare la risorsa giusta per approfondire con calma.
- Non serve leggere ogni link elencato: sono alternative e integrazioni, non un elenco di compiti a casa. Scegli quelle che si adattano al tuo modo di imparare (alcune persone preferiscono leggere documentazione, altre guardare video, altre seguire corsi strutturati passo passo).
- Molte risorse sono in inglese: è normale, ed è anche una buona palestra, perché la quasi totalità della documentazione tecnica "primaria" (GitHub, AWS, OWASP...) nasce in inglese e le traduzioni arrivano dopo, quando arrivano.

Le risorse sono raggruppate in 7 aree, che corrispondono ai grandi blocchi del corso.

1. Fondamenti di informatica

(per la sezione 2 — Fondamenti di informatica)

- **MDN Web Docs (Mozilla)** — Documentazione ufficiale — Il riferimento più completo e autorevole per capire come funzionano web, browser, HTML/CSS/JS e i concetti di base di Internet, scritto in modo accessibile anche a chi non è sviluppatore. — <https://developer.mozilla.org/>
- **roadmap.sh — Computer Science roadmap** — Corso gratuito / percorso guidato — Una mappa visuale gratuita che elenca, in ordine logico, tutti i concetti di base dell'informatica (hardware, reti, sistemi operativi) con link di approfondimento per ciascuno. — <https://roadmap.sh/>
- **freeCodeCamp** — Corso gratuito — Piattaforma gratuita con corsi introduttivi su informatica, programmazione e reti, pensata per chi parte da zero e vuole progredire con esercizi pratici. — <https://www.freecodecamp.org/>
- **Coursera** — Corso gratuito (con opzione certificazione a pagamento) — Ospita corsi introduttivi di informatica di università reali (ad esempio corsi di "Computer Science 101" o "IT fundamentals"), spesso frequentabili gratuitamente in modalità "audit". — <https://www.coursera.org/>

- **Video — canale ufficiale di un'università o di un provider cloud** — Video — Cerca su YouTube playlist introduttive tipo “how computers work” o “networking basics” pubblicate da canali istituzionali (università, Microsoft, AWS, Google): sono generalmente più affidabili di canali amatoriali. — <https://www.youtube.com/>
 - **Microsoft Learn — Fondamenti di Azure (AZ-900)** — Documentazione ufficiale / corso gratuito — Percorso gratuito e guidato di Microsoft che parte dai concetti di base del cloud, utile anche solo per capire il vocabolario di base dell'informatica moderna. — <https://learn.microsoft.com/training/courses/az-900t01>
-

2. Sviluppo software e Git/GitHub

(per le sezioni 3-4 — Come nasce un software, Git e GitHub)

- **Git — documentazione ufficiale** — Documentazione ufficiale — Il sito ufficiale del progetto Git, con la guida di riferimento e il libro gratuito “Pro Git” scaricabile in italiano e altre lingue. — <https://git-scm.com/>
 - **GitHub Docs** — Documentazione ufficiale — La documentazione ufficiale di GitHub: guide passo passo su repository, pull request, issue e tutte le funzionalità della piattaforma. — <https://docs.github.com/>
 - **Learn Git Branching** — Corso gratuito interattivo — Un tutorial visuale e interattivo (gratuito) che fa vedere graficamente cosa succede a branch e commit man mano che esegui comandi Git; ottimo per capire il “modello mentale” di Git senza usare il terminale. — <https://learngitbranching.js.org/>
 - **GitHub Skills** — Corso gratuito interattivo — Corsi interattivi ufficiali di GitHub, per imparare facendo (pull request, collaborazione, GitHub Actions). — <https://skills.github.com/>
 - **Atlassian Git Tutorials** — Articolo / documentazione — Una serie di guide molto chiare (con immagini) sui concetti fondamentali di Git: branching, merge, workflow di team. — <https://www.atlassian.com/git>
 - **roadmap.sh — Backend / Full Stack roadmap** — Corso gratuito / percorso guidato — Mappe gratuite che mostrano, in ordine, i concetti che uno sviluppatore impara per costruire software, utili per capire il “vocabolario” dei tuoi colleghi sviluppatori. — <https://roadmap.sh/>
-

3. Agile, Scrum, Kanban

(per le sezioni 5-7 — Agile, Scrum, Kanban)

- **Manifesto for Agile Software Development** — Documentazione ufficiale — Il testo originale del Manifesto Agile (disponibile anche in italiano), con i 4 valori e i 12 principi che hanno dato origine a tutte le metodologie Agile. — <https://agilemanifesto.org/>
- **Scrum Guide (Scrum Guides)** — Documentazione ufficiale — La Guida Scrum ufficiale, scritta dai creatori di Scrum (Ken Schwaber e Jeff Sutherland), disponibile gratuitamente in molte lingue incluso l'italiano. — <https://scrumguides.org/>

- **Scrum.org** — Documentazione ufficiale / community — Il sito dell'organizzazione fondata da Ken Schwaber, con articoli, blog e risorse gratuite su Scrum e sul ruolo di Scrum Master. — <https://www.scrum.org/>
 - **Atlassian — Agile Coach** — Articolo / corso gratuito — Una guida online gratuita e molto completa su Agile, Scrum e Kanban, scritta con un linguaggio pratico e molti esempi concreti. — <https://www.atlassian.com/agile>
 - **Atlassian — Kanban Guide** — Articolo — Guida dedicata specificamente a Kanban: colonne, WIP limit, flusso continuo, con esempi visuali. — <https://www.atlassian.com/agile/kanban>
 - **Kanbanize (Businessmap) — Kanban Guide** — Articolo — Una risorsa gratuita e molto dettagliata dedicata interamente al metodo Kanban, alle sue origini (Toyota) e alla sua applicazione nello sviluppo software. — <https://kanbanize.com/kanban-resources>
 - **Scrum Alliance** — Community / documentazione — Una delle principali organizzazioni internazionali dedicate a Scrum, con articoli, risorse gratuite e informazioni sulle certificazioni (utile anche solo per capire il vocabolario delle certificazioni che potresti sentir citare). — <https://www.scrumalliance.org/>
-

4. Project Management

(per la sezione 8 — Project Management)

- **PMI — Project Management Institute** — Documentazione ufficiale / community — L'organizzazione internazionale di riferimento per il project management, con articoli gratuiti, standard di settore e informazioni sulle certificazioni (come il PMP). — <https://www.pmi.org/>
 - **PMI — What is Project Management** — Articolo — Pagina introduttiva ufficiale del PMI che spiega cos'è il project management, i suoi principi e le competenze richieste, utile come base comune di vocabolario. — <https://www.pmi.org/about/learn-about-pmi/what-is-project-management>
 - **Atlassian — Project Management Guide** — Articolo — Guida pratica e gratuita, orientata al software, su tecniche di project management applicate a team di sviluppo (roadmap, stime, gestione del rischio). — <https://www.atlassian.com/agile/project-management>
 - **Coursera — corsi di Project Management** — Corso gratuito (con opzione certificazione) — Diverse università e aziende (incluso Google) pubblicano su Coursera corsi introduttivi di project management, spesso frequentabili gratuitamente in modalità "audit". — <https://www.coursera.org/>
 - **freeCodeCamp — articoli su Project Management** — Articolo — Il blog di freeCodeCamp pubblica periodicamente guide gratuite e accessibili su tecniche di project management applicate al software. — <https://www.freecodecamp.org/news/>
-

5. DevOps e CI/CD

(per le sezioni 9-10 — DevOps, CI/CD)

- **GitHub Docs — GitHub Actions** — Documentazione ufficiale — La documentazione completa e ufficiale di GitHub Actions, lo strumento di pipeline CI/CD integrato in GitHub: workflow, runner, artifact, con guide passo passo. — <https://docs.github.com/actions>
 - **Martin Fowler — Continuous Integration** — Articolo — Un articolo di riferimento, scritto da uno degli autori più citati nel mondo dello sviluppo software, che spiega in profondità cos'è la Continuous Integration e perché conta. — <https://martinfowler.com/articles/continuousIntegration.html>
 - **Atlassian — CI/CD Pipeline** — Articolo — Guida pratica con schemi visuali su cosa sono le pipeline di CI/CD e come si differenziano CI, delivery continua e deployment continuo. — <https://www.atlassian.com/continuous-delivery/continuous-integration>
 - **Docker — documentazione ufficiale** — Documentazione ufficiale — Se nelle pipeline del team senti parlare di container e immagini Docker, questa è la documentazione ufficiale con cui iniziare a orientarsi. — <https://docs.docker.com/>
 - **roadmap.sh — DevOps roadmap** — Corso gratuito / percorso guidato — Una mappa gratuita che elenca, in ordine logico, tutti i concetti e strumenti dell'ecosistema DevOps, con link di approfondimento per ciascuno. — <https://roadmap.sh/devops>
-

6. Architetture, Cloud, Sicurezza

(per le sezioni 11-13 — Architetture software, Cloud, Sicurezza)

- **Microsoft Learn — Azure Architecture Center** — Documentazione ufficiale — Raccolta ufficiale di pattern architetturali, best practice e glossario dedicati alle architetture cloud moderne (utile anche solo per orientarsi tra i diagrammi che il team potrebbe mostrarti). — <https://learn.microsoft.com/azure/architecture/>
- **Microsoft Learn — Fondamenti di Azure** — Corso gratuito — Il percorso formativo gratuito e ufficiale Microsoft per capire i concetti base del cloud: risorse, regioni, modelli di servizio (IaaS/PaaS/SaaS). — <https://learn.microsoft.com/training/azure/>
- **AWS — Cloud concepts / What is Cloud Computing** — Documentazione ufficiale — La spiegazione ufficiale di Amazon Web Services su cosa significa “cloud computing”, utile per un confronto con la terminologia usata da Microsoft Azure. — <https://aws.amazon.com/what-is-cloud-computing/>
- **Martin Fowler — bliki (Microservices e architetture)** — Articolo — Articoli di riferimento sul tema delle architetture software moderne, inclusi i microservizi, scritti con un linguaggio più discorsivo rispetto alla documentazione tecnica pura. — <https://martinfowler.com/microservices/>
- **OWASP Top 10** — Documentazione ufficiale — La classifica ufficiale e gratuita delle vulnerabilità di sicurezza più critiche per le applicazioni web, un riferimento citato in moltissimi contratti e audit di sicurezza. — <https://owasp.org/www-project-top-ten/>
- **OWASP Foundation** — Documentazione ufficiale / community — Il sito della fondazione OWASP, con decine di progetti e guide gratuite sulla sicurezza applicativa, oltre a eventi e community locali. — <https://owasp.org/>

- **Microsoft Learn — Zero Trust e concetti di sicurezza** — Documentazione ufficiale — Introduzione ufficiale Microsoft ai principi moderni di sicurezza applicati al cloud, utile per capire il vocabolario che i colleghi tecnici usano nelle discussioni su sicurezza e accessi. — <https://learn.microsoft.com/security/zero-trust/>
 - **Microsoft Learn — DevSecOps** — Documentazione ufficiale — Approfondimento su come la sicurezza si integra nel ciclo DevOps, con pratiche come lo “shift left” già incontrato nella sezione 13. — <https://learn.microsoft.com/devops/operate/security-in-devops>
-

7. Intelligenza artificiale

(per la sezione 15 — Intelligenza artificiale)

- **GitHub Copilot** — Prodotto / pagina ufficiale — La pagina ufficiale dello strumento di AI generativa integrato nell’editor di codice che il team usa quotidianamente per scrivere codice più velocemente. — <https://github.com/features/copilot>
 - **GitHub Docs — GitHub Copilot** — Documentazione ufficiale — La documentazione ufficiale di GitHub su come funziona Copilot, come si configura e come usarlo in modo responsabile all’interno di un team. — <https://docs.github.com/copilot>
 - **DeepLearning.AI** — Corso gratuito / community — La piattaforma fondata da Andrew Ng, con corsi (alcuni gratuiti in modalità “audit”) che spiegano in modo accessibile Machine Learning, Deep Learning e AI generativa, anche per chi non ha background tecnico. — <https://www.deeplearning.ai/>
-

Come restare aggiornato nel tempo

Il mondo DevOps si evolve rapidamente: nuove funzionalità di GitHub, nuove versioni degli strumenti, nuove pratiche di sicurezza. Non serve rincorrere ogni novità, ma un aggiornamento periodico e leggero ti aiuterà a non restare indietro. Qualche suggerimento pratico:

- **Segui i blog ufficiali degli strumenti che usa il team:** sia GitHub (The GitHub Blog) sia i principali cloud provider (Microsoft Azure, AWS) pubblicano regolarmente novità, cambi di funzionalità e guide pratiche — bastano 10 minuti ogni tanto per restare al passo.
 - **Iscriviti a una newsletter di settore:** esistono diverse newsletter gratuite (via email) dedicate ad Agile, DevOps o Project Management che riassumono settimanalmente le notizie più rilevanti, così non devi andare a cercarle una per una.
 - **Partecipa a community e, se possibile, a un evento o webinar all’anno:** gruppi locali o online di Scrum Master/Agile Coach, conferenze di settore (anche solo guardando le registrazioni gratuite pubblicate dopo l’evento) e community come quella di Scrum.org o PMI ti mettono in contatto con altre persone che affrontano le tue stesse sfide quotidiane.
-

Collegamenti

- [16. Glossario](#) — per ripassare rapidamente la terminologia incontrata in queste risorse
- [17. Piano di studio](#) — il percorso settimanale in cui inserire, se vuoi, l'approfondimento di alcune di queste risorse

20. Backend Engineering Avanzato

Nella sezione [2 \(Fondamenti di informatica\)](#) hai visto le basi: cos'è HTTP, cos'è un'API, come funziona in linea di massima un database relazionale. Nella sezione [13 \(Sicurezza\)](#) hai visto la differenza tra autenticazione e autorizzazione, e il ruolo dei token. Questa sezione riprende esattamente da lì e **va più a fondo**: è un approfondimento pensato per il momento in cui, seduto in una revisione tecnica o in una discussione su una stima, senti termini come “sharding”, “idempotenza di un endpoint”, “isolation level” o “cache-aside” e vuoi capire non solo cosa significano, ma **perché il team li discute e quali trade-off si nascondono dietro ciascuna scelta**.

Non ti verrà chiesto di scrivere questo codice. Ma capire *perché* Marco insiste per aggiungere un indice prima di rilasciare una funzionalità, perché Giulia propone di introdurre una cache invece di “semplicemente scalare il server”, o perché una certa modifica al database richiede una transazione e non due semplici scritture separate, ti renderà un interlocutore molto più efficace nelle discussioni tecniche — e ti aiuterà a stimare meglio il rischio reale di un cambiamento, non solo la sua dimensione in righe di codice.

Come sempre, restiamo su **ShopFacile**: la stessa piattaforma e-commerce con Marco e Giulia (developer), Ahmed (developer junior, in crescita), Sara (Product Owner) e Luca (Scrum Master).

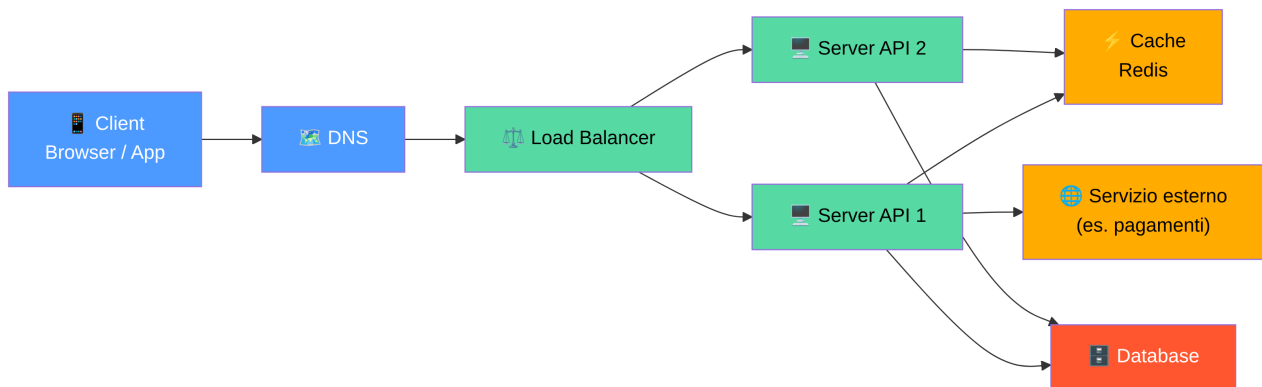
Obiettivi della sezione

Alla fine di questa sezione saprai:

- descrivere l'architettura di un sistema backend “in profondità”, dal client fino al database, includendo load balancer, cache e servizi esterni;
- spiegare cosa succede davvero durante un handshake TLS (HTTPS) e perché rende una connessione sicura;
- distinguere sessioni server-side e **JWT**, e ricostruire un flusso **OAuth 2.0** completo con i suoi attori;
- riconoscere una API REST progettata bene da una progettata male, secondo una checklist concreta (versioning, paginazione, errori, idempotenza);
- spiegare le proprietà **ACID** di una transazione e collegarle a un caso reale (es. un trasferimento di saldo);
- distinguere i principali **livelli di isolamento** di una transazione e i problemi di concorrenza che prevengono (dirty read, lost update, phantom read);
- spiegare il **teorema CAP** e usarlo per motivare la scelta tra database relazionale e NoSQL;
- distinguere **replication** e **sharding** come due strategie di scaling complementari, non intercambiabili;
- descrivere le principali strategie di **caching** (cache-aside, write-through, write-behind) e i problemi di invalidazione che portano con sé;
- spiegare, a grandi linee, come Node.js gestisce migliaia di richieste concorrenti con un solo thread (event loop);
- riconoscere le cause più comuni di una query lenta e sapere cosa chiedere a un developer che dice “il problema è nel database”.

20.1 Architettura di un sistema backend, in profondità

Nella sezione 2 hai visto client, server e database come tre blocchi separati. Nella realtà, tra il client e il database si interpone quasi sempre una catena più lunga di componenti, ciascuno con un compito preciso. Vale la pena vederla per intero almeno una volta, perché ogni “anello” della catena è un punto dove qualcosa può rallentare o rompersi — e sapere dove si trova ciascun anello ti aiuta a seguire una diagnosi tecnica senza perderti.



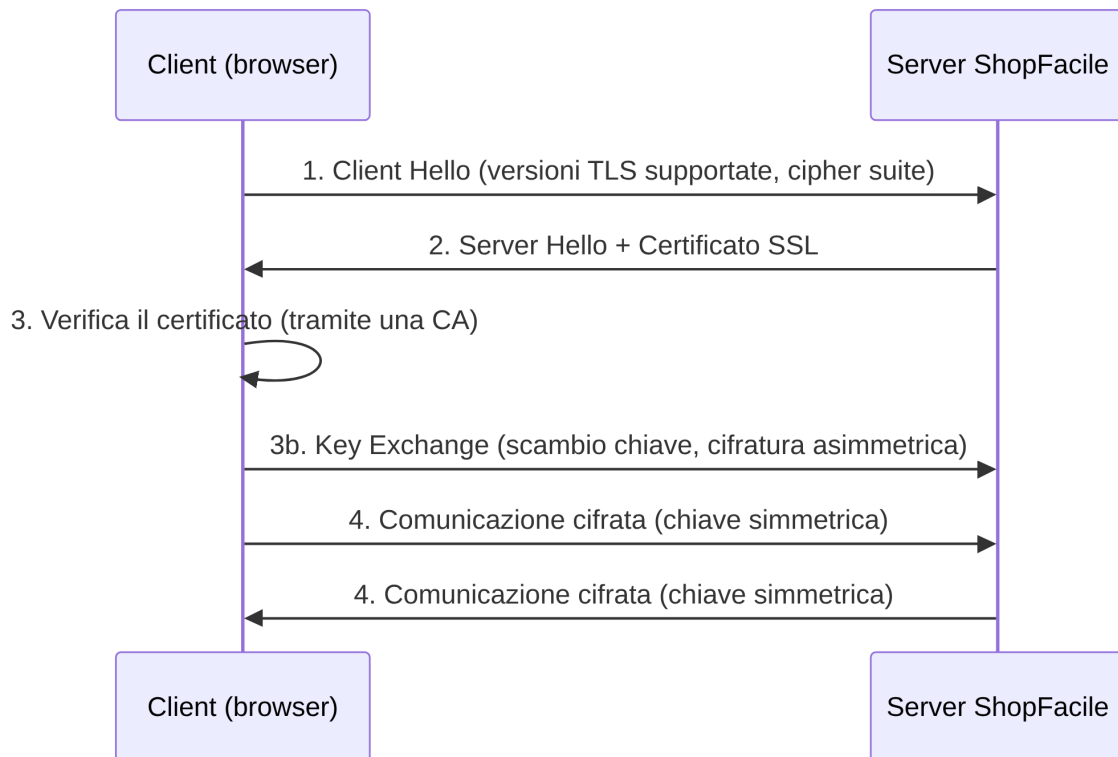
Il pezzo nuovo rispetto alla sezione 2 è il **Load Balancer** (bilanciatore di carico): un componente che riceve tutte le richieste in arrivo e le distribuisce su più server applicativi identici (**Server API 1** , **Server API 2** , ...), invece di farle gestire a un unico server. Questo risolve due problemi insieme: se un server va sotto carico o si guasta, il traffico continua a fluire sugli altri; e se il traffico cresce, basta aggiungere un altro server dietro lo stesso bilanciatore (è esattamente la **scalabilità orizzontale** già vista nel glossario) senza toccare il codice dell'applicazione.

💡 **Analogia:** il load balancer è come il maître di un ristorante molto frequentato: non cucina lui stesso, ma decide a quale dei cuochi disponibili (i server) assegnare ogni nuovo ordine (richiesta) che entra, in modo che nessun cuoco resti sovraccarico mentre un altro è a mani vuote.

Perché questa catena funzioni davvero, però, il modello **Stateless** visto nella sezione 2 non è un dettaglio elegante: è una condizione necessaria. Se un server API salvasse “in memoria” i dati della sessione di un cliente, e il load balancer inviasse la richiesta successiva di quello stesso cliente a un server diverso, quel server non saprebbe nulla di lui. Per questo, come vedremo nel paragrafo 20.3, i sistemi moderni tendono a spostare lo stato fuori dal server applicativo — in un database, in una cache condivisa o in un token che il client stesso porta con sé a ogni richiesta.

20.2 Dentro una richiesta HTTPS: il TLS handshake

Sai già, dalla sezione 2, che HTTPS è “HTTP cifrato” e che vedi il lucchetto  nel browser quando è attivo. Vale la pena vedere **cosa succede davvero**, in pochi passaggi, prima che il primo byte di dati venga scambiato — è una delle domande più frequenti nei colloqui tecnici di backend, ed è un buon esempio di come un sistema apparentemente “istantaneo” (clicchi un link e la pagina compare) in realtà nasconde una piccola negoziazione a più fasi.



Il punto chiave, che spiega anche perché HTTPS non è “gratis” in termini di performance: la prima fase (passi 1-3b) usa **crittografia asimmetrica** — più robusta, ma più lenta — solo per concordare in sicurezza una chiave condivisa; una volta stabilita quella chiave, tutto il resto della comunicazione (passo 4, che dura per l'intera sessione) passa a **crittografia simmetrica**, molto più rapida. È un compromesso deliberato: usare la parte costosa solo per il minimo indispensabile, la parte economica per tutto il resto.

Il **certificato SSL** che il server presenta al passo 2 è firmato da una **Autorità di Certificazione (CA)** riconosciuta: è quello che permette al browser di verificare al passo 3 che sta davvero parlando con **shopfacile.it** e non con un impostore che si finge tale (un attacco noto come *man-in-the-middle*). Se il certificato è scaduto o non è firmato da una CA fidata, il browser mostra l'avviso “connessione non sicura” che avrai visto capitare qualche volta.

 **Esempio pratico:** durante un audit, il team scopre che il certificato SSL di un sottodominio interno di ShopFacile scadrà tra tre giorni. Ahmed lo rinnova in tempo grazie a un allarme di monitoraggio (sezione 9.9) impostato apposta per i certificati in scadenza — senza quell'allarme, alla scadenza tutti i clienti che visitano quel sottodominio vedrebbero un errore di sicurezza nel browser, anche se il resto del sito continuerebbe a funzionare perfettamente.

20.3 Autenticazione stateless: sessioni, JWT e OAuth 2.0 in pratica

Nella sezione 13 hai visto la differenza tra autenticazione, autorizzazione e token in generale. Qui vediamo **come si costruisce concretamente** un sistema di autenticazione stateless, e perché è diventato lo standard per API e app mobile.

Sessione server-side vs JWT

Il modo “classico” di gestire il login è la **sessione**: dopo l'autenticazione, il server crea un record (in memoria o in un database) che dice “l'utente 42 è loggato”, e restituisce al browser un identificatore di sessione (di solito in un cookie). Ad ogni richiesta successiva, il server consulta quel record per sapere chi sta parlando.

Il **JWT (JSON Web Token)** rovescia l'approccio: invece di far ricordare al server chi sei, il server ti consegna un token **autocontenuto** — i dati sull'identità stanno dentro il token stesso, firmato in modo che il server possa verificarne l'autenticità senza dover consultare nulla.

	Sessione server-side	JWT
Dove vive lo stato	Sul server (o in un database condiviso)	Sul client, dentro il token
Scalabilità	Richiede uno store di sessioni condiviso tra i server	Naturalmente compatibile con più server dietro un load balancer
Revoca immediata	Facile: basta eliminare il record	Difficile: il token resta valido finché non scade
Caso d'uso tipico	App web tradizionali, server-rendered	API, app mobile, architetture a microservizi

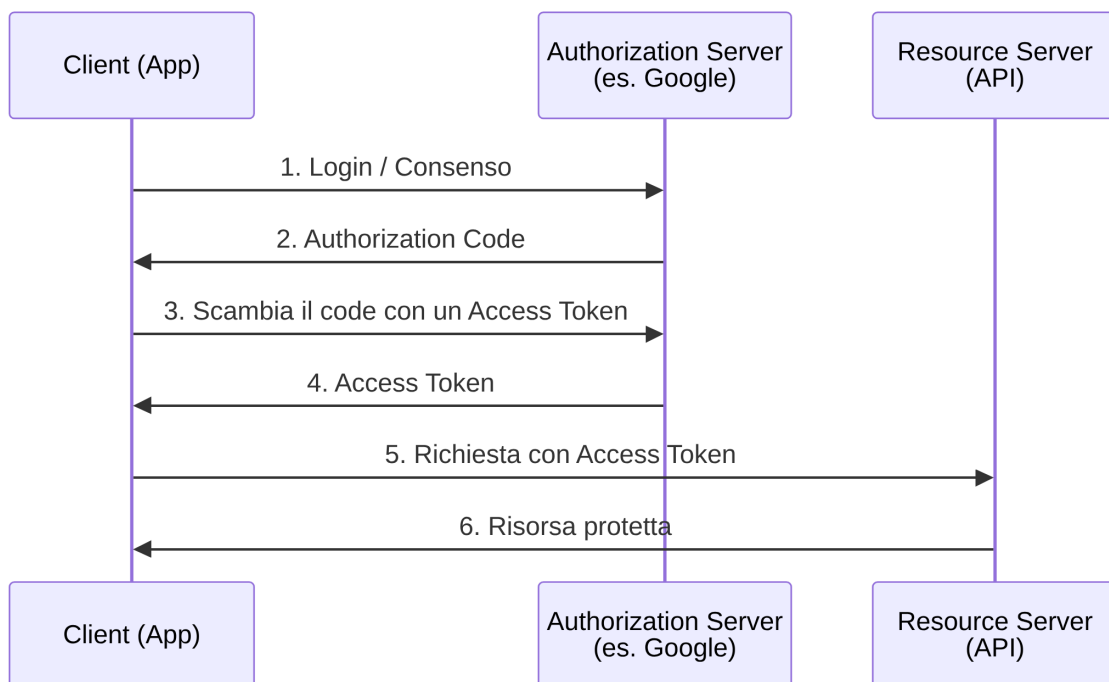
Un **JWT** è composto da tre parti separate da un punto: `header.payload.signature`. L'header dichiara l'algoritmo di firma usato; il payload contiene i dati (es. id utente, ruolo, scadenza); la firma è calcolata cifrando le prime due parti con una chiave segreta nota solo al server — è quella firma che rende impossibile, per chi intercetta il token, modificarne il contenuto (es. cambiare `"role": "user"` in `"role": "admin"`) senza che il server se ne accorga alla verifica successiva.

💡 **Analogia:** una sessione server-side è come un braccialetto di un evento con solo un numero stampato sopra — chi controlla l'ingresso deve consultare una lista per sapere a chi appartiene quel numero. Un JWT è come un biglietto che ha già scritto sopra, in modo inalterabile (sigillato), “ingresso VIP, valido fino alle 23:00”: chi lo controlla legge direttamente il biglietto, senza consultare nessuna lista — ma se lo perdi, chiunque lo trovi può usarlo fino a quell'orario.

Proprio per questo, la scadenza (**exp**) di un JWT va tenuta **breve** (minuti, non giorni): è l'unico argine reale contro un token rubato, dato che non può essere “disattivato” a metà strada come una sessione. La pratica comune è accompagnare un JWT a vita breve con un **refresh token** a vita più lunga, che il client usa solo per richiederne uno nuovo quando quello corrente scade, senza dover richiedere di nuovo username e password ogni volta.

OAuth 2.0: delegare un accesso senza cedere la password

La sezione 13 ha già introdotto OAuth come “il meccanismo dietro al pulsante accedi con Google”. Vediamone il flusso completo, con i suoi tre attori:



Nota che l'app (**Client**) non vede **mai** la password dell'utente: la inserisce direttamente sulla pagina dell' **Authorization Server** (es. Google), che al termine restituisce solo un **Access Token** con permessi **limitati e revocabili** (es. “leggi solo l'email e il nome”) — esattamente la definizione di delega che hai già incontrato nel glossario.

 **Esempio pratico:** ShopFacile integra un pulsante “accedi con Google”. Sara chiede: “e se un cliente stacca l’accesso da Google, cosa succede sul nostro sito?” Marco spiega che l’access token concesso a ShopFacile smette di funzionare alla prima richiesta successiva a quella revoca — è proprio quel meccanismo di delega revocabile a rendere sicura l’integrazione: ShopFacile non ha mai avuto la password del cliente da “dimenticare”.

20.4 Progettare una API REST che invecchia bene

Nella sezione 2 hai visto lo stile REST e i verbi HTTP. Qui vediamo alcune regole pratiche che distinguono una API REST progettata con cura da una scritta “di corsa” — regole che diventano costose da correggere una volta che clienti esterni si sono già integrati con l’API.

Idempotenza: perché conta davvero

Un’operazione è **idempotente** se eseguirla più volte con lo stesso input produce lo stesso risultato di eseguirla una volta sola. Non è un dettaglio accademico: è la proprietà che rende sicuro **ritentare** una richiesta quando la risposta si è persa per un problema di rete (il client non sa se il server ha effettivamente elaborato la richiesta o no).

Verbo	Idempotente?	Perché
GET	Sì	Legge, non modifica nulla
PUT	Sì	“Imposta questa risorsa a questo stato”: ripeterlo produce lo stesso stato finale
DELETE	Sì	Dopo la prima cancellazione, le successive non cambiano nulla (la risorsa è già assente)
POST	No	Ogni chiamata crea una nuova risorsa: ripeterla crea duplicati
PATCH	Di solito no	Dipende dall’operazione: “incrementa di 1” non è idempotente, “imposta il campo X a Y” lo è

 **Esempio pratico:** un cliente di ShopFacile clicca “Conferma ordine”, la connessione si interrompe per un attimo e il browser ritenta automaticamente la stessa richiesta **POST /ordini**. Senza precauzioni, il cliente si ritrova con **due ordini identici** addebitati. Per questo Giulia introduce una **idempotency key**: il client genera un identificatore univoco per quel tentativo di

ordine e lo manda nell'header della richiesta; se il server vede due richieste con la stessa chiave, la seconda restituisce semplicemente il risultato già calcolato la prima volta, senza creare un secondo ordine.

Versionamento, paginazione, errori

La sezione 2 ha già introdotto OpenAPI come “contratto” tra team. Una API che deve convivere a lungo con client esterni ha bisogno di altre tre convenzioni, sempre concordate in anticipo:

- **Versionamento** (già visto nel glossario in forma generale): tipicamente nell'URL (`/api/v1/ordini`), così una modifica che rompe la compatibilità (rimuovere un campo, cambiarne il significato) può convivere temporaneamente con la versione precedente, invece di rompere di colpo tutti i client esistenti.
- **Paginazione**: restituire migliaia di righe in un'unica risposta `GET /api/ordini` è impraticabile. Le due tecniche principali sono la **paginazione a offset** (`?page=2&limit=20` , semplice ma lenta su dataset molto grandi, perché il database deve comunque scorrere tutte le righe saltate) e la **paginazione a cursore** (`?after=id_ultimo_visto` , più efficiente su grandi volumi, perché il database può partire direttamente da quel punto).
- **Formato degli errori**: un'API ben progettata restituisce errori in un formato **coerente e prevedibile** (stesso schema JSON per ogni errore, con un codice, un messaggio leggibile e magari un campo che indica quale campo della richiesta ha fallito la validazione), non un testo libero diverso a ogni endpoint.

💡 **Trade-off**: aggiungere versionamento fin dal primo giorno sembra un costo “premature” quando l'API ha ancora un solo cliente (magari l'app stessa di ShopFacile). Ma il costo di introdurlo *dopo*, quando già esistono client esterni in produzione che si aspettano l'URL senza versione, è enormemente più alto: richiede coordinare un cambiamento con ogni consumatore dell'API, non solo modificare il proprio codice.

20.5 Transazioni e proprietà ACID

Hai già incontrato SQL e le tabelle nella sezione 2. Ora affrontiamo un concetto che diventa cruciale ogni volta che un'operazione tocca **più di una riga, o più di una tabella, che devono cambiare insieme o non cambiare affatto**.

Una **transazione** è una sequenza di operazioni sul database trattata come **un'unica unità di lavoro**: o tutte le operazioni hanno successo (`COMMIT`), o nessuna viene applicata (`ROLLBACK`) — non esiste una via di mezzo dove solo metà delle operazioni sono state salvate.

```

BEGIN;
UPDATE conti SET saldo = saldo - 1000 WHERE id = 1; -- addebito
UPDATE conti SET saldo = saldo + 1000 WHERE id = 2; -- accredito
COMMIT; -- se qualcosa va storto: ROLLBACK;

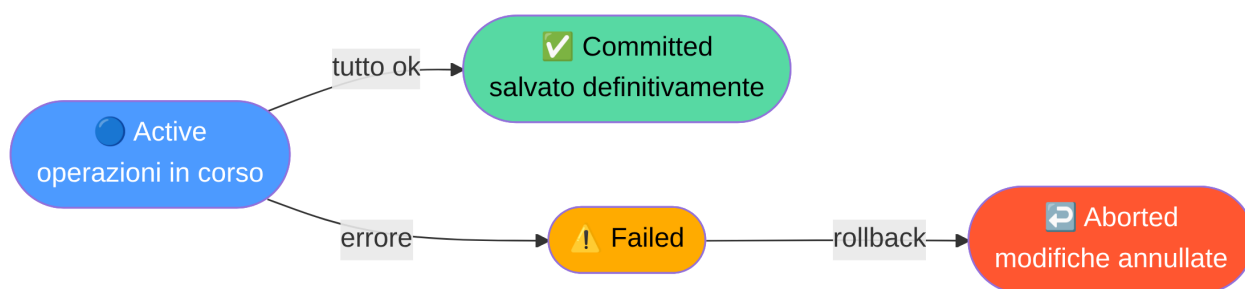
```

💡 **Analogia:** pensa a un trasferimento di denaro come a due persone che si scambiano una busta con dei soldi. Se una lascia cadere la busta a metà passaggio, non puoi lasciare che una persona abbia “già dato” e l'altra non abbia “ancora ricevuto”: la transazione garantisce che l'intero scambio avvenga, oppure che sia come se non fosse mai cominciato — nessuno stato intermedio è ammesso.

Le proprietà che ogni transazione deve garantire si chiamano **ACID**:

Proprietà	Cosa garantisce	Esempio ShopFacile
Atomicity (atomicità)	Tutto o nulla	L'addebito e l'accredito avvengono insieme, o nessuno dei due
Consistency (coerenza)	Il database resta in uno stato valido	Il saldo non può mai diventare negativo, se esiste quel vincolo
Isolation (isolamento)	Transazioni concorrenti non si “vedono” a metà strada	Due clienti che prenotano l'ultimo posto disponibile non finiscono entrambi per “vincerlo”
Durability (durabilità)	Una volta confermata, la modifica resta anche dopo un crash	Un ordine confermato non “svanisce” se il server si riavvia un secondo dopo

Le transazioni passano per **stati** ben definiti:



20.6 Isolamento e concorrenza: cosa può andare storto tra due transazioni

La proprietà di **Isolation** vista sopra non è un interruttore tutto-o-nulla: i database offrono diversi **livelli di isolamento**, ognuno con un compromesso diverso tra correttezza e velocità. Per capire perché servono, guardiamo prima **cosa può andare storto** quando due transazioni girano nello stesso momento sugli stessi dati:

Problema	Cosa succede	Esempio
Dirty read (lettura sporca)	Una transazione legge dati scritti da un'altra transazione non ancora confermata	T1 legge un saldo che T2 ha modificato ma poi annulla con ROLLBACK
Non-repeatable read (lettura non ripetibile)	La stessa query, eseguita due volte nella stessa transazione, restituisce risultati diversi	T1 legge il prezzo di un prodotto due volte e ottiene due valori diversi, perché T2 lo ha modificato nel frattempo
Phantom read (lettura fantasma)	Una query che seleziona un intervallo di righe restituisce righe diverse se ripetuta, perché nel frattempo ne sono state inserite di nuove	T1 conta gli ordini di oggi due volte, T2 ne inserisce uno in mezzo, il secondo conteggio è più alto
Lost update (aggiornamento perso)	Due transazioni leggono lo stesso valore, lo modificano entrambe, e l'ultima sovrascrive silenziosamente il lavoro della prima	Due operatori aggiornano lo stock dello stesso prodotto contemporaneamente: l'ultimo "vince", il primo aggiornamento sparisce senza errori

I livelli di isolamento standard, dal più permissivo (rapido, ma più rischioso) al più rigoroso (sicuro, ma più lento):

Livello	Dirty read	Non-repeatable read	Phantom read	Note
Read Uncommitted	Possibile	Possibile	Possibile	Raramente usato in pratica
Read Committed	Evitato	Possibile	Possibile	Predefinito in PostgreSQL e SQL Server
Repeatable Read	Evitato	Evitato	Possibile	Predefinito in MySQL (InnoDB)
Serializable	Evitato	Evitato	Evitato	Massima coerenza, ma più lento e con più blocchi

Come i database moderni riducono il costo dell'isolamento: MVCC

Il modo più semplice per garantire l'isolamento sarebbe **bloccare** i dati ogni volta che qualcuno li sta modificando (**locking pessimistico**): chi vuole leggere deve aspettare che chi scrive finisca. Funziona, ma penalizza la concorrenza — molte letture restano in coda dietro poche scritture.

La maggior parte dei database moderni (PostgreSQL, MySQL InnoDB) usa invece il **MVCC (Multi-Version Concurrency Control)**: quando una riga viene modificata, il database non la sovrascrive subito, ma mantiene **più versioni** della stessa riga contemporaneamente. Chi sta leggendo continua a vedere la versione coerente con l'inizio della propria transazione, mentre chi scrive lavora su una nuova versione — **i lettori non bloccano gli scrittori, e viceversa**. È il motivo per cui, in pratica, un database moderno riesce a gestire moltissime letture concorrenti senza che rallentino a vicenda con le scritture in corso.

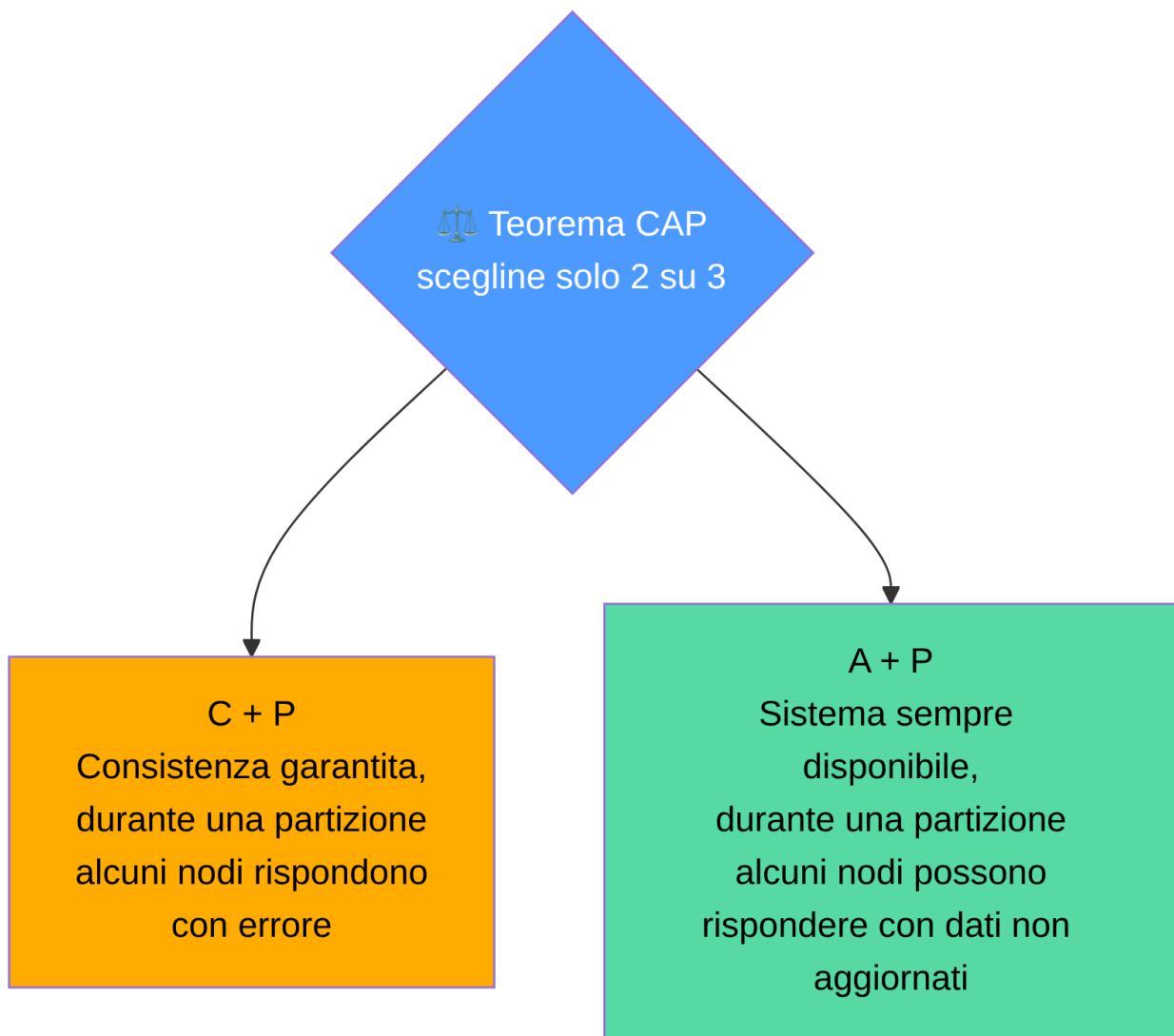
 **Trade-off:** MVCC non è gratuito — mantenere più versioni della stessa riga richiede spazio e un processo periodico (in PostgreSQL si chiama *vacuum*) che elimina le versioni ormai inutili. Un database che non viene mai “pulito” accumula versioni vecchie e finisce, paradossalmente, a rallentare per lo stesso motivo che doveva risolvere.

20.7 NoSQL e il teorema CAP

La sezione 2 ha introdotto i database NoSQL come alternativa ai relazionali quando i dati sono meno strutturati o i volumi molto grandi. Il motivo tecnico per cui esistono famiglie di database così diverse tra loro è riassunto da un teorema fondamentale per chi progetta sistemi distribuiti.

Il **teorema CAP** afferma che, in un sistema distribuito (dati sparsi su più macchine collegate in rete), è possibile garantire **solo due delle tre proprietà seguenti allo stesso tempo**:

- **Consistency (C)**: ogni nodo del sistema vede sempre gli stessi dati, nello stesso istante;
- **Availability (A)**: il sistema resta sempre disponibile a rispondere, anche se qualcosa va storto;
- **Partition Tolerance (P)**: il sistema continua a funzionare anche se la rete tra i nodi si interrompe temporaneamente (una “partizione”).



Nota che la **Partition Tolerance** non è davvero opzionale in un sistema reale distribuito su più macchine o data center: le reti si interrompono, è un fatto della vita, non una scelta di design. La scelta pratica che un team fa davvero è quindi tra **C+P** e **A+P**:

Scelta	Comportamento durante un problema di rete	Esempi
C + P (Consistenza)	Alcuni nodi rispondono con un errore, piuttosto che con un dato potenzialmente non aggiornato	MongoDB (configurazione predefinita)
A + P (Disponibilità)	Il sistema risponde sempre, anche con dati leggermente non aggiornati (coerenza eventuale)	Cassandra, DynamoDB

 **Esempio pratico:** Sara chiede perché il carrello di ShopFacile usa un database diverso (orientato a disponibilità) rispetto al sistema di pagamento (orientato a consistenza). Marco spiega: se il carrello risponde per un istante con un dato leggermente vecchio durante un

problema di rete, il peggio che succede è che un cliente veda un articolo che ha già rimosso — fastidioso, non grave. Se invece il sistema di pagamento “inventasse” un saldo per restare sempre disponibile, il rischio è addebitare due volte la stessa carta: lì la consistenza vale più della disponibilità, sempre.

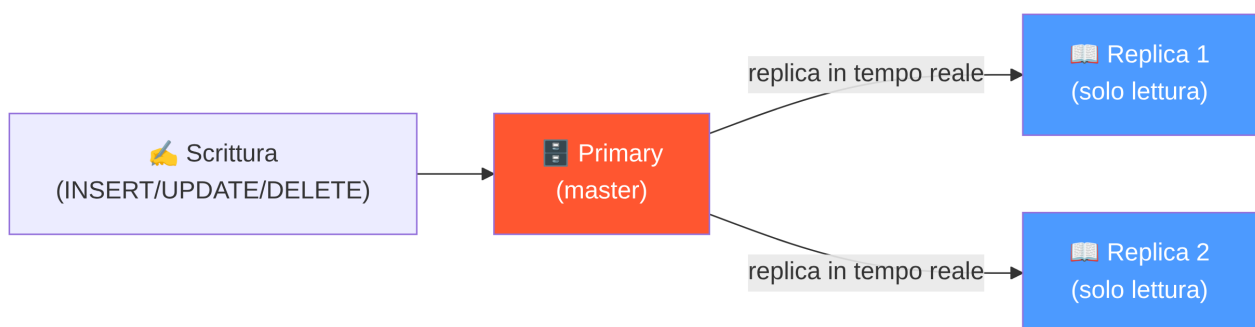
Questo spiega anche la riga della sezione 2 su “**Transazioni**: Forte supporto ACID (SQL) / Consistenza eventuale, spesso BASE (NoSQL)”: **BASE** (Basically Available, Soft state, Eventually consistent) è, in un certo senso, l’opposto filosofico di ACID — un compromesso deliberato in favore della disponibilità e della scalabilità, accettando che i dati possano essere temporaneamente non sincronizzati tra i nodi.

20.8 Scalare un database: replication e sharding

Quando un singolo database non basta più — troppe letture, troppe scritture, o troppi dati per una sola macchina — esistono due strategie distinte, spesso confuse tra loro ma complementari.

Replication: copie per scalare le letture

La **replication** mantiene **più copie identiche** dello stesso database: un nodo **primario** (o master) gestisce tutte le scritture, e le propaga automaticamente a uno o più nodi **replica** (o slave), che gestiscono le letture.

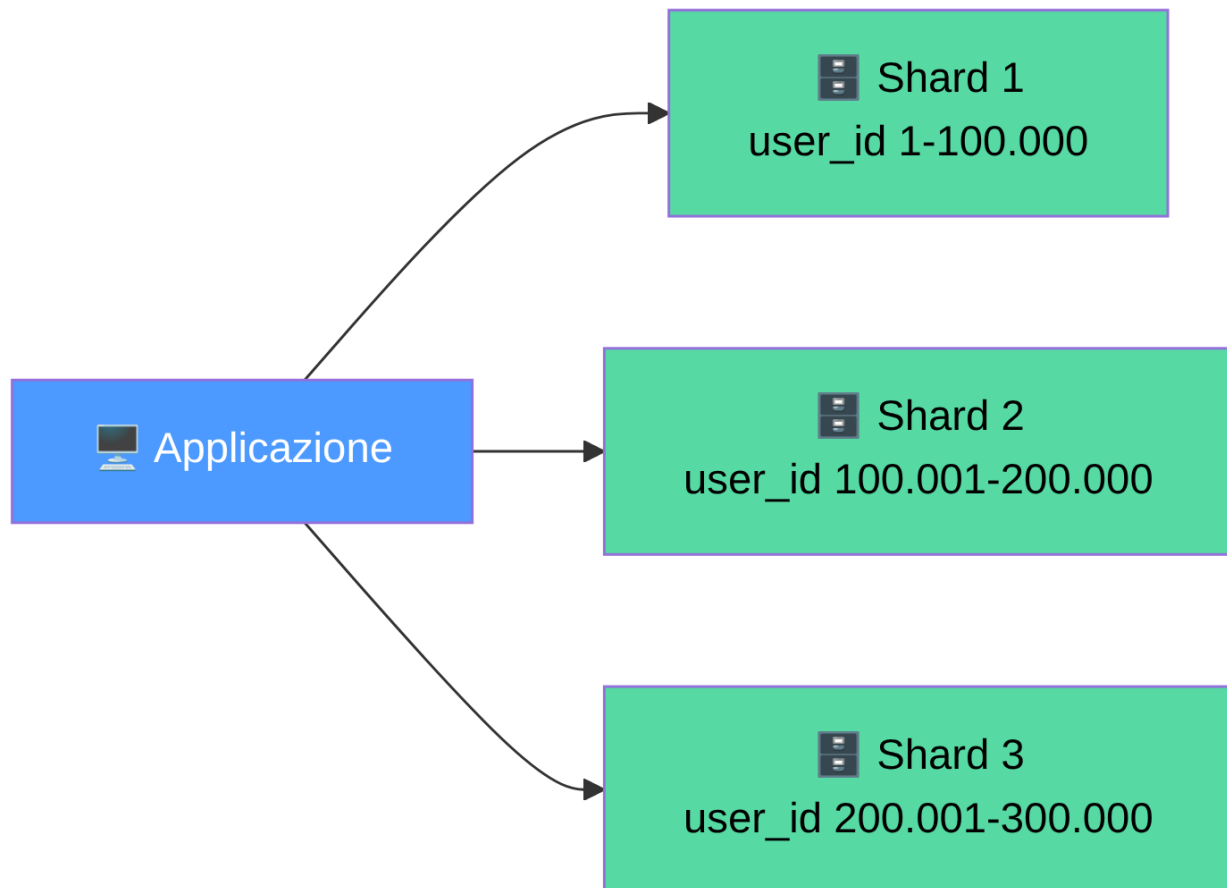


Il beneficio è duplice: le letture (spesso la maggioranza del traffico su un e-commerce come ShopFacile: catalogo, ricerche, dettagli prodotto) si distribuiscono su più macchine, e se il primario si guasta, una replica può essere promossa rapidamente al suo posto, garantendo alta disponibilità.

💡 **Trade-off:** la propagazione dalla primary alle replica richiede un istante, non è mai istantanea. Se un cliente aggiorna il proprio indirizzo e la pagina successiva legge da una replica che non ha ancora ricevuto quella modifica, potrebbe vedere per una frazione di secondo il vecchio indirizzo. È lo stesso concetto di “coerenza eventuale” visto a proposito del teorema CAP, applicato qui a un database relazionale “normale”.

Sharding: dividere i dati per scalare le scritture

Il **sharding** (o partizionamento orizzontale) affronta un problema diverso: quando i **dati stessi** sono troppo grandi o le **scritture** troppo frequenti per una singola macchina, anche con più repliche in lettura. La soluzione è dividere i dati in porzioni indipendenti (**shard**), distribuite su macchine diverse, in base a una **chiave di sharding** (es. `user_id`).



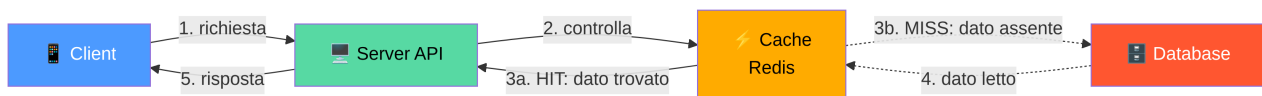
	Replication	Sharding
Problema che risolve	Troppe letture	Troppi dati o troppe scritture
Cosa contiene ogni nodo	L'intero database, in copia	Solo una porzione dei dati
Complessità	Relativamente semplice da introdurre	Più complessa: query che toccano più shard, ribilanciamento se una chiave cresce troppo
Si possono combinare?	Sì: ogni shard può a sua volta avere le proprie repliche	—

 **Esempio pratico:** ShopFacile cresce e il database ordini diventa troppo grande per una macchina sola. Il team valuta di shardare per `region` (area geografica) invece che per `user_id`: significa che tutti gli ordini di una stessa area finiscono sullo stesso shard, il che rende semplici i report regionali ma rischia di sovraccaricare lo shard dell'area più popolosa in modo

squilibrato rispetto agli altri — la scelta della chiave di sharding, come spesso accade, è un compromesso senza soluzione perfetta, solo trade-off diversi da valutare rispetto ai pattern di accesso reali.

20.9 Caching: velocità in cambio di un problema nuovo

Aggiungere una cache è probabilmente la singola modifica più efficace per velocizzare un sistema che legge spesso gli stessi dati — ma introduce un problema nuovo, spesso riassunto dalla battuta (vera, non solo umoristica) che l'invalidazione della cache è “uno dei due problemi difficili dell'informatica”.



Questo pattern si chiama **cache-aside** (o *lazy loading*): l'applicazione controlla prima la cache; se il dato c'è (**cache hit**), lo restituisce immediatamente, senza toccare il database; se non c'è (**cache miss**), lo recupera dal database, lo salva in cache per la prossima volta, e solo allora risponde. È lo schema più diffuso perché è semplice e “fallisce bene”: se la cache non è disponibile, il sistema continua a funzionare appoggiandosi al database, solo più lentamente.

Esistono strategie alternative, ciascuna con un trade-off diverso:

Strategia	Come funziona	Trade-off
Cache-aside	L'app scrive nel DB, la cache si popola solo alla prima lettura successiva	Il primo accesso dopo una modifica è sempre un cache miss
Write-through	Ogni scrittura aggiorna cache e database insieme, nella stessa operazione	Sempre coerente, ma ogni scrittura è più lenta
Write-behind (write-back)	Si scrive subito in cache, e nel database in modo asincrono, dopo	Scritture molto rapide, ma rischio di perdere dati se la cache si guasta prima che il database venga aggiornato

Il problema vero: l'invalidazione

Una volta che un dato è in cache, cosa succede quando quel dato **cambia** nel database? Se la cache non viene aggiornata o rimossa, il client continua a ricevere un dato **vecchio** — un bug particolarmente insidioso perché “sembra funzionare” (il sistema risponde, velocemente, solo con il dato sbagliato). Le tecniche principali:

- **TTL (Time To Live)**: ogni voce in cache scade automaticamente dopo un tempo prefissato — semplice, ma per quella finestra di tempo il dato può essere non aggiornato;

- **Invalidazione esplicita:** quando il dato cambia nel database, il codice stesso rimuove (o aggiorna) la voce corrispondente in cache;
- **Basata su eventi:** un cambiamento pubblica un evento (collegandosi a quanto visto sull'architettura event-driven nella sezione 11) che un consumer usa per invalidare la cache, anche in altri servizi che non hanno scritto direttamente quel dato.

 **Analogia:** la cache è come un foglietto con gli appunti delle risposte più richieste, tenuto a portata di mano sulla scrivania invece che nell'archivio in fondo al corridoio (il database). È velocissimo da consultare — ma se qualcuno aggiorna l'informazione nell'archivio e nessuno aggiorna anche il foglietto, chi consulta il foglietto continua a dare la risposta vecchia, con piena sicurezza di avere ragione.

Quando la cache si riempie, un'ultima decisione riguarda **cosa buttare via** per far spazio a dati nuovi (**eviction policy**): le più comuni sono **LRU** (*Least Recently Used*, elimina ciò che non viene consultato da più tempo) e **LFU** (*Least Frequently Used*, elimina ciò che viene consultato meno spesso in assoluto) — Redis, la cache in-memory più diffusa, supporta entrambe le politiche in configurazione.

20.10 Concorrenza nel backend: thread, processi ed event loop

Un server backend reale gestisce **migliaia di richieste al secondo**, non una alla volta. Come questo sia possibile dipende dal modello di concorrenza scelto dal linguaggio o dal framework — e cambia sostanzialmente il modo in cui un problema di performance va diagnosticato.

Processo vs Thread

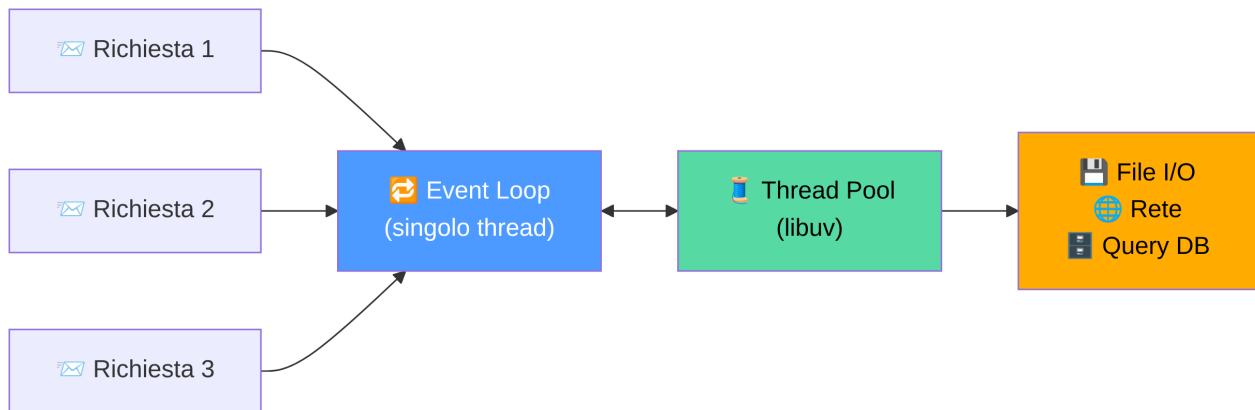
	Processo	Thread
Definizione	Programma indipendente, con propria memoria	Unità di esecuzione più leggera, dentro un processo
Memoria	Separata da ogni altro processo	Condivisa con gli altri thread dello stesso processo
Isolamento	Alto: se un processo va in crash, gli altri non ne sono toccati	Basso: un crash di un thread può travolgere l'intero processo
Costo di creazione	Più lento	Più rapido

Concorrenza vs parallelismo

Sono due parole spesso confuse, ma indicano cose diverse: la **concorrenza** significa gestire più attività **senza che debbano avanzare necessariamente nello stesso istante** (es. un solo core che alterna rapidamente tra più richieste); il **parallelismo** significa farle avanzare **davvero nello stesso istante**, su core diversi. Un server può essere concorrente senza essere parallelo.

L'event loop: come Node.js gestisce migliaia di richieste con un thread

Node.js (uno dei linguaggi elencati nella sezione 2 tra le tecnologie backend comuni) adotta un modello particolare: un **singolo thread principale** che non blocca mai in attesa di un'operazione lenta (lettura da disco, chiamata di rete, query al database). Quando una di queste operazioni viene richiesta, il thread principale la delega a un **thread pool** interno e continua immediatamente a occuparsi di altre richieste; quando l'operazione lenta finisce, il suo risultato rientra nel flusso principale tramite una **callback**.



💡 Analogia: pensa a un solo cameriere (l'event loop) che gestisce molti tavoli. Non resta impalato al tavolo 3 aspettando che la cucina finisca un piatto: prende l'ordine, lo passa in cucina (il thread pool) e va immediatamente a servire il tavolo 4. Quando il piatto del tavolo 3 è pronto, la cucina lo avvisa e lui lo porta — un solo cameriere può servire molti tavoli, a patto che non debba **mai** restare fermo ad aspettare.

Il rovescio della medaglia: se qualcuno scrive per errore del codice che **blocca** quell'unico thread principale (es. un calcolo molto pesante eseguito in modo sincrono), **tutte** le altre richieste restano in coda fino a quando quel calcolo non finisce — è uno degli errori più comuni e più dannosi in un'applicazione Node.js, ed è il motivo per cui si insiste tanto su operazioni asincrone (`async` / `await`) per ogni cosa lenta.

Race condition: il problema della concorrenza vera

Quando più thread (o processi) **accedono e modificano lo stesso dato condiviso contemporaneamente**, senza coordinamento, si crea una **race condition**:

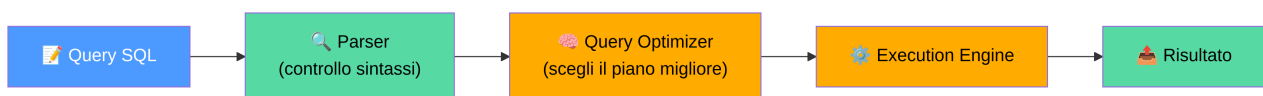
```
int contatore = 0;
// Thread 1: contatore++;
// Thread 2: contatore++;
// Il valore finale può essere 1 invece di 2, se le due operazioni
// si sovrappongono nel momento sbagliato
```

La soluzione classica è un **lock** (o mutex): un meccanismo che garantisce che, in un dato momento, solo un thread alla volta possa accedere a quella porzione di codice condivisa (la “sezione critica”). Un **read-write lock** è una variante più permissiva: permette a molti thread di leggere insieme, ma richiede accesso esclusivo per chi scrive — utile quando le letture sono molto più frequenti delle scritture, come spesso accade in un catalogo prodotti.

20.11 Query optimization: come una query lenta diventa rapida

Quando un developer del team dice “questa pagina è lenta perché la query sotto è lenta”, ecco cosa succede davvero, e cosa si può fare.

Come viene eseguita una query



Il pezzo interessante è il **Query Optimizer**: prima di eseguire davvero la query, il database valuta più modi possibili di eseguirla (con o senza un certo indice, in che ordine unire le tabelle) e stima quale sarà il più rapido, basandosi su statistiche interne sui dati. Il comando **EXPLAIN** (visto nella sezione 2 a proposito degli indici) mostra esattamente **quale piano** ha scelto — è lo strumento numero uno per capire perché una query è lenta, prima ancora di modificare qualcosa.

Le cause più comuni di una query lenta

- **Full table scan**: il database legge **tutta** la tabella riga per riga perché non esiste un indice utile sulla colonna del **WHERE** — su una tabella da poche migliaia di righe non si nota, su una da milioni diventa il primo sospetto;
- **Indici mancanti o sbagliati** su colonne usate spesso in **WHERE**, **JOIN** o **ORDER BY**;
- **SELECT *** quando servono solo 2-3 colonne: si trasferiscono e si elaborano dati inutili;
- **Funzioni applicate a colonne indicizzate** nel **WHERE** (es. **WHERE YEAR(data) = 2026**): impediscono al database di usare l'indice su **data**, anche se esiste — la versione corretta confronta direttamente l'intervallo di date (**WHERE data >= '2026-01-01' AND data < '2027-01-01'**);
- **Paginazione a offset molto alto** (**LIMIT 20 OFFSET 500000**): il database deve comunque scorrere le prime 500.000 righe scartate — è proprio il motivo per cui la paginazione a cursore, vista al paragrafo 20.4, è preferibile su dataset grandi.

 **Esempio pratico**: Ahmed nota che la pagina “i miei ordini” di ShopFacile è diventata lenta dopo che il catalogo clienti è cresciuto. Esegue **EXPLAIN** sulla query e scopre che il piano di esecuzione mostra **type: ALL** (full table scan) sulla colonna **user_id** della tabella ordini: non esiste nessun indice su quella colonna, quindi ogni volta il database scorre l'intera tabella per trovare gli ordini di un cliente. Crea l'indice (**CREATE INDEX idx_orders_user_id ON**

`ordini(user_id)`) e il tempo di risposta scende da secondi a millisecondi — un cambiamento piccolo, ma con un impatto enorme, esattamente il tipo di intervento che vale la pena chiedere per primo quando qualcuno segnala “il database è lento”.

💡 **Trade-off:** un indice non è mai “gratis”. Accelera le letture, ma ogni scrittura (`INSERT` / `UPDATE` / `DELETE`) deve aggiornare anche tutti gli indici esistenti su quella tabella — troppi indici su una tabella con scritture frequenti possono rallentarle sensibilmente. È un equilibrio da valutare caso per caso, non una regola “più indici, sempre meglio”.

20.12 Checklist finale: un’API pronta per la produzione

Riprendendo tutti i fili di questa sezione, ecco una checklist pratica che un team maturo verifica prima di considerare un’API davvero pronta per essere usata da altri (non solo “funziona sul mio computer”):

1. **Nomi ed endpoint coerenti:** nomi al plurale, sostantivi non verbi (`/ordini` , non `/creaOrdine`), maiuscole/minuscole consistenti.
2. **Verbi e status HTTP corretti:** `POST` per creare, `GET` per leggere, `404` quando la risorsa non esiste, `401` / `403` quando manca autenticazione/autorizzazione — non tutto ridotto a `200` con un campo `"success": false` dentro il corpo.
3. **Autenticazione e autorizzazione** implementate (paragrafo 20.3), sempre su HTTPS (paragrafo 20.2).
4. **Validazione dell’input** lato server, non solo lato client (che un utente può bypassare facilmente).
5. **Idempotenza** garantita dove serve (paragrafo 20.4), specialmente su operazioni economicamente sensibili come pagamenti e ordini.
6. **Paginazione** su ogni endpoint che può restituire liste grandi.
7. **Versionamento** pianificato fin dal primo rilascio pubblico, anche se sembra prematuro.
8. **Rate limiting:** un limite al numero di richieste che un singolo client può fare in un intervallo di tempo, per evitare abusi o un singolo client “affamato” che degrada il servizio per tutti gli altri.
9. **Documentazione** con OpenAPI/Swagger (sezione 2), mantenuta allineata al comportamento reale.
10. **Monitoraggio e log** (sezione 9) su ogni endpoint critico, per scoprire i problemi prima che li segnalino gli utenti.

20.13 Riepilogo

- Un sistema backend reale è una catena di componenti — DNS, load balancer, server applicativi, cache, database, servizi esterni — non un singolo blocco; il modello **stateless** è ciò che rende possibile distribuire il traffico su più server senza problemi.
- **HTTPS** stabilisce un canale sicuro tramite un handshake TLS che negozia una chiave condivisa usando crittografia asimmetrica solo all’inizio, per poi passare a crittografia simmetrica, più rapida, per tutta la sessione.

- Le **sessioni server-side** mantengono lo stato sul server; i **JWT** spostano lo stato, firmato e verificabile, sul client — una scelta architetturale con conseguenze dirette su scalabilità e revocabilità. **OAuth 2.0** permette di delegare un accesso limitato senza mai cedere la password.
- Una API REST “che invecchia bene” ha idempotenza dove serve, versionamento, paginazione ed errori coerenti pensati fin dal principio, non aggiunti in fretta quando è già troppo tardi.
- Le **transazioni** garantiscono le proprietà **ACID**; i **livelli di isolamento** governano quali problemi di concorrenza (dirty read, lost update, phantom read...) sono ammessi in cambio di maggiore velocità; il **MVCC** è la tecnica con cui i database moderni riducono il costo dell'isolamento senza bloccare tutto.
- Il **teorema CAP** spiega perché database diversi fanno scelte diverse tra consistenza e disponibilità durante un problema di rete — ed è la base razionale per scegliere tra SQL e NoSQL, non una moda tecnologica.
- **Replication** scala le letture con copie dei dati; **sharding** scala scritture e volumi dividendo i dati stessi: sono strategie complementari, non alternative.
- Il **caching** (cache-aside, write-through, write-behind) accelera un sistema, ma introduce il problema dell'invalidazione — spesso la parte più difficile, non la parte più “furba” del pattern.
- Il modello a **event loop** (Node.js) gestisce moltissime richieste concorrenti con un solo thread, a patto che nessuna operazione lo blocchi; le **race condition** nascono quando thread diversi modificano dati condivisi senza coordinamento, e si prevengono con lock e strutture dati pensate per la concorrenza.
- La maggior parte delle **query lente** ha una causa individuabile con **EXPLAIN**: indici mancanti, full table scan, funzioni su colonne indicizzate, paginazione a offset molto alto — quasi sempre risolvibile senza riscrivere l'applicazione.

Con questa sezione hai completato il ponte tra i concetti di base visti nella sezione 2 e le decisioni architetturali reali che il team di ShopFacile discute ogni settimana. Non serve saper scrivere questo codice da solo — ma la prossima volta che qualcuno propone “aggiungiamo una cache” o “dobbiamo shardare il database”, saprai esattamente quale domanda fare per capire se è la scelta giusta.



Esercizi pratici

1. **Ricostruisci la catena di una richiesta.** Disegna a mano lo schema della sezione 20.1 (client → DNS → load balancer → server → database) e aggiungi, con un colore diverso, dove inseriresti una cache — a monte o a valle del load balancer, e perché.  **Come verificare:** il tuo schema ha almeno 5 “anelli” distinti e sai spiegare a voce, senza guardare gli appunti, cosa succederebbe se uno qualsiasi di essi si guastasse.
2. **Racconta il TLS handshake con parole tue.** Spiega a un collega non tecnico, in 4-5 frasi, perché HTTPS richiede uno “scambio di presentazioni” prima di iniziare a parlare, usando un'analogia diversa da quella della busta sigillata già vista in questa sezione.  **Come verificare:** chi ti ascolta riesce a spiegarti a sua volta, con parole proprie, perché la prima parte della connessione è “più lenta” della seconda.

3. **Sessione o JWT?** Per tre scenari diversi — (a) un sito web tradizionale con login, (b) un'app mobile che parla con un'API, (c) un sistema di microservizi interni — decidi se useresti sessioni server-side o JWT, motivando la scelta con almeno un criterio della tabella del paragrafo 20.3 (non solo “perché è più moderno”).  **Come verificare:** per ciascuno dei tre scenari la tua motivazione cita esplicitamente scalabilità o revocabilità, non solo una preferenza generica.
4. **Trova un endpoint non idempotente nel tuo progetto reale.** Chiedi a un developer del team se esiste (o è mai esistito) un bug legato a una richiesta duplicata (doppio ordine, doppio pagamento, doppia email) e fatti raccontare come è stato risolto.  **Come verificare:** sai riassumere in una frase se la soluzione trovata dal team corrisponde a una idempotency key, un controllo lato database, o un altro meccanismo — e la differenza fra questi approcci.
5. **Applica ACID a un caso non bancario.** Pensa a un'operazione del tuo progetto che tocca più tabelle insieme (non necessariamente denaro: es. “cancella un utente e tutti i suoi ordini”) e spiega, proprietà per proprietà, cosa garantirebbe l'atomicità, la coerenza, l'isolamento e la durabilità in quel caso specifico.  **Come verificare:** per ogni lettera di ACID hai scritto una frase specifica al tuo esempio, non la definizione generica del paragrafo 20.5.
6. **Riconosci un problema di concorrenza.** Rileggi le quattro righe della tabella al paragrafo 20.6 (dirty read, non-repeatable read, phantom read, lost update) e inventa un piccolo scenario di ShopFacile (o del tuo progetto) per ciascuno, diverso da quelli già scritti nella sezione.  **Come verificare:** i tuoi quattro scenari sono tra loro distinguibili — se due si somigliano troppo, probabilmente hai confuso due dei quattro problemi.
7. **CAP theorem, in una frase.** Scegli due sistemi che usi ogni giorno (es. un'app di messaggistica e l'home banking) e per ciascuno decidi se, a tuo giudizio, privilegia consistenza o disponibilità, spiegando il “cosa succederebbe se sbagliasse” per ciascuna delle due scelte.  **Come verificare:** la tua spiegazione per ognuno dei due sistemi fa riferimento a una conseguenza concreta (es. “vedo un messaggio in ritardo” vs “vedo un saldo sbagliato”), non solo al nome della proprietà.
8. **Replication o sharding?** Per tre problemi diversi — (a) troppe letture sul catalogo prodotti, (b) un'unica tabella ordini troppo grande per una macchina, (c) un picco di traffico solo durante il Black Friday — indica quale delle due strategie (o entrambe) userebbe ShopFacile, motivando con il criterio della tabella al paragrafo 20.8.  **Come verificare:** per il caso (c) hai considerato che potrebbe non richiedere né l'una né l'altra, ma semplicemente più server applicativi dietro il load balancer — non ogni problema di scala è un problema di database.
9. **Progetta una cache-key.** Per un endpoint `GET /api/prodotti/{id}` di ShopFacile, scrivi una chiave di cache sensata (seguendo l'esempio “buono vs cattivo” del materiale di questa sezione) e stabilisci un TTL plausibile, motivando la scelta in base a quanto spesso pensi che il prezzo o la disponibilità di un prodotto possano cambiare.  **Come verificare:** la tua chiave include l'id del prodotto (non è generica come `dati` o `temp`) e il TTL scelto è accompagnato da una motivazione, non un numero a caso.

10. **Diagnostica una query lenta (esercizio guidato).** Chiedi a un developer del team di mostrarti, anche solo per 10 minuti, l'output di un `EXPLAIN` su una query reale del progetto (lenta o no) e fatti indicare il valore della colonna `type / rows` corrispondente.  **Come verificare:** sai indicare, a parole tue, se quella query sta usando un indice o facendo una scansione completa della tabella, guardando l'output che ti è stato mostrato.
-

Collegamenti

- [2. Fondamenti di informatica](#) — le basi di HTTP, DNS, API, REST, SQL e JOIN su cui questa sezione costruisce
- [11. Architetture software](#) — monolite, microservizi ed event-driven architecture, il contesto in cui vivono le scelte di caching e scaling viste qui
- [13. Sicurezza](#) — autenticazione, autorizzazione, token, OAuth e OpenID Connect nella loro forma generale
- [16. Glossario](#) — tutti i termini di questa sezione, in ordine alfabetico

Risorse

- [MDN Web Docs — HTTP](#) — riferimento completo su protocollo HTTP, metodi e status code
- [jwt.io — Introduzione ai JSON Web Token](#) — spiegazione interattiva della struttura di un JWT
- [OAuth.net — Introduzione a OAuth 2.0](#) — documentazione ufficiale del protocollo
- [PostgreSQL — Livelli di isolamento delle transazioni](#) — documentazione ufficiale, con esempi concreti dei problemi di concorrenza
- [Martin Kleppmann — Designing Data-Intensive Applications](#) — il libro di riferimento su database distribuiti, CAP theorem, replication e sharding
- [Redis — Documentazione sulle strategie di caching](#) — pattern di caching ed eviction policy
- [Node.js — Event Loop, spiegato ufficialmente](#) — come funziona davvero l'event loop
- [Use The Index, Luke! — Guida agli indici SQL](#) — guida pratica, indipendente dal database, su indici e query optimization
- [Microsoft REST API Guidelines](#) — linee guida concrete per progettare API REST solide